LLD Handbook Study Edition — Vol 1: Concepts (OOP to Concurrency)

OOP · UML · SOLID · Clean Code · Refactoring · Patterns · Concurrency · 45 case studies in Java — every topic, readable length

12 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 2 — OOP Fundamentals (Study Edition)**
   - 2.1 What object-oriented programming is
   - 2.2 Classes, objects, constructors, `this`, `static` and `final`
   - 2.3 Encapsulation
   - 2.4 Abstraction
   - 2.5 Inheritance
   - 2.6 Polymorphism (and the full bank example)
   - 2.7 Compile-time vs runtime polymorphism: overloading vs overriding
   - 2.8 Interfaces (default, static, private methods, functional interfaces)
   - 2.9 Abstract classes vs interfaces
   - 2.10 Enums with fields and behaviour
   - 2.11 Composition over inheritance
   - 2.12 Object identity vs equality: the `equals` / `hashCode` contract
   - 2.13 Immutability and records
   - 2.14 Access modifiers and packages
   - 2.15 Generics basics for LLD
   - 2.16 Coupling and cohesion (the yardsticks)
   - 2.17 Object-oriented analysis and design (OOA / OOD / OOP)
   - Chapter summary: in the interview
   - Self-check (whole chapter)
   - TL;DR
2. **Chapter 3 — Class Relationships & UML (Study Edition)**
3. **Part A — Class Relationships**
   - 3.1 The big picture: what a "relationship" really claims
   - 3.2 Association (uni-directional, bi-directional, multiplicity)
   - 3.3 Aggregation
   - 3.4 Composition
   - 3.5 Dependency
   - 3.6 Generalization (inheritance)
   - 3.7 Realization (interface implementation)
   - 3.8 Decision guide: choosing the relationship between two classes
4. **Part B — UML**
   - 3.9 What UML is (structural vs behavioural diagrams)
   - 3.10 Use-case diagrams
   - 3.11 Class diagrams
   - 3.12 Sequence diagrams
   - 3.13 Activity diagrams
   - 3.14 State machine diagrams
5. **Part C — UML in the interview**
   - 3.15 How much UML to draw in an interview
   - 3.16 Chapter self-check
   - 3.17 TL;DR
6. **Chapter 4 — SOLID Principles (Study Edition)**
   - 4.1 What SOLID is and why you should learn it
   - 4.2 Single Responsibility Principle (SRP)
   - 4.3 Cohesion
   - 4.4 Coupling
   - 4.5 Separation of Concerns (SoC)
   - 4.6 SRP vs cohesion vs coupling vs SoC — one table
   - 4.7 Open/Closed Principle (OCP)
   - 4.8 Liskov Substitution Principle (LSP)
   - 4.9 Interface Segregation Principle (ISP)
   - 4.10 Dependency Inversion Principle (DIP)
   - 4.11 Refactoring one class through all five principles
   - 4.12 SOLID and design patterns
   - 4.13 SOLID in the design interview
   - 4.14 Common questions about SOLID (with short answers)
   - 4.15 Over-engineering: the warnings in one place
   - Chapter self-check
   - TL;DR
7. **Chapter 5 — More Design Principles (Study Edition)**
   - 1. DRY — Don't Repeat Yourself
   - 2. KISS — Keep It Simple, Stupid
   - 3. YAGNI — You Aren't Gonna Need It
   - 4. Law of Demeter (Principle of Least Knowledge)
   - 5. Tell, Don't Ask
   - 6. Composition over Inheritance
   - 7. Program to an Interface, Not an Implementation
   - 8. Encapsulate What Varies
   - 9. Strive for Loosely Coupled Designs
   - 10. The Hollywood Principle (Inversion of Control)
   - 11. Separation of Concerns (SoC)
   - 12. Principle of Least Astonishment (POLA)
   - 13. Fail Fast
   - 14. Immutability
   - 15. Design by Contract (DbC)
   - 16. GRASP — General Responsibility Assignment Software Patterns
   - 16.1 Information Expert
   - 16.2 Creator
   - 16.3 Controller
   - 16.4 Low Coupling
   - 16.5 High Cohesion
   - 16.6 Polymorphism
   - 16.7 Pure Fabrication
   - 16.8 Indirection
   - 16.9 Protected Variations

- 17. Domain-Driven Design (DDD) Essentials
- Chapter summary — how the principles fit together

8. **Chapter 6 — Clean Code (Study Edition)**
   - What is clean code, and why bother?
   - 1. Meaningful Names
   - 2. Functions
   - 3. Comments
   - 4. Formatting
   - 5. Objects and Data Structures
   - 6. Error Handling
   - 7. Boundaries
   - 8. Unit Tests
   - 9. Classes
   - 10. Systems
   - 11. Emergence — Kent Beck's Four Rules of Simple Design
   - 12. Concurrency
   - 13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)
   - 14. Smells and Heuristics (the checklist)
   - 15. Code Review Checklist (best practices)
   - 16. Error-Handling Strategy (best practices for real services)
   - 17. Testing Strategies (unit vs integration, test doubles, mocks)
   - Chapter summary
9. **Chapter 7 — Refactoring (Study Edition)**
   - 7.1 Refactoring fundamentals
   - 7.2 Code smells
   - 7.3 The refactoring catalogue
   - 7.4 Smell → refactorings lookup table
   - 7.5 Worked example — from messy to clean in six compiling steps
   - 7.6 Refactoring toward patterns
   - 7.7 Putting it together
   - 7.8 Full source code and output
10. **Chapter 8 — Design Patterns: Introduction & Creational Patterns (Study Edition)**
11. **Part A — Design patterns: the big picture**
    - 8.1 What is a design pattern?
    - 8.2 A short history: from buildings to the "Gang of Four"
    - 8.3 The three categories and all 23 GoF patterns
    - 8.4 The OO principles behind every pattern
    - 8.5 How to pick a pattern
    - 8.6 Anti-patterns: when patterns go wrong
12. **Part B — Creational patterns**
    - 8.7 Creational patterns at a glance
    - 8.8 Singleton
    - 8.9 Simple Factory (an idiom, not a GoF pattern)
    - 8.10 Factory Method
    - 8.11 Abstract Factory
    - 8.12 Builder
    - 8.13 Prototype
    - 8.14 Bonus — Object Pool
    - 8.15 Creational patterns compared
    - 8.16 Chapter self-check
    - 8.17 Chapter TL;DR
13. **Chapter 9 — Structural Design Patterns (Study Edition)**
    - 9.0 What structural patterns are
    - 9.1 Adapter Pattern
    - 9.2 Bridge Pattern
    - 9.3 Composite Pattern
    - 9.4 Decorator Pattern
    - 9.5 Facade Pattern
    - 9.6 Flyweight Pattern
    - 9.7 Proxy Pattern
    - 9.8 Putting it all together — comparison
14. **Chapter 10 — Behavioral Design Patterns (Part 1) · Study Edition**
    - 10.0 Behavioral patterns: the big picture
    - 10.1 Chain of Responsibility
    - 10.2 Command
    - 10.3 Iterator
    - 10.4 Mediator
    - 10.5 Memento
    - 10.6 Interpreter
    - 10.7 Chapter recap: the six patterns side by side
15. **Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF · Study Edition**
    - 11.1 Observer
    - 11.2 State
    - 11.3 Strategy
    - 11.4 Template Method
    - 11.5 Visitor
    - 11.6 GoF behavioral patterns: quick-reference summary
    - 11.7 Beyond GoF: patterns every LLD interview expects
    - 11.8 Choose the right behavioral pattern
    - 11.9 Architectural patterns, and how they differ from design patterns
16. **Chapter 12 — Concurrency & Multithreading for LLD (Java) · Study Edition**
17. **Part A — Foundations**
    - 12.1 Concurrency vs parallelism
    - 12.2 Processes vs threads
    - 12.3 Thread lifecycle & states
    - 12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

18. **Part B — What goes wrong**
    - 12.5 Race conditions & critical sections
    - 12.6 Java Memory Model basics: visibility, happens-before, `volatile`
19. **Part C — Locks**
    - 12.7 Mutex (mutual exclusion) and `synchronized`
    - 12.8 `ReentrantLock` and reentrancy
    - 12.9 Try-lock & timed locking
    - 12.10 Read-write locks: `ReadWriteLock` and `StampedLock`
    - 12.11 Coarse-grained vs fine-grained locking
20. **Part D — Coordination**
    - 12.12 Semaphores
    - 12.13 Condition variables: `wait/notify` and `Condition`
    - 12.14 `CountDownLatch` and `CyclicBarrier`
21. **Part E — Lock-free**
    - 12.15 Compare-and-swap (CAS) & atomics
    - 12.16 Concurrent collections
22. **Part F — Liveness failures**
    - 12.17 Deadlock
    - 12.18 Livelock
    - 12.19 Starvation (and fairness)
23. **Part G — Concurrency patterns**
    - 12.20 Signaling pattern
    - 12.21 Thread pool pattern
    - 12.22 Producer-consumer pattern
    - 12.23 Reader-writer pattern
    - 12.24 Immutability & thread confinement
24. **Part H — Concurrency in LLD interviews**
    - 12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)
    - 12.26 Choosing the right tool (cheat sheet)
    - 12.27 Chapter self-check
    - 12.28 Chapter TL;DR
25. **Chapter 13 — Classic Concurrency Problems · Study Edition**
    - 13.1 Print FooBar Alternately (LeetCode 1115)
    - 13.2 Print Zero Even Odd (LeetCode 1116)
    - 13.3 Fizz Buzz Multithreaded (LeetCode 1195)
    - 13.4 Building H2O (LeetCode 1117)
    - 13.5 Dining Philosophers (LeetCode 1226)
    - 13.6 Thread-safe cache with TTL
    - 13.7 Design a (simplified) Concurrent HashMap — lock striping
    - 13.8 Thread-safe bounded Blocking Queue (two ways)
    - 13.9 Concurrent Bloom Filter
    - 13.10 Multi-threaded Merge Sort (Fork/Join)
    - 13.11 Rate limiter — token bucket (thread-safe)
    - 13.12 Chapter summary
    - 13.13 Chapter self-check
    - 13.14 Chapter TL;DR

Chapter 2 — OOP Fundamentals (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full programs and outputs are in the Full Edition (`README.md`).
>
> **You will learn:** the class basics, the four pillars on one bank example, and the Java OOP tools LLD relies on (sections 2.2–2.17 map one-to-one to the Full Edition).

Run all examples: `cd chapters/02-oop-fundamentals/code && javac --release 21 -d /tmp/lld_02 $(find . -name '*.java') && java -cp /tmp/lld_02 lld.ch02.<package>.Main`

2.1 What object-oriented programming is

What it is

OOP bundles data with the only code allowed to modify it (an **object**). Procedural code keeps data apart, so any function can change it. Analogy: cash on a table vs a vault with a teller (the methods) who checks every request.

The problem it solves

A rule like "balance never negative" lives **once**, next to its data, instead of in every caller; designs grow by **adding classes**.

Why this matters for the interview

LLD tests **splitting a problem into objects, giving each responsibilities and data, and justifying it**.

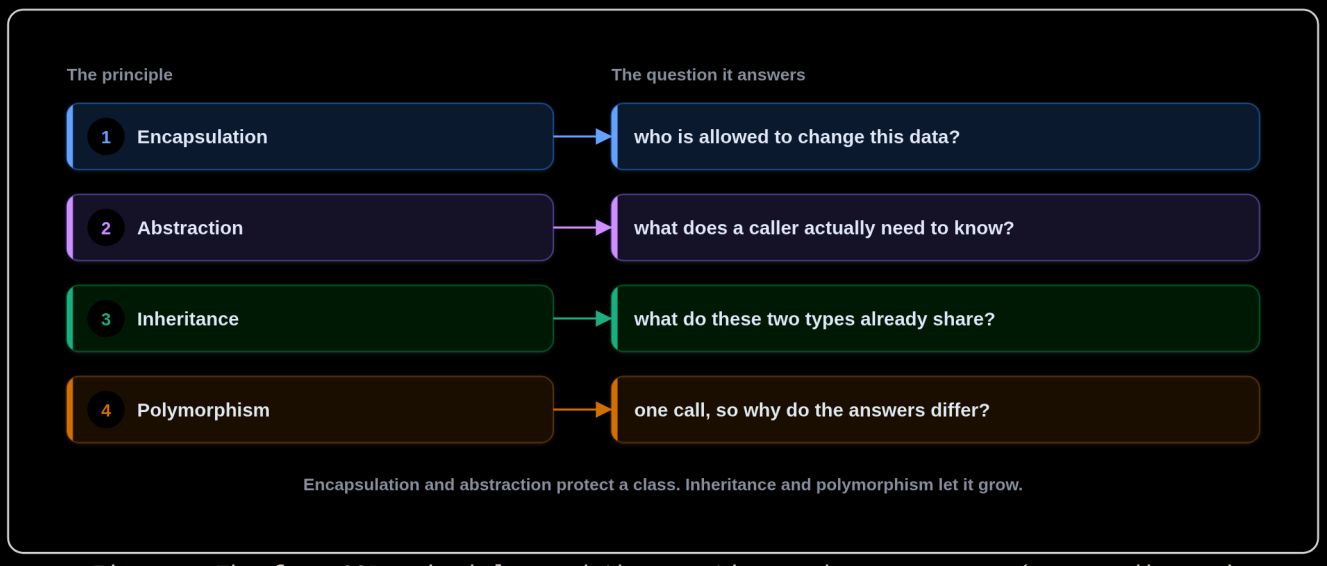


Figure — The four OOP principles and the question each one answers (course diagram)

Pillar	Question it answers
Encapsulation	Who may change this data?
Abstraction	What does a caller need to know?
Inheritance	What do these types share?
Polymorphism	One call, why do answers differ?

The first two guard one class; the last two let the design grow with new classes.

2.2 Classes, objects, constructors, `this`, `static` and `final`

What it is

Class = blueprint; **object** = one instance made with `new`. **Constructor** = named like the class, **no return type**, makes the object **valid**. `this` = current object. `static` = one per class, shared (else per-object **instance** members). `final` = variable set once / method not overridable / class not extendable.

How it works

1. `new` sets fields to defaults (0, `null`, `false`), then runs the constructor.
2. **Order:** static initialisers run **once** at first use; per `new`: parent constructor (`super(...)`) → instance initialisers in source order → constructor body.
3. No constructor written → compiler adds a public no-arg **default constructor**; write any → it doesn't.
4. **Chain** overloads to one canonical constructor with `this(...)` so validation lives once; in Java 21 it must be the **first statement** (Java 25, JEP 513, relaxes this).
5. `this`: disambiguate (`this.id = id`), pass self, chain, `return this` for **fluent** calls.
6. `static` methods have **no** `this`: utilities and factories.
7. Variables hold **references**: `Account alias = a;` is the **same** object.

Java example

```java
// File: code/lld/ch02/classes/Account.java
public class Account {
    // static = belongs to the CLASS, shared by all objects
    private static int openedCount = 0;
    // ... MIN_OPENING_CENTS constant, final id/owner, balanceCents

    public Account(String id, String owner, long openingCents) {
        if (openingCents < MIN_OPENING_CENTS) throw new IllegalArgumentException("negative opening balance");
        this.id = id;                 // 'this.id' is the field, 'id' is the parameter
        this.owner = owner;
        this.balanceCents = openingCents;
        openedCount++;                // shared counter
    }

    public Account(String id, String owner) {
        this(id, owner, 0);
    }

    public Account deposit(long cents) {   // returns 'this' so calls can be chained
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        this.balanceCents += cents;
        return this;
    }
    // ... accessors, static openedCount(), toString
}
```

Full runnable code: code/lld/ch02/classes/ (see the Full Edition).

A refused `new Account("X", "Bad", -5)` leaves the counter unchanged: **validate first, then change state**.

Trade-offs

Static members are shared but are global state (hard to test, thread-risky). Many constructor overloads confuse past 3–4 (use a Builder or static factories). `final` fields are safer but must be set in every constructor.

Common mistakes

Per-object data in a `static` field; heavy work or overridable calls in constructors; `id = id;`; thinking `=` copies an object.

Real-world examples

`Math.max` (static); `StringBuilder.append` returns `this`; `String` is `final`.

In the interview

> **30-second answer:** "A class is the blueprint, an object an instance with its own state. Constructors create valid objects; I chain overloads with `this(...)`. Static members are shared, so I keep them for constants, utilities and factories, never mutable state."

Self-check

- **Q:** Copies of a `static` field for 1,000 objects? **A:** One.
- **Q:** Parent constructor or child initialisers first? **A:** Parent constructor.

TL;DR

Class = blueprint, object = instance; chain with `this(...)`; `static` = per class; variables hold references.

2.3 Encapsulation

What it is

Hide data (**private**) and allow change **only through guarded methods**. Analogy: a child-proof cap.

The problem it solves

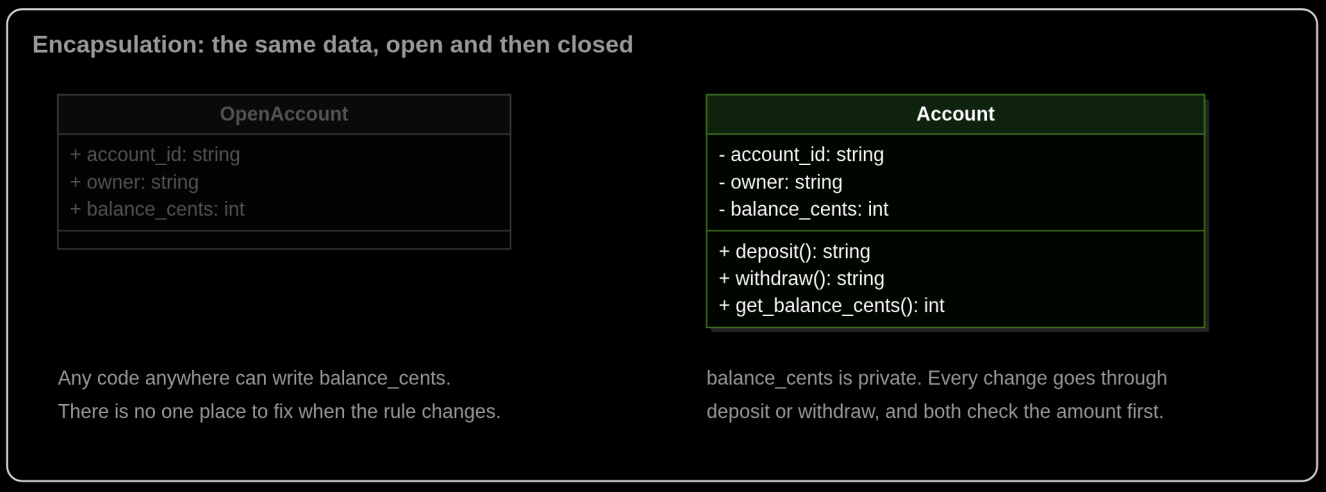


Figure — The same account data, first open (public fields anyone can write) and then closed (private fields changed only through deposit and withdraw) (course diagram)

With a private balance, **every route to the data passes a method that can say no**; public fields leave no central place for the rule.

How it works

Fields `private` (and `final`); expose **behaviour** (`withdraw`), not setters; validate there, **once**; return copies/unmodifiable views of collections; decide **who may modify each field** and say it.

Java example

`Account` in §2.6: private balance, `final deposit`/`withdraw`.

Trade-offs · When not to use it (strictly)

Rules in one place and freely changeable internals, for a little more code. Rule-free data carriers (DTOs, events): use a **record**.

Common mistakes

Getter + setter for every field (anemic, private in name only); returning the live internal list.

Real-world examples

`String`, `ArrayList`; `LocalDate.of(2026, 2, 30)` throws.

In the interview

> **30-second answer:** "State hides behind methods that hold every rule about it. If only this class may change a field, it's private and changed through `withdraw`-style methods, not setters." A rule-free setter is a public field in disguise.

Self-check

- **Q:** Where does "no negative balance" live? **A:** In `Account.withdraw`, once.

TL;DR

Private data + rule-enforcing methods; behaviour, not setters.

2.4 Abstraction

What it is

Give callers **only the operations they need**, hide **how**. Encapsulation hides **state**; abstraction hides **decisions**. Analogy: pedals work the same for petrol or electric.

The problem it solves

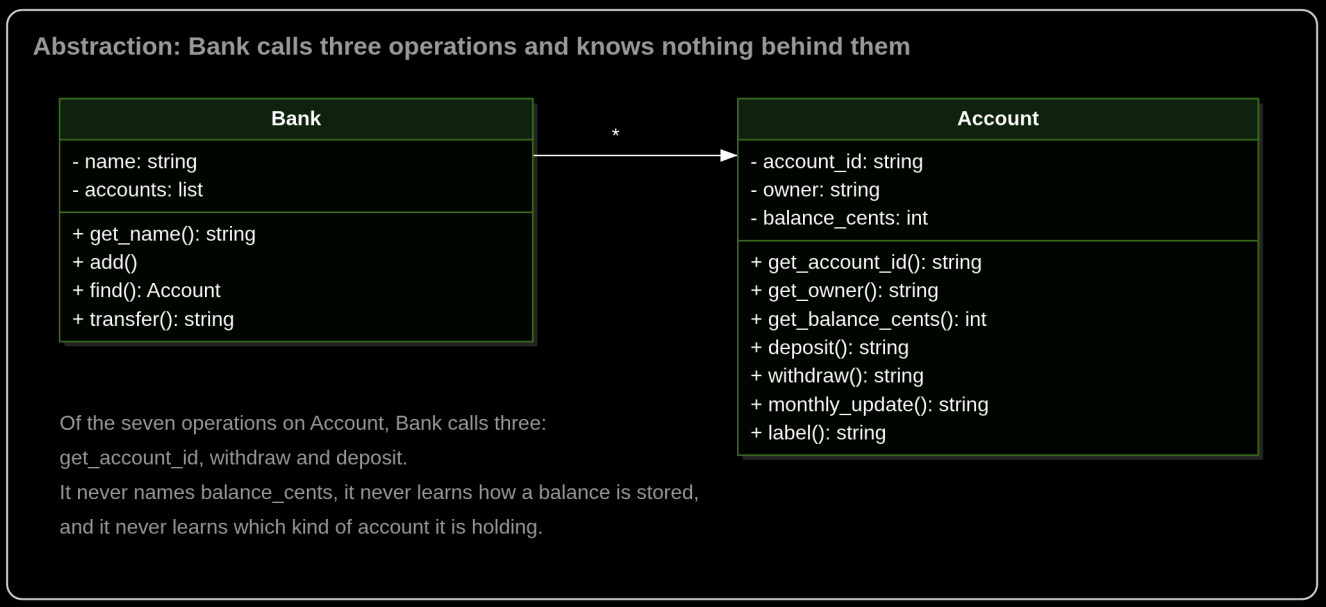


Figure — Of Account's seven public operations, Bank relies on three and sees nothing of what lies behind them (course diagram)

`Bank.transfer` uses only `id()`, `withdraw()`, `deposit()` (our Java `Account` has six public operations, the diagram seven). It doesn't know about cents or subclasses, so storage could move to a database untouched.

How it works

Smallest set of operations, named by **what** they do, on an interface/public surface; callers depend only on it.

Java example

`Bank.transfer` in §2.6.

Trade-offs

Simpler callers, enables polymorphism; a wrong abstraction is costly, too many layers confuse.

Common mistakes

Leaky abstractions (`getHashMapOfSeats()`); abstracting before two concrete cases.

Real-world examples

`List`, JDBC `Connection`, `InputStream`.

In the interview

> **30-second answer:** "Expose only what callers need; whenever 'the caller doesn't need to know that' is true, it moves behind the interface." Encapsulation protects **data**; abstraction hides **complexity**.

Self-check

- **Q:** What does `transfer` need from `Account`? **A:** `id()`, `withdraw()`, `deposit()`.

TL;DR

Minimum surface, hidden how.

2.5 Inheritance

What it is

A **child** gets everything its **parent** has, then **adds** or **overrides**: `class SavingsAccount extends Account` (a savings account **is an** account).

The problem it solves

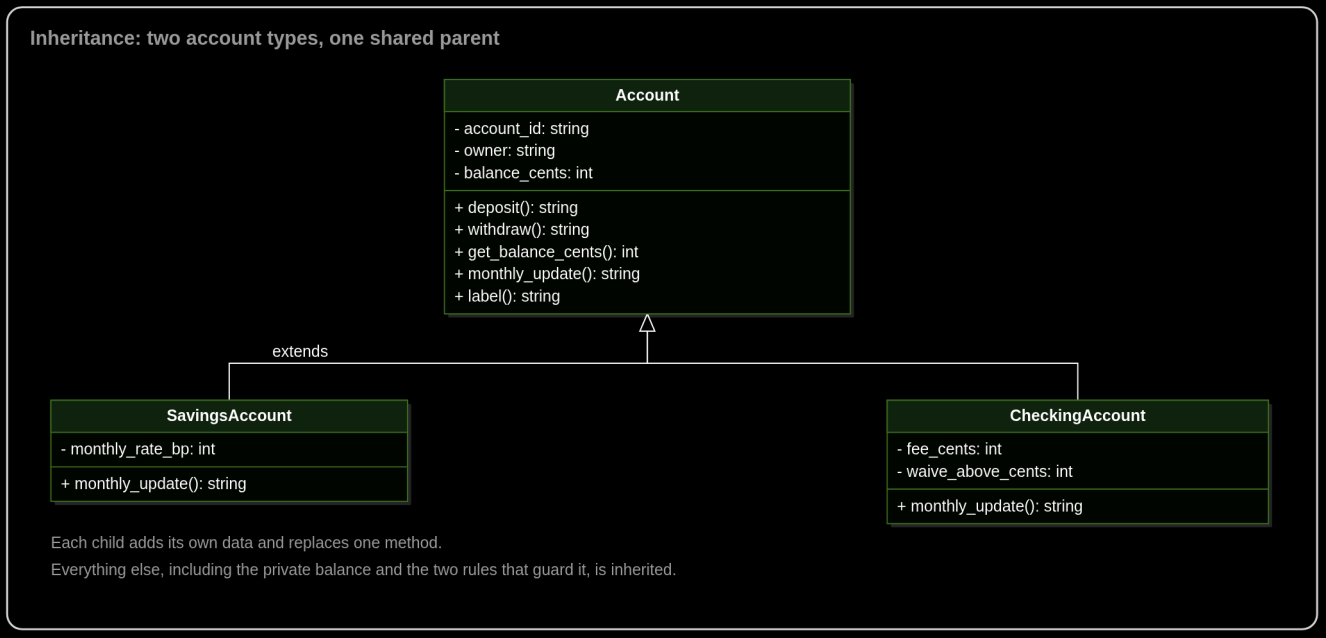


Figure — SavingsAccount and CheckingAccount extend Account; each adds its own data and replaces one method (course diagram)

Both children share the parent's balance and rules, add 1–2 fields and **override one method**. If is-a were false, the difference would be a **field**.

How it works

1. **One** parent class, many interfaces. **Constructors are not inherited**; private fields exist but are reached via parent methods.
2. Child constructor calls `super(...)` first (implicit `super()` fails if the parent lacks a no-arg constructor). `super.method()` reuses the parent's code.
3. **Overriding rules:**

Rule	Detail
Signature	Same name and parameter types
Return type	Same or a **subtype** (*covariant*); primitives exact
Access	Same or **wider** only
Checked exceptions	Fewer/narrower only
Cannot override	`final`; `static` (**hidden**); `private`
Annotation	Always `@Override`

4. Every class extends `Object` (`toString`, `equals`, `hashCode`, `getClass`). The **is-a test** must hold always.

Java example

`SavingsAccount` / `CheckingAccount` (§2.6); `Circle` (§2.7).

Trade-offs

Reuse and polymorphism for true families; but **tight coupling** to parent internals (fragile base class), compile-time fixed, single parent, hard past 2–3 levels.

When not to use it

Code reuse without is-a (use composition); a child that must **disable** a parent method (breaks **Liskov**, Chapter 4); data-only differences (field/enum).

Common mistakes

`Square extends Rectangle` with independent setters; missing `@Override` (accidental overload); overridable calls in parent constructors.

Real-world examples

Good: `FileInputStream extends InputStream`. JDK mistakes: `Stack extends Vector`, `Properties extends Hashtable`.

In the interview

> **30-second answer:** "Inheritance is right only when 'child is a kind of parent' always holds. I use `@Override`, and if a child must disable a parent method I switch to composition, since that breaks Liskov."

- *"Override a static method?"* — No, it's **hidden** (chosen by compile-time type).

Self-check

- **Q:** Override from `public` to `protected`? **A:** No.

TL;DR

True is-a only; `super(...)` first, `@Override` always.

2.6 Polymorphism (and the full bank example)

What it is

One call, different behaviour per receiving object. Code holds the **parent type**; the actual object decides. Analogy: one "start" button on a car, washer, laptop.

The problem it solves

Removes `if (a instanceof SavingsAccount) ... else if ...` chains that every new type must edit.

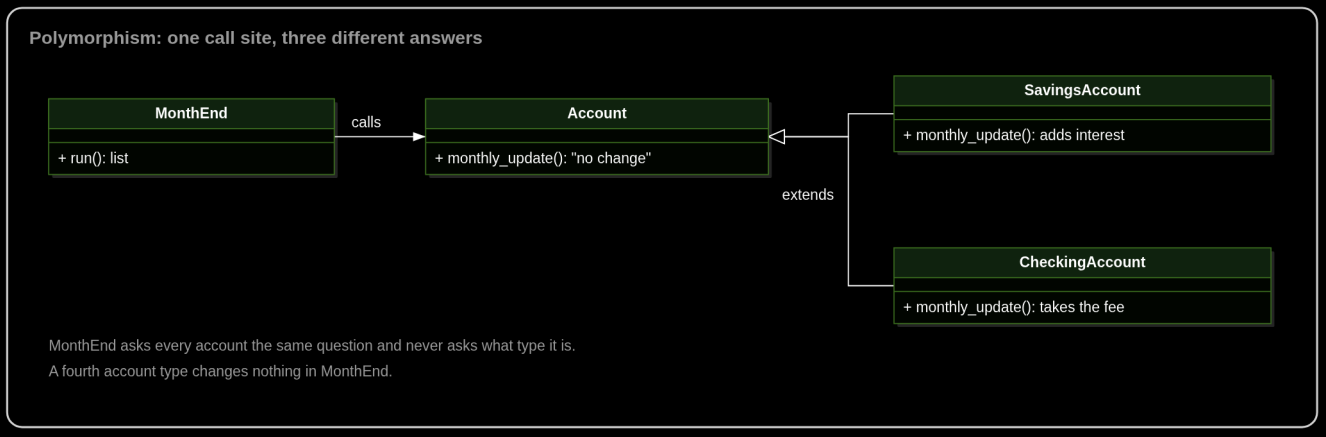


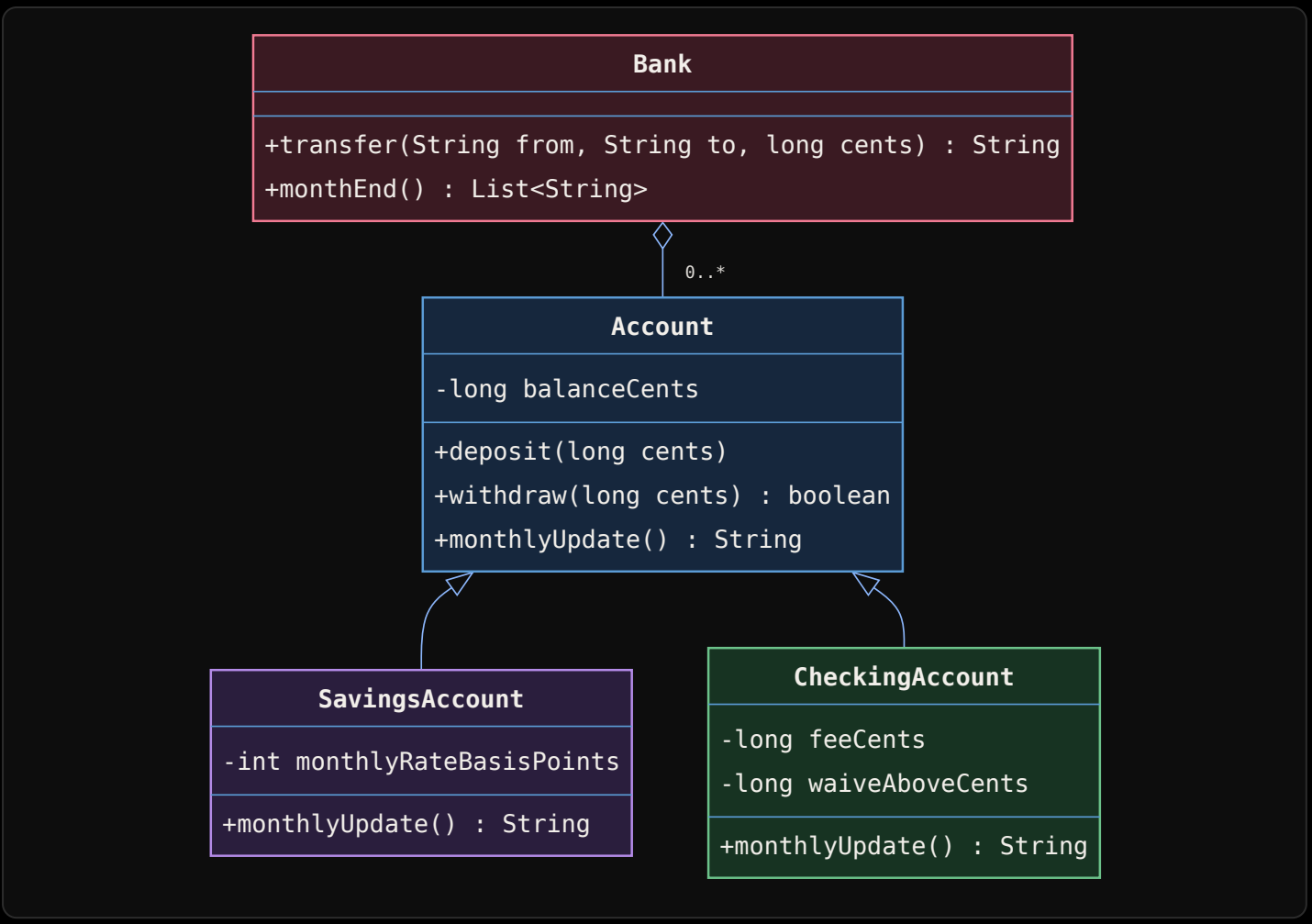
Figure — MonthEnd calls monthlyUpdate on every account and never asks which type it holds (course diagram)

The month-end loop has **no type** `if`; a fourth account type needs no edit.

How it works

Declare on the parent/interface → children override → callers hold the parent type → the JVM runs the **actual class's** version: **dynamic dispatch** (late binding).

Java example: the bank (encapsulation + abstraction + inheritance + polymorphism)




```java
// File: code/lld/ch02/bank/Account.java
public class Account {
    private long balanceCents;                      // hidden: nobody outside can set it to -5
    // ... id, owner, constructor

    public final boolean withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) return false;   // the one place the rule lives
        balanceCents -= cents;
        return true;
    }
    // ... final deposit() and accessors

    public String monthlyUpdate() { return "no change"; }
}
```

```java
// File: code/lld/ch02/bank/SavingsAccount.java
    @Override
    public String monthlyUpdate() {
        long interest = balanceCents() * monthlyRateBasisPoints / 10_000;
        if (interest > 0) deposit(interest);
        return "interest " + Money.format(interest);
    }
```

```java
// File: code/lld/ch02/bank/Bank.java
    public List<String> monthEnd() {
        List<String> lines = new ArrayList<>();
        for (Account a : accounts) {
            String what = a.monthlyUpdate();                // dynamic dispatch picks the override
            lines.add(a.id() + " (" + a.owner() + "): " + what + ", balance " + Money.format(a.balanceCents()));
        }
        return lines;
    }
```

Full runnable code: code/lld/ch02/bank/ (see the Full Edition).

```
S-1 rejected, not enough money
month end:
  S-1 (Ada): interest $1.00, balance $401.00
  C-1 (Kai): fee $2.50, balance $177.50
  P-1 (Ravi): no change, balance $30.00
```

- The $900 transfer is refused inside `withdraw` (encapsulation); `CheckingAccount` waives its fee above a threshold; one call gives three results with no type `if` (polymorphism).
- **Why `long` cents:** exact, unlike `double`, but integer division **truncates** fractions of a cent. For interest/tax use `BigDecimal` with a `RoundingMode` (§2.13). 1 basis point = 0.01%.

How the four fit together

Encapsulation + abstraction decide **what is hidden**; inheritance + polymorphism make **new requirements into added classes**.

Trade-offs

Kills type switches and underpins most patterns; but behaviour spreads across classes (virtual-call cost rarely matters).

Common mistakes

Type `switch`/`if` chains (**pointing it out earns more credit than refactoring**); `instanceof` chains as the main dispatch.

Real-world examples

`List.add` on `ArrayList` vs `LinkedList`; `Comparator.compare`.

In the interview

> **30-second answer:** "I code against the parent type or interface, each subtype overrides, and the JVM picks at run time, so a new type is a new class, not an edit." In a parking lot: pricing, payments, spot selection.

Self-check

- **Q:** `Account a = new SavingsAccount(...)`: whose `monthlyUpdate`? **A:** `SavingsAccount`'s, at run time.

TL;DR

Hold the parent type, let the object decide; type switches signal missing polymorphism.

2.7 Compile-time vs runtime polymorphism: overloading vs overriding

What it is

	Overloading (compile-time)	**Overriding (runtime)**
What	Same name, **different parameters**	Same signature in a **subclass**
Chosen by	**Compiler**, from **declared** argument types	**JVM**, from the receiver's **actual class**
Return type	May differ (not alone)	Same or covariant
Inheritance?	No	Yes

How it works: overload resolution

Phases, first match wins: (1) exact or **widening** (`int` → `long` → `double`, `Circle` → `Shape`); (2) **boxing**; (3) **varargs**. Ties → **most specific**, else ambiguous error. **Not polymorphic:** fields (**hidden**), `static` and `private` methods.

Java example

```
// File: code/lld/ch02/polymorphism/Calculator.java
    public String add(int a, int b)       { return "add(int,int) = " + (a + b); }
    public String add(long a, long b)     { return "add(long,long) = " + (a + b); }
    // ... add(double,double), add(int...)
    public String greet(Shape s)  { return "greet(Shape)"; }
    public String greet(Circle c) { return "greet(Circle)"; }
```

```
// File: code/lld/ch02/polymorphism/Circle.java
public class Circle extends Shape {
    public String kind = "circle";                          // HIDES Shape.kind, does not override it
    // ...
    @Override public double area() { return Math.PI * r * r; }  // OVERRIDE: same signature

    @Override public Circle copy() { return new Circle(r); }    // covariant return type is allowed
    // ... static describe() (hidden), who() calls super.who()
}
```

Full runnable code: code/lld/ch02/polymorphism/ (see the Full Edition).

With `Shape s = new Circle(1.0)`: `calc.add(2L, 3)` → `add(long,long)` (widening); `s.area()` → 3.14 (dispatch); `calc.greet(s)` → **`greet(Shape)`** (overloads use the declared type: a favourite question); `s.kind` → `shape` (**never hide fields**); `s.copy()` is a `Circle`. `s instanceof Circle c` is pattern matching (Java 16+).

Common mistakes

Expecting overloads to follow runtime type (needs double dispatch: Visitor, Chapter 11); `equals(Point p)` overload.

In the interview

> **30-second answer:** "Overloading is compile-time, chosen by declared types; overriding is runtime, dispatched on the actual object. Fields and statics aren't polymorphic."

Self-check

- **Q:** Differ only in return type? **A:** Compile error. **Q:** Varargs phase? **A:** Last.

TL;DR

Overloading = compiler, declared types; overriding = JVM, actual object.

2.8 Interfaces (default, static, private methods, functional interfaces)

What it is

A **contract**: methods a class promises. Analogy: a socket standard. Abstract methods are `public abstract`, fields `public static final`; a class **implements many**. **Default** (Java 8) bodies let interfaces evolve (`Collection.stream()`); **static** (Java 8) belong to the interface, not inherited; **private** (Java 9) help defaults. A **functional interface** has **exactly one abstract method** (Object/default/static don't count) → a **lambda** can implement it; `@FunctionalInterface` enforces it.

The problem it solves

Callers depend on **what**, not a class: add implementations without edits, test with fakes, multiple inheritance of **type** (not state).

How it works

Smallest method set; implement via classes or lambdas; callers use the interface type. Same default in two interfaces → class **must** override (may call `A.super.m()`); a superclass method **wins** over defaults.

Java example

```java
// File: code/lld/ch02/interfaces/Notifier.java
public interface Notifier {
    String send(String to, String message);        // abstract method: every implementer must provide it

    default String sendAll(java.util.List<String> recipients, String message) {
        StringBuilder sb = new StringBuilder();
        for (String r : recipients) sb.append(send(r, clean(message))).append('\n');
        return sb.toString().stripTrailing();
    }

    static Notifier console() { return (to, msg) -> "[console] " + to + ": " + msg; }

    private String clean(String message) { return message.strip(); }
}
```

Full runnable code: code/lld/ch02/interfaces/ (see the Full Edition).

`EmailNotifier` and a lambda both implement `Notifier`; `DiscountRule` is an `@FunctionalInterface` with a default `then` combinator. JDK ones: `Function`, `Supplier`, `Consumer`, `Predicate`.

Trade-offs

Loose coupling, testability, evolvable; but an extra type, no state, and overused defaults become mini base classes.

When not to use it

One implementation, no realistic second, no test double.

Common mistakes

Fat interfaces (ISP); constant-only interfaces.

Real-world examples

`Comparable`, `Runnable`, `AutoCloseable`, Spring `Repository`.

In the interview

> **30-second answer:** "Services depend on interfaces like `PaymentGateway` so implementations can be added and faked. Interfaces have default and static methods; one abstract method makes them lambda-friendly." Defaults exist to add methods like `stream` without breaking implementers.

Self-check

- **Q:** Is `Comparator` functional? **A:** Yes; only `compare` is abstract.

TL;DR

Depend on contracts; defaults evolve, statics are factories, one abstract method = lambda.

2.9 Abstract classes vs interfaces

What it is

An **abstract class** can't be instantiated; it may hold fields, constructors, finished and **abstract** methods. Analogy: a partly pre-printed form.

How it works

```java
// File: code/lld/ch02/interfaces/Report.java
public abstract class Report {
    private final String title;                    // state: interfaces cannot have instance fields

    protected Report(String title) { this.title = title; }

    public final String render() {                 // final: subclasses cannot change the skeleton
        return header() + "\n" + body() + "\n-- end of " + title + " --";
    }

    protected String header() { return "== " + title + " =="; }   // concrete, overridable
    protected abstract String body();                              // must be supplied
}
```

`SalesReport` supplies only `body()`: **Template Method** (Chapter 11).

	Abstract class	Interface
State / constructors	Yes	Constants only / no
Bodies	Any	`default`, `static`, `private`
Per class	**One**	**Many**
Expresses	"is a"	"can do"
Use	Shared state + partial implementation	Contracts, roles, strategies, test seams

Rule of thumb

Interface for the contract; abstract class beneath when implementations share state or an algorithm (`List` + `AbstractList` + `ArrayList`).

Common mistakes

Abstract class to share one helper; everything abstract "for flexibility".

In the interview

> **30-second answer:** "Interfaces define capabilities and allow many; abstract classes share state and partial implementation, one parent. Interface first; abstract base for real shared state or a template."

Self-check

- **Q:** Abstract class without abstract methods? **A:** Allowed. **Q:** Interface constructor? **A:** No.

TL;DR

Interface = contract, many, stateless; abstract class = partial implementation, one.

2.10 Enums with fields and behaviour

What it is

A **fixed set of named instances**; in Java a full class with **fields, constructors, methods, interfaces**, even per-constant bodies.

The problem it solves

String/int constants allow typos and carry no behaviour. Enums are **type-safe** and hold each value's data and rules, removing switches: LLD's home for "**different data, not behaviour**".

How it works

Implicitly extend `Enum`; private constructors; `values()`, `valueOf` (throws on unknown), `name()`, `ordinal()` (**never persist**); compare with `==`; per-constant bodies make tiny state machines; `EnumMap` / `EnumSet`; exhaustive `switch` needs no `default`; a one-element enum is the simplest safe **Singleton**.

Java example

```java
// File: code/lld/ch02/enums/OrderStatus.java
public enum OrderStatus {
    PLACED    { @Override Set<OrderStatus> next() { return Set.of(PAID, CANCELLED); } },
    PAID      { @Override Set<OrderStatus> next() { return Set.of(SHIPPED, CANCELLED); } },
    SHIPPED   { @Override Set<OrderStatus> next() { return Set.of(DELIVERED); } },
    DELIVERED { @Override Set<OrderStatus> next() { return Set.of(); } },
    CANCELLED { @Override Set<OrderStatus> next() { return Set.of(); } };

    abstract Set<OrderStatus> next();          // constant-specific method bodies

    public boolean canMoveTo(OrderStatus target) { return next().contains(target); }
    // ... isFinal()
}
```

Full runnable code: code/lld/ch02/enums/ (see the Full Edition).

`Coin` carries `cents` as a field (`PENNY(1) ... QUARTER(25)`), so summing needs no switch. `OrderStatus` refuses `PLACED -> SHIPPED`: the "**enum + guarded method**" alternative to the State pattern.

Trade-offs

Type-safe, data + behaviour; but closed (recompile to add), and large per-constant logic belongs in State/Strategy classes.

When not to use it

Values created at run time, or needing big independent behaviour.

Common mistakes

Persisting `ordinal()`; switches over the enum instead of methods on it.

Real-world examples

`DayOfWeek`, `TimeUnit.SECONDS.toMillis(5)`, `Thread.State`.

In the interview

> **30-second answer:** "Enums model fixed sets like vehicle types and statuses, carrying data like 'which spots fit' or 'which status may follow'." Enum if only data differs; classes if behaviour does.

Self-check

- **Q:** `Coin.valueOf("dime")`? **A:** `IllegalArgumentException` (case-sensitive).

TL;DR

Type-safe fixed sets with data and behaviour; `==`; no persisted ordinals.

2.11 Composition over inheritance

What it is

Hold other objects and **delegate** ("has-a") instead of inheriting ("is-a"); GoF's "favour composition over inheritance". Analogy: a phone **has** an upgradable camera module.

The problem it solves

Inheritance ties the child to the parent's **internals** (**fragile base class**), fixed at compile time. Composition stays independent and **swappable at run time**.

How it works

Put the behaviour behind an interface → hold it in a private field from the constructor → **forward** calls, adding your logic.

Java example

```java
// File: code/lld/ch02/composition/InheritingCountingSet.java
    @Override public boolean add(E e) { attempts++; return super.add(e); }

    @Override public boolean addAll(Collection<? extends E> c) { attempts += c.size(); return super.addAll(c); }
```

```java
// File: code/lld/ch02/composition/CountingSet.java
public class CountingSet<E> {
    private final Set<E> inner;
    // ...
    public boolean addAll(Collection<? extends E> c) {
        attempts += c.size();
        boolean changed = false;
        for (E e : c) changed |= inner.add(e);     // talk to the inner set, not to our own add()
        return changed;
    }
}
```

Full runnable code: code/lld/ch02/composition/ (see the Full Edition).

```
inheritance: added 3, attempts counted = 6
composition: 4 attempts counted = 4, size = 3
```

`HashSet.addAll` calls `add` per element (parent **self-use**, not promised), so the subclass double counts. `CountingSet` works with any `Set`. `Car` **has an** `Engine`, swapped at run time.

Trade-offs

	Inheritance	Composition
Relationship / coupling	is-a / tight	has-a / loose
Run-time change	No	Yes
Boilerplate	Low	Forwarding methods

When not to use it

A genuine is-a class **designed for extension** is fine to extend.

Common mistakes

Extending concrete library classes (`HashMap`); exposing the inner object (`getInner()`).

Real-world examples

`BufferedReader` wraps a `Reader`; Spring DI; `Comparator` in `TreeMap`.

In the interview

> **30-second answer:** "I hold an interface-typed collaborator and delegate: no fragile base class, run-time swapping, and it underlies Strategy, Decorator, State. Inheritance only for true is-a." Example of breakage: the counting set.

Self-check

- **Q:** Why `inner.add`, not `this.add`? **A:** No double counting; relies only on the public contract.

TL;DR

Has-a + delegate beats is-a for reuse.

2.12 Object identity vs equality: the `equals` / `hashCode` contract

What it is

Identity `==`: same object. **Equality** `equals`: same value (default is identity; `String`, `Integer`, records override). Two copies of a book: equal, not identical.

The problem it solves

Hash collections pick a bucket by `hashCode()`, then compare with `equals()`; get either wrong and objects "disappear".

How it works: the contracts

`equals`: reflexive, symmetric, transitive, consistent, `equals(null)` false. `hashCode`: stable, **equal objects → equal hashes**, collisions allowed. **Override both on the same fields.**

Java example

```java
// File: code/lld/ch02/equality/Point.java
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // reflexive fast path
        if (!(o instanceof Point p)) return false;           // also handles null
        return x == p.x && y == p.y;
    }

    @Override public int hashCode() { return Objects.hash(x, y); }
```

Full runnable code: code/lld/ch02/equality/ (see the Full Edition).

A `HashSet<Point>` holding two equal points has size 1. `BrokenPoint` (only `equals`) gives `set.contains(equal one): false`; `javac -Xlint` warns `[overrides]`: treat as an error. Compare boxed numbers with `equals`: `Integer` caches only small values (at least −128..127).

Trade-offs / design choice

Value objects (Money, Point): value equality, immutable. **Entities** (User, Order): identity or stable **ID**.

Common mistakes

`equals` without `hashCode`; `equals(Point p)`; **mutable fields in `hashCode`** then mutating a key; mixing `getClass()` / `instanceof`.

Real-world examples

`String`, `LocalDate`, `List.equals`; `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is **false** (scale; use `compareTo`).

In the interview

> **30-second answer:** "`==` is identity, `equals` value. I override `hashCode` with `equals` on the same fields, since hash collections locate the bucket by hash. Records for values, IDs for entities."

Self-check

- **Q:** Unequal objects, same hash? **A:** Allowed. **Q:** Equal objects, different hashes? **A:** Broken contract.

TL;DR

Override `equals` and `hashCode` together, on immutable fields.

2.13 Immutability and records

What it is

Immutable objects never change; "changes" return **new** objects. A **record** (Java 16+) declares an immutable data carrier in one line. Analogy: a printed receipt.

The problem it solves

Shared mutable state causes surprise changes and needs locks. Immutable objects are **thread-safe without locks** and safe hash keys.

How it works

Checklist: `final` class (or private constructors + static factories); `private final` fields; no setters; **defensive copies** in and out; don't leak `this`. **Records** generate fields, canonical constructor, accessors (`id()`), `equals` / `hashCode` / `toString`; they're `final`, allow no extra instance fields, may

implement interfaces; a **compact constructor** validates. **Only shallowly immutable**: copy mutable components.

Java example

```java
// File: code/lld/ch02/immutability/Money.java
    private Money(BigDecimal amount, String currency) {
        this.currency = Objects.requireNonNull(currency);
        // UNNECESSARY = refuse (ArithmeticException) anything finer than a cent instead of silently rounding it
        this.amount = amount.setScale(2, RoundingMode.UNNECESSARY);
    }

    public static Money of(String amount, String currency) { return new Money(new BigDecimal(amount), currency); }

    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(amount.add(other.amount), currency);   // new object; 'this' is untouched
    }
```

```java
// File: code/lld/ch02/immutability/Order.java
public record Order(String id, List<String> items, Money total) {
    public Order {
        if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
        items = List.copyOf(items);                // unmodifiable copy: caller's list cannot leak in
    }
    // ... withItem(item, price) returns a modified copy ("wither")
}
```

Full runnable code: code/lld/ch02/immutability/ (see the Full Edition).

Scale fixed at 2 makes `Money.of("5")` equal `Money.of("5.00")`; `items()` is an unmodifiable copy.

Trade-offs

Lock-free safety, no boilerplate; but a new object per change, and records can't hide components or extend classes.

When not to use it

Entities whose state changes (occupied spot, balance) stay **mutable but encapsulated**.

Common mistakes

No defensive copies; thinking `final` freezes the referenced object; records for entities.

Real-world examples

`String`, `BigDecimal`, `java.time`, `List.of`.

In the interview

> **30-second answer:** "Value objects like Money or Ticket are immutable records with compact-constructor validation and defensive copies: thread-safe, safe keys. Entities stay mutable but encapsulated." Records are only shallowly immutable.

Self-check

- **Q:** `final List<String> items` guarantees? **A:** The reference, not the contents.

TL;DR

Final, private final, no setters, defensive copies; records do it (shallowly).

2.14 Access modifiers and packages

What it is

A **package** groups classes (namespace + folder); **access modifiers** set visibility:

Modifier	Class	Package	Subclass elsewhere	Everyone
`private`	✔	✘	✘	✘
(none) package-private	✔	✔	✘	✘
`protected`	✔	✔	✔ (via inheritance)	✘
`public`	✔	✔	✔	✔

Top-level classes: `public` or package-private only.

The problem it solves

Encapsulation becomes **compiler-enforced**; packages hide implementation behind a small API.

How it works

Start `private`; group by **feature**; package-private helpers behind a `public` facade; `protected` only for subclass hooks (it **also** opens the package); Java 9 **modules** export chosen packages.

Java example

```java
// File: code/lld/ch02/access/client/PremiumWallet.java
    public void grantBonus(long cents) {
        bonusCents += cents;                    // OK: protected, accessed through inheritance
        // auditCount++;                        // would NOT compile: package-private in another package
        // balanceCents += cents;               // would NOT compile: private
    }

    @Override public String describe() {       // may WIDEN protected -> public when overriding
        return "[premium] " + super.describe();
    }
```

Full runnable code: code/lld/ch02/access/core/ and code/lld/ch02/access/client/ (see the Full Edition).

`Wallet` (in `core`) has one field per access level; `AuditLog` is a package-private class exposed only through the public `Audits` facade.

Common mistakes

Everything `public`; `protected` fields instead of methods; one package for a whole solution.

In the interview

> **30-second answer:** "Private by default, package-private for feature collaborators, protected only for subclass hooks, public as the deliberate API; packages like model, repository, service, strategy."

Self-check

- **Q:** Non-subclass in another package reads `protected`? **A:** No.

TL;DR

private < package-private < protected < public; default private.

2.15 Generics basics for LLD

What it is

Type-safe code for **any type**: `List<String>`, `Repository<String, User>`; `T`, `ID`, `E` are **type parameters**. Analogy: a "SHOES ONLY" box checked on the way in.

The problem it solves

No `Object` + casts failing at run time: errors become **compile errors**, and one `Repository<ID, T>` serves every entity.

How it works

Generic types and methods; **bounds** (`T extends X`) let you call X's methods; wildcards `? extends T` (producer), `? super T` (consumer): **PECS**; diamond `new Repository<>()`; **erasure**: no `new T()`, no `instanceof List<String>`, no primitives; avoid **raw types**.

Java example

```java
// File: code/lld/ch02/generics/Repository.java
public class Repository<ID, T extends Identifiable<ID>> {
    private final Map<ID, T> store = new LinkedHashMap<>();

    public void save(T item)           { store.put(item.id(), item); }
    public Optional<T> findById(ID id) { return Optional.ofNullable(store.get(id)); }
    public List<T> findAll()           { return List.copyOf(store.values()); }
}
```

Full runnable code: code/lld/ch02/generics/ (see the Full Edition).

The same class stores `User`s by `String` and `Product`s by `Integer`. `copy(List<? extends T> src, List<? super T> dst)` moves integers into a `List<Number>` (PECS). `ArrayList<String>` and `ArrayList<Integer>` share one runtime class (erasure).

Common mistakes

Raw types; thinking `List<Integer>` is a `List<Number>`; over-generic designs.

Real-world examples

Collections, `Optional<T>`, Spring Data `CrudRepository<T, ID>`.

In the interview

> **30-second answer:** "Generics give type-safe infrastructure like `Repository<ID, T>`; I know bounds, PECS, and erasure."

Self-check

- **Q:** Why no `new T()`? **A:** Erasure; pass a `Supplier<T>`.

TL;DR

Type-safe reuse; bounds, PECS, erasure.

2.16 Coupling and cohesion (the yardsticks)

Cohesion (how well a class's duties belong together) should be **high**; **coupling** (dependence on others' details) should be **low**. Encapsulation and access modifiers cut coupling, single-responsibility raises cohesion, interfaces and composition decouple. Chapters 4–5 name these as SOLID, Law of Demeter, GRASP.

2.17 Object-oriented analysis and design (OOA / OOD / OOP)

What it is

Activity	Question	Talks about
Analysis (OOA)	What is the system, what must it do?	Domain terms: books, members, loans
Design (OOD)	Which objects with which responsibilities?	`Book`, `Member`, `Loan`, who holds whom
Implementation (OOP)	How in code?	`ArrayList`, `HashMap`

The problem it solves

Choosing operations before objects are settled puts them on the wrong class.

How it works: the four steps

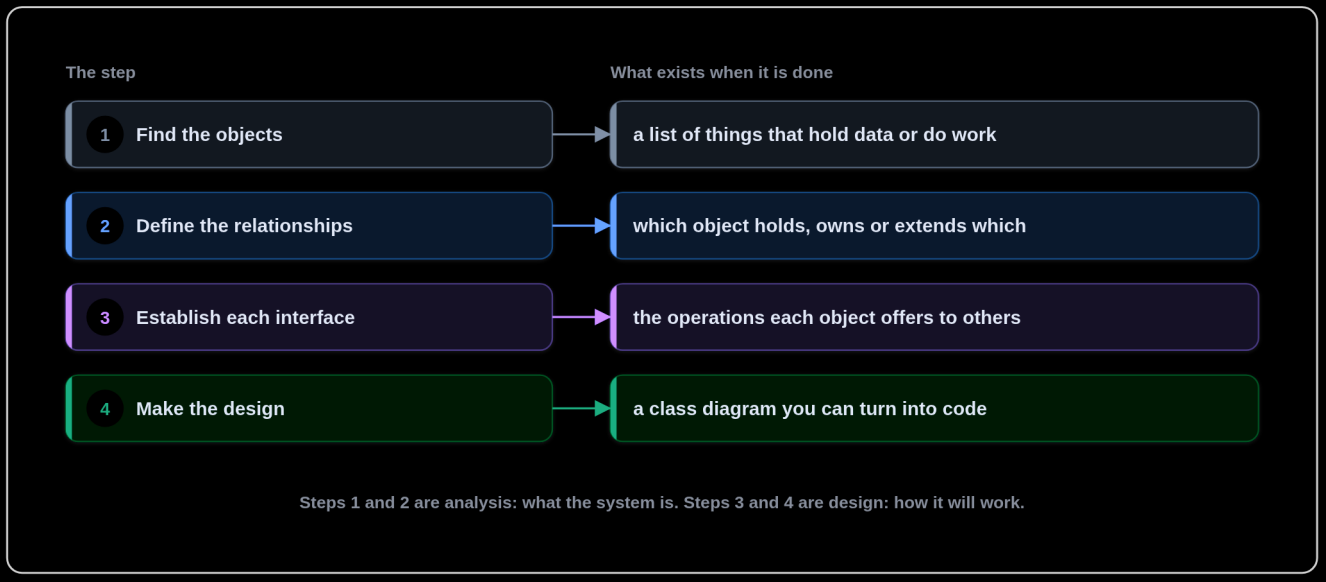


Figure — The four OOAD steps and what exists after each; steps 1–2 are analysis and steps 3–4 are design (course diagram)

1. **Find the objects** (things that store info or do work; single values become fields).
2. **Define relationships** (who references whom, ownership, how many).
3. **Establish each interface** (operations offered to others; the rest is **private**).
4. **Make the design** (one class diagram, checked against the description). Fixed order.

Worked example: "library members borrow books; each book on loan must be returned by a due date"

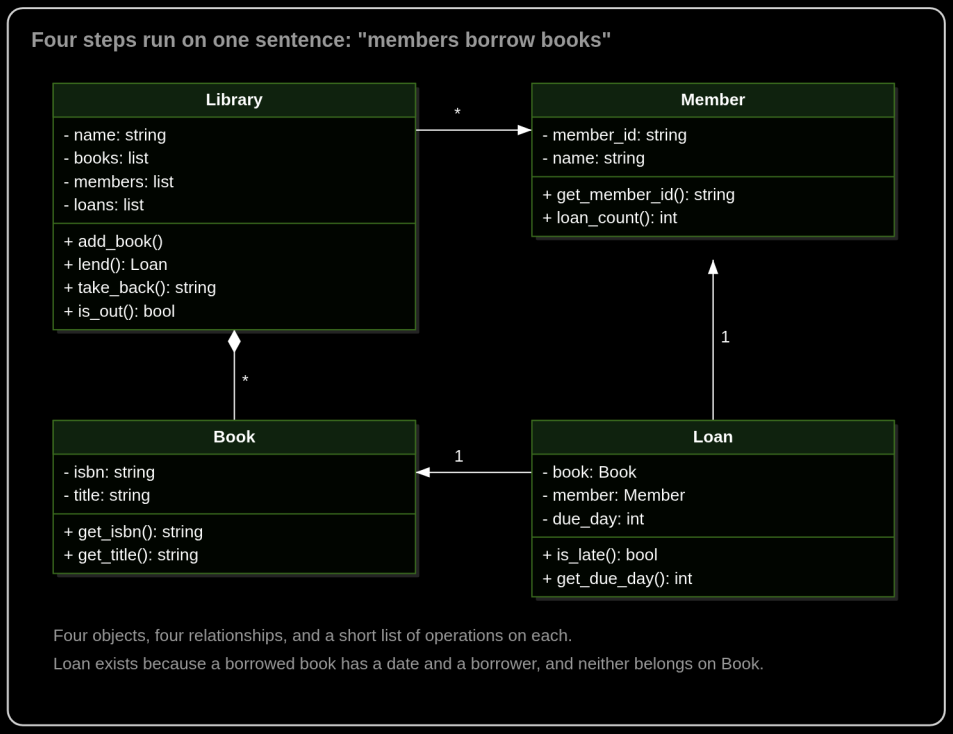
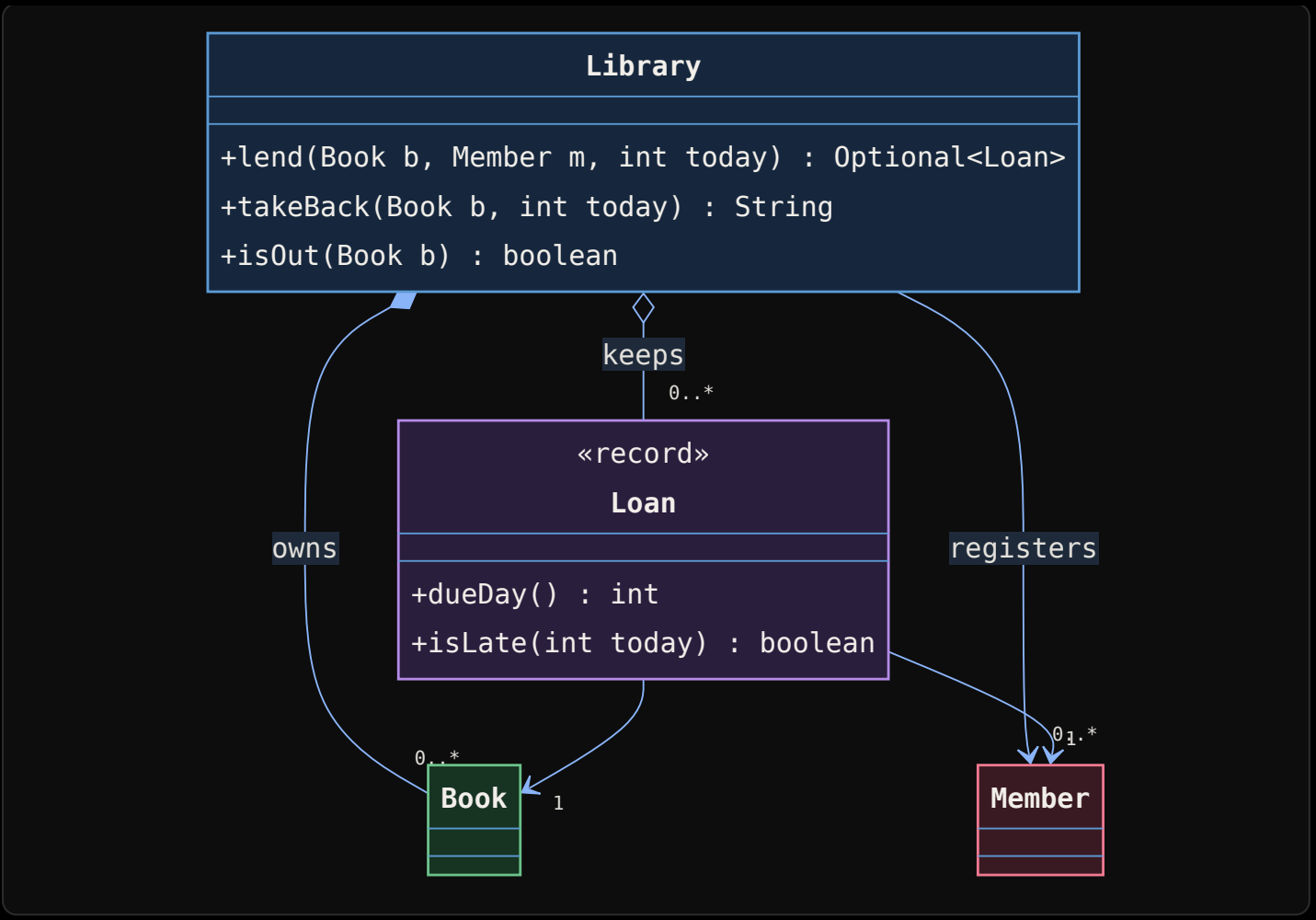


Figure — The four steps applied to one sentence: Library, Member, Book and the connection object Loan (course diagram)

Objects: `Library`, `Member`, `Book`, and **`Loan`** (most often missed); *date* is a field. **Rule: a link with data of its own becomes an object** (`Loan`, `Reservation`, `Ticket`, `Enrollment`). Library **owns** books (composition), **registers** members (association); a loan **points to** one book and one member. Only `Library` can answer "is this book out?" because it keeps the loans.




```java
// File: code/lld/ch02/ooad/Library.java
    public static final int LOAN_DAYS = 14;
    public static final int MAX_LOANS_PER_MEMBER = 5;
    // ...
    public Optional<Loan> lend(Book b, Member m, int today) {
        if (!books.contains(b) || !members.contains(m)) return Optional.empty();
        if (isOut(b) || m.loanCount() >= MAX_LOANS_PER_MEMBER) return Optional.empty();
        Loan loan = new Loan(b, m, today + LOAN_DAYS);
        loans.add(loan);
        m.loanStarted();
        return Optional.of(loan);
    }
```

Full runnable code: code/lld/ch02/ooad/ (see the Full Edition).

Limits became constants; `Loan` is a record with `isLate(today)`; `Member.loanStarted/loanEnded` are package-private so only `Library` changes counts.

Why UML

UML is the standard notation because each symbol records a decision: **filled diamond** = ownership, **plain arrow** = reference, **+ / −** = public/private (Chapter 3).

Three ways this goes wrong

Classes named after tables (`UserRow`) that do nothing; **missing the connection object** (`dueDate` on `Book`); **listing every imaginable operation**.

In the interview

> **30-second answer:** "Analysis first: objects from nouns, relationships with ownership and multiplicity. Then design: each object's public operations (which decides what's private), then

the diagram. A link with its own data becomes a class."

Self-check

- **Q:** Analysis steps? **A:** Find objects, define relationships. **Q:** Parking-lot connection object? **A:** The ticket. **Q:** When is encapsulation decided? **A:** Step 3.

TL;DR

OOA = what; OOD = how with objects; OOP = code. Connections with data become objects.

Chapter summary: in the interview

30-second answer ("pillars of OOP"): "Encapsulation keeps data and its rules in the owning class; abstraction exposes only what callers need; inheritance reuses a parent when is-a truly holds; polymorphism lets one call behave per object, removing type switches. I prefer interfaces and composition, enums for fixed sets, records for values, and consistent equals/hashCode."

- *"Make a class immutable?"* — Final class, private final fields, no setters, defensive copies, or a record.

Self-check (whole chapter)

- **Q:** Dynamic dispatch? **A:** The JVM choosing the override from the actual class at run time.
- **Q:** Where does "which spots fit a truck" live? **A:** On the `VehicleType` enum.
- **Q:** hashCode rule? **A:** Equal objects, equal hash codes.
- **Q:** OOAD steps? **A:** Objects, relationships, interfaces, design.

TL;DR

- OOP = data bundled with the only code allowed to change it.
- Pillars: encapsulation, abstraction, inheritance (true is-a), polymorphism (kills type switches).
- Overloading compile-time; overriding run-time; fields/statics not polymorphic.
- Interfaces = contracts; abstract classes = shared state; enums = fixed sets; composition over inheritance.
- `equals` + `hashCode` together; immutable records for values; encapsulated mutable entities.
- Private by default; generics for type-safe infrastructure; OOA → OOD → OOP.

Chapter 3 — Class Relationships & UML (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full programs and outputs are in the Full Edition (`README.md`).
>
> **You will learn:** the six class relationships and their Java shape, how to choose one, the five UML diagrams interviews use, and how much to draw.

Parts: A. Relationships (3.1–3.8) · B. UML (3.9–3.14) · C. Interview (3.15–3.17). One runnable package per relationship, plus `library` and `statemachine`.

Run: `cd chapters/03-relationships-and-uml/code && javac --release 21 -d /tmp/lld_03 $(find . -name '*.java') && java -cp /tmp/lld_03 lld.ch03.<package>.Main`

Part A — Class Relationships

3.1 The big picture: what a "relationship" really claims

What it is

A line between two classes makes two claims: about **lifetime** (if A is destroyed, is B too?) and about **substitution** (can a B be used wherever an A is expected?). Analogy: body and heart share a lifetime (composition); a car and its driver don't (association); a sports car *is a* car (generalization).

Family	Relationship	Meaning	Coupling
uses-a	**Dependency**	A uses B briefly (parameter, local, return type)	weakest
has-a	**Association**	A holds a reference to B; nobody owns anybody	↓
has-a	**Aggregation**	A is a whole of parts B; parts can outlive A	↓
has-a	**Composition**	A *owns* parts B; parts die with A	↓
is-a	**Realization**	A implements interface B	↓
is-a	**Generalization**	A extends class B	strongest

Coupling = how much one class must change when the other does; inheritance couples to every parent detail. (The order is a discussion rule of thumb, not UML.)

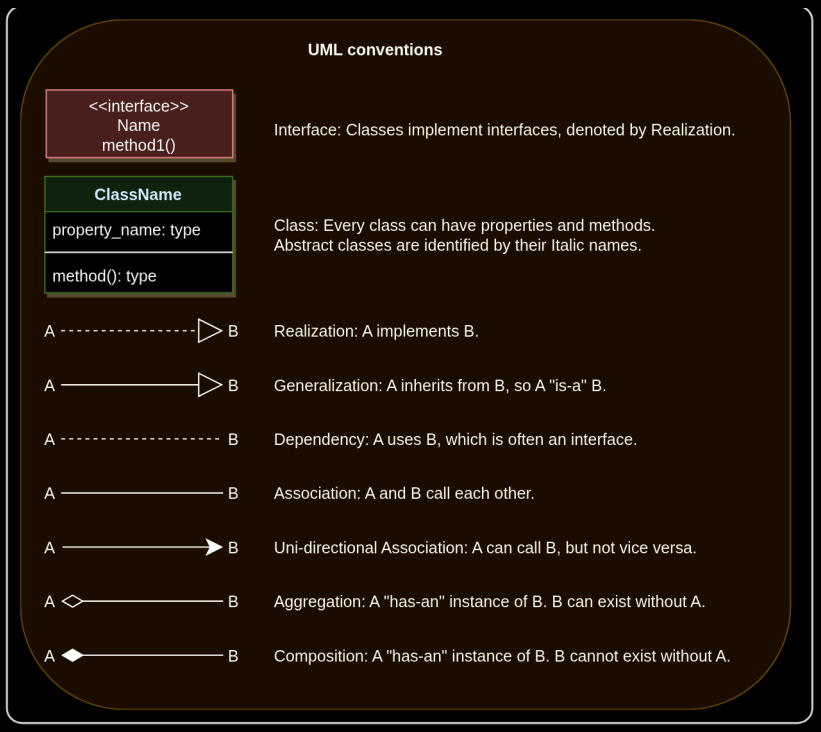


Figure — The UML conventions legend used throughout this book: class and interface boxes, and the line style for realization, generalization, dependency, association, uni-directional association, aggregation and composition (course diagram)

Line ends (triangle, diamond, arrowhead) plus solid vs dashed are the whole vocabulary. (This legend draws one-way association with a *filled* head and dependency with *no* head; standard UML and our Mermaid use an **open** arrowhead for both.)

The problem it solves

"Order has a Customer" only points at something living elsewhere; "ParkingFloor has ParkingSpots" creates, owns and deletes them. Confusing them deletes data you don't own or leaks orphans, visibly, before any code.

TL;DR

A line claims **who owns whom (lifetime)** and **who can stand in for whom (substitution)**.

3.2 Association (uni-directional, bi-directional, multiplicity)

What it is

One class keeps a **reference (field)** to another and calls it over time; **neither owns the other**. Analogy: a ticket refers to a spot; throw the ticket away and the spot remains.

- **Navigability: uni-directional** (one side has the field) or **bi-directional** (both do).
- **Multiplicity:** `1`, `0..1`, `*` / `0..*`, `1..*`, `2`, `2..4`.
- Optional **role/association name** (`placed by`).

The problem it solves

Objects collaborate over time (an `Order` must know its `Customer`). Association records "A knows B" without claiming ownership; it is the most common relationship, correctly.

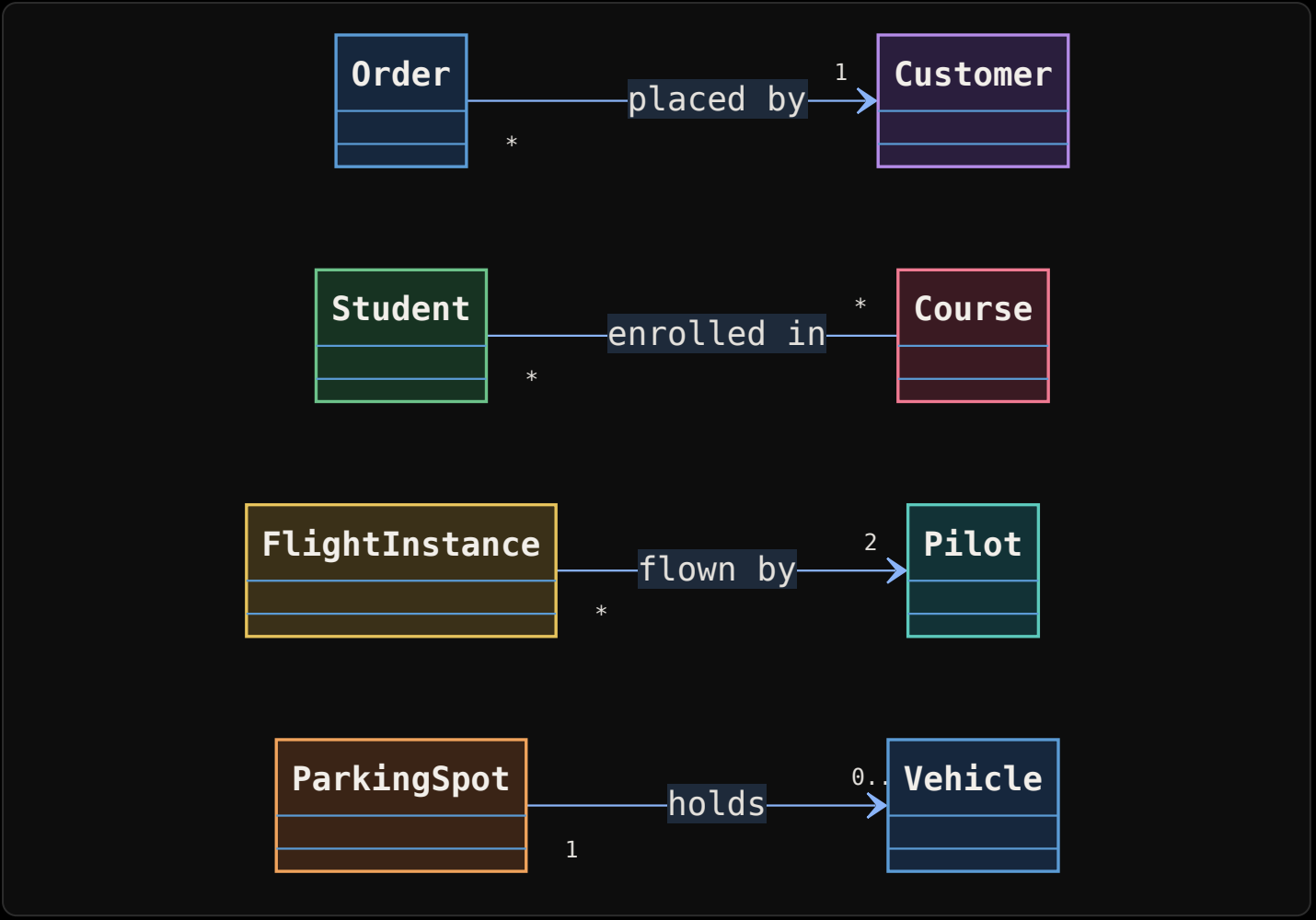
How it works

1. If only one side calls the other, make it **uni-directional**.
2. **Bi-directional** only if real use cases need both directions.
3. For bi-directional, **one side owns the update** and changes both ends in one method.

4. **Enforce multiplicity in code:** `1` → `Objects.requireNonNull`; `0..1` → `Optional<T>`; `*` → collection; `1..*` → non-empty collection; exact numbers → size check in the constructor.

UML notation

Plain solid line = association (navigability unspecified; read as "both know"). **Open arrowhead** = uni-directional (tail knows head). **×** = explicitly not navigable. Numbers sit next to the class they count: `Order "*" --> "1" Customer` = many orders, each with one customer.



Java example

```java
// File: code/lld/ch03/association/Course.java
    public void enroll(Student s) {
        if (students.contains(s)) return;
        if (students.size() == capacity) {
            throw new IllegalStateException(code + " is full (max " + capacity + ")");
        }
        students.add(s);
        s.linkCourse(this);
    }

    public void drop(Student s) {
        if (students.remove(s)) s.unlinkCourse(this);
    }
```

Full runnable code: code/lld/ch03/association/ (see the Full Edition).

- `Order` holds a `final Customer` set with `Objects.requireNonNull`; `Customer` has no field back → **uni-directional**, multiplicity `1`.
- `Student` / `Course` hold sets of each other → **bi-directional many-to-many**; only `Course.enroll/drop` change the link (`Student.linkCourse` is package-private): the **owning-side rule**.
- `FlightInstance` rejects anything but **2** pilots; `ParkingSpot` exposes **0..1** as `Optional<Vehicle>`.

Trade-offs

Uni-directional is simple and low-coupling, but the reverse question needs a query. Bi-directional navigates both ways but has two fields to sync and recursion risk in `toString` / `equals`/JSON. Strict multiplicity checks make invalid states impossible for a little constructor code.

When not to use it · Common mistakes · Real-world examples

Avoid when: Needed only inside one method → a **dependency** (3.5). Don't default to bi-directional. **Mistakes:** Every line bi-directional "just in case"; updating only one side; no multiplicity (and so no check); both sides in `toString()` / `equals()` → stack overflow. **Seen in:** JPA `@ManyToOne` (uni); a bi-directional `@OneToMany(mappedBy=...)` needs an owning side and helpers setting both ends.

In the interview

> **30-second answer:** "Association: a class holds a reference to another and calls it, with no ownership. I make it one-way unless both directions are needed, and write multiplicity on every line because it changes the code: `1` is required, `0..1` an Optional, `1..*` a non-empty collection."

- *Spot holds two motorbikes?* — `0..1` becomes `0..2`: a small collection with a size check.

Self-check

- **Q:** `Order "*" --> "1" Customer`: who has the field? **A:** `Order` (arrow points at the referenced class).

TL;DR

"A knows B", no ownership. Prefer one-way; enforce multiplicity in constructors/types.

3.3 Aggregation

What it is

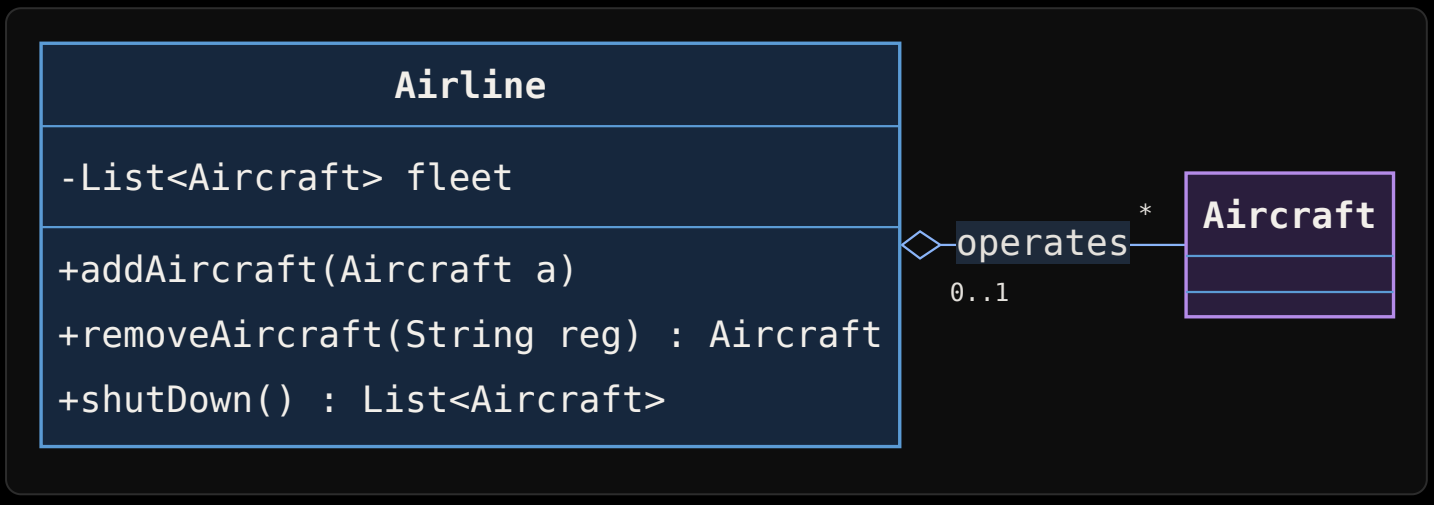
Whole–part association where **parts have their own life**: they exist before, can move, and survive the whole (*shared aggregation*). Analogy: an airline's aircraft are sold, not scrapped.

The problem it solves

It says "these form a group" while making clear the whole does **not** control their lifetime: don't delete parts with the whole.

How it works

1. Parts are **created outside** the whole. 2. They are **passed in** (`addAircraft(a)`). 3. Removing a part **returns it alive**; destroying the whole **releases** parts. 4. **Hollow diamond at the whole**.



Java example

```
// File: code/lld/ch03/aggregation/Airline.java
    public void addAircraft(Aircraft a) { fleet.add(a); }        // part is passed in
    // ... removeAircraft(registration) hands the part back, still alive

    /** Shutting down the airline releases the parts instead of destroying them. */
    public List<Aircraft> shutDown() {
        List<Aircraft> released = List.copyOf(fleet);
        fleet.clear();
        return released;
    }
```

Full runnable code: code/lld/ch03/aggregation/ (see the Full Edition).

Aircraft are built in `Main` before any airline; one is transferred between airlines; after `skyways = null` its released aircraft join `BlueJet`.

Trade-offs

Aggregation adds whole–part **meaning** but usually **no code difference** from association (both are a field). The UML spec leaves its meaning to the modeller and authors like Fowler advise sparing use; interviews still expect it for clear whole–part groups.

When not to use it · Common mistakes · Real-world examples

Avoid when: No whole–part idea (ticket and spot) → association. Parts created by and dying with the whole → composition. **Mistakes:** Diamond at the **part's** end (it goes at the whole); choosing randomly (test: *if the whole is gone, is the part still meaningful?* Yes → aggregation); deleting parts in the whole's close method. **Seen in:** Archiving a Slack channel keeps its users; deleting a playlist keeps the songs.

In the interview

> **30-second answer:** "Aggregation is whole–part where parts outlive the whole, hollow diamond at the whole. Shut down an airline and the aircraft remain. In Java parts are created outside, passed in, never destroyed by the whole."

- *Airline with no aircraft allowed?* — That's multiplicity (`1..*`), not aggregation vs composition.

Self-check

- **Q:** A library closes; do members vanish? **A:** No: Library ◇— Member is aggregation.

TL;DR

"Has-a, parts live on"; passed in, never destroyed; hollow diamond at the whole.

3.4 Composition

What it is

The whole **owns** its parts: each belongs to **exactly one** whole, is usually **created by** it and **dies with** it (or is explicitly removed first). Analogy: house and rooms; floor and spots.

The problem it solves

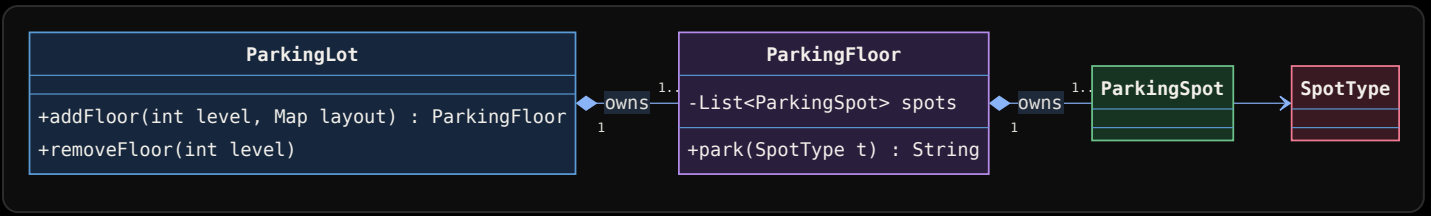
Some objects make no sense alone; letting outsiders create or share them causes bugs (two floors sharing a spot, orphan spots). One owner handles the full life cycle.

How it works

1. The whole **creates** parts (constructor or `addX(data)`, not a ready-made part).

2. The part's constructor is **not public**.
3. Never hand out the internal collection: read-only views or copies.
4. Removing the whole drops the only reference → part becomes garbage.
5. **Filled diamond at the whole**.

Java has no destructors: parts die **logically** (unreachable → GC); for resources the whole's `close()` closes its parts.



Java example

```java
// File: code/lld/ch03/composition/ParkingFloor.java
    ParkingFloor(int level, Map<SpotType, Integer> layout) {
        this.level = level;
        int n = 1;
        for (var e : layout.entrySet()) {
            for (int i = 0; i < e.getValue(); i++) {
                spots.add(new ParkingSpot("L" + level + "-" + n++, e.getKey()));
            }
        }
        if (spots.isEmpty()) throw new IllegalArgumentException("a floor needs 1..* spots");
    }

    public List<ParkingSpot> spots() { return Collections.unmodifiableList(spots); }
```

Full runnable code: code/lld/ch03/composition/ (see the Full Edition).

`ParkingLot.addFloor(level, layout)` takes a *description* and builds the floor itself; `ParkingSpot`'s constructor is package-private; `spots().clear()` throws; `removeFloor(1)` takes its 4 spots with it. Spot kinds are a `SpotType` enum, not subclasses.

Trade-offs

Composition: the whole creates the part, no sharing, part doesn't outlive it; strong encapsulation, lower flexibility. Aggregation: someone else creates it, sharable, outlives the whole; weaker encapsulation, more flexibility.

When not to use it · Common mistakes · Real-world examples

Avoid when: The part has its own business identity (person, aircraft, song) or must be shared. **Mistakes:** Accepting a ready-made part (`addSpot(ParkingSpot s)`) and calling it composition; returning the internal list; confusing it with "favour composition over inheritance" (3.6), where "composition" means *holding* an object, including association/aggregation. **Seen in:** `HashMap`'s internal `Node`s; JPA `cascade = ALL, orphanRemoval = true` (delete an `Order`, lose its `OrderLine`s).

In the interview

30-second answer: "Composition: the whole creates its parts, they belong only to it and die with it; filled diamond at the whole. In Java I build parts inside the owner, hide their constructor, and expose read-only views."

- *With GC, how does a part 'die'?* — Logically: the whole held the only reference.

Self-check

- **Q:** Why is `ParkingSpot`'s constructor package-private? **A:** Only the owning floor may create spots.

TL;DR

"Owns, parts die with it"; whole creates, never shares, read-only views; filled diamond.

3.5 Dependency

What it is

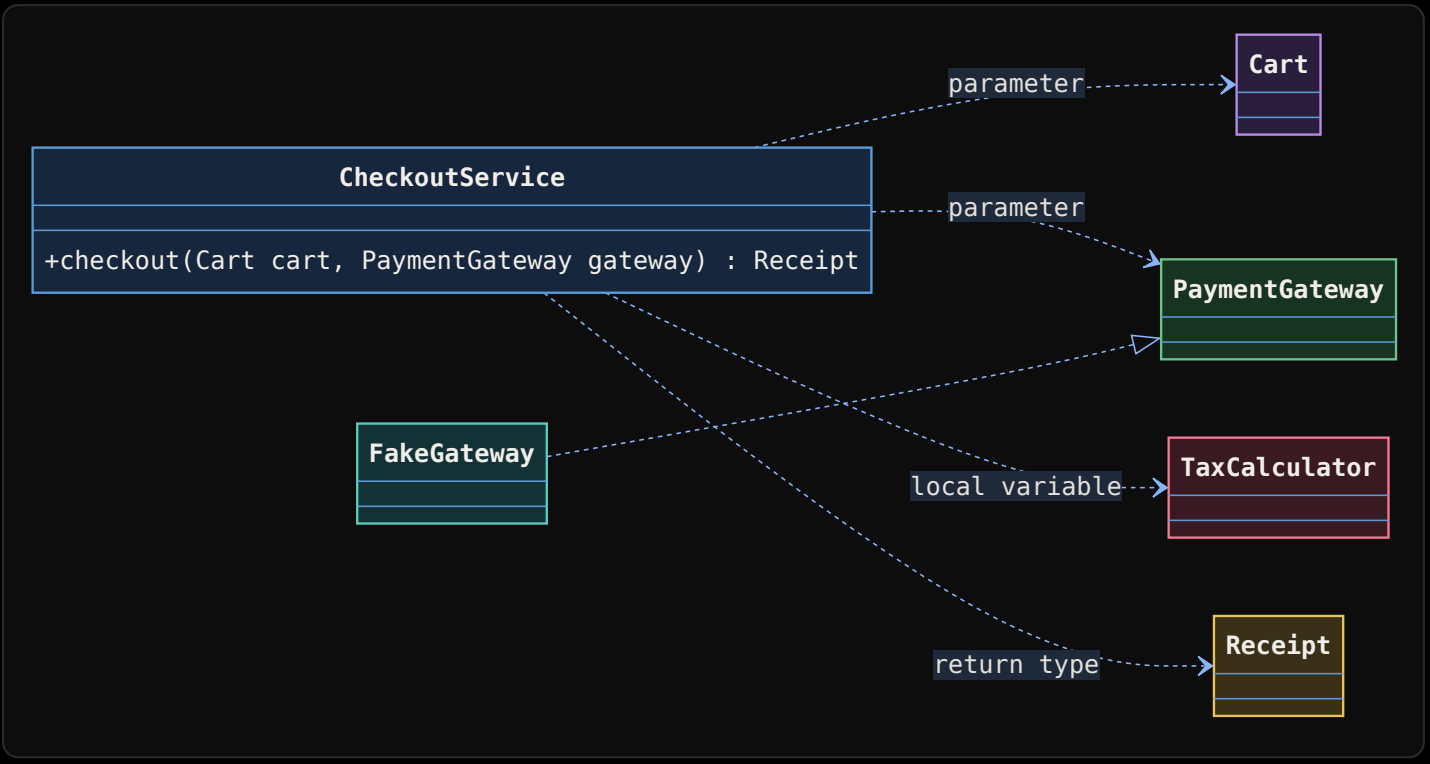
A **uses** B briefly, inside one method, with **no field**: B is a **parameter**, **local variable**, **return type** or **static call**. Analogy: a chef uses today's delivery van.

The problem it solves

Records the weakest link ("if B's interface changes, A may change"); keeps classes stateless and lets callers pass a different collaborator per call.

How it works

B as parameter / `new` local / return type / static call; no field of type B. UML: **dashed line, open arrowhead** from A to B, optionally `«use»` / `«create»`.



Java example

```java
// File: code/lld/ch03/dependency/CheckoutService.java
public final class CheckoutService {

    public Receipt checkout(Cart cart, PaymentGateway gateway) {
        TaxCalculator tax = new TaxCalculator();            // local-variable dependency
        Money subtotal = cart.subtotal();                   // parameter dependency
        Money total = subtotal.plus(tax.taxOn(subtotal));
        String txn = gateway.charge(total);                 // parameter dependency (interface)
        return new Receipt(txn, subtotal, total);           // return-type dependency
    }

    public record Receipt(String txnId, Money subtotal, Money total) { }
}
```

Full runnable code: code/lld/ch03/dependency/ (see the Full Edition).

`CheckoutService` has **no fields**; `Main` passes a `FakeGateway` and then a lambda gateway on the next call. `Money` uses `BigDecimal`.

Trade-offs

Dependency: lowest coupling, stateless, swap per call, but callers pass B every time. Association (field): higher coupling and state, swap per object, pass once.

When not to use it · Common mistakes · Real-world examples

Avoid when: When A uses B across many methods (a service and its repository), a constructor-injected field is cleaner. **Mistakes:** Solid association line for a mere parameter; drawing every dependency (`String`, `List`...); hiding `new ConcreteThing()` deep inside where the caller should decide (blocks testing). **Seen in:** `Collections.sort(list, comparator)`; `LocalDate.now(Clock clock)`.

In the interview

> **30-second answer:** "Dependency is 'uses briefly': parameter, local or return type, no field; dashed arrow. It's the loosest coupling, so a collaborator needed in one method is passed in, not stored."

- *Relation to DI?* — DI is *who supplies* the collaborator; it may be stored (association) or passed per call (dependency). DIP adds "depend on interfaces" (Chapter 4).

Self-check

- **Q:** Dashed + open arrow vs dashed + hollow triangle? **A:** Dependency vs realization.

TL;DR

"Uses for one call"; no field; dashed arrow.

3.6 Generalization (inheritance)

What it is

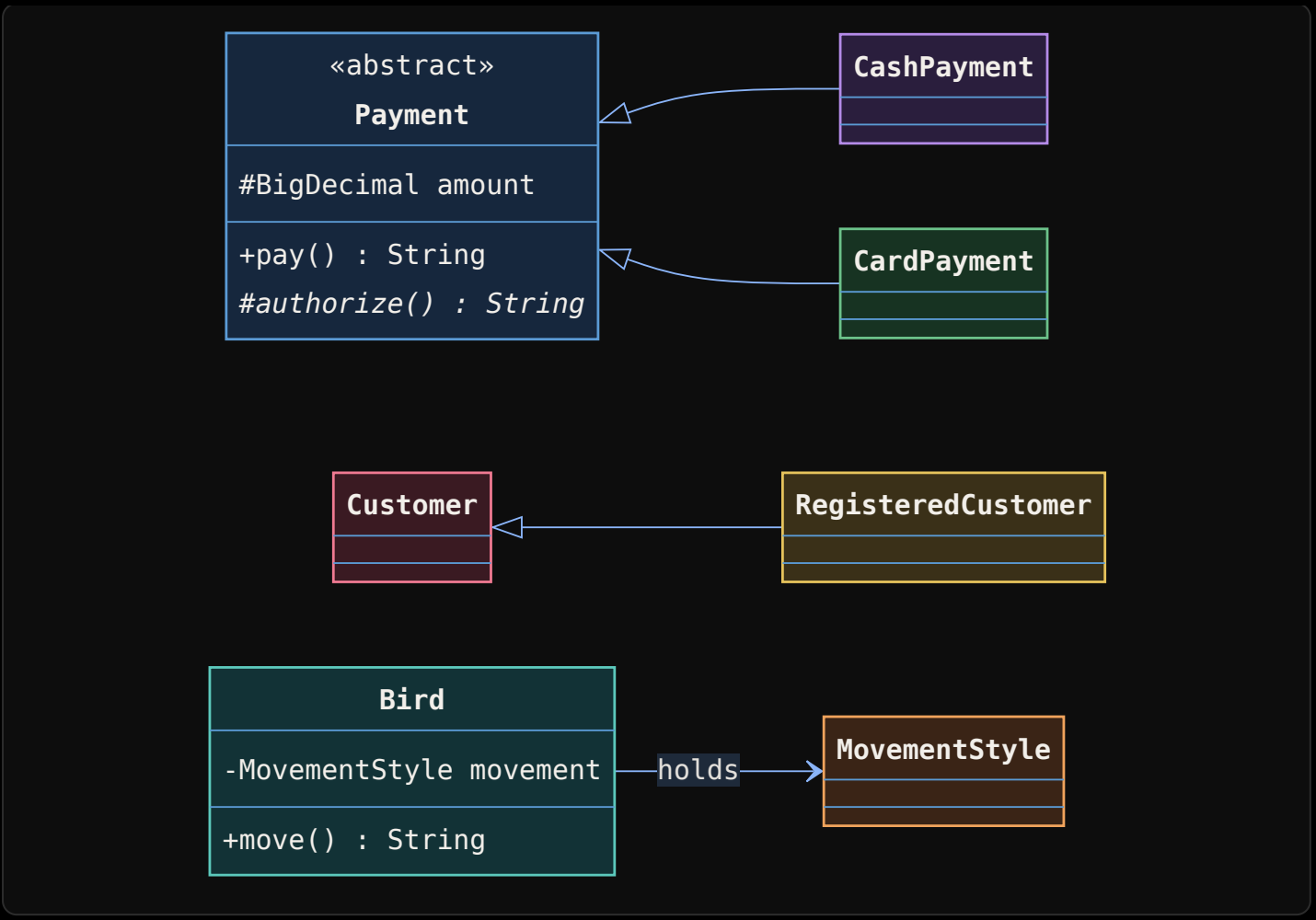
UML's word for **inheritance**: B `extends` A, so B *is a kind of* A. Analogy: a card payment *is a* payment.

The problem it solves

Classes sharing a contract **and** behaviour keep shared parts in one place, and callers use the general type (**polymorphism**).

How it works

1. Run the **substitution test** (Liskov, Chapter 4): can B go anywhere A is expected with no refused or broken method?
2. Yes → shared state/workflow in A (often `abstract`), subclasses override varying steps.
3. No → **hold instead of extend** (an interface-typed field).
4. UML: **solid line, hollow triangle at the parent**; abstract names in *italics* or `{abstract}`.



Java example

```java
// File: code/lld/ch03/generalization/Payment.java
public abstract class Payment {
    protected final BigDecimal amount;   // '#' protected: visible to subclasses
    // ... constructor rejects non-positive amounts

    public final String pay() {
        String ref = authorize();
        return getClass().getSimpleName() + " of " + amount + " -> " + ref;
    }

    protected abstract String authorize();   // abstract operation (italics in UML)
}
```

Full runnable code: code/lld/ch03/generalization/ (see the Full Edition).

- **Good hierarchy:** cash and card both truly take money, so `Main` loops over `List<Payment>`; amounts are `BigDecimal` compared with `compareTo`.
- **Fix #1, subclass that must refuse:** a `Guest extends Customer` that can't order breaks substitution → `Customer` only browses; ordering moves to `RegisteredCustomer`.
- **Fix #2, subclass that only carries a constant:** `CompactSpot` / `LargeSpot` → a `SpotType` enum field.
- **Hold, don't extend:** `Penguin` breaks a `FlyingBird` hierarchy; `Bird` holds a `MovementStyle` field (FLY, SWIM, WALK) and `setMovement` swaps it at run time.

Trade-offs

Inheritance: automatic reuse, fixed at compile time, tight coupling (fragile base class), one parent; best for true is-a with different behaviour. Holding an object: explicit delegation, swappable at run time, loose coupling, many fields/interfaces; best for varying or combinable behaviour.

When not to use it · Common mistakes · Real-world examples

Avoid when: Just sharing fields/helpers; a child would throw `UnsupportedOperationException`; data-only variation; mixable behaviours (fly + swim + walk → class explosion). **Mistakes:** Hierarchies deeper than 2–3; `Stack extends Vector` style leaks; triangle pointing at the child; inheriting when the relation holds only *sometimes* (use a role/interface). **Seen in:** `ArrayList extends AbstractList`; `FileInputStream extends InputStream`; `IOException extends Exception`. Counter-examples: `Stack extends Vector`, `Properties extends Hashtable`.

In the interview

> **30-second answer:** "Generalization is inheritance, solid line with a hollow triangle at the parent. Only when the child passes the substitution test. A subclass that only passes a constant becomes an enum field; varying behaviour I hold as an interface."

- *Why not `CompactSpot extends ParkingSpot`?* — No behaviour; a `SpotType` field does it.

Self-check

- **Q:** Guest can't order but `Customer.placeOrder()` exists. **A:** LSP broken; move `placeOrder` to `RegisteredCustomer`.

TL;DR

"Is-a", `extends`, only after the substitution test; triangle at the parent.

3.7 Realization (interface implementation)

What it is

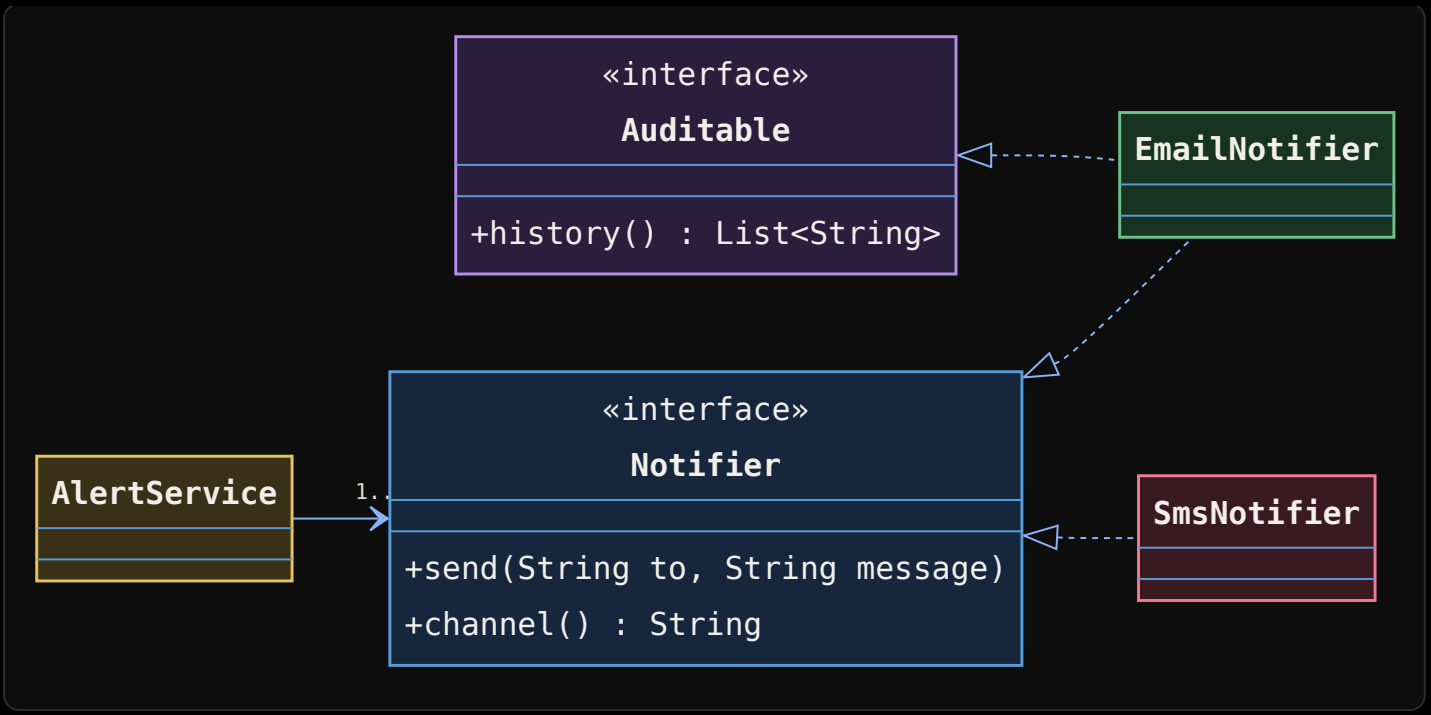
A class **implements an interface** (a stateless contract): `class EmailNotifier implements Notifier`. Analogy: a plug that realizes a socket standard.

The problem it solves

Callers depend on the **contract**: new implementations need no caller change, fakes for tests, several roles per class.

How it works

Define the operations callers need; classes `implement` them; callers hold the **interface type**; `default`/`static` methods allowed. UML: **dashed line, hollow triangle at the interface**, `«interface»` (or a "lollipop"). Same is-a idea as generalization, but only the contract is inherited.



Java example

```java
// File: code/lld/ch03/realization/AlertService.java
public final class AlertService {
    private final List<Notifier> notifiers;

    public AlertService(List<Notifier> notifiers) { this.notifiers = List.copyOf(notifiers); }

    public void alert(String to, String message) {
        for (Notifier n : notifiers) {
            System.out.println("via " + n.channel());
            n.send(to, message);
        }
    }
}
```

Full runnable code: code/lld/ch03/realization/ (see the Full Edition).

`EmailNotifier` realizes **two** interfaces; `SmsNotifier` overrides the `default channel()`; a **lambda** also realizes `Notifier` (functional interface). `AlertService` depends only on the interface, so a `PushNotifier` needs no change.

Trade-offs

Interface: many per class, no instance fields, shared code only via `default`; for roles, plug-ins, test seams. Abstract class: one parent, any fields and code; for a family sharing state and workflow.

When not to use it · Common mistakes · Real-world examples

Avoid when: A single never-varying implementation with no test need (YAGNI); empty "marker" interfaces (annotations serve better). **Mistakes:** Solid instead of dashed line; fat interfaces (ISP); variables typed with the concrete class. **Seen in:** JDBC `Connection` realized by vendor drivers; SLF4J `Logger` realized by Logback/Log4j.

In the interview

> **30-second answer:** "Realization: a class implements an interface, dashed line with a hollow triangle at the interface. I add interfaces where behaviour varies or I need a test double (gateway, notifier, pricing), and the design depends only on them."

- *Interface or abstract class?* — Interface for capabilities (many allowed); abstract class for shared state/workflow; combine as `List` → `AbstractList` → `ArrayList`.

Self-check

- **Q:** Generalization vs realization drawn? **A:** Same hollow triangle; solid vs dashed line.

TL;DR

"Implements": dashed line + hollow triangle at `«interface»`; depend on the interface.

3.8 Decision guide: choosing the relationship between two classes

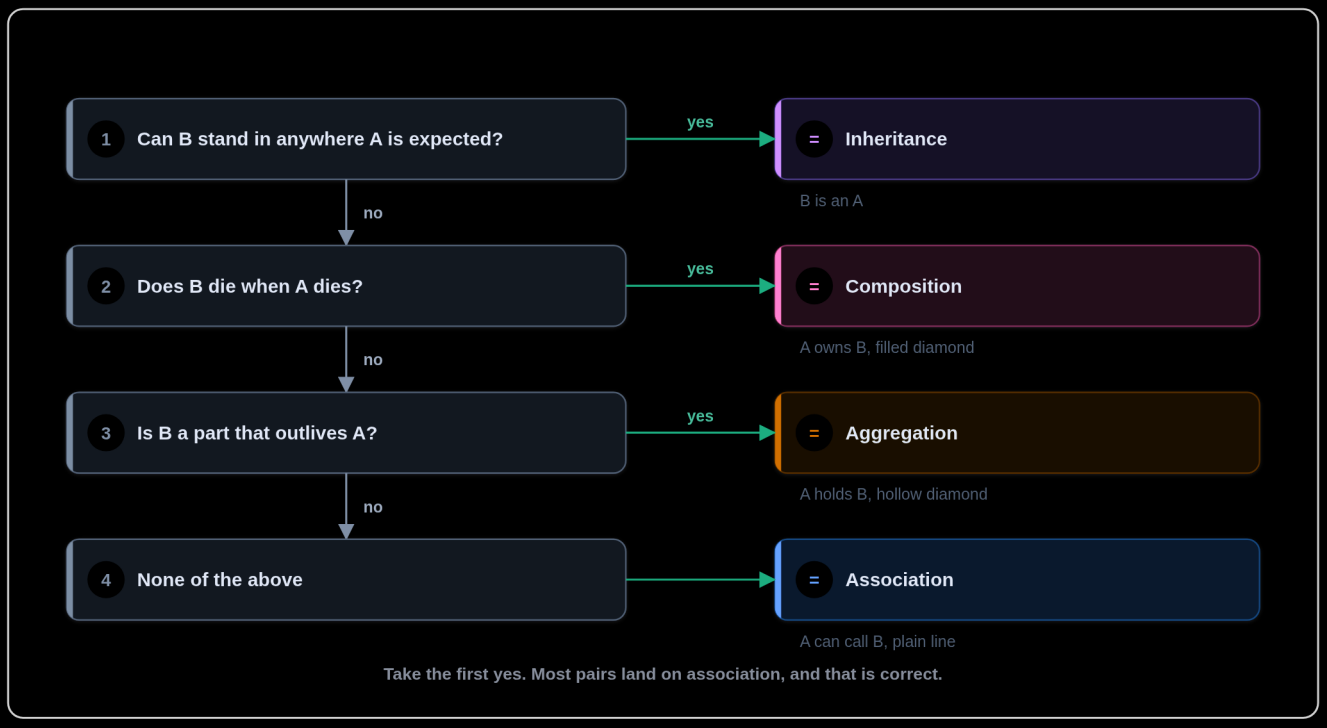
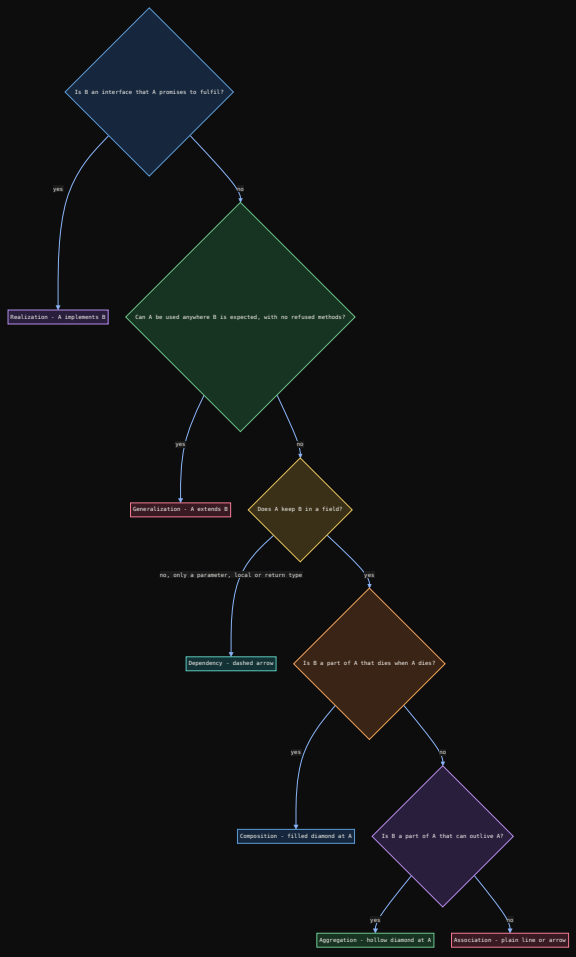


Figure — The four-question decision: inheritance, then composition, then aggregation, else association (course diagram)

Ask in order; stop at the first yes. Inheritance comes first only because it's easiest to **rule out**; most pairs end as association.



Summary table (keep this one)

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency	A *uses* B	dashed, open arrow → B	parameter/local/return/static call	none	`CheckoutService` uses `PaymentGateway`
Association	A *knows* B	solid (arrow if one-way)	field, B made elsewhere	independent	`Order` → `Customer`
Aggregation	A *has* B	solid, hollow ◇ at A	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition	A *owns* B	solid, filled ◆ at A	A creates B, hides it	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Mermaid cheat-sheet: `A ..> B` dependency · `A --> B` association A→B · `A -- B` association (both/unspecified) · `A o-- B` aggregation (A whole) · `A *-- B` composition (A whole) · `B <|.. A` A realizes B · `B <|-- A` A extends B · `A "1" *-- "1..*" B` with multiplicities.

Common mistakes (all relationships)

Diamond not at the **whole**; triangle not at the **parent/interface**; lines without multiplicity; inheritance to share data.

In the interview

Say the test **aloud** per line ("remove the floor and its spots go: composition"); collapse constant-only subclasses yourself. Expect "why not inheritance?" (substitution test) and multiplicity changes.

Self-check

- **Q:** `Library` / `Member`? **A:** Aggregation or association. **Q:** `Flight` / `WeeklySchedule`? **A:** Composition.
- **Q:** `ParkingTicket` / `ParkingSpot`? **A:** Association. **Q:** `Report.render(Formatter f)`? **A:** Dependency.

TL;DR

Is-a only after the substitution test; composition if parts die with the whole, aggregation if they live on, association if one just knows the other, dependency if only used in a method.

Part B — UML

3.9 What UML is (structural vs behavioural diagrams)

What it is

UML is the OMG's standard design notation (UML 2.5.1): not a programming language, a whiteboard suffices. It merged Booch, OMT (Rumbaugh) and OOSE (Jacobson) in the 1990s, hence its size.

The problem it solves

Makes a design **visible**, forces **explicit decisions** (filled or hollow diamond?), and makes **disagreement cheap** (the interviewer taps a line and asks why).

How it works — the 14 diagram types

Structural (what it is made of): **Class** (always), Object, Package, Component, Composite structure, Deployment, Profile. **Behavioural** (what it does): **Use case**, **Activity**, **State machine**, and the **interaction** diagrams **Sequence**, Communication, Interaction overview, Timing. Component/deployment belong to system design rounds; profile is never used in interviews.

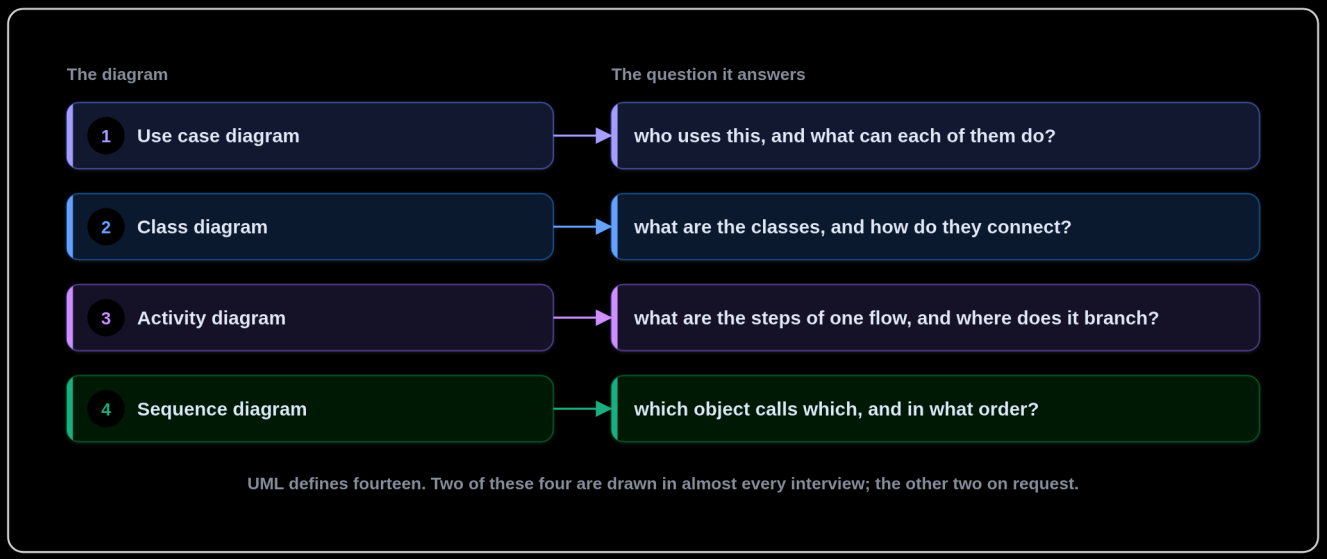


Figure — The four diagrams an OOD interview uses and the question each answers (course diagram)

Interview set: **use case** (who does what; first), **class** (the answer, maps to code), **activity** (steps and branches of one flow), **sequence** (who calls whom), plus **state machine** when an object has a lifecycle.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Shared vocabulary and tools (PlantUML, Mermaid) vs a big spec, whiteboard time, stale diagrams. Avoid unmaintained formal docs; sketch UML, C4 or Mermaid beside the code is the norm, UML-lite for interviews. **Mistakes:** Thinking notation is graded: what's graded is whether **diagram and words agree** (say what a symbol means if unsure). A line you can't justify hurts more than a missing one. **Seen in:** Mermaid in GitHub/GitLab design docs; PlantUML in Confluence.

In the interview

> **30-second answer:** "UML has 14 diagrams: structural (class, object, component...) and behavioural (use case, activity, state machine, sequence...). In an OOD round I use a quick use-case list, one class diagram, and an activity or sequence diagram for the key flow."

Self-check

- **Q:** Sequence diagram family? **A:** Behavioural (interaction). **Q:** How many types? **A:** 14 (7 + 7).

TL;DR

Standard symbols, not code; structural vs behavioural; interviews use class, use case, activity, sequence, state.

3.10 Use-case diagrams

What it is

Shows **who** uses the system and **what** each can achieve; no classes, data or order.

- **Actor** — a *role* outside the system (stick figure); may be non-human (payment gateway, timer).
- **Use case** — one goal with a result the actor cares about (oval, verb phrase): "Borrow a book", not "click search".
- **System boundary** — rectangle: inside = what you build.

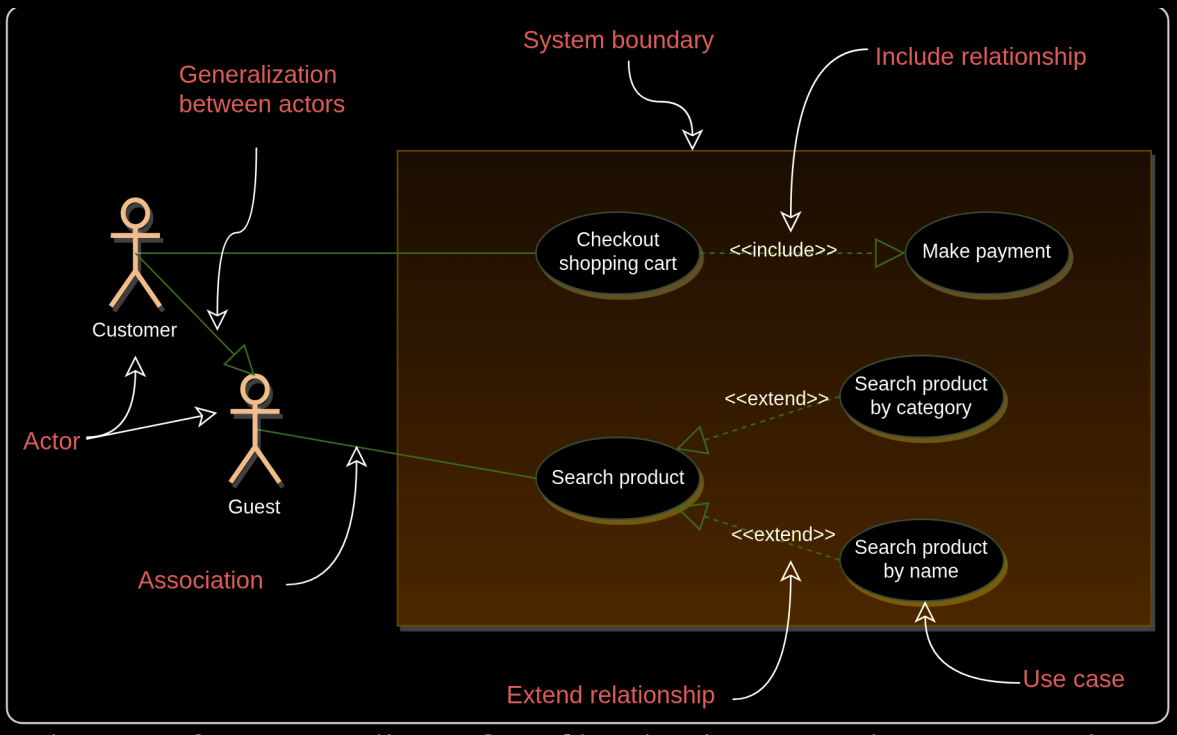


Figure — The parts of a use-case diagram for online shopping: actors (Customer, Guest), use cases, system boundary, generalization between actors, and «include» / «extend» relationships (course diagram)

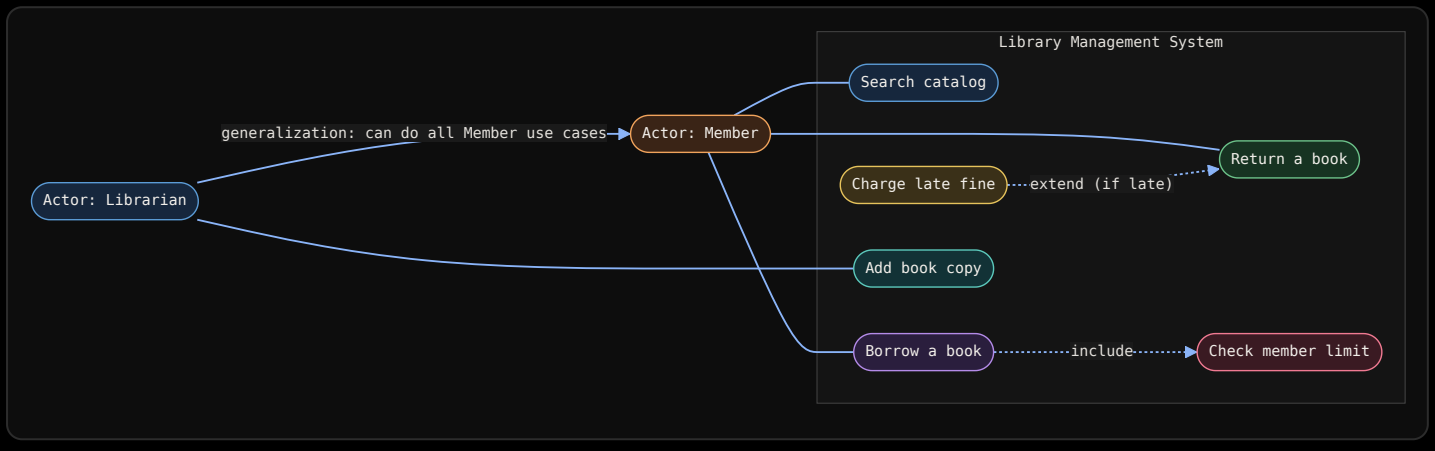
The problem it solves

Turns a one-liner into an **agreed scope** ("these nine are in; refunds and reporting are out"). **Actors usually become classes; use cases become methods.**

How it works

1. Find actors: *who starts something? who administers it* (most forgotten)? *who gets told? what acts with nobody present* (timers, jobs)?
2. Primary actors left, secondary (gateway) right; use cases inside the boundary, joined to actors by plain lines.
3. **«include»** = *always*; dashed arrow **base → included** (*Borrow* «include» *Check limit*). **«extend»** = *sometimes*, at an extension point; arrow **extension → base** (*Charge fine* «extend» *Return*). Test: can the base complete without it? Yes → extend.
4. **Generalization** (hollow triangle) between actors or use cases. Keywords are singular; old «uses» = «include».

In the course figure the triangle at Guest means a Customer can do all a Guest can, plus check out.



From use cases to code (see `lld.ch03.library`)

Use-case element	Java
Actor *Member* / *Librarian*	`class Member` / `class Librarian extends Member`
Borrow a book + «include» *Check limit*	`Member.checkout(...)` calling private `checkLimit()` **every** time
Return a book + «extend» *Charge fine*	`Member.returnBook(...)` calling `chargeFine(...)` **only if** late
Add book copy	`Librarian.addCopy(...)`

A starting point: some use-case methods belong on a service class.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Fast scope agreement and finds forgotten actors, but shows no order or data and bloats with UI steps. If requirements are already precise, a spoken list suffices. **Mistakes:** UI steps as use cases; one actor per person (a borrowing librarian is two roles); reversed «extend»; «include» for something occasional; no boundary. **Seen in:** *Place order* «include» *Make payment*; *Apply gift card* «extend»s it.

In the interview

> **30-second answer:** "Before classes I list actors (roles, including admins, external services, timers) and their goals, with a boundary for scope. «include» is always, «extend» sometimes. Actors become classes, use cases methods." Takes 3–5 minutes, early.

Self-check

- **Q:** «include» arrow direction? **A:** Base → included.
- **Q:** Is a payment gateway an actor? **A:** Yes, secondary.

TL;DR

Actors outside, goals inside, boundary = scope; include always, extend sometimes.

3.11 Class diagrams

What it is

The **static structure**: classes, attributes, operations, relationships with multiplicity. The answer to an OOD question, and the only UML diagram mapping directly to code.

The problem it solves

Puts the whole design on one page to review, challenge and code; other diagrams lead to it or test it.

How it works — the class box

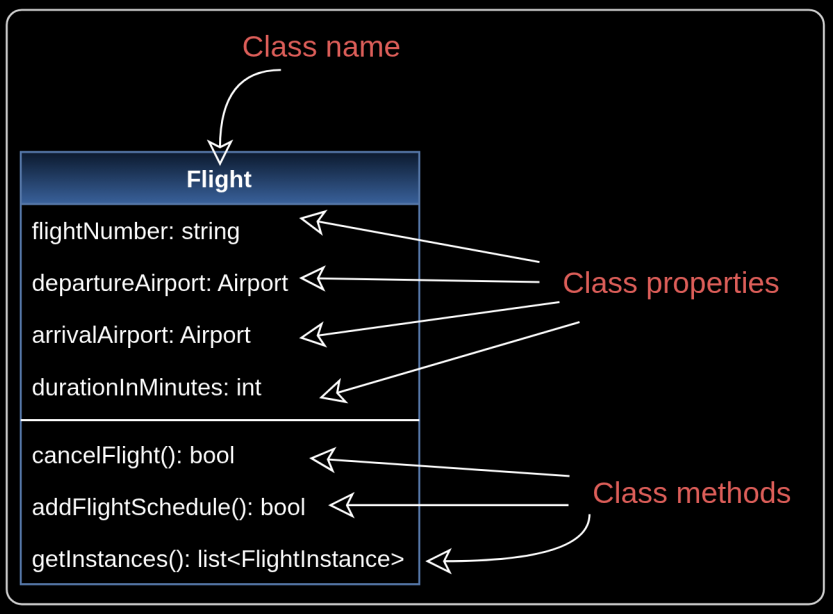


Figure — A class box: name on top, properties (attributes) in the middle, methods (operations) at the bottom, using a Flight class (course diagram)

```
visibility name : Type [multiplicity] = default      e.g.  - spots : ParkingSpot [1..*]
visibility name(param : Type, ...) : ReturnType      e.g.  + park(v : Vehicle) : Ticket
```

Visibility: `+` public, `-` private, `#` protected (Java's `protected` also opens the package), `~` package (no modifier). *Italic* or `{abstract}` = abstract; `«interface»`, `«enumeration»`; underlined = static; `{readOnly}` = `final`.

A name-only box is fine early. Mermaid: same visibility symbols, `*` = abstract, `$` = static, `List~T~` for generics.

How it works — the relationships (all arrow types)

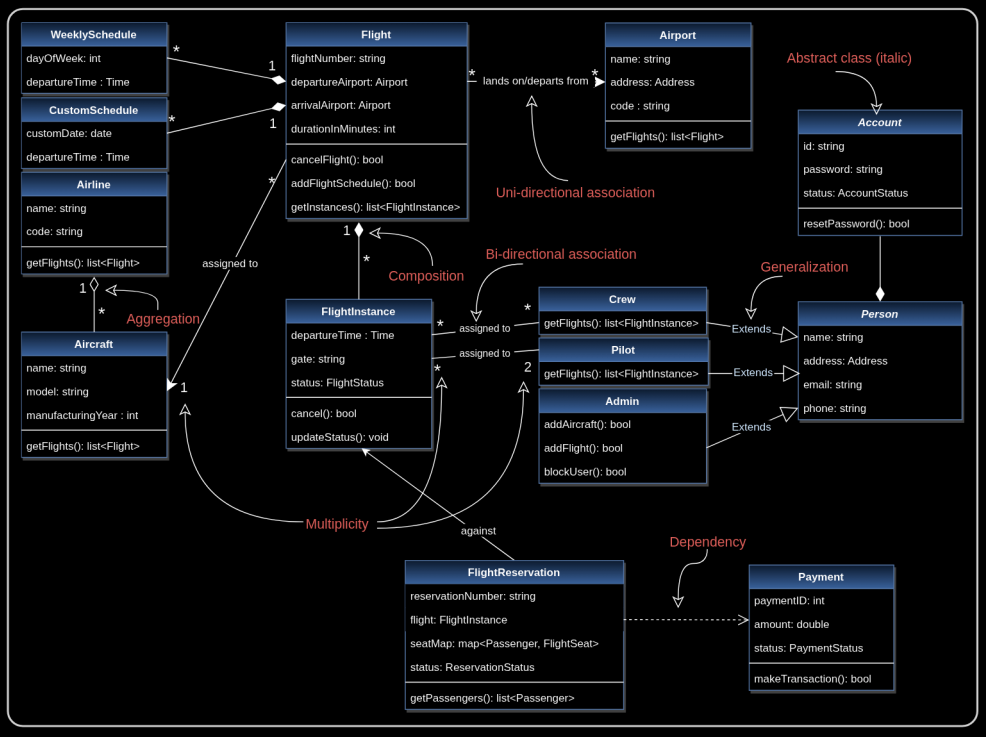
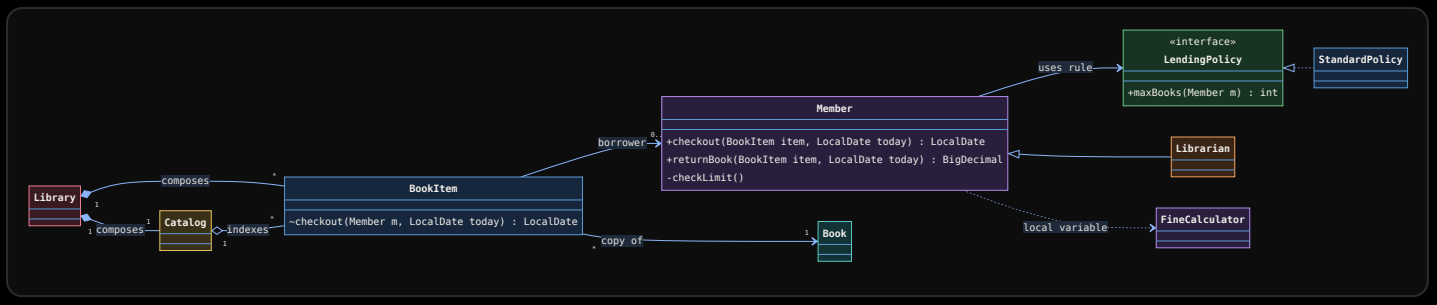


Figure — An airline-system class diagram using every relationship: associations (e.g. Flight to Aircraft, uni-directional), multiplicities (FlightInstance has 2 Pilots), aggregation (Airline–Aircraft), composition (Flight–WeeklySchedule), generalization (Person to Crew/Pilot/Admin) and dependency (course diagram)

The figure uses every line: association FlightInstance — Pilot (two-way) and Flight → Airport (one-way); multiplicity `*` — `2` Pilot; aggregation Airline ◇— Aircraft; composition Flight ◆— WeeklySchedule; generalization Pilot/Crew/Admin → Person; dependency FlightReservation ⇢ Payment. Keep arrows consistent with fields.

Also recognise: **association class** (dashed-attached class holding link data, e.g. `Enrollment{grade}`) and **qualified association** (key box, `Library [isbn] → Book` = a `Map<Isbn, Book>`).

Worked example — the library design (code in `lld.ch03.library`)



Every line type appears; `~` marks package-private methods like `BookItem.checkout`, callable only by `Member`.

How much detail to draw

10–15 named boxes; attributes/operations only where flows need them; **every line typed with multiplicity**; **drop attributes before relationships**.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Detailed = near code-ready but 10+ minutes; sketch = fast and changeable. It shows no order or time: use behaviour diagrams for that. **Mistakes:** Unlabelled lines; diamonds/triangles at the wrong end; every getter and `String` field listed; lines with no matching field in code (or vice versa). **Seen in:** JPA entity diagrams are class diagrams with multiplicities.

In the interview

> **30-second answer:** "Named boxes from nouns and actors, then every line with type and multiplicity, saying the test aloud; only the attributes and methods the key flows need. `+ - # ~` for visibility, «abstract», «interface»."

- `#` *nuance in Java?* — Also visible within the package. *Static?* — Underline (Mermaid `$`).

Self-check

- **Q:** `~`? **A:** Package visibility. **Q:** Aggregation vs composition test? **A:** Is the part meaningful once the whole is gone?

TL;DR

Box = name / attributes / operations; seven line types plus multiplicity on every line.

3.12 Sequence diagrams

What it is

Which object calls which, in what order, for one scenario: objects across the top, **time down the page**.

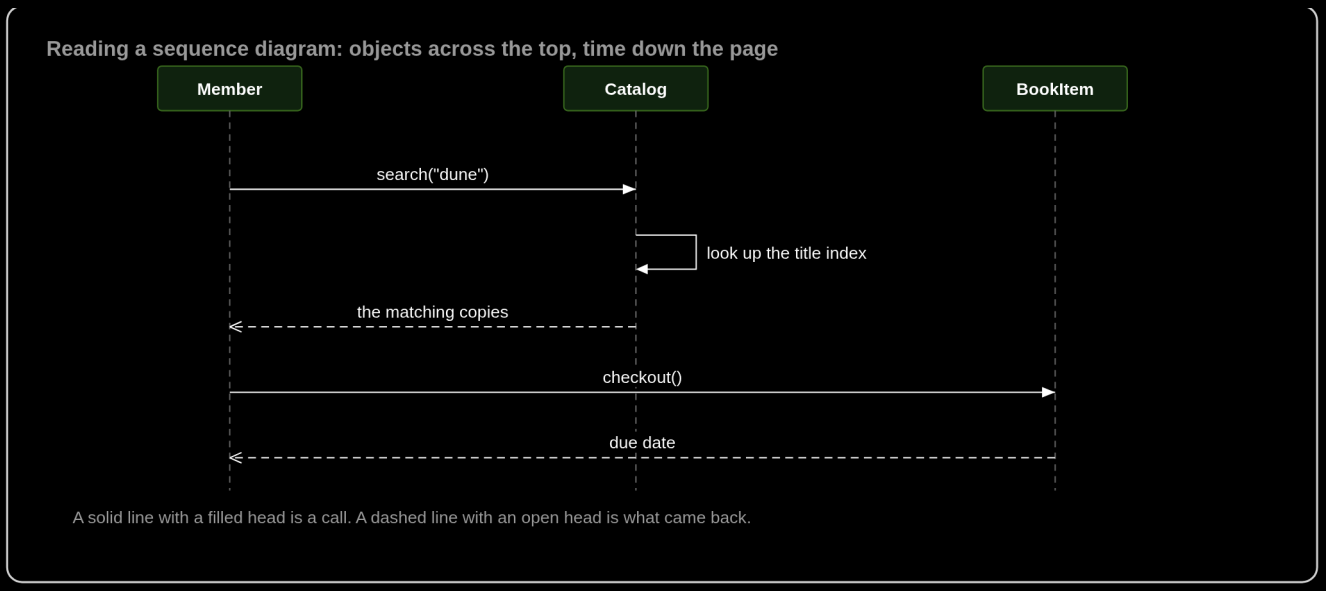


Figure — A member searches the catalog for "dune" and borrows a copy: lifelines, a self-call on Catalog, calls with filled arrowheads and dashed returns (course diagram)

The problem it solves

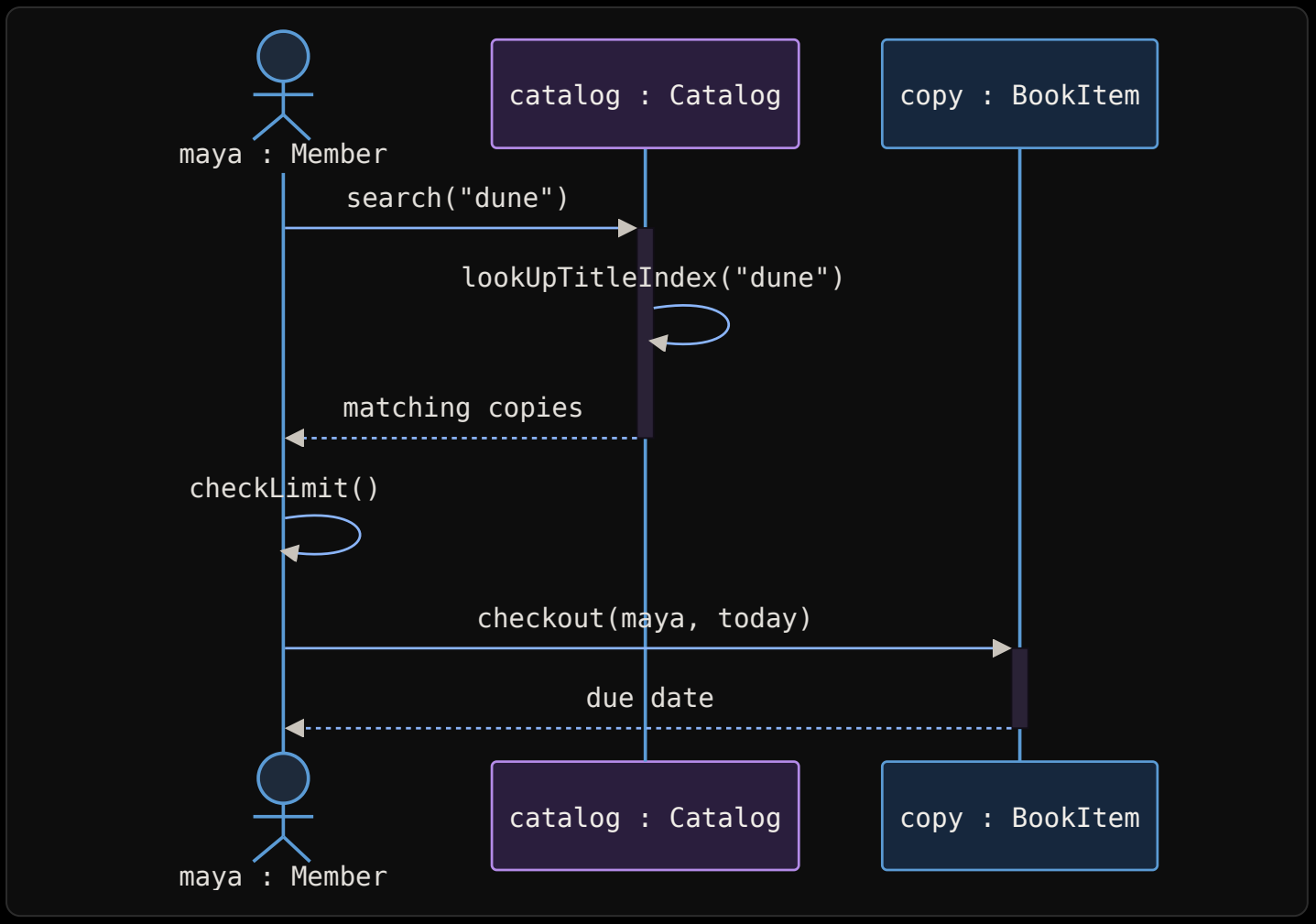
Shows what objects *actually* say and when: vital when order is easy to get wrong (reserve → pay → confirm), when several objects take part, or to explain a class diagram.

How it works — the notation

- **Lifeline:** box + dashed vertical line, one object (`catalog : Catalog`); **actor** (stick figure) starts the flow.
- **Sync message:** solid, **filled** head (caller waits). **Async:** solid, **open** head (fire-and-forget). **Return:** dashed, open head (only when it matters). **Self-message:** loops back.
- **Activation bar:** thin rectangle, object busy. **Create:** dashed «create» arrow to a new box; **destroy:** **X**.

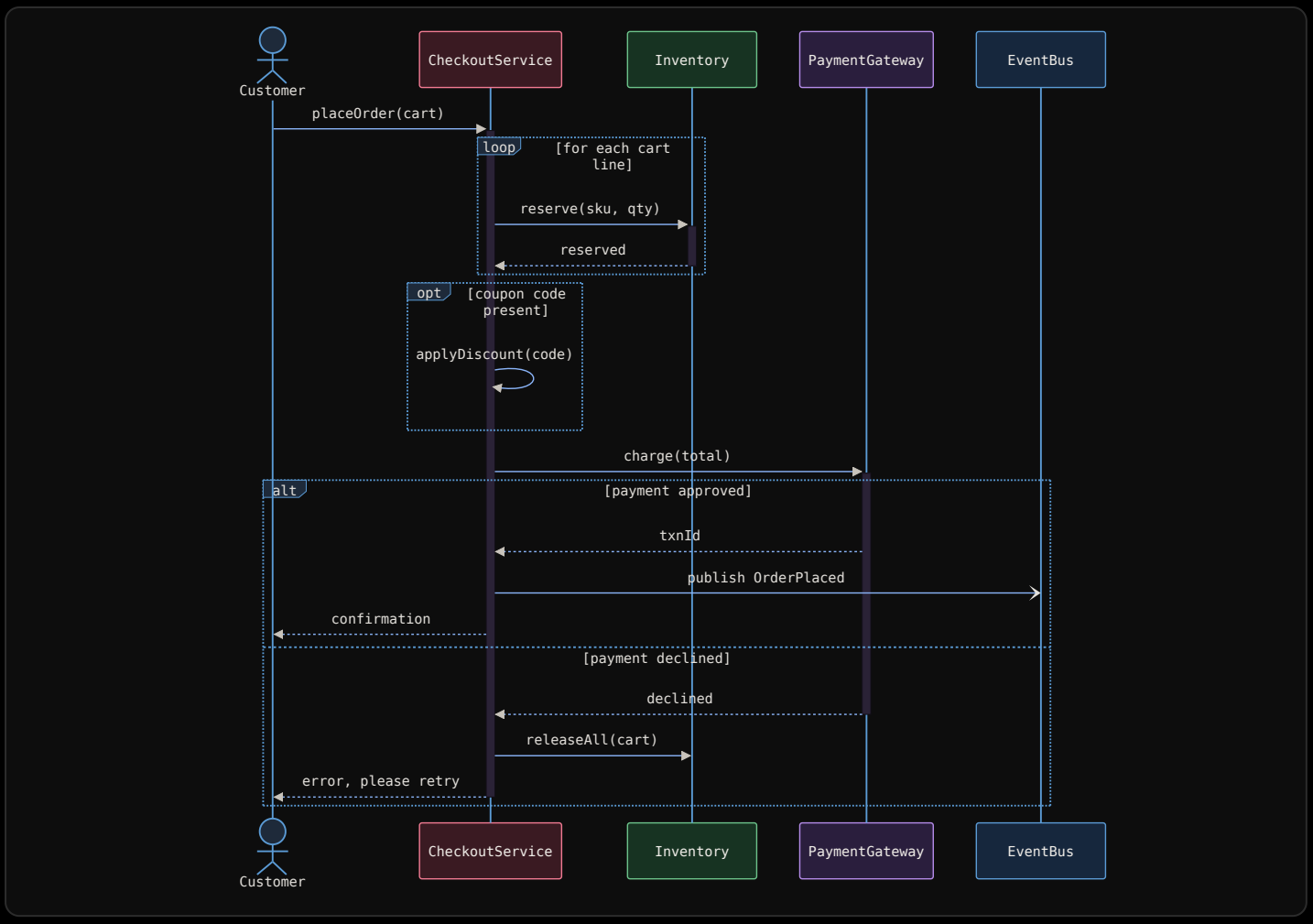
Fragments (guards in `[ ]`): `alt` if/else, `opt` if, `loop`, `par` parallel, `break` early exit, `critical` atomic, `ref` another diagram (others like `neg`, `strict` aren't needed). Whiteboards usually skip bars and fragments.

Example 1 — the library checkout, matching the Java code



The flow starts outside; the catalog **calls itself**; only two returns are drawn; the member talks **directly** to the copy, a choice the interviewer can question.

Example 2 — all fragments and async messages (checkout with payment)



`-)` is an **asynchronous** message (publishing doesn't wait). Point: reserve stock *before* charging; release it if the charge fails.

Java code for Example 1 (`lld.ch03.library`)

```
// File: code/lld/ch03/library/Member.java
    public LocalDate checkout(BookItem item, LocalDate today) {
        checkLimit();                                  // <<include>>: happens every time
        LocalDate due = item.checkout(this, today);
        onLoan.add(item);
        return due;
    }

    public BigDecimal returnBook(BookItem item, LocalDate today) {
        if (!onLoan.remove(item)) throw new IllegalStateException(name + " does not hold " + item.barcode());
        LocalDate due = item.checkIn();
        BigDecimal fine = new FineCalculator().fineFor(due, today);
        if (fine.signum() > 0) chargeFine(fine);       // <<extend>>: only when late
        return fine;
    }
```

Full runnable code: code/lld/ch03/library/ (see the Full Edition). `Catalog.search` calls its private `lookUpTitleIndex` (the self-call).

Output (excerpt): `Maya borrowed B-1, due 2026-03-15`, `Blocked: Maya already has 2 books (limit)`, `Returned B-3 3 days late, fine = 15.00`. **Every arrow is a real method; a message with no method means the class diagram is missing one.**

Trade-offs

Good at call order, sync vs async, spotting missing methods; poor at branch-heavy flows, the whole design, and speed.

When not to use it · Common mistakes · Real-world examples

Avoid when: Three or more decisions → **activity diagram** (a second `alt` / `opt` is the warning sign). Unprompted in most OOD rounds, it steals class-diagram time. **Mistakes:** A return for every call; messages that aren't methods; mixed-up arrowheads; drawing classes when two objects of one class take part (`from : Account`, `to : Account`). **Seen in:** OAuth/OIDC login flows; payment "create order → redirect → webhook" (async webhook); sagas with compensating calls.

In the interview

> **30-second answer:** "One scenario, objects across, time down. Filled arrows sync, open async, dashed returns; `alt` / `opt` / `loop` for if/else, if, loops. Three or four objects, six to eight messages, and every arrow is a method on my class diagram."

- *Sequence vs activity?* — Sequence when *who calls whom* matters; activity when branching does.

Self-check

- **Q:** Thin rectangle on a lifeline? **A:** Activation bar. **Q:** "Retry up to 3 times"? **A:** `loop [attempt <= 3]`.

TL;DR

Objects across, time down; filled sync, open async, dashed return; keep it small.

3.13 Activity diagrams

What it is

Traces **one flow end to end** with its **branches** (and parallel steps): the flowchart of UML.

The problem it solves

Makes the flow's `if/unless/otherwise` logic checkable, and **tests the class diagram**: for every action, *which class does this?* No home = missing class.

How it works — the symbols

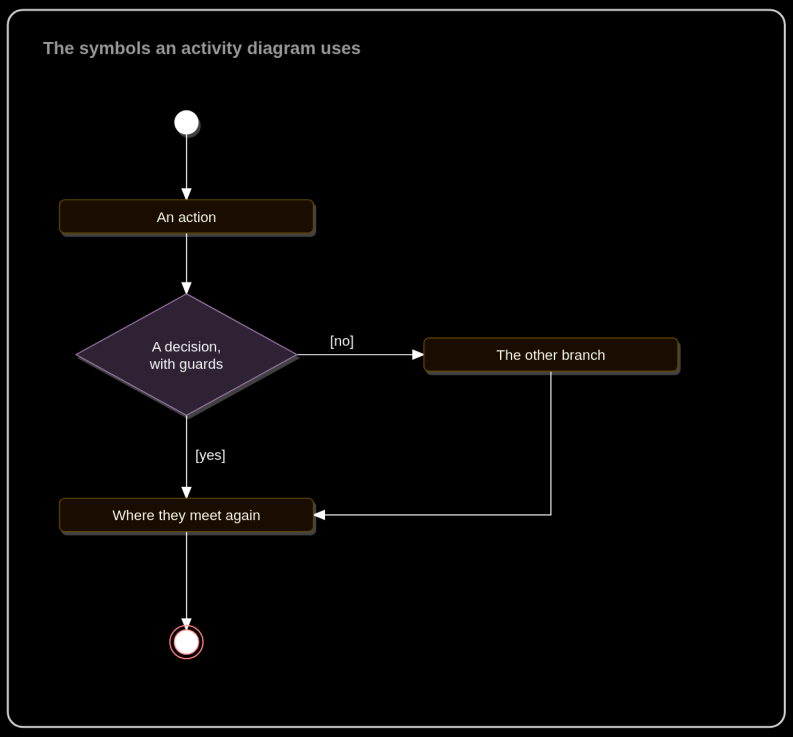


Figure — The core activity-diagram symbols: start, action, a decision with [yes]/[no] guards, the other branch, where the branches meet again, and the end (course diagram)

Initial node (filled circle, draw one) · **action** (rounded rectangle, one step) · **decision/merge** (diamond; **guards** must cover every case, use `[else]`) · **fork/join** (thick bar: start parallel actions / wait for **all**) · **activity final** (bullseye; whole activity stops) · **flow final** (circle with X; only this branch) · **swimlane** (who does each action) · object nodes and signals for data and events.

Steps: pick the flow with an "if/unless/otherwise"; actions in business language ("System calculates the fee", not `calculateFee()`); guard every outcome; fork/join only for true parallelism; swimlanes if helpful; end node(s).

Example 1 — online shopping (course)

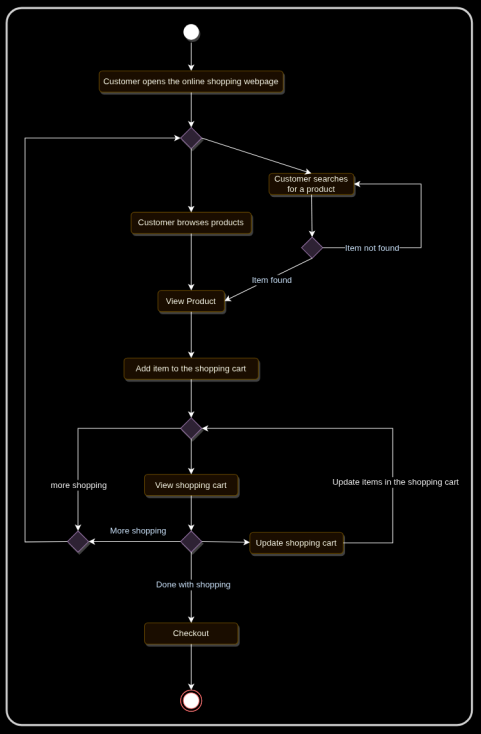
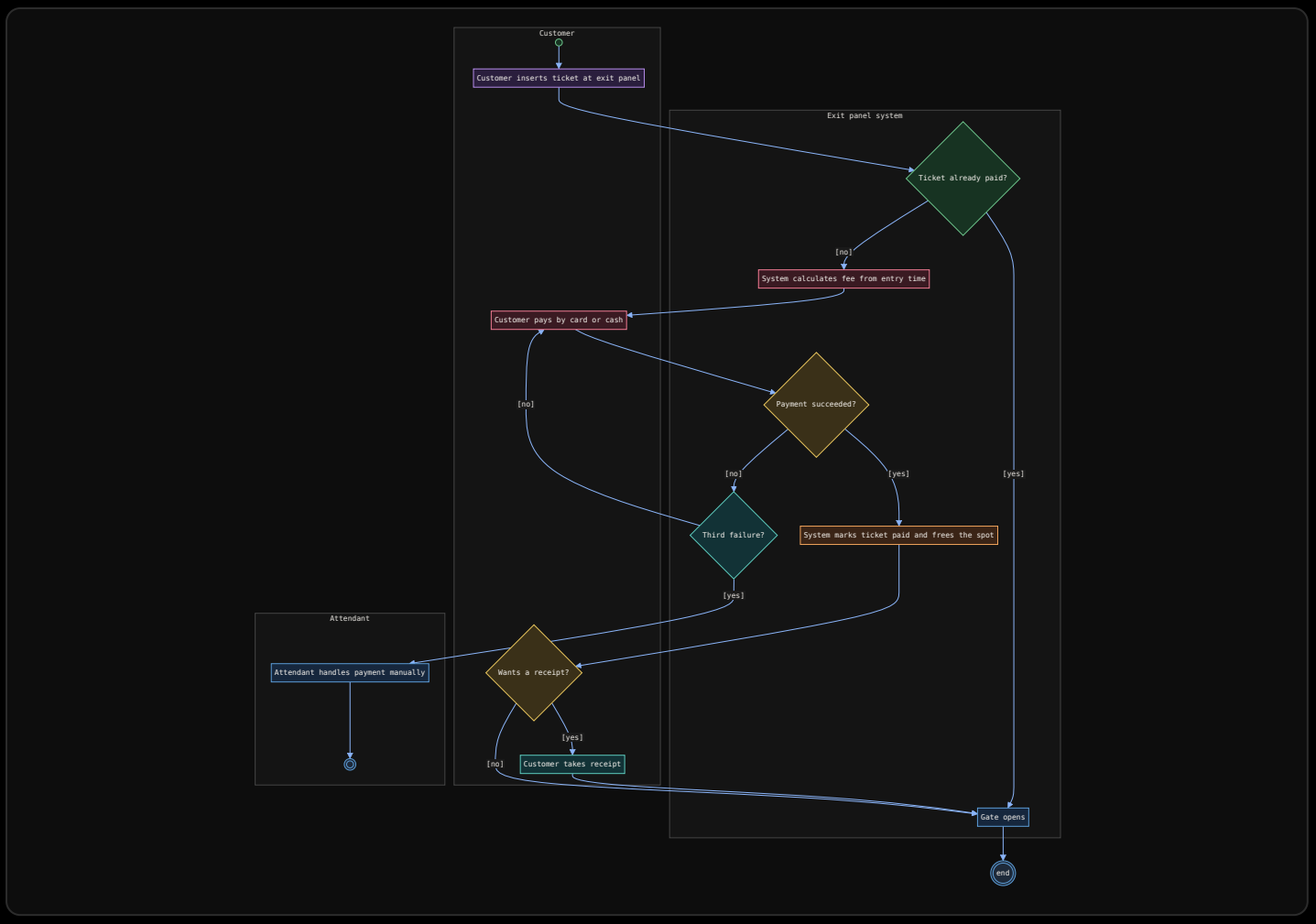


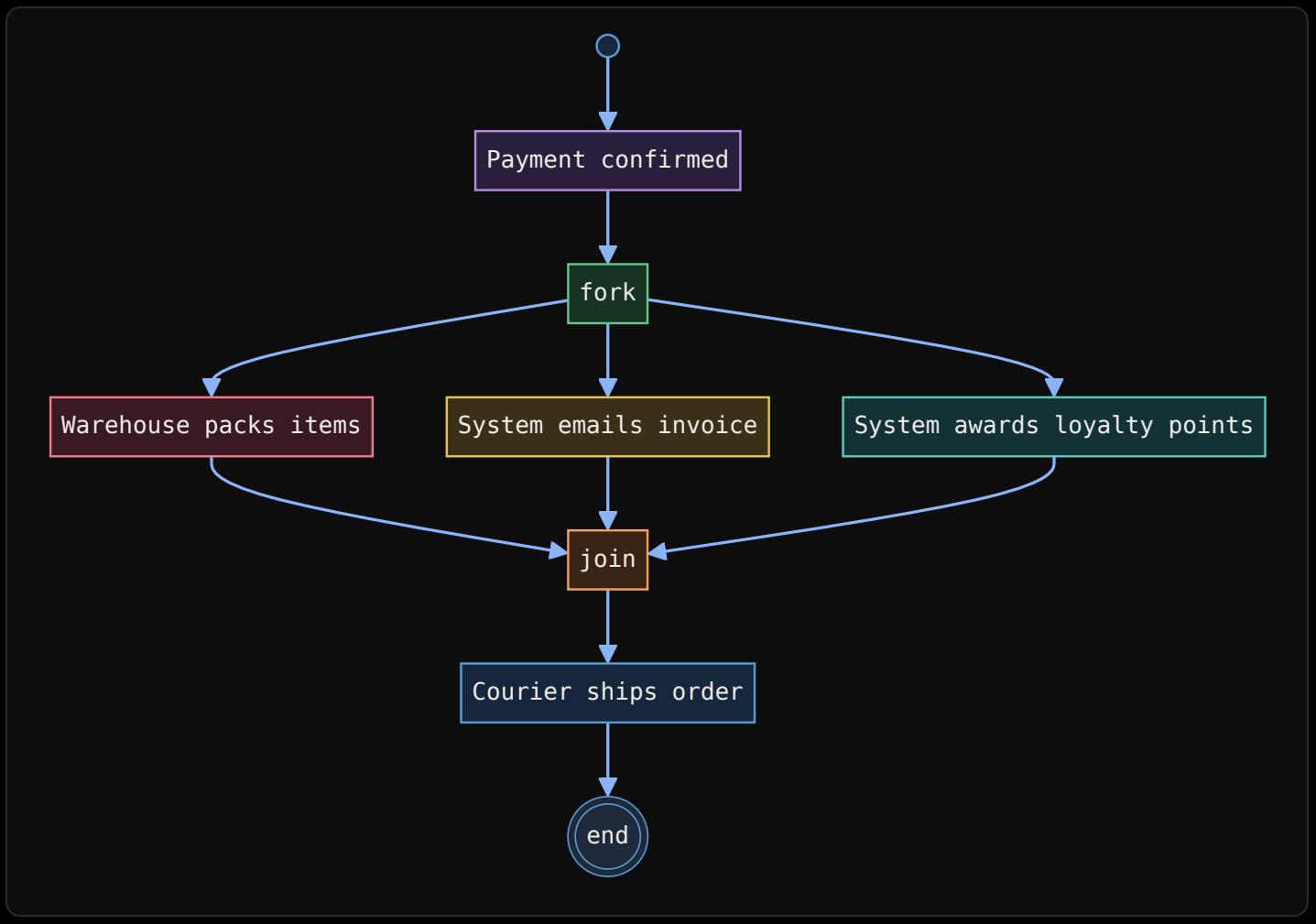
Figure — A customer shopping online: search, item found or not found, add to cart, view/update the cart, more shopping or checkout (course diagram)

Example 2 — pay at the parking exit (three decisions, with swimlanes)

Three requirement-driven decisions (ticket paid? payment succeeded? receipt?), a retry loop, and hand-off to an attendant after three failures (Parking Lot case study, Chapter 14):



Example 3 — fork and join (order fulfilment)



After the **fork** bar the three actions run in parallel; the **join** waits for all three before shipping.

Activity vs sequence diagram

Activity shows a process as steps, handles branches naturally and answers "what happens?"; sequence shows objects messaging, handles branches awkwardly (nested `alt`) and answers "who calls whom?". Rule of thumb: **three branches** → activity; **four objects** with tricky call order → sequence.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Readable by anyone, great at branches and parallelism; but no "who" without swimlanes, and big flows sprawl. Skip straight-line flows; use sequence for call order. **Mistakes:** A diamond with one exit; actions written as code; guards missing a case (`> 100` and `< 100`: what about 100?); fork/join for merely sequential steps. **Seen in:** BPMN (Camunda); CI/CD pipelines (parallel jobs = fork, `needs:` = join).

In the interview

> **30-second answer:** "One flow: start, actions, decisions with complete guards, merges, fork/join for parallel steps, swimlanes, ends. I draw the branching flow and check every action has a class."

- *Activity final vs flow final?* — Whole activity vs one path.

Self-check

- **Q:** What must every arrow leaving a decision have? **A:** A guard; together covering all cases.

TL;DR

Start → actions → guarded decisions → merge → end; fork/join, swimlanes; use it to test the class diagram.

3.14 State machine diagrams

What it is

The **states one object** can be in and the **transitions** that events trigger.

- **State** (rounded rectangle), **initial pseudostate** (filled circle), **final state** (bullseye).
- **Transition** label `event [guard] / action`: trigger, boolean condition, effect.
- Internal `entry / exit / do` activities; **self-transition**; **choice** pseudostate (diamond); **composite state** with sub-states and **history** `(H)`; **orthogonal regions** active together.

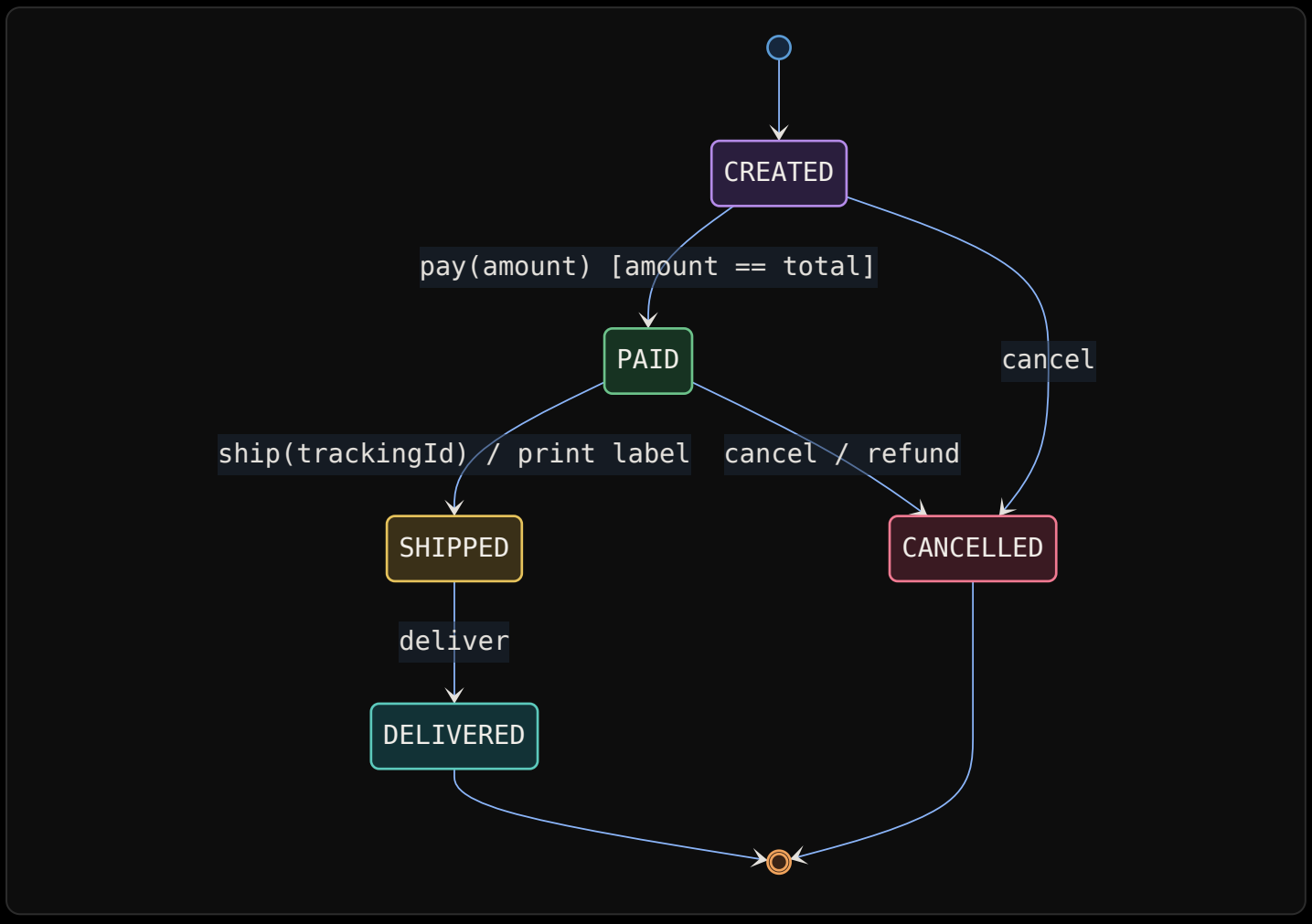
Analogy: a traffic light, always in one state, moving in a fixed order.

The problem it solves

Lifecycle bugs (shipping a cancelled order, paying twice). The diagram lists **every legal transition**, so everything else is rejected; it maps to an `enum` + transition method or the **State pattern** (Chapter 11).

How it works

Pick **one** class with a lifecycle → list states → per state: accepted events, guard, target, action → **everything else is illegal** → mark initial and final states.



For recognition, an elevator uses a **composite state** `Active` (sub-states `Moving` ⇄ `DoorsOpen`) entered from `Idle` on `request(floor) [floor != current]`, with `Active --> Maintenance : fault`.

Java example (`lld.ch03.statemachine`)

```
// File: code/lld/ch03/statemachine/Order.java
    public void handle(OrderEvent event) {
        OrderState from = state;
        OrderState to = switch (event) {
            case OrderEvent.Pay p when state == OrderState.CREATED && p.amount().compareTo(total) == 0 ->
OrderState.PAID;
            case OrderEvent.Pay p when state == OrderState.CREATED ->
                    reject(event, "guard failed: paid " + p.amount() + " but total is " + total);
            case OrderEvent.Ship s when state == OrderState.PAID -> {
                log.add("  action: print label " + s.trackingId());
                yield OrderState.SHIPPED;
            }
            case OrderEvent.Deliver d when state == OrderState.SHIPPED -> OrderState.DELIVERED;
            // ... two Cancel cases (from CREATED; from PAID with "refund" action)
            default -> reject(event, "no transition from " + state);
        };
        state = to;
        log.add(from + " --" + event.getClass().getSimpleName() + "--> " + to);
    }
```

Full runnable code: code/lld/ch03/statemachine/ (see the Full Edition).

`OrderEvent` is a **sealed** interface of records; each `case ... when` line is one diagram arrow, and `default` rejects the rest (e.g. `Ship rejected, no transition from CANCELLED`). The guard uses `compareTo` since `BigDecimal.equals` compares scale.

Trade-offs

enum + switch: all transitions in one place; best for few states with simple actions; a new state edits the switch. **State pattern (Ch. 11):** each state's behaviour in its own class; best when states behave very differently; a new state is a new class (Open/Closed).

When not to use it · Common mistakes · Real-world examples

Avoid when: A flag or two with no lifecycle; flows across many objects (activity diagram). A state machine is about **one** object. **Mistakes:** States named as actions ("Paying" vs PAID); missing or default-allowed transitions; overlapping guards; a state machine for the whole system. **Seen in:** `Thread.State`; TCP states; order tracking (cancel only before shipping); Spring Statemachine.

In the interview

> **30-second answer:** "For a lifecycle object I draw states with `event [guard] / action` transitions; anything not drawn is illegal and rejected. In Java: an enum with one transition method, or the State pattern when states carry lots of behaviour."

- *Guard?* — A boolean in `[ ]` that must hold, e.g. `[amount == total]`.
- *Thread-safe transitions?* — `synchronized` method, a lock, or CAS on an `AtomicReference<State>`.

Self-check

- **Q:** Read `PAID --> CANCELLED : cancel / refund`. **A:** In PAID, `cancel` moves to CANCELLED and refunds.

TL;DR

One object's states, `event [guard] / action` arrows; everything undrawn is illegal.

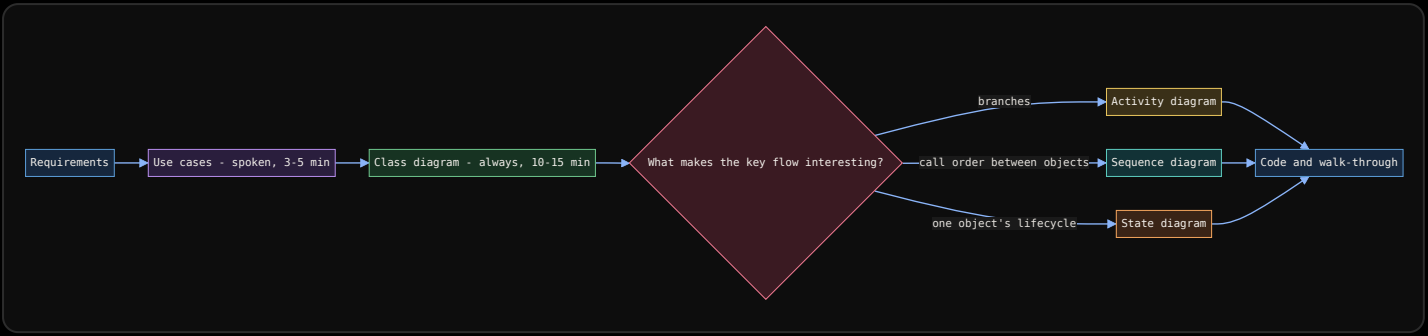
Part C — UML in the interview

3.15 How much UML to draw in an interview

Arrowheads aren't marked; what's checked is that **diagram and words agree**. A missing line costs less than an unjustifiable one.

Minute	Diagram	Depth
0–5	**Use case**	Usually spoken: actors, goals, out of scope
5–20	**Class diagram**	*Always*: 10–15 boxes, typed lines with multiplicity
20–25	**Activity**, **sequence** or **state**	One, for the key flow: branches / call order / lifecycle
25–45	Code + discussion	Walk the flow against the diagrams

Rules: one picture → the class diagram; drop attributes before relationships; label every line; say the test aloud; every activity action and sequence arrow must be a method; sequence diagrams 3–4 objects, 6–8 messages; machine coding → a 2-minute sketch; forgot a symbol → say what you mean.



In the interview

> **30-second answer:** "I list use cases verbally for scope, spend most diagram time on one class diagram with typed, multiplicity-labelled lines, then one behaviour diagram for the key flow to check every step has a class and method."

3.16 Chapter self-check

1. **Q:** Six relationships, weakest to strongest coupling? **A:** Dependency, association, aggregation, composition, realization, generalization.
2. **Q:** `CompactSpot` / `LargeSpot` subclasses passing a constant? **A:** One `ParkingSpot` with a `SpotType` enum.
3. **Q:** «include» vs «extend»? **A:** Always, base → included; sometimes, extension → base.
4. **Q:** Sequence arrowheads? **A:** Filled = sync, open = async, dashed = return.
5. **Q:** 14 UML diagrams? **A:** 7 structural (class, object, package, component, composite structure, deployment, profile), 7 behavioural (use case, activity, state machine, sequence, communication, interaction overview, timing).
6. **Q:** Only one diagram? **A:** The class diagram.

3.17 TL;DR

- **Relationships claim lifetime and substitution:** dependency (dashed arrow) · association (line/arrow) · aggregation (◇ at whole, parts live on) · composition (◆ at whole, parts die) · realization (dashed △) · generalization (solid △).
- **Choose in order:** substitution test → dies with whole? → outlives? → association; parameter-only → dependency.
- **Multiplicity changes code:** `1` required, `0..1` Optional, `1..*` non-empty, exact numbers checked.
- **UML:** 14 diagrams; interviews use use case, class (the answer), activity, sequence, state machine.

- **Draw little, label everything, say the test aloud,** and test the class diagram with behaviour diagrams.

Chapter 4 — SOLID Principles (Study Edition)

> **Study Edition.** Every topic of Chapter 4, without the padding. Code blocks are excerpts from the real files (`// ...` marks trimmed parts); full listings, outputs and longer discussion are in the Full Edition (`README.md`).
>
> **What you will learn.** The five SOLID principles, one at a time and then together, plus the ideas beneath them: **cohesion**, **coupling** and **separation of concerns**. Read the principles in order; 4.11 (one class through all five) is the best interview rehearsal.

Run the code (from `chapters/04-solid-principles/code/`):

```
javac --release 21 -d /tmp/lld_04 $(find . -name '*.java')
java -cp /tmp/lld_04 lld.ch04.srp.Main        # also: cohesion, coupling, soc, ocp, lsp, isp, dip, checkout
```

Packages: `srp` (Employee split by actor), `cohesion`, `coupling` (three steps), `soc` (rules vs storage), `ocp` (invoice formats), `lsp` (cars, contract rules, birds), `isp` (printers, over-splitting, roles), `dip` (notifications, DI, locator, plug-ins), `checkout` (one class through all five).

Money. All amounts are `BigDecimal` built from strings, and anything multiplied by a rate is rounded with `setScale(2, RoundingMode.HALF_EVEN)`. Never `double` (`0.1 + 0.2` prints `0.30000000000000004`). `BigDecimal.equals` compares scale (`2.0` ≠ `2.00`), so compare with `compareTo` or `signum()` .

4.1 What SOLID is and why you should learn it

What it is

Letter	Principle	The one-line rule
S	Single Responsibility (SRP)	A class should have one reason to change.
O	Open/Closed (OCP)	Add new behaviour by adding code, not by editing working code.
L	Liskov Substitution (LSP)	A subtype must be usable anywhere its parent type is expected.
I	Interface Segregation (ISP)	No client should be forced to depend on methods it does not use.
D	Dependency Inversion (DIP)	Depend on abstractions, not on concrete details.

Terms: a **module** is a unit bigger than a class (package, library, service); a **client** is code that uses other code; an **abstraction** is an interface or abstract class (*what*, not *how*); a **concrete class** is one you can `new` .

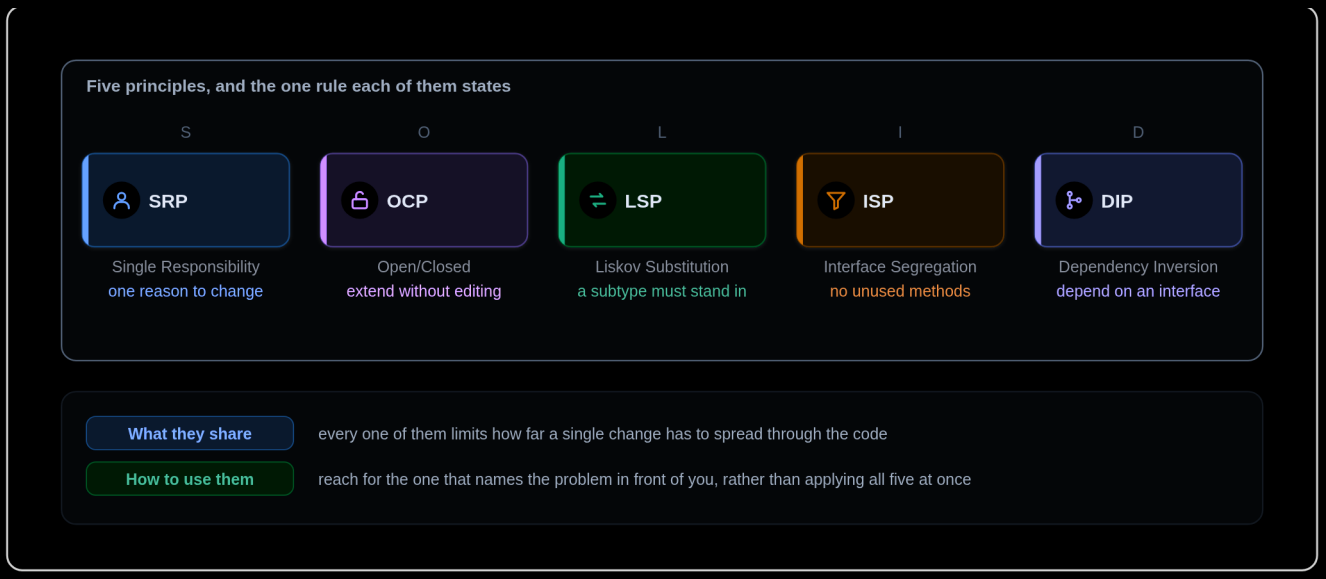


Figure — the five principles and the single rule each one states (course diagram)

The common thread: each principle **limits how far one change spreads**. Martin collected them around 2000; Feathers coined the acronym; OCP is Meyer's (1988), LSP is Liskov's (1987).

The problem it solves

Code keeps changing; without guidelines classes grow until one edit breaks three features. SOLID gives cleaner, more flexible, reusable code with fewer bugs — and **a shared vocabulary** ("this breaks ISP") so design debates are settled by argument, not taste.

How to use SOLID (the honest version)

Treat each principle as a **name for a problem**; when code hurts to change, find the name; apply the fix **when a real change asks for it**; always state the cost. Applied blindly, SOLID yields a maze of one-implementation interfaces.

TL;DR

Five guidelines that keep one change from spreading. Diagnose with them, apply on real change, state the cost.

4.2 Single Responsibility Principle (SRP)

What it is

A class should have only one reason to change — sharpened in *Clean Architecture* to **one, and only one, actor** (a person or group who can request a change). Analogy: a Swiss army knife has three jobs, so three reasons to change; a kitchen knife has one. Note: SRP is about classes/modules; "a function should do one thing" is a separate *Clean Code* rule.

The problem it solves

When one class serves several groups, every edit for one group ships inside code the others rely on: a payroll tweak can break an operations report.

What counts as one responsibility?

"One thing" has no yardstick; the **actor** test does: *who can send a change request for this code?*

Method	Who asks for the change	Example request
`calculatePay`	Finance	"Overtime starts after 38 hours, not 40."
`reportHours`	Operations	"Show the week-ending date."
`save`	Database team	"Move from SQL to a document store."

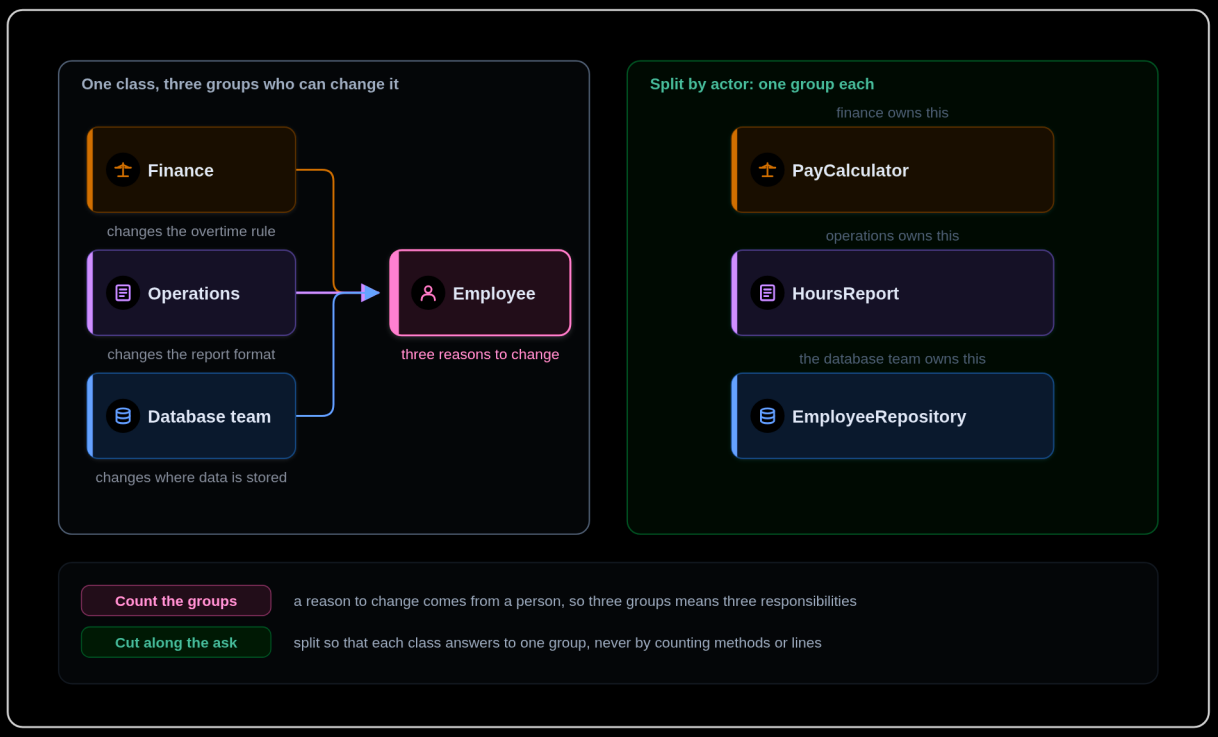


Figure — finance, operations and the database team all reach into one Employee class; splitting by actor gives each group its own class (course diagram)

Martin's trap: `calculatePay` and `reportHours` share a helper `regularHours()`; finance changes it and the operations report silently breaks. **Accidental duplication** — code that looks the same but changes for different reasons must not be merged.

How it works

List methods → name who would change each → group by actor → one class per group, named after its job → plain data in a `record` → run the three checks below.

Java example: splitting `Employee` by actor

Before, `Employee` holds `calculatePay()`, `reportHours()` and `save()`. After:

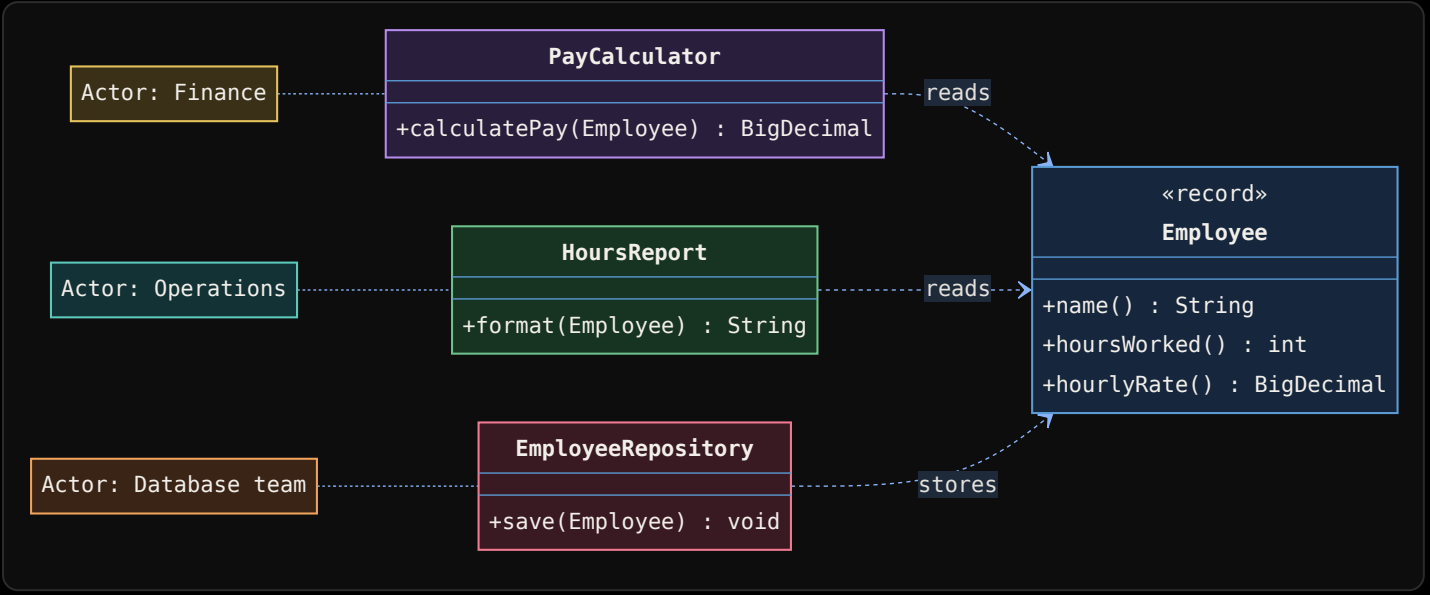
```java
// File: code/lld/ch04/srp/after/Employee.java
public record Employee(String name, int hoursWorked, BigDecimal hourlyRate) { }

// File: code/lld/ch04/srp/after/PayCalculator.java
/** Only Finance asks for changes to this class. */
public class PayCalculator {
    private static final int OVERTIME_THRESHOLD = 40;
    private static final BigDecimal OVERTIME_MULTIPLIER = new BigDecimal("1.5");

    public BigDecimal calculatePay(Employee e) {
        int overtime = Math.max(0, e.hoursWorked() - OVERTIME_THRESHOLD);
        int normal = e.hoursWorked() - overtime;
        BigDecimal normalPay = e.hourlyRate().multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = e.hourlyRate().multiply(OVERTIME_MULTIPLIER)
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN); // cents
    }
}
// HoursReport.format(Employee)        -- Operations
// EmployeeRepository.save(Employee)   -- Database team
```

Full runnable code: code/lld/ch04/srp/ (`before/`, `after/`) (see the Full Edition).

Output is identical before and after (`Pay: 950.00`); what changed is who must be consulted before an edit.



Where splitting goes wrong

Chopping by verb gives tiny classes always edited together — worse than no SRP. Three checks: (1) **Do the pieces change together?** (same commits = one responsibility); (2) **Does each name stand alone?** (`PayCalculator` yes, `EmployeeHelper` no); (3) **Could it be used elsewhere?** Also, **splitting doesn't automatically reduce dependencies** — deciding what each piece may know of the other is cohesion (4.3) and coupling (4.4).

Why SRP is important

Simpler maintenance, reuse, growth by adding classes, high cohesion — and a seam that *enables* low coupling (only an abstraction across it finishes the job).

Trade-offs

You gain	You pay
One group's change touches only its code	More classes and files
Small classes, easy tests, clear ownership	Wiring; risk of over-splitting

When not to use it

You can't name two actors; short-lived scripts; pieces that always change together.

Common mistakes

Splitting by method count or verbs; `Helper` / `Utils` classes; merging "duplicates" owned by different actors; thinking SRP means "one method"; confusing it with "a function does one thing".

Code smells that point to SRP (from *Refactoring*)

Divergent Change (SRP broken), **Large Class**, and **Shotgun Surgery** (often from *over*-splitting). Fix: **Extract Class**, **Move Function/Field**.

Real-world examples

Spring `@Controller` / `@Service` / `@Repository`; `DateTimeFormatter` vs `LocalDate`; Logback/Log4j 2 logger, appender and layout.

In the interview

> **30-second answer:** "One reason to change — really, one actor. I ask who would request changes to each method. If finance, operations and the DBAs can all edit `Employee`, I split it into `PayCalculator`, `HoursReport` and `EmployeeRepository` and keep `Employee` as data. It costs classes and wiring, so I only split when I can name the groups."

"Would you always split?" — No; if the halves always change together I've made two files for nothing.

Self-check

1. **Q:** What is an actor? **A:** A person or group who can request a change.
2. **Q:** The three checks for a good split? **A:** Change together? Name stands alone? Usable elsewhere?
3. **Q:** Does splitting always lower coupling? **A:** No — the dependency must point at an abstraction.

TL;DR

One class, one actor. Split along actors, keep data in a record, don't split what always changes together.

4.3 Cohesion

What it is

How closely the contents of one class belong together. **High**: every method serves one purpose. **Low**: unrelated jobs bundled.

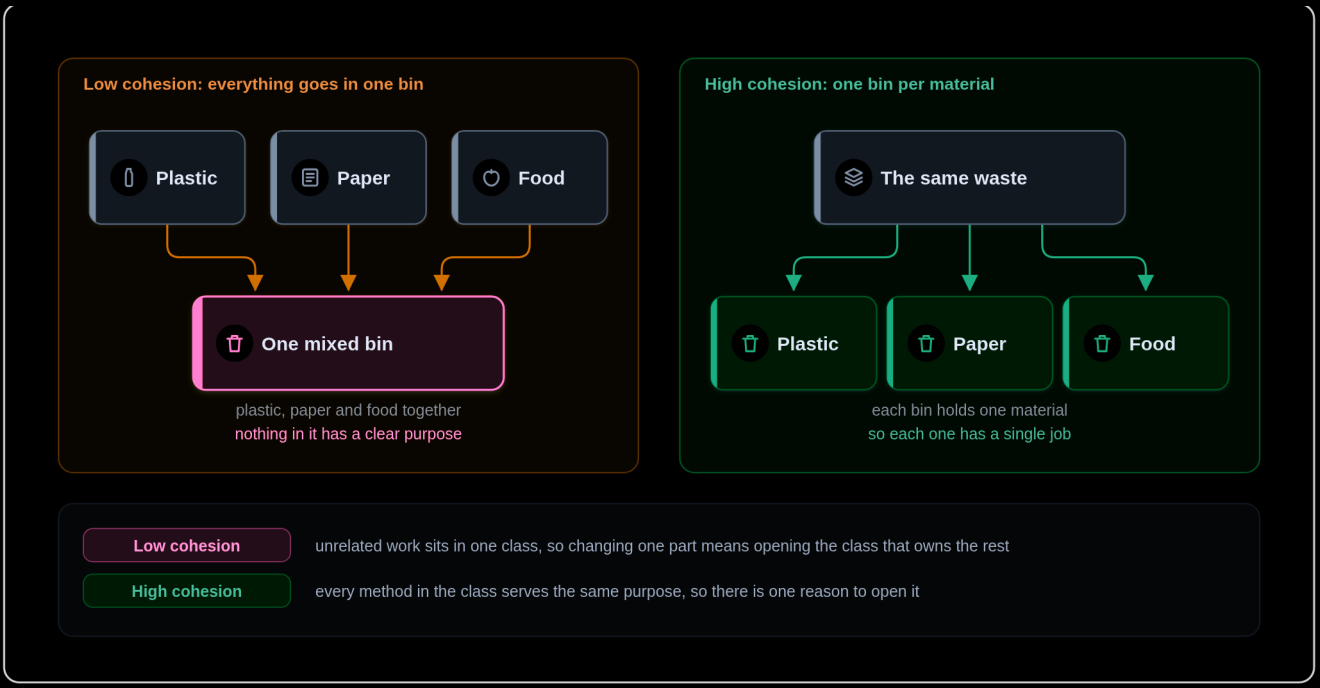


Figure — one mixed bin (low cohesion) next to one bin per material (high cohesion) (course diagram)

The problem it solves

Both directions hurt: unrelated jobs in one class (edit one, redeploy all), and one job **scattered** across classes (hunt for every piece).

The classic types of cohesion

Constantine, Yourdon, Myers (1970s), worst → best:

Type	What groups the code	Example
Coincidental	Nothing	`Utils` with `formatDate`, `sendEmail`, `parseCsv`
Logical	Category chosen by a flag	`handle(type)` with a big `switch`
Temporal	Run at the same time	`startup()` opens DB, warms caches, prints banner
Procedural	Steps in order	validate, log, retry
Communicational	Same data	`ReportBuilder` methods on one `Order` list
Sequential	Output feeds next step	`parse → validate → transform`
Functional (best)	One well-defined task	`PayCalculator`

Aim for functional; treat coincidental (the `Utils` class) as a warning.

How it works

Check which fields each method uses; disjoint groups for different purposes → split them; gather scattered pieces of one job.

Java example

`UserManager` mixed user bookkeeping (`users`) with audit-file work (`auditLines`). After the split:

```java
// File: code/lld/ch04/cohesion/after/AuditFile.java
/** HIGH cohesion: only audit-log work lives here. */
public class AuditFile {
    private final List<String> lines = new ArrayList<>();

    public void writeLine(String line) {
        // ...
    }

    public List<String> readAll() {
        // ...
    }
}
// after/UserManager keeps only users: addUser, deleteUser
```

Full runnable code: code/lld/ch04/cohesion/ (`before/`, `after/`) (see the Full Edition).

Same output; a new audit format now edits only `AuditFile`. Cost: the caller holds two objects.

Cohesion and SRP

High cohesion = one job = SRP. Aim for cohesion and SRP tends to follow.

Trade-offs

Gain: easy to name, read, test; changes land in one class. Cost: more classes to wire.

When not to use it

Don't split an already functionally cohesive class just to shrink it.

Common mistakes

Calling one mixed class a "coupling problem" (coupling needs two classes); `Utils` dumping grounds.

Real-world examples

`java.lang.Math` (functional); `java.util.Collections` (drifting toward logical).

In the interview

> **30-second answer:** "Cohesion is how much the things inside one class belong together. Two method groups touching different fields for different reasons mean low cohesion, so I split. It's SRP seen from inside the class." A big class can still be cohesive — size is a hint, not the measure.

Self-check

1. **Q:** Worst kind? **A:** Coincidental.
2. **Q:** One job spread over three classes — cohesion problem? **A:** Yes.

TL;DR

One purpose per class; split disjoint groups; gather scattered pieces.

4.4 Coupling

What it is

How much one class depends on another — always a **pair**. **Loose**: change, replace or test one without the other. **Tight**: changing one forces changing the other. Analogy: a remote for one TV vs a universal remote using an agreed signal — the signal is the **interface**.

The problem it solves

Tight coupling blocks swapping (MySQL → Postgres), blocks unit tests without the real dependency, and spreads changes in how the dependency is constructed.

The classic types of coupling

Worst → best:

Type	What A knows about B	Example
Content	B's internals	Reflection setting a private field
Common	Shared mutable globals	Both use a `public static Map`
External	A shared external format	Both parse one hand-rolled file format
Control	A flag telling B *how*	`print(doc, boolean asPdf)`
Stamp	Whole structure for one field	`sendInvoice(Customer c)` reads `c.email()`
Data	Only needed values	`sendInvoice(String email, Money total)`
Message (lowest)	A message through an interface	`notifier.send(message)`

Goal: **depend on interfaces, pass only what's needed, never share mutable globals.**

How it works

Find where A **names** B (field type, `new B()`, `B.x()`); if A only needs *something that does the job*, describe the job as an interface, accept it in A's constructor, let the entry point choose.

Java example: three steps with `Student`

Start: `Student` holds data and talks to MySQL (a cohesion problem). **Step 1:** extract `MySqlDatabase` — cohesion fixed, coupling **tight** (`private final MySqlDatabase database = new MySqlDatabase(); // hard-wired`). **Step 2:** depend on an interface:

```java
// File: code/lld/ch04/coupling/decoupled/Student.java
/** Step 2: Student names no database at all -> LOOSE coupling. */
public class Student {
    private final String studentId;
    private final StudentRepository repository;

    public Student(String studentId, StudentRepository repository) {
        this.studentId = studentId;
        this.repository = Objects.requireNonNull(repository);
    }

    public void save() {
        repository.save(studentId);
    }
}
// StudentRepository { void save(String studentId); }
// implemented by MySqlStudentRepository and InMemoryStudentRepository (test double)
```

Full runnable code: code/lld/ch04/coupling/ (`start/`, `extracted/`, `decoupled/`) (see the Full Edition).

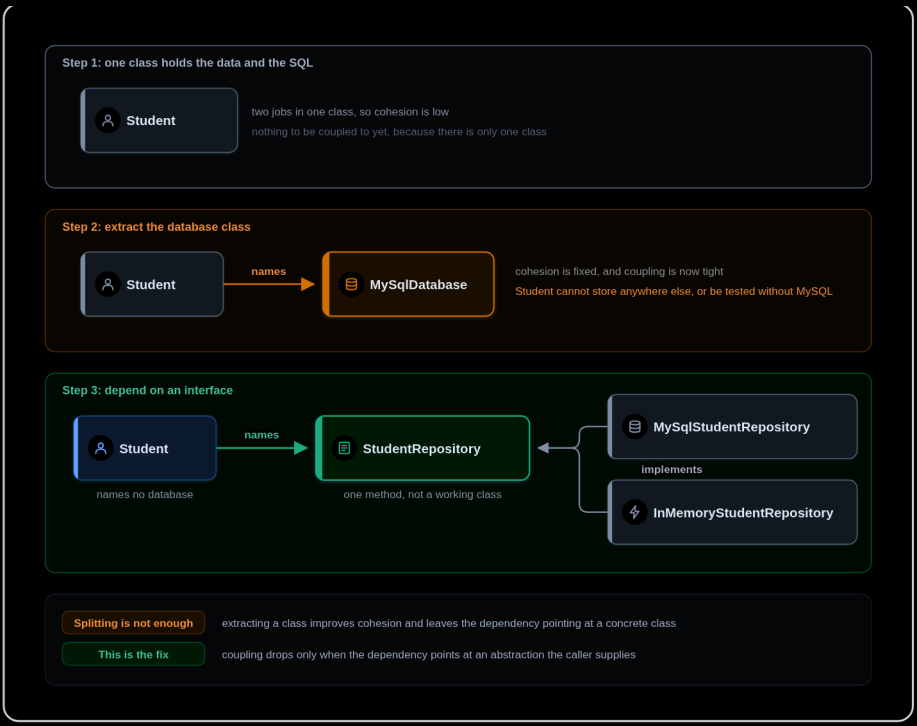


Figure — the three steps: one class with two jobs, an extracted but tightly coupled class, and a class that depends on an interface (course diagram)

Postgres is now a new class passed in; a test passes the in-memory one. This is DIP (4.10), the standard cure for tight coupling.

Coupling and SRP

SRP fixed cohesion but left `Student` welded to MySQL; pointing the dependency at an interface fixed coupling. SRP *usually* lowers coupling, never *guarantees* it.

Trade-offs

Loose coupling gains	Costs
Swap implementations; fakes in tests; no ripple	Extra interface and parameter; someone must wire it; one more hop to the real code

When not to use it

Not for **stable** classes (`String`, `LocalDate`, pure calculations); a one-implementation interface with no fake is just a file.

Common mistakes

"I extracted a class so coupling is lower"; hiding `new ConcreteThing()` in a constructor; stamp coupling.

Real-world examples

JDBC interfaces with vendor drivers; SLF4J; USB/HTTP/SQL as agreed signals.

In the interview

> **30-second answer:** "Coupling is how much one class depends on another. Where a class names a concrete collaborator but only needs the job done, I make the job an interface and pass the implementation in — swap MySQL for Postgres, test with a fake. Not for stable things like `String`."
> Can SRP increase coupling? Yes, if the extracted class is built around its parent's exact needs.

Self-check

1. **Q:** Can one class be tightly coupled? **A:** Not to itself; two jobs in one class is cohesion.

2. **Q:** Control coupling? **A:** A flag telling the other code how to behave.

TL;DR

Depend on an interface and pass the implementation in; extracting a class alone isn't enough.

4.5 Separation of Concerns (SoC)

What it is

Split a program so each part handles one **concern** (storage, validation, display). Dijkstra, 1974. Analogy: kitchen, waiters and cashier — a new menu doesn't change payment.

The problem it solves

Mixed concerns force edits in unrelated code. Separation gives modularity, maintainability, reusability, testability.

How it works

List concerns, one module each, small clear methods between them; **test: count the modules a likely change opens — ideally one.**

Java example: user registration

`UserStore` stores without judging; `UserService` decides and asks storage to save:

```java
// File: code/lld/ch04/soc/UserService.java
    public boolean registerUser(String username, String email) {
        if (!isValidUsername(username)) {
            System.out.println("Rejected: " + username + " is too short");
            return false;
        }
        if (!isValidEmail(email)) {
            System.out.println("Rejected: " + email + " is not an email address");
            return false;
        }
        store.addUser(username, email);
        return true;
    }
```

Full runnable code: code/lld/ch04/soc/ (see the Full Edition).



Figure — UserService decides what is acceptable and asks UserStore to save it; each change lands in one module (course diagram)

Change	Module you open
In-memory list → database	`UserStore` only
Usernames ≥ 8 characters	`UserService` only
Test rules without a DB	Give `UserService` a stand-in store

SoC and SRP

Same instinct at different scale: SRP for a class, SoC for modules and layers.

Trade-offs

Gain: local changes, isolated tests. Cost: more boundaries; cross-cutting concerns (logging, security, transactions) still span modules and need aspects or decorators.

When not to use it

A 50-line script. Separate concerns that change for different reasons or speeds.

Common mistakes

Validation hidden in the DAO; layers calling each other in every direction.

Real-world examples

MVC; HTML/CSS/JS; controller → service → repository; Spring `@Transactional` proxies.

In the interview

> **30-second answer:** "Each module handles one area of work. My test: how many modules does a requirement change open? Ideally one. It's SRP at module size."

Self-check

1. **Q:** SoC vs SRP? **A:** Mostly size — class vs modules/layers.

TL;DR

One module per area of work; measured by modules opened per change.

4.6 SRP vs cohesion vs coupling vs SoC — one table

Idea	Question it asks	Scope	Relation to SRP
SRP	How many actors can ask this class to change?	Class/module	—
Cohesion	Do the parts belong together?	Inside one unit	SRP usually raises it
Coupling	How much does A depend on B?	Between two units	SRP tends to lower it; needs abstractions
SoC	Is each area of work in its own module?	Modules, layers	SRP everywhere is one route to it

"High cohesion, low coupling": SRP gets the first half, DIP the second.

4.7 Open/Closed Principle (OCP)

What it is

Open for extension, closed for modification: add behaviour by adding code, not editing working code. **Meyer's OCP** (1988): a published class is closed and extended by **inheritance**. **Martin's polymorphic OCP** (1996): stable code depends on an **interface**; new behaviour arrives as new implementations — the version used today. Analogy: a console gains a racing wheel through a standard port without being opened.

The problem it solves

A growing `if/else` / `switch` on a type code: each case edits tested code, often in several copies. **Quick test:** *can I add the next case without opening this file?*

How it works

Wait for a real second case → identify the part that varies → make it an interface → one class per variant → the stable class **delegates** → new variants are new classes → choose the variant once, at the edge (entry point, factory, map, config).

Java example: invoice formats

Before: `generate(String invoiceType)` branches on `"basic"` / `"international"` and throws for unknown types — each new type edits working code. After:

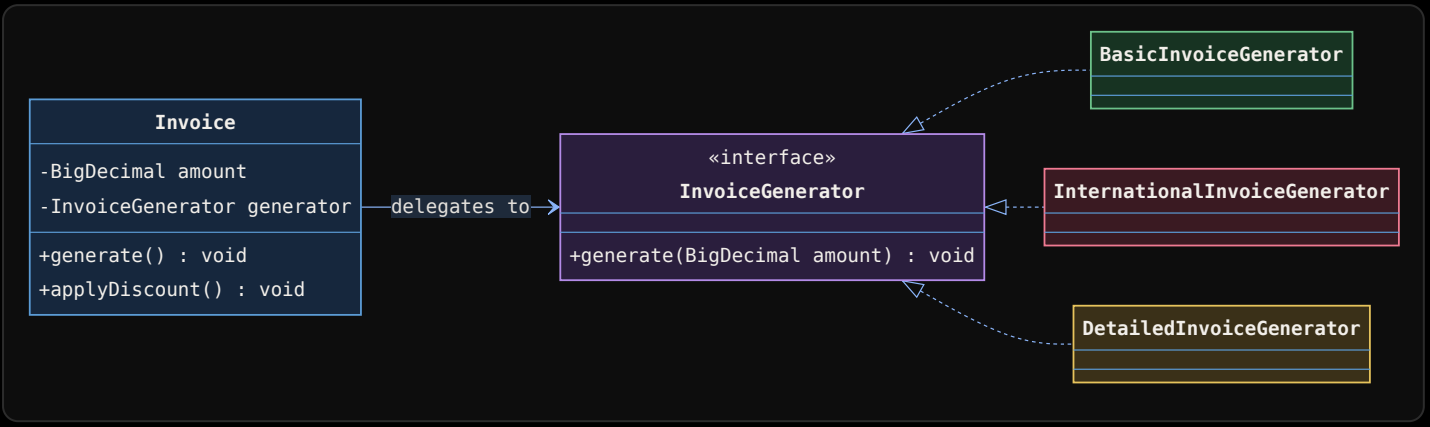
```java
// File: code/lld/ch04/ocp/after/InvoiceGenerator.java
@FunctionalInterface
public interface InvoiceGenerator {
    void generate(BigDecimal amount);
}

// File: code/lld/ch04/ocp/after/Invoice.java
public class Invoice {
    private final BigDecimal amount;
    private final InvoiceGenerator generator;
    // ...
    public void generate() {
        generator.generate(amount); // no branching
    }

    // Still closed? No -- a new discount rule would still edit this class.
    // That is fine until discounts actually start to vary.
    public void applyDiscount() {
        // ...
    }
}
// BasicInvoiceGenerator, InternationalInvoiceGenerator, and the ONLY new code
// for format three: DetailedInvoiceGenerator
```

Full runnable code: code/lld/ch04/ocp/ (`before/`, `after/`) (see the Full Edition).

The classes are named *generators* (not `BasicInvoice`, which would imply a subtype). As a `@FunctionalInterface`, a **lambda** is also an extension — OCP with zero new files.



This shape is the **Strategy pattern** — OCP for a varying algorithm.

Why not just subclass `Invoice`?

Subclassing is also OCP (Meyer's style) and fine when small. Delegation wins with **a second dimension** (3 formats × 4 currencies = 12 subclasses vs 3 + 4 = 7 classes) and **run-time choice** (pick or replace the generator once the country is known).

What is still not closed

`applyDiscount` still edits `Invoice` — fine. **A class is closed against a specific kind of change**, one you've seen happen. Martin: **strategic closure**.

Approaches to OCP

Approach	Pattern / Java feature	Good when
Interface + delegation	Strategy, `Comparator`	Algorithm varies, chosen at run time
Inheritance + overriding	Template Method	Fixed skeleton, a few steps vary
Wrapping	Decorator (`BufferedInputStream`)	Add features around behaviour
Listening	Observer	New reactions to events
Plug-ins / registry	map, `ServiceLoader`	Third parties add features
Functions	lambdas, `java.util.function`	Small one-method variations
Data / configuration	rules engines, feature flags	Variants are just numbers

The `switch` often survives — moved to **one** place at the edge (a factory or `Map<String, InvoiceGenerator>`).

Importance of OCP

Extend without touching the tested core; test only the new class; fewer regressions; adaptability.

Caution: do not over-engineer

Write the `if` first; add the interface at the second or third case (**rule of three**). Unused extension points are **Speculative Generality**.

How OCP relates to SRP

Changing parts end up in single-job classes; SRP usually comes first because the extension point stays hidden until the class has one job.

Trade-offs

You gain	You pay
New variants without editing; each tested alone; run-time choice	Interface + class per variant; flow spread over files; someone must choose the variant

When not to use it

Nothing has varied; variants are just numbers; the set is small and fixed — there Java 21 **sealed interfaces + pattern-matching** `switch` fit (compiler checks every case; easy to add operations, hard to add types).

Common mistakes

"Never edit a file" (bug fixes and refactoring still edit); plug-in systems for one implementation; moving one `switch` into five factories.

Refactorings and smells

Smell **Repeated Switches**; fixes **Replace Conditional with Polymorphism**, **Replace Type Code with Subclasses**.

Real-world examples

`list.sort(comparator)`; Java I/O decorators; servlet filters / `HandlerInterceptor`; JDBC drivers via `ServiceLoader`; IDE and browser plug-ins.

In the interview

> **30-second answer:** "I should add the next variation by adding code. When a `switch` over a type keeps growing, I put the varying behaviour behind an interface, one class per variant, and the stable

class delegates — Strategy. Only once the variation is real, and closure is against a specific kind of change." Late requirement ("trucks pay a different rate"): "add a class" beats "add a branch".

Self-check

1. **Q:** Is `Invoice` closed against new discounts? **A:** No, only formats — fine until discounts vary.
2. **Q:** Which pattern is OCP for an algorithm? **A:** Strategy.
3. **Q:** Quick OCP test? **A:** "To add the next case, must I open this file?"

TL;DR

Varying part behind an interface, variants as classes, chosen at the edge — once variation is real.

4.8 Liskov Substitution Principle (LSP)

What it is

A subtype must be usable wherever its parent is expected without making the program incorrect (Liskov 1987; with Wing 1994, *behavioural subtyping*). "Substitutable" means **correct results**, not "it compiles". Analogy: promised "a car that takes petrol", handed an electric car.

The problem it solves

If a subclass secretly behaves differently, callers must check `instanceof` — polymorphism and OCP are lost.

Example 1: `Car` and `ElectricCar`

```java
// File: code/lld/ch04/lsp/car/before/ElectricCar.java
/** VIOLATION: inherits a promise (refuel) it cannot keep. */
public class ElectricCar extends Car {
    @Override
    public void refuel() {
        throw new UnsupportedOperationException("Electric cars don't need refueling");
    }
    // ... plus its own recharge()
}
```

LSP is tested by code that **doesn't know** the subclass: `service(Car car)` calls `start`, `refuel`, `stop` and crashes.

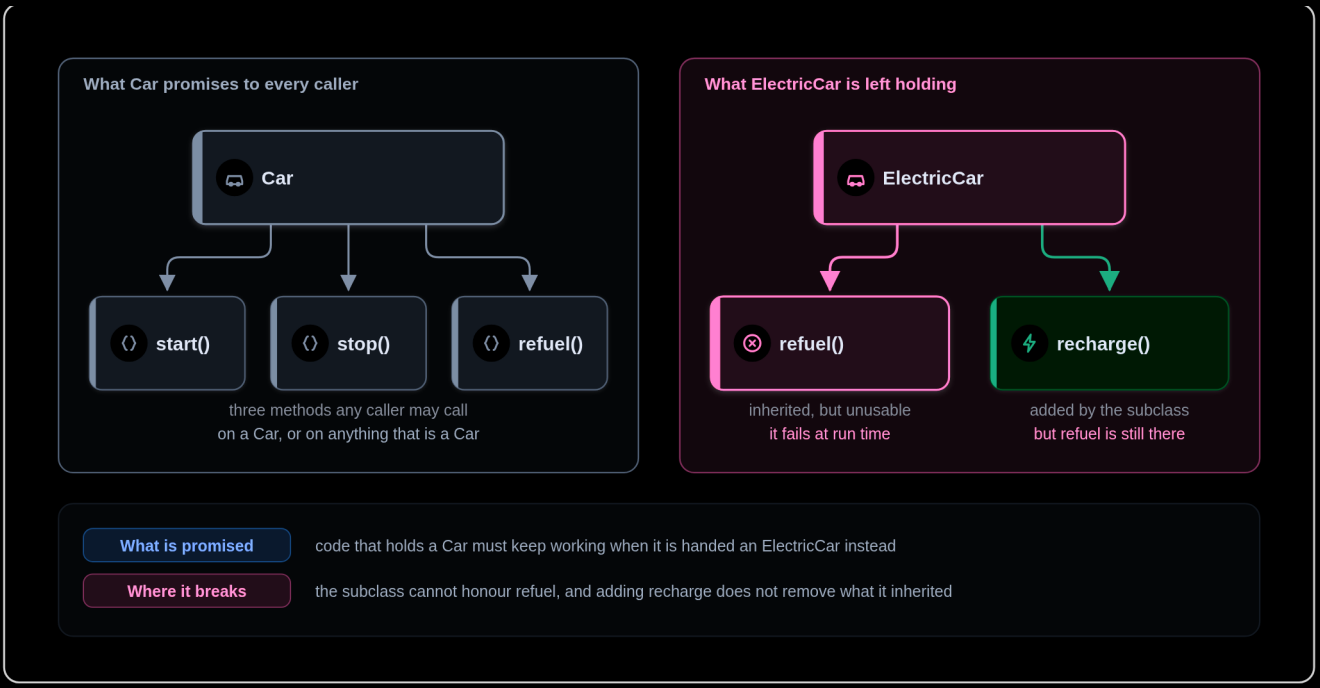


Figure — what Car promises every caller, and the unusable refuel() that ElectricCar is left holding (course diagram)

Root cause: `refuel()` names a **mechanism**, not the shared **goal**. **Raise a general parent** — `abstract class Vehicle { start(); stop(); abstract replenishEnergy(); }` — with `Car` and `ElectricCar` as **siblings**. Secretly charging inside `refuel()` would keep a lying name; adding only `recharge()` is invisible to `Vehicle` callers; and substitution is **child → parent, never between siblings**.

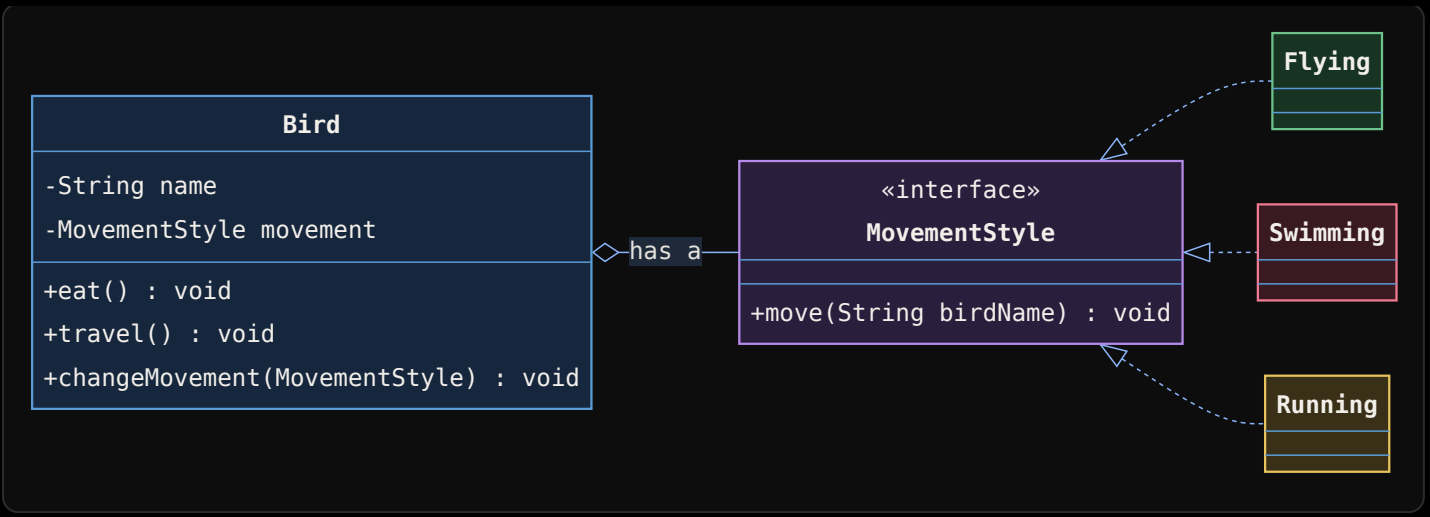
Break a hierarchy when a subclass can't implement a parent method, a base method names a mechanism, callers test the type, or subclasses share almost nothing. Fix: raise a parent, or move the varying behaviour out with composition.

Example 2: birds, and composition instead of inheritance

`fly()` on `Bird` forces `Penguin` to throw (**Refused Bequest**). Inheritance is **is-a** (gets every method); composition is **has-a**. Fix: no bird subclasses — each `Bird` gets a `MovementStyle`:

```java
// File: code/lld/ch04/lsp/birds/composition/Bird.java
/** A bird HAS-A movement style. No subclasses, so nothing is inherited and refused. */
public final class Bird {
    private final String name;
    private MovementStyle movement;
    // ...
    /** Tell, don't ask: callers say "travel", the bird decides how. */
    public void travel() {
        movement.move(name);
    }

    /** Composition can change behaviour at run time (e.g. an injured bird). */
    public void changeMovement(MovementStyle movement) {
        this.movement = Objects.requireNonNull(movement);
    }
}
// MovementStyle { void move(String birdName); } -> Flying, Swimming, Running
```



	Inheritance	Composition
Link	Penguin **is a** Bird	Bird **has a** MovementStyle
Change behaviour	add a subclass	supply another object
At run time?	no	yes
LSP risk	high	low

Inherit when the child truly keeps every promise; compose when behaviour varies or a subclass rejects something (throws, does nothing, "never call this on X").

Tell, Don't Ask

(Hunt & Thomas) **Tell an object what to do; don't inspect it and decide for it.** `if (bird instanceof Penguin) { swim(bird); } else { fly(bird); }` must know every subtype; `bird.travel()` works with any substitute. `instanceof` on a base type hints the base promises the wrong thing.

The rules a subclass must follow

The dangerous violations are **silent**. `Square extends Rectangle` couples the setters: set width 5, height 4, expect 20, get **16**. `Rectangle`'s unwritten promise was *width and height change independently*. **Base inheritance on the promises in code, not on real-world classification.**

A subclass may not **require more**, **give less**, or **break an assumption** (Meyer's Design by Contract: preconditions, postconditions, invariants):

1. **Preconditions may not be strengthened** — `SmallOrderDiscount` rejects amounts ≥ 100 that `Discount` accepts.
2. **Postconditions may not be weakened** — `OverdraftAccount.withdraw` lets the balance go negative where `Account` guaranteed ≥ 0.
3. **Invariants must be preserved** — `Square` (also describable as a weakened `setWidth` postcondition). Hardest: invariants are rarely written down.
4. **No new exceptions** — `ElectricCar.refuel()`. Java checks only **checked** exceptions. Loudest, least harmful.
5. **History constraint** (Liskov & Wing) — no state changes the parent forbids: `AdjustablePrice extends Price` adds `adjust()`, so a quote of 100 is charged 120. Why `String`/`Integer` are `final`.

```java
// File: code/lld/ch04/lsp/rules/Square.java
/** RULE 3 BROKEN: to stay square it couples the two setters -- a silent wrong answer. */
public class Square extends Rectangle {
    // ...
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }
    // ...
}
```

The fix: no shared mutable promise — immutable shapes that promise only `area()`:

```java
// File: code/lld/ch04/lsp/shapes/Shape.java
public sealed interface Shape permits Rect, Sq {
    int area();
}

// File: code/lld/ch04/lsp/shapes/Rect.java
public record Rect(int width, int height) implements Shape {
    public Rect withWidth(int w) {
        return new Rect(w, height);
    }
    // ...
}
```

Full runnable code: code/lld/ch04/lsp/ (`car/before/`, `car/after/`, `birds/inheritance/`, `birds/composition/`, `rules/`, `shapes/`) (see the Full Edition).

```
OverdraftAccount: balance after payment: -30.00   <-- the caller never expected this
Square:    expected area 20, got 16
quoted 100.00, charged 120.00
```

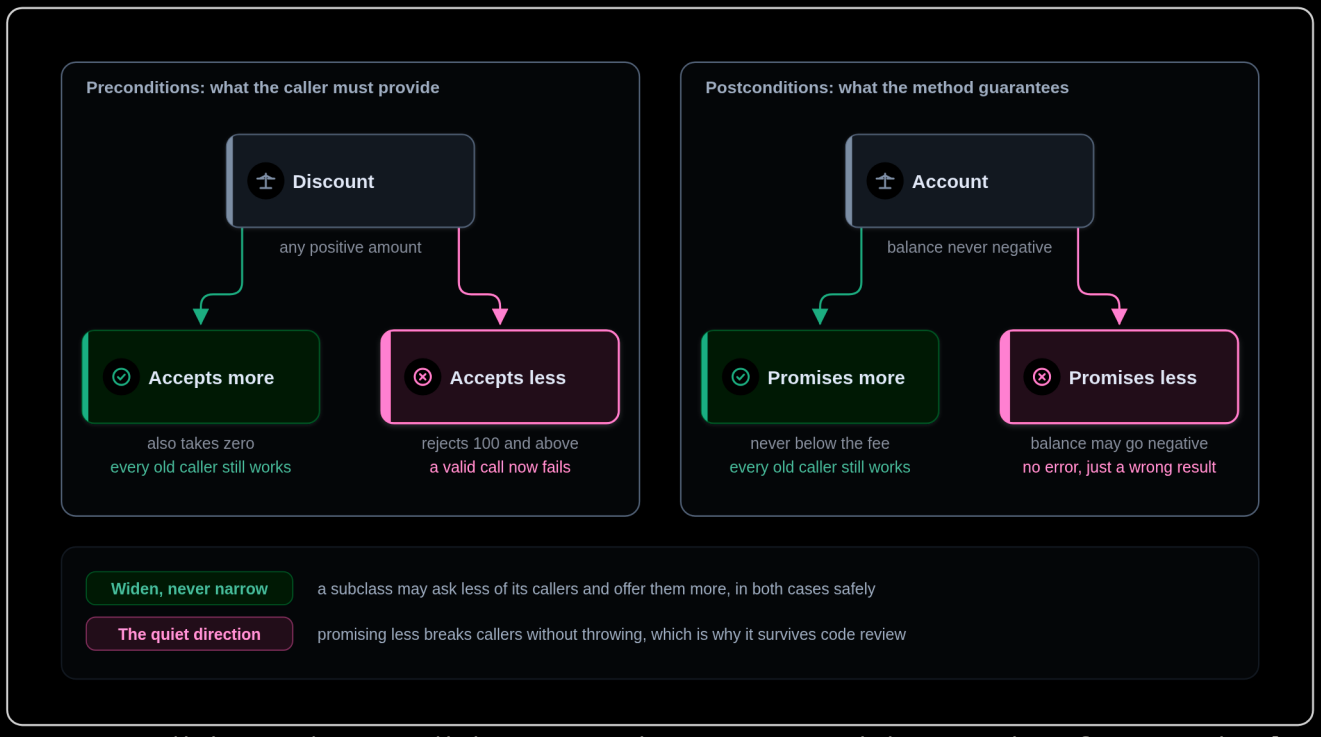


Figure — preconditions and postconditions: accepting more or promising more is safe; accepting less or promising less breaks callers (course diagram)

Rule	The subclass may not…	Usually shows up as
Preconditions	demand more	valid input rejected
Postconditions	guarantee less	a returned value breaks an assumption
Invariants	break a rule that always held	a silently wrong result
Exceptions	throw something new	crash with no matching `catch`
History	allow changes the parent forbids	a "fixed" value changes between reads

Signature rules (Java checks these): covariant returns allowed; no new/broader checked exceptions; no reduced visibility; parameter types must match exactly (else it's an **overload** — use `@Override`). Java checks **none** of the behavioural rules.

How to check a subclass

1. List the parent's **promises** (not signatures) first. 2. **Write one test against the parent type and run it for every subclass** (`resizeAndCheck(Rectangle)` fails for `Square`) — an abstract contract test in JUnit.

Why LSP is important

No fragile surprises; polymorphism without `instanceof` (what makes OCP work); correct results; safe reuse.

Trade-offs

Gain: callers trust the base type; safe new subclasses; run-time flexibility via composition. Cost: contracts to think through and document; "natural" hierarchies abandoned; more small objects.

When not to use it

LSP is never optional with polymorphism; what you can skip is inheritance — if a subclass can't keep the promises, don't make it one.

Common mistakes

Modelling the real world; "fixing" by silently doing nothing; thinking LSP is only about exceptions; expecting siblings to substitute.

Refactorings and smells

Refused Bequest → Replace Subclass/Superclass with Delegate, Extract Superclass, Push Down Method.

Real-world examples

`java.sql.Date` (`getHours()` throws); `java.sql.Timestamp` (asymmetric `equals`, "implementation inheritance"); `Stack extends Vector` (use `Deque`); `List.of(...).add` throws (an "optional operation").

In the interview

> **30-second answer:** "A subclass must work anywhere its parent is expected with correct results. It can't demand more, promise less, break invariants, throw new exceptions or allow forbidden state changes. Mutable Square/Rectangle is the classic silent break. I test with one base-type test for every subclass; when a subclass must refuse a method, I raise a better parent or compose."

"Is `ElectricCar` a substitute for `Car` after the fix?" — No, nor should it be; both substitute for `Vehicle`.

Self-check

1. **Q:** Subclass accepts fewer inputs — which rule? **A:** Preconditions strengthened.
2. **Q:** Why is Square worse than `refuel()` throwing? **A:** Silent wrong answer vs loud failure.

3. **Q:** History constraint? **A:** No state changes the parent forbids.

TL;DR

Accept at least as much, deliver at least as much, keep invariants and history, throw nothing new. Inherit by promises; otherwise raise a parent or compose.

4.9 Interface Segregation Principle (ISP)

What it is

Clients should not be forced to depend on methods they do not use. Origin: Martin at Xerox, where one `Job` class used by every task meant any change rebuilt all tasks. A **fat interface** grew feature by feature; a **role interface** describes one kind of caller's needs. Analogy: the 40-page menu for someone who wants coffee.

The problem it solves

Implementers must throw or no-op (also an LSP break); callers depend on methods they never call; test fakes grow long.

Example: a bloated `Printer` interface

```java
// File: code/lld/ch04/isp/fat/BasicPrinter.java
/** VIOLATION: forced to "implement" three methods it can only refuse. */
public class BasicPrinter implements Printer {
    // ...
    @Override
    public void scanDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot scan");
    }
    // ... faxDocument and stapleDocument throw too
}
```

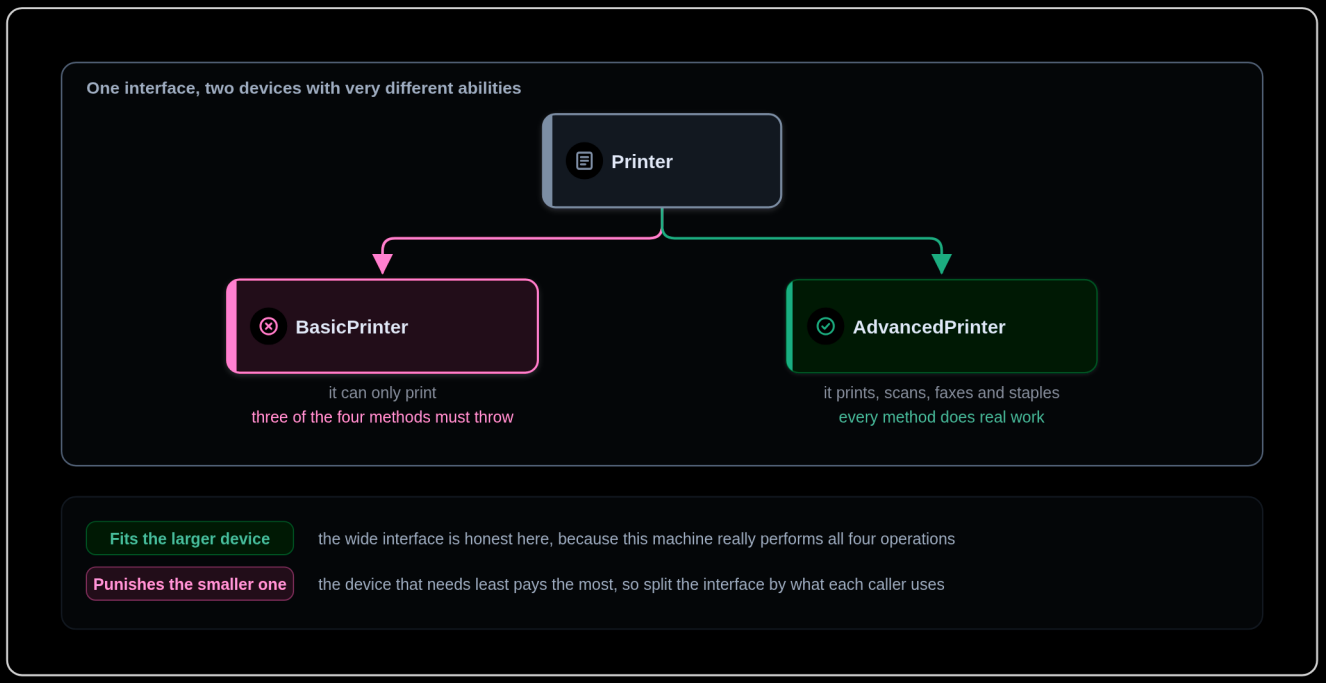


Figure — one interface, two devices: it fits the larger device and punishes the smaller one (course diagram)

Restructuring: first attempt, one interface per capability

`Printer`, `Scanner`, `Fax`, `Stapler`; `BasicPrinter` implements one, `AdvancedPrinter` all four. Valid ISP — but **one interface per method is not the goal**.

How small should an interface be?

Group by caller, not by method. Right size when both are "no": *is any implementer forced to supply what it can't honour?* and *does any caller depend on methods it never calls?*

Over-splitting cost: `OfficeWorkflow` (scan, print, staple) takes three parameters and gets **the same device three times** — `new OfficeWorkflow(adv, adv, adv)`. Grouping by role instead:

```java
// File: code/lld/ch04/isp/roles/DocumentProcessor.java
/** Role interface for callers that scan, print AND staple (e.g. OfficeWorkflow). */
public interface DocumentProcessor extends Printer {
    void scanDocument(String doc);

    void stapleDocument(String doc);
}
// OfficeWorkflow(DocumentProcessor) -- one dependency
// PrintQueue(Printer)               -- a BasicPrinter is enough
```

Full runnable code: code/lld/ch04/isp/ (`fat/`, `split/`, `roles/`) (see the Full Edition).

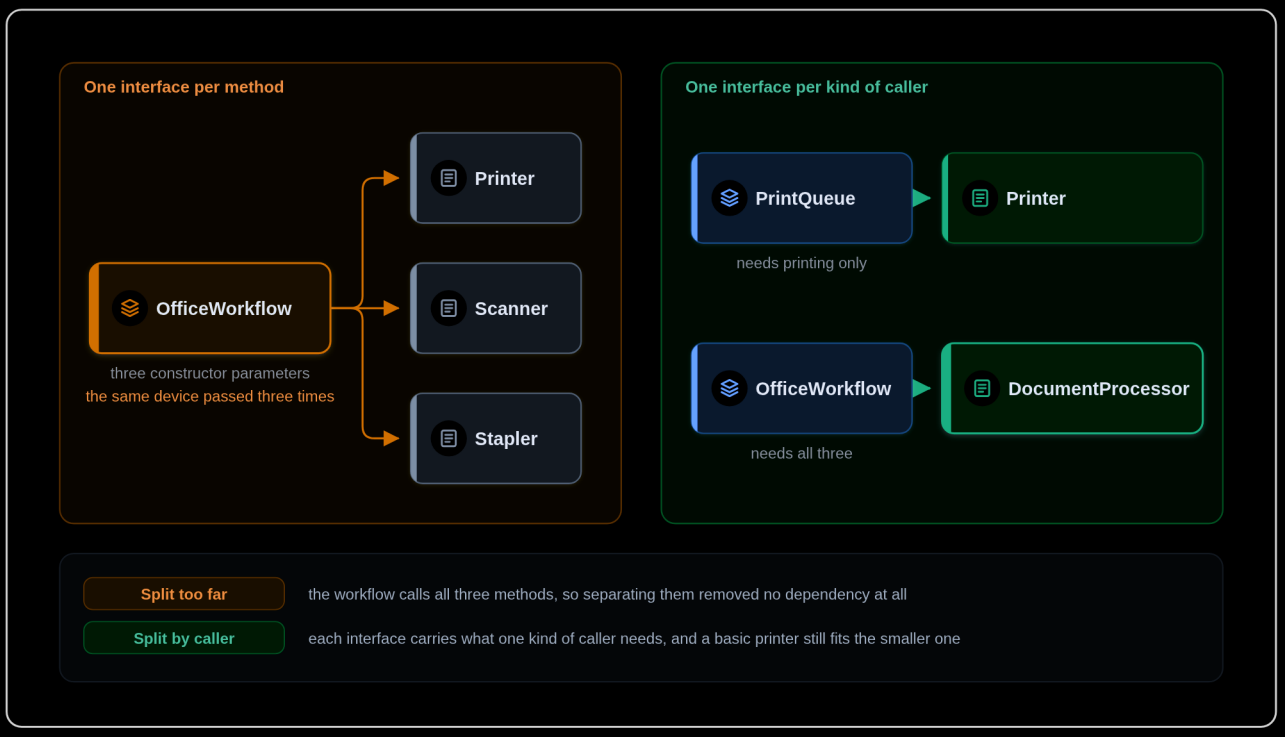


Figure — split too far (three interfaces, the same device passed three times) vs split by caller (one role per kind of caller) (course diagram)

	Four one-method interfaces	Two role interfaces
Forced/unused methods?	No	No
Dependencies for `OfficeWorkflow`	3	1
Names to learn	4	2

Rule of thumb: split only when you can **point to the caller** needing the smaller piece; merge interfaces always implemented and used together. In Python/JavaScript the interface is **the set of methods a caller actually calls** — the same needless dependency exists, just unchecked (`typing.Protocol` / `abc` can document it).

Five signals of an ISP violation

Signal	When it shows up	Example
Low cohesion in the interface	Reading it	`MediaDevice { playAudio; playVideo; recordVideo }` for a speaker
Fat interface, mechanism names	Reading it	`processCreditCard/PayPal/BankTransfer` → `PaymentMethod.pay(amount)`
Empty / throwing methods	Reading an implementer	`Vehicle { drive; fly; sail }` for a car
One interface, many roles	Comparing implementers	`Employee { calculateSalary; handleCustomerQueries; manageTeam }`
Long test stand-ins	Writing a test	dozens of empty bodies for one call

How it works (step by step)

List each caller's used methods → identical lists form a **role** → one interface per role (may extend each other) → classes implement every role they can honestly fill → each caller takes the narrowest role.

Importance of ISP

No bloated implementers, no ripple across roles, simpler tests, clearer design.

How ISP relates to the other principles

SRP for interfaces; OCP via new small interfaces; fat interfaces **force** LSP violations; DIP should depend on the smallest role.

Trade-offs

Gain: no throwing methods, insulated callers, tiny fakes (lambdas for one-method roles). Cost: more names; over-splitting adds wiring; harder to see a device's full ability in one place.

When not to use it

Every implementer supports everything and every caller uses it all; or you can't name the caller that wants less.

Common mistakes

One interface per method everywhere (most common after learning ISP); splitting by implementer features; fat interface + empty adapter class (AWT `MouseAdapter`).

Real-world examples

`Runnable`, `Comparable`, `AutoCloseable`, `Iterable`, `java.util.function`; `MouseListener`/`MouseAdapter`; Kotlin `List` vs `MutableList`; Spring Data `Repository`/`CrudRepository`/`PagingAndSortingRepository`.

In the interview

> **30-second answer:** "No caller should depend on methods it doesn't use. Symptoms: throwing implementations, fakes full of empty methods. I split by caller — one role per kind of caller, not per method — and only when I can name that caller." *SRP vs ISP?* SRP protects the class, ISP protects the caller.

Self-check

1. **Q:** The two sizing questions? **A:** Forced-to-supply? Depends-on-unused?
2. **Q:** Why didn't four interfaces help `OfficeWorkflow`? **A:** It uses all three methods; no dependency was removed.

TL;DR

One interface per kind of caller. Throwing methods = too wide; same object passed thrice = too thin.

4.10 Dependency Inversion Principle (DIP)

What it is

1. **High-level modules should not depend on low-level modules; both depend on abstractions.**
2. **Abstractions should not depend on details; details depend on abstractions.**

High level = policy (*what*); low level = detail (*how*: SMTP, MySQL). **Inversion:** the call still goes policy → detail, but the **source-code dependency is reversed** — the detail implements an interface owned by the policy. Analogy: a lamp and a wall socket.

The problem it solves

Policy tied to a vendor; untestable without infrastructure; library changes ripple up.

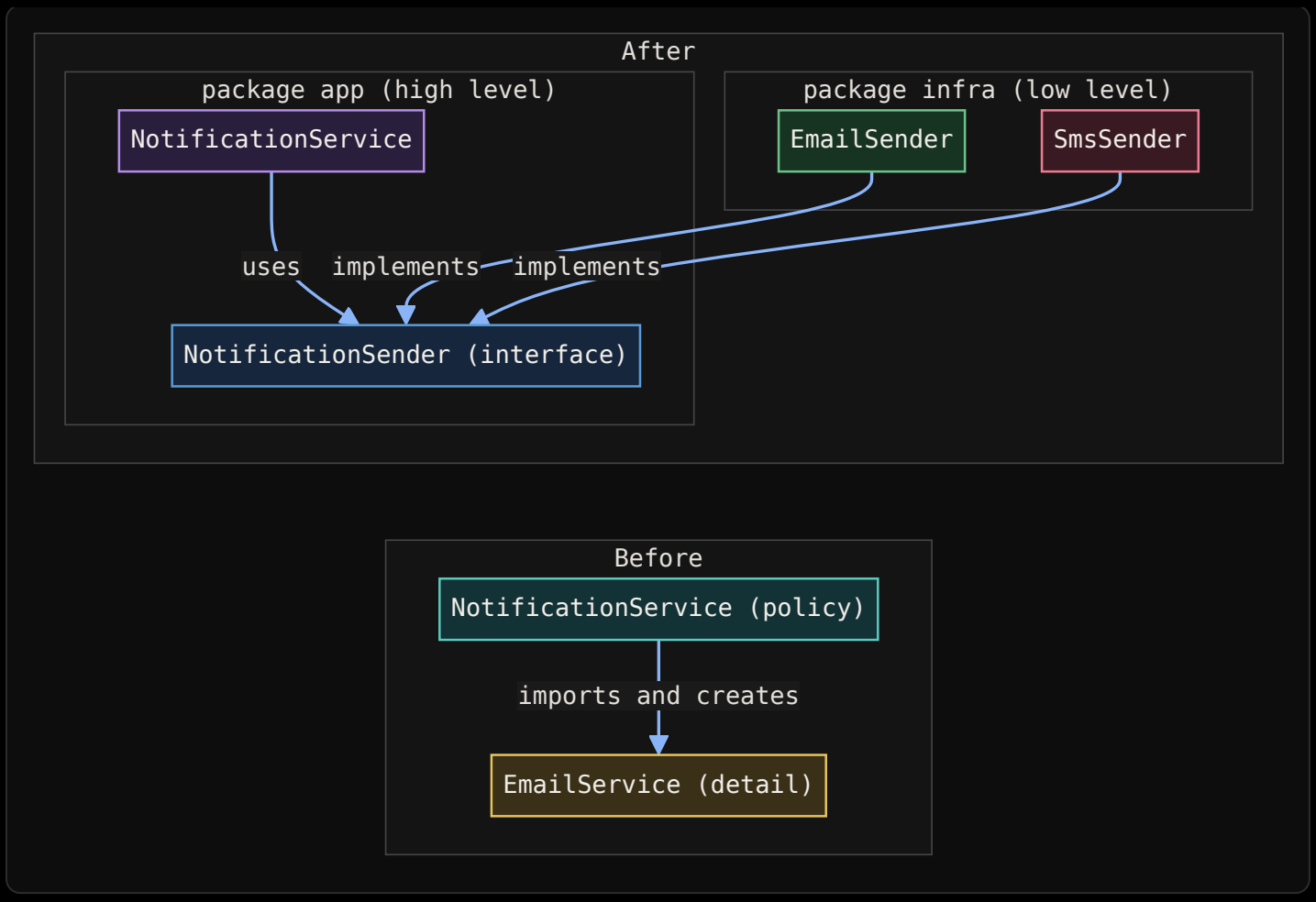
Example: notifications

Before: `NotificationService` does `new EmailService()`. After: an interface **in the policy's package** (`app`, not `infra`) — **the high-level side owns the abstraction**:

```java
// File: code/lld/ch04/dip/app/NotificationService.java
/** High-level policy: depends only on an abstraction it owns. Imports nothing from infra. */
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) { // constructor injection
        this.sender = Objects.requireNonNull(sender);
    }

    public void notifyUser(String userId, String message) {
        if (message.isBlank()) {
            throw new IllegalArgumentException("empty notification");
        }
        sender.send(userId, message);
    }
}
// app.NotificationSender { void send(String recipient, String message); }
// infra: EmailSender, SmsSender, RecordingSender (test double) implement it
```



DIP in practice: four real-world techniques

1. Dependency Injection — **constructor** (default; required, immutable), **setter** (optional with default), **method** (varies per call). The one place that does all the `new`-ing is the **composition root** (each `Main`).

2. Service Locator — a registry hands out services; `CheckoutFlow` depends on a `Locator` interface: `locator.paymentService(serviceName).processPayment(amount)`. It satisfies DIP, but hides real dependencies and fails late on a missing registration (Seemann: an anti-pattern). Prefer constructor injection.

3. IoC containers — Spring injects a single constructor automatically (`@Autowired` optional since 4.3). Without Spring: `new OrderService(new StripePayments())`. **The framework is a wiring convenience, not the principle.**

4. Plug-in architecture — the core owns the contract; plug-ins implement it:

```java
// File: code/lld/ch04/dip/plugin/MediaPlayer.java
/** The core. New formats arrive as plug-ins; this class is never edited for them. */
public class MediaPlayer {
    private final Map<String, AudioDecoder> decoders = new HashMap<>();

    public void install(AudioDecoder decoder) {
        decoders.put(decoder.extension(), decoder);
    }
    // ... play(fileName) looks up the decoder by extension
}
```

Full runnable code: code/lld/ch04/dip/ (`before/`, `app/`, `infra/`, `locator/`, `plugin/`) (see the Full Edition).

DI is small-scale (how one class gets collaborators); plug-ins are system-level (whole modules discovered at run time: `ServiceLoader`, OSGi).

DIP vs DI vs IoC — do not mix them up

Term	Kind	Example
DIP — depend on abstractions the policy owns	Principle	`NotificationService` → `NotificationSender`
DI — pass dependencies in	Technique	`new NotificationService(new SmsSender())`
IoC — the framework calls you (Hollywood Principle)	Broad idea; DI is one form	Spring beans, servlet `doGet`, JUnit `@Test`
IoC container	Tool	Spring `ApplicationContext`, Guice

Injection without inversion: a `MySqlDatabase`-typed parameter. Inversion without a framework: this whole chapter.

Stable vs volatile dependencies

Invert **volatile** dependencies (DBs, networks, clocks, third-party APIs, varying rules); use **stable** ones (`String`, `List`, `LocalDate`) directly.

Importance of DIP

Swappable implementations, detail changes stay in the detail, fakes instead of real infrastructure.

How DIP relates to the other principles

SRP (wiring moves to the composition root); OCP (new implementations without edits); LSP (implementations must be proper substitutes); ISP (own the smallest role).

How it works (step by step)

Find policy naming a volatile detail → write its **need** in its own words → make it an interface **in the high-level package** → detail implements it → inject (constructor) → move concrete `new`s to the composition root.

Trade-offs

Gain: technology-independent policy, easy tests, swappable parts. Cost: an interface per volatile dependency, wiring code, an extra hop.

When not to use it

Stable classes; a single implementation forever with no fake; tiny scripts.

Common mistakes

Interface in the low-level package; interface shaped by a vendor API (`sendSmtpMessage`); breaking clause 2 (returning `ResultSet`, throwing `MessagingException` — use domain types); "we use Spring so we follow DIP"; `new ConcreteThing()` "just this once".

Real-world examples

JDBC + `ServiceLoader`; SLF4J; Spring / Jakarta CDI; JPA `EntityManager`; **hexagonal architecture** (ports and adapters — DIP for a whole service).

In the interview

> **30-second answer:** "Business logic depends on an abstraction it owns, not on technical details. `NotificationService` depends on a `NotificationSender` in its own package; email and SMS implement it; I inject via the constructor and wire in `main`, so it's testable with a fake. DI is the technique, DIP the principle, Spring a wiring tool." *Who owns the interface?* The high-level module.

Self-check

1. **Q:** What is inverted? **A:** The source-code dependency.
2. **Q:** Composition root? **A:** The one place concrete classes are created and wired.
3. **Q:** `String` behind an interface? **A:** No — it's stable.

TL;DR

Policy owns the interfaces; details implement them; `main` wires them.

4.11 Refactoring one class through all five principles

The principles usually appear **in sequence**, each exposing the next. One checkout class that validates, prices, saves and emails is refactored five times.

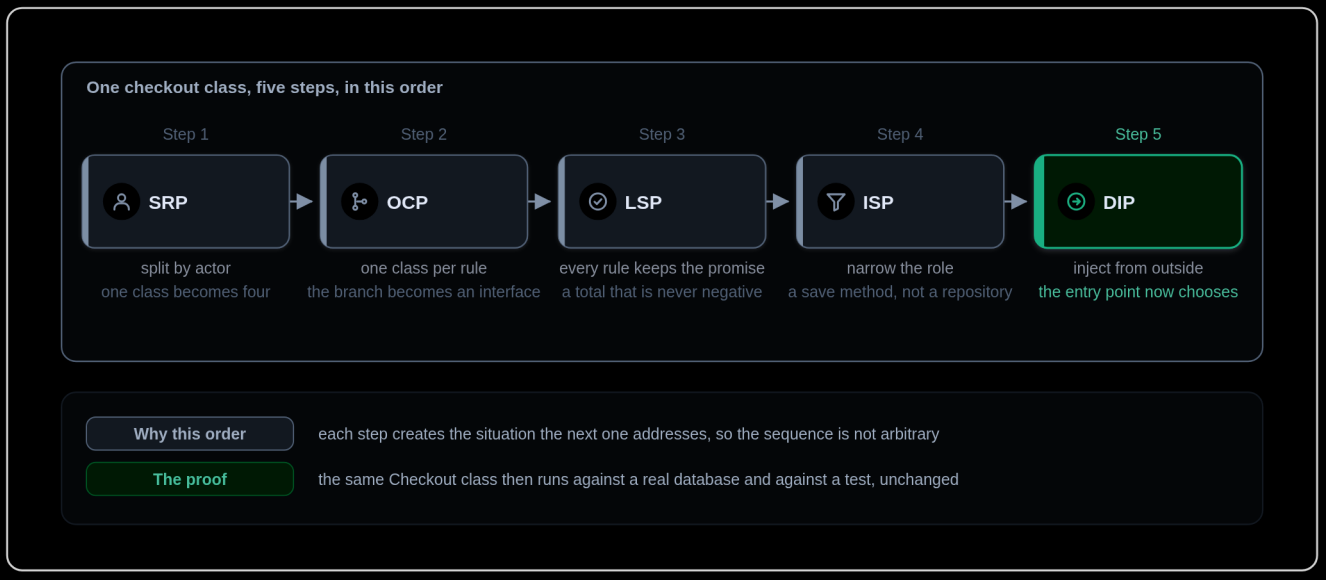


Figure — one checkout class, five steps: SRP, OCP, LSP, ISP, DIP, each creating the situation the next addresses (course diagram)

Step 0: the starting class

`OrderService.process(CustomerOrder)` does it all: rejects non-positive amounts (business-rules owner), branches on `customerType` for `"member"` (0.90) and `"staff"` (0.70) (finance), prints an `INSERT` (database team), and emails the total (marketing). It works — and every principle objects. (Code: `code/lld/ch04/checkout/step0/`.)

Step 1: Single Responsibility — split by actor

Four actors → `OrderValidator`, `PriceCalculator`, `OrderRepository`, `Notifier`, coordinated by a `Checkout` that still `new`s them itself. The customer-type `switch` survives inside `PriceCalculator` (`code/lld/ch04/checkout/step1/`).

Step 2: Open/Closed — one class per pricing rule

```
// File: code/lld/ch04/checkout/pricing/PricingRule.java
/**
 * STEP 2 (OCP): one class per rate; a new rate is a new class.
 * CONTRACT every implementation must keep (STEP 3, LSP):
 *   precondition : amount >= 0
 *   postcondition: result >= 0, rounded to 2 decimals   (a total is never negative)
 */
@FunctionalInterface
public interface PricingRule {
    BigDecimal apply(BigDecimal amount);
}
// RegularPrice, MemberDiscount, StaffDiscount; Order no longer carries a type string
```

`step2.PriceCalculator.total(order, rule)` is now just `rule.apply(order.amount())` — no branch (`code/lld/ch04/checkout/step2/`).

Step 3: Liskov Substitution — every rule keeps the promise

A careless fixed discount of 20 turns a 10 order into **−10**, silently saved. Every rule must return a total **never below zero**; one contract test (`PricingRuleContract.holds`, in `code/lld/ch04/checkout/step3/`) runs against every implementation:

```
// File: code/lld/ch04/checkout/pricing/FixedDiscount.java
    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.subtract(off).max(BigDecimal.ZERO).setScale(2, RoundingMode.HALF_EVEN);
    }
```

Step 4: Interface Segregation — the checkout needs a save method, not a repository

The repository grows (`save`, `findById`, `listAll`) but the checkout only saves, so it depends on a one-method role, `OrderSaver`; reporting uses `OrderFinder`; `SqlOrderRepository` implements both.

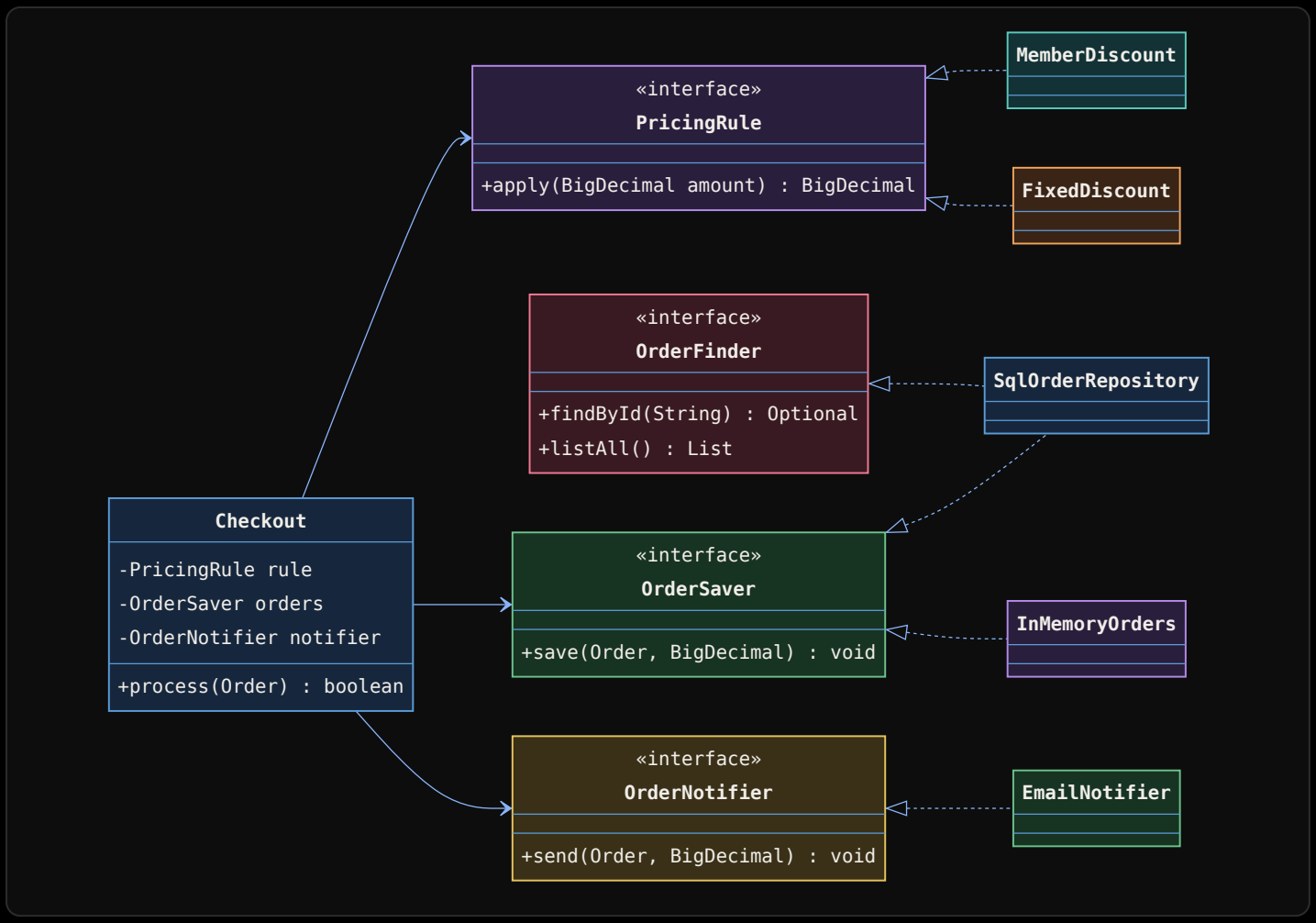
Step 5: Dependency Inversion — the entry point decides

```
// File: code/lld/ch04/checkout/fin/Checkout.java
public class Checkout {
    private final PricingRule rule;
    private final OrderSaver orders;
    private final OrderNotifier notifier;
    // ... constructor takes all three (requireNonNull)
    public boolean process(Order order) {
        // Validation has not varied yet, so it stays a plain check (see "What we did not do").
        if (order.amount().signum() <= 0) {
            System.out.println("  rejected: amount must be positive");
            return false;
        }
        BigDecimal total = rule.apply(order.amount());
        orders.save(order, total);
        notifier.send(order, total);
        return true;
    }
}
```

Full runnable code: code/lld/ch04/checkout/ (`step0/`–`step3/`, `pricing/`, `fin/`) (see the Full Edition).

```
  CarelessFixedDiscount broke the contract: apply(0.00) = -20.00
  4/5 rules are safe substitutes
Production wiring:
  INSERT INTO orders VALUES ('A-9', 180.00)
Test wiring, same Checkout class:
  kept A-9 in memory
```

One unmodified `Checkout` ran with production collaborators, test doubles (`InMemoryOrders`, `SilentNotifier`) and lambdas — **never edited or subclassed**.



Reading the five steps back

Step	The question it asked	What changed
SRP	Who might request changes?	One class → four, one per actor
OCP	What breaks when a new case appears?	Each pricing branch → a class
LSP	Does each rule work through the interface?	All rules guarantee total ≥ 0
ISP	Is the caller tied to more than it calls?	Checkout needs one save method
DIP	Who picks the concrete class?	The entry point

The order has a reason: the varying part stays hidden until each class has one job; LSP matters at the second implementation; ISP when new callers appear; DIP needs abstractions to point at.

What we did not do

`Order` stays a plain record; validation still edits `Checkout` (until a third rule is asked for); stable `BigDecimal` math isn't injected. **Every step was triggered by a real change request, not a checklist.**

Self-check

1. **Q:** Why SRP before OCP? **A:** The varying branch was buried in a four-job method.
2. **Q:** Step 3's contract? **A:** Any amount ≥ 0 gives a total ≥ 0.
3. **Q:** Proof it worked? **A:** One `Checkout`, three wirings, zero edits.

TL;DR

SRP → OCP → LSP → ISP → DIP, each triggered by a real change.

4.12 SOLID and design patterns

Principles say what good structure looks like; patterns are named recipes for it.

Pattern	Principle(s)	How
Strategy	OCP, DIP	Varying algorithm behind an interface
Template Method	OCP	Fixed skeleton, hook steps
Decorator	OCP, SRP	Behaviour added by wrapping
Adapter	DIP, ISP	Caller keeps its own interface
Factory Method / Abstract Factory	DIP, OCP	Concrete constructors in one place
Observer	OCP, DIP	New reactions = new subscribers
Composite	LSP, OCP	Leaves and groups share one interface
Proxy	SRP, OCP	Access control/caching in a separate class
Command	SRP, OCP	Each request an object
Visitor	OCP (operations)	Easy to add operations, hard to add types
Null Object	LSP	Do-nothing implementation that keeps the contract

Head First states the same ideas (encapsulate what varies, composition over inheritance, program to an interface, …); *Clean Code*'s "isolating from change" is DIP.

4.13 SOLID in the design interview

Keeping the design honest: five checks over your class diagram

Run five quick checks (~5 minutes) over the diagram you drew: **SRP** (one-sentence job; an "and" means two), **OCP** (a `switch` over a type?), **LSP** (would any inherited method throw or refuse? recheck late-added arrows), **ISP** (empty methods in big interfaces?), **DIP** (does `ParkingLot` construct a `CreditCardTransaction`? depend on `Payment`).

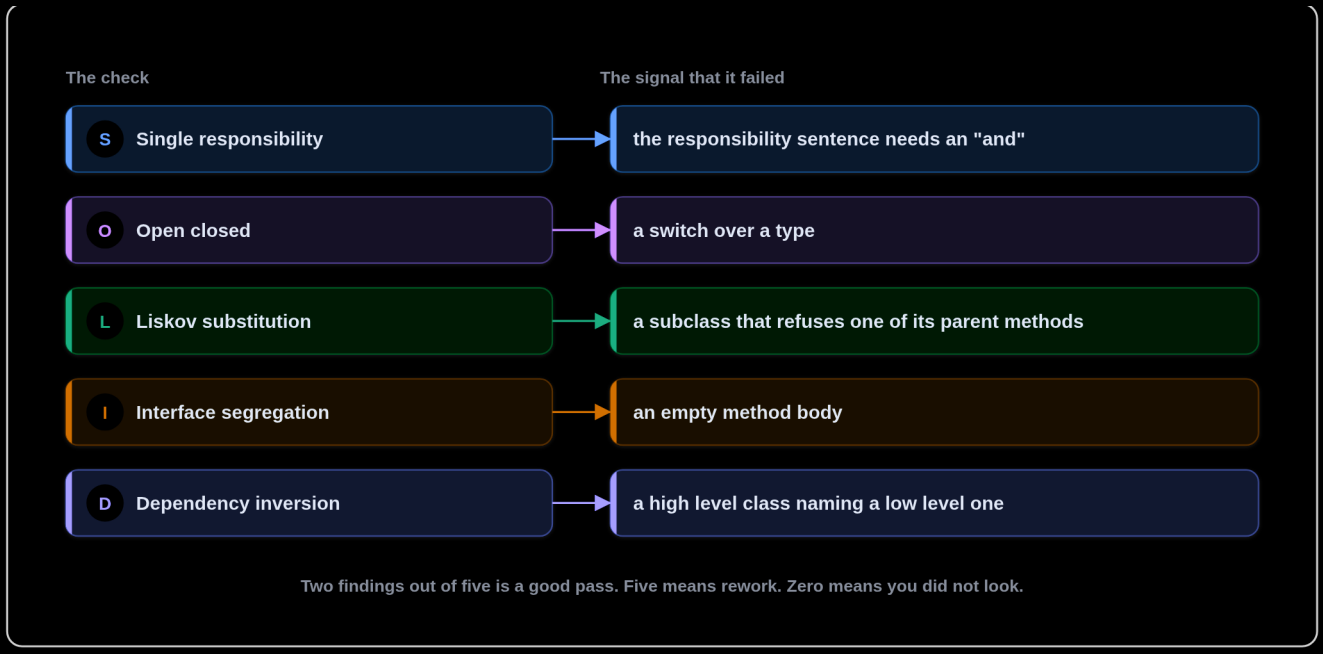


Figure — each check and the signal that it failed: an "and" in the sentence, a switch over a type, a refused parent method, an empty method body, a high-level class naming a low-level one (course diagram)

Two findings out of five is a good pass; five means rework; zero means you didn't look. SRP and OCP earn most credit. Example (parking lot): SRP — `ParkingLot` issues tickets *and* finds spots; LSP — five spot subclasses with no behaviour collapse into one class with a `type` field; OCP, ISP, DIP pass. Don't add one-implementation interfaces or split beyond what a reader can hold.

Naming a design problem out loud

Interviewers weigh: spotting and naming a problem; defending a choice ("pricing changes every quarter" beats "SOLID says so"); knowing when to hold back. **Pattern:** *what you see → what it will cost → what you'd do and what that costs.* Say the observation before the principle's name. To "would you always do that?", the honest answer is almost never "yes". Lose marks by reciting definitions, applying everything at once, or confidently naming the wrong principle. **Drill:** who can ask me to change this class? what's the next feature? which dependencies can't I fake?

4.14 Common questions about SOLID (with short answers)

1. **SRP vs ISP?** SRP guards the class (reasons to change); ISP guards the caller (unused methods).
2. **When not to apply?** Nothing has varied; throwaway code; arbitrary splits; hurts readability. Flexibility later is bought with simplicity now.
3. **Still relevant?** As diagnostic vocabulary, not absolute rules.
4. **Inversion vs injection?** Principle vs technique; a concrete parameter is injection without inversion.
5. **LSP example?** Square/Rectangle, plus the rule: demand no more, guarantee no less.
6. **OCP forbids editing?** No; closure is against a kind of change.
7. **Composition always?** No; when behaviour varies or a subclass would refuse a method.
8. **Principles vs patterns?** Patterns are principles made concrete.
9. **Most violated?** SRP (accumulation), or ISP in reverse (an interface per method).

> **Interview tip:** expand on "when not to apply" and "inversion vs injection"; strong answers always **admit a limit**.

4.15 Over-engineering: the warnings in one place

Warning sign	Misapplied	Instead
`XHelper` / `XUtils` after a split	SRP	Split by actor
Small classes always in the same commit	SRP — *Shotgun Surgery*	Merge back
Interface + factory + registry for one implementation	OCP — *Speculative Generality*	Write the `if`; rule of three
Subclass that silently does nothing	LSP	Better parent or composition
One interface per method; same object thrice	ISP	Group by caller
Interface in front of `String`-like classes	DIP	Invert only volatile deps
DI container for 200 lines	DIP	Wire in `main`

YAGNI is the counterweight: SOLID says **what** a good seam looks like; YAGNI says **when** to cut it.

Chapter self-check

1. **Q:** `switch (vehicleType)` computing fees? **A:** OCP — `ParkingRate` interface, one class per rate.
2. **Q:** `ReadOnlyList extends ArrayList` throws on `add`? **A:** LSP; compose or use a read-only interface.
3. **Q:** `Robot` implements `Worker { work(); eat(); }`? **A:** ISP; split by caller into `Workable` / `Eater`.
4. **Q:** `private final MySqlOrderDao dao = new MySqlOrderDao();`? **A:** DIP violation, untestable; inject an `OrderSaver` the service owns.
5. **Q:** DIP vs DI vs IoC? **A:** Principle; technique; broad idea (containers perform DI).

TL;DR

- **SOLID** names the ways code becomes painful to change — diagnose, don't checklist.
- **SRP** one actor per class; **OCP** varying part behind an interface; **LSP** keep every promise, compose when a subclass would refuse; **ISP** one role per kind of caller; **DIP** policy owns the abstraction, `main` wires.
- **Order**: SRP → OCP → LSP → ISP → DIP, each triggered by a real change.
- **Interview**: observation → cost → fix → price of the fix; know when *not* to apply a principle.

Chapter 5 — More Design Principles (Study Edition)

> **Study Edition.** Every principle of Chapter 5, without the padding. Code blocks are excerpts from the real files (`// ...` marks trimmed parts); full listings and outputs are in the Full Edition (`README.md`). Each topic is one `code/lld/ch05/<topic>/Main.java` with nested `Before` and `After` classes (`javac --release 21`).

SOLID is only five rules. This chapter adds rules of thumb (DRY, KISS, YAGNI), how objects talk (Demeter, Tell-Don't-Ask, loose coupling, Hollywood), the GoF/*Head First* OO principles (encapsulate what varies, composition, program to an interface), correctness (fail fast, immutability, Design by Contract), **GRASP** — nine guidelines for *which class does what* — and **Domain-Driven Design** essentials (*which classes exist and where the boundaries are*).

Key terms. **Coupling** — how much one class depends on another. **Cohesion** — how closely a class's contents belong together. **Abstraction** — a simplified view (interface/abstract class) hiding details. **Encapsulation** — private data changed only through methods. **Invariant** — a fact that must always hold ("balance never negative"). **Side effect** — anything a method changes besides its return value. **Responsibility** — something a class *knows* or *does*.

Chapter map

#	Principle	One-line idea	Comes from
1	DRY	One home per piece of knowledge	Hunt & Thomas (1999)
2	KISS	Simplest design that works	US Navy (1960), Kelly Johnson
3	YAGNI	Don't build before it's needed	Extreme Programming (Ron Jeffries)
4	Law of Demeter	Talk only to immediate friends	Holland / Lieberherr (1987)
5	Tell, Don't Ask	Tell objects what to do	Hunt & Thomas, after Alec Sharp (1997)
6	Composition over inheritance	Prefer has-a for reuse	GoF (1994), *Effective Java*
7	Program to an interface	Depend on supertypes	GoF
8	Encapsulate what varies	Isolate the changing part	GoF, *Head First*
9	Loose coupling	Interacting objects know little	*Head First*
10	Hollywood / IoC	"Don't call us, we'll call you"	*Head First*; frameworks
11	Separation of Concerns	One concern per part	Dijkstra (1974)
12	Least Astonishment	Behave as readers expect	Design folklore
13	Fail fast	Detect problems early, loudly	Jim Shore (2004)
14	Immutability	Never change after creation	*Effective Java*
15	Design by Contract	Pre/postconditions, invariants	Bertrand Meyer, Eiffel (1986)
16	GRASP (9)	Assigning responsibilities	Craig Larman (1997; nine from 2001)
17	Domain-Driven Design	Bounded contexts, entities vs value objects, aggregates, events	Eric Evans (2003); Vaughn Vernon (2013)

1. DRY — Don't Repeat Yourself

What it is

Every piece of **knowledge** (rule, formula, validation, constant) has **one authoritative representation**. Analogy: one master price list. DRY is about knowledge, not look-alike text: identical code representing *different* rules is not a violation.

The problem it solves

Copies **drift** — one is updated, the others aren't — and the system gives two answers to one question.

How it works

Ask "must all copies change together?" If yes, give the rule one home (constant, method, value object, enum, config) and reference it. **Rule of Three** (Don Roberts, via Fowler): tolerate the second copy, extract at the third.

Java example — before/after

Before, two services disagree about `bob@localhost`. After:

```
// File: code/lld/ch05/dry/Main.java
        /** The ONLY definition of "valid email" in the system. */
        record EmailAddress(String value) {
            private static final Pattern FORMAT = Pattern.compile("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
            EmailAddress {
                if (value == null || !FORMAT.matcher(value).matches())
                    throw new IllegalArgumentException("invalid email: " + value);
            }
        }
        // ... TaxPolicy holds the 18% RATE in one private constant
```

Full runnable code: code/lld/ch05/dry/ (see the Full Edition). The compact constructor makes an invalid `EmailAddress` impossible.

Trade-offs

Gain: one change point, no drift, named rules. Cost: a shared abstraction couples its users; a wrong abstraction is worse than duplication.

When not to use it

Coincidental look-alikes ("max username 30" vs "max tag 30"); across microservices (shared libraries couple releases); tests (DAMP readability).

Common mistakes

Monster functions with flags; extracting after one repeat; forgetting duplication across code, DB constraints and docs.

Real-world examples

Spring `@Value`; database normalisation; Bean Validation `@Email`; OpenAPI/Protobuf codegen.

In the interview

> **30-second answer:** "One authoritative home per rule — rules, not look-alike text. I extract when copies must change together, e.g. validation into a value object, using the Rule of Three." *Always a violation?* No — different rules stay apart.

Self-check

Q: What bug does duplication cause? → A: Drift.

TL;DR

One rule, one home; leave coincidental look-alikes alone.

---

2. KISS — Keep It Simple, Stupid

What it is

The plainest design that fully solves the real problem (US Navy c. 1960; Kelly Johnson's field-repairable jets). *Simple* = easy to understand and change, not fewest lines.

The problem it solves

Accidental complexity — added by us, not the problem — is the biggest source of bugs and slow delivery.

How it works

State the requirement in one sentence → most direct solution with standard features → "can a newcomer get it in two minutes?" → add structure only for a concrete need.

Java example — before/after

Three fixed tiers. Before: a priority-sorted rule engine of predicates. After:

```
// File: code/lld/ch05/kiss/Main.java
            return switch (tier) {
                case GOLD -> times(amount, GOLD_FACTOR);
                case SILVER -> times(amount, SILVER_FACTOR);
                case REGULAR -> amount;
            };
```

Full runnable code: code/lld/ch05/kiss/ (see the Full Edition). A `switch` *expression* over an enum without `default` must cover every constant, so a new tier won't compile until handled.

Trade-offs

Simple: fast to read and test, standard constructs, may need refactoring when variation arrives. "Flexible": handles hypotheticals but costs every day and needs docs.

When not to use it

Not an excuse to drop real complexity (payments *need* idempotency, retries, auditing); if users edit rules at run time, a rule engine *is* the requirement.

Common mistakes

Simple ≠ short (five nested lambdas); skipping tests or validation; copy-paste instead of simplifying.

Real-world examples

Unix philosophy; Go's small feature set; `Optional` kept small.

In the interview

> **30-second answer:** "Simplest design that meets the requirements; patterns only when a requirement demands them." *KISS vs YAGNI? How* you build vs *what* you build.

Self-check

Q: Accidental complexity? → A: Complexity from our solution, not the problem.

TL;DR

Solve the real problem plainly.

3. YAGNI — You Aren't Gonna Need It

What it is

Don't build a capability until you need it (XP, Ron Jeffries). The opposite is **speculative generality**.

The problem it solves

Early features cost four times: building, maintaining, surrounding complexity, and being wrong.

How it works

Build today's requirements; keep code clean and tested so later change is cheap; extend when the need arrives; delete unused options.

Java example — before/after

Requirement: CSV export. Before: `export(orders, format, encoding, compress)` where four of five formats throw `UnsupportedOperationException`. After:

```java
// File: code/lld/ch05/yagni/Main.java
        static String toCsv(List<Order> orders) {
            return orders.stream()
                    .map(o -> String.join(",", o.id(), o.customer(), o.total().toPlainString()))
                    .collect(Collectors.joining("\n", "id,customer,total\n", ""));
        }
```

Full runnable code: code/lld/ch05/yagni/ (see the Full Edition).

Trade-offs

YAGNI: less code, design fits real needs, requires test and refactoring discipline. Building ahead: pays only if the guess is right; leaves dead options.

When not to use it

Hard-to-reverse decisions (public APIs, big schemas, security, retention laws); known quality requirements are requirements.

Common mistakes

Excuse for messy code; ignoring a stated requirement; `interface` + `Impl` everywhere.

Real-world examples

MVPs; feature flags (LaunchDarkly, Unleash); iteration planning.

In the interview

> **30-second answer:** "I build today's needs and keep the design easy to extend; I add `PaymentStrategy` only if variants were hinted." *Conflict with OCP?* No — OCP where variation is known; YAGNI stops invented variation.

Self-check

Q: When design ahead anyway? → A: Expensive-to-reverse decisions.

TL;DR

Build for today with clean, changeable code.

---

4. Law of Demeter (Principle of Least Knowledge)

What it is

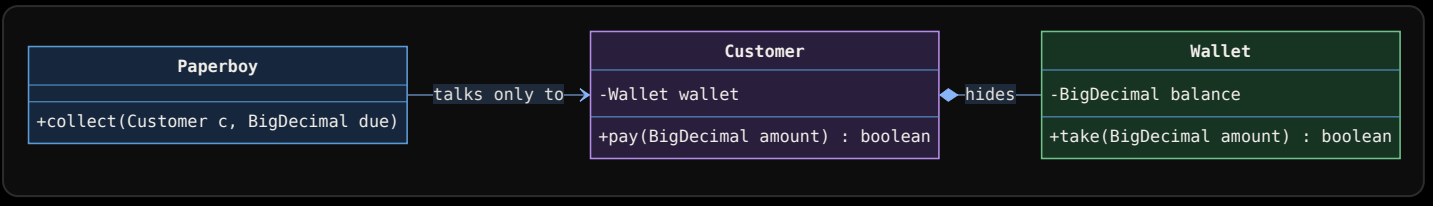
A method `m` of `O` calls only: `O` itself, `m`'s parameters, objects `m` creates, `O`'s own fields (Holland & Lieberherr, 1987; originally also globals). **"Use only one dot"** — `a.getB().getC().doIt()` is a **train wreck**. Analogy: the paperboy asks the customer to pay rather than taking their wallet.

The problem it solves

Chains couple the caller to the whole structure: switch the customer from wallet to UPI and the paperboy breaks.

How it works

Ask what the caller really wants → add that method on the first object (`customer.pay(amount)`) → keep internals private.



Java example — before/after

Before: `c.getWallet().setBalance(c.getWallet().getBalance().subtract(due))`. After:

```
// File: code/lld/ch05/demeter/Main.java
            /** Customer decides HOW to pay. */
            boolean pay(BigDecimal amount) { return wallet.take(amount); }
        // ...
                if (c.pay(due)) {                    // talk only to a parameter: allowed
```

Full runnable code: code/lld/ch05/demeter/ (see the Full Edition). Stream chains aren't violations — each call returns the same kind of thing.

Trade-offs

Following: internal changes stay local, one collaborator to mock, but wrapper methods accumulate. Ignoring: structural changes ripple and tests build object graphs.

When not to use it

Fluent APIs, builders, streams; DTOs/records (`order.shippingAddress().city()`); not dozens of pass-throughs (**Middle Man**).

Common mistakes

Counting dots, not knowledge (temp variables still violate it); getters for everything.

Real-world examples

Facade (*Head First*); Spring Data `findByCustomerId`; `Files.readString(path)`.

In the interview

> **30-second answer:** "A method calls only itself, parameters, objects it creates and its fields. For `a.getB().getC()` I add an intent method — `customer.pay()`." *Streams?* Not a violation.

Self-check

Q: The four friends? → A: `this`, parameters, created objects, own fields.

TL;DR

Only talk to friends.

---

5. Tell, Don't Ask

What it is

Don't pull data out and decide for an object — **tell** it what you want (Hunt & Thomas, after Sharp). Analogy: tell the taxi "airport".

The problem it solves

Ask-then-act spreads rules across callers; one forgets part ("frozen accounts can't withdraw"), the invariant breaks, and the object becomes an **Anemic Domain Model**.

How it works

Find read-decide-write code → move the decision into a method → reject illegal requests → remove setters.

Java example — before/after

Before, the ATM checks `isFrozen()`; the app forgets. After:

```java
// File: code/lld/ch05/telldontask/Main.java
            void withdraw(long amt) {
                if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
                if (frozen) throw new IllegalStateException("account is frozen");
                if (amt > balancePaise) throw new IllegalStateException("insufficient funds");
                balancePaise -= amt;
            }
```

Full runnable code: code/lld/ch05/telldontask/ (see the Full Edition).

Trade-offs

Tell: invariants in one place, richer objects, risk of bloated objects. Ask: duplicated rules; fine for pure data.

When not to use it

Display/reporting queries; decisions combining several objects' data (Information Expert elsewhere).

Common mistakes

A getter "just for this check"; emails and DB calls inside entities.

Real-world examples

`List.removeIf`; `Map.computeIfAbsent`/`merge`; `AtomicInteger.incrementAndGet()`.

In the interview

> **30-second answer:** "`account.withdraw(100)`, not read-then-set the balance — invariants stay in one place." *Concurrency?* Ask-then-act is a check-then-act race; one method can be atomic.

Self-check

Q: Anti-pattern it fights? → A: Anemic Domain Model.

TL;DR

Tell the object what you want.

6. Composition over Inheritance

What it is

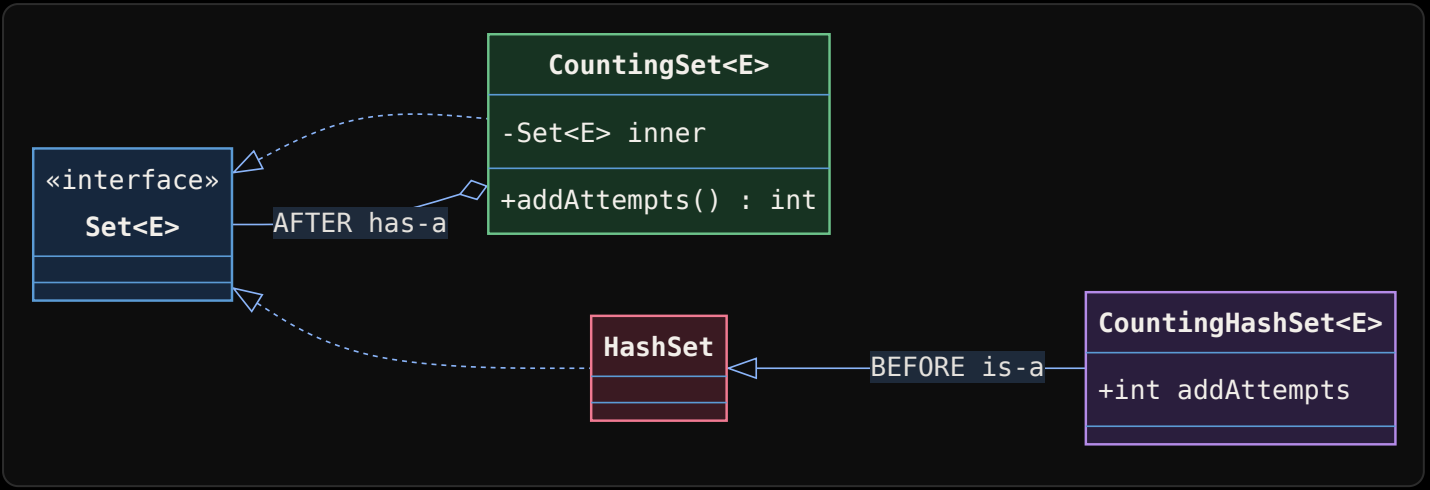
Favour composition over class inheritance (GoF). Is-a reuses by subclassing; has-a holds a reference and **delegates**. Analogy: a car *has* a swappable engine.

The problem it solves

Inheritance is the tightest coupling: **fragile base class** (breaks when parent internals change), fixed at compile time, **class explosion**, every parent method exposed.

How it works

Replace `extends A` with a private interface-typed field, forward calls, inject via constructor; inherit only for substitutable is-a with parents **designed for extension**.



Java example — before/after

The *Effective Java* trap:

```
// File: code/lld/ch05/composition/Main.java
            @Override public boolean addAll(Collection<? extends E> c) {
                addAttempts += c.size();
                return super.addAll(c);   // HashSet's inherited addAll calls add() for each element -> counted
twice!
            }
    // ...
        static final class CountingSet<E> extends AbstractSet<E> {
            private final Set<E> inner;   // "has-a", not "is-a"
            // ...
            @Override public boolean addAll(Collection<? extends E> c) { addAttempts += c.size(); return
inner.addAll(c); }
```

Full runnable code: code/lld/ch05/composition/ (see the Full Edition).

```
BEFORE: added 3 items, counter says 6
AFTER : added 3 items, counter says 3
```

`HashSet` inherits `AbstractCollection.addAll`, which calls our overridden `add` — an unpromised "self-use" detail. The wrapper also forwards `contains`/`remove` (else `AbstractSet` scans, O(n)) and works over a `TreeSet` chosen at run time.

Trade-offs

Composition	Inheritance
Coupled to public API; swap at run time; combine freely	Coupled to internals; fixed; class explosion
Forwarding boilerplate; needs an interface for polymorphism	Free reuse; automatically a subtype

When not to use it

True is-a with a base designed for extension (`AbstractList`, Template Method); sealed closed variants; classes that evolve together.

Common mistakes

`OrderService extends BaseDao`; composition without an interface; exposing the wrapped object.

Real-world examples

`Stack extends Vector`, `Properties extends Hashtable` (admitted JDK mistakes); `BufferedInputStream`; `Collections.unmodifiableList`; Unity's Entity–Component model.

In the interview

> **30-second answer:** "Inheritance couples to parent internals, is fixed and explodes combinatorially; composition delegates to an interface, swappable at run time. Inherit only for true is-a designed for extension." *Composition-based patterns?* Strategy, Decorator, Composite, State, Bridge, Proxy, object Adapter.

Self-check

Q: Why 6? → A: Inherited `addAll` calls the overridden `add`.

TL;DR

Reuse by has-a; is-a only for substitutable subtypes designed for extension.

---

7. Program to an Interface, Not an Implementation

What it is

Type variables, fields, parameters and returns by the **most general supertype** you need ("interface" = any supertype). Analogy: a charger built for the USB-C standard.

The problem it solves

Code typed on `ArrayList` or `MySqlUserStore` is untestable without the real thing and not swappable.

How it works

Name the role → interface with only needed methods → type by it → create concrete objects at the edges and inject them (**DI**) → widest parameter type (`Collection` > `List` > `ArrayList`).

Java example — before/after

Before, `GreetingService` does `new MySqlUserStore()` (can't run on a laptop) and `countLongNames` takes `ArrayList`. After:

```java
// File: code/lld/ch05/interfaces/Main.java
        interface UserRepository { Optional<User> findById(String id); }
        // ...
            private final UserRepository users;                  // any implementation
            GreetingService(UserRepository users) { this.users = users; }
        // ...
        static int countLongNames(Collection<String> names) {   // works for List, Set, Queue...
```

Full runnable code: code/lld/ch05/interfaces/ (see the Full Edition).

Trade-offs

Interface-typed: swappable, testable, but no implementation-only methods and one more type. Concrete-typed: simple, but changes ripple.

When not to use it

Value types; one implementation with no test seam (`FooServiceImpl` ceremony); genuinely specific needs (`LinkedHashMap` access order for LRU).

Common mistakes

30-method mirror interfaces (ISP); declaring `List` then casting; still `new`-ing inside.

Real-world examples

Collections Framework; JDBC; SLF4J; Spring injection by interface.

In the interview

> **30-second answer:** "I type dependencies by roles and inject implementations — in-memory for tests. It's DIP in everyday form." *Every class an interface?* No.

Self-check

Q: Where does `new ConcreteClass()` go? → A: The edges: `main`, factories, DI config.

TL;DR

Depend on roles; pick the concrete class once, at the edge.

---

8. Encapsulate What Varies

What it is

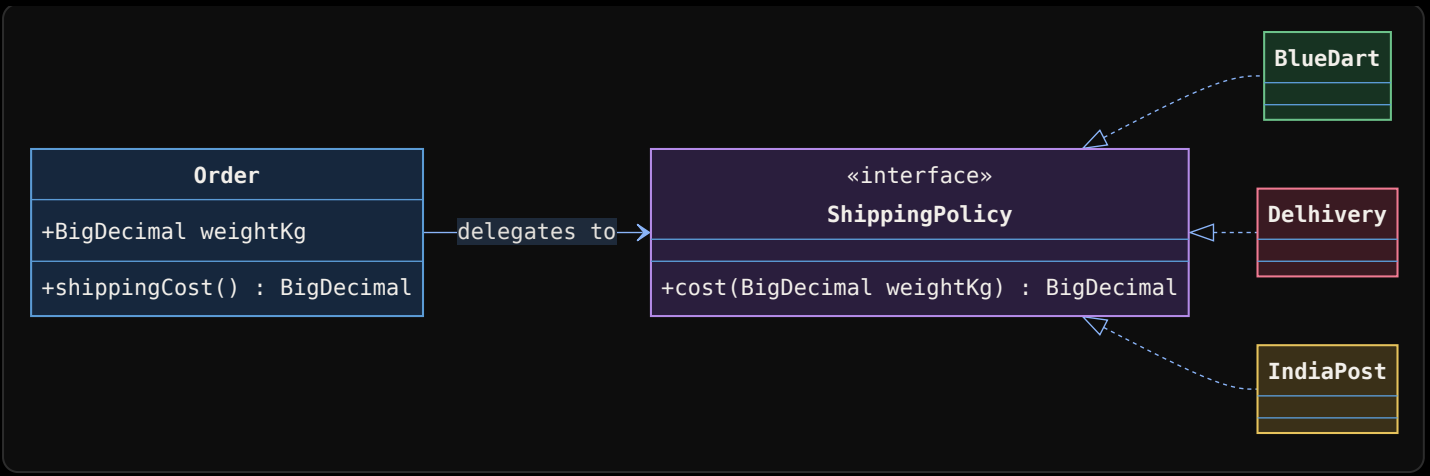
Separate what **changes** from what stays the same; put the varying part behind an abstraction (*Head First*'s first principle). Analogy: camera body, swappable lenses.

The problem it solves

Volatile rules inline force edits to stable classes; type `switch`es grow forever (OCP broken).

How it works

Find what changed recently or differs per customer/region → extract behind an interface/lambda → delegate → add variants. Behind Strategy, State, Factory, Observer, Template Method.



Java example — before/after

Before, `Order.shippingCost()` branches on carrier strings. After:

```
// File: code/lld/ch05/encapsulatevaries/Main.java
        interface ShippingPolicy { BigDecimal cost(BigDecimal weightKg); }

        /** Stable catalogue of today's policies. Adding one = adding one entry; Order never changes. */
        static final Map<String, ShippingPolicy> POLICIES = Map.of(
                "BLUEDART", w -> linear("50", "20", w),
                "DELHIVERY", w -> linear("40", "25", w),
                "INDIA_POST", w -> linear("30", "10", w));

        record Order(BigDecimal weightKg, ShippingPolicy shipping) {
            BigDecimal shippingCost() { return shipping.cost(weightKg); }
        }
```

Full runnable code: code/lld/ch05/encapsulatevaries/ (see the Full Edition). A promo policy was added with zero edits to `Order`.

Trade-offs

Encapsulated: new variants without touching stable code, each testable; more indirection. Inline: all visible, fine for two fixed variants, grows into long conditionals.

When not to use it

It has never varied (YAGNI); extract at the third variant.

Common mistakes

Wrong axis of variation; leaving the type `switch` in the client.

Real-world examples

`Comparator`; `ThreadPoolExecutor`'s `RejectedExecutionHandler`/`ThreadFactory`; payment gateways.

In the interview

> **30-second answer:** "I hide what's likely to change — pricing, payment method — behind an interface; it's behind Strategy and State." *How do you know what varies?* Requirements, history, the interviewer's hints.

Self-check

Q: Smell of missing encapsulation? → A: A growing type `switch`.

TL;DR

Put the changing part behind an interface.

9. Strive for Loosely Coupled Designs

What it is

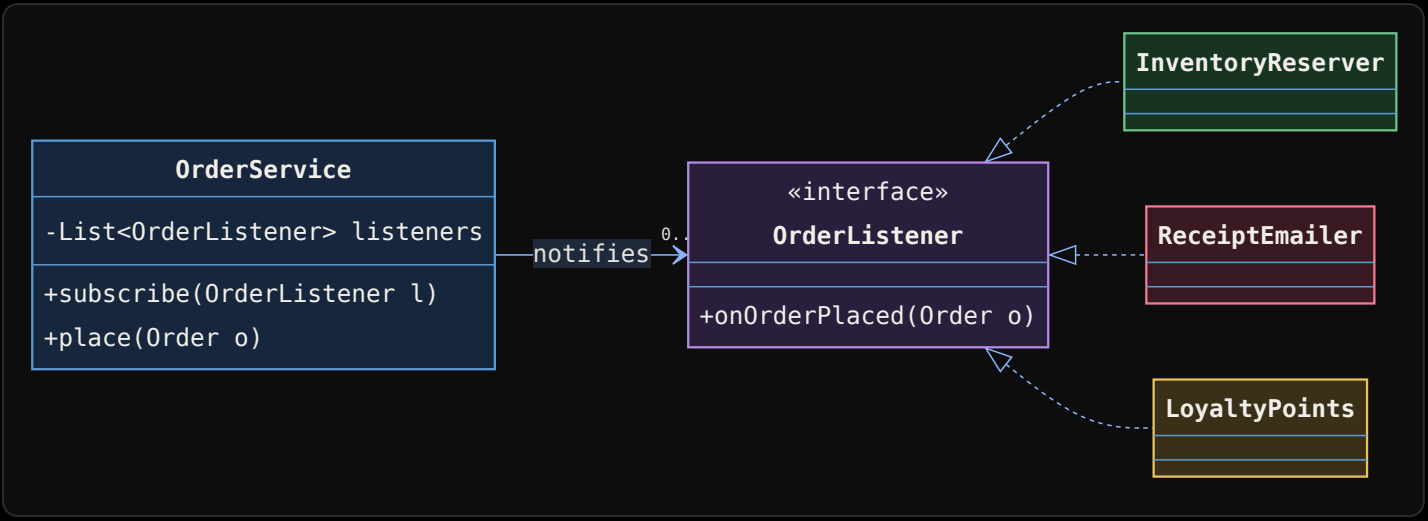
Interacting objects know as little as possible about each other — ideally a small interface (*Head First*). Analogy: a radio station doesn't know its listeners.

The problem it solves

A class that creates and calls many concrete collaborators is edited for every change, not reusable, not testable alone.

How it works

Interfaces; injection; **events** when the sender shouldn't know receivers (Observer); narrow interfaces; immutable values. Coupling, tightest → loosest: content → common → external → control → stamp → data → message.



Java example — before/after

Before, `OrderService` creates `EmailSender`, `Inventory`, `Analytics`. After:

```
// File: code/lld/ch05/loosecoupling/Main.java
            private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
            void subscribe(OrderListener l) { listeners.add(l); }
            void place(Order o) {
                System.out.println("placed " + o.id());
                listeners.forEach(l -> l.onOrderPlaced(o));   // doesn't know who listens or what they do
            }
```

Full runnable code: code/lld/ch05/loosecoupling/ (see the Full Edition). Loyalty points became one more `subscribe`; `CopyOnWriteArrayList` allows safe subscription during notification.

Trade-offs

Loose	Tight
Independent change and tests; features = listeners	Flow easy to follow and debug
"Who handles this?"; listener order and errors need design	Every change edits the central class

When not to use it

Small cohesive modules; ordered, atomic steps (debit then credit) — orchestrate explicitly.

Common mistakes

A global event bus for everything; shared static state; one throwing listener stopping the rest.

Real-world examples

UI listeners; Spring `@EventListener`; Kafka (`OrderPlaced` consumed independently by inventory, notifications, analytics).

In the interview

> **30-second answer:** "Collaborators share only small interfaces — interfaces, DI, events; the order service publishes 'order placed' and features subscribe." *Downside?* Harder to trace flow, ordering, failures.

Self-check

Q: Classic one-to-many pattern? → A: Observer.

TL;DR

Small interfaces, injection and events.

---

10. The Hollywood Principle (Inversion of Control)

What it is

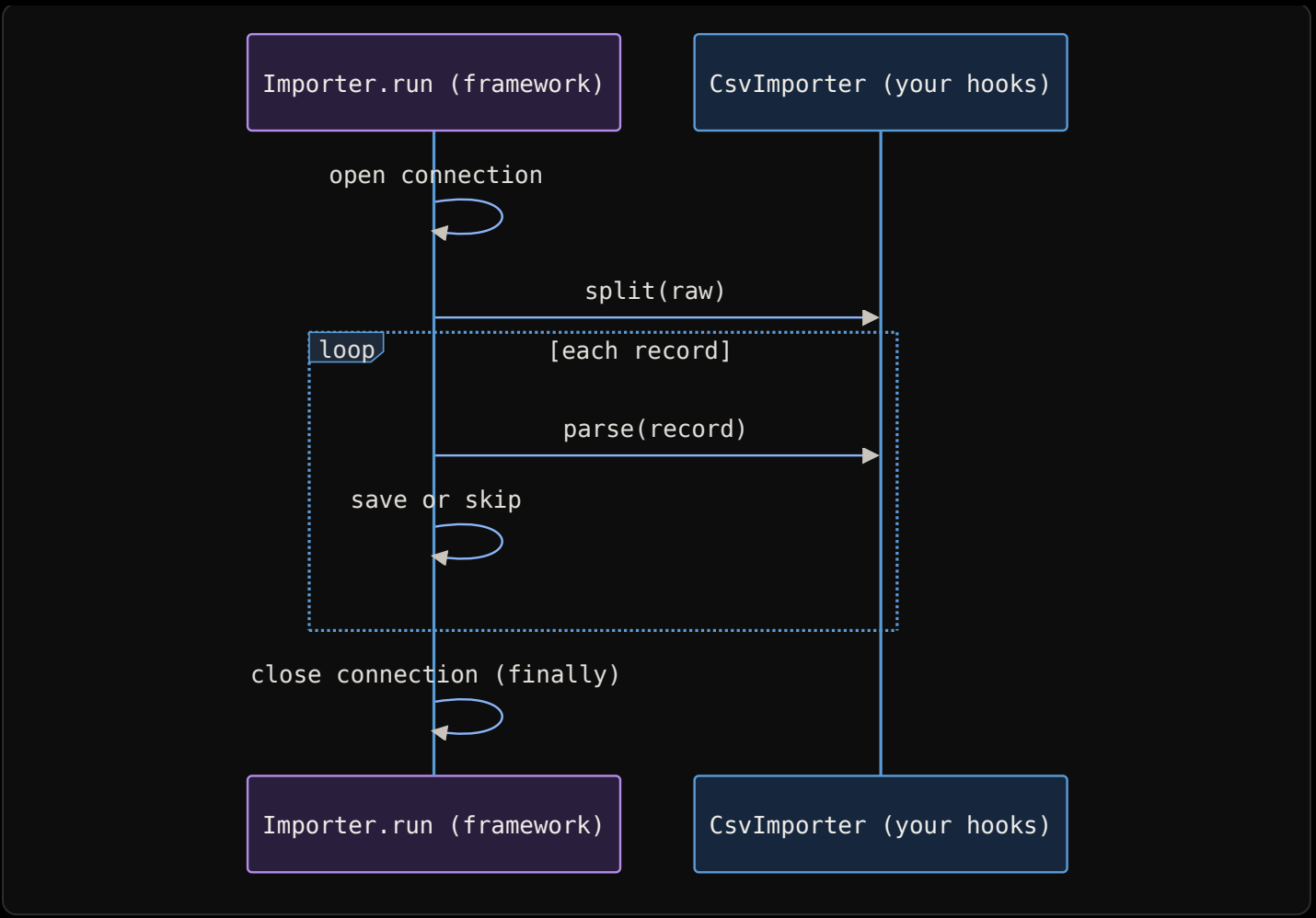
"Don't call us, we'll call you." High-level components own the flow and call low-level hooks, which never call up — **Inversion of Control. You call a library; a framework calls you.**

The problem it solves

Each component re-implements the skeleton, forgets steps (closing connections), and levels depend on each other (**dependency rot**).

How it works

Template Method (`final` algorithm, abstract hooks); **callbacks**; **DI**; **lifecycle management** (`@PostConstruct`, servlet `init`/`destroy`).



Java example — before/after

Before, `CsvImporter.run` drives everything and forgets to close. After:

```java
// File: code/lld/ch05/hollywood/Main.java
    /** The fixed algorithm. final = subclasses cannot reorder or skip steps. */
    public final void run(String raw) {
        System.out.println("  open connection");
        try {
            // ... for each record from split(raw): parse, then save or skip
        } finally {
            System.out.println("  close connection");       // guaranteed by the framework
        }
    }
    protected abstract List<String> split(String raw);      // hooks: WE call YOU
    protected abstract String[] parse(String record);
```

Full runnable code: code/lld/ch05/hollywood/ (see the Full Edition). A tiny `Container` also starts `Lifecycle` components in order and stops them in reverse (as Spring does).

Trade-offs

IoC: guaranteed flow, ordering and cleanup, no duplicated skeletons; but "magic", harder to debug, tied to hooks. Direct control: visible flow, duplicated skeletons.

When not to use it

Small scripts with no repeated skeleton.

Common mistakes

Hooks calling back up; template method not `final`; confusing IoC (idea) with DI (one form).

Real-world examples

JUnit `@Test`; servlets/Spring MVC; Spring lifecycles; Android `onCreate`; `AbstractList`; `JdbcTemplate` + `RowMapper`.

In the interview

> **30-second answer:** "The high-level component controls flow and calls hooks — what separates a framework from a library; Template Method, callbacks and DI are forms." *vs DIP?* DIP: depend on abstractions; Hollywood: who calls whom.

Self-check

Q: Why `final` template method? → A: Subclasses can't reorder or skip steps.

TL;DR

The framework owns the flow and calls your hooks.

---

11. Separation of Concerns (SoC)

What it is

A **concern** is one aspect (input, rules, storage, presentation, security, logging); each part handles one (Dijkstra, 1974, EWD 447). Big-picture SRP, at every level: functions, classes, layers (MVC), services, languages (HTML/CSS/JS).

The problem it solves

Spaghetti: can't change HTML without risking the grading rule; can't test the rule without parsing strings.

How it works

One unit per concern with typed inputs/outputs; data flows one way (parse → rules → store → present); lower layers don't know higher ones; **cross-cutting concerns** via filters, decorators, proxies, aspects.



Java example — before/after

Before, one `handle(String request)` parses, grades, stores and returns HTML. After, the rules class:

```java
// File: code/lld/ch05/soc/Main.java
        /** Concern 2: business rules only. No HTML, no storage. */
        static final class GradingService {
            // ...
            Graded record(ResultRequest r) {
                if (r.marks() < 0 || r.marks() > 100) throw new IllegalArgumentException("marks must be 0..100");
                String grade = r.marks() >= 90 ? "A" : r.marks() >= 75 ? "B" : r.marks() >= 50 ? "C" : "F";
                repo.save(r.name(), r.marks());
                return new Graded(r.name(), r.marks(), grade);
            }
        }
```

Full runnable code: code/lld/ch05/soc/ (see the Full Edition). `RequestParser`, `ResultRepository` and `HtmlView` hold the other concerns; a JSON view would not touch rules.

Trade-offs

Separated: safe changes, parallel team work, more types. Mixed: quick one-offs, tangled and hard to test.

When not to use it

Throwaway scripts; avoid "lasagna code" (layers that only pass data through).

Common mistakes

Rules in controllers or SQL; separating only by technical type while a feature is scattered.

Real-world examples

MVC; HTML/CSS/JS; TCP/IP layers; Spring AOP, servlet filters.

In the interview

> **30-second answer:** "Each module handles one aspect so each changes and tests alone — controller, service, repository, domain in the middle." *Cross-cutting?* Logging, auth, transactions via AOP/proxies/filters.

Self-check

Q: Four concerns of a web request? → A: Parsing, rules, persistence, presentation.

TL;DR

Each concern its own home.

---

12. Principle of Least Astonishment (POLA)

What it is

A component behaves as users expect from its name, signature and context; if not, change the behaviour or the name. Analogy: a light switch that also locks the door.

The problem it solves

Surprises cause bugs even when code "works": mutating queries, mutate-and-return-`this` aliasing, inconsistent names, 0/1-based confusion, swallowed errors.

How it works

Truthful names (`find` doesn't modify; `getOrCreate` if it creates); platform conventions; consistency; immutable results; `Optional` for absence; document unavoidable surprises.

Java example — before/after

Before, `Money.add` mutates `this` and `findUser` silently creates users. After:

```
// File: code/lld/ch05/leastastonishment/Main.java
        Money plus(Money other) { return new Money(paise + other.paise); }   // returns a NEW value
    // ...
        Optional<String> findUser(String id) { return Optional.ofNullable(users.get(id)); }
        String getOrCreateUser(String id) { return users.computeIfAbsent(id, k -> "new-user-" + k); }
```

Full runnable code: code/lld/ch05/leastastonishment/ (see the Full Edition).

```
new BigDecimal("2.0").equals(new BigDecimal("2.00")) = false
new BigDecimal("2.0").compareTo(new BigDecimal("2.00")) = 0
Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)
```

JDK gotchas: use `compareTo` for numeric equality; `Arrays.asList` is fixed-size.

Trade-offs

A longer name or extra method is cheap next to surprise bugs; "expected" depends on your platform's conventions.

When not to use it

Never surprise on purpose, but fix a name rather than bend to a wrong expectation.

Common mistakes

Getters with side effects; overloading `equals(Money)` instead of overriding `equals(Object)`; hidden global flags.

Real-world examples

`java.time` fixing `Date`/`Calendar`; `URL.equals` doing DNS (prefer `URI`); `rm` without undo; "Cancel" never saves.

In the interview

> **30-second answer:** "Queries don't mutate, creation is named, absence is `Optional`, values immutable." *CQS?* Command-Query Separation (Chapter 6) makes state changes obvious.

Self-check

Q: `findUser` on a missing user? → A: Empty `Optional`, never silent creation.

TL;DR

Do what the name promises and nothing more.

---

13. Fail Fast

What it is

Detect invalid state or input **where it arises** and stop with a clear error. Analogy: a smoke alarm at the first smoke.

The problem it solves

A `null` accepted in a constructor crashes hours later, pointing at the symptom; bad data may be saved first.

How it works

Validate at boundaries (constructors, public entry, API input, startup config); guard clauses (`requireNonNull`, `IllegalArgumentException`); specific messages; invalid objects **unconstructable**; never swallow exceptions.

Fail fast vs fail safe: fail-*safe* = failure leaves a safe state. Interview literature (not the JDK, which says "snapshot"/"weakly consistent") calls `CopyOnWriteArrayList`/`ConcurrentHashMap` iterators "fail-safe" (never throw `ConcurrentModificationException`); **fail-fast** iterators (`ArrayList`) throw on structural modification during iteration — **best-effort** only.

Java example — before/after

Before, `new RetryConfig(null, 0, null)` succeeds and an NPE appears far away. After:

```java
// File: code/lld/ch05/failfast/Main.java
        record RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
            RetryConfig {
                Objects.requireNonNull(endpoint, "endpoint");
                Objects.requireNonNull(backoff, "backoff");
                if (maxAttempts < 1) throw new IllegalArgumentException("maxAttempts must be >= 1, was " +
maxAttempts);
                if (backoff.isNegative()) throw new IllegalArgumentException("backoff must not be negative");
            }
        }
```

Full runnable code: code/lld/ch05/failfast/ (see the Full Edition). The demo also triggers an `ArrayList` `ConcurrentModificationException` and shows the safe `removeIf`.

Trade-offs

Fail fast: bugs near their cause, clear diagnostics, upfront validation, may stop a process for a minor issue. Fail slow: "keeps running", corruption, confusing symptoms.

When not to use it

Fail fast inside components, but be resilient at edges (a widget failure shouldn't crash the UI; a batch job may skip and log). Detect early, then handle deliberately.

Common mistakes

Swallowing exceptions or returning `null` / `-1`; validating deep inside; `assert` for public arguments (disabled by default).

Real-world examples

`List.of(null)`; Spring Boot startup checks; circuit breakers (Resilience4j); Erlang "let it crash"; CI stopping at first failure.

In the interview

> **30-second answer:** "Validate at the boundary and throw clearly, near the cause — guard clauses so invalid objects can't exist; at system level, circuit breakers." *Bad for users?* Fail fast inside; degrade gracefully at the edge.

Self-check

Q: Why not `assert` for public checks? → A: Disabled by default.

TL;DR

Check early, throw clearly, never swallow.

---

14. Immutability

What it is

An object whose observable state can't change after construction; "changing" creates a new one (`String`, `LocalDate`, `BigDecimal`, records of immutables). Analogy: reprint a receipt, don't edit it.

The problem it solves

Aliasing bugs, **broken invariants**, **thread-safety problems**, **hash-collection bugs**. Immutables are inherently thread-safe, cacheable, safe keys.

How it works

Effective Java: no mutators; not extendable (`final`; records are); all fields `final` and `private`; **defensive copies** of mutable components in and out. Plus "withers". `Collections.unmodifiableList` is a live *view*; `List.copyOf` / `List.of` are independent. Records are **shallowly** immutable.

Java example — before/after

Before, `Itinerary` keeps and returns the caller's list — outsiders clear it. After:

```
// File: code/lld/ch05/immutability/Main.java
        record Itinerary(List<String> cities, LocalDate start) {
            Itinerary {
                cities = List.copyOf(cities);   // copy in: caller's later edits cannot reach us
            }
            Itinerary withStart(LocalDate newStart) { return new Itinerary(cities, newStart); }
            // ... plusCity(c) copies, adds, returns a new Itinerary
        }
```

Full runnable code: code/lld/ch05/immutability/ (see the Full Edition). `t1.cities().clear()` throws; `plusCity(...).withStart(...)` yields a new `t2`. `LocalDate.of(2026,1,31).plusMonths(1)` returns Feb 28.

Trade-offs

Immutable: thread-safe, safe keys, cacheable; an allocation per change (usually cheap), copying large objects. Mutable: in-place updates, needs synchronisation, aliasing bugs.

When not to use it

Hot loops (local `StringBuilder`, expose the immutable result); entities with changing state (encapsulate; make their values immutable); JPA entities.

Common mistakes

`final` field ≠ immutable; returning the internal list; `unmodifiableList` over a caller-held list; `this` escaping the constructor.

Real-world examples

`List.of`, `Map.of`, records; Git commits; Kafka records; Redux/React state.

In the interview

> **30-second answer:** "Final class, private final fields, no setters, defensive copies — thread-safe without locks, safe as keys. Value objects immutable; entity state encapsulated." *Records?* Shallowly immutable.

Self-check

Q: `unmodifiableList` vs `List.copyOf`? → A: Live view vs independent copy.

TL;DR

No setters, final everything, copy in and out.

---

15. Design by Contract (DbC)

What it is

(Meyer, Eiffel) **Precondition** — caller's duty (`pop` needs a non-empty stack); **postcondition** — method's guarantee (after `push(x)` size +1, top `x`); **class invariant** — always true between calls (`0 ≤ size ≤ capacity`).

The problem it solves

Without contracts, duplicated defensive checks or none, ambiguous returns (`-1`), lost data.

How it works

Document the contract → check preconditions at entry (`IllegalArgumentException` / `IllegalStateException`) → check postconditions/invariants in dev/tests → offer queries (`isEmpty()`, `isFull()`) → subclasses may **weaken** preconditions and **strengthen** postconditions, keeping the invariant (LSP).

Java example — before/after

Before, a full stack drops pushes and an empty one returns `-1`. After:

```
// File: code/lld/ch05/contract/Main.java
            /** pre: !isFull(). post: size() == old size + 1 and peek() == x. */
            void push(int x) {
                require(!isFull(), "push requires !isFull()");
                int oldSize = size;
                items[size++] = x;
                ensure(size == oldSize + 1 && peek() == x, "push postcondition");
                checkInvariant();
            }
```

Full runnable code: code/lld/ch05/contract/ (see the Full Edition). Output: `precondition failed: push requires !isFull()`.

Trade-offs

Explicit: clear duties, precise errors, executable docs; runtime cost (postcondition/invariant checks can be off in production). Implicit: less code, ambiguity, silent corruption.

When not to use it

Full checks in hot loops — keep preconditions on public APIs, move expensive checks to tests or `assert`.

Common mistakes

`assert` for public preconditions; subclasses strengthening preconditions (`SavingsAccount.withdraw`; `Square.setWidth` is the postcondition mirror); special return values.

Real-world examples

Java API docs (`List.get`, `equals` / `hashCode`); Guava `Preconditions`, Spring `Assert`; Kotlin `require()` / `check()`; Eiffel, Ada 2012.

In the interview

> **30-second answer:** "Preconditions for the caller, postconditions guaranteed, invariants always true; guard clauses and query methods; subclasses weaken pre, strengthen post — LSP." *vs defensive programming?* DbC assigns each check to one party.

Self-check

Q: Whose duty is a precondition? → A: The caller's.

TL;DR

State and enforce who promises what.

---

16. GRASP — General Responsibility Assignment Software Patterns

What it is

Craig Larman's nine patterns for **"which class should be responsible for this?"** (knowing or doing) — a vocabulary for justifying decisions, not code templates. (The 1997 edition's ninth was "Don't Talk to Strangers"; 2001 replaced it with **Protected Variations**.)

#	Guideline	Question	Short answer
1	Information Expert	Who does this job?	The class with the information
2	Creator	Who creates A?	One that contains, records, closely uses or initialises A
3	Controller	Who first handles a UI event?	A non-UI use-case or system object
4	Low Coupling	Reduce impact of change?	Few dependencies
5	High Cohesion	Keep classes focused?	Related work per class
6	Polymorphism	Behaviour varies by type?	Each type implements the operation
7	Pure Fabrication	No domain class fits?	Invent a helper (repository)
8	Indirection	Avoid direct coupling?	An intermediate object
9	Protected Variations	Shield from change?	Stable interface around instability

The problem it solves

A god UI/`Manager` class plus anemic data, or logic scattered at random; GRASP gives explainable reasons ("`total()` is on `Sale` — Expert").

How it works

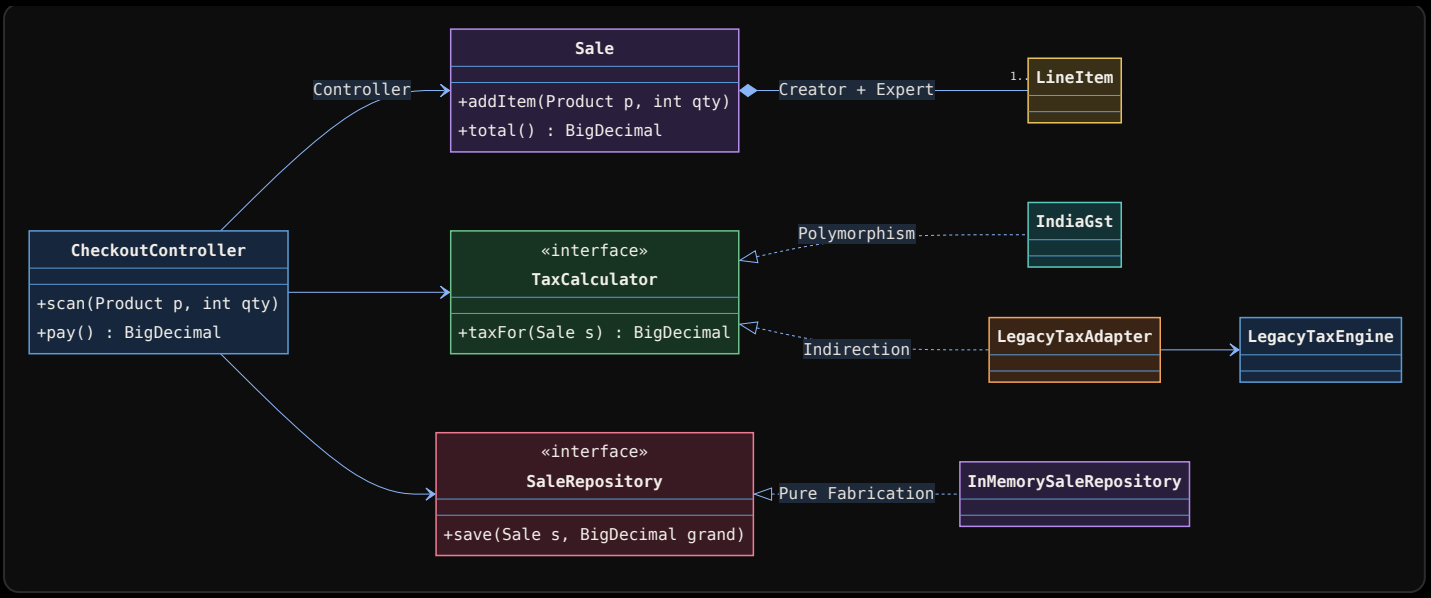
List system events → **Controller** each → **Expert** for what's known/computed → **Creator** for each object → judge by **coupling/cohesion** → **Polymorphism** for type variation, **Protected Variations** via **Indirection** for instability → **Pure Fabrication** where no domain object fits.

Java example — before/after (all nine in one checkout)

Before, `CheckoutScreen` creates items, totals them, hard-codes regional tax and builds SQL. After (each Javadoc names its guideline):

```
// File: code/lld/ch05/grasp/Main.java
        /** INFORMATION EXPERT: a line item knows its product and quantity, so it computes its own subtotal. */
        record LineItem(Product product, int qty) {
            BigDecimal subtotal() { return product.price().multiply(BigDecimal.valueOf(qty)); }
        }
        // ...
            void addItem(Product p, int qty) { items.add(new LineItem(p, qty)); }        // Creator
        // ...
        /** PROTECTED VARIATIONS: tax rules vary by region/vendor -> hide them behind a stable interface. */
        interface TaxCalculator { BigDecimal taxFor(Sale sale); }
        // ... IndiaGst, FlatUsSalesTax (POLYMORPHISM); LegacyTaxAdapter (INDIRECTION); SaleRepository (PURE
FABRICATION)
            BigDecimal pay() {
                BigDecimal grand = current.total().add(tax.taxFor(current)).setScale(2, RoundingMode.HALF_UP);
                repo.save(current, grand);
                return grand;
            }
```

Full runnable code: code/lld/ch05/grasp/ (see the Full Edition). Grand totals: `IndiaGst` 377.60, `FlatUsSalesTax` 342.40, `LegacyTaxAdapter` 384.00.



16.1 Information Expert

What it is

The class that **has the information** does the job: `LineItem.subtotal()`, `Sale.total()`.

The problem it solves

Managers pulling data out (*Feature Envy*).

How it works

Job → needed information → who holds it → partial experts cooperate.

Trade-offs

Behaviour with data; blindly applied, puts SQL in `Sale`.

When not to use it

When it adds persistence, UI or network work to domain objects.

Common mistakes

Everything in services via getters; entities that save or email themselves.

Real-world examples

`String.length()`, `LocalDate.isLeapYear()`; a `Cart` totalling itself.

In the interview

"`total()` is on `Order` because it owns the line items."

Self-check

Q: Who computes a line's subtotal? → A: `LineItem`.

TL;DR

The one with the data does the work.

16.2 Creator

What it is

B creates A if B *contains*, *records*, *closely uses* or *has initialising data* for A: `Sale` creates `LineItem`s.

The problem it solves

Random `new` spreads coupling to A's constructor.

How it works

Prefer the container; complex or varying creation → Factory/Builder (Chapter 8).

Trade-offs

Low-coupling for plain objects; unsuitable for complex creation.

When not to use it

Complex creation; DI-managed objects.

Common mistakes

UI code doing `new LineItem(...)`.

Real-world examples

`HashMap` nodes; `Document` → `Paragraph`; `ExecutorService` via `ThreadFactory`.

In the interview

"`Sale.addItem` creates the item because the sale contains it." *vs Factory?* A Factory is a Pure Fabrication for complex creation.

Self-check

Q: Strongest reason to create? → A: Containment.

TL;DR

The container creates.

16.3 Controller

What it is

The first **non-UI** object receiving a **system operation**: **facade** (`Register`, `ParkingLotSystem`) or **use-case** (`CheckoutController`) controller.

The problem it solves

Keeps logic out of UI so web, mobile and tests share it.

How it works

Assign each system event to a controller that **delegates**.

Trade-offs

Clean split; a controller doing real work gets **bloated** — split by use case.

When not to use it

Trivial programs.

Common mistakes

Fat controllers with pricing and SQL; a `JFrame` computing totals.

Real-world examples

`@RestController` → `@Service`; `DispatcherServlet`; `TripService.requestRide()`.

In the interview

"`ParkingLotController.park()` receives the event and delegates."

Self-check

Q: Should it calculate tax? → A: No; it delegates.

TL;DR

A thin coordinator per system event.

16.4 Low Coupling

What it is

Evaluative: prefer fewer dependencies; the controller knows only `TaxCalculator` and `SaleRepository`.

The problem it solves

Rippling changes, no reuse.

How it works

Compare alternatives: interfaces, fewer collaborators, one direction, no cycles, DI.

Trade-offs

Stable dependencies (JDK) are harmless; chasing zero bloats indirection.

When not to use it

Don't decouple from standard libraries.

Common mistakes

Package cycles; UI knowing the DB.

Real-world examples

Microservices over APIs; JDBC drivers.

In the interview

"I depend on `PaymentGateway`, not Stripe's SDK." *Zero coupling?* Impossible.

Self-check

Q: Is `java.util.List` bad coupling? → A: No; it's stable.

TL;DR

Fewer, more stable dependencies.

16.5 High Cohesion

What it is

Evaluative: each class's responsibilities focused and related.

The problem it solves

`Utils`/`Manager` classes changing for many reasons.

How it works

"Is this related to what the class does?" Larman's levels: *very low* (many different areas) → *low* (one big complex job) → *moderate* (light unrelated jobs) → *high* (related jobs, collaborating for the rest — the goal).

Trade-offs

More, simpler files; improves alongside coupling.

When not to use it

Rarely — coarse-grained remote APIs, on purpose.

Common mistakes

Utils dumping grounds; entities sending emails.

Real-world examples

`LocalDate`, `Pattern`, Spring Data repositories.

In the interview

"I split `BookingService` into seat locking, pricing, payment."

Self-check

Q: A low-cohesion smell? → A: A `Utils`/`Manager` class.

TL;DR

One related set of things per class.

16.6 Polymorphism

What it is

When behaviour **varies by type**, each type implements a shared operation (`taxFor`), no type `switch`.

The problem it solves

Repeated type checks; new types edit every branch.

How it works

Declare the varying operation on an interface; implement per variant.

Trade-offs

Easy to add types, hard to add operations (Visitor, Chapter 11, flips it).

When not to use it

Closed type sets gaining operations — `sealed` + pattern-matching `switch`.

Common mistakes

Variation that doesn't exist; leftover `instanceof` chains.

Real-world examples

`Comparator`, `InputStream`, `Runnable`/`Callable`, card/UPI/wallet payments.

In the interview

"Each `Vehicle` answers `requiredSpotSize()`."

Self-check

Q: Which change is easy? → A: Adding a type.

TL;DR

Each type implements the varying behaviour.

16.7 Pure Fabrication

What it is

A **non-domain** class invented to keep cohesion high and coupling low: `SaleRepository`.

The problem it solves

Persistence, logging, IDs fit no domain class cleanly.

How it works

When Expert drags technical work into a domain class, create a focused service (`IdGenerator`).

Trade-offs

Good cohesion and reuse; overuse makes an anemic domain.

When not to use it

A domain object is a natural home — Expert first.

Common mistakes

Domain rules in "Manager" services.

Real-world examples

DAOs, Jackson `ObjectMapper`, `Clock`, Strategy/Adapter/Factory objects.

In the interview

"`TicketRepository` keeps persistence out of `Ticket`."

Self-check

Q: Is `Sale` a fabrication? → A: No; `SaleRepository` is.

TL;DR

Invent a helper when no domain class fits.

16.8 Indirection

What it is

An **intermediate object** so two components aren't directly coupled: `LegacyTaxAdapter` before `LegacyTaxEngine`.

The problem it solves

Direct dependencies on external or volatile parts.

How it works

Interface plus adapter, facade, mediator or proxy.

Trade-offs

Decoupling vs an extra hop (Wheeler's "another level of indirection" — except too many levels).

When not to use it

Stable classes that change together.

Common mistakes

Pure pass-through layers.

Real-world examples

Adapter/Facade/Mediator/Proxy; `DriverManager`; API gateways; brokers.

In the interview

"`SmsSender` with a `TwilioSmsSender` adapter." *vs PV?* Indirection is the mechanism, PV the goal.

Self-check

Q: GoF forms? → A: Adapter, Facade, Mediator, Proxy (and Bridge).

TL;DR

A middleman between strangers.

16.9 Protected Variations

What it is

Wrap **predicted variation** in a **stable interface** (`TaxCalculator`). Larman: Parnas's *information hiding* and OCP as a responsibility rule; Demeter is one mechanism.

The problem it solves

One vendor/regulation/format change forcing edits everywhere.

How it works

Find variation points → stable interface → variants behind it.

Trade-offs

Protection costs; protect only predicted variation.

When not to use it

Stable code (YAGNI).

Common mistakes

Leaky interfaces (`StripeCharge` from `PaymentGateway`).

Real-world examples

JDBC, JPA, SLF4J, `ServiceLoader`, `FileSystem`.

In the interview

"Payment providers vary, so `PaymentGateway` is my protected variation."

Self-check

Q: Closest SOLID principle? → A: OCP (also DIP).

TL;DR

Wrap what's unstable behind a stable interface.

GRASP in the interview (whole set)

> **30-second answer:** "Expert and Creator place most behaviour, a thin Controller takes system events, Polymorphism and Protected Variations handle variation, Indirection and Pure Fabrication keep technical concerns out of the domain, and Low Coupling/High Cohesion judge every choice." *vs GoF?* Strategy = Polymorphism + PV; Adapter = Indirection; Repository = Pure Fabrication.

Self-check (GRASP)

1. Q: Which two are evaluative? → A: Low Coupling, High Cohesion.
2. Q: Guideline for inventing `IdGenerator`? → A: Pure Fabrication.

TL;DR

Work where the data is, containers create, thin controllers, polymorphism behind stable interfaces, fabricate when needed.

---

17. Domain-Driven Design (DDD) Essentials

What it is

Eric Evans's approach (*Domain-Driven Design*, 2003) for making code mirror the business (the **domain**). **Strategic design** splits a large business into **bounded contexts**, each with its own model and **ubiquitous language**; **tactical design** gives the building blocks inside one context: **entities**, **value objects**, **aggregates**, **domain events**, **repositories** (plus domain services). The other principles shape classes; DDD decides *which classes exist and where the boundaries are*. Analogy: "patient" means different things at a hospital's reception, operating theatre and billing — each department is a bounded context with its own file.

Term	Meaning
Ubiquitous language	One shared vocabulary for experts and code: `place()`, not `setStatus(2)`.
Bounded context	Boundary where one model and language hold (Sales "order" vs Shipping "consignment"); often one team/service.
Context map / ACL	How contexts relate; an **anti-corruption layer** translates another (or legacy) model into yours.
Entity	Defined by **identity** that survives changes; equality by id.
Value object	Defined by **values**; immutable; equality by value (`Money`, `Address`).
Aggregate / root	Cluster changed as one unit; only the **root** is reachable from outside and guards invariants.
Domain event	Past-tense business fact (`OrderPlaced`) others react to.
Repository	Collection-like load/save **per aggregate root** (Chapter 11, §11.7.3).

The problem it solves

One model for every department (big ball of mud); developer vs business vocabulary drift; **anemic models** (data bags + rules in services — Tell, Don't Ask violated at scale); no consistency boundary, so invariants are checked in some paths only; teams coupled through each other's tables.

How it works

Strategic: learn the language with domain experts → draw contexts where words change meaning (Catalog, Sales, Payments, Shipping) → choose relationships (published events/API, shared kernel sparingly, conformist, **ACL** for legacy/third party) → map contexts to teams and services (see the Strangler Fig notes for splitting a monolith).

Tactical: prefer **value objects** over primitives (immutable, self-validating); entities get an id value object and intention-revealing methods. **Aggregate rules** (Vaughn Vernon, 2013): protect true invariants inside; keep aggregates **small**; **reference other aggregates by id**; **one aggregate per transaction**, others follow via events (eventual consistency). Publish events **after commit** via a transactional outbox (see Saga, Event-Driven). Repositories only for roots. Application services load → call one method → save. Separate read models = **CQRS** (notes); events as the source of truth = Event Sourcing (§11.7.6).

Java example — before/after

Before: public fields, `double` total, string status — a line with qty −5 is added after "SHIPPED". After:

```java
// File: code/lld/ch05/ddd/Main.java
        record Money(long paise) {
            Money { if (paise < 0) throw new IllegalArgumentException("Money cannot be negative"); }
            // ... rupees(), plus(), times()
        }
        static final class Order {                              // aggregate root
            // ... id, customer (CustomerId, not Customer), lines, events, status
            void addItem(Sku sku, Money unitPrice, int quantity) {
                requireDraft("add items");
                // ... at most 10 units per SKU, then add an OrderLine entity
            }
            void place() {
                requireDraft("place");
                if (lines.isEmpty()) throw new IllegalStateException("Cannot place an empty order");
                status = OrderStatus.PLACED;
                // ... events.add(new OrderPlaced(id, customer, quantities, total()))
            }
            @Override public boolean equals(Object o) { return o instanceof Order other && other.id.equals(id); }
// entity: by identity
        }
        // ... InMemoryOrderRepository.save(order): store, then publish order.pullEvents()
        // ... ShippingTranslator (ACL): OrderPlaced -> Consignment(reference, parcels, weightGrams)
```

Full runnable code: code/lld/ch05/ddd/ (see the Full Edition).

```
total=0.30000000000000004 status=SHIPPED lines=3 (a line was added after shipping, with qty -5)
rejected: At most 10 units of TEA-250 per order
   shipping context: created Consignment[reference=SO-1001, parcels=1, weightGrams=920]
rejected: Cannot add items: order is PLACED
two Order objects with id 1001 equal? true   (entity: identity decides)
```

Trade-offs

Business-shaped code, clear team/service boundaries, clear transaction unit, testable rich models — versus expert time, more types, eventual consistency to design, translation at borders, ORM friction.

When not to use it

CRUD/admin apps, one small team with one small model, algorithm-heavy technical problems. Value objects and ubiquitous names are still worth borrowing.

Common mistakes

Anemic aggregates with setters; huge aggregates; one repository per table; publishing events before commit; one shared model across services; DDD as folder names; primitive obsession.

Real-world examples

Microservices split by bounded context (Sam Newman's *Building Microservices*; Microsoft's eShopOnContainers uses aggregates and domain events); `java.time` and `BigDecimal` as value objects; Axon aggregates (CQRS notes); in this book, the grocery POS (Ch 19) and shipping locker (§14.7).

In the interview

> **30-second answer:** "Bounded contexts with their own language, connected by events or APIs and translated at the border; inside, entities for identity, immutable value objects like Money, small aggregates whose root guards invariants, one aggregate per transaction, domain events for the rest, one repository per root." *Keep two aggregates consistent?* Event via outbox → handler or saga.

Self-check

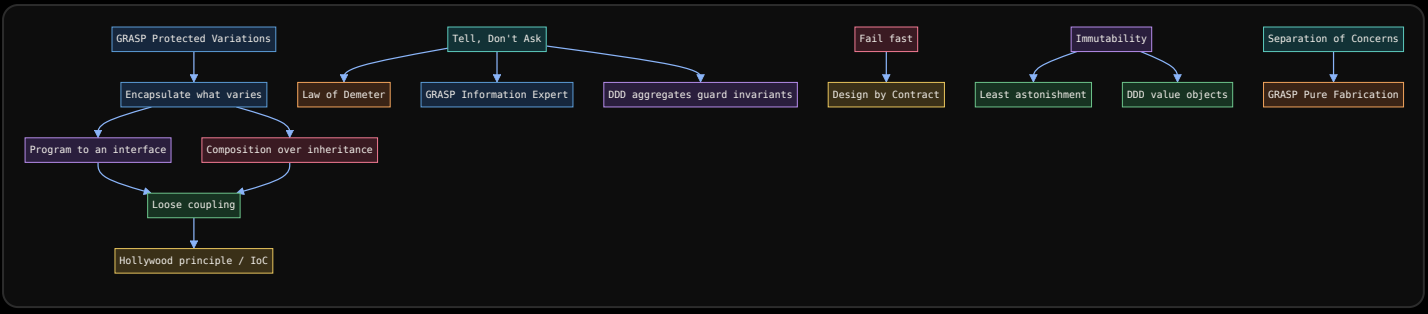
1. Q: Entity or value object? → A: Tracked over time by identity → entity; replaced when changed → value object.

2. Q: Why `CustomerId` inside `Order`? → A: Small, independent aggregates; no loading/locking the customer.
3. Q: ACL? → A: Translates a foreign model into ours at the border.

> **TL;DR**

Speak the business's language; bounded contexts with their own models; entities, value objects, small aggregates guarding invariants, domain events, repositories per root.

Chapter summary — how the principles fit together



Principle	Main goal	Smell it fixes	Key tool
DRY	One home per rule	Drifting duplicates	Value objects
KISS	Simplicity	Over-engineering	`switch` expressions
YAGNI	Only what's needed	Speculative generality	Delete unused code
Demeter	Hide structure	Train wrecks	Intent methods
Tell, Don't Ask	Behaviour with data	Anemic model	Command methods
Composition	Flexible reuse	Fragile base class	Forwarding
Program to interface	Substitutability	Hard-wired deps	Interfaces + DI
Encapsulate varies	Isolate change	Type switches	Strategy, lambdas
Loose coupling	Independent change	God orchestrators	Observer, events
Hollywood / IoC	Framework owns flow	Duplicated skeletons	Template Method
SoC	One concern per unit	Spaghetti	Layers, MVC
POLA	Predictability	Hidden side effects	Honest names, `Optional`
Fail fast	Early errors	Distant NPEs	Guard clauses
Immutability	Safe sharing	Aliasing, races	Records, `List.copyOf`
DbC	Clear obligations	Magic error values	Pre/postconditions
GRASP	Place responsibilities	God classes	Expert, Creator…
DDD	Model the business	Anemic model, big ball of mud	Value objects, aggregate roots, events

Tensions: DRY vs loose coupling; YAGNI vs encapsulate-what-varies/OCP; Demeter vs cohesion (Middle Man); Information Expert vs SoC (domain computes, fabrications persist); DRY vs bounded contexts (share events and ids, not models).

Chapter self-check

1. Q: `order.getCustomer().getAddress().getCity()` in a shipping service? → A: Demeter (and TDA); ask `order.shippingZone()`.
2. Q: A `find` that inserts a row? → A: Least Astonishment (and CQS).
3. Q: A record holding a list — truly immutable how? → A: `List.copyOf` in the compact constructor.
4. Q: DDD: why is `Money` a value object but `Order` an entity? → A: Money is defined by its values and replaced; an order keeps its identity while it changes.

TL;DR (chapter)

One home per rule, keep it simple, build only what's needed, let objects own behaviour, depend on small interfaces and compose, let frameworks call you, separate concerns, don't surprise, fail early, prefer immutable values, state contracts, use GRASP to decide who does what — and DDD to decide which classes exist.

Chapter 6 — Clean Code (Study Edition)

> **Study Edition.** Every topic of Chapter 6, without the padding. Code blocks are excerpts from the real files (`// ...` marks trimmed parts); full listings and outputs are in the Full Edition (`README.md`). Code lives in `code/lld/ch06/<topic>/` .

Design principles decide *which classes exist*; **clean code** is what happens *inside* them. In machine-coding rounds interviewers read your code line by line, so clean code often separates "hire" from "no hire". This chapter follows Robert C. Martin's *Clean Code* (2008) in order (the case studies are summarised in §13), then adds a **code review checklist**, an **error-handling strategy** and **testing strategies**.

Key terms. **Code smell** — a symptom that *usually* signals a deeper problem. **Refactoring** — changing structure, not behaviour (Chapter 7). **Level of abstraction** — `processPayroll()` high, `sb.append(',')` low. **Side effect** — any change besides the return value. **Test double** — a stand-in dependency (dummy, stub, spy, mock, fake). **TDD** — failing test, pass, refactor, in short cycles.

What is clean code, and why bother?

What it is

Code that reads like well-written prose, does one thing, has tests, has no duplication, expresses intent and looks written by someone who **cares**. Working definition: *intent obvious to the next reader, easy to change safely.* Analogy: an organised kitchen anyone can cook in.

The problem it solves

Mess slows teams more and more until productivity nears zero and a "grand redesign" reproduces the mess. We read code far more than we write it (often quoted >10:1). "Clean it later" rarely happens — *later equals never* (LeBlanc's law).

How it works

Treat readability as a requirement; follow the **Boy Scout Rule** — *leave the code a little cleaner than you found it*; write tests so you can clean without fear; make it work, then make it right, in small steps.

Trade-offs

Slightly slower the first time; faster on every later read and change. Throwaway scripts differ.

When not to use it

Unreadable code is never a goal, but avoid **gold-plating** clear, stable code.

Common mistakes

"Working" = "done"; blaming deadlines (the mess slows you immediately); big-bang rewrites instead of continuous small improvements.

Real-world examples

Google's readability reviews and certification; linters/formatters in CI (Checkstyle, SpotBugs, SonarQube, google-java-format).

In the interview

> **30-second answer:** "Obvious intent, cheap to change: intention-revealing names, small functions, exceptions not codes, immutable values, a few tests — and time to refactor once it works."

Self-check

Q: Boy Scout Rule? → A: Leave the code a little cleaner than you found it.

TL;DR

Obvious intent + safe to change; clean a little every time.

---

1. Meaningful Names

What it is

A good name answers **why it exists, what it does, how it's used**. If it needs a comment, it isn't good enough. Analogy: "Kitchen — glasses — fragile" vs "Box 7".

The problem it solves

Bad names force a **mental map** ("`d` is elapsed days, 4 means done") that every reader pays for, and misreadings cause bugs.

How it works — the rules

Rule	Bad	Good
Reveal intention	`int d;`	`int elapsedTimeInDays;`
Avoid disinformation	`accountList` for a `Set`; `l` / `O`	`accounts`
Meaningful distinctions (no number series, noise words)	`a1, a2`; `ProductInfo` vs `ProductData`	`source, destination`
Pronounceable	`crtdTs`	`createdAt`
Searchable	`7`, `e`	`MAX_LOGIN_ATTEMPTS`
No encodings (Hungarian, `m_`, `I` prefix)	`strName`, `IShapeFactory`	`name`, `ShapeFactory`
No mental mapping	`r`	`normalizedPath`
Classes = nouns	`Manager`, `Data`, `Info`	`Customer`, `AddressParser`
Methods = verbs; `get`/`is`/`has`	`data()`	`postPayment()`, `isPosted()`
Named static factories	`new Money(23)`	`Money.ofRupees(23)`
Don't be cute	`nuke()`	`deleteAll()`
One word per concept	`fetch` / `retrieve` / `get`	pick one
Don't pun	`add` = sum and append	`append`, `insert`
Solution-domain names (technical)	—	`OrderRepository`, `RetryPolicy`
Problem-domain names (business)	—	`Premium`, `Claim`
Add meaningful context	lone `state`	`shippingState` or an `Address`
No gratuitous context	`AcmeShopCustomerAddress`	`Address`

The longer the scope, the longer the name — `i` suits a 3-line loop.

Java example — before/after

Before: `class DtaRcrd102 { String strNm; int i1; int i2; boolean flg; }` and `Mgr.getThem()` filtering `x.i1 == 4 && x.i2 > 18000 && !x.flg`. After:

```java
// File: code/lld/ch06/names/Main.java
        record Task(String title, TaskStatus status, Duration timeSpent, boolean archived) {
            boolean isCompleted() { return status == TaskStatus.DONE; }          // boolean method reads as a question
            boolean tookLongerThan(Duration limit) { return timeSpent.compareTo(limit) > 0; }
        }
        // ...
            private static final Duration LONG_RUNNING = Duration.ofHours(5);  // named constant, not a magic number
            // ...
            List<Task> findCompletedLongRunningTasks() {                       // says exactly what it returns
```

Full runnable code: code/lld/ch06/names/ (see the Full Edition). The rule now reads aloud; a status code became an enum; `Temperature.ofCelsius(25)` is unambiguous.

Trade-offs

A few keystrokes (IDEs autocomplete). A very long name may mean the function does too much — split, don't abbreviate.

When not to use it

Conventional short names: `i`/`j` in tiny loops, `e` for exceptions, `x`/`y`, `T`/`K`/`V`.

Common mistakes

Not renaming ("everyone knows `tmp2`"); author-only abbreviations (`calcAmtPrc`); `customer`/`client`/`user` for one concept.

Real-world examples

JDK: `isEmpty()`, `computeIfAbsent()`, `List.of()`, `Duration.ofMinutes()`; Google Java Style casing.

In the interview

> **30-second answer:** "Code should read like the requirement: nouns for classes, verbs for methods, booleans as questions, named constants, enums instead of status codes, one word per concept." *`I` prefix on interfaces?* Not in Java; name the role (`PaymentGateway`), implementations specifically (`RazorpayGateway`).

Self-check

1. Q: `ProductInfo` and `ProductData` together? → A: Noise words, no real distinction.
2. Q: A naming pun? → A: One word for two meanings.

TL;DR

Reveal intent, pronounceable, searchable, one word per concept.

---

2. Functions

What it is

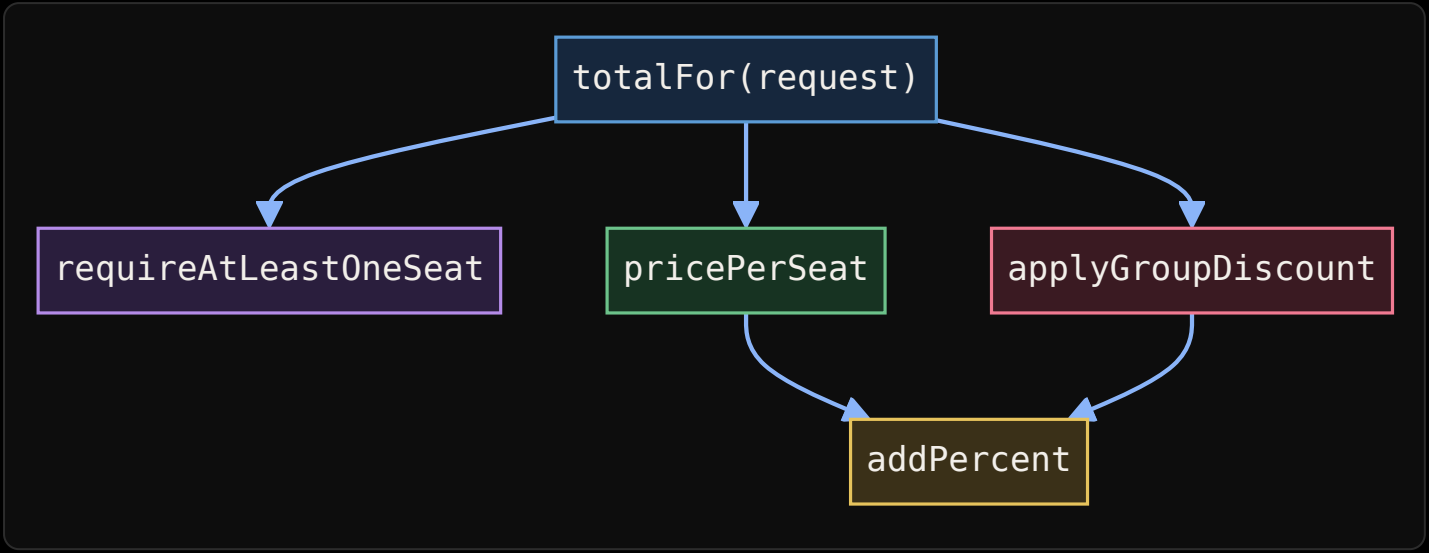
Clean functions are **small**, **do one thing**, stay at **one level of abstraction**, take **few arguments**, have **no hidden side effects**, and either **do** or **answer** — never both. Analogy: a recipe of named steps, each with its own sub-recipe.

The problem it solves

Long functions mix detail levels, hide bugs, can't be named, reused or tested in pieces; flags and output arguments confuse; hidden side effects surprise; error codes clutter callers.

How it works — the rules

1. **Small, then smaller** — rarely 20 lines; block bodies ideally one call; indentation ≤ 1–2 levels.
2. **Do one thing** — nothing meaningful left to extract; "sections" mean several things.
3. **One level of abstraction** per function. 4. **Stepdown Rule** — read top-down, each function followed by its callees.
4. **Switch statements** — only **once**, buried low (a factory creating polymorphic objects); else polymorphism or enum data.
5. **Descriptive names** beat enigmatic ones and comments.
6. **Few arguments** — zero, one, two; avoid three; more needs justification. One-arg forms: question, transform, event. **Flag arguments** mean two things — split or use an enum. **Argument objects** (`book(BookingRequest)`); varargs count as one; **verbs and keywords** (`copyFromTo(source, target)`).
7. **No side effects** — hidden ones create **temporal coupling**. 9. **No output arguments** — `letter.addSignature()`.
8. **Command-Query Separation** — change state *or* answer (`if (enable("darkMode"))` is ambiguous).
9. **Exceptions, not error codes**. 12. **Extract try/catch** — error handling is one thing. 13. **No "error code magnet"** enum.
10. **DRY**. 15. **Structured programming** matters for big functions; small ones may return early (never `goto`).
11. **How to get there** — rough draft with tests, then refine, keeping tests green.



Java example — before/after

Before: `static int book(String seatType, int count, boolean weekend, List<String> log)` — returns `-1` on error, switches on seat strings, mutates the caller's `log`, writes a global `SESSION`. After:

```java
// File: code/lld/ch06/functions/Main.java
        /** Argument object: three related arguments become one concept. */
        record BookingRequest(SeatType seat, int seats, DayType day) {}
        // ...
            int totalFor(BookingRequest request) {                      // level 1: the story
                requireAtLeastOneSeat(request);
                int total = pricePerSeat(request) * request.seats();
                return applyGroupDiscount(total, request.seats());
            }
        // ...
            void book(BookingRequest request) {
                // ... price it and append to history (a command: returns nothing)
            }
            List<String> history() { return List.copyOf(history); }
```

Full runnable code: code/lld/ch06/functions/ (see the Full Edition).

Flag → `DayType`; type switch → `SeatType` enum carrying its price, parsed once at the boundary; `-1` → `InvalidBookingException` with a message; output argument and `SESSION` → state owned by `BookingService`; **CQS**: `book` is a void command, `history` a query (returning a created ID is an accepted exception); `bookFromForm` does only error translation.

Trade-offs

Many small functions	Few large functions
Nameable, testable, reusable	Everything in one place
Need good names; over-splitting scatters logic	Hard to understand and test; mixed levels

When not to use it

Don't split a clear 8-line function into 8 one-liners; size is rarely a performance excuse (the JIT inlines small methods).

Common mistakes

Boolean parameters in public APIs; state-changing getters; catching deep and returning `null`; names that say one thing while the body does three.

Real-world examples

`list.sort(comparator)` returns `void`; `Iterator.next()` / `Queue.poll()` are known CQS exceptions; `Files.readString`.

In the interview

> **30-second answer:** "Small, single-purpose functions at one level, stepdown order; enums instead of flags, parameter objects, CQS, and descriptive exceptions instead of error codes." *How small?* One clear name and level — often 3–10 lines, rarely over ~20.

Self-check

1. Q: When is a function "one thing"? → A: No meaningful function can be extracted from it.
2. Q: When is a `switch` acceptable? → A: Once, buried in a factory, never repeated.

TL;DR

Small, one thing, one level, top-down, ≤2–3 args, no flags, CQS, exceptions not codes.

---

3. Comments

What it is

At best a necessary evil — compensation for failing to express ourselves in code. Code changes; comments drift and **lie**. **Explain yourself in code first**. Analogy: a "Bridge closed" sign long after it reopened.

The problem it solves

Comments as a crutch for unclear code, redundant comments, outdated misleading ones.

How it works

`// free shipping if big and domestic` + `if (order.total() >= 499 && !order.isInternational())` becomes `if (order.qualifiesForFreeShipping())`.

Good comments: legal (licence header); informative (what a regex matches); explanation of intent (*why*); clarification (obscure library values); warning of consequences ("not thread-safe", "billed per call");

TODO (with an owner, reviewed); amplification ("the trim is critical"); Javadoc on public APIs.

Bad comments: mumbling; redundant (`i++; // increment i`); misleading; mandated (`@param a the a`); journal; noise; scary noise; comment instead of a function/variable; position markers; closing-brace comments; attributions; **commented-out code** (delete — git remembers); HTML; nonlocal information; too much information; inobvious connection; function headers; Javadoc on non-public code.

Java example — before/after

Before: journal comment, `@param a the a`, a comment instead of a name over a five-part condition, `return true; // return true`, commented-out code, `} // end eligible`. After:

```
// File: code/lld/ch06/comments/Main.java
        static boolean isEligibleForPremiumLoan(Applicant a) {
            return isWorkingAge(a) && hasGoodCredit(a) && earnsEnough(a);
        }
        // ...
        // GOOD: informative comment -- clarifies what a regex matches (format: 10-character PAN, e.g. ABCDE1234F)
        private static final Pattern PAN_FORMAT = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");
```

Full runnable code: code/lld/ch06/comments/ (see the Full Edition). The kept comments are the good kinds: legal header, informative, intent, public Javadoc, a billing warning and an owned `TODO(loans-team)`.

Trade-offs

Cheap to write, expensive to maintain: a few precise "why" comments help; many "what" comments hurt.

When not to use it

"No comments" isn't the goal: public libraries need Javadoc; complex algorithms deserve an explanation of approach and why.

Common mistakes

Updating code but not its comment; keeping commented-out code; TODO lists nobody reviews.

Real-world examples

JDK Javadoc (`HashMap` iteration order, `ConcurrentHashMap` weakly consistent iterators); SonarQube flags commented-out code; linters reject TODOs without tickets.

In the interview

> **30-second answer:** "Expressive code over comments — named functions and constants replace most 'what' comments. I keep why, warnings, legal, owned TODOs and public contracts, and delete commented-out code."

Self-check

1. Q: Commented-out code? → A: Delete it.
2. Q: Why do comments lie? → A: Code changes, comments don't.

TL;DR

Say it in code; comment only why, warnings, legal and public contracts.

---

4. Formatting

What it is

Layout communicates what belongs together and what matters. One team style, applied automatically.

The problem it solves

Random layout makes readers decode structure before logic, and produces noisy diffs.

How it works

Vertical: small files (many Java projects ~200 lines, few >500); **newspaper metaphor**; **openness** (blank lines between concepts); **density** (related lines together); **distance** (locals near first use, instance variables at the top, **caller above callee**, conceptual affinity); **ordering** high-level first. **Horizontal:** short lines (Martin 120, Google 100); spaces around assignments, none between name and `(`; **no column alignment**; indentation shows scope; make **dummy scopes** (empty loop bodies) visible. **Team rules:** one style, enforced by a formatter in CI.

Java example — before/after

Before, `Playlist` is crammed onto a few lines with fields scattered and aligned in columns. After (newspaper order):

```java
// File: code/lld/ch06/formatting/Main.java
            // 1. Fields at the top, together (vertical density for closely related lines).
            // ...
            public String next() {
                if (songs.isEmpty()) {
                    return null;
                }
                String song = displayTitle(cursor);    // caller ABOVE callee (dependent functions close together)
                cursor = (cursor + 1) % songs.size();
                return song;
            }

            // 4. Private helpers just below the functions that use them.
            private String displayTitle(int index) {
```

Full runnable code: code/lld/ch06/formatting/ (see the Full Edition). Behaviour is identical; only reading speed changes.

Trade-offs

Almost none once automated; agree the rules once.

When not to use it

Generated code (e.g. Protobuf) — exclude it from formatters.

Common mistakes

Mixing formatting and logic changes in one commit; personal styles; column alignment.

Real-world examples

google-java-format, Spotless, Prettier, gofmt, Black; EditorConfig.

In the interview

> **30-second answer:** "Related lines together, concepts separated, callers above callees, fields at the top, one automated team style."

Self-check

Q: Where does a private helper go? → A: Just below its caller.

TL;DR

Headline first, related things close, one automated style.

---

5. Objects and Data Structures

What it is

An **object** hides data and exposes behaviour (`Order.isMetroDelivery()`); a **data structure** exposes data with no meaningful behaviour (record `Address`, DTO). **Data abstraction** isn't getters/setters on private fields — it's exposing the *essence* (`fuelPercentRemaining()`, not tank capacity and litres).

The problem it solves

Hybrids (public data *and* behaviour) make both new functions and new types hard. Knowing the difference tells you which change is easy.

How it works — the data/object anti-symmetry

	Procedural (data structures + functions)	OO (objects)
Add a **function** (`perimeter`)	**Easy** — one function	**Hard** — every class
Add a **type** (`Triangle`)	**Hard** — every function	**Easy** — one class

"Everything is an object" is a myth: expect new types → objects; new operations → data + functions (or Visitor). Java 21 `sealed` + records + pattern-matching `switch` make the procedural style compiler-checked. **Demeter** applies to objects (no `order.getCustomer().getAddress().city()`), not strictly to data structures. **DTOs** carry pure data across boundaries (bean form is framework ceremony, not encapsulation). **Active Record** — a DTO with `save()`/`find()`; keep business rules out.

Java example — before/after

```java
// File: code/lld/ch06/objects/Main.java
            static double perimeter(Shape s) {                // NEW FUNCTION: easy -- no shape class changes
                return switch (s) {
                    case Square sq -> 4 * sq.side();
                    case Circle c -> 2 * Math.PI * c.radius();
                };
            }
        // ...
        record Triangle(double base, double height) implements Shape {   // NEW TYPE: easy -- nothing else changes
        // ...
            boolean isMetroDelivery() { return shipTo.isMetro(); }        // tell the order what you need
```

Full runnable code: code/lld/ch06/objects/ (see the Full Edition). The train wreck became `order.isMetroDelivery()` while the `Address` record freely exposes `city`; `OrderDto` is a pure data carrier.

Trade-offs

See the anti-symmetry table: choose by which change you expect more.

When not to use it

Don't push behaviour into DTOs crossing process boundaries — keep them dumb and versioned.

Common mistakes

Reflexive getters/setters; hybrids; business logic in JPA entities that are also API bodies.

Real-world examples

Records; `java.awt.Point` (public fields); `LocalDate` (object); Spring `@RequestBody` DTOs; ActiveRecord / JPA entities.

In the interview

> **30-second answer:** "Objects hide data and expose behaviour; data structures expose data. OO makes new types easy, procedural makes new functions easy. Records for DTOs, rich objects for domain logic, no hybrids or train wrecks." *Pattern for adding operations in OO?* Visitor.

Self-check

1. Q: Which style makes new shape types easy? → A: OO.
2. Q: A hybrid? → A: Exposed data plus significant behaviour.

TL;DR

Pick objects or data structures by the change you expect; no hybrids.

---

6. Error Handling

What it is

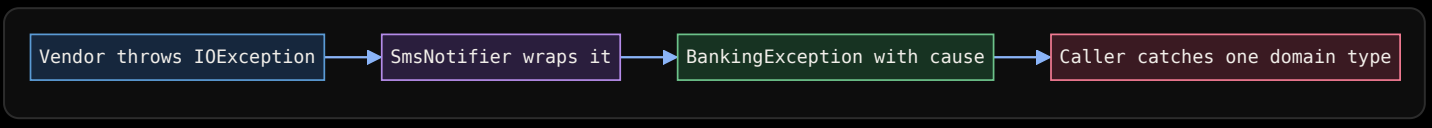
Important — but if it **obscures the logic** it's wrong. Keep the happy path readable; make failures explicit, informative and hard to ignore.

The problem it solves

Error codes clutter callers and get forgotten; `null` returns become distant NPEs; checked exceptions leak into every signature; vague messages slow debugging.

How it works — the rules

1. **Exceptions, not codes.** 2. **Write the** `try-catch-finally` **first** (a transaction); prefer **try-with-resources**.
2. **Unchecked exceptions** — checked ones break OCP by rippling through signatures; checked only when callers must recover.
3. **Context** — what, which values, why; keep the **cause**. 5. **Define exceptions by the caller's needs**; **wrap third-party APIs**.
4. **Special Case pattern** (Fowler) — an object for expected absence (`NoMembership`). 7. **Don't return** `null`. 8. **Don't pass** `null`.



Java example — before/after

Before: `transfer` returns `E_NO_ACCOUNT` (which account?), `history` returns `null`, `loyaltyPoints` returns a `null` `Integer` that crashes on unboxing. After:

```java
// File: code/lld/ch06/errors/Main.java
        static final class InsufficientFundsException extends BankingException {
            InsufficientFundsException(String id, long requested, long available) {
                super("account '" + id + "' has " + available + " but " + requested + " was requested");
            }
        }
        // ...
        enum NoMembership implements Loyalty { INSTANCE; public int points() { return 0; } public String tier() {
return "NONE"; } }
        // ...
            void send(String phone, String text) {
                try { client.push(phone, text); }
                catch (IOException e) { throw new BankingException("could not send SMS to " + phone, e); }
            }
        // ...
            List<String> history(String id) { return List.of(); }                          // empty, never null
            Optional<Long> findBalance(String id) { return Optional.ofNullable(balances.get(id)); }
```

Full runnable code: code/lld/ch06/errors/ (see the Full Edition).

Output includes `account 'B' has 100 but 900 was requested` and `could not send SMS to 000123 (cause: socket timeout to sms-gw-3)`. An unchecked `BankingException` hierarchy with context; the vendor's checked `IOException` wrapped with its cause; a try-with-resources `AuditFile` is closed even when an exception is thrown.

Trade-offs

Choice	Pro	Con
Unchecked exceptions	Clean signatures, OCP-friendly	No compiler reminder; document them
Checked exceptions	Compiler-enforced	Signature ripple; often swallowed
`Optional` return	Absence in the type	Not for fields/parameters
Special Case	No checks in callers	Can hide absence that *should* be noticed
Result/Either	Expected failures as values	Verbose in Java (§16)

When not to use it

Exceptions aren't control flow (don't throw to exit a loop); expected, frequent outcomes (row validation) may be cleaner as result values (§16).

Common mistakes

`catch (Exception e) {}`; log-and-rethrow at every layer; losing the cause (`new MyException(e.getMessage())`); `null` from collection methods.

Real-world examples

Spring `DataAccessException` translating `SQLException`s (e.g. `DuplicateKeyException`); `Collections.emptyList()`, `Optional.empty()`; `requireNonNull`; `UncheckedIOException`.

In the interview

> **30-second answer:** "Unchecked domain exceptions with context, not codes or null; wrap vendor exceptions; empty collections, `Optional` or special cases for absence; reject nulls at entry; try-with-resources." *Checked?* Only for conditions callers must recover from.

Self-check

1. Q: Why are error codes bad? → A: Clutter, easily ignored, no context.
2. Q: Why keep the cause? → A: The original stack trace and message.

TL;DR

Exceptions not codes; unchecked with context; wrap vendors; no null in or out; always clean up.

---

7. Boundaries

What it is

Where your code meets code you don't control (libraries, frameworks, other teams, even general JDK types like `Map`). Keep it at the edge behind interfaces shaped for *your* needs.

The problem it solves

Providers want broad APIs; users want focused ones. A raw `Map<String, Session>` passed around lets anyone `clear()` it and forces everyone to know the key format; library changes break every usage site.

How it works

1. **Wrap boundary types** — expose only what the app needs (`SessionRegistry.open/activeSession/close`); keep the boundary type private.
2. **Learning tests** — small tests of the library as you'll use it; free documentation, re-run on upgrades.
3. **Use code that doesn't exist yet** — define the interface you wish you had (`FraudCheck`), use a fake, later write an **Adapter**.
4. Keep boundary knowledge in few places.

Java example — before/after

Before, an admin tool calls `sessions.clear()` on the shared map. After:

```
// File: code/lld/ch06/boundaries/Main.java
        static final class SessionRegistry {
            private final Map<String, Session> byToken = new ConcurrentHashMap<>();   // boundary type stays
private

            void open(String token, Session session) { byToken.put(token, session); }
            Optional<Session> activeSession(String token, Instant now) {
                return Optional.ofNullable(byToken.get(token)).filter(s -> now.isBefore(s.expiresAt()));
            }
            void close(String token) { byToken.remove(token); }
        // ...
        interface FraudCheck { boolean isSuspicious(String userId, long amountPaise); }
```

Full runnable code: code/lld/ch06/boundaries/ (see the Full Edition). `JdkLearningTest` pins real facts: `HashMap` allows one `null` key, `ConcurrentHashMap` rejects it, `Map.of` rejects duplicates, `"a,b,,".split(",")` drops trailing empties (use limit `-1`).

Trade-offs

A class and some delegation — worth it once the boundary type is used in more than one place or may change.

When not to use it

A `Map` used inside one method is fine; the rule is about *passing* boundary types around.

Common mistakes

Returning vendor types (`StripeCharge` from `PaymentGateway`); discovering library behaviour in production.

Real-world examples

DDD **Anti-Corruption Layer**; SLF4J; Spring `RestClient`/`WebClient`; Pact contract tests.

In the interview

> **30-second answer:** "Third-party code stays at the edge behind my interfaces; learning tests when adopting a library; for a missing API, my own interface, a fake, and an adapter later."

Self-check

Q: A learning test? → A: A test that pins down how a third-party API behaves.

TL;DR

Foreign code at the edge behind your interfaces; learn it with tests.

---

8. Unit Tests

What it is

Small automated tests of one unit of behaviour in isolation. Test code is **as important as production code** — it's what lets you change production code without fear.

The problem it solves

No tests → fear of change → rot. *Dirty* tests → break constantly, get abandoned → rot anyway.

How it works

Three laws of TDD (a ~30-second cycle): (1) no production code without a failing test; (2) write only enough test to fail (not compiling counts); (3) write only enough code to pass. Beck's coarser **Red → Green → Refactor**. **Clean tests** enable the "-ilities". Readability: **Build–Operate–Check** (Arrange–Act–Assert, Given–When–Then); a **domain-specific testing language** (`cart("TEA", 3)`); a **dual standard** (clean, not necessarily efficient). **One concept per test** (several asserts are fine for one concept).

F.I.R.S.T.	Meaning	In our code
Fast	Milliseconds	In-memory doubles
Independent	No order dependence	Fresh instance per test
Repeatable	Same result anywhere	Fixed `Clock`
Self-validating	Pass/fail automatically	Assertions, not `println`
Timely	Just before the code	TDD

Java example — a JUnit-free runner, clean tests and a dirty one

`MiniTest` (~60 lines) finds `@Test` methods by reflection, runs each on a **new instance** and reports PASS/FAIL. `Checkout` has four collaborators behind interfaces (`PriceCatalog`, `CouponRepository`, `ReceiptSender`, `AuditLog`) plus a `Clock`.

```
// File: code/lld/ch06/testing/CheckoutTest.java
    private static final Clock FIXED_CLOCK = Clock.fixed(Instant.parse("2026-09-28T10:00:00Z"), ZoneOffset.UTC);
// Repeatable
    // ...
    @Test void expired_coupon_is_ignored() {
        var coupons = new FakeCoupons().add(new Checkout.Coupon("OLD", 50, TODAY.minusDays(1)));
        long total = checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 1), "OLD");
        assertEquals(12_000L, total);
    }
```

Full runnable code: code/lld/ch06/testing/ (`Checkout`, `CheckoutTest`, `CheckoutIntegrationTest`, `BeforeStyleTest`, `MiniTest`, `Test`, `Main`) (see the Full Edition).

`test1` "passes" but proves nothing: three concepts, real clock, printing, meaningless name. `CheckoutTest` methods test one concept each, are named as specifications, use Given/When/Then and a fixed clock; `CheckoutIntegrationTest` wires real in-memory components. Output: `TOTAL: 8 passed, 0 failed`.

Trade-offs

Well-tested: fearless refactoring, executable documentation, but test code to maintain. Untested: faster first write, fear of change and knowledge in heads.

When not to use it

TDD suits spikes and pure UI layout less; don't unit-test trivial getters or code you don't own (learning tests instead).

Common mistakes

Testing implementation details; shared mutable fixtures; `Thread.sleep` or real clock/network; names like `test1`.

Real-world examples

JUnit 5 (new instance per test method by default), AssertJ, Mockito; `java.time.Clock` with `Clock.fixed`.

In the interview

> **30-second answer:** "Small, fast, independent tests — one concept each, Given/When/Then, doubles behind interfaces, an injected `Clock`. In a machine-coding round at least a `main` exercising key flows plus a few unit tests."

Self-check

1. Q: What's wrong with printing in a test? → A: Not self-validating.
2. Q: Repeatable time-based tests? → A: Inject a `Clock`; `Clock.fixed`.

TL;DR

Tests are first-class: TDD, one concept per test, Given/When/Then, F.I.R.S.T.

---

9. Classes

What it is

Standard **organisation**; **small** (by responsibilities); **one reason to change**; **cohesive**; **organised for change** so new features are new classes.

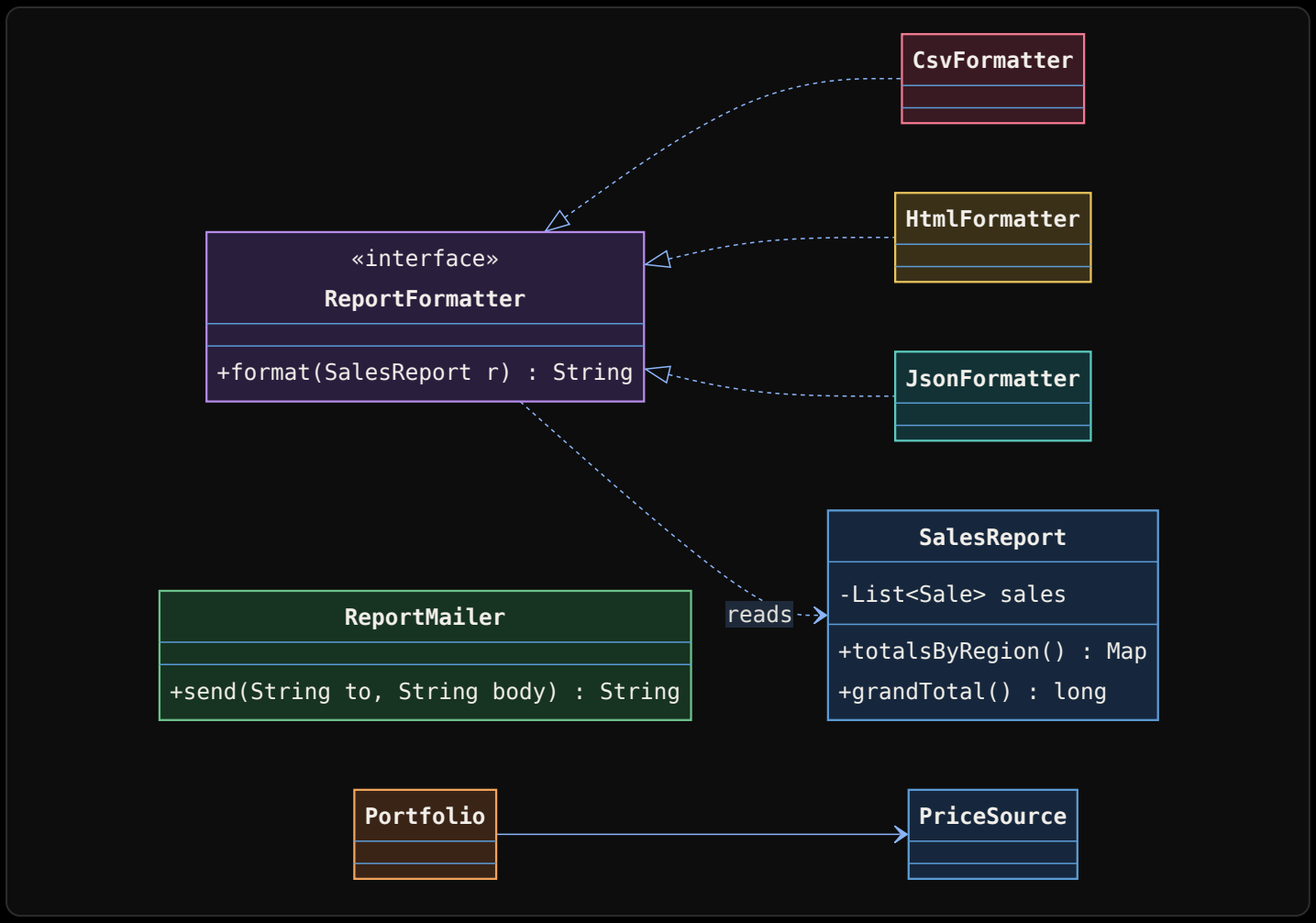
The problem it solves

God classes change for many reasons; growing `if/else` over formats reopens the same class repeatedly.

How it works

1. **Organisation**: public static constants → private static variables → private instance variables → public functions → private helpers after their caller.
2. **Encapsulation**: loosen for tests only as a last resort.
3. **Small** by responsibilities: if the name needs *Manager/Processor/Super* or "and/or/but", it does too much; describable in ~25 words.
4. **SRP**: many small classes beat a few large ones — same code, better organised.
5. **Cohesion**: maximal when every method uses every field; a subset of methods using a subset of fields wants its own class.
6. **Organise for change**: new features as new classes (OCP) — one `ReportFormatter` per format.

7. **Isolate from change**: depend on abstractions (`Portfolio` → `PriceSource`, DIP).



Java example — before/after

Before, `ReportManager` totals, renders CSV/HTML in a growing `if/else` and emails — three reasons to change. After:

```java
// File: code/lld/ch06/classes/Main.java
        /** Organised for change: a new format is a NEW class; existing formatters are never reopened (OCP). */
        interface ReportFormatter { String format(SalesReport report); }
        // ...
        static final class JsonFormatter implements ReportFormatter {   // added later, nothing else edited
        // ...
        /** Isolating from change: Portfolio depends on an abstraction, not on a live stock-exchange client. */
        interface PriceSource { long pricePaise(String symbol); }
```

Full runnable code: code/lld/ch06/classes/ (see the Full Edition). `JsonFormatter` was added with no edits; `Portfolio` is tested with fixed prices.

Trade-offs

Many small classes: easy to understand and test, localised changes, need good names and packages. Few big: fewer files, every change touches the same class, merge conflicts.

When not to use it

Don't fragment a small cohesive class into micro-classes that always change together.

Common mistakes

`*Manager` / `*Helper` / `*Utils`; splitting only by technical layer; everything `public` for tests.

Real-world examples

`java.time` (`LocalDate`, `Duration`, `DateTimeFormatter`…) vs old `Date`/`Calendar`; `JdbcTemplate` delegating to `RowMapper`.

In the interview

> **30-second answer:** "Small by responsibility, highly cohesive; when a feature would add a branch I split so it's a new class; interfaces isolate volatile details."

Self-check

1. Q: How is class size measured? → A: Responsibilities.
2. Q: Maximal cohesion? → A: Every method uses every field.

TL;DR

Small, cohesive, open for extension, depending on abstractions.

---

10. Systems

What it is

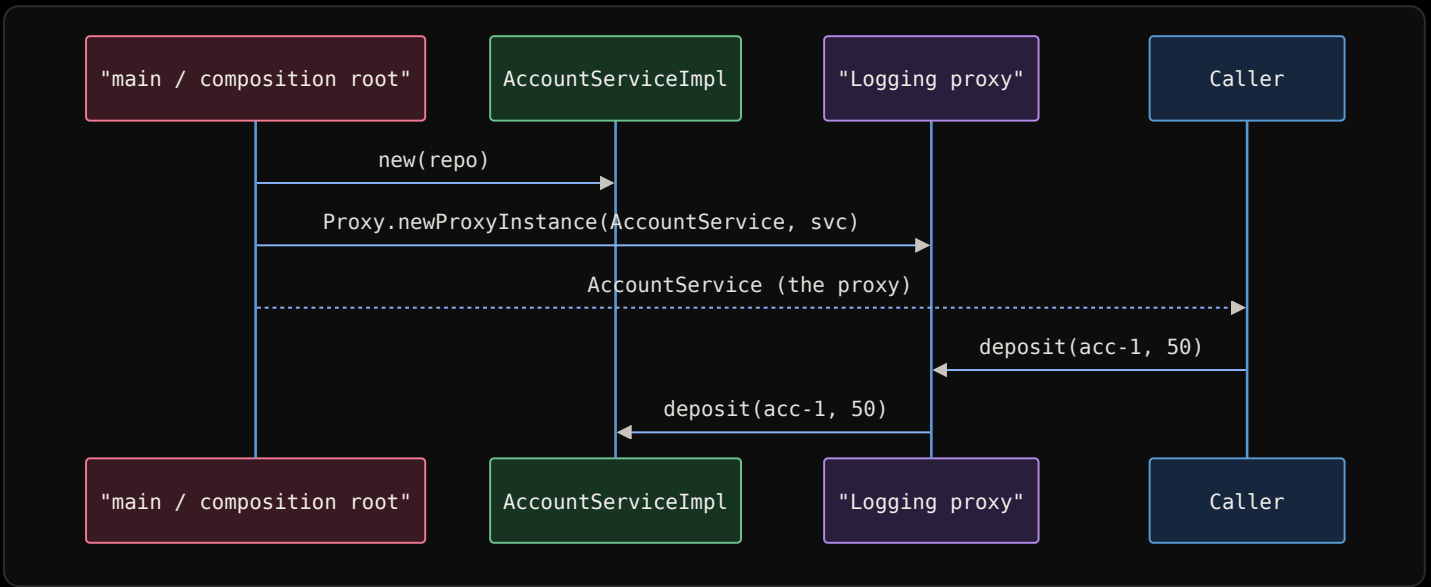
Separate startup (construction and wiring) from runtime logic, use **Dependency Injection**, handle **cross-cutting concerns** in one place, grow the architecture **incrementally**.

The problem it solves

Lazy init in business code (`if (service == null) service = new MyServiceImpl()`) hard-wires classes and blocks testing; logging copied into every method is noisy.

How it works

1. **Separate construction from use**: the **composition root** in `main` builds everything; **Abstract Factory** when the app decides *when* to create; **DI** (IoC for dependencies; constructor injection preferred).
2. **Scale up incrementally**. 3. **Cross-cutting concerns** via **dynamic proxies**, **AOP** (Spring `@Transactional`, `@Cacheable`), **AspectJ**, decorators, filters.
3. **Test-drive the architecture** with POJOs; 5. **postpone decisions** to the last responsible moment; 6. **use standards wisely**; 7. **DSLs** make code read like the domain.



Java example — before/after

Before, `AccountServiceImpl` lazily builds its repository and repeats `LOG enter/exit` in every method. After:

```
// File: code/lld/ch06/systems/Main.java
            private final AccountRepository repo;
            AccountServiceImpl(AccountRepository repo) { this.repo = repo; }
        // ...
        static <T> T withLogging(Class<T> type, T target) {
            // ... an InvocationHandler logs, calls method.invoke(target, args), and on failure
            // rethrows e.getCause() (the real exception, not the reflection wrapper)
            return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type}, handler);
        }

        /** Composition root: the ONLY place that knows concrete classes and wires them together (the "main"
side). */
        static AccountService buildApplication() {
            AccountRepository repo = new InMemoryAccountRepository();
            AccountService core = new AccountServiceImpl(repo);
            return withLogging(AccountService.class, core);
        }
```

Full runnable code: code/lld/ch06/systems/ (see the Full Edition). Logging is written once; tests wire the core with another repository.

Trade-offs

DI + composition root + proxies	Self-constructing objects
Testable, swappable, one wiring place; concerns in one place	Simple for tiny programs
Framework "magic"; proxies only intercept calls *through* the proxy	Duplicated, inconsistent concerns

Spring gotcha: self-invocation of a `@Transactional` method bypasses the proxy — no transaction.

When not to use it

Small programs and interviews need no DI framework — manual constructor injection in `main` is expected.

Common mistakes

Service Locator calls in business code; `@Autowired` field injection (hidden, blocks immutability); framework annotations everywhere.

Real-world examples

Spring/Spring Boot, Guice, Dagger, Jakarta CDI; Spring AOP via JDK proxies or CGLIB; servlet filters, `HandlerInterceptor`.

In the interview

> **30-second answer:** "A composition root in `main` builds and wires everything; classes get dependencies through constructors; logging and transactions live once in a decorator, proxy or aspect." *DI vs Service Locator?* DI pushes (explicit); a locator pulls (hidden).

Self-check

1. Q: Composition root? → A: The one place objects are constructed and wired.
2. Q: Three ways to handle cross-cutting concerns? → A: Dynamic proxies, AOP/AspectJ, decorators (also filters).

TL;DR

Build in one place and inject; cross-cutting concerns in proxies/aspects; grow incrementally.

11. Emergence — Kent Beck's Four Rules of Simple Design

What it is

Good design **emerges** from four rules. *Clean Code* (Jeff Langr's chapter) orders them: **1. runs all the tests, 2. no duplication, 3. expresses intent, 4. minimises classes and methods. Beck's own order** (per Fowler's "BeckDesignRules"): **passes the tests, reveals intention, no duplication, fewest elements** — the middle two get swapped and reinforce each other. In interviews, quote Beck's order and mention the swap.

The problem it solves

An ordered checklist that pushes code toward SRP, DIP and good structure.

How it works

1. **Runs all the tests** — testability itself drives small, decoupled classes (DI, interfaces).
2. **Refactor with rules 2–4**: remove duplicate lines and implementations (`isEmpty()` as `size() == 0`), higher-level duplication via **Template Method**; be **expressive** (names, small units, pattern names, tests as docs, *care*); **minimise** classes and methods — lowest priority, loses to SRP and no-duplication.

Java example — before/after

Before, `calcIN` and `calcUS` copy one algorithm with magic `18`/`45` and `15`/`30`, plus a purposeless `LeaveCalcStrategyProvider`. After:

```
// File: code/lld/ch06/emergence/Main.java
        abstract static class LeavePolicy {
            /** The shared algorithm, written once (rule 2: no duplication). */
            final int earnedLeaveDays(Employee e) {
                if (e.onProbation()) return 0;
                return Math.min(daysPerYear() * e.monthsWorked() / 12, maxCarryForwardDays());
            }
            protected abstract int daysPerYear();              // what varies, with intention-revealing names
(rule 3)
            protected abstract int maxCarryForwardDays();
        }
```

Full runnable code: code/lld/ch06/emergence/ (see the Full Edition). `LeavePolicyTest` proves old and new behaviour match (rule 1); the useless interface is gone (rule 4).

Trade-offs

Rules 2–4 can conflict; resolve by priority or find a design satisfying both.

When not to use it

Always applicable — a mindset, not a process.

Common mistakes

Refactoring without tests; minimising classes at SRP's expense.

Real-world examples

XP teams; `AbstractList` removing duplication like our Template Method.

In the interview

> **30-second answer:** "Beck's rules in priority order: passes the tests, reveals intention, no duplication, fewest elements (*Clean Code* swaps the middle two). Get it working with a quick test, then refactor duplication and names, and add no class that doesn't earn its place."

Self-check

Q: Lowest-priority rule? → A: Minimal classes and methods.

TL;DR

Tests pass → no duplication → clear intent → nothing extra.

---

12. Concurrency

What it is

A **decoupling strategy** separating *what* is done from *when*. Clean concurrency keeps concurrency code small, separate and built on library tools.

The problem it solves

Myths: it always speeds things up (only with waiting or spare cores); design doesn't change; containers make it irrelevant. Reality: overhead, non-repeatable bugs, a different design.

How it works — defence principles

1. **SRP** — concurrency code separate. 2. **Limit shared data**. 3. **Use copies**. 4. **Independent threads**. 5. **Know your library** (`ConcurrentHashMap`, `ExecutorService`, atomics, `CountDownLatch`/`Semaphore`) — not `wait/notify`. 6. **Execution models** (Chapter 13): producer–consumer, readers–writers, dining philosophers. 7. **Dependencies between synchronised methods** — client- or server-based locking. 8. **Small synchronised sections**. 9. **Plan shutdown** (`shutdown()`, `awaitTermination()`, `shutdownNow()`). 10. **Testing**: one-off failures are bugs; non-threaded code first; pluggable, tunable thread counts; more threads than cores; several platforms; jiggling (jcstress).

Vocabulary: **bound resources**, **mutual exclusion**, **starvation**, **deadlock**, **livelock** (Chapters 12–13; *Clean Code* Appendix A goes deeper).

Java example — before/after

Before, four threads do `theCount++` on a shared `int` — a read-modify-write race that loses updates. After:

```
// File: code/lld/ch06/concurrency/Main.java
                    tasks.add(() -> {
                        Map<String, Integer> local = new HashMap<>();       // thread-confined: no sharing, no
locks
                        for (int r = 0; r < repeats; r++)
                            for (String page : PAGES) countWords(page).forEach((w, c) -> local.merge(w, c,
Integer::sum));
                        return local;
                    });
                // ...
                for (Future<Map<String, Integer>> f : pool.invokeAll(tasks))
                    f.get().forEach((w, c) -> total.merge(w, c, Integer::sum));   // Future.get() safely hands
over each result
                return Map.copyOf(total);                                          // publish an immutable copy
            } finally {
                pool.shutdown();                                                   // clean shutdown is part of
the design
                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();
            }
```

Full runnable code: code/lld/ch06/concurrency/ (see the Full Edition).

Before: `count(the) = 182526 expected 320000 -> lost 137474 updates`; after, every thread count matches exactly. The racy count varies per run (and is occasionally right — why such bugs escape tests). The fix separates pure `countWords` from threading, confines data per task, merges once, and is tested with 1, 4 and 16 threads.

Trade-offs

Confinement + merge: no locks or races, scales, but extra memory and a merge step. Shared state + locks: simple for small cases, but contention, deadlock risk, every access must be synchronised.

When not to use it

No concurrency without a measured need; prefer parallel streams (pure CPU) or Java 21 virtual threads (blocking I/O) over manual threads.

Common mistakes

Unsynchronised shared `HashMap` / `ArrayList`; check-then-act (`containsKey` then `put` — use `putIfAbsent` / `computeIfAbsent`); unshut executors; holding locks while calling unknown code.

Real-world examples

`java.util.concurrent` (Doug Lea), `ForkJoinPool`; MapReduce/Spark; Node.js and Redis event loops.

In the interview

> **30-second answer:** "Concurrency separate from business logic; minimal shared mutable state — confinement, immutability, merging results; `java.util.concurrent` over raw threads; planned shutdown; tests with more threads than cores." *Why isn't* `count++` *safe?* Read, add, write — updates get lost; use `AtomicInteger` / `LongAdder` or a lock.

Self-check

1. Q: What does concurrency decouple? → A: What from when.
2. Q: A one-off failure in concurrent code? → A: Treat it as a real threading bug.

TL;DR

Separate it, share little, use the library, small locks, plan shutdown, test hard.

---

13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)

These are case studies showing the *process*, not new rules.

Chapter 14 — Successive Refinement (an `Args` parser): write it dirty, then clean it — never stop at dirty; stop adding features once the code rots; refactor in tiny tested steps; absorb variation by design (one class per argument type behind a "marshaler" interface — OCP); gather error handling into one exception class.

Chapter 15 — JUnit Internals (`ComparisonCompactor`, by Beck and Gamma — already good): drop encoding prefixes (`fExpected` → `expected`, N6); encapsulate and prefer positive conditionals (`canBeCompacted()`, G28/G29); rename functions hiding a second job (`formatCompactedComparison`, N7/G30); make temporal coupling explicit (G31); name `+1` / `-1` boundary arithmetic (G33). Even good code can be left cleaner.

Chapter 16 — Refactoring SerialDate (JCommon): first make it work — coverage rose from ~50% to ~92% and exposed real bugs; then make it right — rename (`DayDate`), enums for int constants (`Month`, J3), move methods to their data (G14/G17), an **Abstract Factory** so the base class doesn't know its subclass (G7), delete dead code, duplication and noise (G9, G5, C3); tests green throughout.

TL;DR: make it work, then make it right, in small tested steps.

---

14. Smells and Heuristics (the checklist)

What it is

Clean Code's catalogue of coded **smells and heuristics** (C1, G5, N1…), extending Fowler's smells (Chapter 7). Hints, not laws.

The problem it solves

You must *recognise* problems fast; codes give a shared review vocabulary ("G28 — encapsulate that conditional").

How it works — the catalogue, grouped (names listed; meaning in our words)

Comments (C): C1 inappropriate information (history, authors → VCS); C2 obsolete comment; C3 redundant comment; C4 poorly written comment; C5 commented-out code (delete).

Environment (E): E1 build requires more than one step; E2 tests require more than one step.

Functions (F): F1 too many arguments; F2 output arguments; F3 flag arguments; F4 dead function.

General (G):

Theme	Heuristics
Correctness and safety	G2 obvious behaviour unimplemented (least surprise); G3 incorrect behaviour at the boundaries (test edges); G4 overridden safeties (warnings off, skipped tests); G21 understand the algorithm; G26 be precise (no floating-point currency; nulls, concurrency, ties); G31 hidden temporal couplings (make order explicit); G33 encapsulate boundary conditions (`lastIndex = size - 1`)
Duplication and dead weight	**G5 duplication** ("one of the most important rules" — use methods, polymorphism, Template Method, Strategy); G9 dead code; G12 clutter
Structure and placement	G1 multiple languages in one file; G6 code at wrong level of abstraction; G7 base classes depending on derivatives; G8 too much information (small interfaces); G10 vertical separation; G13 artificial coupling; G14 feature envy; G17 misplaced responsibility; G18 inappropriate static; G22 make logical dependencies physical; G35 configurable data at high levels; G36 avoid transitive navigation (Demeter)
Readability and intent	G11 inconsistency; G15 selector arguments; G16 obscured intent; G19 explanatory variables; G20 names say what functions do (`applyDiscountPercent(5)`); G23 polymorphism over if/switch ("one switch"); G24 conventions; G25 named constants; G27 structure over convention; **G28 encapsulate conditionals** (`isRefundable(order)`); G29 avoid negative conditionals; G30 one thing; G32 don't be arbitrary; G34 one level of abstraction

Java (J): J1 wildcard imports to shorten lists (dated — Google style forbids them; follow your team); J2 don't inherit constants (static imports); J3 enums over `int` constants.

Names (N): N1 descriptive; N2 right level of abstraction (`notify(recipient)`, not `sendSms(phone)`); N3 standard nomenclature (`...Decorator`); N4 unambiguous; N5 long names for long scopes; N6 no encodings; N7 names describe side effects (`getOrCreateUser`).

Tests (T): T1 insufficient tests; T2 use a coverage tool (JaCoCo); T3 don't skip trivial tests; T4 an ignored test is a question about an ambiguity; T5 test boundary conditions; T6 exhaustively test near bugs; T7 patterns of failure are revealing; T8 coverage patterns are revealing; T9 tests should be fast.

Java example — several heuristics on one function

Before: `static double calc(Parcel p, boolean express)` with `double` money, a double negative `!(p.membership() != Membership.PRIME)`, a tangled `&& !p.fragile() == false` condition, magic numbers and a flag. After:

```java
// File: code/lld/ch06/smells/Main.java
        static BigDecimal shippingCost(Parcel p, Speed speed) {
            BigDecimal baseFee = isPrime(p) ? BigDecimal.ZERO : BASE_FEE;
            BigDecimal handlingFee = needsSpecialHandling(p) ? SPECIAL_HANDLING_FEE : BigDecimal.ZERO;
            BigDecimal weightCharge = BigDecimal.valueOf(p.weightKg()).multiply(RATE_PER_KG);   // explanatory
variables
            BigDecimal cost = baseFee.add(handlingFee).add(weightCharge).multiply(speed.multiplier);
            if (isPrime(p) && speed == Speed.EXPRESS) cost = cost.multiply(PRIME_EXPRESS_DISCOUNT);
            return cost.setScale(2, RoundingMode.HALF_UP);                          // round once, explicitly, to
paise
        }

        private static boolean isPrime(Parcel p) { return p.membership() == Membership.PRIME; }   // positive
conditional
        private static boolean needsSpecialHandling(Parcel p) {                              // encapsulated
conditional
            boolean heavy = p.weightKg() > HEAVY_KG;
            boolean fragileLongHaul = p.distanceKm() > LONG_HAUL_KM && p.fragile();
            return heavy || fragileLongHaul;
        }
```

Full runnable code: code/lld/ch06/smells/ (see the Full Edition). A "same" column over eight parcel/speed cases proves behaviour unchanged. Fixed: G29, G16/G28, G25, F3 (→ `Speed` enum), G19, N1/G20 (`calc` → `shippingCost`), G26 (`BigDecimal`, one `HALF_UP`).

Trade-offs

A checklist can be applied dogmatically; when items conflict, favour readability and correctness.

When not to use it

Don't block a critical fix on nits — log follow-ups.

Common mistakes

Gaming smell counts; following dated notes like J1 over your team's guide.

Real-world examples

SonarQube, PMD, Checkstyle, SpotBugs, Error Prone.

In the interview

30-second answer: "I look for duplication, long functions, flag/output arguments, magic numbers, negative or complex conditionals, feature envy, train wrecks, misplaced responsibilities, dead code and weak tests — fixed with small refactorings: extract method, introduce constant, replace conditional with polymorphism."

Self-check

1. Q: What does G28 ask? → A: Encapsulate conditionals in a named function.
2. Q: What does T4 say about a disabled test? → A: It marks an unresolved requirement ambiguity.

TL;DR

C/E/F/G/J/N/T as shared review vocabulary; fix with small tested refactorings.

---

15. Code Review Checklist (best practices)

What it is

A peer reading a change before merge to catch defects, share knowledge and keep consistency — and, in interviews, what you run over your own code before saying "done".

The problem it solves

Unreviewed changes bring bugs, security holes and inconsistency; unfocused reviews chase nits and miss real issues.

How it works — the checklist (from most to least important)

1. **Correctness** — meets the requirement; edge cases (empty, null, zero, max, duplicates, time zones, rounding); errors handled, resources closed; concurrency (shared state, check-then-act, thread-safe collections).
2. **Design** — fits the architecture; right class/layer (SRP, GRASP); DIP, OCP; no over-engineering (YAGNI) or duplication (DRY).
3. **Readability** — names, small functions, no magic numbers or flags; *why* comments; no commented-out code.
4. **Tests** — new behaviour covered; regression test per bug fix; F.I.R.S.T., one concept each.
5. **Security** — validation; no injection (parameterised queries); output encoding; no secrets in code/logs, PII masked; authorisation on sensitive operations.
6. **Performance and resources** — N+1 queries, repeated work, unbounded collections, missing pagination; timeouts; closing.
7. **Operability** — useful logs and metrics; feature flags; backwards-compatible APIs, messages, migrations.
8. **Style** — automated.

Reviewing well: small changes; the author describes what/why/how-tested and self-reviews; the reviewer is specific and kind, explains why, marks "nit:"; automate the mechanical.

Java example

No separate program: reviewing `Before.book` in §2 would flag F3, error codes, F2, the hidden `SESSION` side effect, G23 and G25 — exactly its *after* fixes.

Trade-offs

Slightly slower merges; defects caught early, knowledge spread. Too many reviewers or nit-heavy reviews slow teams without quality gains.

When not to use it

Emergency hotfixes may be reviewed after merge — but still reviewed.

Common mistakes

Style-only reviews; huge PRs; personal preferences as rules.

Real-world examples

GitHub PRs, GitLab MRs, Gerrit; Google's published review guidance; CODEOWNERS.

In the interview

> **30-second answer:** "Correctness first — requirements, edge cases, errors, concurrency — then design, readability, tests, security, performance; formatting left to tools; specific feedback, nits marked optional."

Self-check

Q: Why small PRs? → A: Large diffs get shallow reviews.

TL;DR

Correctness → design → readability → tests → security → performance → operability; automate style.

---

16. Error-Handling Strategy (best practices for real services)

What it is

§6 covered rules in code; a **strategy** is the system-wide policy: which exception types exist, where errors are translated, which are retried, how they're logged, what callers see.

The problem it solves

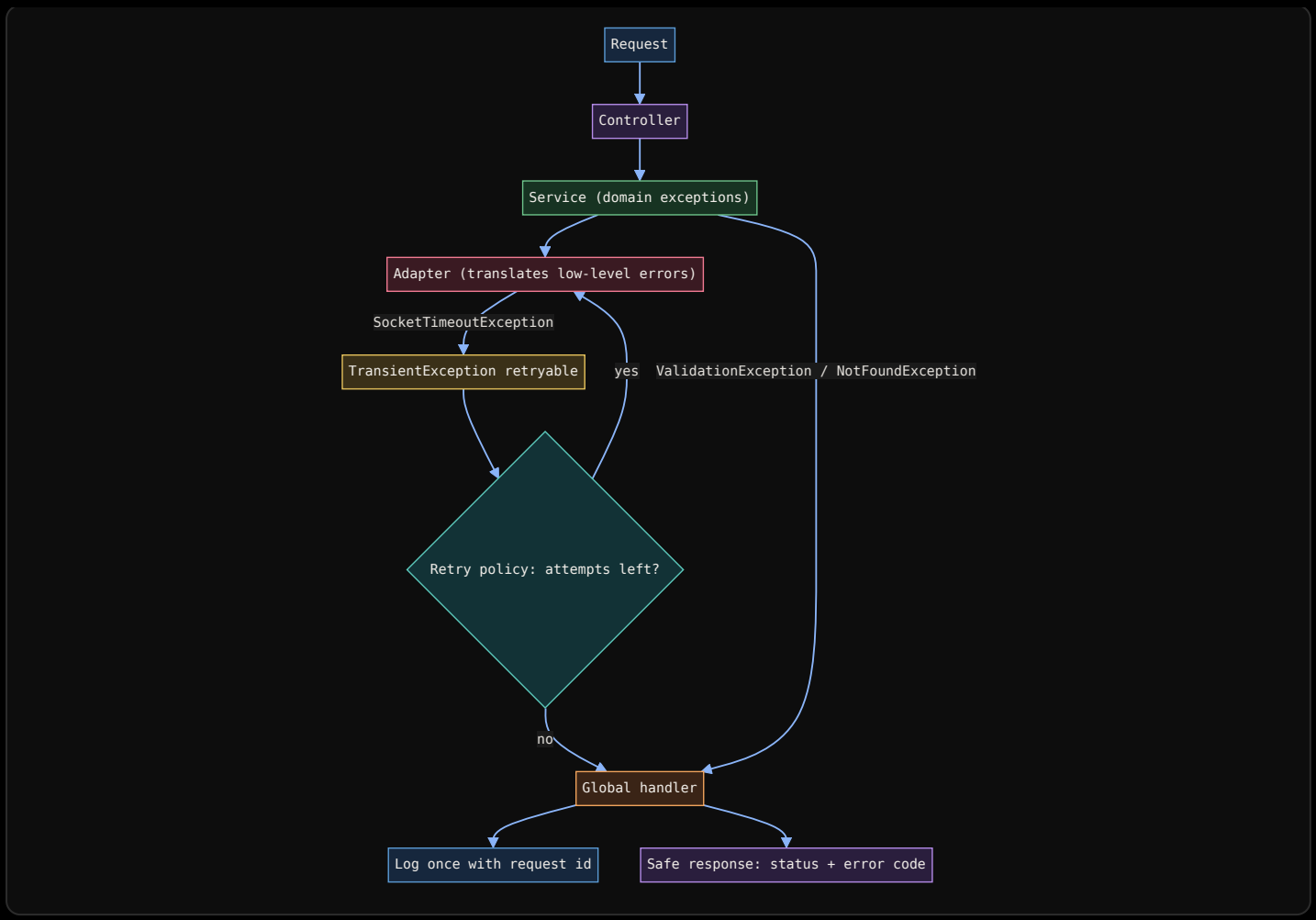
Without one: nulls, log-and-rethrow everywhere, stack traces to users, retried permanent failures overloading services.

How it works

1. **Classify errors:**

Class	Examples	Handling
Validation / client	bad input	Reject, 4xx, no retry
Business rule	insufficient funds, seat taken	Domain exception or result, 4xx (409)
Not found	unknown id	404 / empty `Optional`
Transient infrastructure	timeouts, 503, deadlock victim	Retry with backoff — only if **idempotent**
Permanent infrastructure	DB auth failure, misconfiguration	Fail fast, alert
Programming bugs	NPE, index out of bounds	Don't catch locally; generic 500, log, fix

2. **One exception hierarchy** with an error code and a *retryable* flag. 3. **Translate at boundaries** (adapters), keeping the cause. 4. **Retry only transient** errors: max attempts, **exponential backoff**, **jitter**, plus timeouts and circuit breakers. 5. **Result values for expected per-item failures** (collect all row errors). 6. **Global handler at the edge**: status, stable code, safe message, **log once** with a request id; never leak internals. 7. **Stay consistent**: transactions or compensation; idempotent handlers.



Java example

```java
// File: code/lld/ch06/errorstrategy/Main.java
    static <T> T withRetry(Supplier<T> action, int maxAttempts, long initialBackoffMs) {
        long backoff = initialBackoffMs;
        for (int attempt = 1; ; attempt++) {
            try {
                return action.get();
            } catch (AppException e) {
                if (!e.retryable || attempt == maxAttempts) throw e;
                // ...
                sleep(backoff);
                backoff *= 2;                                   // exponential backoff (add jitter in production)
            }
        }
    }
```

Full runnable code: code/lld/ch06/errorstrategy/ (see the Full Edition). req-1: a timeout is retried (10 ms, 20 ms) then succeeds; req-2: `NotFoundException` isn't retryable → 404; req-3: gives up after 3 attempts → 503; req-4: a bug → generic 500 with only a request id; a sealed `Result` (`Ok`/`Err`) collects all bulk-import row errors, then the request is rejected with 400.

Trade-offs

Choice	Benefit	Cost
Retries	Survive blips	Duplicate side effects unless idempotent; amplify load
Global handler	Consistent responses, log once	Must not swallow context
Result types	Explicit expected failures	Verbose in Java; needs a clear rule vs exceptions

When not to use it

Don't retry non-idempotent operations ("charge card") without an **idempotency key**; never retry validation or authorisation failures.

Common mistakes

Log-and-rethrow; catching `Exception` in business code; tight retry loops without backoff; 200 with an error body.

Real-world examples

Spring `@ControllerAdvice` / `@ExceptionHandler`; RFC 9457 Problem Details (`ProblemDetail`); Resilience4j; AWS SDK backoff with jitter; Stripe idempotency keys.

In the interview

> **30-second answer:** "Classify errors; one hierarchy with codes and a retryable flag; translate in adapters; retry only transient, idempotent calls with backoff and jitter; one global handler returns safe responses and logs once." *Why jitter?* To avoid a thundering herd.

Self-check

1. Q: Which errors are retried? → A: Transient ones, for idempotent operations.
2. Q: Where are errors logged? → A: Once, at the global handler, with a correlation id.

TL;DR

Classify, translate, retry transient + idempotent with backoff, results for expected failures, one global handler.

---

17. Testing Strategies (unit vs integration, test doubles, mocks)

What it is

Deciding *which kinds* of tests, *how many*, and *how to isolate* code, building on §8.

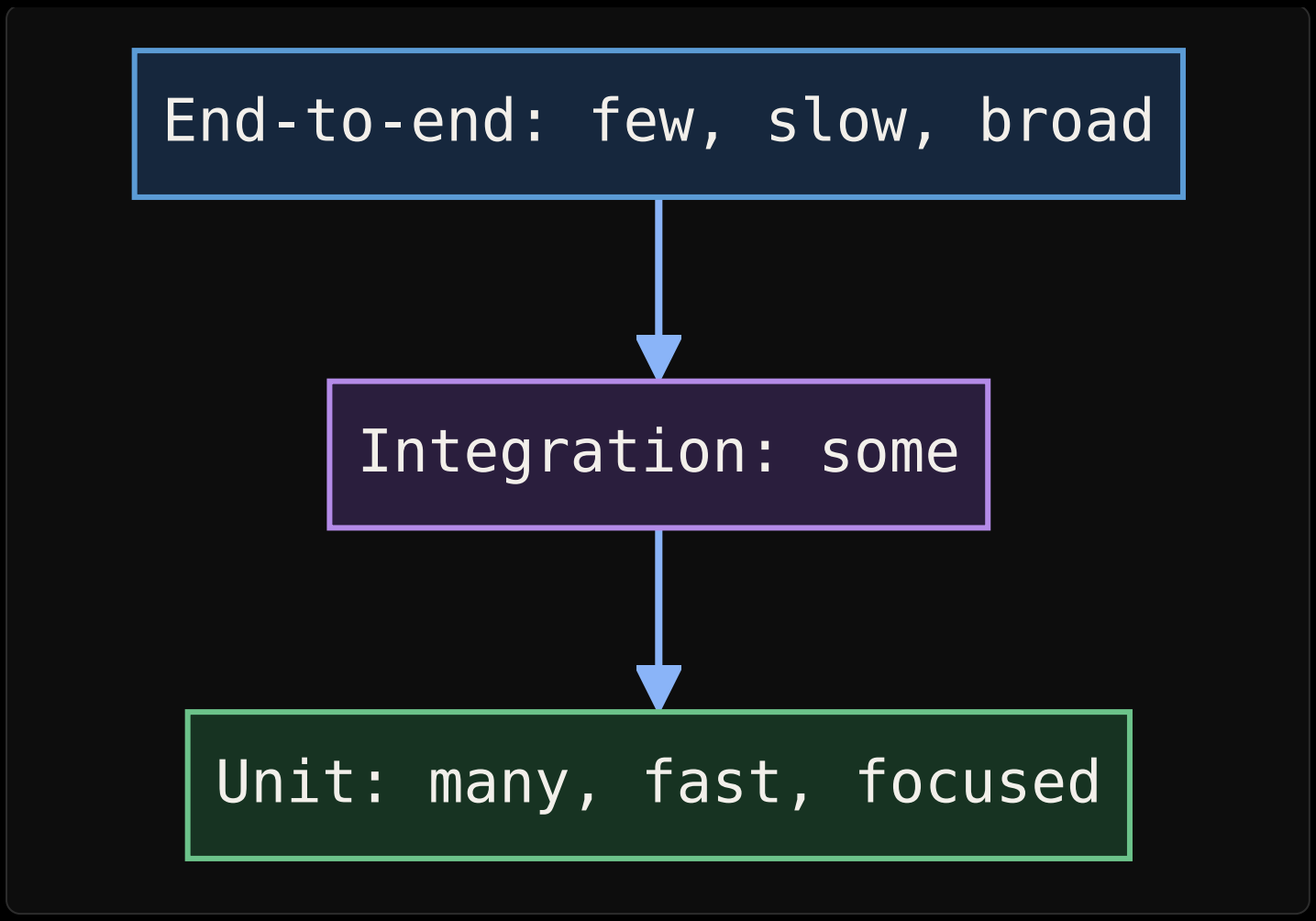
The problem it solves

Too few tests → production bugs; mostly slow E2E tests → flaky pipelines; over-mocking → tests pass while the system is broken.

How it works

Kind	Scope	Speed	Catches
Unit	One class, doubles for collaborators	ms	Logic errors
Integration	Real components together (DB, broker)	s	Wiring, queries, serialisation, config
Contract	Consumer–provider agreement	Fast	API mismatches
End-to-end	Whole system via UI/API	Slow	User-journey breakage
Others	Performance, security, exploratory, property-based, mutation	Varies	Non-functional issues, weak tests

Test Pyramid (Mike Cohn): many unit, some integration, few E2E — not the "ice-cream cone".



Test doubles (Meszaros, via Fowler):

Double	What it does	In our code
Dummy	Fills a parameter; never used	`DummyAuditLog` (throws if called)
Stub	Canned answers	`StubCatalog`
Spy	Stub that records calls	`SpySender.sent`
Mock	Pre-programmed expectations, verifies itself	`MockAuditLog.expect(...).verify()`
Fake	Working simplified implementation	`FakeCoupons`

State vs behaviour verification: prefer state (stubs/fakes); mocks for unobservable outgoing side effects. **Test** public behaviour, edge cases, error paths, every bug fix — not private methods or the framework. **Mocking:** don't mock types you don't own (wrap them) or value objects; many mocks means too many dependencies. **Coverage** (JaCoCo) is necessary, not sufficient; mutation testing (PIT) checks the tests.

Java example

The code is in §8 (`lld.ch06.testing`): `CheckoutTest` uses all five doubles, `CheckoutIntegrationTest` wires real in-memory parts, `BeforeStyleTest` shows what to avoid. The mock records calls in `actual` and its `verify()` asserts them against `expected`.

Trade-offs

More unit tests: fast, precise, may miss wiring bugs; over-mocking couples to implementation. More integration/E2E: higher confidence, slow, flaky, need environments.

When not to use it

Thin framework glue is better covered by an integration test; push edge cases down from E2E to unit tests.

Common mistakes

Mocking everything (even value objects or the class under test); leaving flaky tests unfixed; testing internal calls instead of outcomes; no error-path tests.

Real-world examples

JUnit 5, Mockito, AssertJ, Testcontainers, WireMock, Pact, PIT, JaCoCo; Spring Boot `@WebMvcTest`, `@DataJpaTest`.

In the interview

> **30-second answer:** "The test pyramid: many fast unit tests with doubles, fewer integration tests with real components (Testcontainers), a handful of E2E. State verification with stubs and fakes; mocks only for outgoing side effects; never mock types I don't own." *Mock vs stub?* A stub supplies answers for state checks; a mock verifies the calls itself.

Self-check

1. Q: The five doubles? → A: Dummy, stub, spy, mock, fake.
2. Q: Is 100% coverage proof? → A: No — mutation testing checks assertions catch bugs.

TL;DR

Many unit, some integration, few E2E; the right double; state over interactions; don't mock what you don't own.

Chapter summary

Topic	Remember this
Names	Reveal intent, pronounceable, searchable, one word per concept
Functions	Small, one thing, one level, stepdown, ≤2–3 args, no flags, CQS, exceptions
Comments	Explain in code; comment why/warnings/legal/public API
Formatting	Newspaper order, related things close, automated style
Objects vs data	Objects hide data; data structures expose it; anti-symmetry decides
Error handling	Unchecked with context, wrap vendors, no null in or out
Boundaries	Wrap third-party code; learning tests; interfaces you need
Unit tests	TDD laws, one concept per test, F.I.R.S.T.
Classes	Small by responsibility, cohesive, organised for change
Systems	Composition root + DI; cross-cutting via proxies/aspects
Emergence	Beck: tests → intent → no duplication → fewest elements (*Clean Code* swaps the middle two)
Concurrency	Separate it, share little, use the library, test hard
Smells	C/E/F/G/J/N/T checklist
Code review	Correctness → design → readability → tests → security → performance
Error strategy	Classify, translate, retry transient, handle globally, log once
Testing strategy	Pyramid; right double; state over behaviour verification

Chapter self-check

1. Q: `boolean saveAndValidate(Order o, boolean sendEmail)` breaks what? → A: Does two things, flag argument (F3), likely breaks CQS.
2. Q: `findOrders(customerId)` with no results? → A: An empty list, never `null`.
3. Q: Testing code that calls `LocalDate.now()`? → A: Inject a `Clock`; `Clock.fixed` in tests.

TL;DR (chapter)

Write for the next reader: honest names, tiny single-purpose functions, few "why" comments, consistent layout, exceptions instead of codes or nulls, foreign code behind your interfaces, clean fast tests, small cohesive classes wired from one place, simple design through refactoring, careful concurrency — and review against a shared checklist.

Chapter 7 — Refactoring (Study Edition)

Study Edition. A shorter version of Chapter 7: every topic, all 24 smells and all 61 refactorings are here, but in compact form. The full chapter (README.md) has longer explanations and the complete source code.

What you will learn. What refactoring is, when (and when not) to do it, how tests make it safe, the 24 **code smells** that tell you *where* to refactor, the 61 **refactorings** of Fowler's *Refactoring* (2nd ed., 2018) that tell you *how*, a smell → refactoring lookup table, a worked Java example in six compiling steps, and how refactoring leads to design patterns.

Why it matters for LLD interviews. Machine-coding rounds often end with "now add feature X" or "what would you clean up?". You want to say: "this switch is repeated in three places — Repeated Switches — so I'd Replace Conditional with Polymorphism, which gives a Strategy."

7.1 Refactoring fundamentals

What it is

Refactoring (noun): a change to the internal structure of software that makes it easier to understand and cheaper to modify, *without changing its observable behaviour*. **Refactoring (verb):** applying a *series* of such small changes.

- **Behaviour-preserving** — same outputs, side effects and errors for the same inputs. Only internals (names, where code lives, how it is split) change.
- **Small steps** — each step compiles and passes the tests. If your code is broken for an hour, you are rewriting, not refactoring.

Term — observable behaviour. What callers or users can see: return values, output, data written, exceptions. Call stacks, class structure and private method names are *not* observable behaviour.

Activity	Refactoring?	Why
Renaming a method with the IDE	Yes	Behaviour unchanged, tiny step
Fixing a bug / adding a feature	No	Behaviour changes on purpose
Performance tuning	No	Goal is speed, not clarity
Two days of "nothing works until the end"	No	That is restructuring/rewriting

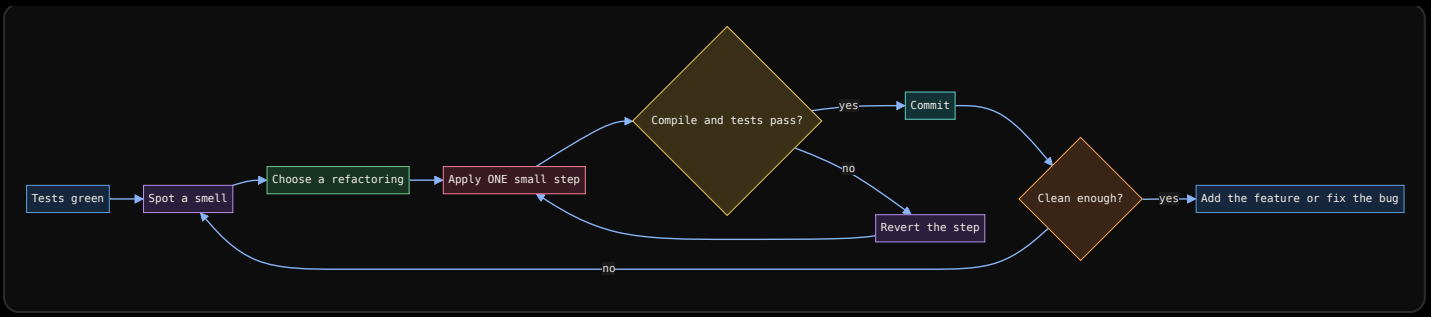
Term — restructuring. The general word for reorganising code; refactoring is the small-step, behaviour-preserving kind.

The problem it solves

Code decays: every quick fix and copy-paste makes the next change harder. This **technical debt** is the extra cost on every future change. Refactoring pays it down continuously, instead of trying to get the design right up front (which never fully works, because requirements change).

How it works

1. Have tests that pin current behaviour — green. 2. Spot a smell (7.2). 3. Pick a named refactoring (7.3, 7.4). 4. Apply **one** small step (ideally IDE-automated). 5. Compile and test: green → commit locally; red → **revert** (don't debug) and take a smaller step. 6. Repeat until clean enough for the job at hand.



7.1.1 The two hats

Kent Beck: at any moment you wear one hat. **Adding function** — add behaviour and tests, don't restructure. **Refactoring** — restructure, don't add behaviour (tests for *existing* behaviour are fine). Swap often, but never mix: if a test fails while you did both, you can't tell which change broke it.

7.1.2 Why refactor

- **Improves the design** — stops decay; removes duplication so each fact lives in one place.
- **Makes code easier to understand** — moves understanding out of your head into the code.
- **Helps find bugs** — clear structure leaves bugs fewer places to hide.
- **Helps you program faster** — Fowler's **Design Stamina Hypothesis**: teams that keep internal quality high soon add features *faster*. Speed is the economic argument, not "clean code for its own sake".

7.1.3 When to refactor

Trigger	Meaning
Rule of Three (Don Roberts)	1st time do it, 2nd time wince and duplicate, 3rd time refactor
Preparatory	Just before a feature: "make the change easy, then make the easy change" (7.5 step 3)
Comprehension	Rename/extract as you come to understand code
Litter-pickup	Boy Scout Rule — fix small mess you pass; note bigger mess for later
Planned vs opportunistic	Mostly opportunistic, inside feature work; a quarterly "refactoring sprint" is a warning sign
Long-term	Gradual big changes, e.g. **Branch by Abstraction** (abstraction → move callers → swap impl → delete old)
Code review	Suggest or pair on refactorings, ideally with the author

> **Term — Rule of Three.** Don't abstract on the 1st or 2nd occurrence (you don't know what varies yet); abstract on the 3rd.

Ask your manager? Fowler: no more than for writing tests; discuss only large planned efforts.

7.1.4 When NOT to refactor

- Messy code behind a clean API that you never touch.
- When a rewrite is cheaper (7.1.10).
- Without a safety net — add tests first (7.1.11).
- An immovable deadline may postpone a *big* refactoring (skipping small ones usually slows you).
- **Published interfaces** — possible, but keep the old version working and deprecate it (Change Function Declaration, migration mechanics).

7.1.5 Problems and costs of refactoring

Problem	Mitigation
"Slows down features"	Refactor *because* it speeds features; focus on code you're about to change
Code ownership (callers in another repo)	Keep old name as a delegating `@Deprecated` method; migrate later
Long-lived feature branches	Continuous Integration / trunk-based — integrate at least daily
Missing tests	Add tests first; only IDE-automated refactorings until then
Databases	Evolutionary database design; small versioned migrations; **parallel change**

> **Term — parallel change (expand / contract).** *Expand* (add new beside old), *migrate* (move users), *contract* (remove old). Nothing is ever broken.

7.1.6 Refactoring, architecture and YAGNI

Because you can reshape code safely later, build only what today's requirements need — **YAGNI** ("You Aren't Gonna Need It"). This does not mean "no architecture": think hard up front only about things truly hard to change later (public APIs, database schemas).

7.1.7 Tests as the safety net

Self-testing code = automated tests that check themselves and run in seconds.

- Run tests after **every** step; on failure, **revert** and redo smaller.
- Test **behaviour** (public API), not private structure — otherwise every refactoring breaks tests.
- **Characterisation tests** (Feathers): record what code *currently* does, even if wrong. 7.5 uses a byte-for-byte output checker.
- **Mutation testing** (e.g. PIT) checks that your tests actually catch injected bugs.

> **Term — safety net.** Anything that tells you quickly you broke something: tests, compiler type checks, IDE-automated refactorings.

7.1.8 IDE automation

IDEs refactor using the **syntax tree and symbol resolution**, not text search — Rename changes only that symbol and its real references. (History: Opdyke's 1992 thesis → the Smalltalk *Refactoring Browser* → Java IDEs.)

Refactoring	IntelliJ (Win/Linux · macOS)	Eclipse
Rename	Shift+F6 · Shift+F6	Alt+Shift+R
Extract Method	Ctrl+Alt+M · Cmd+Opt+M	Alt+Shift+M
Extract Variable	Ctrl+Alt+V · Cmd+Opt+V	Alt+Shift+L
Inline	Ctrl+Alt+N · Cmd+Opt+N	Alt+Shift+I
Change Signature	Ctrl+F6 · Cmd+F6	Alt+Shift+C
Move	F6 · F6	Alt+Shift+V

Large-scale tools: **OpenRewrite** recipes, Google's **Refaster** templates in Error Prone.

> **Caution.** Automated refactorings can miss **reflection**, names in **config files**, **serialised field names** (JSON/Jackson, JPA) and other repositories. Renaming a serialised field changes your API's JSON —

> an observable behaviour change.

7.1.9 Performance and refactoring

1. Most refactorings have negligible cost — the JIT inlines small methods.
2. Programs spend most time in a small fraction of code (Chrysler payroll story: profiling found a hot spot nobody guessed).
3. Well-factored code is **easier to tune** — the hot spot is small and isolated.

Order: **make it work → make it clean → make it fast (profile, optimise hot spots)**.

Care is still needed: see the `DoubleStream.sum()` trap in 7.5 step 2.

7.1.10 Refactoring vs rewriting

	Refactoring	Rewriting
Unit of change	Many tiny safe steps	One big replacement
System works during change?	Always	Usually not until the end
Risk	Low	High — old bug fixes lost; old system keeps evolving
Delivers value	Continuously	Only at the end
Good when	Code mostly works	Code doesn't work at all, or technology must change

Even replacements go piece by piece: the **Strangler Fig** builds the new system around the old, routing one feature at a time until the old one can be switched off.

7.1.11 Legacy code

Feathers (*Working Effectively with Legacy Code*, 2004): **legacy code = code without tests**. The loop: identify change points → find test points → break dependencies at a **seam** (a place to change behaviour without editing there, e.g. inject an interface instead of `new Database()`, or override in a test subclass) → write characterisation tests → change and refactor.

Also: **Sprout Method/Class** (new tested code, called by a one-line change), **Wrap Method/Class** (rename old; new method with the old name calls old + new behaviour); automated refactorings only until tests exist; clean only what you touch.

7.2 Code smells

What it is

A **code smell** (Kent Beck's term) is a *surface sign* that usually points to a deeper design problem. It is not a bug — it tells you *where to look*. Smells are **heuristics, not rules**.

The problem it solves

You must know *when* and *where* to refactor, not just how. Fowler and Beck give no metrics ("max 10 lines"); they give **named** smells with their usual cures. Names help reviews: "Feature Envy" says in two words what would take a paragraph.

How it works

Notice discomfort → match it to a named smell → look up its cures (7.4) → apply in small steps under tests.

Smell groups (refactoring.guru classification)

Group	Idea	Smells (2nd-ed name; *guru name*)
Bloaters	Grown too big	Long Function *(Long Method)*, Large Class, Primitive Obsession, Long Parameter List, Data Clumps
OO Abusers	Wrong/incomplete OO	Repeated Switches *(Switch Statements)*, Temporary Field, Refused Bequest, Alternative Classes with Different Interfaces
Change Preventers	One change forces many	Divergent Change, Shotgun Surgery *(+ Parallel Inheritance Hierarchies, 1st ed.)*
Dispensables	Pointless; removing helps	Comments, Duplicated Code, Lazy Element *(Lazy Class)*, Data Class, Speculative Generality *(+ Dead Code)*
Couplers	Too much coupling/delegation	Feature Envy, Insider Trading *(Inappropriate Intimacy)*, Message Chains, Middle Man *(+ Incomplete Library Class)*
2nd-edition extras		Mysterious Name, Global Data, Mutable Data, Loops

> **Term — coupling / cohesion.** Coupling: how much one piece depends on another. Cohesion: how strongly the things inside one unit belong together. Aim for low coupling, high cohesion.

Below, all 24 smells in the book's order: what it is, why it hurts, a tiny smelly → cured sketch, and the cures. (Snippets use `double` for money to stay short; real money needs `BigDecimal` or a `Money` type — see `code/smells/Money.java`.)

7.2.1 Mysterious Name

What. Names that don't say what a thing does or holds: `data`, `tmp`, `proc()`, `x2`, `Manager`. **Hurts.** Every reader must decode the body; if you *can't* find a good name, the thing may do too much. `calc(l, d)` → `averageDailyRevenue(dailyRevenues, daysInPeriod)`. **Cures:** Change Function Declaration, Rename Variable, Rename Field.

7.2.2 Duplicated Code

What. The same code structure in more than one place. **Hurts.** Every change must find every copy; one gets missed. `net + net * 0.18` in two methods → one `withTax(net)` using `GST_RATE`. **Cures:** Extract Function; Slide Statements; Pull Up Method.

7.2.3 Long Function

What. A function too long to grasp at a glance. Heuristic: when you feel a block needs a **comment**, extract it and name it after its *intent*. **Hurts.** Hard to understand, reuse and test. `printOwing` → `printBanner(); outstandingOf(inv); recordDueDate(inv); printDetails(...)`. **Cures:** Extract Function (~99% of the time), plus Replace Temp with Query, Decompose Conditional, Split Loop and others (7.4).

7.2.4 Long Parameter List

What. Many parameters, often same-typed, or boolean flags. **Hurts.** Callers mix up order; each new parameter touches every caller; hides a missing concept. `book(hotelId, from, to, adults, children, breakfast)` → `book(hotel, Stay, Guests, MealPlan)`. **Cures:** Introduce Parameter Object, Preserve Whole Object, Remove Flag Argument (+ 7.4).

7.2.5 Global Data

What. Data readable and writable from anywhere: globals, public static mutable fields, mutable singletons. **Hurts.** No way to find who changed it; bugs appear far from their cause. One of the most pungent smells. `public static double discountRate` → private field + `discountRate()` / `setDiscountRate()`; better still, inject the value. **Cures:** Encapsulate Variable first, always; then narrow its scope.

7.2.6 Mutable Data

What. Data changed after creation, especially when shared or reused for different purposes. **Hurts.** A change in one place breaks code elsewhere that assumed the value stayed put — worse with threads. `public List<Item> items; public double total;` → private list with `add()`, `items()` returning `List.copyOf`, and `total()` computed on demand. **Cures:** Encapsulate Variable, Replace Derived Variable with Query, Remove Setting Method, Change Reference to Value (+ 7.4).

7.2.7 Divergent Change

What. One class changed for many different reasons (new database *and* new instrument *and* new report format). **Hurts.** Mixed responsibilities break SRP; changes interfere. `ProductService` (save + price + CSV) → `ProductRepository`, `PricingService`, `ProductCsvWriter`. **Cures:** Split Phase, Extract Class (+ Move/Extract Function).

7.2.8 Shotgun Surgery

What. The opposite: **one change forces small edits in many classes**. **Hurts.** Easy to miss one place; the concept is scattered. The "free shipping at 500" rule in four classes → one `ShippingPolicy` with `FREE_SHIPPING_THRESHOLD`, `isFree()`, `charge()`. **Cures:** Move Function, Move Field (+ 7.4); sometimes Inline first, then re-extract.

	Divergent Change	Shotgun Surgery
Shape	One class, many reasons to change	One reason, many classes
Fix	Split the class	Gather the scattered code

7.2.9 Feature Envy

What. A method more interested in another class's data than its own (calls many getters). **Hurts.** Behaviour lives away from its data. Rule: **put behaviour next to the data it uses** (Strategy/Visitor separate them on purpose). `order.lineTotal(line)` → `line.total()`. **Cures:** Move Function (Extract Function first if only part is envious).

7.2.10 Data Clumps

What. The same group of values always travels together: `(street, city, zip)`, `(start, end)`, `(amount, currency)`. **Hurts.** An unnamed concept; its behaviour gets duplicated. **Test:** delete one item — if the rest makes no sense alone, make an object. `(LocalDate start, LocalDate end)` → `record DateRange` with `contains()` (see `code/smells/DateRange.java`). **Cures:** Extract Class, Introduce Parameter Object, Preserve Whole Object — then watch for Feature Envy moving onto the new class.

7.2.11 Primitive Obsession

What. Primitives for domain concepts: money as `double`, phone as `String`, status as `"ACTIVE"`. **Hurts.** No validation, duplicated formatting, easy mix-ups (`transfer(amount, fee)` swapped compiles), and `0.1 + 0.2 = 0.30000000000000004`. `charge(double, String)` → `charge(Money)`. **Cures:** Replace Primitive with Object (+ Replace Type Code with Subclasses, 7.4).

7.2.12 Repeated Switches

What. The *same* switch on the same type code in several places. (1st ed. "Switch Statements" flagged any switch; the 2nd ed. flags only the repetition.) **Hurts.** A new case means finding every switch — Shotgun Surgery. Three `switch (b.type)` methods → `sealed interface Bird` with `speed()`, `plumage()`. **Cures:** Replace Conditional with Polymorphism. (With a `sealed` type or `enum`, switch *expressions* are exhaustiveness-checked, so a few are acceptable.)

7.2.13 Loops

What. A `for` loop that filters/maps/aggregates where a pipeline says it better (new in 2nd ed.). **Hurts.** The reader must simulate the loop. Loop adding IT names →

`employees.stream().filter(...).map(Employee::name).toList()`. **Cures:** Replace Loop with Pipeline. Careful: `toList()` is **unmodifiable**; use `Collectors.toCollection(ArrayList::new)` if callers mutate.

7.2.14 Lazy Element

What. A class/function/subclass that doesn't pull its weight (1st ed. "Lazy Class"). **Hurts.** Every element costs a name, a file, an indirection. `NameFormatter.format(s)` that only trims → `rawName.trim()`. **Cures:** Inline Function, Inline Class, Collapse Hierarchy.

7.2.15 Speculative Generality

What. Hooks and abstractions "we might need someday": abstract class with one subclass, unused parameters, methods used only by tests. **Hurts.** Cost with no benefit; violates **YAGNI**. `PaymentProcessorFactoryProvider` with one implementation, `futureOptions` never used → `pay(Money m)`. **Cures:** Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code (delete the tests too).

7.2.16 Temporary Field

What. A field set and used only in some situations, null the rest of the time. **Hurts.** Readers expect every field to matter — "is this null a bug?". `Planner.tmpDistances`, `tmpTarget` → a short-lived `RouteSearch` object holding that state. **Cures:** Extract Class, Move Function, Introduce Special Case.

7.2.17 Message Chains

What. `order.getCustomer().getAddress().getCity()`. **Hurts.** Client coupled to the whole navigation path; violates the **Law of Demeter**. → `order.shippingCity()` (see `code/smells/Order.java`). Don't hide every delegate (creates Middle Men). **Cures:** Hide Delegate, Extract Function, Move Function.

7.2.18 Middle Man

What. A class whose methods *mostly* just forward to another class. **Hurts.** A layer with no value; every delegate feature needs a new forwarder. `Person` with ten `dept.getX()` forwarders → `department()` accessor. **Cures:** Remove Middle Man, Inline Function, Replace Superclass/Subclass with Delegate. Message Chains and Middle Man pull in opposite directions — balance them.

7.2.19 Insider Trading

What. Modules poking at each other's internals; subclasses knowing too much about parents (1st ed. "Inappropriate Intimacy"). **Hurts.** Can't change one without the other. `customer.internalLoyaltyTable.get(customer.tierCode)` → `customer.loyaltyDiscount()`. **Cures:** Move Function, Move Field, Hide Delegate, Replace Subclass/Superclass with Delegate.

7.2.20 Large Class

What. A class doing too much — a **God Class**. **Hurts.** Magnet for duplication, bottleneck for changes, breaks SRP. `Employee` with identity + address + payroll + performance fields → `Address`, `Compensation`, `PerformanceRecord`. **Cures:** Extract Class, Extract Superclass, Replace Type Code with Subclasses.

7.2.21 Alternative Classes with Different Interfaces

What. Classes doing the same job with different signatures: `EmailSender.sendMail(to, body)` vs `SmsGateway.dispatch(msg, phone)`. **Hurts.** No polymorphism → callers write `if email ... else sms`. → `interface Notifier { void send(String recipient, String message); }`. **Cures:** Change Function Declaration, Move Function, Extract Superclass; for third-party classes, an **Adapter** (Ch. 9).

7.2.22 Data Class

What. Only fields and getters/setters; others manipulate its data. **Hurts.** Behaviour lives elsewhere (Feature Envy), duplicated; public setters break invariants. **Exceptions:** immutable records — Split Phase results, DTOs, value objects — are fine. `if (acc.getBalance() >= amt) acc.setBalance(...)` → `account.withdraw(amount)`. **Cures:** Encapsulate Record, Remove Setting Method, Move Function (+ 7.4).

7.2.23 Refused Bequest

What. A subclass that doesn't want what it inherits — or refuses the *interface* (throws `UnsupportedOperationException`). **Hurts.** Refusing implementation is mild; refusing the interface breaks **Liskov** (Ch. 4). `Stack<T> extends ArrayList<T>` (cf. `java.util.Stack extends Vector`) → `Stack` holding a `Deque`. (Check: empty `pop()` now throws a different exception and `ArrayDeque` rejects nulls.) **Cures:** Push Down Method/Field, Replace Subclass with Delegate, Replace Superclass with Delegate.

7.2.24 Comments

What. Comments used as **deodorant** — explaining *what* confusing code does (Fowler: a "sweet" smell). **Hurts.** Comments go stale. **Good comments:** *why*, warnings, algorithm/ticket references, uncertainty notes, Javadoc. `// check senior discount` + complex `if` → `if (isEligibleForSeniorDiscount(customer))`; `// rate never negative` → `assert rate >= 0`. **Cures:** Extract Function, Change Function Declaration, Introduce Assertion.

1st-edition smell names (renamed or dropped in the 2nd edition)

1st-edition smell	Where it went
Long Method	**Long Function**
Switch Statements	Narrowed to **Repeated Switches**
Lazy Class	**Lazy Element**
Inappropriate Intimacy	**Insider Trading**
Parallel Inheritance Hierarchies	Special case of Shotgun Surgery; Move Function/Field so one hierarchy disappears
Incomplete Library Class	Wrapper/utility (1st ed. Introduce Foreign Method / Local Extension) or Adapter
Dead Code	The refactoring **Remove Dead Code**

7.3 The refactoring catalogue

What it is

A **named refactoring** is a small, well-defined transformation with a name, a motivation, safe mechanics and examples. Fowler's 2nd edition has **61**, in book chapters 6–12 (online at refactoring.com/catalog). Old 1st-edition names appear as aliases. Many come in **inverse pairs** (Extract ↔ Inline Function) — go whichever way makes the code clearer for the change you're about to make.

How to read each entry

Each row: **name** (*alias*; ⇄ inverse), **when/why**, and a tiny **before → after** sketch (illustrative; the fully compiling examples are in `code/`).

Book chapter	Count
6 A First Set · 7 Encapsulation · 8 Moving Features · 9 Organizing Data · 10 Simplifying Conditionals · 11 Refactoring APIs · 12 Inheritance	11 + 9 + 9 + 5 + 6 + 10 + 11 = **61**
refactoring.com extras	5 more

7.3.1 A First Set of Refactorings (book chapter 6)

The everyday ones.

Refactoring	When / why	Before → After
Extract Function (*Extract Method*; ⇄ Inline Function)	Separate *intention* from implementation: if you must study a fragment to see *what* it does, extract it and name it that. Even one line is worth it.	inline print lines → `printDetails(invoice, outstanding)`
Inline Function (*Inline Method*; ⇄ Extract Function)	Body as clear as the name, or a tangle of bad small functions (inline, then re-extract), or pure delegation	`moreThanFiveLateDeliveries(d) ? 2 : 1` → `d.lateDeliveries() > 5 ? 2 : 1`
Extract Variable (*Introduce Explaining Variable*; ⇄ Inline Variable)	Name parts of a complex expression; if the name matters beyond the function, extract a function instead	long price formula → `basePrice - quantityDiscount + shipping`
Inline Variable (*Inline Temp*; ⇄ Extract Variable)	Name says no more than the expression, or blocks another refactoring	`double basePrice = order.basePrice(); return basePrice > 1000;` → `return order.basePrice() > 1000;`
Change Function Declaration (*Rename Function/Method, Add/Remove Parameter, Change Signature*)	Names and parameters are the joints of the system. **Simple mechanics**: change all callers at once. **Migration mechanics**: new function, old one delegates + `@Deprecated`, migrate, delete.	`circum(r)` → `circumference(r)` + deprecated `circum` delegating
Encapsulate Variable (*Encapsulate Field, Self-Encapsulate Field*)	Turns "reorganise data" into "reorganise functions"; one place for validation/logging. First step for Global Data	`public static Owner defaultOwner` → private + `defaultOwner()` / `setDefaultOwner()`
Rename Variable	The wider the scope, the more the name matters	`double a = height * width` → `double area = ...`
Introduce Parameter Object	A Data Clump of parameters becomes one object — often the seed of a domain class	`(station, min, max)` → `(station, NumberRange range)` with `contains(v)`
Combine Functions into Class	Several functions share the same data argument → a class holding it as fields	`base(r)`, `taxableCharge(r)` → `ReadingCalculator(reading).base()`
Combine Functions into Transform	Read-only source data: one function returns an enriched copy with all derived values (use a class if the source can change — the copy would go stale)	scattered `base`, `taxable` → `enrichReading(r)` returning `EnrichedReading`
Split Phase	Code doing two things in sequence (parse then compute; compute then format) → two phases joined by an intermediate data structure. Cures Divergent Change; see 7.5 step 3	`priceOrder(line)` → `price(parse(line))` via `record OrderRequest`

7.3.2 Encapsulation (book chapter 7)

Deciding what each module *hides*: data structures, temps, relationships.

Refactoring	When / why	Before → After
Encapsulate Record (*Replace Record with Data Class*)	Raw maps/arrays/public-field structs expose layout	`Map.of("name", "Acme", ...)` → `record Organization(String name, String country)`
Encapsulate Collection	Getter returning the internal collection lets clients change it behind the owner's back	`getCourses()` returning the list → `courses()` unmodifiable + `addCourse` / `removeCourse`
Replace Primitive with Object (*Replace Data Value with Object, Replace Type Code with Class*)	A simple value soon needs validation/formatting/comparison. Cures Primitive Obsession	`priority().equals("high") || ...("rush")` → `enum Priority` with `higherThan(NORMAL)`
Replace Temp with Query	A temp is visible only in its function; a method is usable class-wide and eases extraction. Best if assigned once and side-effect free	`double basePrice = quantity * itemPrice` → `basePrice()` method
Extract Class (⇄ Inline Class)	A subset of fields/methods that go (and change) together. Cures Large Class, Divergent Change	`Person.officeAreaCode, officeNumber` → `record TelephoneNumber`
Inline Class (⇄ Extract Class)	Class no longer does enough; or merge-then-re-split	`Shipment.trackingInfo` → fields directly in `Shipment`
Hide Delegate (⇄ Remove Middle Man)	`a.getB().do()` couples client to B; add `a.do()`. Cures Message Chains	`person.getDepartment().getManager()` → `person.manager()`
Remove Middle Man (⇄ Hide Delegate)	Too many forwarders; let clients call the delegate	`person.manager()` → `person.department().getManager()`
Substitute Algorithm	Replace a whole body with a clearer algorithm or a library call (with tests proving they agree)	loop of `if equals "Don"/"John"/"Kent"` → `Set.of(...)` + `stream().filter(candidates::contains).findFirst()`

7.3.3 Moving Features (book chapter 8)

Moving code between classes, between functions, and within a function.

Refactoring	When / why	Before → After
Move Function (*Move Method*)	Put a function where the data it uses most lives. Cures Feature Envy, Shotgun Surgery	`Account.overdraftCharge()` logic → `AccountType.overdraftCharge(daysOverdrawn)`; `Account` delegates
Move Field	A field always passed alongside another object, or changed with it, belongs there (encapsulate first)	`Customer.discountRate` → `Plan.discountRate`; `Customer.discountRate()` delegates
Move Statements into Function (⇄ Move Statements to Callers)	Same statements run with every call → move them inside	title line at every caller → inside `photoData(p)`
Move Statements to Callers (⇄ Move Statements into Function)	A shared function needs different behaviour per caller → move the varying lines out (identical at first; vary later as a separate change)	`emitPhotoData` writes location → each caller writes location itself
Replace Inline Code with Function Call	Code repeats what an existing function does	flag loop over `states` → `states.contains("MA")`
Slide Statements (*Consolidate Duplicate Conditional Fragments*)	Keep related statements together (declare right before use); prepares Extract Function; a statement repeated in every branch slides out. Can't slide past interfering code	reorder so `pricingPlan` is declared next to `baseCharge`
Split Loop	A loop doing two things → two loops, each extractable. Double iteration is rarely a bottleneck	one loop for youngest + total salary → two loops
Replace Loop with Pipeline	`filter` / `map` / `reduce` name each step	skip-blank/comment loop → `lines.stream().filter(...).map(...).toList()`
Remove Dead Code	Unused code costs readers; version control remembers it. Don't comment it out	`if (false) { legacyDiscount(); }` → gone

7.3.4 Organizing Data (book chapter 9)

Refactoring	When / why	Before → After
Split Variable (*Remove Assignments to Parameters, Split Temp*)	A variable (not a loop counter/accumulator) assigned twice has two jobs; don't reuse parameters as locals	`temp` = perimeter, then area → `final double perimeter`, `final double area`
Rename Field	Field names explain the data structure; encapsulate first if widely used	`record Organization(String nm, ...)` → `(String name, ...)`
Replace Derived Variable with Query	Stored derived mutable data drifts out of sync; compute on demand (fine to store if the source is immutable)	`production += a.amount()` on every apply → `production()` sums the adjustments
Change Reference to Value (⇄ Change Value to Reference)	Inner object without identity → immutable **value object**, replaced wholesale, equal by value; safe to share across threads. Records are ideal	mutable `TelephoneNumber.setAreaCode` → `record TelephoneNumber` + `setTelephone(new ...)`
Change Value to Reference (⇄ Change Reference to Value)	Many copies of one logical entity must see updates → share one instance via a **repository**	`new Customer(id)` per order → `repo.findOrCreate(id)`

7.3.5 Simplifying Conditional Logic (book chapter 10)

Refactoring	When / why	Before → After
Decompose Conditional	Extract the condition and each branch into named functions (a special Extract Function)	summer date-range `if/else` → `isSummer(date) ? summerCharge() : regularCharge()`
Consolidate Conditional Expression	Several side-effect-free checks with the same result are one condition	three `return 0` checks → `if (isNotEligibleForDisability(e)) return 0;`
Replace Nested Conditional with Guard Clauses	Handle unusual cases first and return; the normal path reads straight down. "One exit point" is not worth unclarity	nested `if/else` in `payAmount` → `if (separated) return 0; if (retired) return 0; return normalPay(e);`
Replace Conditional with Polymorphism	Repeated switches on a type, or base behaviour with variants → class per case, dynamic dispatch. Leads to **Strategy/State** (7.6); 7.5 step 5	`switch (b.type())` in `speed()` → `sealed interface Bird` + one record per bird
Introduce Special Case (*Introduce Null Object*)	Many clients check the same special value (null/"unknown") and do the same thing → an object that answers itself. `Optional` shows absence in the type but callers still decide; a special case centralises the decision	`customer == null ? "occupant" : ...` → `UnknownCustomer`
Introduce Assertion	Make an assumption explicit; program must be correct without it. Java `assert` is off by default (`-ea`) — never use it for external input	`// discountRate never negative` → `assert discountRate >= 0 : ...`

7.3.6 Refactoring APIs (book chapter 11)

Refactoring	When / why	Before → After
Separate Query from Modifier	A value-returning function should have no observable side effects — **Command–Query Separation** (Meyer)	`alertForMiscreant` returns name *and* sets alarms → `findMiscreant()` (query) + `alertForMiscreant()` (modifier)
Parameterize Function (*Parameterize Method*)	Near-identical functions differing only in literals	`tenPercentRaise`, `fivePercentRaise` → `raise(p, factor)`
Remove Flag Argument (*Replace Parameter with Explicit Methods*)	A literal that picks the code path makes calls unreadable (`deliveryDate(order, true)`)	→ `rushDeliveryDate(o)`, `regularDeliveryDate(o)`; `setDimension("height", v)` → `setHeight(v)`
Preserve Whole Object	Don't unpack values from one object just to pass them (unless you want independence from its type)	`withinRange(low, high)` → `withinRange(room.daysTempRange())`
Replace Parameter with Query (*Replace Parameter with Method*; ⇄ Replace Query with Parameter)	The function can get the value itself — don't make callers pass it (unless it adds an unwanted dependency)	`discountedPrice(base, discountLevel())` → `discountedPrice(base)`
Replace Query with Parameter (⇄ Replace Parameter with Query)	Reaching into a global/singleton hurts testing and **referential transparency** — pass the value in	`Thermostat.INSTANCE.currentTemperature()` inside → `targetTemperature(plan, currentTemperature)`
Remove Setting Method	Field shouldn't change after construction → set in constructor, `final`	`setId(id)` → constructor + `final String id`
Replace Constructor with Factory Function (*Replace Constructor with Factory Method*)	Constructors have a fixed name, must return that exact class, need `new`. Factories can be named, return subclasses or cached instances (*Effective Java* Item 1; `List.of`, `Integer.valueOf`)	`new Employee(name, "E")` → `Employee.createEngineer(name)`
Replace Function with Command (*Replace Method with Method Object*; ⇄ Replace Command with Function)	Wrap a function in an object with `execute()`: holds state, supports undo/queueing, and splits a big function whose locals become fields	60-line `score(...)` → `new Scorer(c, exam, guide).execute()`
Replace Command with Function (⇄ Replace Function with Command)	A simple command adds nothing a function couldn't	`new ChargeCalculator(customer, usage).charge()` → `charge(customer, usage)`

7.3.7 Dealing with Inheritance (book chapter 12)

Refactoring	When / why	Before → After
Pull Up Method (⇄ Push Down Method)	Identical methods in siblings → superclass (make them identical first, or use Template Method)	`Salesman.name()`, `Engineer.name()` → `Employee.name()`
Pull Up Field (⇄ Push Down Field)	Same field in siblings (rename to match first)	`protected String name` in both → in `Employee`
Pull Up Constructor Body	Repeated init in subclass constructors → superclass constructor (Java 21 needs `super(...)` first; Java 25 relaxed this, JEP 513)	`Manager` sets name/id → `super(name, id); this.grade = grade;`
Push Down Method (⇄ Pull Up Method)	Method relevant only to some subclasses (callers must know the subclass)	`Employee.quota()` → `Salesman.quota()`
Push Down Field (⇄ Pull Up Field)	Field used by only one subclass	`Employee.quota` → `Salesman.quota`
Replace Type Code with Subclasses (*Extract Subclass, Replace Type Code with State/Strategy*; ⇄ Remove Subclass)	A type code driving behaviour. **Direct** (subclass the class) or **indirect** (subclass a separate type object — needed if the type changes at runtime or the class is already subclassed)	`Employee.type` → `sealed Employee permits Engineer, Manager, Salesman` + `create(name, type)` switch
Remove Subclass (*Replace Subclass with Fields*; ⇄ Replace Type Code with Subclasses)	Subclasses differing only in a constant aren't worth it	`Male`/`Female` returning `"M"`/`"F"` → `Person.genderCode` field
Extract Superclass	Two similar classes duplicate fields/methods (alternative: Extract Class + delegation; you can switch later)	`Department`, `Employee` → `abstract class Party`
Collapse Hierarchy	Parent and child no longer different enough. Cures Lazy Element, Speculative Generality	empty `Salesman extends Employee` → just `Employee`
Replace Subclass with Delegate	Inheritance varies only **one** axis and couples tightly; a delegate can vary independently and change at runtime — "favour composition over inheritance", the core of **State/Strategy**	`PremiumBooking extends Booking` → `Booking.premiumDelegate` set by `bePremium(extras)`
Replace Superclass with Delegate (*Replace Inheritance with Delegation*)	Subclass uses only part of its parent, or parent methods don't make sense (Refused Bequest, broken Liskov) → hold an instance and forward what you need	`Scroll extends CatalogItem` → `Scroll` holding a `catalogItem` delegate

7.3.8 Extra refactorings on refactoring.com

Five more in the online catalogue (not among the 61):

Refactoring	When / why	Before → After
Replace Control Flag with Break (*Remove Control Flag*)	A `found` flag only to stop a loop	flag → `break` / `continue` / `return`
Replace Error Code with Exception	Special codes (`-1` , `null`) get ignored by callers	`return -1` → `throw new InsufficientFundsException(...)`
Replace Exception with Precheck (*Replace Exception with Test*)	Exceptions are for exceptional cases; check first if it's easy	catch `ArrayIndexOutOfBoundsException` → bounds check
Replace Magic Literal (*Replace Magic Number with Symbolic Constant*)	A special-meaning literal needs a name and one home (not for obvious `0`); 7.5 step 4	`9.81` → `STANDARD_GRAVITY`
Return Modified Value	A function secretly updating shared state → return the new value; the data flow becomes visible	`calculateAscent();` → `totalAscent = calculateAscent();`

1st-edition names you may still hear

The old (1999) names are shown as *aliases* in every row above (e.g. Extract Method → Extract Function, Introduce Null Object → Introduce Special Case, Replace Inheritance with Delegation → Replace Superclass with Delegate, Remove Control Flag → Replace Control Flag with Break). The full old → new table is in the Full Edition.

7.4 Smell → refactorings lookup table

Bold = the usual first move.

Smell	Group	Refactorings
Mysterious Name	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
Duplicated Code	Dispensables	**Extract Function**, Slide Statements, Pull Up Method *(also: Replace Inline Code with Function Call, Substitute Algorithm)*
Long Function	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
Long Parameter List	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
Global Data	2nd-ed extra	**Encapsulate Variable** (then shrink scope)
Mutable Data	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value
Divergent Change	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
Shotgun Surgery	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class
Feature Envy	Couplers	**Move Function**, Extract Function
Data Clumps	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
Primitive Obsession	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
Repeated Switches	OO Abusers	**Replace Conditional with Polymorphism**
Loops	2nd-ed extra	**Replace Loop with Pipeline**
Lazy Element	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
Speculative Generality	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code
Temporary Field	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
Message Chains	Couplers	**Hide Delegate**, Extract Function, Move Function
Middle Man	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate
Insider Trading	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate
Large Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
Alternative Classes w/ Different Interfaces	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
Data Class	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
Refused Bequest	OO Abusers	Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
Comments	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

Which principle each smell family violates (Chapters 4–6):

Smell family	Principle violated
Divergent Change, Large Class	Single Responsibility
Repeated Switches, Shotgun Surgery	Open/Closed
Refused Bequest	Liskov Substitution
Long Parameter List with flags, fat interfaces	Interface Segregation / clean functions
Global Data, Mutable Data	Encapsulation / information hiding
Hard-wired `new` of concrete dependencies	Dependency Inversion
Duplicated Code	DRY
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter
Feature Envy, Data Class	Tell, Don't Ask

7.5 Worked example — from messy to clean in six compiling steps

The starting point

A courier prints invoices. Each shipment has a **service type** (`STD`, `EXP`, `FRG`), a weight and a distance.

Type	Charge	Loyalty points
STD	5 + 1.2/kg, + 0.05/km beyond 100 km	1 per full 100 km
EXP	12 + 2.0/kg, + 0.10/km beyond 50 km	1 per full 100 km, +2 bonus
FRG	8 + 1.5/kg + 3 handling; ×1.2 if > 10 kg	1 per full 100 km

New feature request: an HTML invoice now, and an OVERNIGHT service next quarter.

Step 0 (excerpt from `code/worked/step0/Main.java`):

```
    static String print(Inv inv) {
        double tot = 0;
        int pts = 0;
        int unused = 0;                                  // dead code
        StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
        for (Shp s : inv.list) {
            double amt = 0;
            switch (s.t) {
                case "STD":
                    amt = 5 + s.kg * 1.2;
                    if (s.km > 100) amt += (s.km - 100) * 0.05;
                    break;
                // ... EXP and FRG cases ...
                default:
                    throw new IllegalArgumentException("unknown type: " + s.t);
            }
            // loyalty points
            pts += s.km / 100;
            if ("EXP".equals(s.t)) pts += 2;
            // ... append the formatted line ...
            tot += amt;
        }
        // ... append "Amount owed" and "Points earned" ...
        return r.toString();
    }
```

Smell	Where
Mysterious Name	`Shp`, `Inv`, `t`, `kg`, `km`, `amt`, `tot`, `pts`, `r`
Long Function	`print` computes charges, points *and* formats
Divergent Change	HTML would copy everything; a new tariff edits the formatter
Repeated Switches (coming)	`"EXP"` checked in two places
Primitive Obsession	Service type is a `String`
Magic literals	`5`, `1.2`, `100`, `0.05`, ...
Data Class / Mutable Data	Public mutable fields in `Shp`, `Inv`
Dead code	`int unused`

So: **preparatory refactoring** first.

The safety net

`lld.ch07.worked.Main` (`code/worked/Main.java`) is a **characterisation check**: it records step 0's output as "golden" and compares every later step **byte for byte**. The fixture includes boundaries (exactly 100 km STD, exactly 10 kg FRG). Each step lives in its own package `step0` … `step5`.

Step 1 — Extract Function, Rename Variable/Field, Remove Dead Code

Rename Field (`t → type`, `kg → weightKg`, `km → distanceKm`; `Shp → Shipment`, `Inv → Invoice`), Rename Variable (`tot → totalAmount`, `pts → loyaltyPoints`, `r → result`), Extract Function ×3 (`amountFor`, `pointsFor`, `money`), Remove Dead Code. The switch was *moved*, not changed — **move before you change**.

Step 2 — Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements

Inline `thisAmount`; split the three-job loop; turn total and points into queries; make them streams; slide declarations away.

```
    static double totalAmount(Invoice invoice) {
        return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
    }
```

Trap: `mapToDouble(...).sum()` may use compensated summation and differ in the last bit; `reduce(0.0, Double::sum)` adds in order like the loop. (`IntStream.sum()` is exact.) `amountFor` now runs twice per shipment — ignore until a profiler says otherwise.

Step 3 — Split Phase, Encapsulate Record, Encapsulate Collection, Combine Functions into Transform

The key preparatory move: `StatementCalculator.calculate(Invoice)` (phase 1, a transform) → `StatementData` record → `TextRenderer.render(StatementData)` (phase 2, knows no tariffs). `Shipment` / `Invoice` become records; `Invoice` uses `List.copyOf`.

Step 4 — Replace Primitive with Object, Replace Magic Literal

The type code becomes `enum ServiceType { STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG") }` with `fromCode()`, so a bad code fails **at the boundary** (same exception and message; here shipments are always built and printed together, so behaviour is unchanged — check this in real code). Every tariff number gets a name (`STD_FREE_KM`, ...). The switch becomes an exhaustive **switch expression** with no `default`. (`x + 0` equals the old "if not taken" for all doubles except `-0.0`, which can't occur.)

Step 5 — Replace Conditional with Polymorphism, Move Function, then the new feature

Each switch branch becomes a `Tariff` class (a **Strategy**); `ExpressTariff` overrides `loyaltyPoints`, removing the second `"EXP"` check. Move Function puts `charge()` / `loyaltyPoints()` on `Shipment` (Feature Envy cured). The calculator has **no conditionals**.

```java
public sealed interface Tariff permits StandardTariff, ExpressTariff, FragileTariff {

    double charge(double weightKg, int distanceKm);

    /** Default behaviour shared by most tariffs; ExpressTariff overrides it. */
    default int loyaltyPoints(int distanceKm) {
        return distanceKm / 100;
    }
}

public record ExpressTariff() implements Tariff {
    private static final double BASE = 12, PER_KG = 2.0, FREE_KM = 50, PER_EXTRA_KM = 0.10;
    private static final int BONUS_POINTS = 2;
    // ... charge(...) ...
    @Override
    public int loyaltyPoints(int distanceKm) {
        return Tariff.super.loyaltyPoints(distanceKm) + BONUS_POINTS;
    }
}

public record Shipment(ServiceType type, double weightKg, int distanceKm) {
    // ... compact constructor: requireNonNull(type) ...
    public double charge() { return type.tariff().charge(weightKg, distanceKm); }

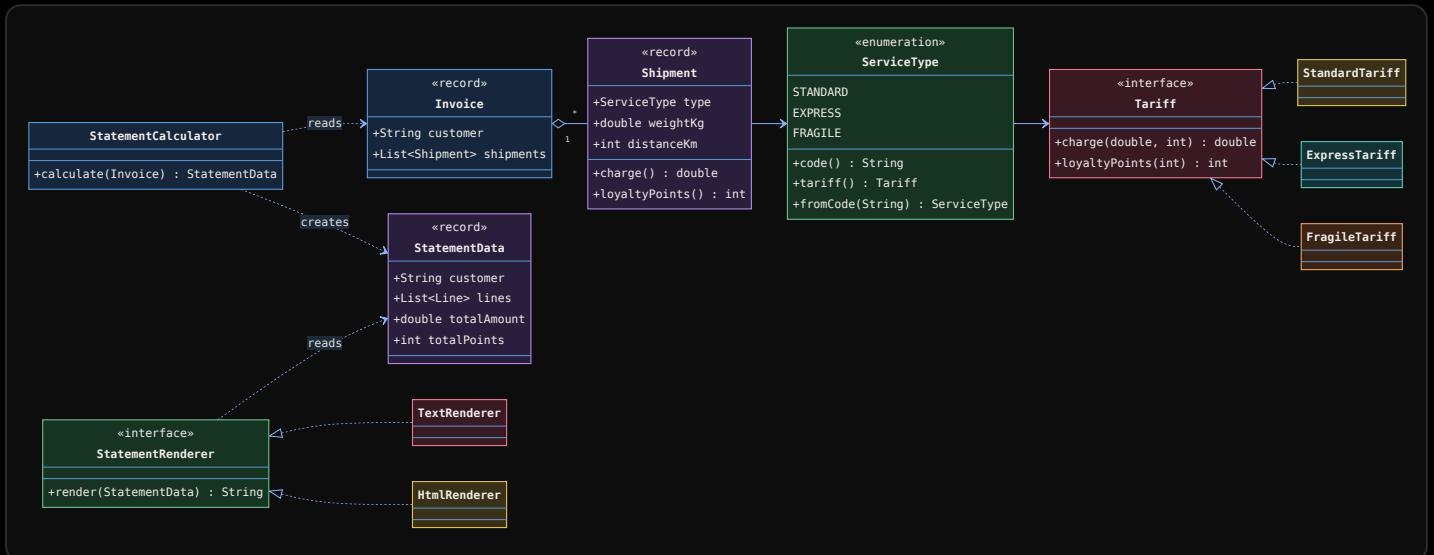
    public int loyaltyPoints() { return type.tariff().loyaltyPoints(distanceKm); }
}

/** Phase 1. No switch left: adding a tariff never touches this class (Open/Closed). */
public final class StatementCalculator {

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), s.charge()))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(Shipment::loyaltyPoints).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }
}
```

Full runnable code: code/worked/step5/ (package `lld.ch07.worked.step5`; see the Full Edition).

Then, **adding-function hat**: extract a `StatementRenderer` interface, add a ~15-line `HtmlRenderer` reusing phase 1 unchanged. OVERNIGHT later = one `Tariff` record + one enum constant + one name in `permits`. Calculator and renderers don't change (Open/Closed).



What changed, step by step

Step	Refactorings	Smells removed
1	Extract Function ×3, Rename Variable/Field, Remove Dead Code	Mysterious Name, part of Long Function, dead code
2	Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements	Long Function, Loops
3	Split Phase, Combine Functions into Transform, Encapsulate Record/Collection	Divergent Change, Data Class, Mutable Data
4	Replace Primitive with Object, Replace Magic Literal	Primitive Obsession, magic numbers
5	Replace Conditional with Polymorphism, Move Function (+ extract renderer interface)	Repeated Switches, Feature Envy

The checker prints `SAME output` for step0 … step5 and ends with `All steps preserve behaviour.`

Deliberately not done: money stays `double`. `BigDecimal` would change rounding — a behaviour change for the other hat, with its own tests and a product decision.

7.6 Refactoring toward patterns

What it is

You often **arrive at a pattern by refactoring** instead of designing it up front. Kerievsky's *Refactoring to Patterns* (2004) catalogues these moves, mostly compositions of Fowler's refactorings. Don't force patterns early (Speculative Generality); refactor *toward* one when a smell appears — and *away* when it stops earning its keep.

How it works

Smell	Toward	Via
Switch choosing an algorithm	**Strategy**	Extract Function per branch → Replace Conditional with Polymorphism (*Replace Conditional Logic with Strategy*)
Methods checking and changing a status field	**State**	Replace Type Code with Subclasses (indirect) → Replace Conditional with Polymorphism (*Replace State-Altering Conditionals with State*)
`if (x == null)` everywhere	**Null Object**	Introduce Special Case
Siblings with same skeleton, different steps	**Template Method**	Extract Function → Pull Up Method (*Form Template Method*)
Confusing constructors	**Creation Methods**	Replace Constructor with Factory Function
Scattered `new ConcreteX()`	**Factory**	+ Move Function to a factory class (*Encapsulate Classes with Factory*)
Complex (Composite) object built by many ordered calls	**Builder**	*Encapsulate Composite with Builder*
Huge function with shared locals; undo/queue	**Command**	Replace Function with Command
Logging/caching mixed into a core class	**Decorator**	*Move Embellishment to Decorator*
Hard-coded notifications / polling	**Observer**	*Replace Hard-Coded Notifications with Observer*
Same job, different interfaces	**Adapter**	*Extract Adapter*, *Unify Interfaces with Adapter*
Subclass explosion for feature combinations	**Strategy / Decorator / Bridge**	Replace Subclass with Delegate
Pattern no longer earning its keep	*Remove it*	Inline Class, Collapse Hierarchy (*Inline Singleton*)

Package `lld.ch07.towardpatterns` (`code/towardpatterns/`) shows five of these with before and after side by side and checks they agree.

Replace Conditional Logic with Strategy

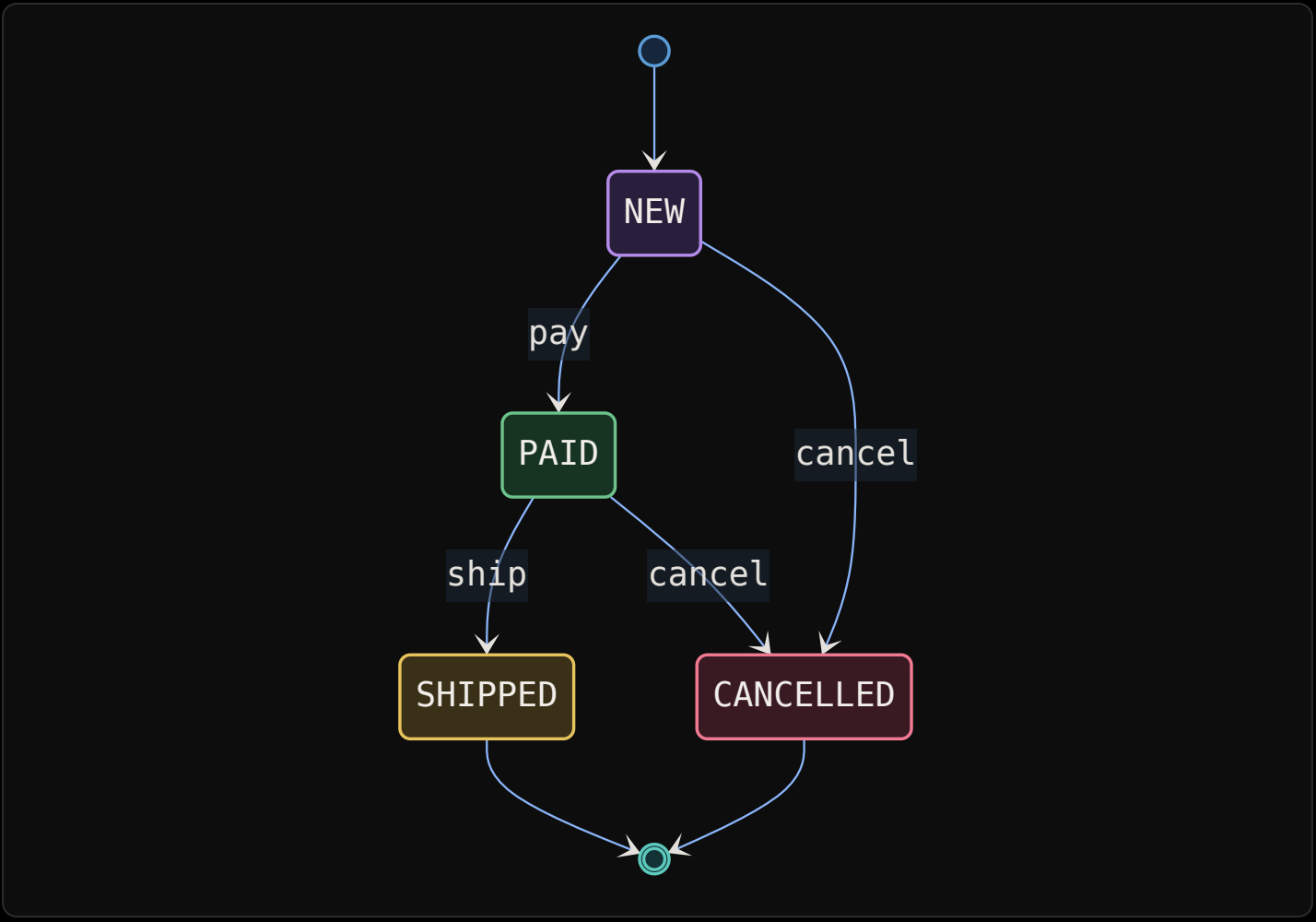
A `switch (tier)` becomes a `DiscountPolicy` per tier stored in an `EnumMap`; new tiers are **registered**, not coded.

```
@FunctionalInterface
interface DiscountPolicy {
    double apply(double amount);
}
// ...
static final class PriceCalculator {
    private final Map<Tier, DiscountPolicy> policies = new EnumMap<>(Tier.class);
    private final DiscountPolicy fallback = amount -> amount;
    // ... constructor registers GOLD, SILVER, NEW lambdas ...
    /** New tiers are registered, not coded into a switch (Open/Closed). */
    void register(Tier tier, DiscountPolicy policy) { policies.put(tier, policy); }

    double price(Tier tier, double amount) {
        return policies.getOrDefault(tier, fallback).apply(amount);
    }
}
```

Replace State-Altering Conditionals with State

Each state is an enum constant overriding only the moves it allows; default methods reject the rest. `Order` just does `state = state.pay(this)`.




```
enum States implements OrderState {
    NEW {
        @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    PAID {
        @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    SHIPPED, CANCELLED
}
```

Strategy vs State: same structure, different intent — a strategy is chosen by the client and stays; a state replaces *itself* as the object moves through its lifecycle.

Introduce Special Case (Null Object)

`find(id)` returns an `UnknownCustomer` answering `"occupant"` / `"basic"`, so clients have no null checks.

Form Template Method

`legacyCsv` and `legacyMarkdown` repeat header/rows/footer. After: `Exporter.export()` is `final` and calls abstract `header()`, `row()`, `footer()`; `CsvExporter` and `MarkdownExporter` fill them in.

Replace Constructors with Creation Methods

`new LegacyLoan(5000, false, 12)` — what is `false`? After: private constructor + `Loan.termLoan(5000, 12)` and `Loan.revolver(2000)`.

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as
if-chains: true
All pattern refactorings preserve behaviour.
```

Full runnable code: code/towardpatterns/ (package `lld.ch07.towardpatterns`; see the Full Edition).

Small smell cures (package `lld.ch07.smells`)

`code/smells/` shows `Money` (Primitive Obsession), `DateRange` (Data Clumps), and `Order` with `shippingCity()` (Hide Delegate), `Line.total()` (Move Function) and a read-only `lines()` (Encapsulate Collection).

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

7.7 Putting it together

Trade-offs

Benefit	Cost / risk
Easier to change; features get faster (Design Stamina)	Takes time now; pay-off later, hard to measure
Bugs become visible	Without tests you can introduce bugs
Design evolves with requirements	Renames/moves cause merge conflicts on long branches
Small steps are safe, can stop anytime	Feel slow to beginners
Small, understandable units	Too much indirection → Middle Men, Lazy Elements
Removes duplication	Premature abstraction → the *wrong* abstraction, costlier than duplication

When not to use it

Not for code you don't touch; not without a safety net; not instead of a rewrite when the code doesn't work at all; not on published interfaces without a migration path; never mixed with behaviour changes in the same step.

Common mistakes

Mistake	Do this instead
Calling any change "refactoring"	Separate refactoring commits from feature/fix commits
Big-bang refactoring	Tiny steps, test after each, commit often
Refactoring without tests	Characterisation tests first; IDE-automated refactorings
Text search-and-replace renames	Symbol-aware IDE Rename; check reflection/config/serialisation
Abstracting on the first duplicate	Rule of Three
Changing rounding/ordering "while there" (`DoubleStream.sum()`, `toList()`)	Know the replacement API's exact semantics
Patterns everywhere "for flexibility"	Refactor toward a pattern when a smell demands it

Real-world examples

- **IDEs** automate catalogue entries; **OpenRewrite** (Spring Boot upgrades, JUnit 4 → 5) and Google's **Refaster** do it at scale.
- **JDK:** `Stack extends Vector`, `Properties extends Hashtable` — Refused Bequest (use `ArrayDeque`). `Date` → `java.time` replaced mutable primitive-ish types with immutable values, as a *new* API beside the published old one.
- **Strangler Fig** for monolith-to-services migrations.

In the interview

> **30-second answer — "What is refactoring?"** "Changing the internal structure of code to make it easier to understand and modify, without changing observable behaviour. I do it in very small steps, each compiling and passing tests, so the system is never broken. Smells like Long Function or Repeated Switches tell me where; named refactorings like Extract Function or Replace Conditional with Polymorphism tell me how. Tests are the safety net, and the goal is economic: clean code lets us add features faster."

- **Q: How do you refactor code with no tests?** A: Characterisation tests first; break dependencies at a seam (inject an interface instead of `new`); until then use only IDE-automated refactorings; Sprout Method/Class for new code.
- **Q: What would you refactor here?** (machine coding) A: Name smell + cure: "`processOrder` is a Long Function → Extract Function; the `vehicleType` switch in three places is Repeated Switches → Replace Conditional with Polymorphism (Strategy); `double price` is Primitive Obsession → `Money`; `getParkingLot().getFloor(1).getSpot(3)` is a Message Chain → Hide Delegate."
- **Q: How do you rename a public API method used by other teams?** A: Migration mechanics: new method, old one delegates and is `@Deprecated`, migrate callers, then delete. For REST: versioning or expand/contract.

Self-check

1. **Q:** Fixing a bug while tidying a method — one refactoring? **A:** No; the fix changes behaviour. Two hats, separate steps and commits.
2. **Q:** `book(hotel, from, to, 2, 1, true)` — smells? **A:** Long Parameter List, Data Clumps, flag argument → Introduce Parameter Object (`Stay`, `Guests`), Remove Flag Argument / an enum like `MealPlan`.
3. **Q:** Why `reduce(0.0, Double::sum)` and not `DoubleStream.sum()`? **A:** `sum()` may use compensated summation and differ in the last bit; `reduce` adds in order like the loop.
4. **Q:** An IDE field rename broke the JSON response — was it a refactoring? **A:** It was meant to be, but the JSON name is observable behaviour, so no. Check serialisation, reflection and config.

TL;DR

- **Refactoring = small, behaviour-preserving changes**, tests after every step, never mixed with behaviour changes (two hats).
- **When:** Rule of Three, preparatory, comprehension, litter-pickup, code review — mostly opportunistic. **Not** for untouched code or without a safety net.
- **24 smells** say *where*: Mysterious Name, Duplicated Code, Long Function, Long Parameter List, Global Data, Mutable Data, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Loops, Lazy Element, Speculative Generality, Temporary Field, Message Chains, Middle Man, Insider Trading, Large Class, Alternative Classes with Different Interfaces, Data Class, Refused Bequest, Comments.
- **61 refactorings** say *how*; the everyday core: Extract/Inline Function, Rename, Move Function, Extract Class, Split Phase, Replace Conditional with Polymorphism.
- **Performance:** clean first, then profile. **Rewrite** rarely, with a Strangler Fig. **Legacy code** = no tests → characterisation tests, seams, sprout/wrap.
- **Patterns** emerge: switch → Strategy/State, null checks → Null Object, duplicated skeleton → Template Method, confusing constructors → factory methods.

7.8 Full source code and output

The complete listings (checker, steps 0–5, toward-patterns, smell cures) are in the Full Edition (README.md, section 7.8). Compile and run with Java 21:

```
cd chapters/07-refactoring/code
javac --release 21 -d /tmp/lld_07 $(find . -name '*.java')
java -cp /tmp/lld_07 lld.ch07.worked.Main          # characterisation check over all six steps
java -cp /tmp/lld_07 lld.ch07.worked.step5.Main    # final design + new HTML feature
java -cp /tmp/lld_07 lld.ch07.towardpatterns.Main  # refactoring toward patterns
java -cp /tmp/lld_07 lld.ch07.smells.Main          # small smell cures
```

Packages: `code/worked/` (checker), `code/worked/step0/`, `code/worked/step1/`, `code/worked/step2/`, `code/worked/step3/`, `code/worked/step4/`, `code/worked/step5/` (steps 0–4 print the text invoice only), `code/towardpatterns/`, `code/smells/`.

Sources: Fowler, *Refactoring*, 2nd ed. (2018) and refactoring.com; refactoring.guru's smell groups; Feathers, *Working Effectively with Legacy Code* (2004); Kerievsky, *Refactoring to Patterns* (2004).

Chapter 8 — Design Patterns: Introduction & Creational Patterns (Study Edition)

> **Study Edition.** A shorter version of Chapter 8 with every topic kept and the padding removed. The full chapter (README.md) has the complete code, full outputs and more follow-ups.
>
> **What you will learn.** What a design pattern is, where the idea came from, all 23 GoF patterns, the OO principles behind them, how to pick (and not pick) a pattern, anti-patterns, and the creational patterns in depth: Singleton, Simple Factory (idiom), Factory Method, Abstract Factory, Builder, Prototype and the bonus Object Pool.
>
> **Code.** `chapters/08-patterns-creational/code/lld/ch08/<pattern>/`, one `Main` per package (Java 21: `javac --release 21`, then `java lld.ch08.<pattern>.Main`).

Part A — Design patterns: the big picture

8.1 What is a design pattern?

What it is

A **design pattern** is a named, reusable description of a good solution to a recurring design problem. It is not a library or paste-in code — more like a *recipe* you adapt. (Analogy: builders learned over hundreds of houses where the front door works best; writing that down, with reasons and costs, is a pattern.)

- **Intent** — the one-sentence purpose ("ensure a class has only one instance").
- **Participants** — the roles that take part (*Creator*, *Product*, *Client*).

The problem it solves

Without patterns every team re-solves the same problems in slightly different, slightly buggy ways. Patterns give proven solutions, readable code, built-in good principles (loose coupling, open/closed), adaptability and faster decisions — and a **shared vocabulary** ("use an Observer" replaces a paragraph), the biggest interview win.

How it works

Every GoF pattern has four parts: **Name**, **Problem** (when to apply, forces in tension), **Solution** (participants and collaborations — a template, not code), **Consequences** (costs and benefits — a pattern without trade-offs is half a pattern). You adapt the template to your language and problem.

When not to use it

Not every problem needs a pattern. Forcing one adds classes and indirection — worse than none (see 8.6).

TL;DR

Pattern = **name + problem + solution template + consequences**. Value: vocabulary and proven trade-offs. Risk: overuse.

8.2 A short history: from buildings to the "Gang of Four"

Year	Event
1977	Architect **Christopher Alexander**, *A Pattern Language* — named solutions for towns and buildings
1987	**Kent Beck** and **Ward Cunningham** apply pattern languages to Smalltalk UI design
1994	**Gamma, Helm, Johnson, Vlissides** — the **Gang of Four (GoF)** — publish *Design Patterns*: **23 patterns** in three categories
2004	*Head First Design Patterns* — the same patterns in Java, OO principles front and centre

The GoF's two central principles:

1. **Program to an interface, not an implementation.**
2. **Favour object composition over class inheritance** ("has-a" black-box reuse over "is-a").

Creational patterns make principle 1 practical: if code must not name concrete classes, *someone* still has to create the objects.

8.3 The three categories and all 23 GoF patterns

What it is

Category	In one line
Creational	Control creation so the system doesn't depend on concrete classes or construction details
Structural	Compose classes/objects into larger, flexible structures
Behavioral	Assign responsibilities and design communication

Second GoF axis, **scope**: **class patterns** use inheritance, fixed at compile time (Factory Method, class Adapter, Template Method, Interpreter); **object patterns** use composition, changeable at run time (the rest).

The full catalogue

#	Pattern	Category	Intent
1	**Abstract Factory**	Creational	Create *families* of related objects without naming concrete classes
2	**Builder**	Creational	Build a complex object step by step, separate from its representation
3	**Factory Method**	Creational	Subclasses decide which class to instantiate via an overridable method
4	**Prototype**	Creational	Create objects by copying a configured instance
5	**Singleton**	Creational	Exactly one instance with a global access point
6	**Adapter** (Wrapper)	Structural	Convert one interface into another
7	**Bridge**	Structural	Split abstraction from implementation so both vary
8	**Composite**	Structural	Treat single objects and trees uniformly
9	**Decorator**	Structural	Add responsibilities dynamically by wrapping
10	**Facade**	Structural	Simple entry point to a complex subsystem
11	**Flyweight**	Structural	Share intrinsic state across huge numbers of objects
12	**Proxy**	Structural	Stand-in controlling access to another object
13	**Chain of Responsibility**	Behavioral	Pass a request along handlers until one handles it
14	**Command**	Behavioral	Request as an object — queue, log, undo
15	**Interpreter**	Behavioral	Grammar as classes; evaluate sentences
16	**Iterator**	Behavioral	Traverse without exposing storage
17	**Mediator**	Behavioral	Centralise communication between objects
18	**Memento**	Behavioral	Capture/restore state without breaking encapsulation
19	**Observer**	Behavioral	Notify dependents when one object changes
20	**State**	Behavioral	Behaviour changes with internal state
21	**Strategy**	Behavioral	Interchangeable algorithms behind one interface
22	**Template Method**	Behavioral	Base class fixes the skeleton, subclasses fill steps
23	**Visitor**	Behavioral	Add operations without changing the classes

Creational → this chapter, structural → Ch. 9, behavioral → Ch. 10–11. **5 + 7 + 11 = 23.** (Some course lists show ten behavioral patterns, omitting the rarely used Interpreter.) **Not GoF but common:** Simple Factory (idiom), Object Pool, Dependency Injection, Null Object, Repository, MVC (a "compound pattern" of Observer + Strategy + Composite).

Self-check

- **Q:** Which creational pattern is a *class* pattern? **A:** Factory Method — subclasses override the creation method.
- **Q:** How many behavioral GoF patterns? **A:** Eleven.

8.4 The OO principles behind every pattern

Principle	Patterns applying it
Encapsulate what varies	Factory Method, Strategy, State, Builder
Composition over inheritance	Abstract Factory (client *holds* a factory), Strategy, Decorator
Program to interfaces	Every creational pattern returns an interface
Loose coupling	Observer, Mediator, factories
Open–Closed	Factory Method, Decorator
Dependency Inversion	All factories — the client never calls `new ConcreteThing()`
Least Knowledge	Facade
Hollywood Principle	Template Method, Factory Method, frameworks/IoC
Single Responsibility	Builder, Iterator

Why creational patterns exist: DIP says don't depend on concrete classes, but `new PdfDocument()` *is* such a dependency — so creational patterns move that `new` somewhere controlled (a factory, subclass, builder, prototype or container).

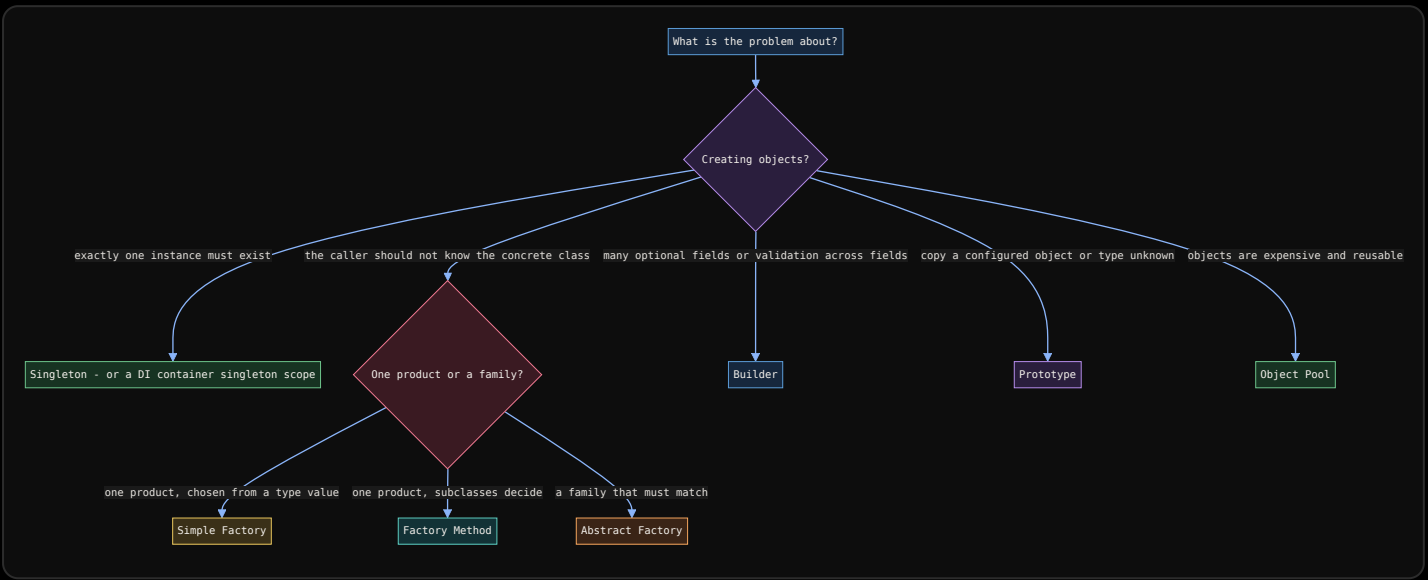
8.5 How to pick a pattern

The problem it solves

Learning 23 patterns and hunting for places to use them is backwards. Start from a **signal** in the requirements.

How it works

1. **Name the problem** in plain words ("pricing will change", "exactly one of these", "ten-argument constructor").
2. **Decide the category** — creating, structuring or behaving.
3. **Match the signal** to a pattern.
4. **Check the cost** — is there something simpler (enum, lambda, constructor, DI container)?
5. **Say what you rejected and why** — worth as much as the pick.



(For fitting objects together: Adapter, Decorator, Facade; for behaviour: Strategy, Observer, State.)

The six patterns that come up in almost every LLD interview

Learn each with its **signal** — forcing a pattern where it doesn't fit costs more than not knowing it.

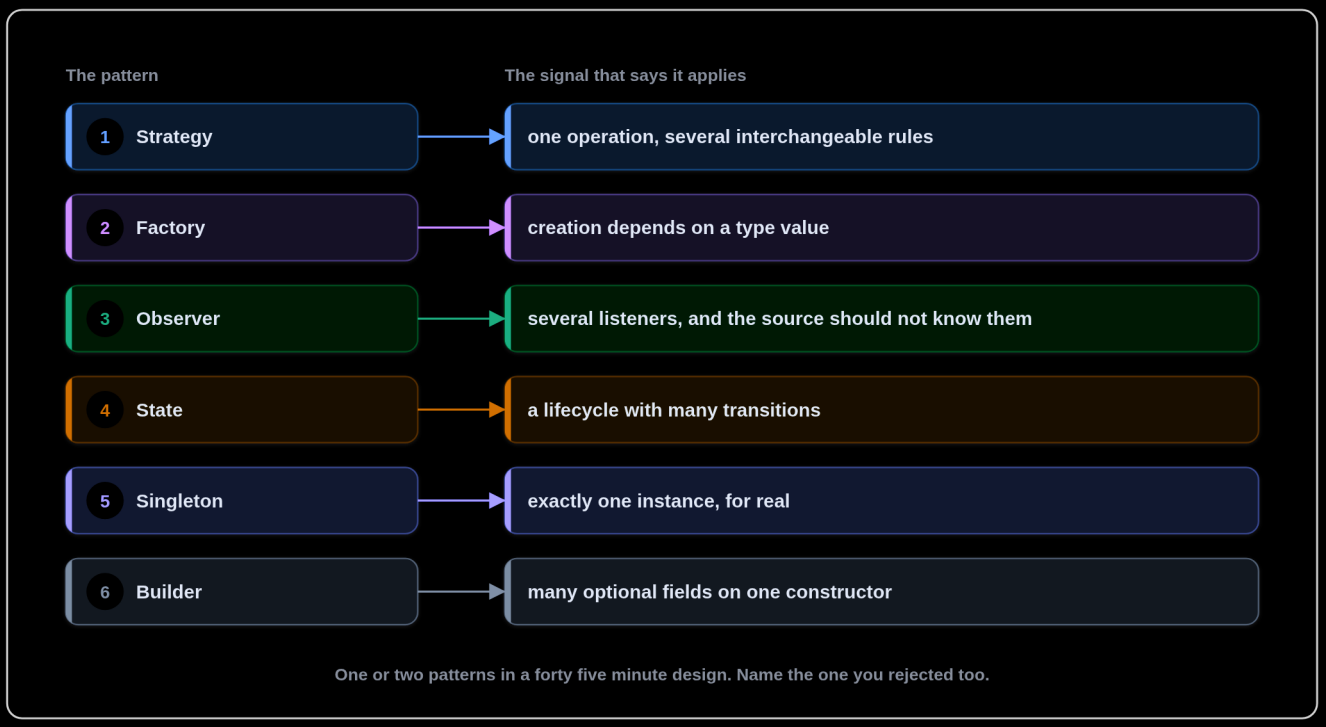


Figure — the six interview patterns and the signal that tells you each one applies (course diagram)

Pattern	Signal	Use when	Don't use when
Strategy	One operation, several swappable versions chosen at run time	Several interchangeable rules	Only one rule, no hint of a second
Factory	Something must be created; the caller shouldn't care which class	Class depends on a type value	Only one concrete class
Observer	One object changes, several others must react unknown to it	Several listeners	A single listener already referenced
State	Behaviour depends on status; allowed moves are fixed	Many states and transitions	3–4 states handled by one guarded method
Singleton	Genuinely one instance	Must be exactly one	You just want a global handle
Builder	Many optional fields; constructor keeps growing	Lots of optional fields	A few required fields

In a parking-lot / library design: **Strategy** — a `ParkingRate` interface (`HourlyRate`, `WeekendRate`, `MonthlyPass`), the answer to "what if pricing changes?"; **Factory** — type value → `Vehicle`, keep it lean (a creation method is often enough in 45 min); **Observer** — spot as subject, boards subscribe (but one reserved-book member needs only a direct call); **State** — ticket active → paid → closed (with 3–4 states an `enum` + guard is clearer); **Singleton** — one `ParkingLot`, raise thread safety and testability yourself; **Builder** — a `BookItem` past ~6 arguments.

Rules of thumb: one or two patterns per 45-minute design, placed where their signal appears; **compare out loud** — "Strategy for rates because a second tariff is coming; no State for the ticket because one guard handles three statuses."

8.6 Anti-patterns: when patterns go wrong

What it is

An **anti-pattern** is a common "solution" that looks reasonable but reliably causes more problems than it solves.

Anti-pattern	Looks like	Fix
God Object (Blob)	`SystemManager` with 80 methods — violates SRP, untestable	Extract Class (Ch. 7); move behaviour to data owners
Golden Hammer	Favourite tool/pattern applied everywhere	Start from the problem's signal (8.5)
Pattern overuse ("patternitis")	Factories for one class, interfaces "just in case" — **Speculative Generality**	YAGNI: add the pattern when the *second* variant arrives
Singletonitis	Singletons as globals — hidden deps, order-dependent tests	Dependency injection (8.8)
Cargo-cult patterns	Structure without intent (a "Builder" that validates nothing)	Know the property the pattern gives you, or drop it

TL;DR

Patterns are a vocabulary, not a checklist. God Object comes from ignoring design; Golden Hammer and overuse from over-applying it.

Part B — Creational patterns

8.7 Creational patterns at a glance

What it is

Creational patterns control *how objects are created*, hiding *which* class is instantiated, *how* it is assembled, and sometimes *whether* a new object is made. The system becomes independent of creation, hides creation complexity, and stays flexible about what/who/how/when.

Pattern	Intent
Singleton	Exactly one instance, one global way to reach it
Factory Method	Base class defines a creation method; subclasses choose the class
Abstract Factory	One factory object creates a matching *family*
Builder	Assemble one complex object step by step
Prototype	Create by cloning a prototypical instance
Simple Factory (idiom)	One method with a switch/registry picks the class
Object Pool (bonus)	Reuse a bounded set of expensive objects

8.8 Singleton

What it is

Singleton restricts a class to **exactly one instance** (per process / class loader) with **one well-known access point** — like a country's one government. **Eager** = create at class initialisation; **lazy** = on first request.

GoF intent and participants. Intent: never more than one instance; a well-known way to reach it. One participant, **Singleton**: a class-level `getInstance()` operation, usually also creating the instance. Clients just call it.

The problem it solves

Some things are one per process: settings loaded from a file, a connection pool, a metrics registry. If billing, reporting and the mailer each build their own `ConfigStore`, the file is parsed three times and one service's change never reaches the others. A plain global has two problems: tests share it (results depend on test order), and someone must assign it before use (wrong start-up order → `null`). Singleton makes **the class decide how many instances exist and create the one it allows**, on demand.

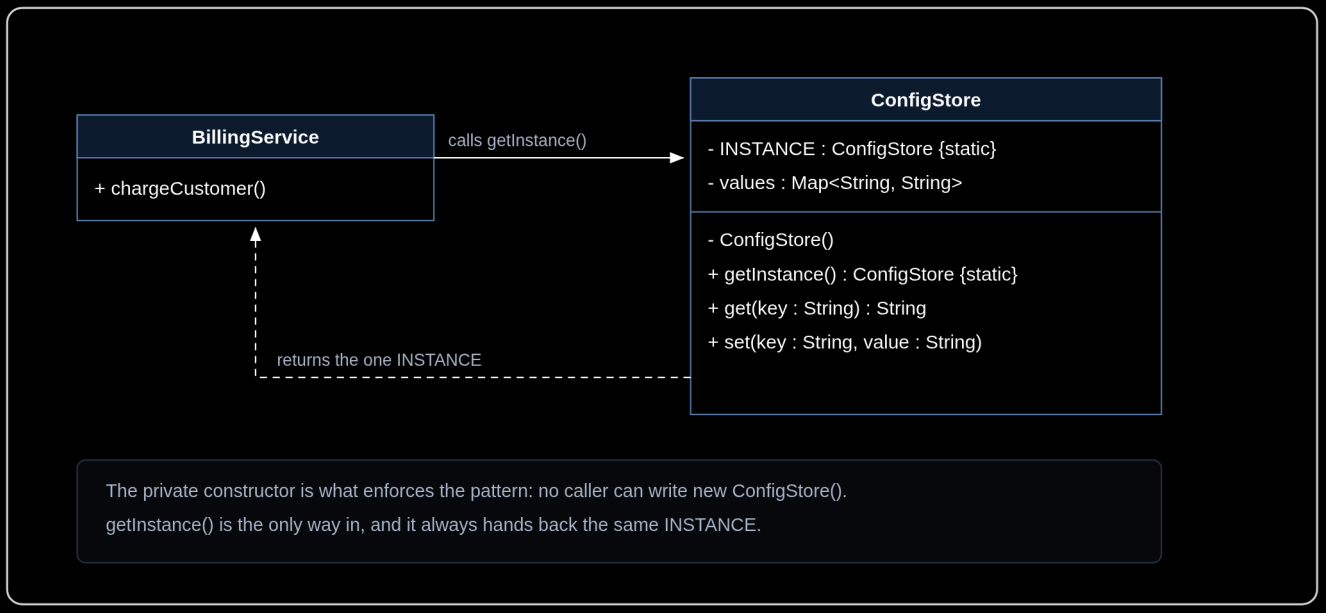
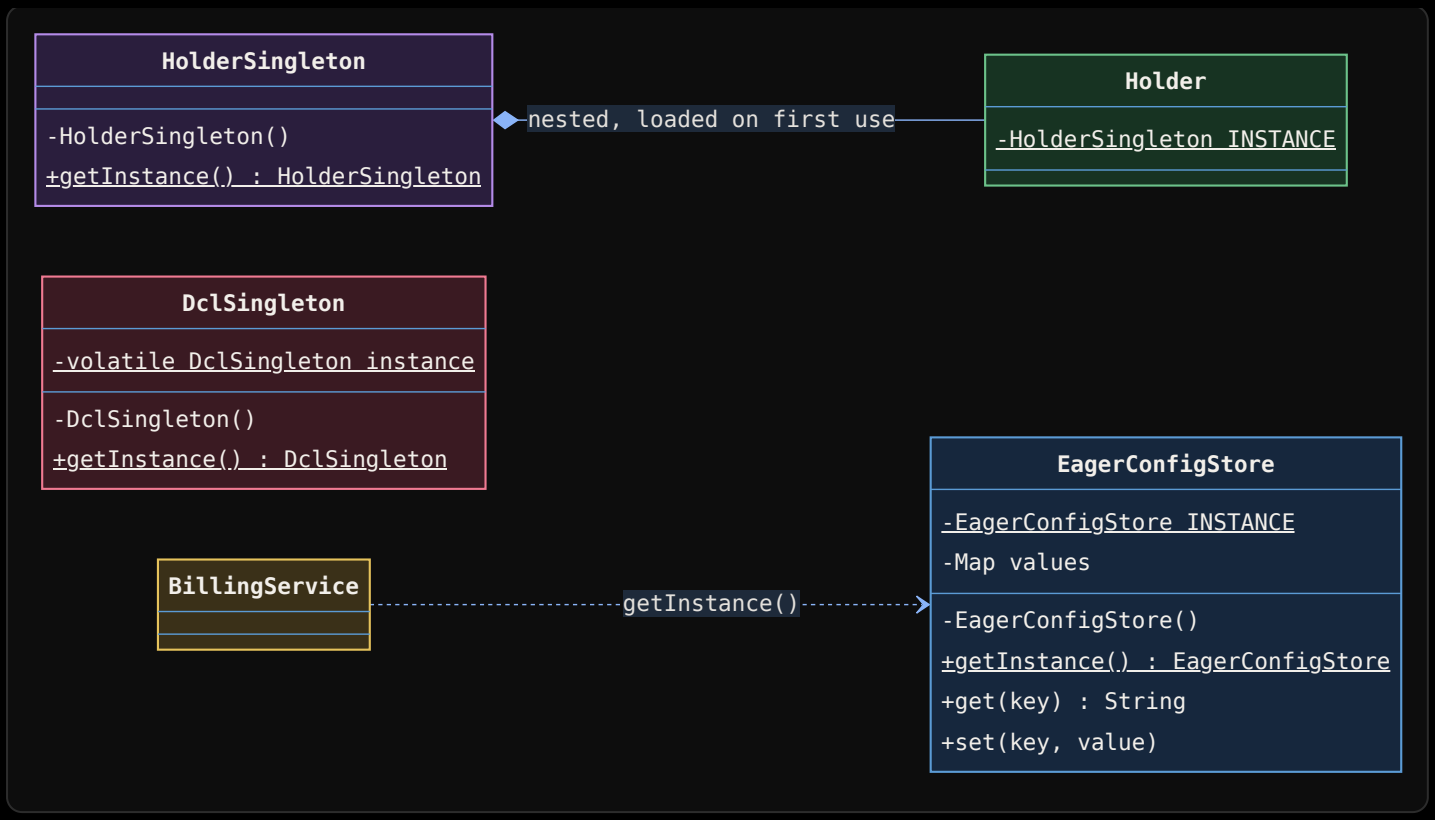


Figure — ConfigStore with a private static INSTANCE, a private constructor and a static getInstance(); BillingService only calls getInstance() (course diagram)

How it works

1. `private` **constructor** — the whole enforcement mechanism.
2. The instance in a `private static` **field** (owned by the class).
3. A `public static getInstance()` (static: callers have no instance yet).
4. Return the instance, creating it (safely) if needed.
5. Ordinary instance methods for the real work.



The six ways to write a Singleton in Java

#	Variant	Lazy?	Thread-safe?	Per-call cost	Survives serialization / reflection?	Verdict
1	**Eager** `static final`	No	Yes — class init runs once	None	No / No	Default when construction is cheap
2	**Naive lazy** `if (instance == null)`	Yes	**No**	None	No / No	Never in multi-threaded code
3	`synchronized` **getInstance()**	Yes	Yes	Lock on *every* call	No / No	Correct but slow under contention
4	**Double-checked locking** + `volatile`	Yes	Yes (Java 5+)	One volatile read	No / No	When creation needs run-time input
5	**Bill Pugh holder**	Yes	Yes — class loading	None	No / No	Best lazy option in plain Java
6	`enum`	No	Yes	None	**Yes / Yes**	Shortest, most robust (*Effective Java*)

- **Eager:** set during class *initialisation* (first active use, not JVM start); JLS §12.4.2 guarantees once-only, safely published. Cost: built even if the class is touched for another reason.
- **Naive lazy:** *check-then-act* — two threads both see `null` and both construct. Passes single-threaded tests; our 8-thread demo built **8** objects.
- `synchronized`: fixes the race but locks forever for a first-call problem.
- **DCL:** check without the lock, lock, **check again**. The field **must be** `volatile`: object creation is allocate → construct → store, and without `volatile` another thread may see the reference before the constructor's writes (a half-built object). `volatile` gives a **happens-before** edge. Copying to a local reads the volatile once on the fast path.
- **Holder:** `private static final class Holder { static final X INSTANCE = new X(); }` and `getInstance()` returns `Holder.INSTANCE`; `Holder` isn't initialised until first touched — lazy, thread-safe, **no locks, no** `volatile`.

- **Enum** (`enum EnumConfigStore { INSTANCE; ... }`): one instance per constant, reflective creation refused, serialization special-cased. Can't extend a class; not truly on-demand.

Which to pick: default **eager**; **holder** once you've *measured* a start-up cost; **DCL** when creation needs a run-time value (and always `volatile`); **enum** for free serialization/reflection safety.

```
/** 4. Double-checked locking: lock only on the first calls. volatile is mandatory. */
final class DclSingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static volatile DclSingleton instance;
    // ... private constructor ...
    static DclSingleton getInstance() {
        DclSingleton local = instance;          // one volatile read on the fast path
        if (local == null) {                    // 1st check, no lock
            synchronized (DclSingleton.class) {
                local = instance;
                if (local == null) {            // 2nd check, under the lock
                    local = new DclSingleton();
                    instance = local;           // volatile write publishes a fully built object
                }
            }
        }
        return local;
    }
}
```

Breaking a Singleton — and the fixes

Attack	How	Fix
Reflection	`setAccessible(true).newInstance()` calls the private constructor	Throw from the constructor if `INSTANCE` exists (reliable for eager/holder, racy for lazy), or **enum**
Serialization	`readObject()` builds a new object without the constructor	`private Object readResolve() { return INSTANCE; }` (+ `transient` fields), or **enum**
Cloning	A `Cloneable` ancestor exposing `clone()`	Override `clone()` to throw (not an issue if no ancestor is `Cloneable` — `Object.clone()` is protected)
Multiple class loaders	Each loader has its own copy	Load from a shared parent loader, or let a container own it
Multiple processes	One per JVM — four servers, four instances	External coordinator (DB, Redis, ZooKeeper) for system-wide uniqueness

`HardenedSingleton` adds all three defences (constructor guard, `readResolve`, throwing `clone`). Output: the naive lazy version under 8 threads — `constructor ran 8 time(s)` (varies; anything above 1 is the bug), DCL — `1`; the hardened and enum versions print `reflection -> blocked` and `serialization -> same? true`.

Subclassing a Singleton

GoF: possible with a **`protected`** constructor, where `getInstance()` picks the subclass **once, at run time** — `AppLogger` chooses `PlainLogger` or `JsonLogger` from a system property. With a `private` constructor it can't be subclassed.

The dependency-injection alternative

Often the best singleton is **not a Singleton class**: a normal class, **one** instance created at the **composition root**, passed in through constructors. Dependencies become visible; tests pass a fake (`new BillingService(testConfig)`). Spring beans are **singleton-scoped by default** — one per bean definition **per container**, not per JVM, and nothing stops `new`. Spring's **`prototype` scope** creates a fresh instance per lookup.

Java implementation

The package also has `EagerConfigStore`, `EnumConfigStore`, `BreakableSingleton`/`HardenedSingleton`, `AppLogger` and `DiAlternative`.

Full runnable code: code/lld/ch08/singleton/ (see the Full Edition).

Trade-offs

Pros	Cons
Controlled access — one instance, one documented door	**Global state** — shared mutable state anyone can change
You choose when — eager or lazy	**Awkward to test** — needs a reset/injection hook that ships to production
Fits shared resources — pool, cache, device handle	**Invisible dependencies** — `getInstance()` inside isn't in the constructor
Setup paid once per process	**Uniqueness ends at the process** (class loader)
Subclassable if designed in (`protected`)	**Hard to undo**; mixes lifecycle management with real work (SRP)

Often confused with: a **static utility class** (not an object — can't implement an interface or be mocked); **DI singleton scope** (same result, better mechanism); **Multiton** (`Logger.getLogger(name)` — one per key); Prototype (opposite intent). Elsewhere: Python module object, Go `sync.Once`, C++ Meyers Singleton.

When not to use it

- **The motive is convenience** — a parameter delivers access just as well.
- **Mutable state tests touch** — tests pass alone, fail in the suite.
- **A framework manages lifecycles** — use singleton scope.
- **Multiple processes** — four servers = four "singletons"; using one for distributed locks or unique IDs is a **correctness bug**.

Common mistakes

Naive lazy in multi-threaded code; DCL without `volatile`; `Serializable` without `readResolve()`; mutable state without thread-safe structures (`ConfigStore` uses `ConcurrentHashMap`); a `public static reset()` "for tests"; assuming one per system rather than per JVM/class loader.

Real-world examples

- **JDK:** `Runtime.getRuntime()` (textbook), `Desktop.getDesktop()`; `Collections.emptyList()` (singleton-like shared constant); `Logger.getLogger(name)` (multiton).
- **Not singletons despite `getInstance()`:** `Calendar.getInstance()`, `NumberFormat.getInstance()`, `Cipher.getInstance()` return **new objects** — static factory methods (8.9).
- Spring singleton beans; connection pools, metrics registries; *Head First*'s chocolate boiler.

In the interview

> **30-second answer:** "Singleton guarantees one instance per JVM (strictly, per class loader) with a single access point: private constructor, static accessor. I'd use eager initialisation — the class loader makes it thread-safe for free — or the holder idiom if construction is expensive; an enum if it must survive serialization or reflection. But it's global state with hidden dependencies, so in a Spring app I'd make it a singleton-scoped bean and inject it."

- *Q: Why does DCL need `volatile`?* — A: Otherwise the reference can be published before the constructor's writes are visible; `volatile` gives a happens-before edge.
- *Q: Unique IDs across 10 servers?* — A: Not a Singleton (one per JVM). Use a DB sequence, Snowflake-style IDs (timestamp + machine id + counter) or UUIDs.

Self-check

- **Q:** Which keyword enforces the pattern? **A:** `private` on the constructor.
- **Q:** Lazy, lock-free, thread-safe variant? **A:** The holder idiom.
- **Q:** What does `readResolve` do? **A:** Replaces the deserialized object with the real instance.

TL;DR

Private constructor + static accessor = one per JVM. Prefer **eager**, then **holder**, then **enum**; **DCL needs** `volatile`; never naive lazy. Know the ways to break it. Mention the costs; DI singleton scope is usually better.

8.9 Simple Factory (an idiom, not a GoF pattern)

What it is

One class (usually one static method) takes a type value and returns the matching object as a common interface. Very common, often right, but **not** GoF — *Head First* calls it an idiom. Analogy: a vending machine — press "B3", get the snack.

The problem it solves

The `switch` turning `"sms"` into `new SmsNotification()` gets copied everywhere. A simple factory **centralises** the decision — it doesn't **remove** it: adding a channel still edits the factory. A registry softens this (types are *registered* at start-up).

How it works

1. Product interface (`Notification`). 2. Concrete classes. 3. One `create(type)` — an exhaustive `switch` over an enum, or a `Map<String, Supplier<Notification>>` registry. 4. Callers hold the interface, never `new`. 5. Reject unknown types loudly.

Java implementation

```java
/** Simple Factory (an idiom, not a GoF pattern): one static method owns the "which class?" decision. */
public final class NotificationFactory {

    public enum Channel { EMAIL, SMS, PUSH }

    private NotificationFactory() { }

    /** Version 1: a switch. Adding a channel means editing this method (and the enum). */
    public static Notification create(Channel channel) {
        return switch (channel) {                    // exhaustive switch: the compiler checks every enum value
            case EMAIL -> new EmailNotification();
            case SMS   -> new SmsNotification();
            case PUSH  -> new PushNotification();
        };
    }

    /** Version 2: a registry of suppliers. New types are registered, not coded into a switch. */
    private static final Map<String, Supplier<? extends Notification>> REGISTRY = new ConcurrentHashMap<>(Map.of(
            "email", EmailNotification::new,
            "sms", SmsNotification::new,
            "push", PushNotification::new));

    public static void register(String type, Supplier<? extends Notification> supplier) {
        REGISTRY.put(type.toLowerCase(Locale.ROOT), supplier);
    }

    public static Notification create(String type) {
        Supplier<? extends Notification> s = REGISTRY.get(type.toLowerCase(Locale.ROOT));
        if (s == null) throw new IllegalArgumentException("Unknown notification type: " + type);
        return s.get();
    }
}
```

- No `default` in the enum switch — the compiler flags an unhandled constant.
- The registry maps names to constructor references; `register("whatsapp", ...)` adds a channel without editing the factory (a lambda works because `Notification` has one abstract method).
- `Locale.ROOT` avoids the Turkish dotless-"i" lower-casing surprise.

Full runnable code: code/lld/ch08/simplefactory/ (see the Full Edition).

Trade-offs

Simple Factory	Factory Method
One method, usually a switch	Decision spread across subclasses
New type = edit the factory	New type = add a class
No inheritance	Creator subclass per product
Not GoF	GoF

Short, stable, config-selected type list → Simple Factory is perfectly respectable. Other developers/modules adding types without editing your class → Factory Method.

When not to use it

Only **one** concrete class (call the constructor); objects differ only in **data** (one class with a field).

Common mistakes

Calling a switch-based factory a "Factory Method"; a `default` that returns `null` or a silent default; letting the factory grow into a God class.

Real-world examples

Static factory methods (*Effective Java* item 1) — not GoF Factory Method, usually not singletons: `Calendar.getInstance()` (new object per call), `EnumSet.noneOf` (hidden `RegularEnumSet`/`JumboEnumSet`), `List.of`, `Integer.valueOf`, `Executors.newFixedThreadPool`; Spring `@Bean` methods.

In the interview

> **30-second answer:** "A simple factory is one method that maps a type value to a concrete class and returns it as an interface. It centralises the switch so callers don't depend on concrete classes. It's an idiom, not GoF — adding a type still edits it. For new types without editing, I'd use a registry of suppliers or Factory Method."

- *Q: Is it Open–Closed?* — A: Not strictly; a registry makes it open via registration.
- *Q: Static factory vs constructor?* — A: Names, caching (`Integer.valueOf`), can return subtypes.

Self-check

- **Q:** What does it centralise but not remove? **A:** The switch — knowledge of every concrete type.
- **Q:** Why no `default` in the enum switch? **A:** So the compiler flags unhandled constants.

TL;DR

One `create(type)` returning an interface. Great for small stable sets. Not GoF. `Supplier` registry = extensible variant.

8.10 Factory Method

What it is

A base class defines a creation method but **subclasses decide the concrete class**; the base class uses the product without naming its type. Analogy: a franchise manual — take order, *prepare the dish* (each branch's own), serve, bill (*Head First*'s `PizzaStore.orderPizza()` with regional `createPizza()`).

GoF intent and participants. Intent: declare an operation that creates an object but let subclasses choose the class — instantiation is deferred to them. **Product** (`Document`), **ConcreteProduct** (`PdfDocument`), **Creator** (`ReportGenerator` — declares the factory method and *calls* it from its own operations), **ConcreteCreator** (`PdfReportGenerator` — overrides it).

> **Not a "static factory method".** `List.of`, `Calendar.getInstance` and Simple Factories are **one static method that decides** — not overridable, no Creator hierarchy. GoF Factory Method is an **instance method overridden in subclasses** and called by the base class. No inheritance + overriding → not this pattern.

The problem it solves

`new PdfDocument()` inside `ReportGenerator` hard-wires PDF. The generator gathers rows, **creates a document**, renders, announces — only creation depends on format. A `switch` works until a second and third method need the same switch. Factory Method leaves a deliberate **gap**: the full procedure in the base class calls an abstract method each subclass fills.

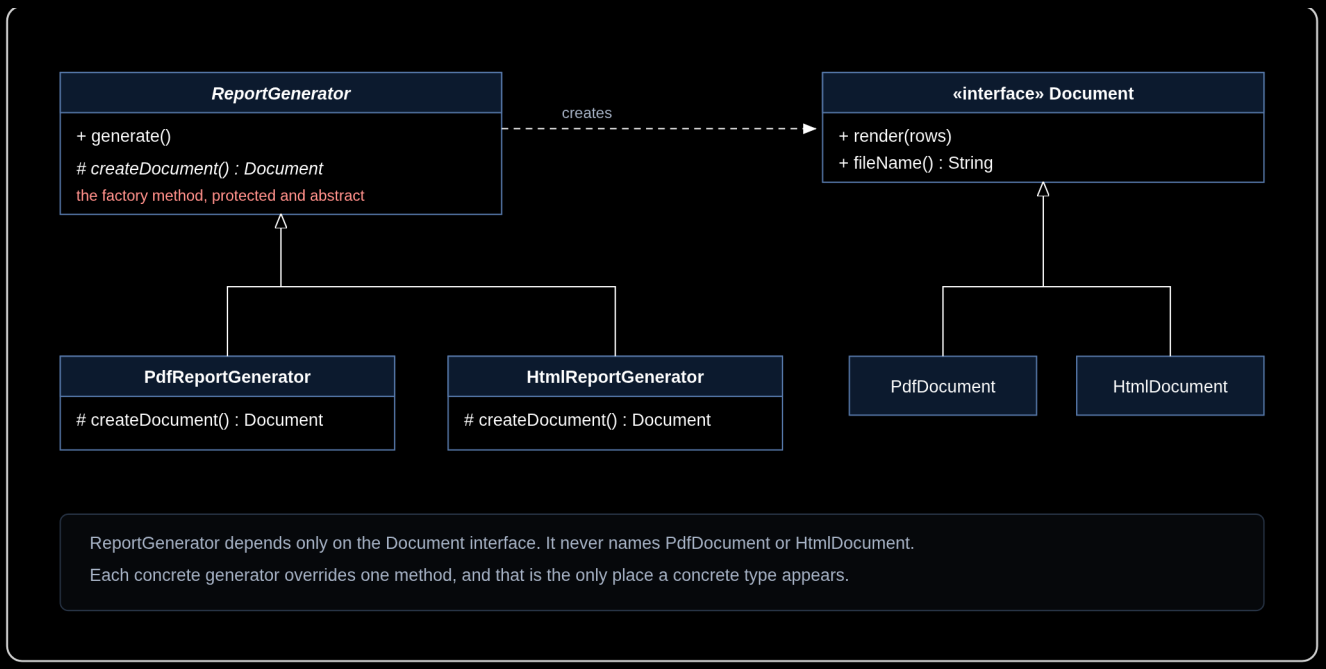
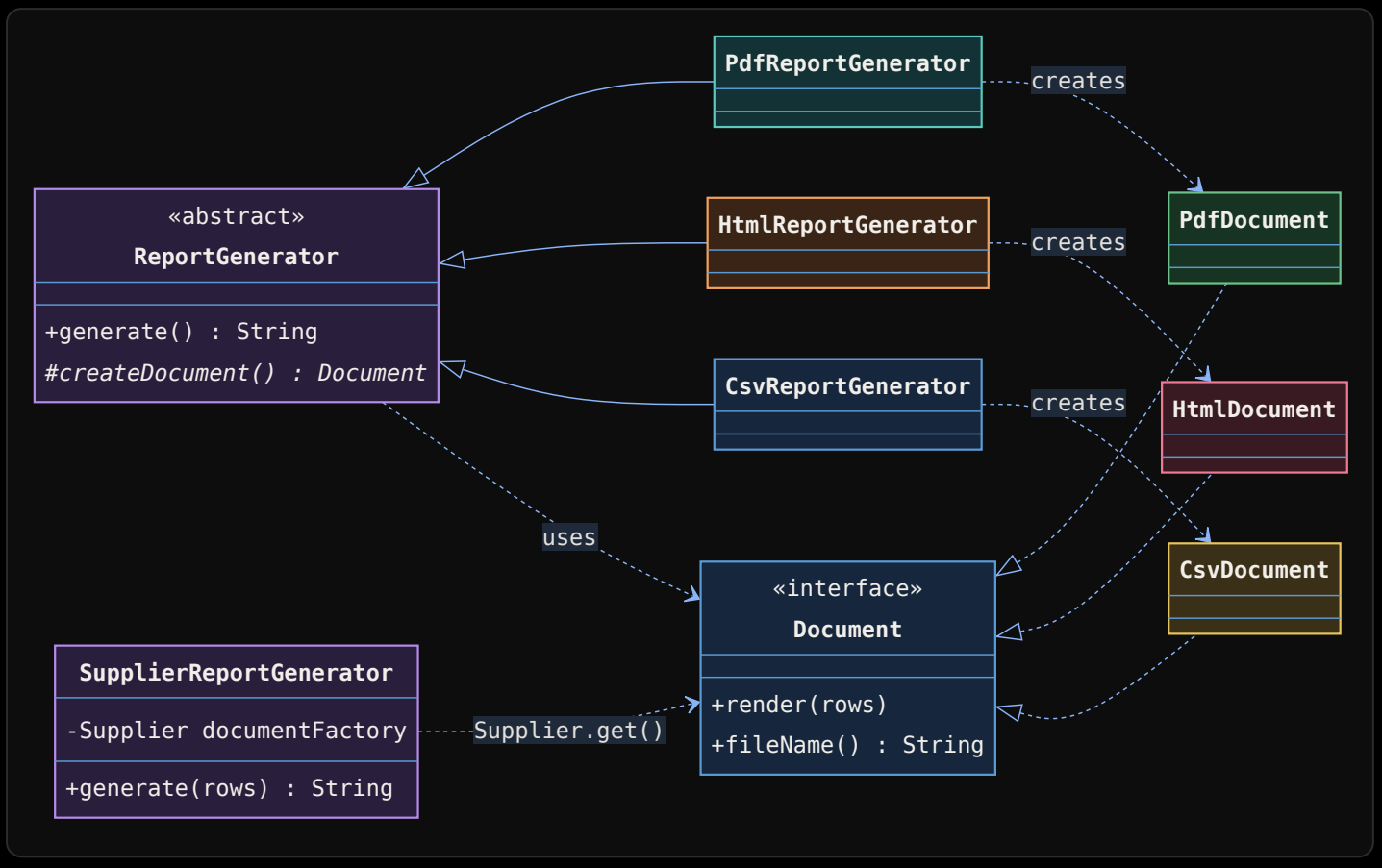


Figure — ReportGenerator declares the protected factory method createDocument(); each concrete generator overrides it to return its own Document (course diagram)

How it works

1. Product interface (`Document`: `render(rows)`, `fileName()`). 2. Concrete products. 3. Abstract creator with an **`abstract`** **`protected`** `createDocument()` (an extension point, not API). 4. Shared logic in the creator (`generate()`) calling it. 5. One concrete creator per product — the *only* place a concrete type is named. 6. The client picks a creator once, then uses `ReportGenerator` and `Document`.



Java implementation

```java
/** Creator: writes the whole procedure once and leaves one hole - the factory method. */
public abstract class ReportGenerator {

    /** The factory method. Protected: a hook for subclasses, not public API. */
    protected abstract Document createDocument();

    /** The template that USES the factory method. It never names a concrete document. */
    public final String generate() {
        System.out.println("  Collecting rows for the monthly sales report...");
        List<String> rows = List.of("north,120", "south,95", "east,140");
        Document document = createDocument();
        document.render(rows);
        System.out.println("  Report ready: " + document.fileName());
        return document.fileName();
    }
}

final class PdfReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new PdfDocument(); }
}
// ... HtmlReportGenerator, CsvReportGenerator (CSV added later - no existing class edited) ...

/** The modern, composition-based variant: pass the "factory method" in as a Supplier. */
public final class SupplierReportGenerator {
    private final Supplier<? extends Document> documentFactory;
    // ... constructor ...
    public String generate(List<String> rows) {
        Document d = documentFactory.get();
        d.render(rows);
        return d.fileName();
    }
}
```

`generate()` is `final` — subclasses change *what* is created, not the procedure. `CsvDocument` + `CsvReportGenerator` were added with no edits (Open–Closed). The `Supplier` variant composes better and a test can pass a lambda. In the output, every generator prints the same "Collecting rows…" line — that code exists once.

Full runnable code: code/lld/ch08/factorymethod/ (see the Full Edition).

Trade-offs

Pros	Cons
Extensible without edits — new product = new class	**More classes** — a creator per product
Common procedure in one place	**Indirection** — which subclass am I holding?
Test seam — subclass returns a fake	**Relies on inheritance** (one superclass)
Nobody holds the full type list	**Tempting too early** — all cost with one product

Often confused with: Simple Factory (one static switch); Abstract Factory (a *family* via a passed-in factory); **Template Method** (same hook mechanism but varies a *step* — `generate()` *is* one); Builder (many calls). In languages with first-class functions, passing a creation function is usually the better default.

When not to use it

One product type and no second coming (most frequent over-design); the base class has no shared work; types differ by data (use `Map<String, Supplier<Document>>`); you need matching objects (Abstract Factory).

Common mistakes

A **public** factory method (callers bypass the procedure); calling it from the base **constructor** (subclass fields not yet initialised); calling a switch-in-one-method a Factory Method.

Real-world examples

- **JDK:** `Collection.iterator()` — each collection (ConcreteCreator) returns its own `Iterator` (ConcreteProduct); `AbstractList.iterator()` built on abstract `get(int)`.
- **JDK XML:** `DocumentBuilderFactory.newDocumentBuilder()`, `TransformerFactory.newTransformer()` — factory methods inside an Abstract Factory (`newInstance()` itself is a *static* factory).
- Spring `FactoryBean<T>.getObject()`; Android `Fragment.onCreateView()`.

In the interview

> **30-second answer:** "Factory Method puts a creation hook in a base class. The base class writes the procedure once against a product interface; each subclass overrides the protected factory method to return its product. New types are new classes — Open–Closed. It pays off only with real shared logic in the base class; otherwise a constructor or simple factory is simpler. In modern Java I'd often pass a `Supplier`."

- *Q: vs Simple Factory?* — A: One static switch that knows every type vs subclasses overriding a hook the base calls. Raise this yourself — calling a switch a Factory Method is a classic slip.
- *Q: How do you test `generate()`?* — A: Subclass the creator (or pass a `Supplier`) returning a fake `Document`.

Self-check

- **Q:** The only place a concrete product is named? **A:** Each concrete creator's override.
- **Q:** What must the base class have for this to be worthwhile? **A:** Shared logic using the product.
- **Q:** Adding `CsvReportGenerator` shows which principle? **A:** Open–Closed.

TL;DR

Base class = procedure + abstract `protected createX()`; subclasses pick the type. One product, varied by inheritance. `Supplier` is the light modern form.

8.11 Abstract Factory

What it is

An interface for creating **families of related objects** that must be used together, without naming concrete classes. Choosing a family = choosing a factory. Analogy: a furniture catalogue — order from "Victorian" and every piece matches. (Course: one factory per OS theme; *Head First*: a regional pizza-ingredient factory.)

GoF intent and participants. Intent: one interface to create a set of related objects without naming their classes. **AbstractFactory** (`StorageFactory` — one create method per product kind), **ConcreteFactory** (`PostgresFactory`), **AbstractProduct** (`Connection`, `QueryBuilder`), **ConcreteProduct** (`PostgresQueryBuilder`), **Client** (`ReportService` — only abstract types). GoF notes one ConcreteFactory per family is typical, hence often a Singleton (passing it in is better).

The problem it solves

Some objects are only correct with partners from the same variant: Postgres quotes identifiers with `"`, MySQL with backticks; macOS buttons with macOS checkboxes; fake payments + fake mailer + fake clock, never a mix. Pairing `PostgresConnection` with `MySqlQueryBuilder` compiles and fails later at the database. Factory Method varies **one** product; Abstract Factory keeps a **set** consistent. *It doesn't help you choose the right combination — it removes the ability to build a wrong one.*

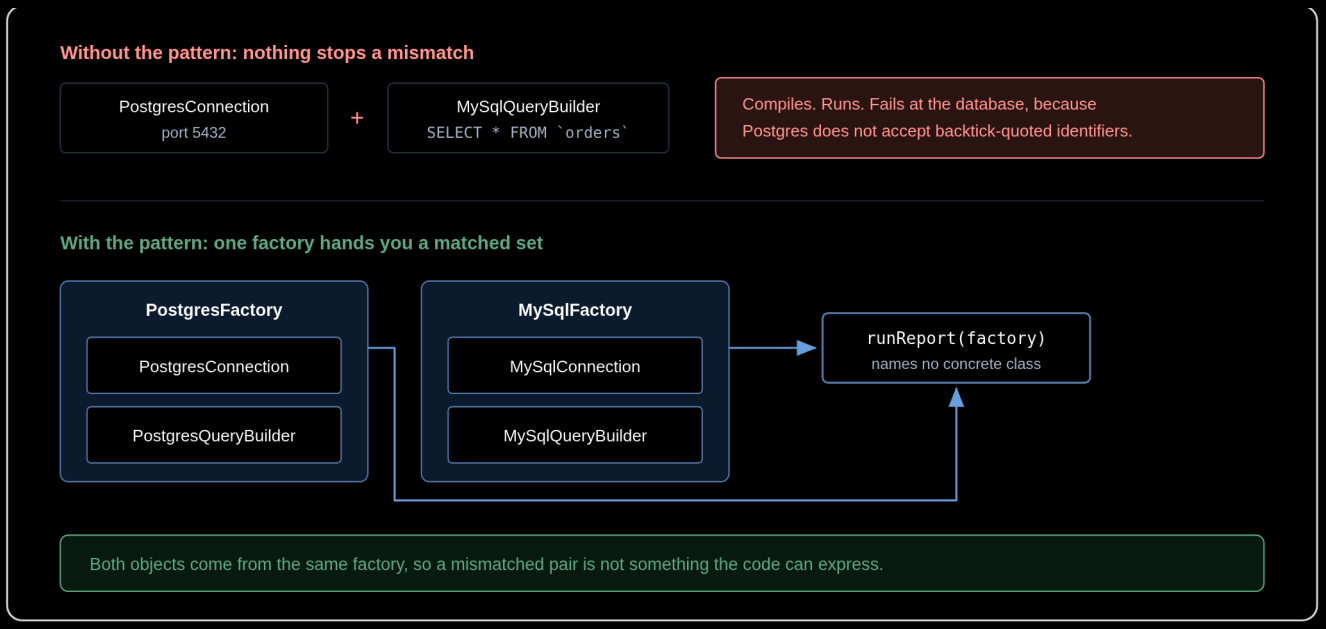
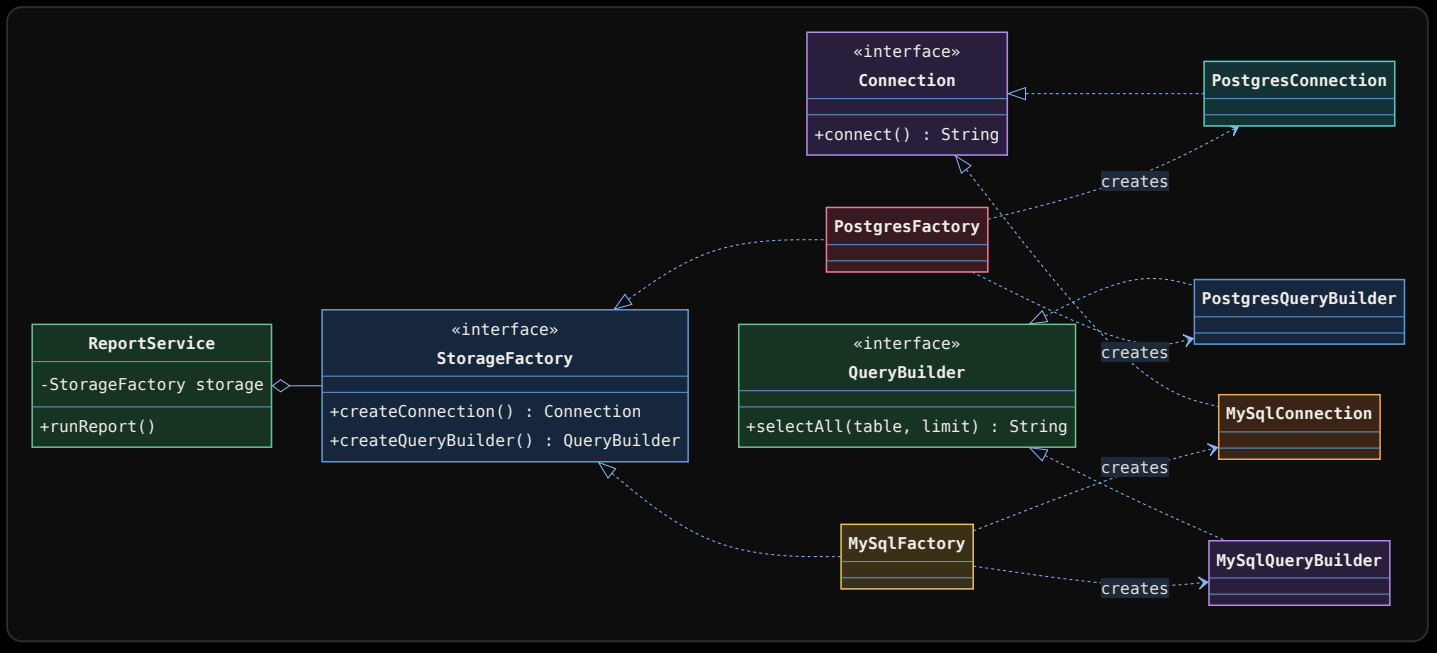


Figure — without the pattern a PostgresConnection can be paired with a MySqlQueryBuilder and fail at the database; with it, each factory hands out a matched set (course diagram)

How it works

1. Interface per **product kind**. 2. **Abstract factory** with one create method per kind. 3. Per variant, concrete products and a **concrete factory implementing every method** (covariant returns document the family). 4. The **client** receives a factory (constructor) and uses only abstract types. 5. Choose the factory **once at the composition root** (near `main`) and pass it down. (A correct Abstract Factory returns **all** products of its family — not one each.)



Java implementation

```java
/** Abstract factory: one creation method per product in the family. */
public interface StorageFactory {
    Connection createConnection();
    QueryBuilder createQueryBuilder();
}

/** Concrete factories implement EVERY method and return only their own family (covariant return types). */
final class PostgresFactory implements StorageFactory {
    @Override public PostgresConnection createConnection()     { return new PostgresConnection(); }
    @Override public PostgresQueryBuilder createQueryBuilder() { return new PostgresQueryBuilder(); }
}
// ... MySqlFactory, FakeStorageFactory ...

/** Client: holds only the abstract factory and abstract products. It cannot mix families. */
public final class ReportService {
    private final StorageFactory storage;

    public ReportService(StorageFactory storage) { this.storage = storage; }

    public void runReport() {
        Connection connection = storage.createConnection();
        QueryBuilder queries = storage.createQueryBuilder();
        System.out.println("  " + connection.connect());
        System.out.println("  " + queries.selectAll("orders", 10));
    }
}
```

`ReportService` has **no branch** on the database. `Main.chooseFactory(db)` — the composition root — holds the only `switch`. `FakeStorageFactory` swaps the **whole environment** for tests in one line.

Full runnable code: code/lld/ch08/abstractfactory/ (see the Full Edition).

Trade-offs

Pros	Cons
Wrong combinations can't be written	**Many types** — interface per kind + class per product per family
One switch for a whole environment (real↔fake, Postgres↔MySQL, themes)	**New product kinds are expensive** — edit every factory
Client never names a concrete class (DIP)	**Another layer to trace**
Test seam — one fake factory	**Pointless with one family**

The asymmetry: new **family** (`SqliteFactory`) = easy, Open–Closed; new **product kind** (`createMigrationRunner()`) = edit every factory.

Abstract Factory vs Factory Method

	Factory Method	Abstract Factory
Creates	**One** product	A **family**
Mechanism	**Inheritance** — subclass overrides	**Composition** — client holds a factory
Who chooses	Creator subclass	Factory passed in
Guarantees	New types without editing the creator	Products from one factory are compatible
New variant	New creator subclass + product	New factory + one product per kind
New product kind	n/a	Edit the interface and **every** factory
Example	`Collection.iterator()`	`DocumentBuilderFactory`, UI toolkits, DB dialects

Often confused with: Builder (one product, many steps); a DI container (same outcome via profiles — prefer it if you have one); Facade (simplifies, no consistency guarantee).

When not to use it

Objects that don't belong together (`createLogger()` + `createInvoice()` is a grab-bag); one variant only; a product-kind list that keeps growing (> 5–6); your DI container already does it (`@Profile`, Guice modules).

Common mistakes

A factory implementing only some methods or mixing families; choosing the factory deep in the call stack; making it a Singleton by reflex.

Real-world examples

- **JDK:** `DocumentBuilderFactory`, `TransformerFactory` (creates `Transformer` and `Templates`), `XPathFactory` — `newInstance()` locates an implementation whose products all match. JDBC `Connection` creates driver-specific `Statement`, `PreparedStatement`, `Blob`.
- UI toolkits (GoF's example), Hibernate `Dialect`, Spring `@Profile` bean sets.

In the interview

> **30-second answer:** "Abstract Factory is an interface with one create method per product in a family; each concrete factory returns a matched set. The client holds only the factory and abstract products, so it can't mix a Postgres connection with a MySQL query builder. I pick the factory once at the composition root. The cost: a new product kind means editing every factory."

- *Q: Add SQLite?* — A: `SqliteConnection`, `SqliteQueryBuilder`, `SqliteFactory`, one line in the composition root.
- *Q: Add a `MigrationRunner`?* — A: New method on the interface and every factory — the weak spot.

Self-check

- **Q:** What does it guarantee? **A:** Products from one factory are always the same family.
- **Q:** What is the composition root? **A:** The one place near start-up where concrete choices are made and passed down.
- **Q:** Easy vs hard change? **A:** New family easy; new product kind hard.

TL;DR

One factory → a matched family. Chosen once, passed down. Easy families, hard product kinds. Prefer DI profiles if you have a container.

8.12 Builder

What it is

Builder separates the *construction* of a complex object from the object. A builder collects values **one named step at a time** (a **fluent interface** — each returns `this`); `build()` produces the finished, usually **immutable**, product. Analogy: ordering at Subway, step by step.

	GoF Builder (1994)	**Bloch's Builder (*Effective Java* item 2)**
Targets	One construction process → **different representations** (RTF reader → ASCII, TeX, text widget)	Many optional parameters; wish for an immutable, validated object
Participants	**Builder** (abstract, one method per part), **ConcreteBuilder** (one representation, `getResult()`), **Director** (runs the steps), **Product**	Product + static nested `Builder`: fluent setters, validating `build()`; no Director or abstract Builder
What varies	Which ConcreteBuilder is plugged in	Only the values

Most real code is Bloch's; interviews may expect the GoF participant names too.

The problem it solves

`new HttpRequest(url, "GET", headers, null, 30, 3, true, false, null)` — which `null` is the body? Swap two booleans and it still compiles.

Approach	**What goes wrong**
Telescoping constructors	Overloads explode; still positional; same-typed params swap silently
JavaBeans (empty constructor + setters)	Half-built objects; can't be immutable; nowhere to validate cross-field rules

Builder: **named** values, **defaults**, **immutable** product, and `build()` as **one gate**. Our rule "a GET must not carry a body" (no defined meaning per RFC 9110 §9.3.1) spans two fields, so only `build()` can enforce it.

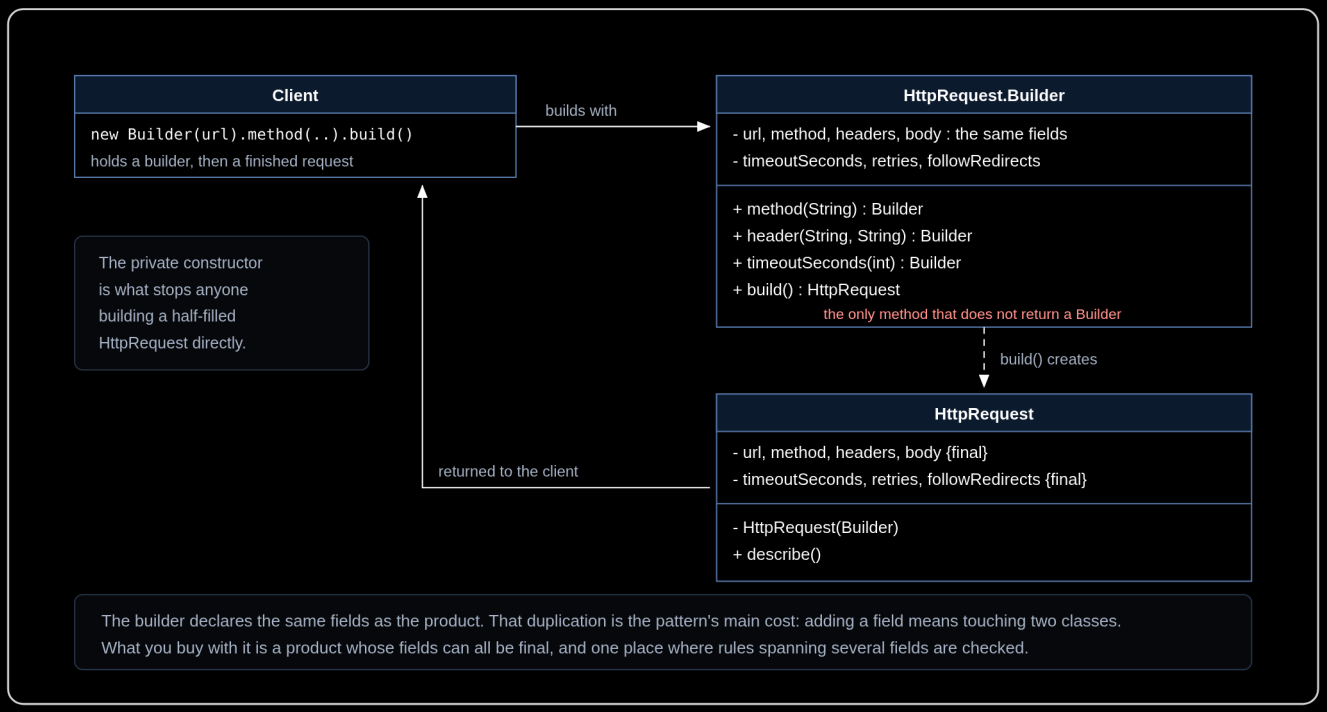
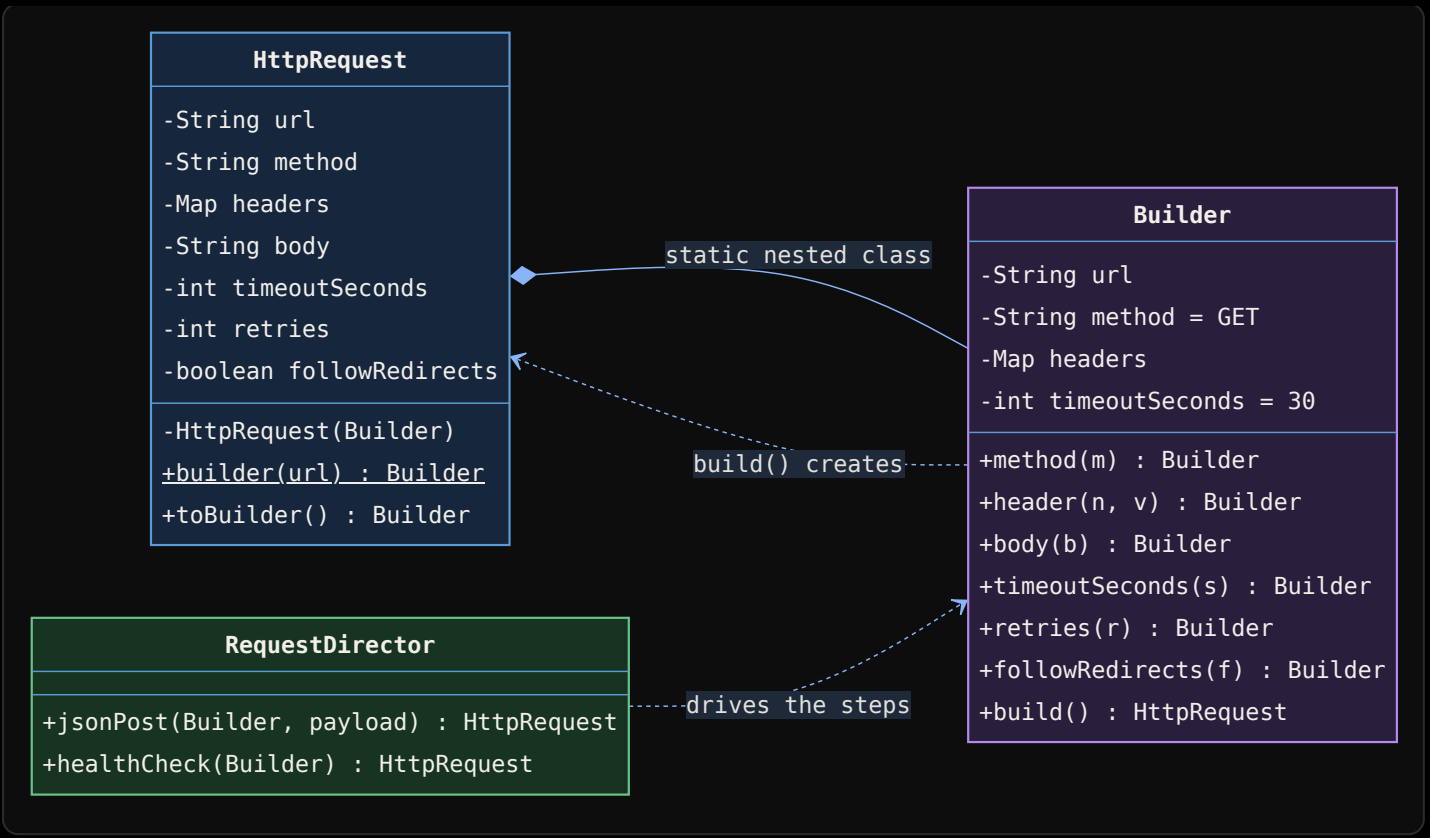


Figure — HttpRequest with final fields and a private constructor; HttpRequest.Builder with the same mutable fields, setters returning Builder, and build() returning HttpRequest (course diagram)

How it works

1. Product fields `final`, constructor `private` taking the builder. 2. `static` **nested** `Builder` with mutable fields and **defaults**. 3. **Required** values in the builder's constructor / `builder(url)`. 4. Named step methods that `return this`. 5. `build()`: validate cross-field rules, **copy mutable collections**, call the private constructor — the only method returning the product. 6. Optional: **Director**, `toBuilder()`, **staged builder**. Price: every field is written twice.



The Director — and why you rarely see one

The GoF **Director** knows the **order of steps** and runs them on whatever builder it gets — a home for a reusable recipe. Day to day, only *values* vary, so callers chain the calls themselves. **Test:** will the Director ever get a *different kind* of builder? If not, drop it. (Our `RequestDirector` only takes `HttpRequest.Builder` — it shows just the "named recipe" half.)

The staged (step) builder

Encode required/ordered steps in **types**: `Email.builder()` returns `ToStep` (only `to()`), then `SubjectStep` (only `subject()`), then `OptionalStep` with `build()`. `Email.builder().build()` **does not compile** — the answer to "how do you stop callers forgetting a required field?".

Lombok-style builders and records

- **Lombok** `@Builder` generates it all (`toBuilder = true`, `@Singular`) but does **not** validate by default. **Records** give the immutable product — validate in the compact constructor. **Immutability checklist:** final class/private constructor, `final` fields, no setters, defensive copies, no leaking getters.

Java implementation

```
// ... HttpRequest: final fields; private HttpRequest(Builder b) copies them, with headers = Map.copyOf(b.headers)
    /** The builder: same fields, mutable, with defaults. Every setter returns this. */
    public static final class Builder {
        private final String url;                                  // required -> constructor
        private String method = "GET";                             // optional -> defaults
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body;
        private int timeoutSeconds = 30;
        private int retries = 3;
        private boolean followRedirects = true;

        private Builder(String url) { this.url = Objects.requireNonNull(url, "url"); }

        public Builder method(String m)            { this.method = m; return this; }
        public Builder header(String n, String v)  { headers.put(n, v); return this; }
        public Builder body(String b)              { this.body = b; return this; }
        // ... timeoutSeconds, retries, followRedirects ...

        /** The single gate: cross-field validation happens here, before any product exists. */
        public HttpRequest build() {
            if (url.isBlank()) throw new IllegalStateException("url is required");
            if (method.equals("GET") && body != null) throw new IllegalStateException("a GET request cannot have a
body");
            if (retries < 0) throw new IllegalStateException("retries cannot be negative");
            if (timeoutSeconds <= 0) throw new IllegalStateException("timeout must be positive");
            return new HttpRequest(this);
        }
    }
}
```

- `TelescopingPizza` is the anti-example (five constructors; `("large", "thin", true, false, true)`).
- `Map.copyOf` matters: without it, two products built from one builder share one map, and a later header leaks into the earlier product. It's also unmodifiable, so `headers().put(...)` throws.
- `toBuilder()` gives "same as this, but…" copies; `RequestDirector.jsonPost/healthCheck` are named recipes; `Email` is the staged builder.

Full runnable code: code/lld/ch08/builder/ (see the Full Edition).

Trade-offs

Pros	Cons
Self-describing call sites	**Duplicated fields** — two edits per new field
Immutable products, thread-safe to share	**Forgettable** `build()`
Home for validation	**Overused** for 3-field objects
Free optional fields via defaults	**Reuse pitfall** — shared collections unless copied
Different representations (GoF Director)	One extra short-lived object (rarely measurable)

Often confused with: Factory Method (one call picks a type); Abstract Factory (several products at once); a **fluent interface** (a calling style — no separate product); Prototype (starts from a finished object). Named/default arguments (Python, Kotlin) usually replace builders.

When not to use it

A few required fields (the most common misuse); named/default arguments available; a builder that validates nothing for a mutable product; a mutable object living inside one method.

Common mistakes

Setters not returning the builder; no validation in `build()`; sharing mutable collections with products; a public product constructor (bypasses validation); a Director when the order never varies.

Real-world examples

- **JDK:** `StringBuilder`, `HttpClient.newBuilder()`, `HttpRequest.newBuilder()`, `Locale.Builder`, `Calendar.Builder`, `Stream.builder()`, `Thread.ofVirtual().name("w").start(task)`, `ProcessBuilder`.
- **Spring/libraries:** `UriComponentsBuilder`, `ResponseEntity.ok()...`, `WebClient.builder()`, OkHttp, Protobuf, Guava `ImmutableList.builder()`.

In the interview

> **30-second answer:** "With many optional parameters I use a Builder: required values in the builder's constructor, one named method per optional field returning the builder, and `build()` that validates cross-field rules and creates an immutable product via a private constructor. I'd mention the GoF Director but usually skip it since only values vary. For tiny objects or languages with named arguments, I wouldn't bother."

- *Q: How do you enforce required fields?* — A: Builder constructor, validation in `build()`, or a staged builder.
- *Q: Is the built object thread-safe?* — A: The immutable product is; don't share the builder across threads.

Self-check

- **Q:** The only builder method not returning the builder? **A:** `build()`.
- **Q:** What bug does copying the headers map prevent? **A:** Two products sharing and mutating one map.
- **Q:** When is a Director worth it? **A:** When the same recipe is reused, possibly with different builders.

TL;DR

Named steps + defaults + one validating `build()` → immutable product. Fixes telescoping constructors and half-built JavaBeans. Director optional; staged builder for compile-time required steps; Lombok/records cut boilerplate.

8.13 Prototype

What it is

Create objects by **copying an existing, configured object** via its own `copy()` — the caller needn't know the concrete class. Analogy: copying a contract template and changing the name and date.

GoF intent and participants. Intent: use a configured instance as the model and make new objects by copying it. **Prototype** (declares the copy operation — `Shape.copy()`), **ConcretePrototype** (`Circle`, `Rectangle`, `Group`; `NotificationTemplate` plays both roles), **Client** (asks a prototype to copy itself — `Main`, or `ShapeRegistry`, a *prototype manager*).

The problem it solves

1. **Expensive construction** — a `NotificationTemplate` parsed from disk; each welcome email differs only in subject and one channel. Re-parsing per recipient repeats the slowest work.
2. **Unknown concrete class** (the more important case) — you hold a `Shape` and need one more of the same; you can't write `new`. The object knows its own class, so it copies itself — no factory or `switch`, and the copy starts from correct values.

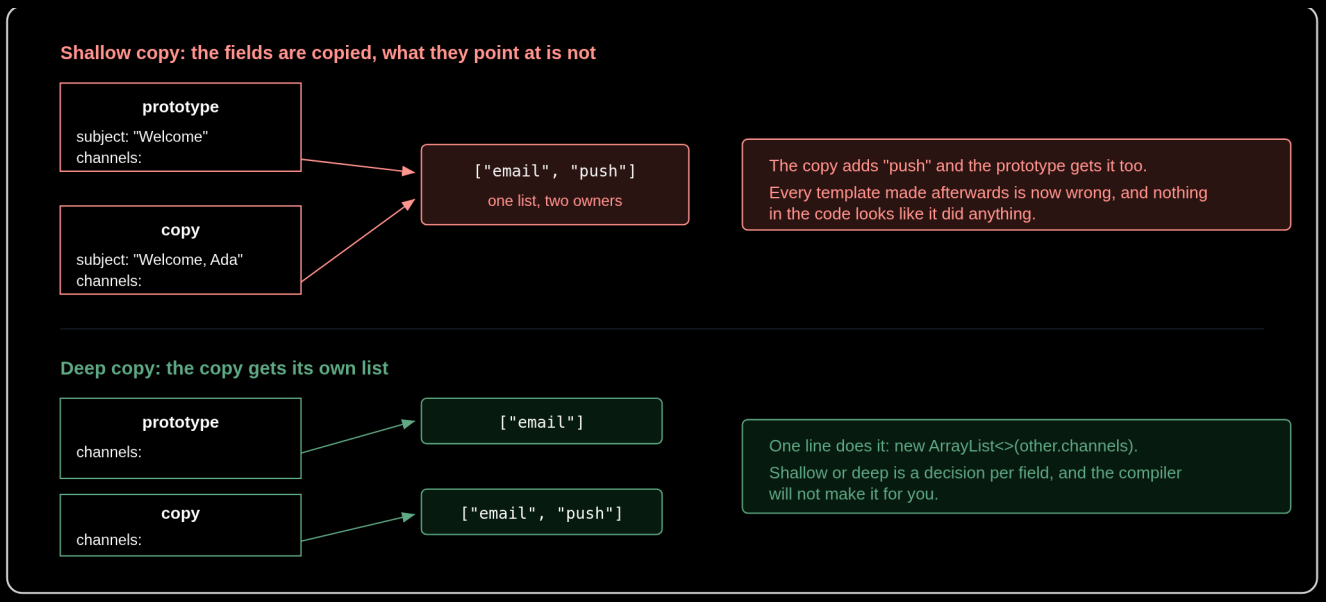
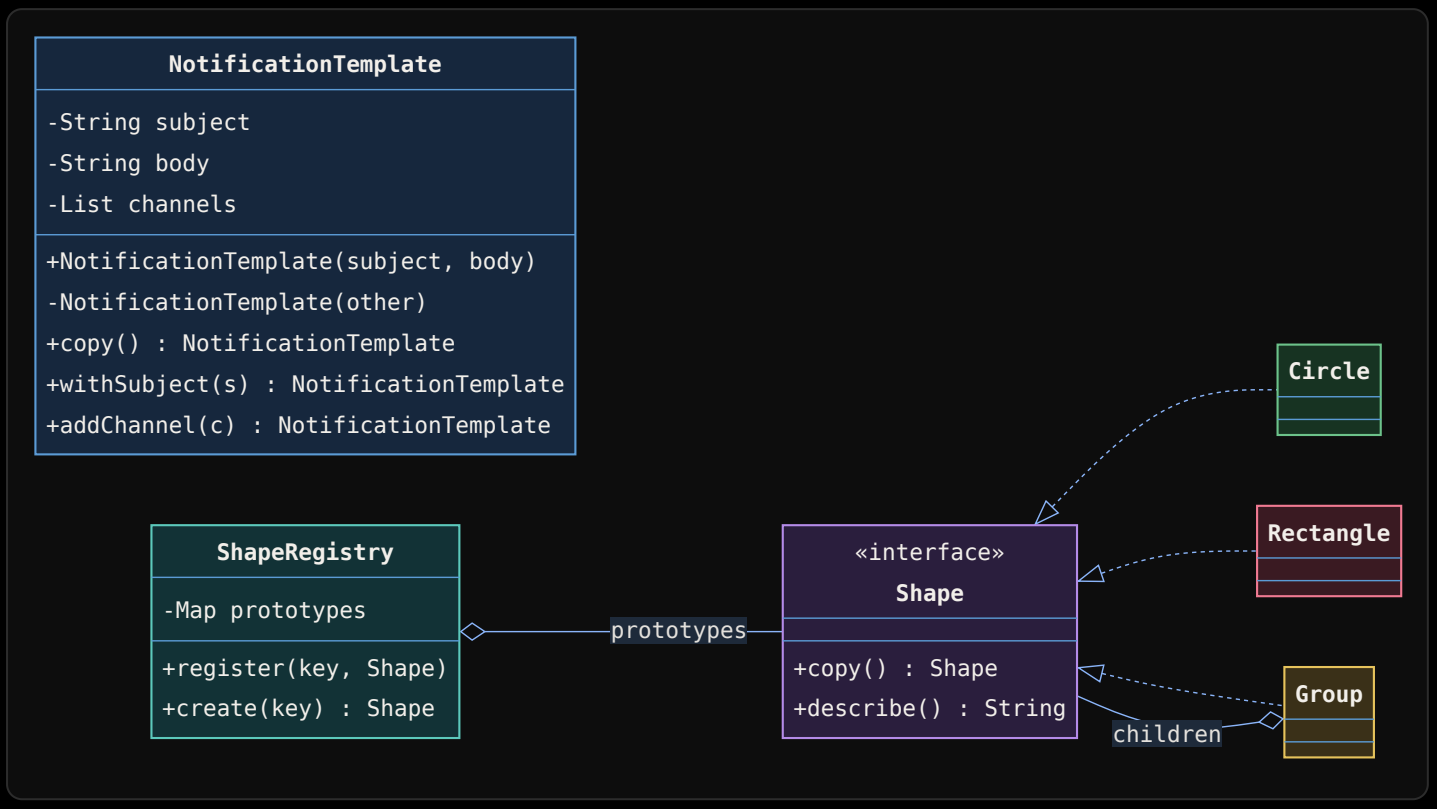


Figure — a shallow copy shares the channels list with the prototype, so adding "push" to the copy corrupts the prototype; a deep copy gets its own list (course diagram)

How it works

1. Two entrances: the **normal constructor** (expensive) and a **copy path** (none of it). 2. Implement the copy as a **private copy constructor**. 3. Per field: immutable → share; mutable → copy (`new ArrayList<>(other.channels)`), recursing for nested mutables. 4. Public `copy()`. 5. A **prototype interface** (`Shape { Shape copy(); }`) only when clients hold several unrelated types. 6. Optional **registry** that always hands out copies. (No separate creator class — the product copies itself.)



Shallow vs deep copy

	Shallow	Deep
Primitives, `String`, immutables	Copied / safely shared	Copied / safely shared
Mutable fields	**Shared**	**Duplicated**
Cost	Cheap	Proportional to the object graph
Risk	Editing the copy changes the original	Cycles; copying caches or resources
Java default	`Object.clone()`	You write it per field

With `this.channels = other.channels`, adding "push" to a copy adds it to the prototype — and **every later copy** inherits it. The fix is one line per mutable field. **Copy depth is a per-field design choice.** Related traps: a **new field not added to the copy** (silently dropped); copying something that **owns a resource** (both try to close it); **cycles** (naive recursive copy never ends — track visited objects).

Why Java's `Cloneable` is a trap

(*Effective Java* item 13.) `Cloneable` is a **marker with no methods**; `Object.clone()` is **`protected`**; it's **shallow**; it **skips constructors** (and their validation); deep-copying needs reassignment — **impossible for `final` fields**; it throws a **checked** exception. **Prefer a copy constructor or static `copyOf(other)`.** Exception: `array.clone()` for primitive arrays is fine.

Java implementation

```java
/** Prototype via a copy constructor: one expensive path, one cheap copy path. */
public final class NotificationTemplate {
    static int parses = 0;

    private String subject;
    private final String body;
    private final List<String> channels;

    /** The expensive path (imagine parsing a template file). */
    public NotificationTemplate(String subject, String body) {
        parses++;
        System.out.println("  Parsing template from disk... (expensive)");
        this.subject = subject;
        this.body = body;
        this.channels = new ArrayList<>(List.of("email"));
    }

    /** The copy path: no parsing. Deep-copies the mutable list. */
    private NotificationTemplate(NotificationTemplate other) {
        this.subject = other.subject;              // String is immutable -> sharing is safe
        this.body = other.body;
        this.channels = new ArrayList<>(other.channels);  // NEW list -> the copy owns its own
    }

    public NotificationTemplate copy() { return new NotificationTemplate(this); }
    // ... withSubject, addChannel, toString ...
}
```

Also in the package: `Group.copy()` (recursive deep copy of children), `Circle.copy()` with a covariant return, `ShapeRegistry.create()` returning `p.copy()` (never the original), and `CloneablePitfall.java` (`ShallowPlaylist` bug vs `DeepPlaylist`, whose `songs` can't be `final`).

Output: "Parsing template from disk" prints **once** for three templates, and the prototype keeps `channels=[email]` while copies add `push` / `sms`.

Full runnable code: code/lld/ch08/prototype/ (see the Full Edition).

Trade-offs

Pros	Cons
Costly setup paid once	**Copy code in every class** — drifts when fields are added
Create without naming a class	**Shallow vs deep per field** — wrong call shares state
New kinds at run time via registered prototypes	**Cyclic graphs** need care
Saved starting points (templates, layouts)	**Hazardous for identity** (keys, handles, connections); Java's `Cloneable` is flawed

Often confused with: Factory Method (fresh object); Builder (from parts); **Object Pool** (lends *existing* objects); **Spring** `prototype` **scope** (constructs anew — no cloning); JS prototypes (a language feature). JS spread is *shallow*; `structuredClone()` and Python `deepcopy` are deep.

When not to use it

Cheap construction; objects with **identity** (two rows sharing an ID is worse than a slow constructor); large graphs; immutable objects (just share them).

Common mistakes

Shallow-copying a mutable collection; trusting `super.clone()`; forgetting to update the copy constructor; handing out the registry's original.

Real-world examples

`ArrayList.clone()` / `HashMap.clone()` (**shallow**), copy constructors (`new ArrayList<>(c)`), `List.copyOf`; Unity `Instantiate(prefab)`; "Duplicate" in Docs/Figma; templates in Notion/Jira.

In the interview

> **30-second answer:** "Prototype creates objects by copying a configured instance via `copy()`. It saves expensive setup and — the deeper reason — lets me create an object when I only hold an interface. The main decision is shallow vs deep per field; a shared mutable list is the classic bug. In Java I'd write a copy constructor, not `Cloneable`, and never copy things with identity like a DB id or connection."

- *Q: Deep copy with cycles?* — A: An identity map original → copy; reuse the copy if already made.
- *Q: Serialization-based deep copy?* — A: Possible but slow, needs `Serializable`, bypasses constructors — last resort.

Self-check

- **Q:** Shallow vs deep? **A:** Shallow copies references; deep also copies the referenced mutable objects.
- **Q:** Why can't `DeepPlaylist.songs` be `final`? **A:** `clone()` must reassign it after `super.clone()`.
- **Q:** What should a registry return? **A:** A copy, never the original.

TL;DR

Copy a configured object — saves setup and works with only an interface. Decide shallow vs deep per field. Copy constructors, not `Cloneable`. Never copy identity.

8.14 Bonus — Object Pool

What it is

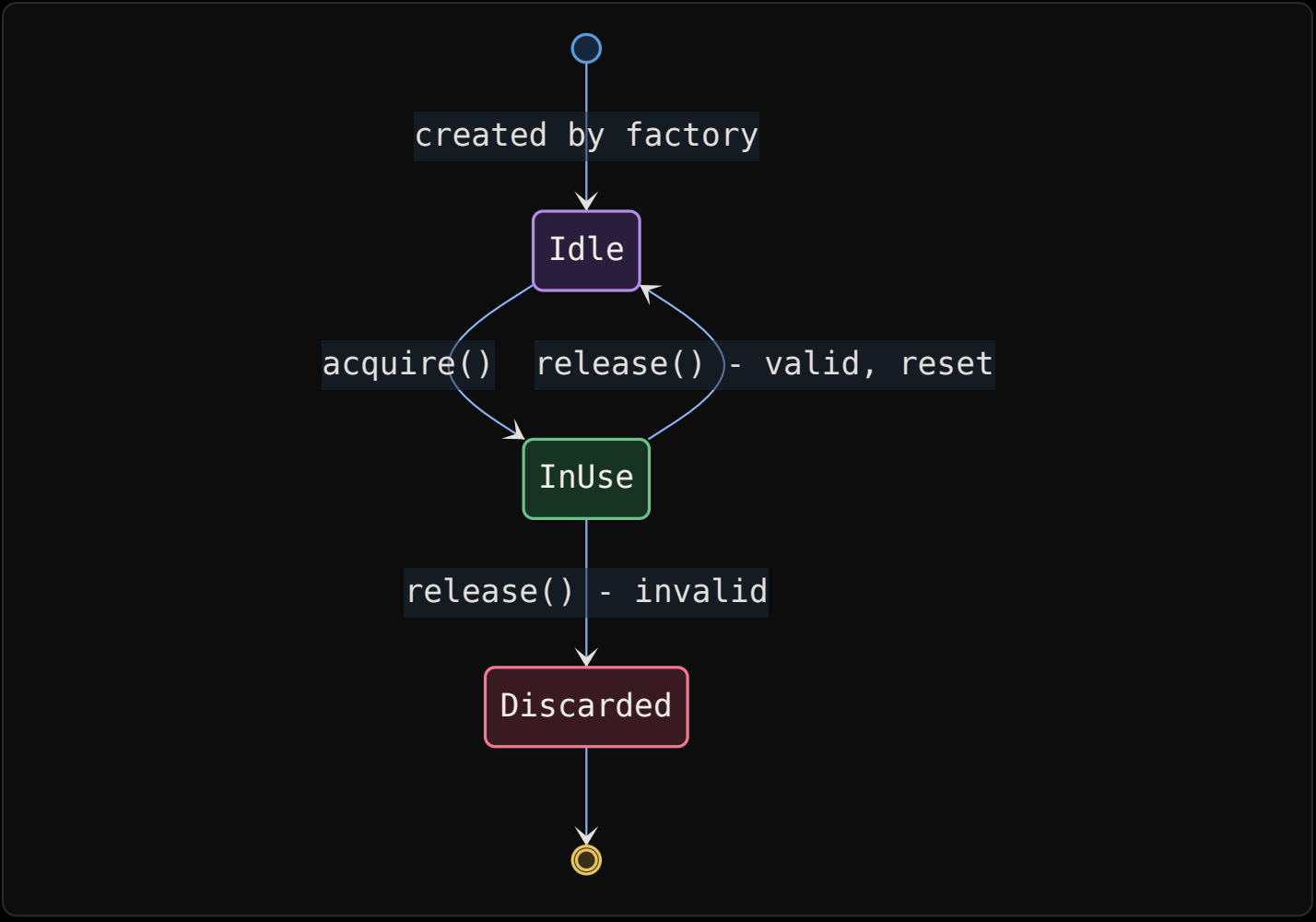
A **bounded set of reusable objects**: clients **acquire**, use, and **release** so others can reuse them. Not GoF, but a known creational pattern. Analogy: a bike-sharing dock — take, ride, return; if none, wait (or give up).

The problem it solves

Some objects are **expensive** (DB connection: TCP + TLS + auth) and/or **limited** (the DB caps connections). One per request is slow and can overwhelm the server. A pool pays creation a few times and **caps** the total.

How it works

1. An **idle queue**, a live count, and a cap (a `Semaphore` with one permit per lendable object).
2. **acquire()**: wait (with timeout) for a permit; take an idle object or create one; fail on timeout.
3. Client **always releases in** `finally` — a leak is lost forever.
4. **release()**: **validate** (discard broken ones), **reset** per-borrower state, return it; always free the permit.
5. Everything **thread-safe**.



Java implementation

```
// ... ObjectPool<T> fields: Semaphore permits (maxSize, fair), ConcurrentLinkedQueue<T> idle,
// ...   Supplier<T> factory, Consumer<T> reset, Predicate<T> isValid, AtomicInteger created
    /** Borrow an object, waiting up to the timeout if all are in use. */
    public T acquire(long timeout, TimeUnit unit) throws InterruptedException {
        if (!permits.tryAcquire(timeout, unit)) {
            throw new IllegalStateException("pool exhausted: timed out after " + timeout + " " + unit);
        }
        // Holding a permit guarantees we are within the cap: reuse an idle object or create one.
        T obj = idle.poll();
        if (obj != null) return obj;
        try {
            obj = factory.get();
            created.incrementAndGet();
            return obj;
        } catch (RuntimeException e) {
            permits.release();                                  // creation failed: give the permit back
            throw e;
        }
    }

    /** Give an object back. Broken objects are discarded so a fresh one can be created later. */
    public void release(T obj) {
        try {
            if (isValid.test(obj)) {
                reset.accept(obj);
                idle.offer(obj);
            } else {
                created.decrementAndGet();                      // discard it; its permit still comes back
            }
        } finally {
            permits.release();                                  // wakes one waiting borrower, if any
        }
    }
```

- The `Semaphore` caps borrowed objects however many threads race; holding a permit means "reuse idle, else safe to create".
- `release()` frees the permit **even when discarding**, so a waiter can create a replacement; a failed factory call also returns its permit.
- Handing objects through a concurrent queue + semaphore makes the previous borrower's writes visible to the next.
- `Main`: 10 tasks on 6 threads with a pool of 3 → `connections ever created: 3`; exhaustion prints `pool exhausted: timed out after 100 MILLISECONDS`.

Full runnable code: code/lld/ch08/objectpool/ (see the Full Edition).

Trade-offs

Pros	Cons
Avoids repeated expensive creation	Complexity: sizing, validation, timeouts, leak detection
Caps resource usage (protects the DB)	Leaks shrink the pool until it deadlocks
Predictable latency after warm-up	Incomplete reset leaks one user's data to the next
	Slower than `new` for cheap objects

When not to use it

Cheap objects (JVM allocation is faster); **virtual threads** (Java 21 — create one per task); when a **library** exists (HikariCP, Commons Pool 2).

Common mistakes

Not releasing in `finally`; not resetting state; unbounded waits; **double release** (here it would queue one object twice *and* add a permit, silently raising the cap — real pools track borrowed objects).

Real-world examples

HikariCP (Spring Boot default), DBCP, c3p0; `ThreadPoolExecutor`; Commons Pool 2 `GenericObjectPool` (Jedis); Netty `PooledByteBufAllocator`. *Not* pools: the `String` intern pool and `Integer` cache (-128..127) share immutables — that's **Flyweight** (Ch. 9).

In the interview

> **30-second answer:** "An object pool reuses a bounded set of expensive objects like DB connections. acquire takes an idle one, creates one if under the cap, or waits with a timeout; release validates, resets and returns it. It must be thread-safe and clients release in finally. For cheap objects it's counter-productive; in production I'd use HikariCP."

- *Q: Sizing?* — A: From measured concurrency and downstream limits; small pools (≈ core-count scale) often win.
- *Q: Leaks?* — A: Record borrower and time; log long holds (HikariCP `leakDetectionThreshold`).

Self-check

- **Q:** Pool vs Flyweight? **A:** Pool lends mutable objects exclusively and takes them back; Flyweight shares immutable objects with everyone at once.
- **Q:** Why validate on release? **A:** So dead objects aren't handed to the next borrower.

TL;DR

Borrow → use → return (in `finally`). Bounded, thread-safe, validated, reset. Only for expensive/limited resources; use a mature library.

8.15 Creational patterns compared

At a glance

Pattern	Distinctive feature	Use when	JDK / Spring
Singleton	One instance	One shared resource	`Runtime.getRuntime()`, singleton beans
Simple Factory	One method maps type → class	Small, stable, config-chosen set	`Calendar.getInstance()`, `EnumSet.noneOf`, `@Bean`
Factory Method	Subclasses choose via overridable method	Shared procedure, varying product	`Collection.iterator()`, `FactoryBean`
Abstract Factory	Families without concrete classes	Products must match	`DocumentBuilderFactory`, JDBC `Connection`, profiles
Builder	Construction separate from representation	Optional fields, validation, immutability	`StringBuilder`, `HttpClient.newBuilder()`
Prototype	Copy an existing object	Expensive setup, or only interface known	Copy constructors, `ArrayList.clone()`, Unity
Object Pool	Reuse a bounded set	Expensive/limited resources	HikariCP, `ThreadPoolExecutor`

Pick by the question you are asking

You are asking…	Pattern
"Only one?"	Singleton (or DI singleton scope)
"Avoid scattered `new ConcreteClass()`?"	Simple Factory
"Let subclasses/plug-ins decide?"	Factory Method
"Make sure these always match?"	Abstract Factory
"Make a 10-argument constructor readable and safe?"	Builder
"Another one of *this*, cheaply or without knowing its class?"	Prototype
"Stop creating and destroying this expensive thing?"	Object Pool

How they combine

- **Abstract Factory** is often built from **Factory Methods**, may use **Prototypes**, and is often a **Singleton** (passing it in is better).
- A **Builder** can use a **Factory** to pick parts; a **Director** can be a stateless shared service.
- Designs often *start* with Factory Method and *evolve* toward Abstract Factory, Prototype or Builder.

8.16 Chapter self-check

1. **Q:** The three GoF categories and counts? **A:** Creational 5, Structural 7, Behavioral 11 = 23.
2. **Q:** The two GoF central principles? **A:** Program to an interface; favour composition over inheritance.
3. **Q:** Why must the DCL field be `volatile`? **A:** So no thread sees a reference to a not-yet-constructed object (happens-before).
4. **Q:** Factory Method vs Abstract Factory? **A:** One product by inheritance vs a matching family by composition.
5. **Q:** How many patterns in a 45-minute design? **A:** One or two — and name the one you rejected.

8.17 Chapter TL;DR

- **Patterns** = named, proven templates with trade-offs; GoF (1994) catalogued 23 in three categories.
- **Pick from the signal**, one or two per design; state cost and alternative.
- **Singleton** — one per JVM; eager / holder / enum; DCL needs `volatile`; prefer DI.
- **Simple Factory** — one `create(type)`; idiom, not GoF.
- **Factory Method** — base-class procedure + subclass-chosen product.
- **Abstract Factory** — one factory, one matching family, chosen at the composition root.
- **Builder** — named steps, one validating `build()`, immutable product.
- **Prototype** — copy a configured object; shallow vs deep; avoid `Cloneable`.
- **Object Pool** — borrow/return expensive objects; use a real library.

Chapter 9 — Structural Design Patterns (Study Edition)

> **Study Edition.** A shorter version of Chapter 9: every pattern and sub-topic is kept, the padding is removed. The full chapter (README.md) has every source file and full outputs.
>
> **Where this fits.** Chapter 8 was about *creating* objects; this chapter is about *putting objects together* — connecting, wrapping, nesting and sharing them. Chapters 10–11 cover how objects *talk* (behavioral patterns).

#	Pattern	One-line intent	Analogy
9.1	Adapter	Translate a class's interface into the one your code uses	Travel plug adapter
9.2	Bridge	Split two independently varying dimensions into two hierarchies joined by a reference	TV remote and TV sold separately
9.3	Composite	Treat one object and a tree of objects the same way	Folders hold files and folders
9.4	Decorator	Add behaviour to one object at run time by wrapping it	Coffee toppings
9.5	Facade	One simple entry point to a complicated subsystem	Hotel concierge
9.6	Flyweight	Share the repeating part of many objects to save memory	One glyph "e" reused on every page
9.7	Proxy	Same-interface stand-in that controls access	Receptionist guarding the manager
9.8	Comparison	Adapter vs Decorator vs Proxy vs Facade vs Bridge	—

Run: `cd chapters/09-patterns-structural/code && javac --release 21 -d /tmp/lld_09 $(find . -name '*.java') && java -cp /tmp/lld_09 lld.ch09.adapter.Main` (or `bridge`, `composite`, `decorator`, `facade`, `flyweight`, `proxy`). All seven compile with zero warnings under `-Xlint:all`; each top-level type has its own file and is package-private.

9.0 What structural patterns are

What it is

The GoF's 23 patterns fall into three families: **Creational** (Ch. 8 — how to create objects), **Structural** (this chapter — how to combine classes and objects into bigger structures) and **Behavioral** (Ch. 10–11 — how objects share work and talk). Structural patterns **simplify complex structures** (Facade, Composite), **promote reuse** (Adapter), **improve flexibility by decoupling** (Bridge, Decorator, Proxy) and **improve maintainability** (clear relationships, local changes). The seven GoF structural patterns: **Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**.

Three words you need first

- **Composition** ("has-a") — an object holds and uses another; almost every structural pattern is built on it.
- **Delegation** — forwarding a call to the held object (`wrapper.pay()` calls `inner.charge()`).
- **Interface** (design sense) — the methods a caller can use on an object.

> **Key insight.** Adapter, Decorator, Proxy and Facade have the *same physical shape* — one object holds another and forwards calls. What separates them is **intent**: what changes as the call passes through (9.8).

9.1 Adapter Pattern

What it is

Adapter lets your code use a class whose methods have the "wrong" shape — one you can't change — through a small translator. GoF intent: *convert the interface of a class into another interface clients expect.* Also called **Wrapper**. Analogy: a travel plug adapter.

Role	Meaning	Ours
Target	Interface your code wants	`PaymentGateway`
Adaptee	Existing class with the wrong interface (usually not yours)	`StripeClient`
Adapter	Implements Target, translates to Adaptee	`StripeGateway`
Client	Uses the Target	`checkout`

The problem it solves

Checkout wants `pay(Money)`; the SDK offers `charge(amountInCents, currencyCode)` (or XML vs JSON, `track(String, Map)` vs your `Event`). **The other side isn't yours to change** — a vendor, another team, many existing callers. Stripe wants integer **cents** + **lowercase** currency; PayPal wants a **decimal string** "42.50" + **uppercase**; your code wants to think in `Money`.

How it works

1. **Target interface in your vocabulary** (`PaymentGateway.pay(Money) -> Receipt`), no vendor names.
2. **One adapter per adaptee**, implementing the Target and **holding** the vendor object.
3. **Translate inputs**, 4. **call the adaptee**, 5. **translate outputs and errors back** (vendor exception → your exception).
4. **Clients depend only on the Target.**

> **The rule:** the adapter's public methods mention **only your types** — parameters, returns *and* exceptions. Let a `StripeCardException` escape and callers depend on Stripe again.

Structure

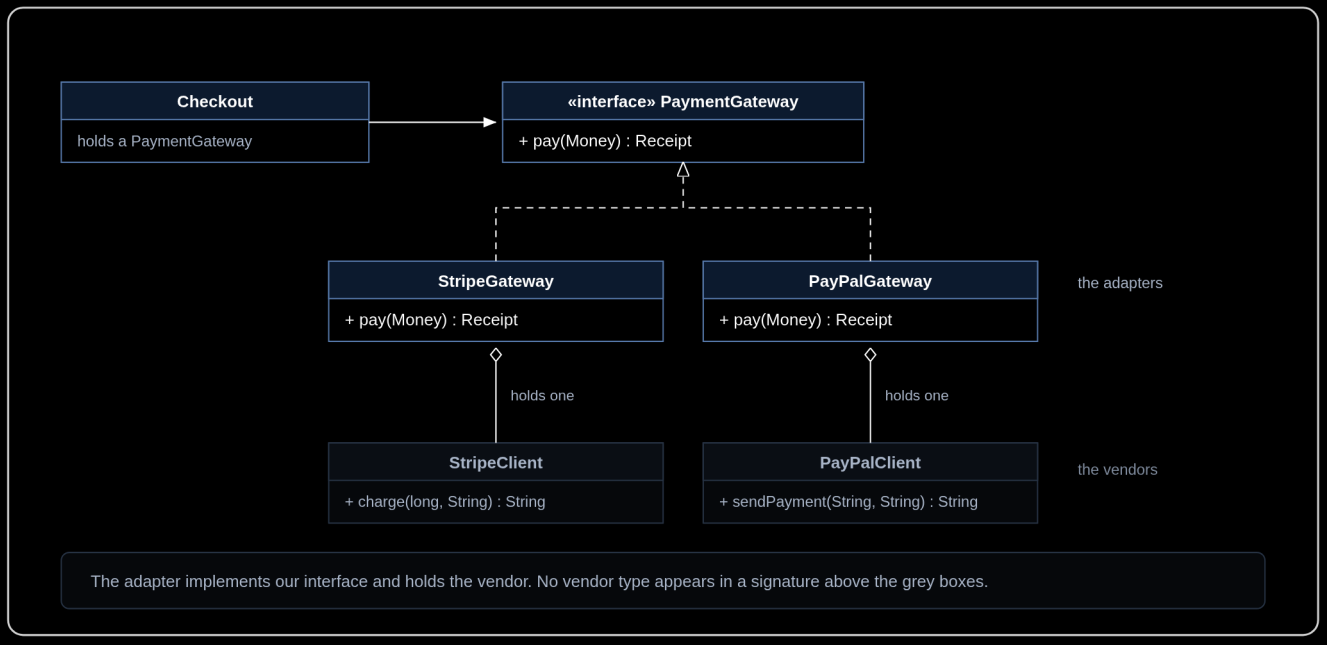
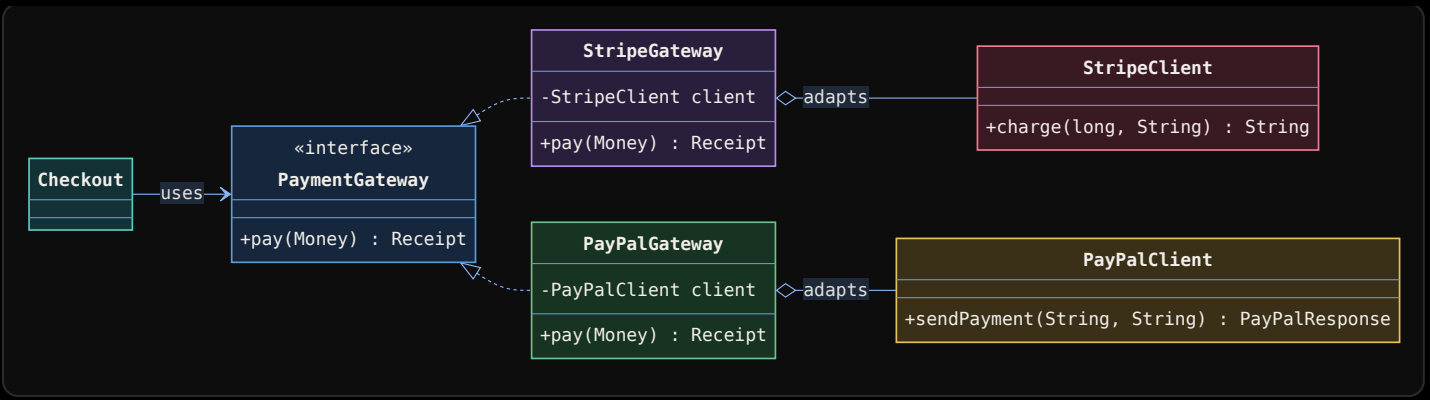


Figure — Adapter class diagram: `Checkout` holds a `PaymentGateway`; `StripeGateway` and `PayPalGateway` implement it and each wrap a vendor client, shown in grey because we do not own them (course diagram)



Java example

```java
/** TARGET interface: the shape our application wants to talk to. */
interface PaymentGateway {
    Receipt pay(Money amount);
}

/** OBJECT ADAPTER: implements our interface, HOLDS the vendor object (composition). */
final class StripeGateway implements PaymentGateway {
    private final StripeClient client;

    StripeGateway(StripeClient client) { this.client = client; }

    @Override
    public Receipt pay(Money amount) {
        try {
            String chargeId = client.charge(amount.cents(), amount.currency().toLowerCase(Locale.ROOT));
            return new Receipt(chargeId, "SUCCEEDED");
        } catch (StripeCardException e) {              // translate the vendor's error too
            throw new PaymentFailedException("Stripe declined: " + e.getMessage(), e);
        }
    }
}
```

`Money` (a record that upper-cases the currency once), `Receipt` and `PaymentFailedException` are *our* vocabulary. `PayPalGateway` does a different translation (`Money.asDecimal()` → "42.50"; `"approved"` → `"SUCCEEDED"`). `checkout(gateway, amount)` has no `if (stripe)` branch. The output shows `charge(4250, "usd")`, `sendPayment("42.50", "USD")`, and a declined Stripe charge reaching the client as `payment failed -> Stripe declined: card_declined` — *our* error type.

Full runnable code: code/lld/ch09/adapter/ (see the Full Edition).

Object adapter vs class adapter

	Object adapter (composition)	**Class adapter (inheritance)**
How	*Holds* an adaptee, implements Target	*Extends* the adaptee, implements Target
Ours	`StripeGateway`, `PayPalGateway`	`ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor`
Adaptee `final`?	Works	**Fails** (our `StripeClient` is final)
Adapts adaptee + subclasses?	Yes	No — bound at compile time
Override adaptee behaviour?	Harder	Yes
Leaks adaptee API?	No	**Yes** (`readFahrenheit()` stays public)
Java constraint	none	Target must be an **interface** (no multiple class inheritance)

Prefer the object adapter (composition, works with `final`, no unwanted methods).

Two-way adapter

Implements **both** interfaces so clients of either side can use one object — `TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor`: new code sees `100.0C`, old code `212.0F`. Handy during **migrations**.

Adapter vs Facade vs Decorator vs Proxy (quick view)

Pattern	Interface	Behaviour
Adapter	**Translated**	Same work, re-expressed
Decorator	Same	**Extended**
Proxy	Same	Same work, **gated** (defer, cache, refuse)
Facade	**Replaced** by one simpler one over many	Same work, fewer calls

Test: *adapters exist because someone else picked the interface* — if the vendor matched yours tomorrow, the adapter would be pointless. **Bridge** looks the same but is planned *before* the classes exist; an Adapter is bolted on *after*.

Trade-offs

Pros	Cons
Foreign format contained in one class	Extra class and call frame
Vendor change is local (Open/Closed)	**Conversions can drop information** silently
Easy to test — fake your one-method interface	Two data models must stay aligned
Your code keeps its language (SRP)	Logic (retries, pricing) creeps in

When not to use it

Both sides are yours (change one); the models truly disagree (redraw the boundary); you want extra behaviour (Decorator); the conversion happens in exactly one place.

Common mistakes

Leaking vendor types/exceptions; business logic in the adapter; class adapter by default; one adapter for many vendors with `if (vendor == STRIPE)`; silently mapping "pending" to "SUCCEEDED".

Real-world examples

`Arrays.asList` (array → `List`, no copy; writes go through, `add` throws); `InputStreamReader` / `OutputStreamWriter` (bytes ↔ chars); `Collections.enumeration`; SLF4J bindings; JDBC drivers; Spring MVC `HandlerAdapter`; every in-house "client" wrapping a vendor SDK.

In modern languages

A one-method Target makes a **lambda** the whole adapter (`FahrenheitSensor oldSensor = () -> 212.0;`); duck/structural typing (Python, TS, Go) makes adapters tiny.

In the interview

> **30-second answer:** "An Adapter converts one interface into another the client expects. I use it at boundaries I don't control — a vendor SDK, a legacy service. It implements *my* interface, holds the vendor object and translates inputs, outputs and exceptions, so no vendor type leaks in. Swapping vendors is one new class. I prefer the object adapter: composition, works with final classes."

- *Q: Adapter vs Decorator?* — A: Adapter changes the interface, keeps behaviour; Decorator keeps the interface, adds behaviour.
- *Q: Why not change the other side?* — A: Say it up front: it's a vendor library / another team / many callers. That constraint justifies the pattern.

Self-check

1. **Q:** The four roles? **A:** Target, Adaptee, Adapter, Client.
2. **Q:** Why can't `StripeClient` be class-adapted? **A:** It's `final`.
3. **Q:** `Arrays.asList(arr).add(x)`? **A:** `UnsupportedOperationException` — a fixed-size view over the array.

TL;DR

Translator between the interface you have and the one you need; implements **yours**, holds the vendor's, translates **in, out and errors**. Prefer object adapters. Only at boundaries you don't control.

9.2 Bridge Pattern

What it is

Bridge splits a class that varies in *two independent ways* into **two hierarchies** joined by a reference (the "bridge"). GoF intent: *decouple an abstraction from its implementation so the two can vary independently* (a.k.a. **Handle/Body**). Analogy: any remote works any TV via a common protocol.

Term	Meaning	Ours
Abstraction	High-level side, holds an Implementor	`Notification`
Refined Abstraction	Its subclasses	`AlertNotification`, `ReminderNotification`, `DigestNotification`
Implementor	Interface of the low-level physical work	`Channel`
Concrete Implementor	Real low-level work	`EmailChannel`, `SmsChannel`, `PushChannel`

> **Naming trap.** "Implementation" here isn't "a class implementing an interface" — it's the **low-level half** of the split (sending bytes, drawing pixels).

The problem it solves — the class explosion

Three notification types × three channels in one hierarchy = **9** classes (`EmailAlert`, `SmsDigest`…); add one of each → 16. With a bridge they **add**: 3 + 3 = 6, then 8. **Smell:** class names made of two nouns, each duplicating build or send code. Bonus: the pairing becomes a **run-time** choice (SMS in tests, email in production). The hard part is choosing which side is the abstraction.

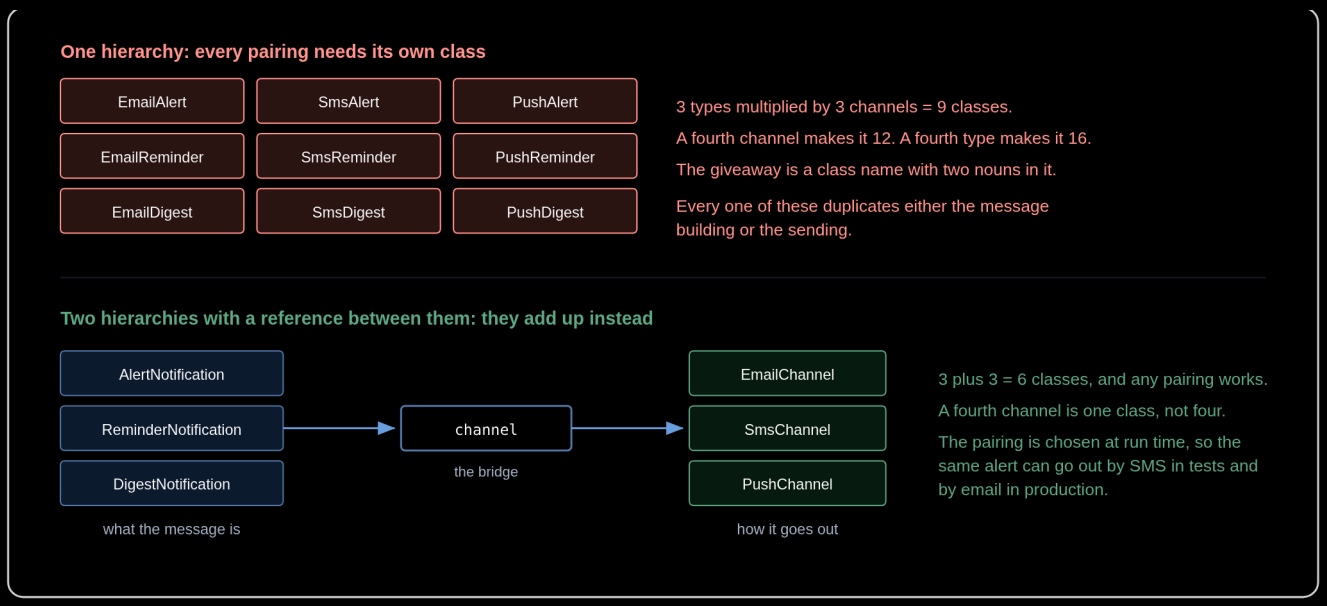
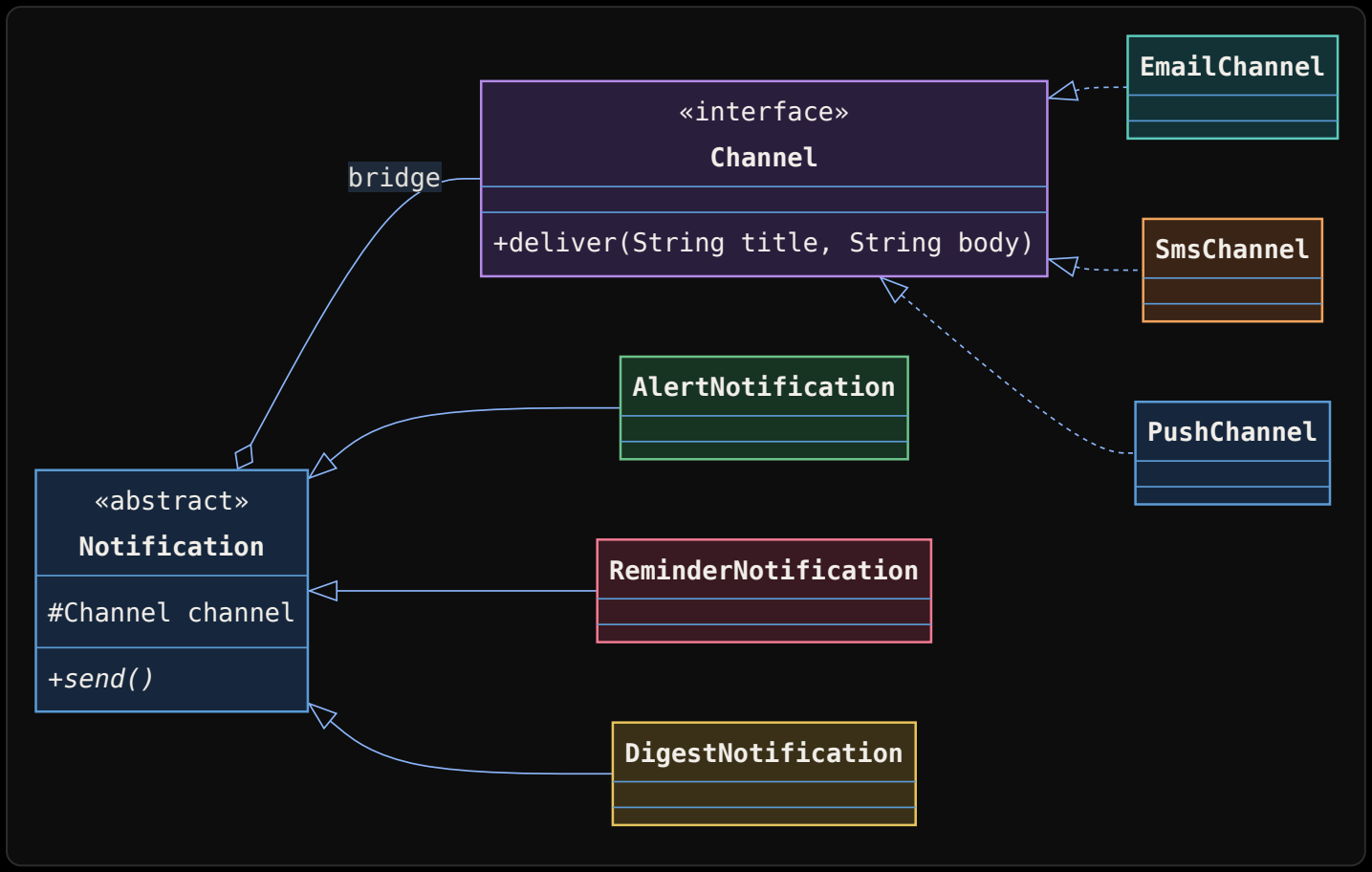


Figure — Top: one hierarchy needs 3 × 3 = 9 classes such as EmailAlert and SmsDigest. Bottom: two hierarchies joined by one `channel` reference need 3 + 3 = 6 (course diagram)

How it works

1. Find the two independent dimensions (what the message *is* vs how it *goes out*).
2. The lower-level one becomes a small, general **Implementor interface** (`Channel.deliver(title, body)`).
3. The higher-level one becomes an **abstract class holding a reference** to it — that field is the bridge.
4. Refined abstractions build content and **delegate** across the bridge, never checking the implementor type.
5. Pair at run time: `new AlertNotification(new SmsChannel(...), ...)`.

Structure



Java example

```java
interface Channel {
    void deliver(String title, String body);
}

abstract class Notification {
    protected final Channel channel;

    protected Notification(Channel channel) { this.channel = channel; }

    /** Builds the message, then hands it across the bridge. Never branches on channel type. */
    abstract void send();
}

/** REFINED ABSTRACTIONS */
final class AlertNotification extends Notification {
    private final String system;
    private final String problem;

    AlertNotification(Channel channel, String system, String problem) {
        super(channel);
        this.system = system;
        this.problem = problem;
    }

    @Override
    void send() { channel.deliver("ALERT " + system, problem + " - act now"); }
}
```

`SmsChannel` truncates long text — an SMS rule living inside SMS only. Adding `SlackChannel` or `PromoNotification` = one class each.

Full runnable code: code/lld/ch09/bridge/ (see the Full Edition).

Bridge vs Strategy

Code is **identical** (hold an interface, delegate); the *reason* differs.

	Bridge	Strategy
What varies	A whole **dimension**	**One algorithm**
When chosen	Once, at construction	Often swapped, even per call
Why	Stop two hierarchies multiplying	Make one behaviour replaceable
Decided	Up front, structurally	Later, when a 2nd algorithm appears
Example	`Notification` → `Channel`	`list.sort(comparator)`

Test: a new implementation that is *a new kind of thing the system supports* → Bridge; *one more way to do one step* → Strategy.

Getting the split wrong

- **Low-level details leaking up** — `if (channel instanceof SmsChannel)` in `AlertNotification`; move it into `SmsChannel`.
- **An implementor shaped around one implementation** — `setEmailReplyTo(...)` on `Channel` forces others to throw. Keep it minimal (two plain strings).

Trade-offs

Pros	Cons
Classes add, not multiply	Two hierarchies to understand
Each side extends independently (Open/Closed)	More indirection
Pairing chosen at run time	**Hard to undo** once both sides grow
Platform code isolated (SRP)	Pointless with one implementor

When not to use it

An axis has one member (add it when the second lands); axes aren't independent (forbidden combos become run-time checks); you can't say in one sentence which side is the abstraction; the second axis is a behaviour (Strategy).

Common mistakes

Bridge "just in case"; abstraction branching on implementor type; implementor mirroring one implementation; confusing it with Adapter (retrofitted vs designed in).

Real-world examples

JDBC (`Connection`/`Statement` + drivers — also called Adapter); SLF4J API vs backend; GoF's `Window`/`WindowImp` per platform and graphics drivers; `java.awt` peers; `NotificationService` with SES/Twilio/FCM providers; DI binding per environment.

In modern languages

A one-method implementor can be a plain function; DI containers wire the pairing per environment.

Other patterns it is confused with

Looks like	Difference
Strategy	Same shape; permanent dimension vs swappable algorithm
Adapter	Retrofitted after vs planned before
Abstract Factory	Decides *which family is created*; often supplies the implementor a bridge receives
Decorator	Shares the wrapped interface and adds behaviour; Bridge sides have unrelated interfaces

In the interview

> **30-second answer:** "Bridge separates an abstraction from its implementation so both vary independently. Notifications by type and by channel in one hierarchy need 3 × 3 = 9 classes, 16 with a fourth of each. I split it: a `Notification` hierarchy holding a `Channel` interface — 6, then 8 — and the pairing is chosen at run time. I wouldn't build it with only one channel."

- *Q: vs Strategy?* — A: Same code; Bridge splits a permanent dimension up front, Strategy makes one algorithm replaceable.
- *Q: Who picks the implementor?* — A: Whoever builds it — often a factory or DI container, per environment.

Self-check

1. **Q:** 5 types × 4 channels, without/with Bridge? **A:** 20 vs 9.
2. **Q:** Which field is the bridge? **A:** `Notification`'s `Channel channel`.
3. **Q:** Bridge added before or after classes exist? **A:** Before (Adapter is after).

TL;DR

Two varying dimensions → two hierarchies + one reference; multiplication becomes addition. Keep the implementor small. Same code as Strategy, different intent.

9.3 Composite Pattern

What it is

Composite gives a single item and a nested group **one common interface**, so clients never check whether they hold a **leaf** or a **tree**. GoF intent: compose objects into **part-whole hierarchies** and treat individual objects and compositions **uniformly**. Analogy: ask any manager or individual contributor "your team's total salary?" — same question for both.

Term	Meaning	Ours
Component	Shared interface	`FileSystemNode`, `MenuComponent`
Leaf	No children; answers itself	`FileNode`, `MenuItem`
Composite	Holds children *typed as Component*, delegates	`FolderNode`, `Menu`
Client	Uses Components only	`report(...)`

The problem it solves

Nested data (folders, invoice bundles, UI panels) makes every operation start with `if (node instanceof Folder) ... else if (node instanceof File) ...`. That test is copied into `size`, `print`, `search`…; a new node type (symlink) must find every copy.

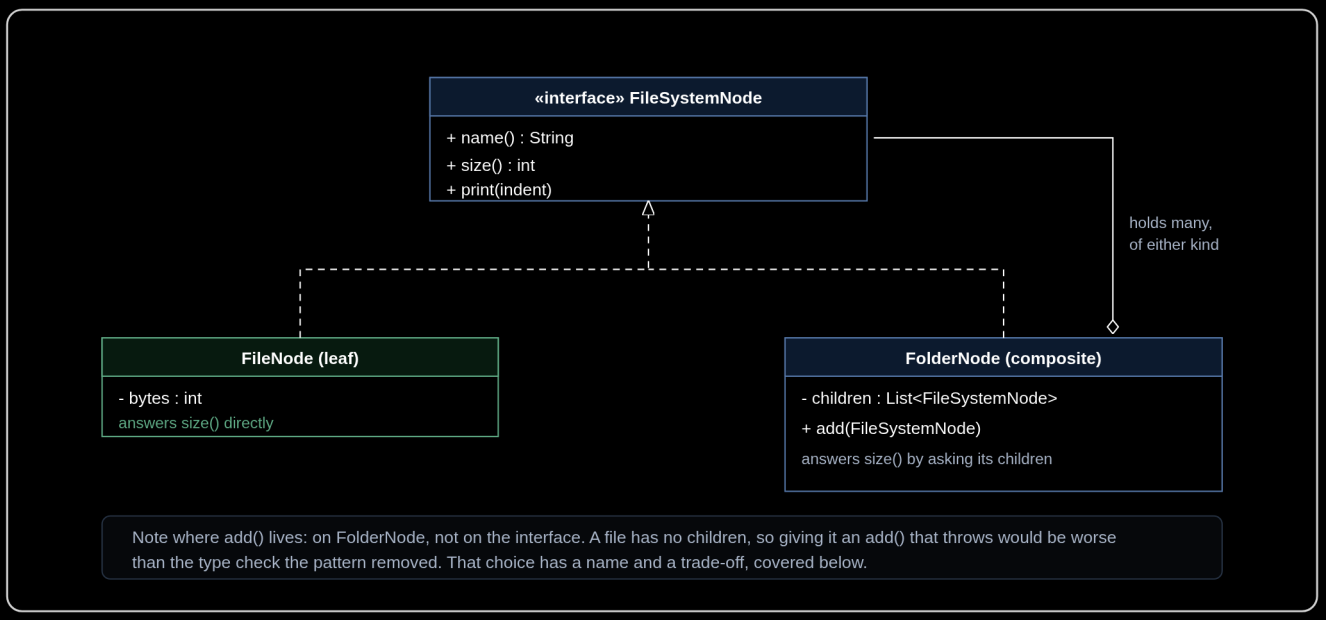
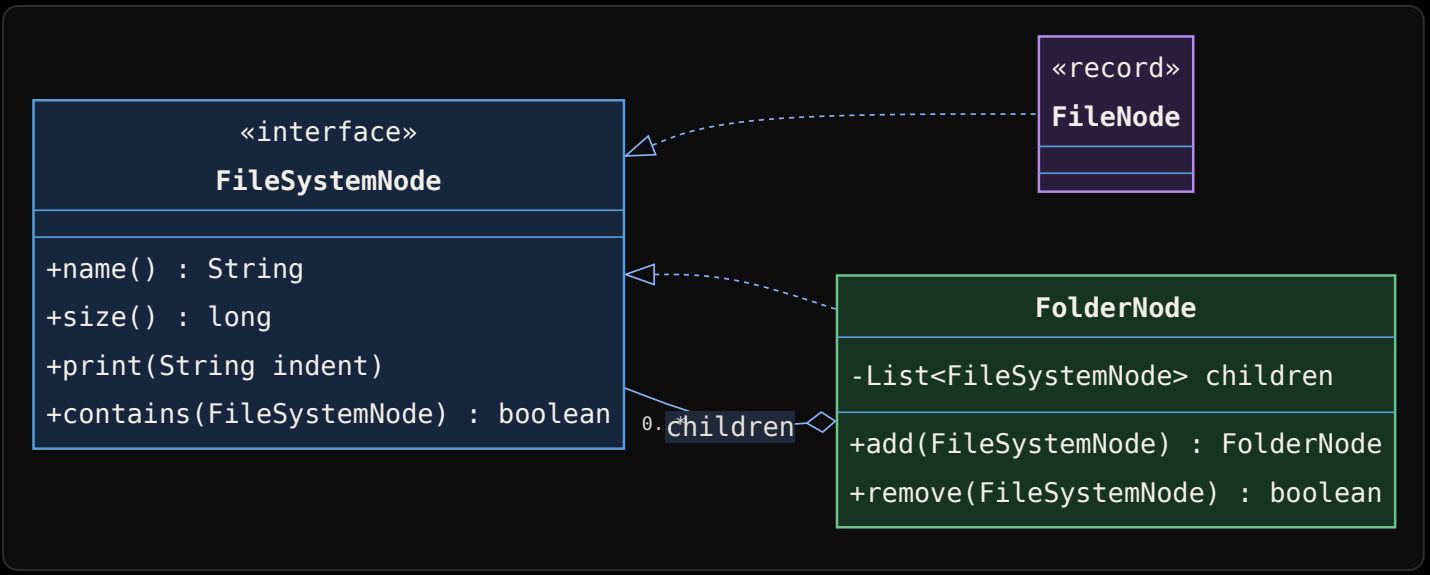


Figure — Composite class diagram: `FileNode` (leaf) and `FolderNode` (composite) implement `FileSystemNode`; the folder holds a list of `FileSystemNode` and has `add()` (course diagram)

How it works

1. A **Component** interface with operations valid for both (`name()`, `size()`, `print()`).
2. **Leaf** implements them directly.
3. **Composite** stores `List<Component>` — the **interface type** — and implements each operation by **asking children and combining**.
4. The recursion is written **once** and works to any depth.
5. **Clients** never type-check.

Structure



Java example

```java
interface FileSystemNode {
    String name();
    long size();
    void print(String indent);

    /** True if 'target' is this node or somewhere below it. Used for the cycle check. */
    default boolean contains(FileSystemNode target) { return this == target; }
}

/** LEAF: no children, answers directly. */
record FileNode(String name, long size) implements FileSystemNode {
    // ... print(indent) ...
}

/** COMPOSITE: holds children typed as the INTERFACE, so folders can hold folders. */
final class FolderNode implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();
    // ... constructor ...
    /** Returns this so trees can be built fluently. Rejects cycles at insert time. */
    FolderNode add(FileSystemNode child) {
        if (child.contains(this)) {
            throw new IllegalArgumentException("cycle: " + child.name() + " already contains " + name);
        }
        children.add(child);
        return this;
    }
    // ... remove, children() returns Collections.unmodifiableList, name(), print, contains ...
    /** The recursion lives in ONE place. */
    @Override
    public long size() {
        long total = 0;
        for (FileSystemNode child : children) total += child.size();
        return total;
    }
}
```

`MenuComponent` / `Menu` / `MenuItem` is the *transparent* variant (*Head First*'s restaurant menu, prices in `BigDecimal`). Output: `report(project)` → 2150 bytes and `report(readme)` → 1200 with the same call; `src.add(project)` is rejected as a cycle; `Tea.add(...)` throws "Tea is a leaf".

Full runnable code: code/lld/ch09/composite/ (see the Full Edition).

Transparent vs safe composite

	Transparent	Safe
`add/remove` live	On the shared Component	Only on the Composite
Uniformity	Full, no casts	Builders must hold a `FolderNode`
Type safety	`leaf.add(...)` compiles, **throws at run time**	Compiler rejects `file.add(...)`
Ours	`MenuComponent` ("Tea is a leaf")	`FileSystemNode` / `FolderNode`
Choose when	Callers add children to arbitrary nodes (UI editor)	Tree built in one place, read in many (most cases)

Neither is wrong — check the choice was **deliberate**. *Head First* notes transparency knowingly bends SRP.

Two traps worth knowing

- **Cycles** — a folder inside its own descendant makes `size()` recurse forever (`StackOverflowError`). **Validate on insert.**
- **Repeated work** — `print()` calling `size()` at every level is ~quadratic; for big trees **cache subtree totals** and invalidate upward (needs parent pointers).

Trade-offs

Pros	Cons
No type checks in callers	Shared interface **bloats**
Recursion in one place	Weaker type safety (transparent)
New node types don't touch callers (Open/Closed)	Deep trees → deep stacks; cycles → endless recursion
Nesting for free	Totals recomputed unless cached

When not to use it

Nothing nests; leaves and containers share little (methods that throw are worse than the type test); deep trees on a hot path; trees modified during traversal.

Common mistakes

Child list typed as the leaf (`List<FileNode>`); returning the mutable child list (bypasses `add()`'s cycle check); `add()` on the interface by accident; no cycle check; using it where one recursive function with one type check would do.

Real-world examples

HTML DOM; `java.awt.Container`, Android `ViewGroup`; file systems; compiler ASTs; `java.io.File` (`listFiles()` returns `null` on a file — the transparent cost); JUnit 3 `TestSuite`; Spring `CompositeCacheManager`.

In modern languages

Sealed types + exhaustive `switch` (Java 21, Kotlin, Rust) are a real alternative — the compiler catches a forgotten node type; prefer them when the set of node types is closed. Functional languages: recursive ADTs + `fold`.

Other patterns it is confused with

Looks like	Difference
Decorator	Wraps **one** and adds behaviour; Composite holds **many** and combines
Iterator	Walks the tree without exposing it; often used together
Visitor	Adds new operations over the structure without editing nodes
Plain recursive function	Enough when only one operation needs the type check

In the interview

> **30-second answer:** "Composite lets a client treat a single object and a tree the same way. Leaf and container share one interface; the container stores children *typed as that interface* — which allows nesting — and delegates and combines. Its value is deleting the `instanceof` in every operation. I choose transparent or safe depending on whether callers need `add` on arbitrary nodes, reject cycles on insert, and cache totals for big trees."

- *Q: Transparent or safe?* — A: Safe if the tree is built in one place; transparent if callers add to arbitrary nodes.
- *Q: New operation without touching every node?* — A: Visitor.

Self-check

1. **Q:** Why `List<FileSystemNode>`, not `List<FileNode>`? **A:** So folders can contain folders.
2. **Q:** Transparent `add` on a leaf? **A:** Compiles, throws `UnsupportedOperationException`.
3. **Q:** Where to detect cycles? **A:** On insert.

TL;DR

Part-whole trees, one interface; leaf answers, composite asks children and combines. Child list holds the **interface**. Safe vs transparent; guard **cycles**; cache big totals.

9.4 Decorator Pattern

What it is

Decorator extends **one object** at run time by wrapping it in an object with **the same interface**; the original class is never edited. GoF intent: *attach additional responsibilities to an object dynamically — a flexible alternative to subclassing.* Also nicknamed **Wrapper**. Analogy: espresso + milk + mocha + whip — still a drink.

Term	Ours
Component	`UserRepository`, `Beverage`, `InputStream`
Concrete Component	`DatabaseUserRepository`, `Espresso`
Decorator (base) — holds a Component, forwards everything	`RepositoryDecorator`, `FilterInputStream`
Concrete Decorator — adds one behaviour	`LoggingRepository`, `CachingRepository`, `Mocha`, `UpperCaseInputStream`

Key property: **a decorator IS-A and HAS-A Component**, so it can wrap the real object, another decorator or a test double.

The problem it solves

Logging, caching, retries, metrics — each caller wants its own mix. **Subclassing**: 4 optional concerns → 2^4 = **16** classes, doubling per concern. **Flags**: `new Repo(true, false, true, true)`. Decorator: **one class per concern** (4), stacked at run time — Open/Closed.

How it works

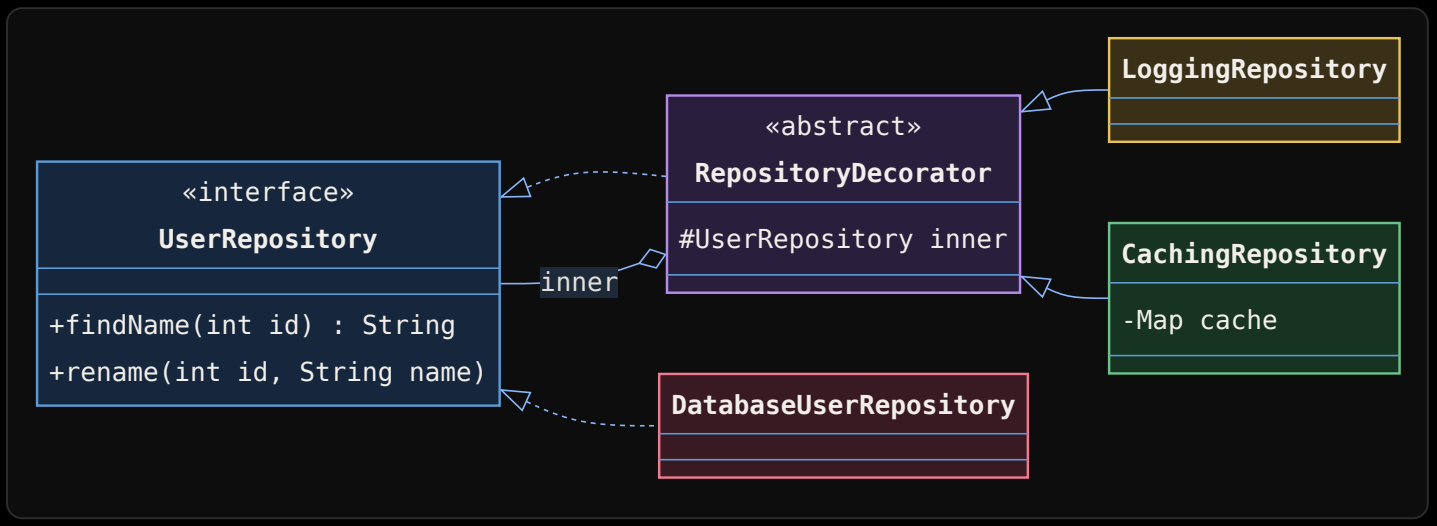
1. **Component interface**. 2. **Concrete component** doing the real work. 3. **Abstract base decorator** holding an `inner` of the **interface type**, **forwarding every method**. 4. **Concrete decorators** override only what they change, working before/after `inner`. 5. **Compose**: `new Logging(new Caching(new Database()))`.



Figure — The same three classes stacked in two orders. Logging outside caching logs every call; caching outside logging means the second call (a cache hit) is never logged (course diagram)

Structure

The bug that bites is **forgotten delegation** — a decorator that overrides `findName` but swallows `rename`. The abstract base decorator forwarding everything is the Java guard.



Java example

```java
abstract class RepositoryDecorator implements UserRepository {
    protected final UserRepository inner;

    protected RepositoryDecorator(UserRepository inner) { this.inner = inner; }

    @Override public String findName(int id)          { return inner.findName(id); }
    @Override public void rename(int id, String name) { inner.rename(id, name); }
}

/** CONCRETE DECORATOR 2 - note it must also invalidate on writes. */
final class CachingRepository extends RepositoryDecorator {
    private final Map<Integer, String> cache = new HashMap<>();

    CachingRepository(UserRepository inner) { super(inner); }

    @Override
    public String findName(int id) {
        if (cache.containsKey(id)) {
            System.out.println("    [cache] hit for " + id);
            return cache.get(id);
        }
        System.out.println("    [cache] miss for " + id);
        String name = super.findName(id);
        cache.put(id, name);
        return name;
    }

    @Override
    public void rename(int id, String name) {
        cache.remove(id);
        super.rename(id, name);
    }
}

/** Add-ons are decorators: they ARE a Beverage and HAVE a Beverage. */
record Milk(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Milk"; }
    public int costCents() { return inner.costCents() + 10; }
}
```

`LoggingRepository` prints then delegates. `new Whip(new Mocha(new Mocha(new Espresso())))` = 1.99 + 0.20 + 0.20 + 0.10 = **$2.49** (double mocha = wrap twice; prices in int cents). `UpperCaseInputStream extends FilterInputStream` is our own `java.io` decorator (ASCII only).

Full runnable code: code/lld/ch09/decorator/ (see the Full Edition).

Layer order changes behaviour

- **Logger outside, cache inside** — every request logged → the log shows *what callers asked*.
- **Cache outside, logger inside** — only misses logged → the log shows *what hit the database*.

Pair	Outside	Inside
Retry vs cache	Repeats the whole lookup	Repeats only the DB call; success fills the cache. Put retry **next to what fails**
Metrics vs cache	Caller-perceived time	DB-only time (what on-call is paged on); often record both
Authorization vs cache	**Correct**	**Security bug** — a cache hit skips the check

The compiler accepts every order — **comment the reason where the stack is built.**

Two related traps

- **Stateful decorators applied twice** (a counter counts every call twice).
- **Lost identity** — `==`, `equals`, `instanceof` on the wrapper differ from the wrapped object.

Decorator vs inheritance

	Subclassing	Decorator
Chosen	Compile time	Run time
How many	One subclass	Any number, any order
Classes for N features	Up to 2^N	N
`final` / not-owned classes	No	Yes (needs only the interface)
Private state	Protected members	No — public interface only

Java I/O streams: the decorator you already use

`new BufferedReader(new InputStreamReader(new FileInputStream("data.txt"), UTF_8))` — `FileInputStream` is the concrete component, `InputStreamReader` is strictly an **adapter** (bytes → chars), `BufferedReader` a **decorator**. `FilterInputStream` is the byte-side base decorator; `BufferedInputStream`, `DataInputStream`, `PushbackInputStream`, `GZIPInputStream`, `DigestInputStream` are concrete decorators. Downside (*Head First*): many small classes.

Trade-offs

Pros	Cons
Any mix without a class per mix (4, not 16)	Deeper stack traces
Concerns tested in isolation (SRP)	**Identity surprises**
Assembled at run time	**Order matters**, unchecked
Wrapped class never edited (Open/Closed)	Over-layering

When not to use it

Exactly one permanent concern; the concern needs private state; the framework already offers it (middleware, interceptors, Spring AOP); the stack would get deep.

Common mistakes

Holding the concrete class instead of the interface (can't stack); forgetting to forward methods; a cache that doesn't invalidate on writes; authorization inside a cache; relying on `instanceof` of the outer object.

Real-world examples

`java.io`; `Collections.unmodifiableList/synchronizedList/checkedList`; `HttpServletRequestWrapper`; HTTP middleware; Resilience4j/Polly retry/circuit-breaker stacks (order documented); Python `@lru_cache`.

In modern languages

Python function decorators, JS higher-order functions, Go struct embedding, C# `DelegatingHandler`.

In the interview

> **30-second answer:** "Decorator adds responsibilities at run time by wrapping an object in another with the same interface. Four optional concerns would be sixteen subclasses; with decorators each concern is one class and I compose the stack. The decorator holds the interface, which lets layers stack. Order matters: retry closest to what fails, authorization outside any cache."

- *Q: Decorator vs Proxy?* — A: Same structure; Decorator **adds behaviour** to an object it's handed (often stacked), Proxy **controls access** and often creates the real object itself. The boundary is fuzzy — a caching wrapper can be either.

- *Q: vs Chain of Responsibility?* — A: In Decorator every layer normally works and forwards; in CoR typically one handler handles the request.

Self-check

1. **Q:** 5 optional features: decorators vs subclasses? **A:** 5 vs up to 32.
2. **Q:** Why is "authorization inside cache" a bug? **A:** A cache hit returns data without the check.
3. **Q:** Is `InputStreamReader` a decorator? **A:** No — an adapter; `BufferedReader` is the decorator.

TL;DR

Wrap to add behaviour, keep the interface (IS-A + HAS-A). N features = N classes, any order — **order matters**. Java I/O, `synchronizedList`, middleware, Resilience4j. A decorator that short-circuits drifts toward Proxy.

9.5 Facade Pattern

What it is

Facade gives a complicated subsystem **one simple front door**. GoF intent: *provide a unified interface to a set of interfaces in a subsystem; a higher-level interface that makes it easier to use.* Analogy: a hotel concierge — you can still call the restaurant yourself.

Term	Ours
Facade — knows which classes to call, in what order, and the cleanup	`OrderFacade`
Subsystem classes — do the work, unaware of the facade	`InventoryService`, `PaymentService`, `ShippingService`, `EmailService`
Client	`Main` / checkout page

The problem it solves

Placing an order = **reserve stock → charge card → create shipment → send confirmation**, repeated by every caller — and each must also undo the reservation if the card is declined. The **undo** is where bugs live: forget it and units stay locked. The subsystem is fine (flexible = many knobs); callers just keep rewriting the standard path.

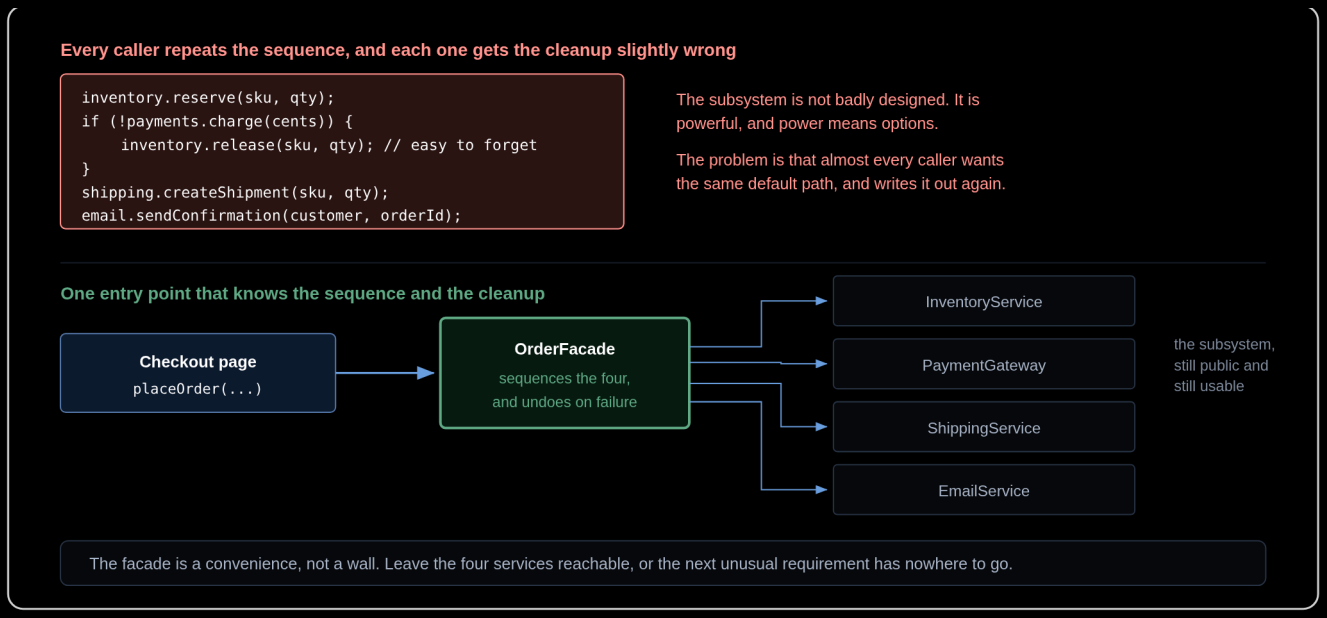


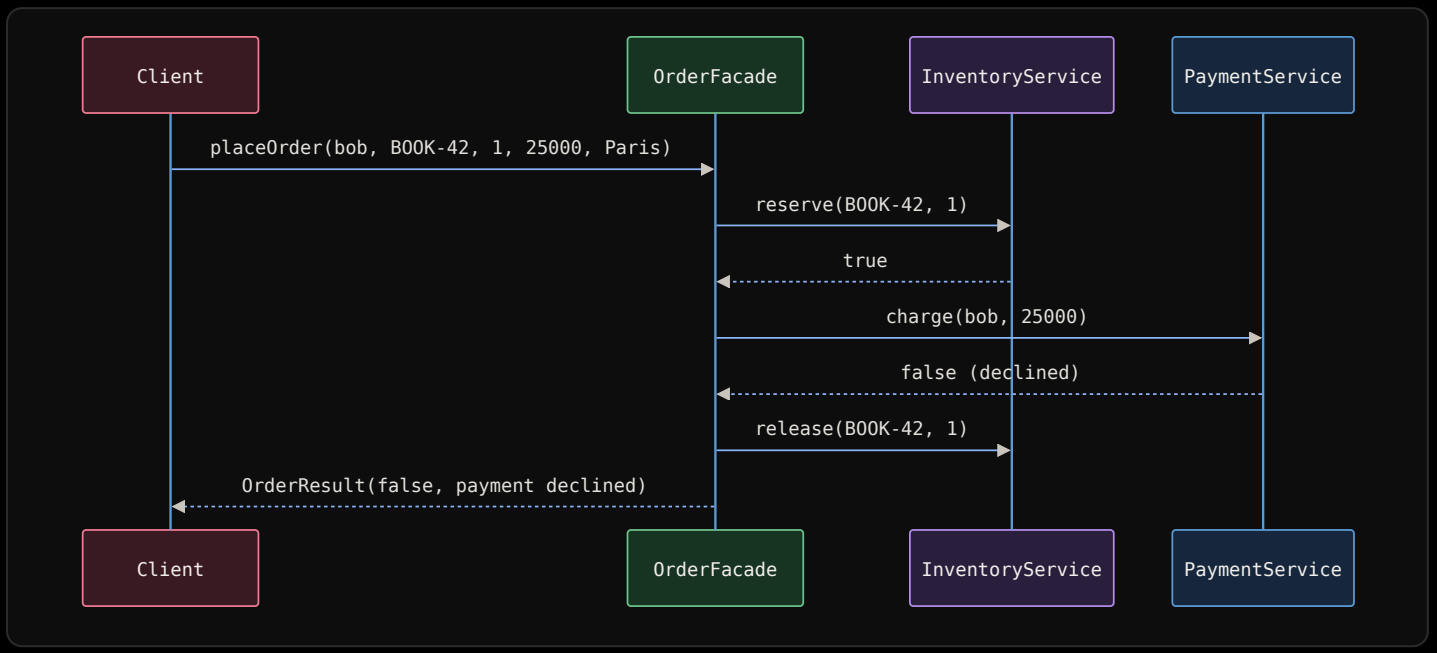
Figure — Top: each caller repeats the reserve/charge/release/ship/email sequence and may forget the release. Bottom: the checkout page calls `OrderFacade.placeOrder(...)`, which sequences the four services and undoes on failure; the services stay public (course diagram)

How it works

1. Identify the **common path**. 2. A **facade class** holding the subsystem objects. 3. One **intention-revealing method** (`placeOrder`) calling them in order. 4. **Own the failure handling** (a *compensating action*: release on decline). 5. **Keep the subsystem public**. 6. **No business rules** — it *sequences*, not *decides*.

Structure

Dependencies run **one way** — no service knows the facade. Usually a concrete class, no interface (unless you'll swap the whole subsystem).



Java example

```
final class OrderFacade {
    private final InventoryService inventory;
    private final PaymentService payments;
    private final ShippingService shipping;
    private final EmailService email;
    private int orderSeq = 100;
    // ... constructor takes the four services ...
    OrderResult placeOrder(String customer, String sku, int qty, long cents, String address) {
        String orderId = "ORD-" + orderSeq++;
        if (!inventory.reserve(sku, qty)) {
            return new OrderResult(false, orderId, "out of stock");
        }
        if (!payments.charge(customer, cents)) {
            inventory.release(sku, qty);            // compensating step callers used to forget
            return new OrderResult(false, orderId, "payment declined");
        }
        String shipmentId = shipping.createShipment(sku, qty, address);
        email.sendConfirmation(customer, orderId);
        return new OrderResult(true, orderId, "shipped as " + shipmentId);
    }
}
```

`OrderResult` is our own small result type (no subsystem types leak). `Main` still calls `inventory.reserve(...)` directly — **a convenience, not a wall**.

Full runnable code: code/lld/ch09/facade/ (see the Full Edition).

Facade vs Adapter vs Mediator

	Facade	Adapter	Proxy	Mediator (Ch. 10–11)
Intent	Simplify a subsystem	Convert an interface	Control access	Coordinate peers
Objects wrapped	**Several**	One	One	Several colleagues
Interface	New, smaller	Client's Target	**Same** as real	Mediator's own
Who chose it	**You**	**Someone else**	Fixed by real object	You
Wrapped objects know it?	**No**	No	No	**Yes** — talk through it
Direction	Client → facade → subsystem	Client → adapter → adaptee	Client → proxy → subject	Colleague ↔ mediator ↔ colleague
Example	`OrderFacade.placeOrder`	`StripeGateway.pay`	`LazyReport`	Air-traffic control; chat room

Fastest test: Adapter/Decorator/Proxy stand in front of **one** object; a Facade in front of **several**. If the subsystem's API became perfect tomorrow, a Facade would still simplify; an Adapter would be pointless. **Facade vs service layer:** same thing while it only sequences; with business rules it has become a domain/application service.

The failure mode to watch for

Facades **pull in features** — a discount rule, then a fraud check, then a **god object**. Rule: **a facade orchestrates; it does not decide.** Related principle (*Head First*): **Least Knowledge / Law of Demeter** — clients talk to the facade, not chains of subsystem objects.

Trade-offs

Pros	Cons
One call replaces a sequence	One more class to maintain
Order and undo in one place	**Feature magnet** → god object
One seam for swapping the subsystem	Opinionated — some caller needs an omitted option
Smaller public surface, lower coupling	Can mask problems; single point of failure

When not to use it

The subsystem is already simple; callers need the full range (every caller bypasses it); covering a mess instead of fixing it; wanting it to be the *only* entrance.

Common mistakes

Business rules in the facade; making the subsystem inaccessible; returning subsystem types; forgetting the failure path; confusing it with Mediator.

Real-world examples

Application service layers (`OrderService`); `java.net.URL.openStream()` (constructors deprecated since Java 20 — use `URI.create(s).toURL()`); SLF4J "Simple Logging Facade"; `FacesContext`; jQuery's `$`; Python `requests` over `urllib3`; e-commerce "Place order" orchestrators / backend-for-frontend.

In modern languages

Often just a module or package exporting one function (or a JPMS module exporting only the facade package).

In the interview

> **30-second answer:** "A Facade gives one simple entry point to a complex subsystem. `OrderFacade.placeOrder` reserves, charges, ships and emails in order, and releases the reservation if payment fails, so no caller has to remember the cleanup. It wraps several objects, unlike Adapter, Decorator and Proxy. I keep the subsystem public and business rules out — the risk is a god object."

- *Q: vs Mediator?* — A: Subsystem classes don't know the facade; colleagues know the mediator and talk through it both ways.
- *Q: Several facades?* — A: Yes — one per use case, or layered.

Self-check

1. **Q:** Fastest structural tell vs Adapter/Decorator/Proxy? **A:** It wraps several objects.
2. **Q:** What happens on a declined payment? **A:** It releases the reservation and returns a failed `OrderResult`.
3. **Q:** Add a 10% discount inside `placeOrder`? **A:** No — the facade sequences, a service decides.

TL;DR

One door into a complex subsystem; names the common path and owns the cleanup. Wraps **several**; subsystem stays **public**; **sequences, never decides**.

9.6 Flyweight Pattern

What it is

Flyweight shares the part of each object's state that is the same across many objects so huge numbers fit in memory. GoF intent: *use sharing to support large numbers of fine-grained objects efficiently.* Analogy: one type design for every "e", placed at different positions.

Term	Meaning	Map example
Intrinsic state	Identical for every holder; lives **inside** the shared flyweight	icon bitmap, colour, size
Extrinsic state	Varies per context; stays **outside**, **passed in** as arguments	latitude, longitude

Role	Ours
Flyweight (shared, immutable, intrinsic only)	`MarkerStyle`
Flyweight factory (owns the pool)	`MarkerStyleFactory`
Context (extrinsic state + reference)	`Marker`
Client	`Main`

GoF's own list: *Flyweight* (interface accepting extrinsic state), *ConcreteFlyweight* (`MarkerStyle` plays both), *UnsharedConcreteFlyweight* (same interface, not shared), *FlyweightFactory*, *Client*; "Context" is a later name (refactoring.guru).

The problem it solves

100,000 map markers, each with its own position plus a 2 MB icon bitmap → ~195 GB: the program **runs out of memory**. Same for a million game trees sharing one mesh, or font data per character.

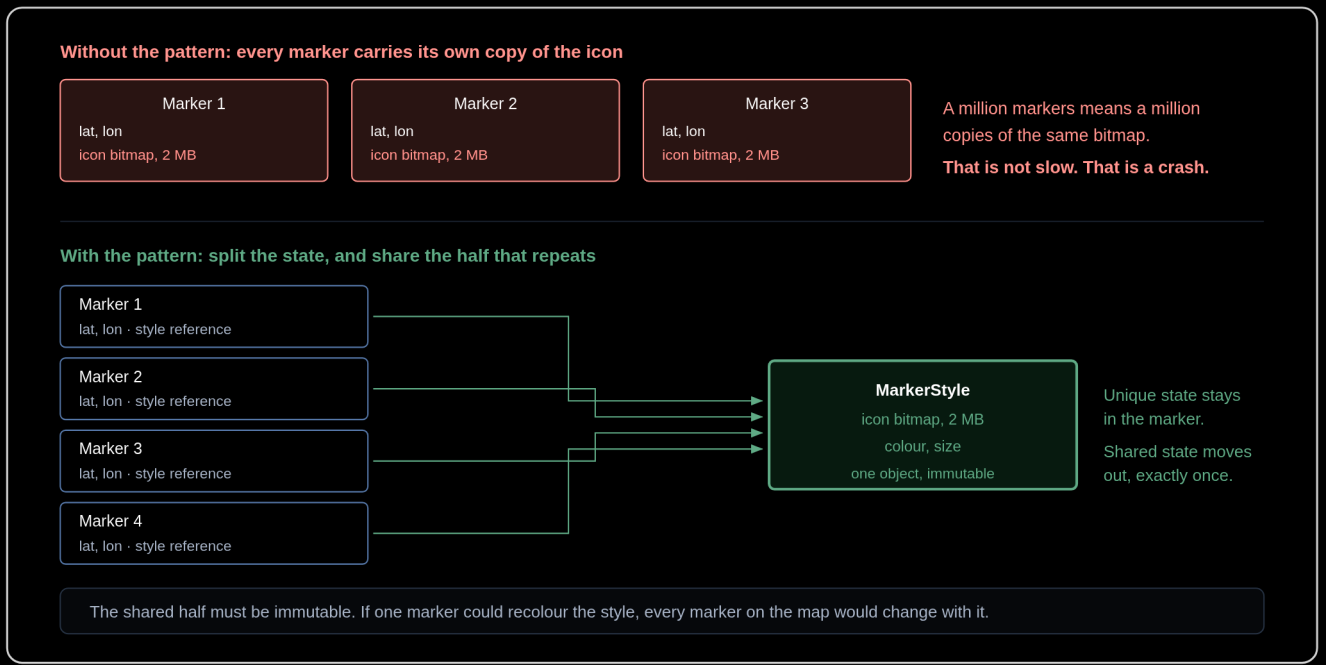


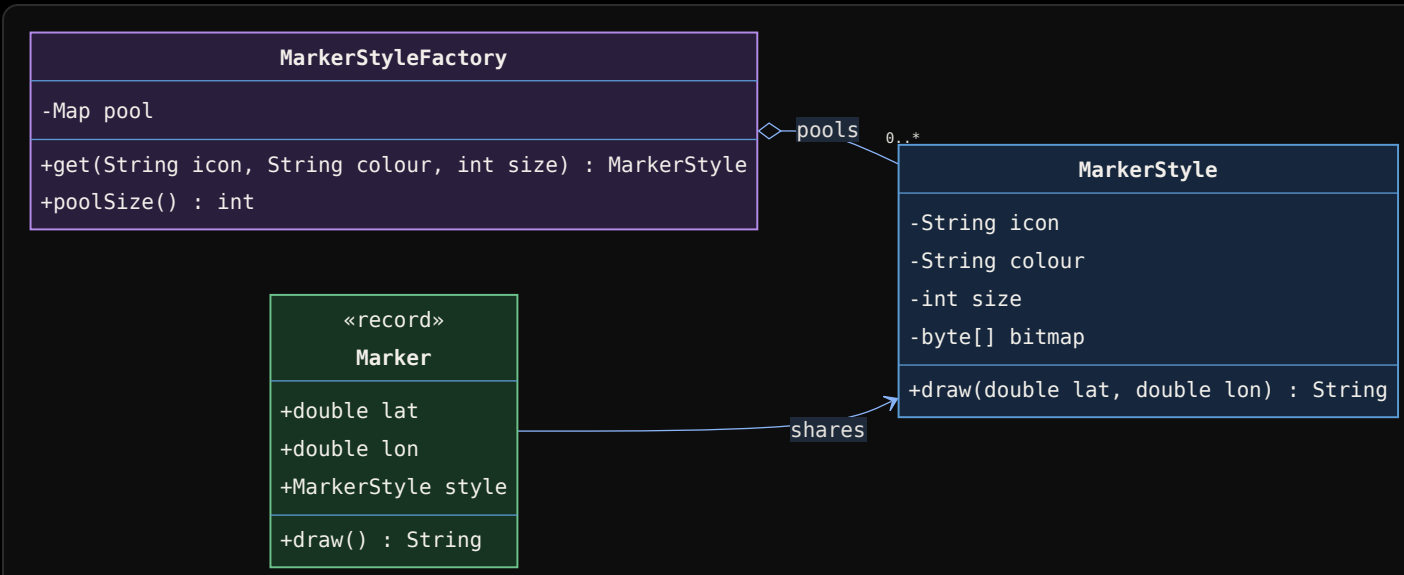
Figure — Top: every marker carries its own 2 MB icon. Bottom: each marker keeps only lat/lon and a style reference; one immutable `MarkerStyle` holds the icon, colour and size (course diagram)

How it works

1. **Split** intrinsic vs extrinsic. 2. Intrinsic → an **immutable** flyweight (`final`, no setters). 3. Extrinsic → **method arguments** (`draw(lat, lon)`). 4. A **factory owns the pool**, keyed by intrinsic values, returning existing or creating once. 5. Clients **only use the factory** (non-public constructor). 6. Contexts hold a **reference**, not a copy.

> **Two rules — break either and the pattern silently fails:** (1) **flyweights never change**; (2) **only the factory creates them**.

Structure



Java example

```java
final class MarkerStyle {
    static final int ICON_BYTES = 2 * 1024 * 1024;   // pretend every icon bitmap is 2 MB

    private final String icon;
    private final String colour;
    private final int size;
    private final byte[] bitmap;   // the heavy part we want exactly one copy of
    // ... package-private constructor loads the bitmap ...
    /** EXTRINSIC state (the position) is passed in, never stored. */
    String draw(double lat, double lon) {
        return String.format("%s %s marker at (%.4f, %.4f)", colour, icon, lat, lon);
    }
    // ... bitmapBytes() ...
}

/** FLYWEIGHT FACTORY: owns the pool. Thread-safe via ConcurrentHashMap.computeIfAbsent. */
final class MarkerStyleFactory {
    /** The pool key is the combination of intrinsic values. */
    private record Key(String icon, String colour, int size) { }

    private final Map<Key, MarkerStyle> pool = new ConcurrentHashMap<>();

    MarkerStyle get(String icon, String colour, int size) {
        return pool.computeIfAbsent(new Key(icon, colour, size),
                k -> new MarkerStyle(k.icon(), k.colour(), k.size()));
    }
    // ... poolSize() ...
}

/** CONTEXT: one per marker. Holds extrinsic state + a REFERENCE to a shared style. */
record Marker(double lat, double lon, MarkerStyle style) {
    String draw() { return style.draw(lat, lon); }
}
```

`computeIfAbsent` makes get-or-create **atomic and thread-safe**; the `Key` record gives correct `equals` / `hashCode`.

```
markers: 100000, styles in pool: 3
icon memory without flyweight: ~200000 MB
icon memory with flyweight:    ~6 MB
Integer 127 == 127 -> true  (cached -128..127)
Integer 128 == 128 -> false (outside default cache)
```

Full runnable code: code/lld/ch09/flyweight/ (see the Full Edition).

Flyweights already in Java

- `Integer.valueOf` (autoboxing) caches **`−128..127`** (JLS-required); 128 may differ (`-XX:AutoBoxCacheMax` can raise the bound). **Compare boxed numbers with `equals`.** `Short`, `Byte`, `Long` (same range), `Character` (0–127), `Boolean.TRUE/FALSE` likewise.
- **`String` pool** — literals and compile-time constants (`"hel" + "lo"`) are interned; `new String("hello")` is new; `intern()` returns the pooled one.
- **Enums** — one instance per constant.

Flyweight is not a cache

	Flyweight	Cache
Purpose	Make data *fit in memory*	Make repeated work *faster*
Contents	One copy of immutable state shared by many	Stored results of costly work
Remove entries?	Not while referenced	Any time
Effect of removal	Lost sharing / duplicates	Only a slower next call

Two mistakes that undo the pattern

- **A mutable flyweight** — one caller changes a colour and every marker changes.
- **An ever-growing pool keyed by open-ended input** — a leak disguised as an optimisation. Safe only for a **small, bounded** set of intrinsic values; otherwise you've built a cache (weak refs, eviction) — call it one.

Measure first

It trades readability for memory — without profiling it's premature optimisation. Answer with numbers: *how many objects at once?* and *how much of each is identical?*

Trade-offs

Pros	Cons
Memory can fall by orders of magnitude	State split across two places
Costly construction once per distinct value	The pool holds strong references (can leak)
CPU-cache friendly	**Thread safety is on you**
"Load once" logic in one home	Tempting too early; extrinsic state passed on every call

When not to use it

You haven't measured; the shared part can't be immutable; counts are small; unique state dominates; the key space is unbounded (that's a cache).

Common mistakes

Extrinsic state inside the flyweight; a public constructor; a non-thread-safe pool (`HashMap` check-then-put); comparing boxed `Integer`s or `String`s with `==`.

Other patterns it is confused with

Looks like	Difference
Cache	See table above
Singleton	One instance total vs one **per distinct intrinsic value**
Object pool	Lends **mutable** objects and takes them back; flyweights are immutable, shared, never returned
Prototype	Makes many objects by copying; Flyweight avoids making them

Real-world examples

`Integer.valueOf`, `Boolean.valueOf`, `String.intern()`, enums (Python interns small ints/strings too); glyphs in word processors (GoF's example); game meshes/textures with GPU instancing; map pins; `ZoneOffset.ofHours(5)` (cached 15-minute multiples).

In modern languages

Any immutable value object + memoising factory (Java record + `computeIfAbsent`, JS `Object.freeze` + `Map`, C++ `shared_ptr<const Style>`).

In the interview

> **30-second answer:** "Flyweight saves memory with huge numbers of similar objects. I split state into intrinsic — shared and invariant, like an icon — and extrinsic — per object, like a position, passed into methods. The intrinsic part is an immutable flyweight from a factory that pools one instance per distinct value. `Integer.valueOf` and the String pool are JDK examples. I'd only do it after measuring."

- *Q: Why is `Integer 128 == 128` false?* — A: `valueOf` only guarantees caching for −128..127; use `equals`.
- *Q: Thread-safe factory?* — A: `ConcurrentHashMap.computeIfAbsent` — creation runs at most once per key.

Self-check

1. **Q:** Chess piece: intrinsic vs extrinsic? **A:** Type, colour, image vs its square.
2. **Q:** What if `MarkerStyle` had `setColour`? **A:** Changing one marker changes every marker sharing it.
3. **Q:** When does a pool leak? **A:** When keys are unbounded (e.g. user input) and nothing evicts.

TL;DR

Share what repeats, pass in what varies. **Immutable** flyweight + **factory-owned**, thread-safe pool. `Integer.valueOf` (−128..127), String pool, enums. **Measure first**; not a cache.

9.7 Proxy Pattern

What it is

Proxy puts a **stand-in with the identical interface** in front of a real object to control **who** may use it, **when** it's created, or **how much** each use costs. GoF intent: *provide a surrogate or placeholder for another object to control access to it* (a.k.a. **Surrogate**). Analogy: a credit card standing in for your bank account.

Term	Ours
Subject	`Report`, `RateService`, `Device`
Real Subject	`RealReport`, `LiveRateService`, `RealDevice`
Proxy	`LazyReport`, `AuditedReport`, `CachingRateProxy`, `RemoteRateProxy`, `SharedDevice` handles

The problem it solves

Callers shouldn't touch the real object directly (or yet): it's **costly and usually unneeded** (a 100 MB report when most screens show only its title); **only some callers are permitted**; it's **on another machine**; **each call must be logged/counted** but the class isn't yours. Same interface → insert it **without editing callers** — which is also the danger: a slow call **looks cheap**.

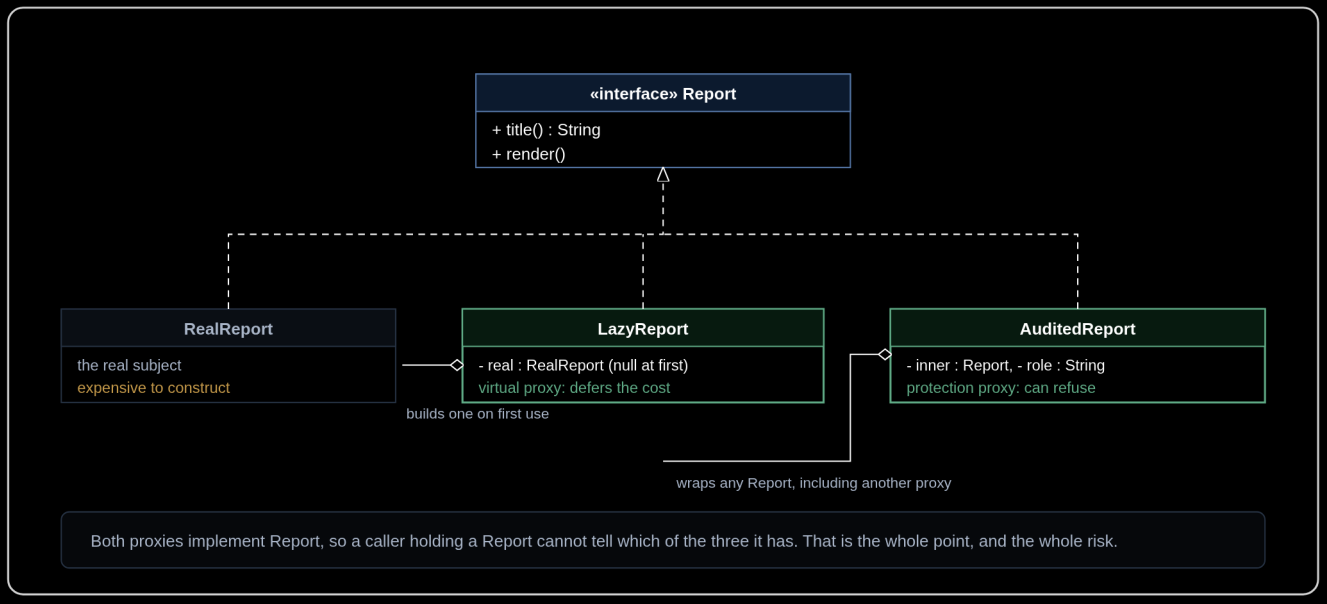


Figure — Proxy class diagram: `RealReport`, `LazyReport` (virtual proxy) and `AuditedReport` (protection proxy) all implement `Report`; `LazyReport` creates the `RealReport` itself, `AuditedReport` wraps any `Report` (course diagram)

How it works

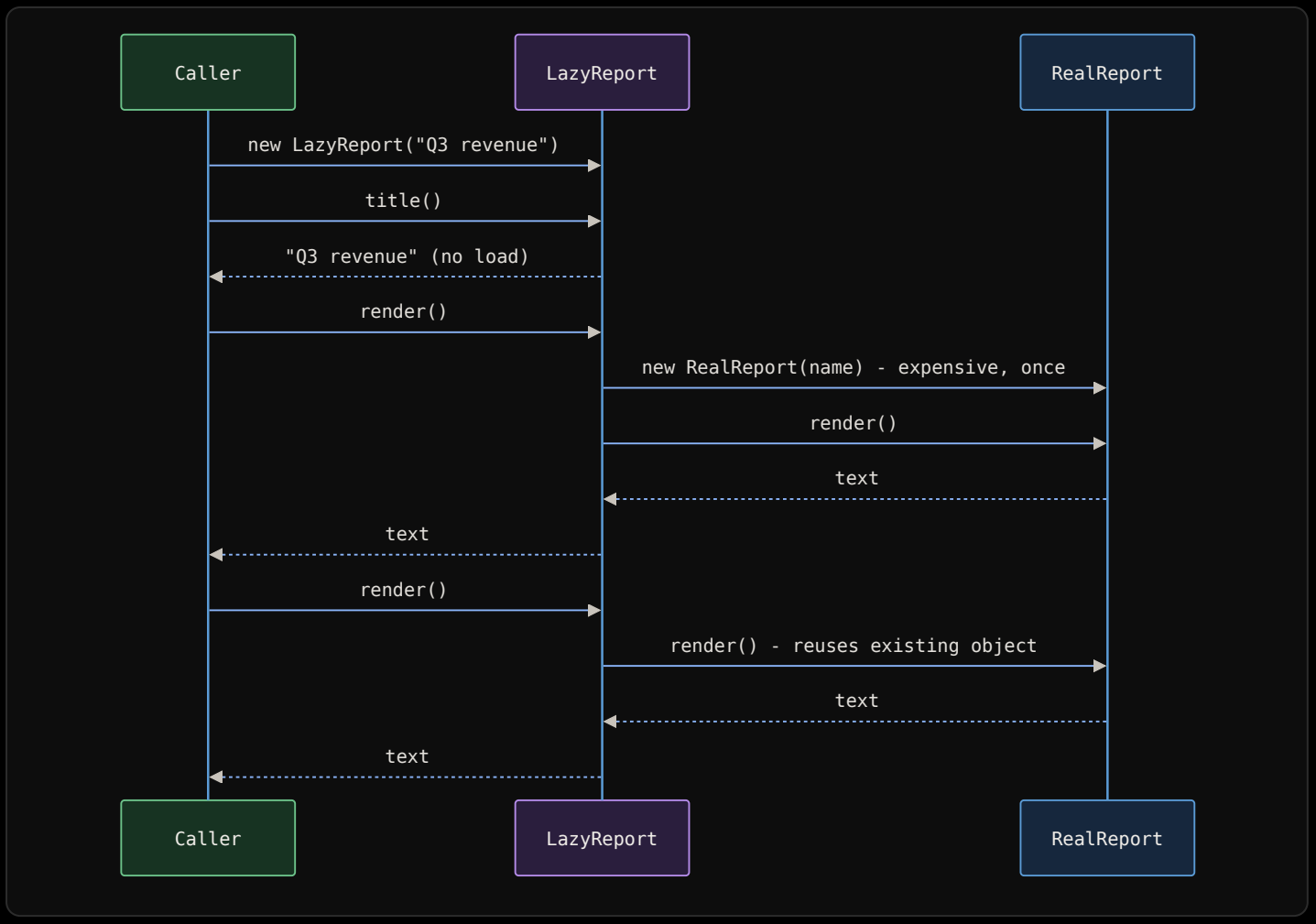
1. Define/reuse the **Subject** interface. 2. A proxy implementing it, holding the real subject **or what's needed to create it**. 3. In each method do the proxy's job (create lazily, check permission, check cache, marshal, count), then **delegate or refuse**. 4. Give callers the proxy (often via a factory or DI).

Kinds of proxy

Kind	Controls	Ours / real
Virtual	*When* it's created (lazy)	`LazyReport`; Hibernate lazy entities
Protection	*Who* may call	`AuditedReport`; `Collections.unmodifiableList`
Remote	*Where* it lives	`RemoteRateProxy` (stub) + `RateServer` (skeleton); RMI/gRPC
Caching	*Whether* the call is needed	`CachingRateProxy` (LRU + TTL)
Logging / monitoring	Records calls	`LoggingHandler` (dynamic proxy)
Smart reference	Bookkeeping: ref counting, locking, load on first use	`SharedDevice` closes on last handle

GoF names four (remote, virtual, protection, smart reference); caching and logging are later variants (GoF would file logging under smart reference).

Structure


Caller
LazyReport
RealReport
new LazyReport("Q3 revenue")
title()
"Q3 revenue" (no load)
render()
new RealReport(name) - expensive, once
render()
text
text
render()
render() - reuses existing object
text
text
Caller
LazyReport
RealReport

Java example

```java
final class LazyReport implements Report {
    private final String name;
    private volatile RealReport real;      // created by the proxy itself, not handed in

    LazyReport(String name) { this.name = name; }

    @Override public String title() { return name; }   // no load needed

    @Override
    public String render() { return realReport().render(); }

    private RealReport realReport() {
        RealReport r = real;
        if (r == null) {
            synchronized (this) {
                r = real;
                if (r == null) real = r = new RealReport(name);
            }
        }
        return r;
    }
}

/** PROTECTION PROXY: checks permission, may REFUSE the call. */
final class AuditedReport implements Report {
    private final Report inner;
    private final Role callerRole;
    private final Set<Role> allowed;
    // ... constructor copies allowed with Set.copyOf ...
    @Override public String title() { return inner.title(); }  // titles are public

    @Override
    public String render() {
        if (!allowed.contains(callerRole)) {
            throw new SecurityException(callerRole + " may not render " + inner.title());
        }
        return inner.render();
    }
}
```

- **Thread-safe lazy init** — double-checked locking; the field **must be** `volatile`, or two threads may build twice (a real bug if construction opens a connection or charges a card).
- `AuditedReport` wraps a `LazyReport`: the viewer is refused **before anything loads**.
- `CachingRateProxy` is **bounded** (LRU via access-ordered `LinkedHashMap` + `removeEldestEntry`) and **time-limited** (TTL): 5 calls → 4 lookups. Rates are `BigDecimal`.
- `RemoteRateProxy` marshals `"rate|USD|INR"` over a simulated network to `RateServer`.
- `SharedDevice` hands out handles; the device opens on the first and closes on the last, all under one lock, `close()` idempotent.

Full runnable code: code/lld/ch09/proxy/ (see the Full Edition).

Static vs dynamic proxies

	Static	Dynamic
Made by	Hand-written per interface	Generated at **run time**
Code	One class each, every method	One `InvocationHandler` for **any** interface
Type safety	Full	Reflective (`Method` + `Object[]`)
Good for	Type-specific logic (lazy load, domain access rules)	**Cross-cutting**: logging, timing, transactions, security
API	—	`Proxy.newProxyInstance(loader, interfaces, handler)`
Limit	Boilerplate	JDK proxies are **interface-only**; classes need CGLIB/ByteBuddy

JDK mechanics: `newProxyInstance` generates `$Proxy0` implementing the interfaces → every call goes to `handler.invoke(proxy, method, args)` → the handler works and calls `method.invoke(target, args)` → **unwrap** `InvocationTargetException` (`e.getCause()`) so callers see the real exception.

The self-invocation trap (and Spring AOP)

In the demo, `convert` called `rate(...)` on `this`, bypassing the logging proxy. **Spring AOP** works the same way: `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize` and `@Aspect` advice wrap your bean in a proxy — a **JDK dynamic proxy** if it implements an interface, else **CGLIB** (Spring Boot 2.0+ defaults to CGLIB). **Self-calls aren't intercepted** (`this.saveAudit()` with `REQUIRES_NEW` is silently ignored); with CGLIB, `final` classes can't be proxied and `final`/`private` methods can't be advised. Fixes: move the method to another bean; inject the bean's own proxy (lazy self-reference); or (discouraged) `AopContext.currentProxy()` with `exposeProxy = true`. AspectJ weaving avoids proxies altogether.

Proxy vs Decorator

GoF: similar implementations, different **purpose** — Decorator *adds responsibilities*, Proxy *controls access*.

	Proxy	Decorator
Purpose	**Controls access**	**Adds behaviour**
Call gets through?	Not necessarily — defer, cache, refuse	Normally yes
Who creates the real object?	**Often the proxy**	**The caller** hands it in
Client knows?	Usually not (factory/DI supplies it)	Usually yes (builds the stack)
Layers	Usually one	Often several

Creation test: `LazyReport(String name)` builds its subject; `LoggingRepository(UserRepository inner)` is given one. `AuditedReport` *is* handed its subject — only intent separates it from a decorator. A caching wrapper is honestly both; ask instead whether its cache is bounded and invalidated.

Where proxies go wrong

Unsynchronised lazy creation (duplicates); a caching proxy with no bound or expiry (leak + stale data); **concealed cost** — a remote proxy used in a loop (fix by making cost visible: batch APIs, `fetch...` names); **ORM N+1** — touching `order.getCustomer().getName()` for 100 orders fires 100 queries.

Trade-offs

Pros	Cons
Pay later for costly construction	Extra indirection in stack traces
No caller changes (Open/Closed)	**Latency and failures disguised** as local calls
Home for cross-cutting rules on classes you can't edit	Lazy creation raises thread-safety questions
Permission checks kept out of the object	Unbounded caches leak and go stale

When not to use it

The real object is cheap; callers need to see the cost (`fetchOverNetwork()`); the framework already generates proxies (ORM, DI, service mesh); you want to add behaviour (Decorator).

Common mistakes

Non-`volatile` DCL field; not unwrapping `InvocationTargetException`; expecting Spring annotations on self-calls, `private` or `final` methods; unbounded caching proxies; remote proxies in tight loops.

Real-world examples

ORM lazy loading (Hibernate `getReference`, lazy associations — the source of N+1 and `LazyInitializationException`); **Spring AOP** (and CGLIB-enhanced `@Configuration` classes unless `proxyBeanMethods = false`); RMI/gRPC/Thrift/SOAP stubs; `java.lang.reflect.Proxy` (Spring Data repositories, MyBatis mappers); `Collections.unmodifiableList`; Envoy/Istio sidecars, Nginx/Cloudflare reverse proxies; C++ `shared_ptr`.

In modern languages

JS built-in `Proxy`; Python `__getattr__`; C# `DispatchProxy` and `Lazy<T>`; C++ smart pointers.

Other patterns it is confused with

Looks like	Difference
Decorator	Adds behaviour to a handed-in object; Proxy controls access, often creates the subject
Adapter	**Different** interface on purpose; Proxy has the **same** one
Facade	New simpler interface over **several**; Proxy the same interface over **one**
A cache	A caching proxy *is* a cache — still needs bounds and eviction

In the interview

> **30-second answer:** "A Proxy is a same-interface stand-in that controls access. I name the kind: virtual defers creation, protection checks permissions, remote hides the network, caching reuses results, smart reference does bookkeeping like ref counting. Callers don't change — which is also the danger, since an expensive call looks cheap. Lazy creation needs thread safety, and ORM lazy proxies cause N+1. In Java I'd use `java.lang.reflect.Proxy` for cross-cutting concerns — that's how Spring AOP does `@Transactional`."

- *Q: Why doesn't `@Transactional` work on a same-class call?* — A: `this.method()` never goes through the proxy.
- *Q: JDK proxy vs CGLIB?* — A: JDK needs interfaces; CGLIB subclasses, so no `final` classes/methods.
- *Q: N+1 fix?* — A: Fetch joins, entity graphs or batch fetching.

Self-check

1. **Q:** Kinds of `LazyReport` and `AuditedReport`? **A:** Virtual; protection.
2. **Q:** Why wasn't the inner `rate()` logged? **A:** Self-invocation — `convert()` called `rate()` on `this`.

3. **Q:** Two must-haves for a caching proxy? **A:** A size bound and an expiry/invalidation policy.

TL;DR

Same interface, controlled access: virtual, protection, remote, caching, logging, smart reference. Static = hand-written; dynamic = `Proxy` + `InvocationHandler`. Spring AOP = proxies → beware self-invocation, `final`, `private`; watch thread safety and hidden costs.

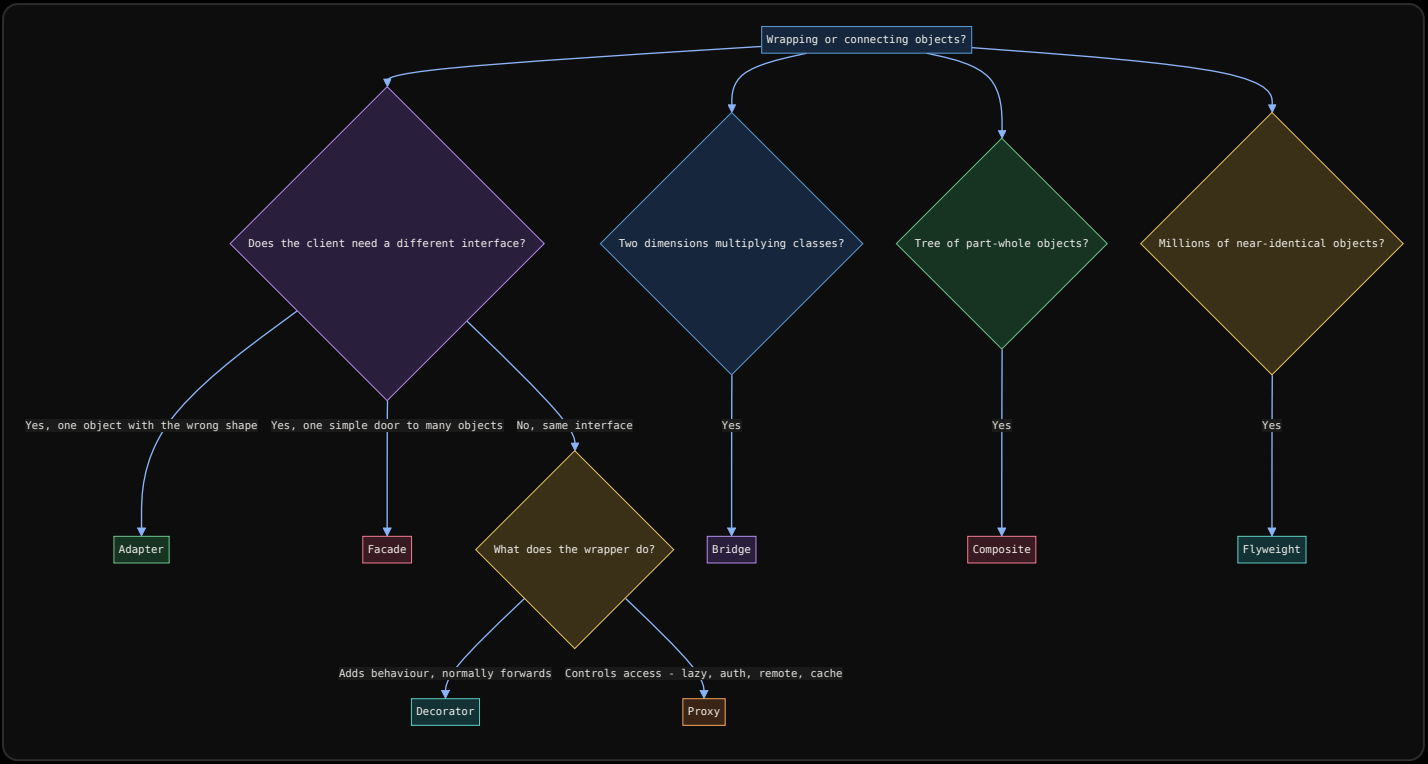
9.8 Putting it all together — comparison

Summary of all seven structural patterns

Pattern	Distinctive feature	Use it when	Example
Adapter	Bridges incompatible interfaces	An existing class doesn't fit	Plug adapter; `StripeGateway`
Bridge	Separates abstraction from implementation	Two dimensions vary independently	Shapes × renderers; notifications × channels
Composite	Part-whole tree, one interface	Treat single and group uniformly	Drawing groups; file system
Decorator	Adds responsibilities dynamically	Run-time combinations of behaviour	Scroll/border on a view; Java I/O
Facade	Simple interface to a subsystem	Callers repeat one common path	"Watch movie" button; `OrderFacade`
Flyweight	Shares common state	Huge numbers of similar objects	Word-processor glyphs; map markers
Proxy	Placeholder controlling access	Lazy, auth, remote, cache, logging, ref counting	Proxy server; Hibernate lazy entity

Adapter vs Decorator vs Proxy vs Facade vs Bridge

	Adapter	Decorator	Proxy	Facade	Bridge
Intent	Convert an interface	Add behaviour, same interface	Control access	Simplify a subsystem	Vary abstraction and implementation independently
Client sees	**Different** (yours)	**Same**	**Same**	**New, simpler**	Abstraction's own
Behaviour	Unchanged (translated)	**Extended**	**Controlled**	Unchanged, fewer calls	Delegated to implementor
Wraps	One	One (stackable)	One	**Several**	One implementor
Creates wrapped?	Passed in	Passed in by client	**Often itself**	Passed in / owned	Passed in
Introduced	**After the fact**	When a concern is added	When access needs control	When callers repeat a sequence	**Up front**
Trigger	"Vendor API doesn't match"	"Sometimes logging, sometimes caching"	"Expensive / remote / restricted"	"Every caller copies 5 steps"	"Two-noun class names"
Example	`Arrays.asList`, `InputStreamReader`	`BufferedInputStream`, `synchronizedList`	`reflect.Proxy`, `@Transactional`, Hibernate	SLF4J, service layers	JDBC + drivers, AWT peers



Memory hooks

Adapter — make it fit · **Decorator** — add to it · **Proxy** — guard it · **Facade** — simplify it · **Bridge** — split it · **Composite** — nest it · **Flyweight** — share it.

Chapter self-check

1. **Q:** Which wrap exactly one object? **A:** Adapter, Decorator, Proxy (Bridge holds one implementor by design). Facade wraps several; Composite holds many.
2. **Q:** Same code shape, different intent/timing? **A:** Adapter vs Bridge (after vs up front); Bridge vs Strategy; Decorator vs Proxy.
3. **Q:** `WindowsButton`, `MacButton`, `WindowsCheckbox`… and Linux coming? **A:** Bridge (widget × platform), often with Abstract Factory choosing the implementor.
4. **Q:** Why is `Integer.valueOf(100) == Integer.valueOf(100)` true? **A:** Flyweight (Integer cache).

Chapter TL;DR

- Structural patterns **compose objects into bigger structures** while staying flexible.
- **Adapter / Decorator / Proxy / Facade** share a shape — tell them apart by **what changes as the call passes** and **how many objects are wrapped**.
- **Bridge** stops two-dimension class explosion; **Composite** makes trees uniform; **Flyweight** shares immutable state.
- In interviews: name the pattern, the **constraint** justifying it, its **trap** (vendor leaks, wrong split, cycles, decorator order, god facade, mutable flyweight, self-invocation) and a **real JDK/Spring example**.

Chapter 10 — Behavioral Design Patterns (Part 1) · Study Edition

> **Study edition.** Every topic of the full chapter, with the padding removed. For complete code listings, full outputs and extra follow-ups, see the Full Edition (`README.md`).
>
> **What you will learn.** What behavioral patterns are, and six of them: **Chain of Responsibility, Command, Iterator, Mediator, Memento** and **Interpreter** — definition, problem, class diagram, core Java 21 code, real uses and interview answers. Chapter 11 covers Observer, State, Strategy, Template Method, Visitor and some non-GoF patterns.

How to run the code

```
cd chapters/10-patterns-behavioral-1/code
javac --release 21 -d /tmp/lld_10 $(find . -name '*.java')
java -cp /tmp/lld_10 lld.ch10.chain.Main        # also: command, iterator, mediator, memento, interpreter
```

10.0 Behavioral patterns: the big picture

What it is

Creational patterns (Ch 8) are about **how objects are made**; structural (Ch 9) about **how they are put together**. **Behavioral patterns** are about **how objects share work and talk** — who is responsible for what, and who calls whom (in a kitchen: who hears about an order first, who passes it on, who can cancel it).

They cut coupling, untangle many-to-many communication and let behaviour change **at run time**.

The eleven GoF behavioral patterns

The GoF book (*Design Patterns*, Gamma, Helm, Johnson, Vlissides, 1994) lists eleven:

Pattern	One-line intent	Ch
Chain of Responsibility	Pass a request down handlers; each handles, stops or forwards it.	10
Command	Wrap a request in an object to store, queue, log or reverse it.	10
Interpreter	A small language's grammar as classes; evaluate sentences.	10
Iterator	Walk a collection without seeing how it is stored.	10
Mediator	Route communication through one coordinator.	10
Memento	Snapshot state and restore it, keeping it private.	10
Observer	Notify all dependants when one object changes.	11
State	Change behaviour when internal state changes.	11
Strategy	Pick an interchangeable algorithm at run time.	11
Template Method	Base class fixes the skeleton; subclasses fill steps.	11
Visitor	Add an operation without changing the classes.	11

Class vs object scope. *Class* behavioral patterns use **inheritance**: only **Template Method** and **Interpreter**. All others are *object* patterns using **composition**, so they can be rewired at run time.

The common idea: take behaviour hidden in a big method (an `if`-chain, a hard-coded call, a loop) and turn it into an **object** you can name, pass, swap, store and test.

10.1 Chain of Responsibility

What it is

Chain of Responsibility (CoR) sends a request through a row of **handlers**. Each handler either handles it, rejects it, or forwards it to its successor. The sender talks only to the first handler and never learns which one handled it.

GoF intent: *avoid coupling sender to receiver by giving several objects a chance to handle the request; chain them and forward until one does.* Participants: **Handler**, **ConcreteHandler**, **Client**. Analogy: support escalation — level 1, level 2, engineer; you dialled one number.

The problem it solves

Before creating an order a request must pass authentication, rate limiting and validation. In one method: the method grows with each rule; the **order of checks is hidden** yet matters; routes needing only some checks lead to flags or copies. CoR makes **each check a small object** linked into a line, so **the line is configuration, not logic**: a new rule = one class + one wiring line; reordering touches only wiring; each route can have its own chain.

How it works

A **Handler** type has `handle(request)` and a **next** reference; each **concrete handler** checks, then **stops** or **passes on**; a **terminal handler** does the real work; the chain is **wired once** at start-up (`auth.setNext(rateLimit).setNext(validate).setNext(createOrder)`) and the client calls only the **first** handler.

Structure (UML)

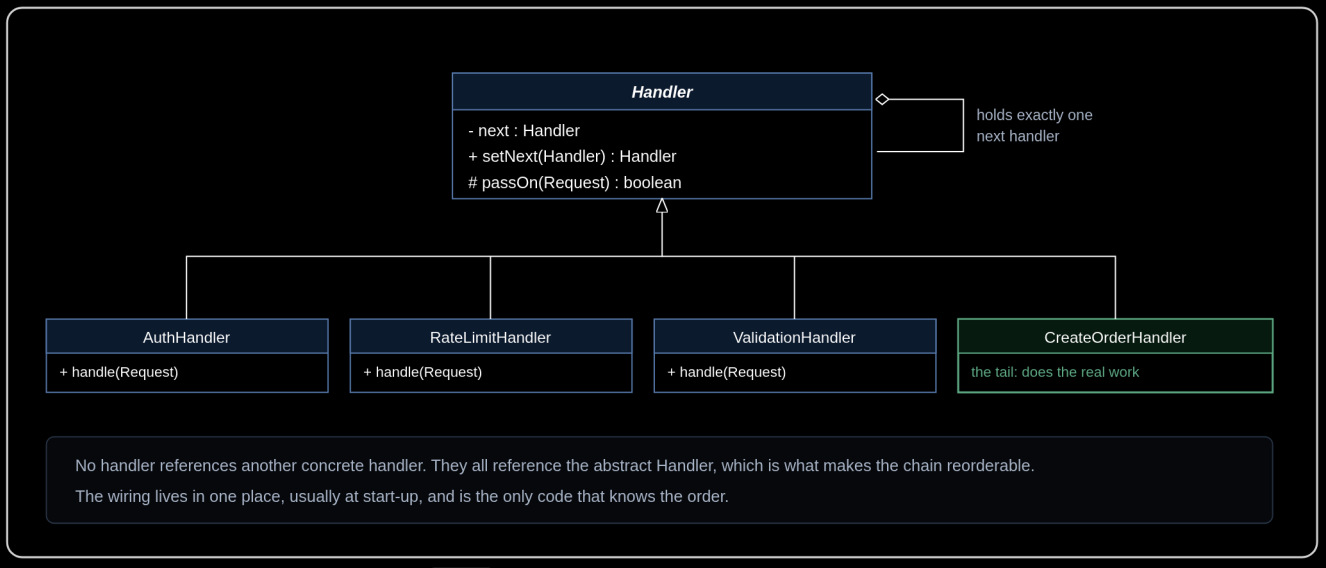
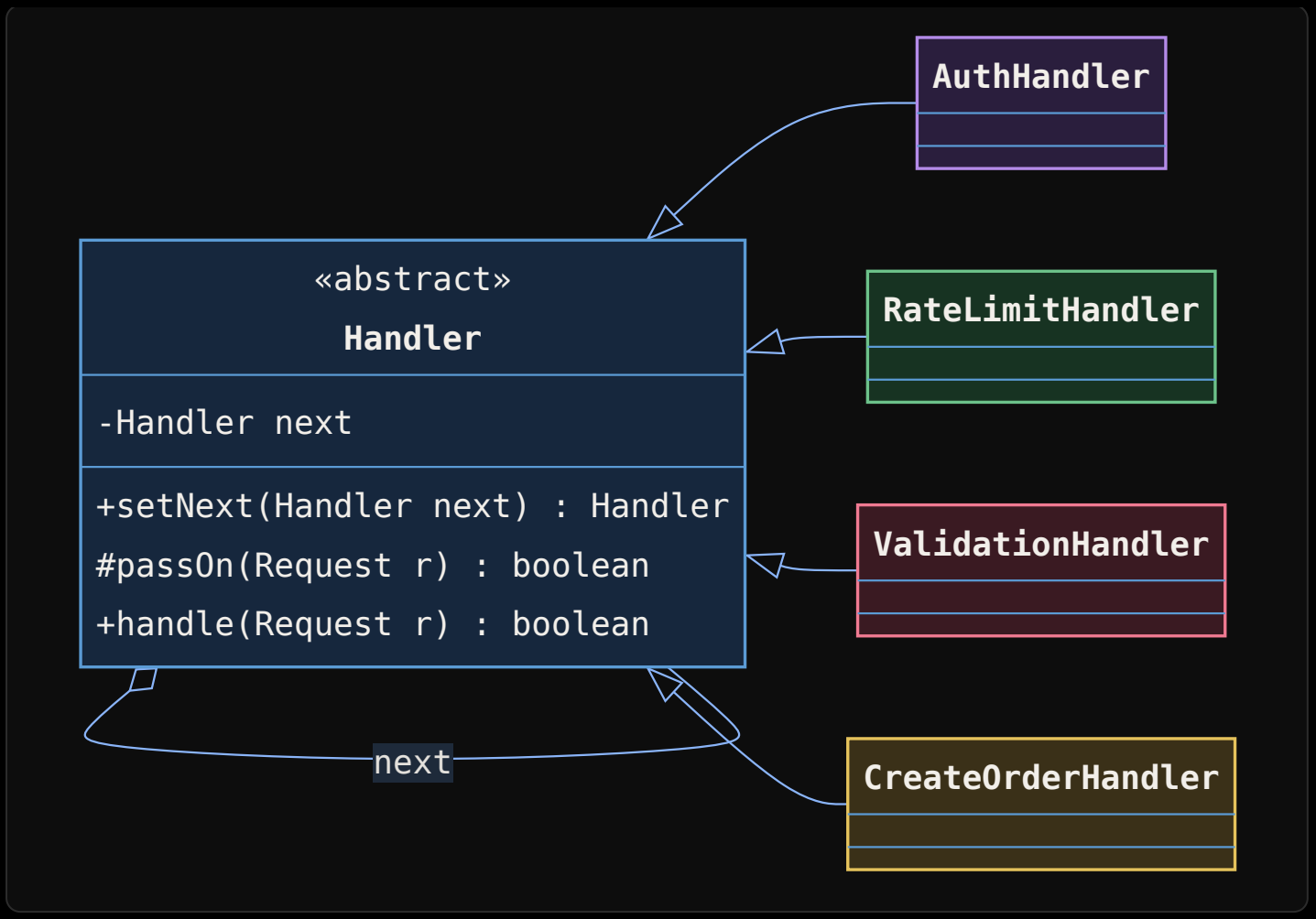


Figure — Abstract Handler holds `next`; concrete handlers extend it; CreateOrderHandler is the tail (course diagram)

> **Our code vs the course.** The course's `passOn` returns `true` when there is no next handler; ours **throws**, because silent "success" when the chain runs out is exactly the invisible failure this pattern is prone to.



Java example

`lld.ch10.chain` shows four flavours: (1) **classic linked handlers** with explicit outcomes (`false` = rejected, `passOn` = continue; `passOn` throws if there's no terminal handler); (2) an **approval chain** (`Approver`, `BigDecimal` limits) where the first capable approver wins and the chain is just data; (3) **servlet-style filters** where `next` is an **argument** (`chain.doFilter(...)`), so a filter can act before *and after* the rest, or stop by not calling it — a `FilterChain` holds a position, so one per request; (4) **logging levels**, where every handler acts if severe enough and always passes on — a pipeline-like variant.

```java
// File: code/lld/ch10/chain/Handler.java
public abstract class Handler {
    private Handler next;

    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    /** Forwards the request. Falling off the end means the chain has no terminal handler: a wiring bug. */
    protected boolean passOn(Request request) {
        if (next == null) {
            throw new IllegalStateException(
                getClass().getSimpleName() + " tried to pass on, but the chain has no terminal handler");
        }
        return next.handle(request);
    }

    public abstract boolean handle(Request request);
}

// File: code/lld/ch10/chain/AuthHandler.java
final class AuthHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.token() == null || r.token().isBlank()) {
            System.out.println("  [auth] rejected: missing token");
            return false;                       // explicit outcome: the chain stops here
        }
        System.out.println("  [auth] ok for " + r.user());
        return passOn(r);                       // explicit outcome: continue
    }
}

// File: code/lld/ch10/chain/Main.java (wiring)
        // Order is a design decision: auth BEFORE rate limiting so the limiter can key on the user,
        // and so anonymous traffic cannot burn a real user's quota.
        Handler pipeline = new AuthHandler();
        pipeline.setNext(new RateLimitHandler(2))
                .setNext(new ValidationHandler())
                .setNext(new CreateOrderHandler());
        // ...
```

In the output, a tokenless request stops at auth (later stages never run); an expense above every limit gets an explicit `REJECTED`; a `DEBUG` log reaches only the console while an `ERROR` reaches all three handlers.

Full runnable code: code/lld/ch10/chain/ (see the Full Edition).

Trade-offs

Pros	Cons
New stage = one class + one wiring line (Open/Closed).	A request's route is spread over the wiring and several files.
Each stage testable against a stub successor; per-route chains.	Nothing verifies the chain is complete or correctly ordered.
Sender doesn't know who handles.	A handler that neither handles nor forwards makes the request vanish.

Order is a design decision no tool verifies: auth first lets the limiter count per *user*; rate-limit first keys on IP but shields expensive auth from floods. **Comment the reason beside the wiring.**

When not to use it

- Every request always goes through every stage in fixed order — four calls in one method are clearer.
- Handlers need each other's results — that's a *pipeline*; pass a context object step by step.
- No terminal handler exists (add one first).
- Long chain on a very hot path.

Common mistakes

1. **The swallowing bug:** a handler returns without `passOn` — the request silently vanishes. Give every path an explicit outcome; test with a stub successor.
2. **No terminal handler**; 3. **wrong, undocumented order**; 4. **handlers that construct each other**; 5. **sharing a stateful** `FilterChain` across requests.

Real-world examples

Servlet filters and Spring Security's `SecurityFilterChain`; Spring `HandlerInterceptor.preHandle` (return `false` to stop); HTTP middleware (Express `next()`, Django, ASP.NET Core, Go handler wrappers), OkHttp interceptors, Netty `ChannelPipeline`; `java.util.logging` parent handlers; exception propagation; DOM event bubbling (`stopPropagation()`); approval and support-escalation workflows.

In the interview

> **30-second answer:** "Chain of Responsibility splits a series of checks into separate handler objects, each linked only to its successor. The request moves down the line until some handler completes or refuses it. Only the wiring knows the order, so adding or reordering rules is cheap. Servlet filters and HTTP middleware are the everyday examples. The classic bug is a handler that forgets to pass on, so I give every path an explicit outcome and put a terminal handler at the end."

- *vs Decorator?* Decorator wrappers all run and always delegate inward to *add* behaviour; a CoR handler may **end** the request.
- *vs pipeline?* A pipeline runs **all** steps; if no stage can stop the flow, it's a pipeline, not CoR.

Self-check

1. **Q:** What does each handler reference? **A:** Only the next handler, via the abstract type.
2. **Q:** Most common CoR bug? **A:** A handler that neither handles nor passes on — the request disappears.
3. **Q:** Why do frameworks pass `next` as a parameter? **A:** A missing `next(...)` is visible in the handler body, and the handler can work before and after the rest.

TL;DR

A request walks a line of handlers; each handles, rejects, or passes on. The chain is data — but **order matters**, **always have a terminal handler**, and **never swallow a request**.

10.2 Command

What it is

Command packages a request ("add 2 widgets to the cart") as an **object** holding everything needed to perform it later. Then you can **store it, queue it, log it, retry it, run it on a worker thread** — and, if it knows its inverse, **undo** it.

GoF intent: *encapsulate a request as an object, so clients can be parameterised with requests, requests can be queued or logged, and operations can be undone.* Analogy: a waiter (invoker) writes an order slip (command) for the cook (receiver) to act on later.

The problem it solves

`cart.setDiscount(10)` runs and is forgotten. That fails once you need **undo**, **a queue**, **a log of intent**, **retry**, or **one action with many triggers** (menu, hot-key, script). Command gives the request a body that outlives the call.

How it works

Name the **three roles** — invoker, command, receiver — plus the client. GoF participants: Command, ConcreteCommand, Client, Invoker, Receiver.

Role	Job	In our code
Command	Interface with `execute()` (optionally `undo()`).	`Command`, `UndoableCommand`
Concrete command	Stores receiver + arguments; performs (and reverses) one operation.	`AddItemCommand`, `ApplyDiscountCommand`, `MacroCommand`
Receiver	Object whose state really changes; unaware of commands.	`Cart`
Invoker	Triggers commands and records them; unaware of carts.	`CommandHistory`, `ExecutorService`
Client	Builds a command and gives it its receiver.	`Main`

Invoker: `execute()` then push on an **undo stack**; **undo** pops, calls `undo()`, pushes on a **redo stack**; **redo** reverses that; **a fresh command clears redo**.

Structure (UML)

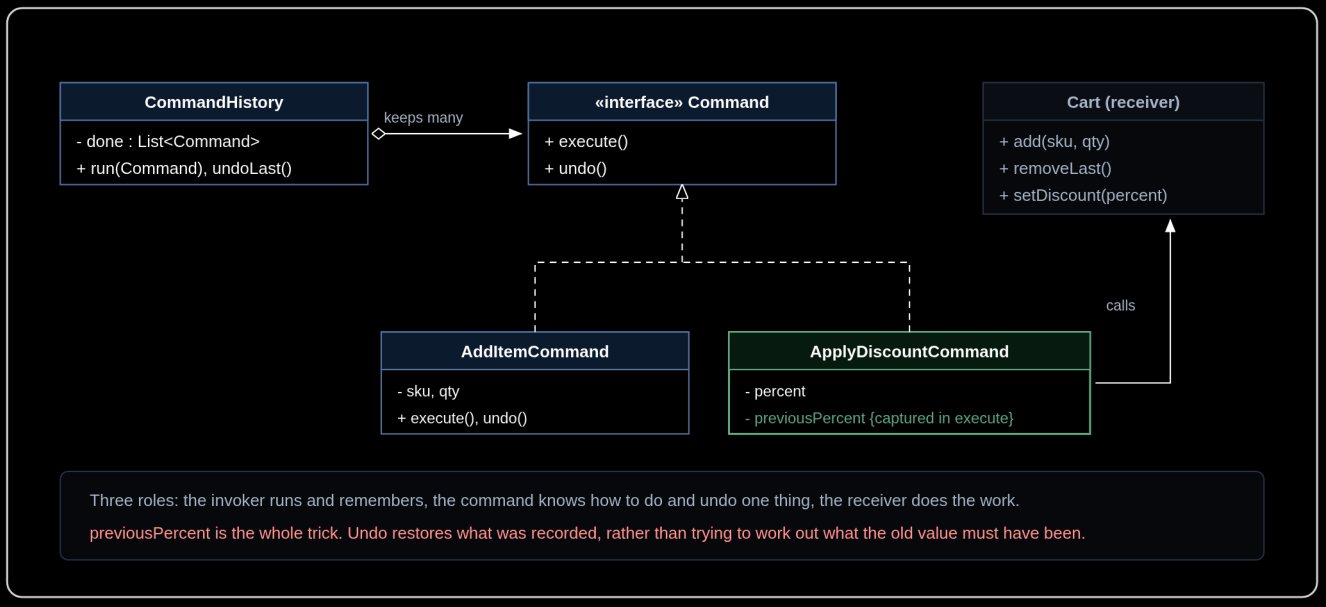
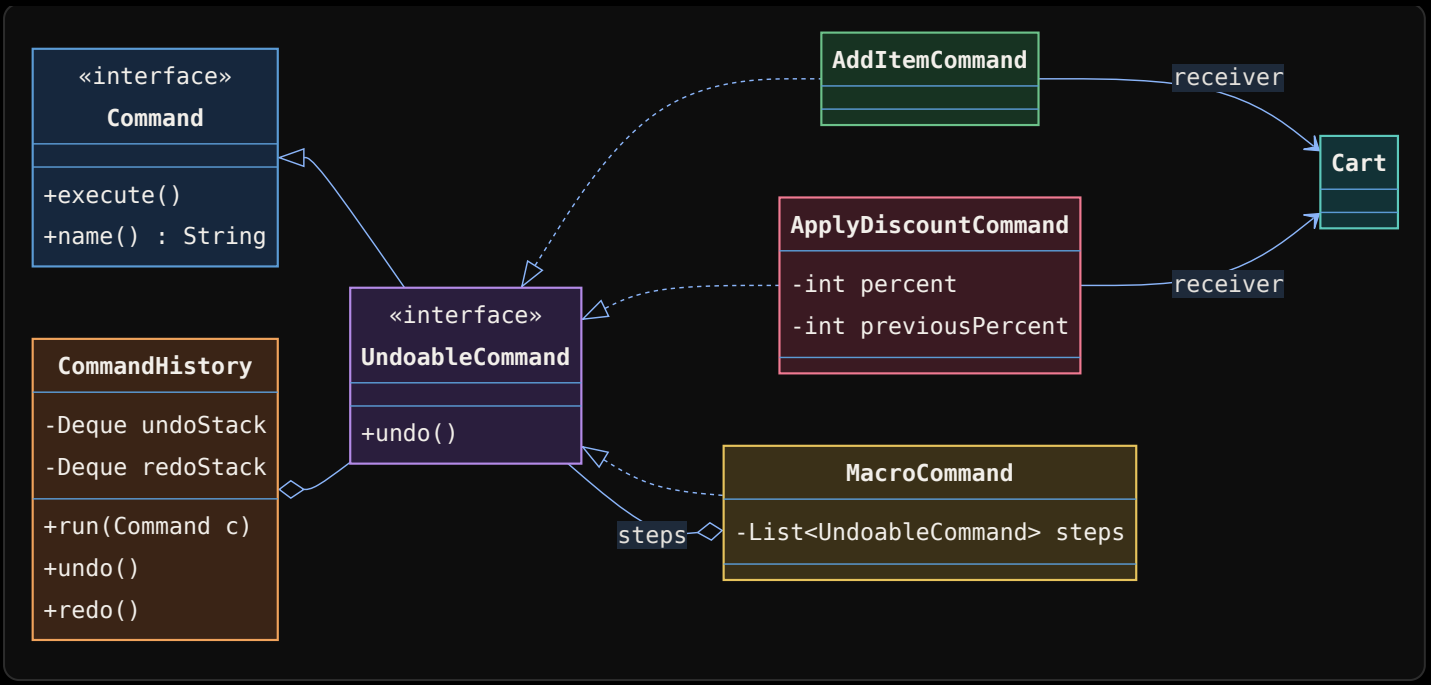


Figure — Command interface with execute/undo, two concrete commands, Cart as receiver, CommandHistory as invoker (course diagram; its `undoLast` / `removeLast` are `undo` / `remove` in our code)

`previousPercent` is the one field that makes undo correct; `CommandHistory` never mentions `Cart`, so one history serves the whole app.



Java example

- `Command` has **one abstract method**, so a lambda is a command; only honestly reversible commands implement `UndoableCommand`, and `CommandHistory` keeps irreversible ones out of the undo stack.
- `ApplyDiscountCommand` **captures the old value in** `execute()`; the cart starts at 5% to show "undo = set to 0" would be wrong.
- `MacroCommand` is a **Composite** of commands: executes in order, undoes in **reverse**.
- `Runnable` + `ExecutorService` is the JDK's version (command + invoker on a worker thread).

```java
// File: code/lld/ch10/command/Command.java
@FunctionalInterface
public interface Command {
    void execute();

    default String name() { return getClass().getSimpleName(); }
}

// File: code/lld/ch10/command/UndoableCommand.java
public interface UndoableCommand extends Command {
    void undo();
}

// File: code/lld/ch10/command/ApplyDiscountCommand.java
final class ApplyDiscountCommand implements UndoableCommand {
    private final Cart cart;
    private final int percent;
    private int previousPercent;          // filled in by execute()
    // ... constructor

    @Override
    public void execute() {
        previousPercent = cart.discountPercent();   // capture BEFORE changing
        cart.setDiscount(percent);
    }

    @Override public void undo()   { cart.setDiscount(previousPercent); }
    @Override public String name() { return "ApplyDiscount(" + percent + "%)"; }
}

// File: code/lld/ch10/command/CommandHistory.java
public final class CommandHistory {
    private final Deque<UndoableCommand> undoStack = new ArrayDeque<>();
    private final Deque<UndoableCommand> redoStack = new ArrayDeque<>();

    public void run(Command command) {
        command.execute();
        if (command instanceof UndoableCommand u) {
            undoStack.push(u);
            redoStack.clear();     // a new action destroys the "future" you could have redone
            System.out.println("  ran  " + command.name());
        } else {
            System.out.println("  ran  " + command.name() + " (irreversible: kept out of history)");
        }
    }

    public void undo() {
        if (undoStack.isEmpty()) { System.out.println("  nothing to undo"); return; }
        UndoableCommand c = undoStack.pop();
        c.undo();
        redoStack.push(c);
        System.out.println("  undo " + c.name());
    }
    // ... redo() mirrors undo(): pop from redo, execute(), push on undo
}
```

Output (excerpt): after undoing the discount the cart shows `discount=5% <- discount restored to the recorded 5%, not 0%`; running a new command then makes `redo` print `nothing to redo`; `SendReceipt` runs but is `kept out of history`.

Full runnable code: code/lld/ch10/command/ (see the Full Edition).

Undo must restore, not recompute

"Reset to 0" is a **guess**. Rule: `execute` **records what** `undo` **needs** (the fields it overwrites). History is **LIFO** — a stack. **When undo is a lie** (email sent, card charged): keep it out of history, or use **compensation** (a **refund** — both events stay on record).

Trade-offs

Pros	Cons
Request outlives the call: undo/redo, queues, audit, retry.	One class per action; an extra hop when reading code.
Generic invoker — one history for all commands; macros compose.	Each command must capture enough to run later and reverse.
One action backs menu, hot-key, script and API.	One faulty `undo` corrupts the whole history.

When not to use it

Simple immediate calls nobody undoes, queues or logs; undo that isn't really possible (use compensation); plain deferral (a lambda/`Runnable` already is a command); commands that must outlive the process (use a durable queue or event log).

Common mistakes

1. Recomputing in `undo` instead of restoring. 2. Not clearing redo on a new command. 3. Business logic in the invoker. 4. Capturing a mutable object instead of values. 5. Unbounded history.

Real-world examples

Editor undo and `javax.swing.undo.UndoManager`; `Runnable` / `Callable` with `ExecutorService` / `Thread` as invokers; Swing `Action`, .NET `ICommand`; Redux actions; job queues (Celery, Sidekiq, Kafka messages), outbox tables; DB migrations with `up` / `down`; event sourcing (Ch 11).

In the interview

> **30-second answer:** "Command wraps a request, with its receiver and arguments, in an object — so it can be kept, queued, logged, retried or reversed. The invoker triggers and records commands without knowing what they touch; each command performs and reverses one operation; the receiver holds the state. For undo, `execute` captures the old value and `undo` restores it; history is a stack and a new command clears redo. Irreversible actions get compensation. In Java, `Runnable` plus an `ExecutorService` is the everyday form."

- *vs Strategy?* Strategy = *how* a job is done, swapped by the caller. Command = *which* job, with arguments, kept to run later or reverse.
- *vs Memento for undo?* Command stores the **action** + a little state (cheap); Memento stores the **whole state** (always correct). Real systems combine them.

Self-check

1. **Q:** What must a command carry? **A:** Its receiver, all arguments, and any state needed for undo.
2. **Q:** Why store `previousPercent`? **A:** So undo restores the real previous value instead of guessing.
3. **Q:** What happens to redo when a new command runs? **A:** It is cleared.

TL;DR

Request as object: **invoker** runs it, **command** knows do/undo, **receiver** changes. Undo **restores captured state**, history is a **stack**, irreversible actions get **compensation**. A lambda is a command when you only need deferral.

10.3 Iterator

What it is

Iterator lets you visit a collection's elements **one at a time without knowing how they are stored** — array, list, tree, or a paginated remote API.

GoF intent: *provide sequential access to an aggregate's elements without exposing its representation.* Participants: **Iterator**, **ConcreteIterator** (tracks position), **Aggregate** (creates iterators), **ConcreteAggregate**. Analogy: a TV remote's "next channel".

The problem it solves

- `get(i)` loops assume an array; move the data into a tree or remote service and every loop breaks.
- Many sources **can't be indexed**: trees (several visit orders), graphs (must track visited), paged APIs. Each caller re-implementing traversal re-implements paging, some wrongly.

Iterator moves walking into a separate object answering `hasNext` and `next`. **The position lives in the iterator, never in the collection** — so two traversals can run at once, and `iterator()` must return a **new** iterator each time.

How it works

The **aggregate** implements `Iterable<T>` (`iterator()`); the **iterator** implements `Iterator<T>` (`hasNext()`, `next()`, optional `remove()`) and keeps its own **cursor**; the client loops `while (it.hasNext()) use(it.next());` — which is what `for (T x : collection)` compiles to.

Structure (UML)

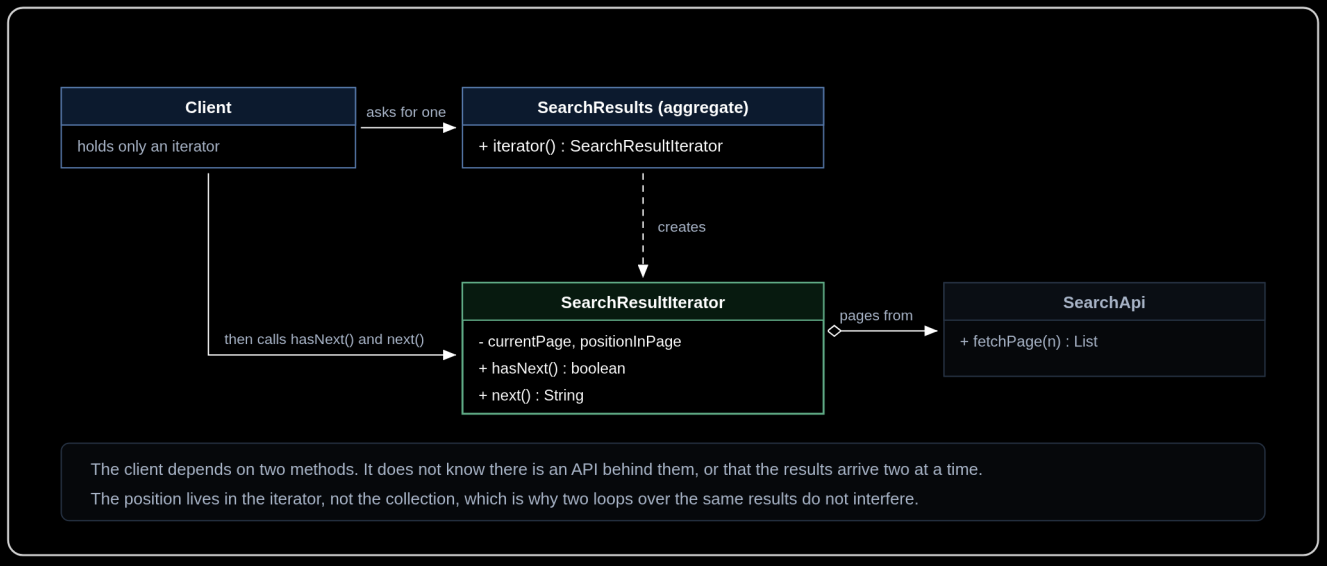
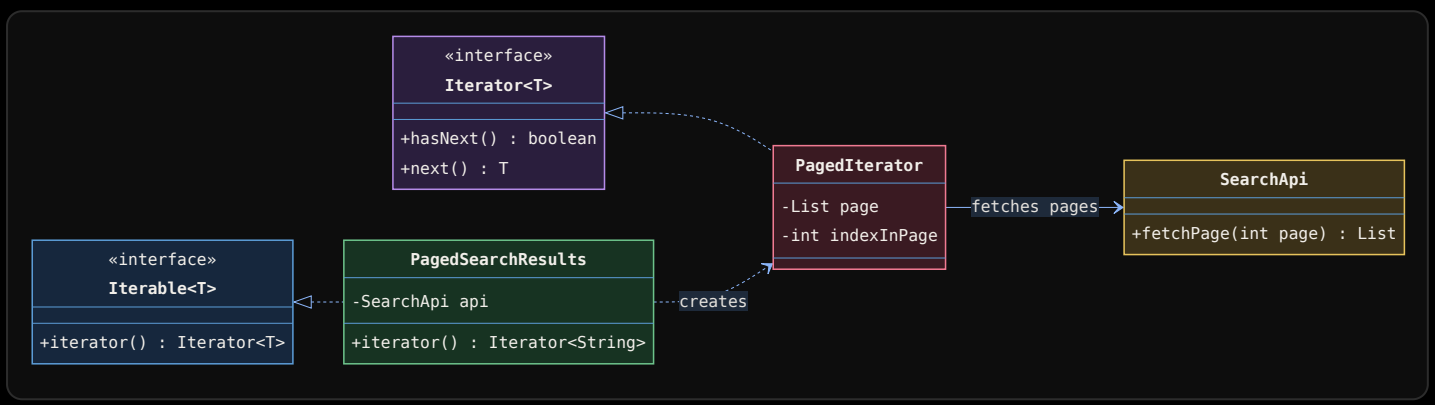


Figure — SearchResults hands out a SearchResultIterator, which calls SearchApi page by page (course diagram)

Here `hasNext` may fetch a page — never assume it is cheap or side-effect-free.



Java example

`lld.ch10.iterator` shows: (1) a **paged remote source behind a for-each**; (2) a **tree iterator** (explicit stack; two iterators independent); (3) **external** (client drives, can `break`) vs **internal** iteration (`forEach`, streams); (4) **fail-fast vs fail-safe** — removing from an `ArrayList` mid-loop *usually* throws `ConcurrentModificationException`, but removing the second-to-last element silently skips the last; use `Iterator.remove()` / `removeIf`; (5) **`Spliterator`** with `trySplit` for parallel streams.

```java
// File: code/lld/ch10/iterator/PagedSearchResults.java
final class PagedSearchResults implements Iterable<String> {
    private final SearchApi api;

    PagedSearchResults(SearchApi api) { this.api = api; }

    @Override
    public Iterator<String> iterator() { return new PagedIterator(); }

    private final class PagedIterator implements Iterator<String> {
        private List<String> page = List.of();
        private int indexInPage = 0;
        private int pageNumber = 0;
        private boolean exhausted = false;

        @Override
        public boolean hasNext() {                // NOT a passive question: may hit the network
            if (indexInPage < page.size()) return true;
            if (exhausted) return false;
            page = api.fetchPage(++pageNumber);
            indexInPage = 0;
            if (page.isEmpty()) {
                exhausted = true;
                return false;
            }
            return true;
        }

        @Override
        public String next() {
            if (!hasNext()) throw new NoSuchElementException();   // the Iterator contract
            return page.get(indexInPage++);
        }
    }
}
```

The tree iterator (`BinarySearchTree`) keeps a `Deque` of nodes still to visit — that stack *is* its position.

Output (excerpt):

```
5 items, 4 network calls (the last one only discovers the end)
b: 20   <- b starts from the beginning: position lives in the iterator
best-effort only: no exception, visited [java, tmp], 'lld' silently skipped
```

Full runnable code: code/lld/ch10/iterator/ (see the Full Edition).

Hide the shape, not the cost

Five results took **four** calls (no total or next-page token, so the last call only discovers the end). **A tidy loop conceals per-step cost** — the shape of the ORM **N+1 problem**. Name the type honestly, offer a batch API, return a total/cursor, count fetches.

Java's iterator family (quick reference)

Type	What it adds
`Iterable<T>`	`iterator()`; enables for-each; also `forEach`, `spliterator()`.
`Iterator<T>`	`hasNext()`, `next()`, optional `remove()`, `forEachRemaining()`.
`ListIterator<T>`	Bidirectional: `hasPrevious()`, `previous()`, indexes, `set()`, `add()`.
`Spliterator<T>`	`tryAdvance`, `trySplit`, `characteristics()` (`ORDERED`, `SIZED`, `DISTINCT`, `IMMUTABLE`).
`Enumeration<E>`	Java 1.0 version: `hasMoreElements()`, `nextElement()`. Prefer `Iterator`.

Fail-fast (`ArrayList`, `HashMap`): throw `ConcurrentModificationException` — best-effort, usually caused by **one thread** modifying its own loop. **Weakly consistent** (`ConcurrentHashMap`): never throw. **Snapshot** (`CopyOnWriteArrayList`): never throw; `remove()` unsupported.

Trade-offs

Pros	Cons
Client doesn't care if data is an array, tree or API; several visit orders.	Mid-loop modification is a bug unless via `Iterator.remove`.
Lazy: huge or infinite sequences work.	A lazy iterator holding a file/DB cursor leaks on early `break`.
Paging hidden behind a plain loop.	Per-step cost invisible → N+1 problems.

When not to use it

Data already an in-memory list (hand it out); callers need random access; a step has side effects the caller must decide on (loading a page is fine, charging a customer is not).

Common mistakes

1. Modifying the collection mid-loop (use `Iterator.remove`/`removeIf`; Python may silently skip elements). 2. Leaking resources on early `break` (try-with-resources). 3. Reusing a one-shot iterator/stream. 4. `next()` returning `null` instead of throwing `NoSuchElementException`. 5. Storing the cursor in the collection.

Real-world examples

Every for-each loop; JDBC `ResultSet`, `Scanner`; AWS SDK v2 paginators and Stripe auto-pagination; `Stream`s on `Spliterator`s; **generators** (Python `yield`, JS `function*`) — the pattern as a function; infinite-scroll feeds.

In the interview

> **30-second answer:** "Iterator pulls traversal into a separate object with `hasNext` and `next`, so callers walk data without caring if it's an array, tree or paginated API. Each iterator keeps its own position, so two loops are independent. I'd implement `Iterable<T>` so for-each works. My example would be the lazy paged case, and I'd flag the risk: when every step can hit the network, the loop hides its cost — that's how N+1 problems start."

- *Internal vs external?* External: client steps and can stop or interleave. Internal: collection loops and calls your function — simpler, parallelisable, harder to stop early.
- *vs Visitor / generator / stream?* Iterator = **how you move**; Visitor = **what you do** at each element; a generator is Iterator as a function; streams add `map`/`filter` on top.

Self-check

1. **Q:** Where does the cursor live? **A:** In the iterator object.

2. **Q:** What does for-each need from a class? **A:** It implements `Iterable<T>`.
3. **Q:** Why 4 calls for 5 results? **A:** With no total or cursor, the end is found only by requesting one more page.

TL;DR

A separate object walks the collection with `hasNext/next`; **position lives in the iterator**. Implement `Iterable`. Know **fail-fast vs snapshot**, **internal vs external**, and don't let a lazy loop **hide its cost**.

10.4 Mediator

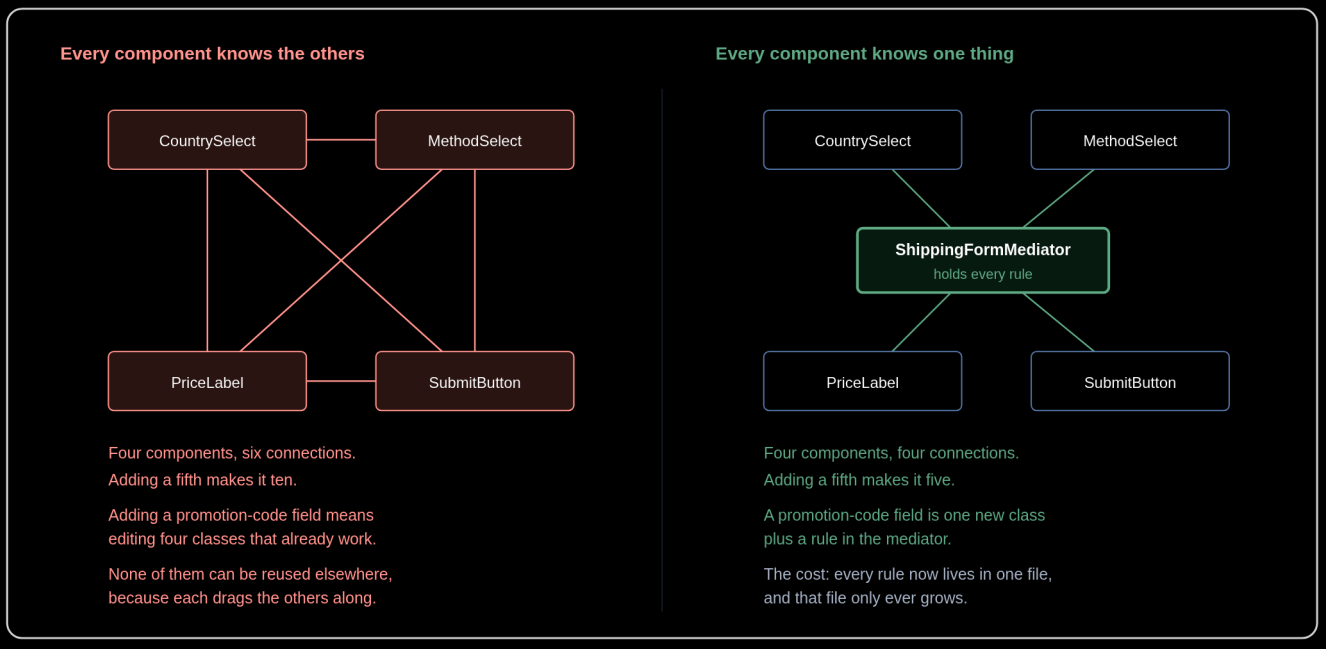


Figure — Left: four components all know each other (six links). Right: each knows only the mediator (four links) (course diagram)

What it is

Mediator makes a group of objects talk through **one coordinator** instead of referencing each other. Each object reports "this happened"; the mediator decides how others respond.

GoF intent: *define an object that encapsulates how a set of objects interact, so they don't refer to each other explicitly and their interaction can vary independently.* Participants: **Mediator**, **ConcreteMediator** (knows colleagues, implements coordination), **Colleague** (knows only its mediator). Sometimes called a *controller*. Analogy: **air traffic control** — planes talk only to the tower.

The problem it solves

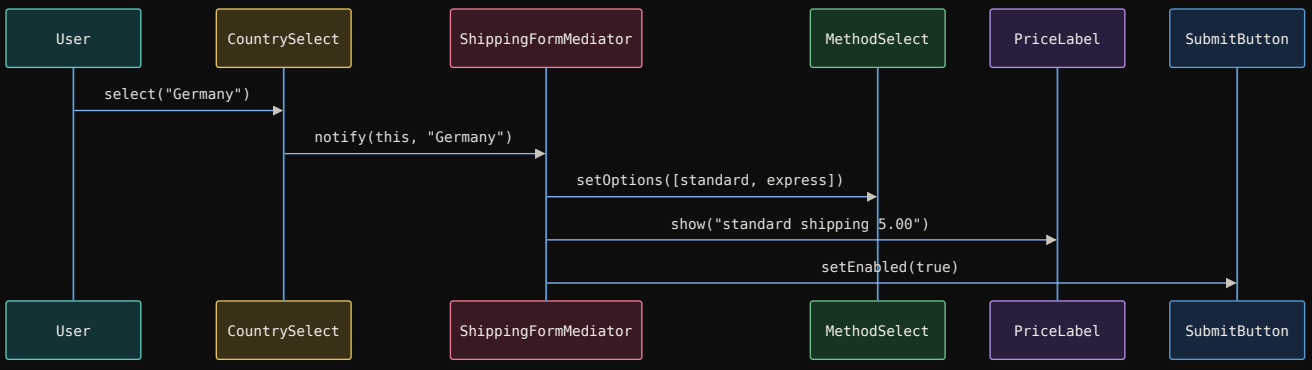
A checkout form: **country**, **delivery method**, **price** label, **submit** button — country refreshes methods, method changes price, price decides submit. Wired directly, four widgets need **six** links, five need **ten** (n(n−1)/2); a new field edits four tested classes; no widget is reusable alone. With a mediator: **one link per widget**, and widgets are reusable.

The honest trade: dependencies aren't removed — **they are relocated** into the mediator, which tends to grow.

How it works

Each **component** holds only the **Mediator** and calls `mediator.notify(this, event)`; the **concrete mediator** holds all components and the rules ("country changed → reload methods, then price, then submit") and calls **down** into them — traffic is **two-way** through one hub.

Structure (UML)



Java example

`lld.ch10.mediator` has three mediators: the **shipping form** (senders identified by reference, rates in `BigDecimal`); a **chat room** (`ChatRoom` routes broadcasts and private messages and applies moderation); **air traffic control** (`ControlTower` owns the runway and holding queue).

```
// File: code/lld/ch10/mediator/Mediator.java
public interface Mediator {
    void notify(Component sender, String event);
}

// File: code/lld/ch10/mediator/CountrySelect.java
    void select(String country) {
        selected = country;
        System.out.println("  [country] user picked " + country);
        mediator.notify(this, country);          // report up; never call a sibling
    }

// File: code/lld/ch10/mediator/ShippingFormMediator.java
    @Override
    public void notify(Component sender, String event) {
        if (sender == country) {
            Map<String, BigDecimal> available = RATES.getOrDefault(event, Map.of());
            methods.setOptions(new ArrayList<>(available.keySet()));
            if (available.isEmpty()) {
                price.show("shipping not available");
                submit.setEnabled(false);
            } else {
                showPrice();                          // ORDER matters: reload methods first, then price
                submit.setEnabled(true);
            }
        } else if (sender == methods) {
            showPrice();
        } else if (sender == submit) {
            System.out.println("  [mediator] order placed: " + country.selected() + " / " + methods.selected());
        }
    }
```

In the output, picking "Germany" reloads methods, shows `standard shipping 5.00` and enables submit — three components updated and **no component called another**.

Full runnable code: code/lld/ch10/mediator/ (see the Full Edition).

Mediator or Observer?

	Mediator	Observer
What travels	An event **and** the decision on what follows	Only the event
Reaction logic	In one place: the mediator	Spread across observers
Parties know each other?	No	No; subject knows only a subscriber list
Flow	Two-way	One-way
Order of reactions	Explicit in the mediator	Not part of the contract

Test: does something need **coordinating in sequence** ("refresh methods, *then* price, *then* submit")? Then Mediator; otherwise the lighter Observer. **With Observer the listeners decide; with Mediator the hub decides.**

Trade-offs

Pros	Cons
Links grow linearly, not quadratically.	Every new rule (and merge conflict) lands in the mediator.
All interaction rules in one testable class.	To learn why a widget changed, read the mediator.
Widgets reusable; new widget = one class + one rule.	Can become a **god object**.

Keep it small: one mediator per screen area/workflow; coordination only (pricing in its own classes); no widget-to-widget shortcuts; dispatch via an event→handler map.

When not to use it

Only two objects interact; interactions are few and stable; nothing needs coordinating (use Observer); the mediator is already a god object (split it first).

Common mistakes

1. Components calling each other "just this once".
2. Computation inside the mediator.
3. Confusing with **Facade**: Facade sits *in front of* a subsystem that doesn't know it (one-way); Mediator sits *among* peers that know it (two-way).
4. Stringly-typed events — use sender reference, an enum or typed events.

Real-world examples

UI controllers/view-models (GoF's dialog box); chat servers; **saga orchestrators** (Temporal, AWS Step Functions), ride-hailing dispatch; Redux store, .NET MediatR; message brokers. `java.util.Timer` / `Executor` are often cited as JDK mediators — a *loose* fit.

In the interview

> **30-second answer:** "Mediator swaps peer-to-peer references for a single hub. Widgets report events; the hub owns the rules and instructs particular widgets. Four widgets go from six links to four; a fifth gives five instead of ten. The dependencies aren't gone — they're concentrated in one class I can read and test. The risk is a god object, so I split mediators per screen area or workflow and push computation into its own classes."

- *vs Facade?* Facade: one-way, subsystem unaware. Mediator: two-way among peers.
- *Microservices?* Orchestration is mediator-style; choreography is observer-style.

Self-check

1. **Q:** Links for 5 fully connected components vs with a mediator? **A:** 10 vs 5.
2. **Q:** Does a mediator remove coupling? **A:** No, it moves it into one class.
3. **Q:** Main failure mode? **A:** God-object mediator.

TL;DR

Peers talk through **one coordinator** holding the interaction rules. Links become linear and components reusable, but coupling **moves** into the mediator — keep it small and split it early.

10.5 Memento

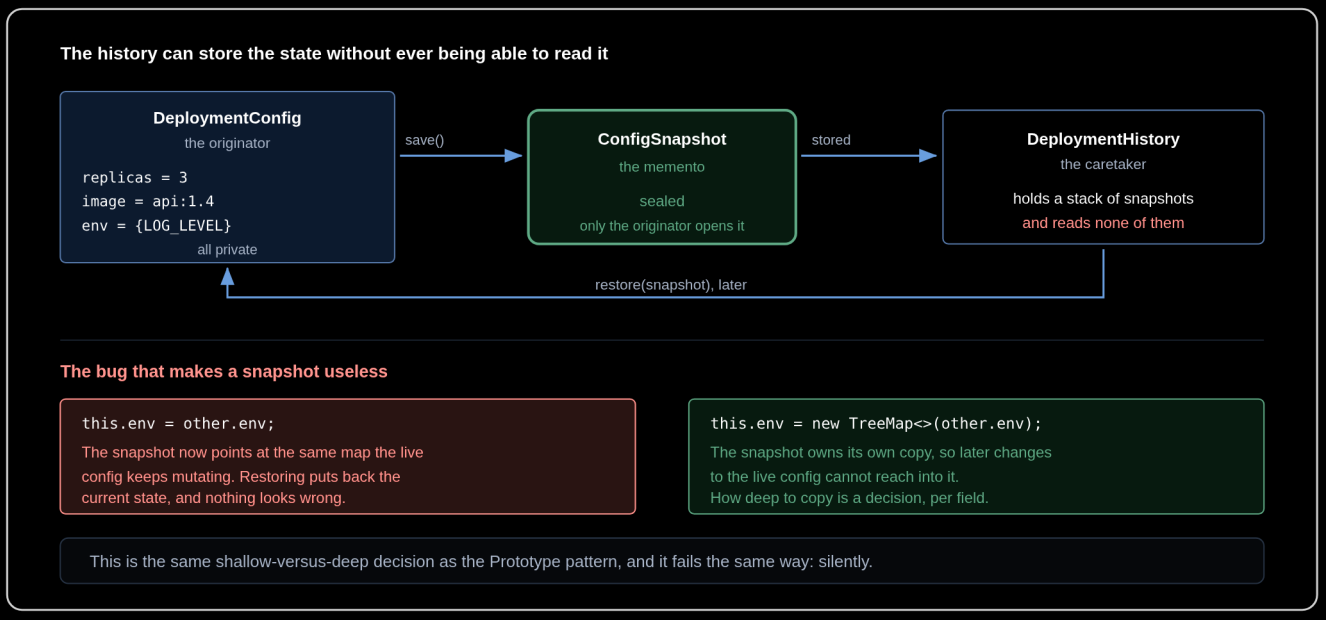


Figure — The originator (DeploymentConfig) saves a sealed snapshot; the caretaker (DeploymentHistory) stores it without reading it; below, the shared-map bug (course diagram)

What it is

Memento lets an object record its internal state in a **snapshot** and later **return** to it — without anyone else **seeing** inside.

GoF intent: *without violating encapsulation, capture and externalize an object's internal state so it can be restored later.* Participants: **Memento** (narrow interface to caretaker, wide to originator), **Originator** (creates and uses mementos), **Caretaker** (keeps mementos, never examines them). Analogy: a **save game** — the menu lists saves but never reads their contents.

The problem it solves

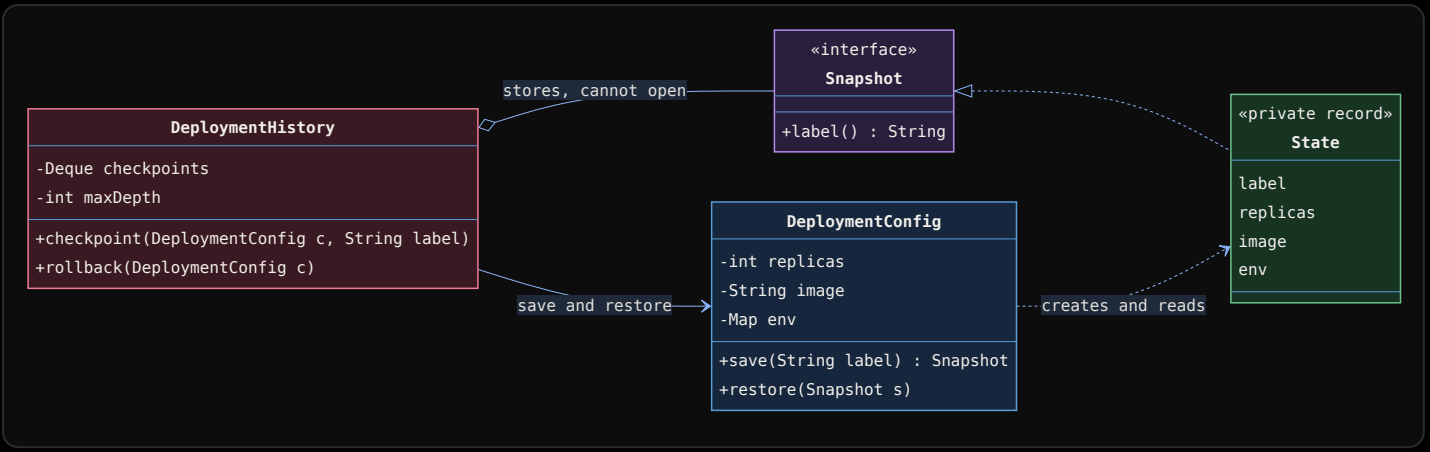
Rolling back a deployment config after a risky change. If an outside class copies every field, **encapsulation** breaks (it must know every field and change with each new one) and **atomicity** breaks (a field-by-field restore can stop half-way). With Memento the object **snapshots itself** into a sealed token **only it can open**, and restore is **one call** it controls.

How it works

Role	Job	In our code
Originator	Owns state; `save` and `restore`.	`DeploymentConfig`
Memento	Carries state; opaque to others.	`Snapshot` (narrow) / private `State` record (wide)
Caretaker	Decides **when** to save/restore; never looks **inside**.	`DeploymentHistory`

Enforcing opacity in Java (GoF narrow vs wide interface): public `Snapshot` exposes only `label()`; the data is a **private nested record** `State`; `restore` checks `instanceof State`, so a forged snapshot is refused. (Reflection can still peek.) C++ uses `friend`; Python/JS rely on convention.

Structure (UML)



Java example

`State`'s compact constructor does `env = Map.copyOf(env)`, so the snapshot **owns its copy** (changing the live map after saving doesn't affect it). `DeploymentHistory` is **bounded** (`maxDepth`) and drops the oldest.

```java
// File: code/lld/ch10/memento/DeploymentConfig.java
public final class DeploymentConfig {

    public interface Snapshot {
        String label();
    }

    private record State(String label, int replicas, String image, Map<String, String> env)
            implements Snapshot {
        State {
            env = Map.copyOf(env);   // DEEP enough: the snapshot owns its own copy, not the live map
        }
    }

    private int replicas;
    private String image;
    private final Map<String, String> env = new TreeMap<>();
    // ... constructor, apply(...), setEnv(...)

    public Snapshot save(String label) {
        return new State(label, replicas, image, env);
    }

    public void restore(Snapshot snapshot) {
        if (!(snapshot instanceof State s)) {
            throw new IllegalArgumentException("not a snapshot produced by DeploymentConfig");
        }
        apply(s.replicas(), s.image(), s.env());
    }
}
```

Output (excerpt):

```
live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}   <- snapshot was a real copy
bad snapshot  = {LOG_LEVEL=debug}  <- changed with the live object
```

Full runnable code: code/lld/ch10/memento/ (see the Full Edition).

The bug that makes a snapshot useless

If the memento keeps a **reference to the live mutable map**, the snapshot drifts with the object and restore silently does nothing. Copy mutable parts (`Map.copyOf`, deep copies — the Prototype decision, Ch 8). Also **bound history depth** and don't snapshot **uncapturable state** (sockets, files, other systems).

Memento or Command undo?

	Memento	Command undo
Saved per step	Full copy of state	The operation + minimal state for reversal
Undo by	Re-installing a copy	Running the inverse
Memory per step	∝ state size	∝ one operation
Correctness risk	Low	Each inverse must be right
Prefer when	Reversal is tricky or state small	Reversal simple or state big

Drawing app (huge canvas) → Command; wizard step changing six interacting fields → Memento. **Real systems combine them:** occasional snapshots + a command log; restore nearest snapshot, replay forward (databases, event sourcing).

Trade-offs

Pros	Cons
Rollback with internals private; new fields need no caretaker change.	Memory grows with each snapshot.
Originator controls restore: all-or-nothing, to a real state.	Mutable parts must be deep-copied.
One mechanism for checkpoints, wizards, saves.	Some state can't be captured; some languages hide only by convention.

When not to use it

Large, frequently changing state (use Command or snapshots + replay); uncapturable state; **immutable objects** (an old instance already is a snapshot); operations with easy exact inverses.

Common mistakes

1. Referencing mutable internals (shallow copy). 2. Caretaker reading/editing fields. 3. Restoring field by field from outside. 4. Unbounded history. 5. Snapshotting connections or locks.

Real-world examples

DB `ROLLBACK` (undo logs); Git commits + `checkout`; **Kubernetes** keeps old ReplicaSets (`revisionHistoryLimit`, default 10) for `kubectl rollout undo`; VM snapshots; game saves, wizard "back". Python `copy.deepcopy`, JS `structuredClone` make quick mementos.

In the interview

> **30-second answer:** "With Memento an object packs its state into a token only it can unpack. The originator makes and consumes snapshots, the memento holds the data, and the caretaker chooses when to save or roll back without inspecting contents. Restore is one call the originator controls, so it's all-or-nothing, and new fields never touch the history code. I'd deep-copy mutable parts, bound the history, and compare with Command undo: Memento stores state, Command stores actions; real systems combine snapshots with a command log."

- *Undo for a large document?* Command per edit + periodic snapshots.

Self-check

1. **Q:** Three roles? **A:** Originator, memento, caretaker.
2. **Q:** Who may read a memento? **A:** Only the originator.
3. **Q:** Why copy `env`? **A:** Otherwise the snapshot shares and changes with the live map.

TL;DR

The object **snapshots itself**; the caretaker **stores but can't read**; restore is **one originator-controlled call**. **Deep-copy**, **bound** history, and pick Memento (state) vs Command (actions) by which is smaller.

10.6 Interpreter

(Covered in the course only by its one-line table entry; this section is from GoF and standard practice.)

What it is

Interpreter represents a small language's **grammar** as classes — **one class per rule** — and evaluates a sentence by building a **tree** of these objects and calling `interpret` on the root. Analogy: a calculator reading `2 + 3 * 4` as `2 + (3 * 4)`.

Key terms: **grammar** (rules of valid sentences); **DSL** (small domain-specific language); **AST** (abstract syntax tree); **terminal** (leaf: number, variable) vs **nonterminal** expression (built from others); **context** (e.g. variable values).

The problem it solves

Rules like "age > 18 AND country = IN" keep appearing; hard-coding each means a redeploy. A tiny interpreted language turns rules into **data** — Interpreter is the simplest OO way to evaluate a **small** grammar.

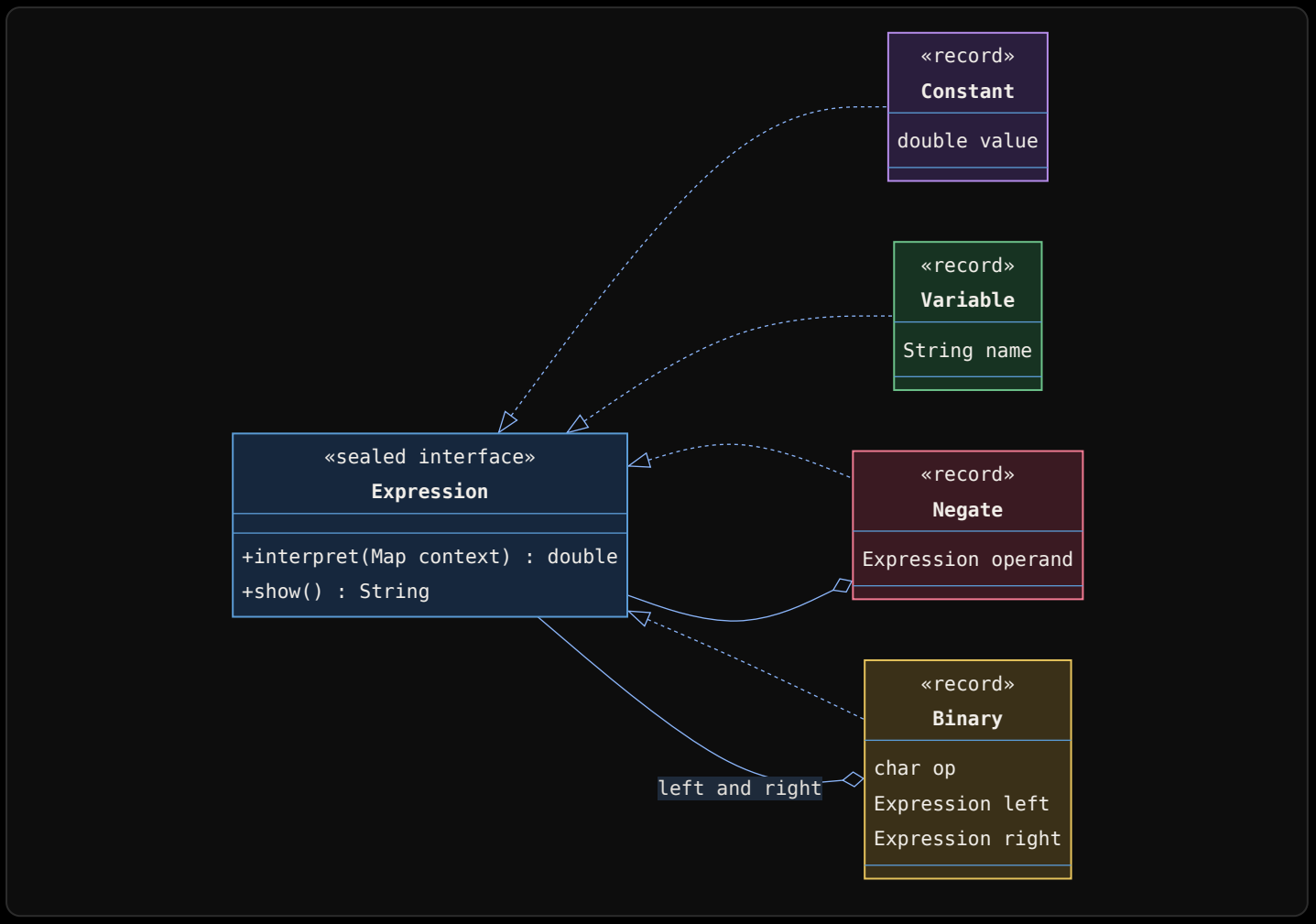
How it works

1. **Write the grammar** (EBNF):

```
expr   := term   (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := NUMBER | NAME | '(' expr ')' | '-' factor
```

2. **One class per rule**: `Constant`, `Variable` (terminals), `Negate`, `Binary` (nonterminals), all implementing `Expression.interpret(context)`.
3. **Build the AST** — GoF leaves **parsing** out; we use a small *recursive-descent* parser (one method per rule).
4. **Interpret** recursively from the root.
5. **Parse once, interpret many times** with different contexts.

Structure (UML)



The structure is a **Composite** (Ch 9): Interpreter ≈ "Composite + `interpret`". The AST for `price * qty - discount` is `Binary -` with children `Binary *` (price, qty) and `Variable discount`.

Java example

`Expression` is a **`sealed interface`** permitting four records (one per rule); `Parser`'s `expr`, `term`, `factor` mirror the grammar. Errors (unknown variable, division by zero, bad syntax) report clearly.

> **Why `double`?** This is a teaching calculator; a real pricing DSL must use `BigDecimal` with an explicit `RoundingMode`.

```java
// File: code/lld/ch10/interpreter/Expression.java
public sealed interface Expression
        permits Expression.Constant, Expression.Variable, Expression.Negate, Expression.Binary {

    double interpret(Map<String, Double> context);
    // ... show()

    record Binary(char op, Expression left, Expression right) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            double l = left.interpret(ctx), r = right.interpret(ctx);
            return switch (op) {
                case '+' -> l + r;
                case '-' -> l - r;
                case '*' -> l * r;
                case '/' -> {
                    if (r == 0) throw new ArithmeticException("division by zero in " + show());
                    yield l / r;
                }
                default -> throw new IllegalStateException("unknown operator " + op);
            };
        }
        // ... show()
    }
    // ... Constant, Variable (looks up the context; unknown name -> error), Negate
}

// File: code/lld/ch10/interpreter/Parser.java
    private Expression expr() {
        Expression left = term();
        while (peek('+') || peek('-')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, term());   // left-associative
        }
        return left;
    }
    // ... term() is the same loop over '*' '/' calling factor()
    // ... factor(): '-' factor | '(' expr ')' | NUMBER | NAME
```

Output (excerpt):

```
price * qty - discount   AST ((price * qty) - discount)       = 700.0
2 + 3 * 4                AST (2 + (3 * 4))                    = 14.0
(2 + 3) * 4              AST ((2 + 3) * 4)                    = 20.0
price * tax              ERROR: unknown variable 'tax'
2 + * 3                  ERROR: unexpected '*' at position 4 in "2 + * 3"
```

Precedence comes from the **grammar**: `term` handles `*` / `/` below `expr`, so they bind tighter.

Full runnable code: code/lld/ch10/interpreter/ (see the Full Edition).

Trade-offs

Pros	Cons
New rule = new class; each rule small and testable.	Large grammars → class jungle; tree-walking is slow.
Rules become data; parse once, evaluate many.	New *operations* touch every class → **Visitor** (Ch 11); you still need a parser.

When not to use it

Large grammars (use **ANTLR** + visitor design); performance-critical evaluation (compile it); an existing language fits (regex, SpEL, JSONPath, CEL); **security** — never let users run a general-purpose script engine; a restricted DSL is safer.

Common mistakes

1. Mixing parsing and evaluation (re-parse each time). 2. Ignoring precedence/associativity (`8 - 3 - 2` = `(8 - 3) - 2`). 3. Error messages without position. 4. Piling many operations on nodes instead of Visitor/pattern matching. 5. Growing a "tiny" DSL into an accidental programming language.

Real-world examples

`java.util.regex.Pattern`; **SpEL** (`SpelNode` AST, `getValue()`); SQL `WHERE` trees, spreadsheet formulas; promotion and feature-flag rule engines, Google **CEL**; cron expressions.

In the interview

> **30-second answer:** "Interpreter models a small language's grammar as classes, one per rule — terminals like numbers and variables, nonterminals like addition. A sentence is parsed into an AST and evaluated by calling `interpret` on the root with a context. It's Composite plus an evaluate operation. Great for small DSLs; for big grammars I'd use ANTLR, and for many operations over the tree I'd switch to Visitor or sealed types with pattern matching."

- *vs Visitor?* Interpreter puts evaluation **in** the nodes; Visitor moves operations **out**.

Self-check

1. **Q:** Terminal vs nonterminal? **A:** Leaves (numbers, variables) vs combinations (operators).
2. **Q:** What is the context? **A:** External info for evaluation, e.g. variable values.
3. **Q:** Why is `2 + 3 * 4` = 14? **A:** `*` lives in `term`, below `expr`, so it binds tighter.

TL;DR

One class per grammar rule; parse to an **AST**; evaluate recursively with a **context**. Perfect for **tiny DSLs**; big grammars need a parser generator and Visitor.

10.7 Chapter recap: the six patterns side by side

Pattern	Object you create	Question it answers	JDK / real example	Often confused with
Chain of Responsibility	A handler per check	*Who* handles this request?	Servlet filters, logging handlers	Decorator
Command	An object per request	How do I store, queue or undo it?	`Runnable`, `UndoManager`	Strategy
Iterator	A cursor object	How do I walk this without knowing its shape?	`Iterator`, `Spliterator`, `ResultSet`	Visitor
Mediator	One coordinator	How do many objects cooperate without knowing each other?	UI controllers, chat servers, saga orchestrators	Observer, Facade
Memento	A sealed snapshot	How do I safely go back?	DB rollback, Git, `kubectl rollout undo`	Command undo, Prototype
Interpreter	A node per grammar rule	How do I evaluate a small language?	`Pattern`, SpEL	Composite, Visitor

Combo facts: the request in a **CoR** is often a **Command**; a **Mediator** often dispatches **Commands**; **Memento + Command** give scalable undo; **Iterator + Composite** walk trees; **Interpreter = Composite + evaluate**, and when operations multiply move to **Visitor**.

Self-check (whole chapter)

1. **Q:** Which two GoF behavioral patterns are class-scoped? **A:** Template Method and Interpreter.
2. **Q:** Auth → rate limit → validation, any step may reject? **A:** Chain of Responsibility.
3. **Q:** Undo/redo for a text editor? **A:** Command per edit, optionally Memento snapshots.
4. **Q:** Five form widgets depend on each other? **A:** Mediator.
5. **Q:** Business users write discount rules? **A:** Interpreter (small DSL), possibly with Specification (Ch 11).

TL;DR

Behavioral patterns turn **hidden control flow** into **named objects**: handlers, requests, cursors, coordinators, snapshots, grammar nodes. Each buys flexibility with indirection — use it when the flexibility is a real requirement.

Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF · Study Edition

> **Study edition.** Every topic of the full chapter, with the padding removed. For complete code listings, full outputs and extra follow-ups, see the Full Edition (`README.md`).
>
> **What you will learn.** The remaining five GoF behavioral patterns — **Observer, State, Strategy, Template Method, Visitor** — and how to tell apart the three that share one diagram (Strategy, State, Bridge). Then six non-GoF patterns interviews expect: **Null Object, Dependency Injection, Repository, Specification, MVC** and **Event Sourcing (lite)**, a "which pattern do I need?" table, and finally the **architectural patterns** (layered, client-server, modular monolith vs microservices, event-driven, pipe-and-filter, hexagonal/clean).

How to run the code

```
cd chapters/11-patterns-behavioral-2/code
javac --release 21 -Xlint:all -d /tmp/lld_11 $(find . -name '*.java')   # compiles with 0 warnings
java -cp /tmp/lld_11 lld.ch11.observer.Main
# also: state, strategy, templatemethod, visitor, nullobject, di, repository, specification, mvc, eventsourcing, architecture
```

The idea running through this chapter: most of these patterns turn a conditional (`if` / `switch`) or a hard-coded call into an **object** that can be swapped, subscribed, or visited.

11.1 Observer

What it is

Observer lets one object (the **subject** / publisher) notify many dependants (the **observers** / subscribers / listeners) when something happens — **without knowing who they are or what they do**. Analogy: subscribers to a YouTube channel get notified of each new video; the channel doesn't care what they do with it.

GoF intent: a one-to-many dependency so that when one object changes, all its dependants are notified and update automatically.

Participant	Role	In `lld.ch11.observer`
Subject	Knows its observers; attach/detach	`BuildPipeline` (`addListener` returns an unsubscribe handle)
Observer	The update interface	`BuildListener`
ConcreteSubject	Holds the state; notifies on change	`BuildPipeline` (one class plays both in modern Java)
ConcreteObserver	Implements the update	`SlackNotifier`, `DashboardUpdater`, `MetricsRecorder`, `DeployTrigger`

The problem it solves

A build pipeline calling Slack, dashboard, metrics and deployer directly depends on a chat service, must be edited for each new reaction, needs four fakes to test, and needs flags to disable deploy in staging. Observer replaces four references with **one list of one interface. The source only announces; every listener chooses its own reaction.**

How it works

An **observer interface** with one method; the **subject** keeps a list with subscribe/unsubscribe; on change it **finishes updating its state first**, then calls each observer; observers come and go at run time.

Push vs pull:

	Push	Pull
Observer receives	All the data (e.g. a `Reading`)	"I changed" + a subject reference
Pros	Simple; no dependence on subject getters	Takes only what it needs; interface stable as data grows
Cons	Sends unneeded data; interface grows with data	Couples observers to the subject; must work out *what* changed

Structure (UML)

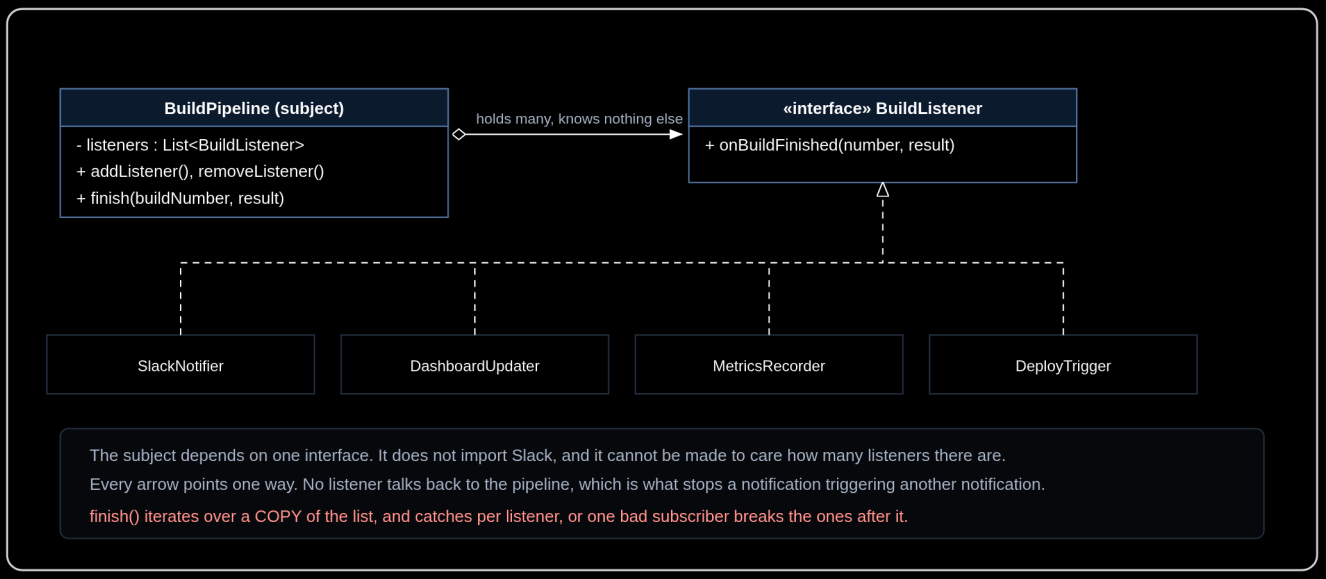
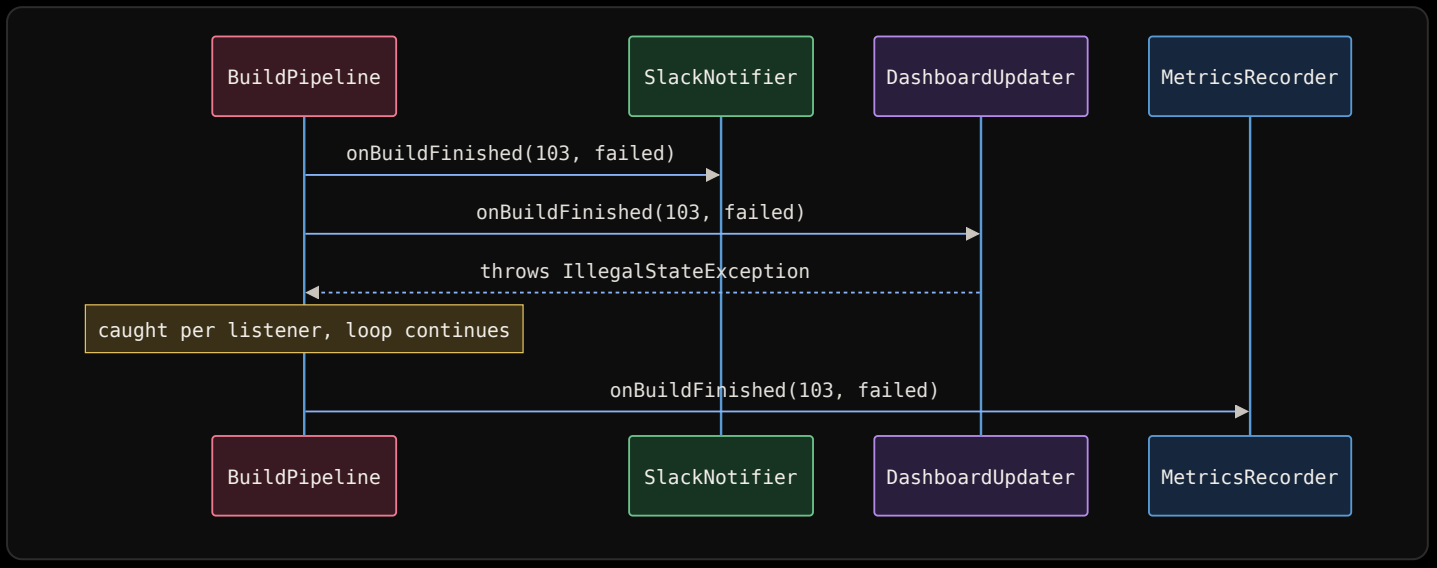


Figure — BuildPipeline holds a list of BuildListener; four concrete listeners implement it (course diagram)

Dependencies run one way: listeners never call the pipeline back.



Java example

`lld.ch11.observer` shows the **build pipeline** (a `CopyOnWriteArrayList` iterates a snapshot, so unsubscribing mid-loop is safe; exceptions caught per listener; `addListener` returns an unsubscribe handle), **push vs**

pull (`WeatherStation`), a tiny typed **`EventBus`** (synchronous, stores nothing — not a queue), and JDK **`PropertyChangeSupport`**, which doesn't fire when old and new values are equal.

```
// File: code/lld/ch11/observer/BuildListener.java
@FunctionalInterface
public interface BuildListener {
    void onBuildFinished(int buildNumber, String result);
}

// File: code/lld/ch11/observer/BuildPipeline.java
public final class BuildPipeline {
    // CopyOnWriteArrayList iterates over a snapshot: a listener that unsubscribes
    // itself mid-notification cannot break the loop, and it is safe across threads.
    private final List<BuildListener> listeners = new CopyOnWriteArrayList<>();

    /** Returns a handle: whoever subscribes owns the matching unsubscribe (prevents leaks). */
    public Runnable addListener(BuildListener listener) {
        listeners.add(listener);
        return () -> listeners.remove(listener);
    }

    public void finish(int buildNumber, String result) {
        System.out.println("Build " + buildNumber + " finished: " + result);
        for (BuildListener l : listeners) {
            try {
                l.onBuildFinished(buildNumber, result);
            } catch (RuntimeException e) {          // one bad listener must not starve the rest
                System.out.println("  [pipeline] a listener failed: " + e.getMessage());
            }
        }
    }
}
```

In the output, build 103's dashboard listener throws (`a listener failed: dashboard is offline`) and metrics still records it.

Full runnable code: code/lld/ch11/observer/ (see the Full Edition).

Why `java.util.Observer` / `Observable` were deprecated

Deprecated since **Java 9**: limited event model, **unspecified order**, notifications not one-for-one with changes; plus `Observable` is a **class** (uses up inheritance), `setChanged()` is protected, `update` is untyped, thread safety is partial. Use instead: `java.beans.PropertyChangeSupport`, your own typed listener interface, `java.util.concurrent` queues, or **`java.util.concurrent.Flow`** (reactive streams with back-pressure).

Trade-offs

Pros	Cons
Source doesn't depend on reactions; add/remove at run time.	Hard to trace; **memory leaks** if never unsubscribed.
Each reaction testable (Open/Closed).	**No ordering/delivery guarantee**; as slow as all listeners combined.

Observer vs Mediator, Chain of Responsibility, callbacks

Looks like	The actual difference
Mediator	Observer broadcasts one way and each listener decides; a mediator decides what happens next, calls components back (two-way) and owns ordering.
Pub-sub with a broker	Same idea plus persistence, redelivery, ordering and a network hop.
Chain of Responsibility	Observer informs **all**; a chain stops at the first handler that accepts.
A callback	One callback is one observer; it becomes the pattern when listeners vary at run time.

Outside Java (JS `EventEmitter`, Python callables, C# `event`), **weak references** (e.g. `weakref.WeakSet`) fix leaks at the cost of listeners vanishing when unreferenced.

When not to use it

One listener forever (call it); you need ordering or guaranteed delivery (use Kafka/RabbitMQ/SQS); slow reactions (notify asynchronously); the flow must be easily traceable.

Common mistakes (the four classic ways observers go wrong, plus one)

1. **No unsubscribe → memory leak** — every `add` needs an owner for its `remove`.
2. **A throwing listener kills the loop** — catch per listener.
3. **Changing the list during notification** → `ConcurrentModificationException`; iterate a copy.
4. **Listeners calling back into the subject** → cycles/recursion.
5. **Notifying before state is updated** → pull observers read stale data.
6. **Relying on notification order** — if order matters, that's a Mediator.

Real-world examples

UI listeners (Swing, DOM `addEventListener`); JDK `PropertyChangeListener`, `Flow`; Spring `@EventListener`; Guava `EventBus`, RxJava/Reactor; MVC model→view; Kafka/SNS pub-sub at system scale; CDC (Debezium).

In the interview

> **30-second answer:** "Observer swaps hard-coded calls for a list of objects sharing one listener interface. The subject says 'this happened' and each observer picks its own response, so reactions plug in and out and are tested alone, and the subject depends on none of them. In production I iterate over a copy of the list, catch exceptions per listener, and pair every subscribe with an unsubscribe. When I need durability, retries or ordering I use a message broker."

- *Observer vs Pub/Sub?* In Observer the subject holds observers directly; in pub/sub a broker sits between, adding durability and retries.

Self-check

1. **Q:** What does the subject know about observers? **A:** Only that they implement the interface.
2. **Q:** Classic Observer leak? **A:** Never-removed listeners stay strongly referenced.
3. **Q:** What replaces `Observable`? **A:** `PropertyChangeSupport`, a typed listener interface, or `Flow`.

TL;DR

One subject, a list of listeners, one-way notification. **Iterate a copy, catch per listener, always unsubscribe.** `Observable` is deprecated; use a broker when you need guarantees.

11.2 State

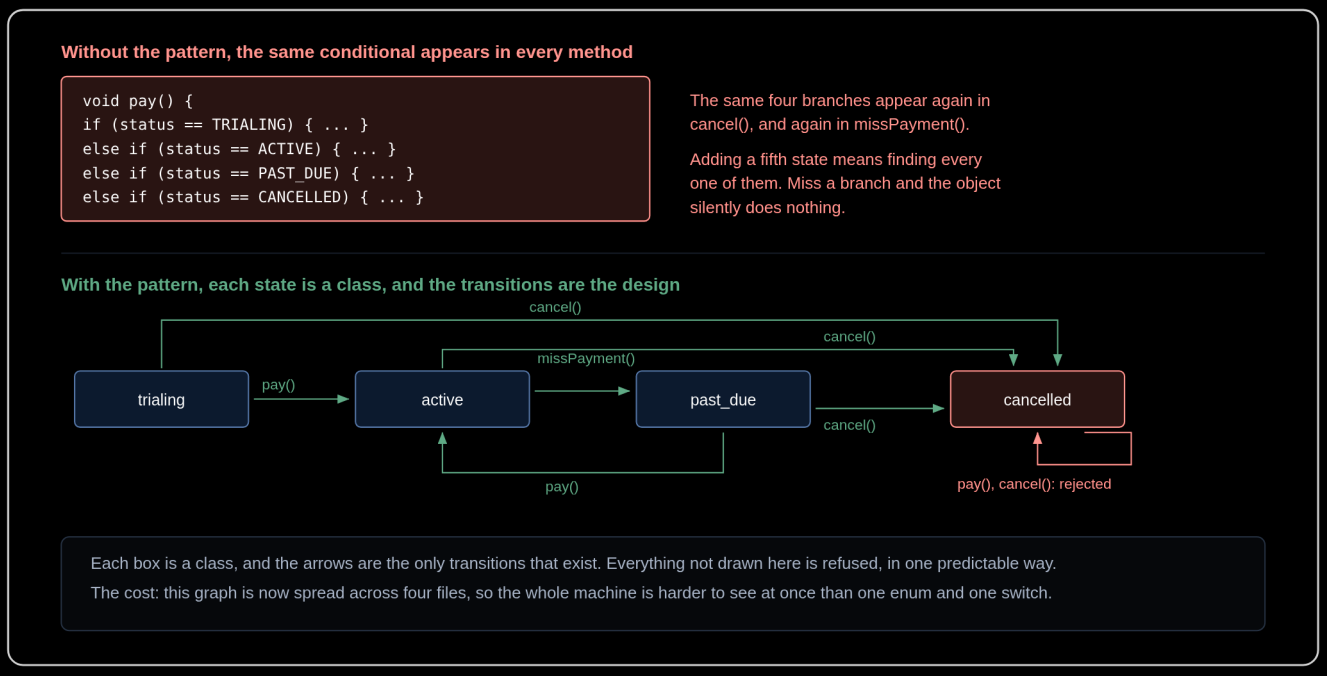


Figure — Top: the same four-branch status check in every method. Bottom: each state is a class; transitions form a graph (course diagram)

What it is

State (GoF): when an object's internal condition changes, **its behaviour changes**, as if it changed class. Instead of every method testing a status field, the object forwards each action to a **state object** and swaps it when the condition changes. Analogy: a phone's ring/vibrate/silent mode.

Participant	Role	In `lld.ch11.state`
Context	Holds current state; forwards requests	`Subscription`, `VendingMachine`
State	One method per forwarded request	`SubscriptionState`, `VendingState`
ConcreteState	Behaviour for one state	`Trialing`, `Active`, `PastDue`, `Cancelled`; `Idle`, `HasMoney`, `SoldOut`

GoF leaves open who performs transitions; letting states choose successors (as here) is flexible but makes states know each other.

The problem it solves

A subscription goes **trialing → active → past due → cancelled**; naively every method repeats the same four-way `if (status == ...)` chain — a fifth state means hunting every copy, and a **forgotten branch** silently does nothing. With State, **each state is a class**, context methods shrink to one line, and one state's rules live in one file.

How it works

A **State interface** has one method per action (every state answers every action, even "no"); the **context** delegates (`state.pay(this)`); each state rules on **legality** and the **successor** (`context.setState(new Active())`); terminal states refuse everything in one place.

States choose their own successors — the key difference from Strategy.

Structure (UML)

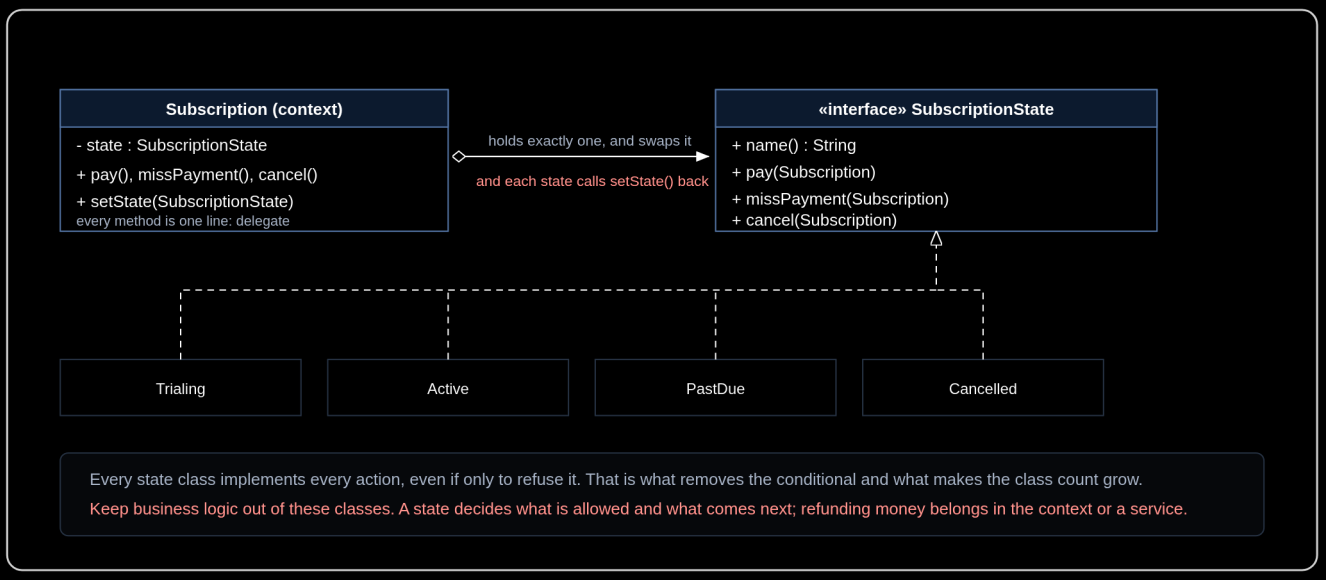
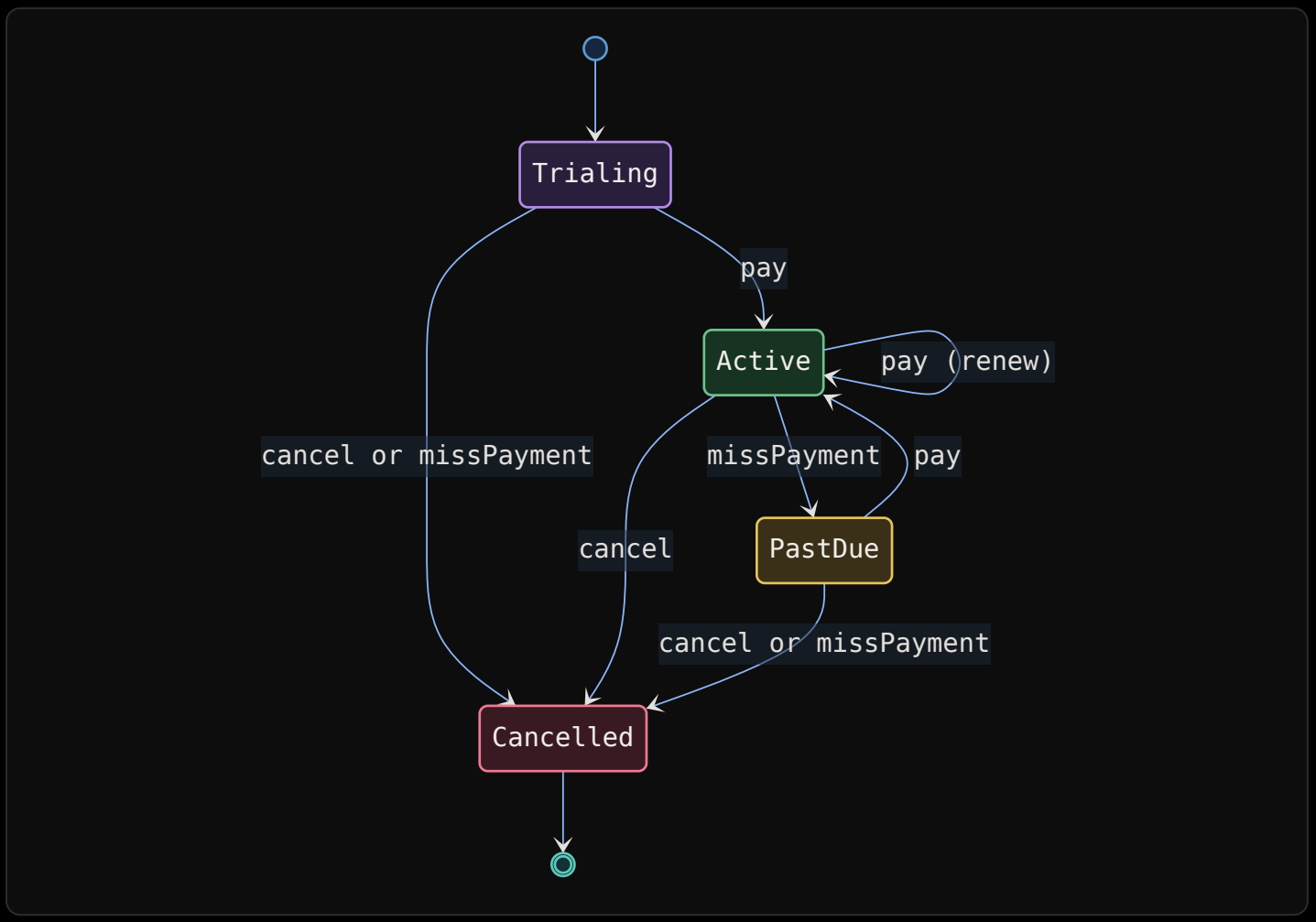


Figure — Subscription (context) holds a SubscriptionState; the four states call setState on the context (course diagram)



(The vending machine runs Idle → HasMoney → Idle or SoldOut; the traffic light RED → GREEN → YELLOW → RED.)

Java example

Three styles: (1) **class per state** — sealed interface, field-less `record` states; (2) **enum machine** (`TrafficLight`) — the idiomatic Java answer for small machines; (3) **vending machine** — **default methods that refuse**, so each state overrides only what it allows (`SoldOut` overrides nothing), and business actions live in the **context**.

```java
// File: code/lld/ch11/state/SubscriptionState.java
public sealed interface SubscriptionState permits Trialing, Active, PastDue, Cancelled {
    String name();
    void pay(Subscription s);
    void missPayment(Subscription s);
    void cancel(Subscription s);
}

// File: code/lld/ch11/state/Trialing.java
public record Trialing() implements SubscriptionState {
    public String name() { return "trialing"; }
    public void pay(Subscription s) {
        System.out.println("  [trialing] first payment taken");
        s.setState(new Active());                    // the state chooses its successor
    }
    public void missPayment(Subscription s) {
        System.out.println("  [trialing] trial ended unpaid");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}

// File: code/lld/ch11/state/Subscription.java
public final class Subscription {
    private SubscriptionState state = new Trialing();

    void setState(SubscriptionState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }
    // ... status()

    public void pay()         { state.pay(this); }
    public void missPayment() { state.missPayment(this); }
    public void cancel()      { state.cancel(this); }
}

// File: code/lld/ch11/state/TrafficLight.java — each constant overrides next()
    RED(30) {
        TrafficLight next() { return GREEN; }
        boolean pedestriansMayCross() { return true; }
    },
    // ... GREEN -> YELLOW, YELLOW -> RED
```

In the output, paying after cancellation prints `[cancelled] refused: no payments after cancellation` — an **illegal action** refused the same way every time, by code in one file.

Full runnable code: code/lld/ch11/state/ (see the Full Edition).

State vs Strategy (and Bridge) — same diagram, different reasons

	State	Strategy	Bridge
Who chooses the object	The **states** themselves	The **caller**	Once at construction/config
How often it changes	Repeatedly	Once, sometimes per call	Effectively never
Alternatives reference each other?	**Yes**	**No**	No
What varies	Current *condition* and allowed actions	*How* one job is done	A whole design dimension
Why it exists	Model a lifecycle	Make an algorithm replaceable	Stop two hierarchies multiplying

Naming test: do the alternatives name each other? Yes → State. No → Strategy (or Bridge, if the choice never changes).

Also: **an enum plus one** `switch` is often the better answer below three repeated switches; **state-machine libraries** (Spring Statemachine, XState) express it as data past about a dozen states.

Trade-offs

Pros	Cons
One state's rules in one file; no branch missed.	More classes; transitions spread over files.
Illegal actions rejected consistently.	Persisting needs one name↔class mapping.

When not to use it

Two states and few methods (a boolean); only data differs (a field); a big formal machine (a transition table `Map<(state,event),handler>` or Spring Statemachine). Rule of thumb: pays off once **three or more methods repeat the same status switch**. If you keep sketching the machine on paper to follow your code, make that sketch (a table) the source of truth.

Common mistakes

1. Successors hard-coded everywhere at 12+ states (move to a table).
2. Business logic in state classes (states rule on legality and successor only).
3. Illegal actions that silently do nothing.
4. Storing state objects in the DB — store a name/enum.
5. Sharing *stateful* state objects between contexts.

State in other languages

Java: enum or `sealed` + exhaustive `switch`. Kotlin/Scala: sealed + exhaustive `when` / `match`. Rust: enums and **typestate** (illegal calls don't compile). Python: a `(state, action)` → handler dict. TypeScript: discriminated unions.

Real-world examples

Order lifecycles; Stripe subscription statuses; **TCP connections** (GoF's example); approval workflows; vending machines, ATMs, elevators. (`Thread.State` is an enum of states, not the pattern.)

In the interview

> **30-second answer:** "State swaps a status field and its repeated switches for one class per state. The context forwards each action to its current state object, which decides if it's legal and which state follows. Each state's rules sit in one file and illegal actions are refused consistently. It draws like Strategy, but states name each other and advance the machine themselves, while strategies are picked from outside. For small Java machines I'd use an enum or a sealed interface."

- *Persist it?* Store the state name/enum; rebuild via one mapping.
- *Concurrency?* Racing "cancel" and "deliver": lock, or optimistic `UPDATE ... WHERE status = 'PLACED'` and check the row count.

Self-check

1. **Q:** Who changes the current state? **A:** The state objects, via the context's `setState`.
2. **Q:** The naming test? **A:** Alternatives reference each other → State; never → Strategy/Bridge.
3. **Q:** What belongs in the context, not the state? **A:** Business actions and data (dispensing, inventory).

TL;DR

A class per state; the context forwards; each state picks its successor. Same diagram as Strategy, but states **know each other**. Enums for small machines, tables/libraries for big ones.

11.3 Strategy

What it is

Strategy (GoF): define a **family** of algorithms, put each in its own class behind a **common interface**, and make them interchangeable, so the algorithm varies independently of the code using it — often chosen at run time. The **context** never branches on which algorithm it holds. Analogy: getting to the airport by car, train or taxi — same goal, different algorithm.

Participant	Role	In `lld.ch11.strategy`
Strategy	Interface all algorithms share	`RetryPolicy` (also `Comparator`)
ConcreteStrategy	One per algorithm	`FixedDelay`, `ExponentialBackoff`, the `RetryPolicy.none()` lambda (course: `NoRetry`)
Context	Configured with a strategy; delegates	`ApiClient`

The problem it solves

Retry = none, fixed delay, or exponential backoff. An `if (policy == ...)` chain puts backoff maths in the API client, needs edits per policy, and can't be tested alone. With Strategy, `ApiClient` asks "how long before the next attempt?" and **never checks which policy it holds**; the choice is made **outside** (config, feature toggle, customer setting).

How it works

A small **strategy interface** (one question: `delayBeforeAttempt(attempt)`); one **concrete strategy** per algorithm; the **context** receives one and calls it; a **factory, config or DI container** chooses — not every call site.

Structure (UML)

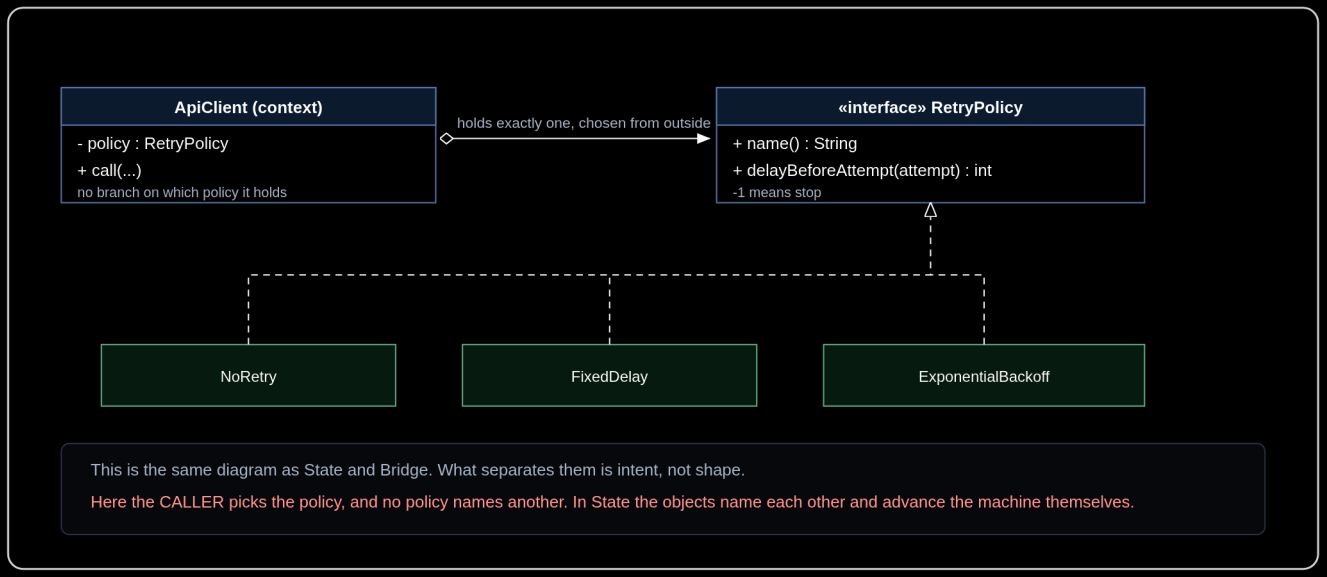
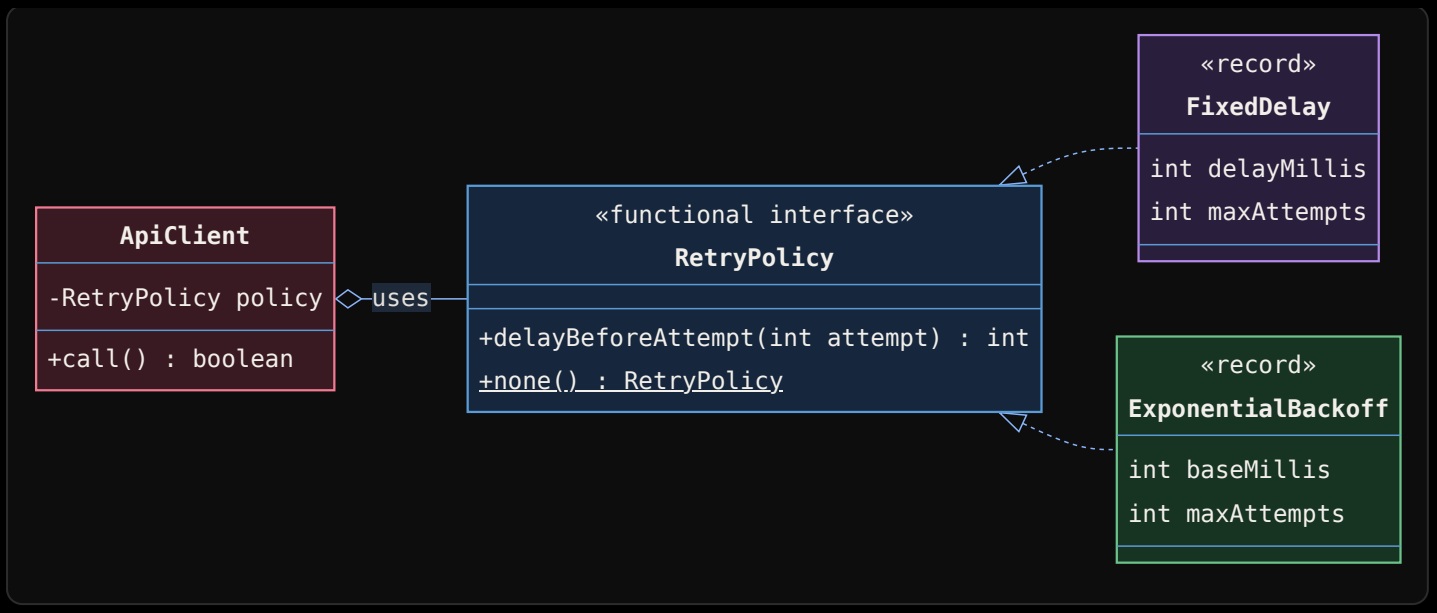


Figure — ApiClient holds a RetryPolicy; NoRetry, FixedDelay and ExponentialBackoff implement it (course diagram)

No strategy references another — that absence tells Strategy from State.



Java example

Retry policies (`none()` is a lambda; configured ones are **records**), `Comparator` composition (`comparing(...).reversed().thenComparing(...)`), and a **registry** `Map` of lambdas chosen by config key.

```
// File: code/lld/ch11/strategy/RetryPolicy.java
@FunctionalInterface
public interface RetryPolicy {
    int STOP = -1;

    int delayBeforeAttempt(int attempt);

    static RetryPolicy none() { return attempt -> STOP; }
}

// File: code/lld/ch11/strategy/ExponentialBackoff.java
public record ExponentialBackoff(int baseMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) {
        return attempt > maxAttempts ? STOP : baseMillis << (attempt - 2);
    }
}

// File: code/lld/ch11/strategy/ApiClient.java
    public boolean call() {
        for (int attempt = 1; ; attempt++) {
            if (server.test(attempt)) {
                System.out.println("  attempt " + attempt + ": ok");
                return true;
            }
            int delay = policy.delayBeforeAttempt(attempt + 1);
            if (delay == RetryPolicy.STOP) {
                System.out.println("  attempt " + attempt + ": failed, giving up");
                return false;
            }
            System.out.println("  attempt " + attempt + ": failed, waiting " + delay + "ms");  // simulated, no
sleep
        }
    }
```

Against a server that is always down, `ExponentialBackoff(100, 4)` waits 100, 200, 400 ms and then gives up. Adding jitter or `Retry-After` support is one more class or lambda; `ApiClient` doesn't change.

Full runnable code: code/lld/ch11/strategy/ (see the Full Edition).

Lambdas as strategies

Strategy is the pattern a plain function replaces most often. Any **functional interface** (one abstract method) — `Comparator`, `Function`, `Predicate`, your `RetryPolicy` — can take a lambda. Keep a named

class/record when the strategy has **configuration or state**, needs a **name for logging**, or must be **discovered/registered** (Spring beans, `ServiceLoader`). Other languages: pass a function (Python dict of functions as a registry; Go one-method interface or `func` field). Money in strategies stays `BigDecimal`, never `double`.

Trade-offs

Pros	Cons
New variants without editing working code; each testable alone.	One class per variant; someone must choose.
Configurable at run time; context keeps to its job.	Interface must fit all variants; overused for one-liners.

When not to use it

Two branches with no third in sight (an `if`); one-line branches (a map of lambdas); variants needing different inputs (forced interface — rethink); a choice fixed for the program's life (decide at wiring, or Bridge).

Common mistakes

1. Every call site choosing — the branch moved instead of disappearing.
2. Bloated interface with parameters half the implementations ignore.
3. Strategies that know each other — that's State.
4. Stateful strategies shared between threads — keep them immutable.

Real-world examples

`Comparator`; `RejectedExecutionHandler` (`AbortPolicy`, `CallerRunsPolicy`, ...); Spring `PasswordEncoder`; Resilience4j/Polly retry intervals; AWT `LayoutManager`; payment methods, tier pricing, map route modes.

In the interview

> **30-second answer:** "Strategy gives every variant of one job its own class behind a common interface, and something outside — usually a factory or config — picks it at run time. The context stops branching: my API client asks its retry policy how long to wait and never which policy it is. Each algorithm is testable alone and new ones don't touch working code. In Java a lambda is often enough, like a `Comparator`; I keep a class when it has configuration. It shares its diagram with State, but strategies never reference each other."

- *vs Template Method?* Template Method swaps **one step** via **inheritance** (fixed at creation); Strategy swaps **the whole algorithm** via **composition** (run time, several axes). Prefer Strategy when both fit.
- *Selection?* Config + factory `Map<String, Strategy>`, DI (Spring can inject a `Map` of beans), or per request.

Self-check

1. **Q:** Who chooses the strategy? **A:** Something outside the context — factory/config/DI.
2. **Q:** Why can `RetryPolicy.none()` be a lambda? **A:** `RetryPolicy` is a functional interface.
3. **Q:** Two JDK strategies? **A:** `Comparator`, `RejectedExecutionHandler` (also `LayoutManager`).

TL;DR

Interchangeable algorithms behind one interface, chosen from outside. The context stops branching. Lambdas for one-liners, records when configured. Not State (states know each other), not Template Method (inheritance, one step).

11.4 Template Method

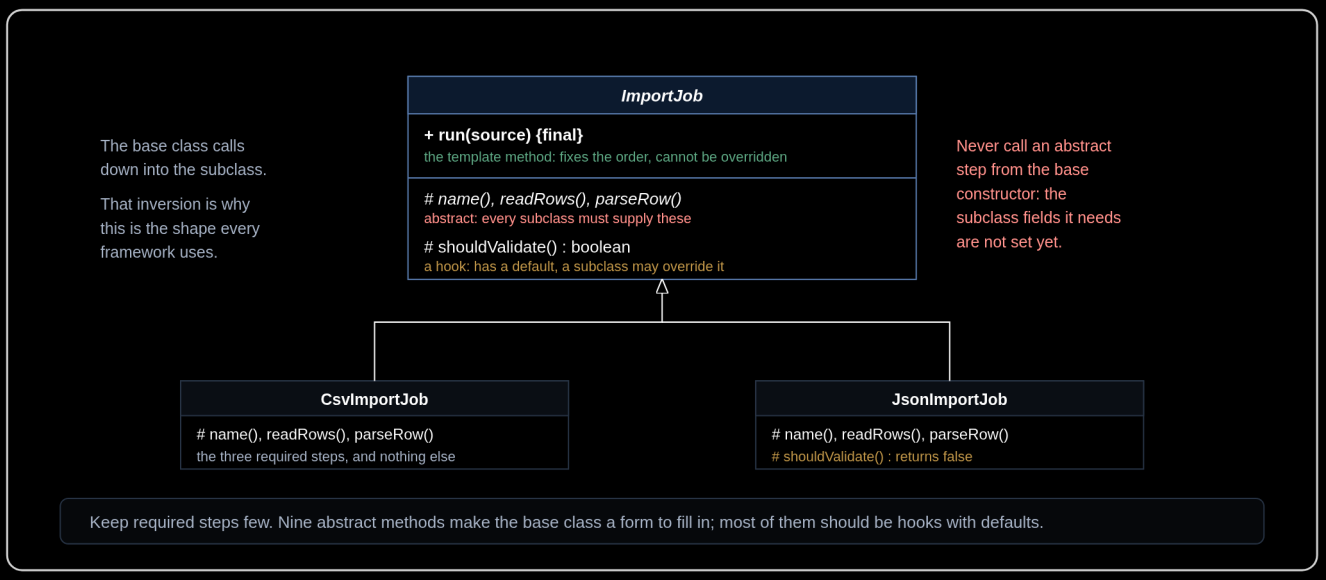


Figure — ImportJob with final run(), abstract steps, hook shouldValidate and private save; CsvImportJob and JsonImportJob override only what differs (course diagram)

What it is

Template Method (GoF): put an **algorithm's outline** in one base-class method (the *template method*) and let **subclasses** fill in certain steps without changing the sequence.

Participant	Role	In `lld.ch11.templatemethod`
AbstractClass	Declares the steps; implements the template method calling them in order	`ImportJob`, `CaffeineBeverage`
ConcreteClass	Implements the steps that differ	`CsvImportJob`, `JsonImportJob`, `Tea`, `Coffee`

Analogy: a recipe card with blanks — "boil water → brew ___ → pour → add ___ (if wanted)".

The problem it solves

An import job always reads, parses, validates, saves, reports; only reading and parsing differ for CSV vs JSON. Copying the flow per class means a change (say, an audit record) touches every copy. Template Method writes the flow once in a `final` method; varying steps are methods subclasses supply.

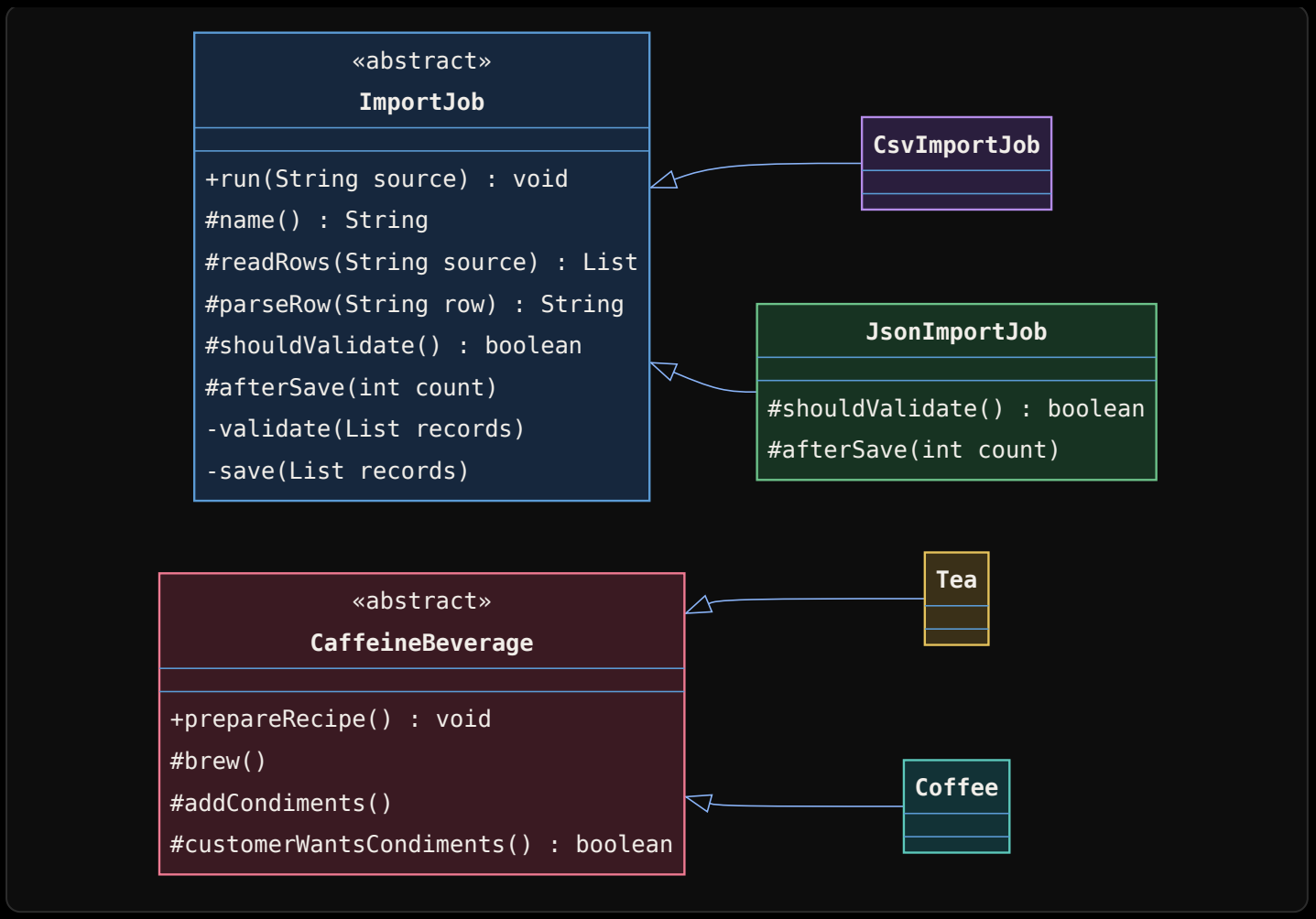
How it works

1. Write the **template method** in an abstract class, `final` so the order is guaranteed.
2. It calls three kinds of steps: **required (abstract)** — compiler-enforced; **hooks** — a default (often empty or `true`) subclasses *may* override, e.g. `shouldValidate()`; **shared** — `private`, unchangeable.
3. Each subclass implements only what differs.

Hollywood Principle — "Don't call us, we'll call you" (*Head First*): the base class decides **when** subclass methods run. This inversion of control is the backbone of frameworks — the framework runs the sequence, you supply pieces.

Structure (UML)

A subclass alone doesn't reveal the sequence it follows — only the parent does. That is the trade-off.



`run()` and `prepareRecipe()` are `final` in code (Mermaid has no marker for it).

Java example

Import jobs (`JsonImportJob` overrides both hooks) and *Head First*'s **beverages**, where `Coffee(black = true)` switches off condiments via a hook.

```java
// File: code/lld/ch11/templatemethod/ImportJob.java
public abstract class ImportJob {

    /** Template method. Marked final: subclasses cannot change the order, drop a step or insert one. */
    public final void run(String source) {
        System.out.println("Running the " + name() + " import:");
        List<String> rows = readRows(source);                // required step
        List<String> records = new ArrayList<>();
        for (String row : rows) records.add(parseRow(row));   // required step
        System.out.println("  parsed " + records.size() + " records: " + records);
        if (shouldValidate()) {                              // hook
            validate(records);
        } else {
            System.out.println("  skipping validation, source is validated upstream");
        }
        save(records);                                       // shared, fixed
        afterSave(records.size());                           // hook with an empty default
        System.out.println("  done");
    }

    // Required steps: the compiler forces every subclass to supply them.
    protected abstract String name();
    protected abstract List<String> readRows(String source);
    protected abstract String parseRow(String row);

    // Hooks: sensible defaults, overriding is optional.
    protected boolean shouldValidate() { return true; }
    protected void afterSave(int count) { }
    // ... private validate(...) and save(...): shared steps written once
}

// File: code/lld/ch11/templatemethod/JsonImportJob.java — overrides both hooks
    @Override protected boolean shouldValidate() { return false; }
```

The JSON run prints `skipping validation, source is validated upstream` and then its `afterSave` hook — the flow is identical, only steps differ.

Full runnable code: code/lld/ch11/templatemethod/ (see the Full Edition).

Template Method vs Strategy

	Template Method	Strategy
What varies	**One step**; algorithm fixed	**The entire** algorithm
Mechanism	**Inheritance**	**Composition**
When chosen	At creation (which subclass); fixed	Run time, per call
Axes that can vary	One (one parent)	Several
One file shows	Only the varying pieces	The complete algorithm

Template Method pays off when **most of the flow is common** and the compiler should **enforce the sequence**. Combined form: fixed sequence, steps passed as **callbacks** — Spring's `JdbcTemplate` owns connect/execute/close and takes a `RowMapper`.

Trade-offs

Pros	Cons
Shared flow written once; order guaranteed (`final`).	No single file tells the whole story; one parent only.
New variants small; compiler reports missing steps.	Tight coupling; base classes swell with abstract methods.

When not to use it

The flow is two lines; variants must combine (encrypted *and* CSV); behaviour must change after creation (Strategy); subclasses override every step.

Common mistakes

1. **Too many required steps** — make most of them hooks.
2. **Calling an overridable step from the base constructor** — subclass fields are still `null` / `0` (in C++ the base version runs). Call steps from the template method only.
3. **Omitting `final`** — subclasses can reorder or skip; the pattern becomes a mere agreement.
4. **Public steps** — callers could run step 3 alone; keep them `protected`.
5. **Violating Liskov** in overrides.

Template Method in other languages

Java: `final` template, `protected abstract` steps (an interface `default` method can't be `final`). Python: `abc.ABC` + `@abstractmethod`, no enforced `final`. TypeScript: `abstract` classes. C++: the **non-virtual interface (NVI)** idiom. Go/Rust: no inheritance — pass an interface/trait of steps, i.e. the Strategy shape.

Real-world examples

`InputStream.read(byte[],int,int)`; `AbstractList` (implement `get` / `size`); `HttpServlet.service()` → `doGet` / `doPost` (else 405); AQS `acquire` → `tryAcquire`; JUnit and Android `Activity` lifecycles; Maven phases; Spring `JdbcTemplate` / `TransactionTemplate`.

In the interview

> **30-second answer:** "Template Method puts an algorithm's outline in one final base-class method; subclasses supply only the steps that differ. Required steps are abstract, optional ones are hooks with a default body, fixed ones are private. It's the Hollywood principle — the base class calls the subclass — which is how servlets, JUnit and Android lifecycles work. It varies one step via inheritance; for run-time choice or several axes I prefer Strategy, or callbacks like JdbcTemplate."

- *vs Factory Method?* A factory method is often one step of a template method — the step whose job is creation (GoF notes the link; not a strict rule).

Self-check

1. **Q:** Which method is `final`? **A:** The template method (`run`, `prepareRecipe`).
2. **Q:** Abstract step vs hook? **A:** Abstract must be implemented; hooks have defaults.
3. **Q:** Why is `JdbcTemplate` more flexible? **A:** It takes the varying step as a callback (composition), not a subclass.

TL;DR

Fixed order in a `final` base method; subclasses fill steps; hooks are optional steps. Hollywood principle. Inheritance-based — prefer Strategy/callbacks for flexibility.

11.5 Visitor

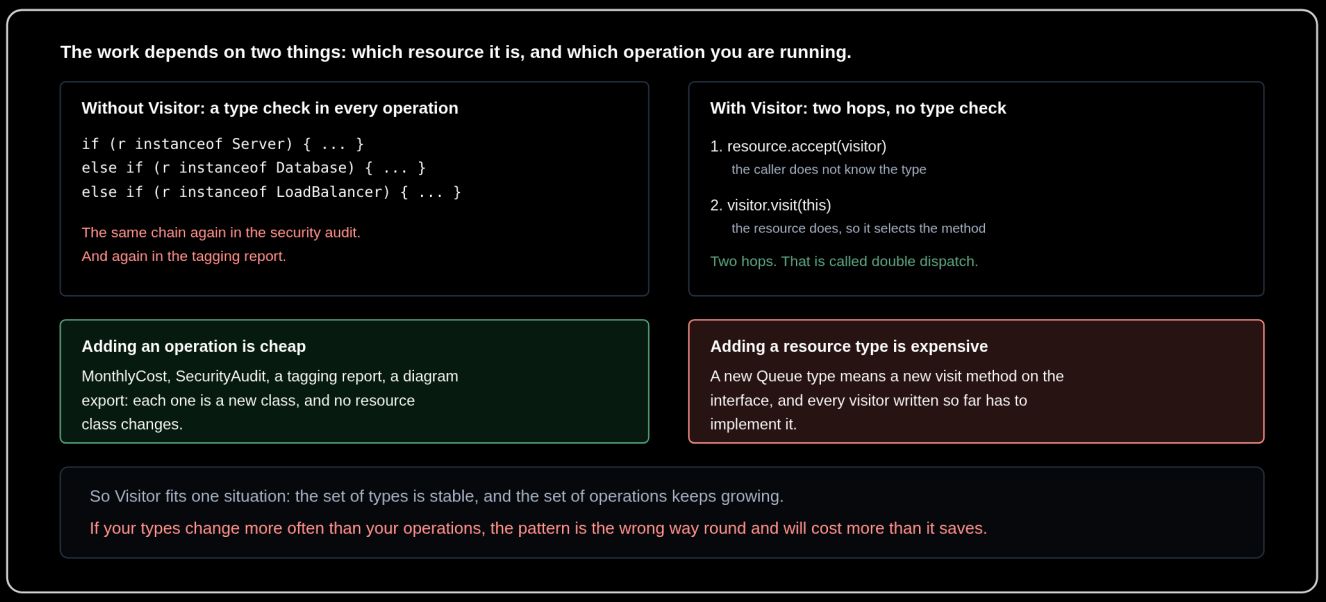


Figure — Without Visitor every operation repeats an instanceof chain; with Visitor two calls (accept, then visit) pick the right code (course diagram)

What it is

Visitor lets you **add new operations** over a set of classes **without changing them**. Each operation is a **visitor** class with one method per element kind; each element keeps a one-line `accept(visitor)` whose body is `visitor.visit(this)`.

GoF intent: represent an operation on the elements of an object structure as a separate object, so new operations don't change the element classes.

Participant	Role	In `lld.ch11.visitor`
Visitor	One `visit` per ConcreteElement	`PlanVisitor`
ConcreteVisitor	One class per operation, often accumulating	`MonthlyCost`, `SecurityAudit`
Element	Declares `accept(visitor)`	`Resource`
ConcreteElement	`accept` = `visitor.visit(this)`	`Server`, `Database`, `LoadBalancer`, `ResourceGroup`
ObjectStructure	Lets a visitor reach every element	the `ResourceGroup` tree in `Main`

Analogy: a tax inspector brings tax rules to different businesses; next year a fire inspector brings different rules — the businesses don't change.

The problem it solves

An infrastructure plan has a **stable** set of kinds (server, database, load balancer) but **operations keep arriving** (billing, security review, export). Inside the classes, unrelated teams edit `Server`; outside with `instanceof` chains, a new kind means finding every chain unaided. Visitor gives each operation a class and uses **double dispatch** instead of type checks.

How it works — double dispatch

Java picks the **override** by the receiver's run-time type, but the **overload** (`visit(Server)` vs `visit(Database)`) at **compile time** from the argument's static type — so `visitor.visit(resource)` with a `Resource` can't compile to the right overload. Visitor makes **two calls**:

1. `resource.accept(visitor)` — dispatch on the **element's** run-time type (`Server.accept` runs).

2. Inside it, `visitor.visit(this)` — `this` is statically `Server`, so `visit(Server)` is chosen; dispatch then happens on the **visitor's** run-time type.

The code that runs depends on **both** types: **double dispatch**.

Structure (UML)

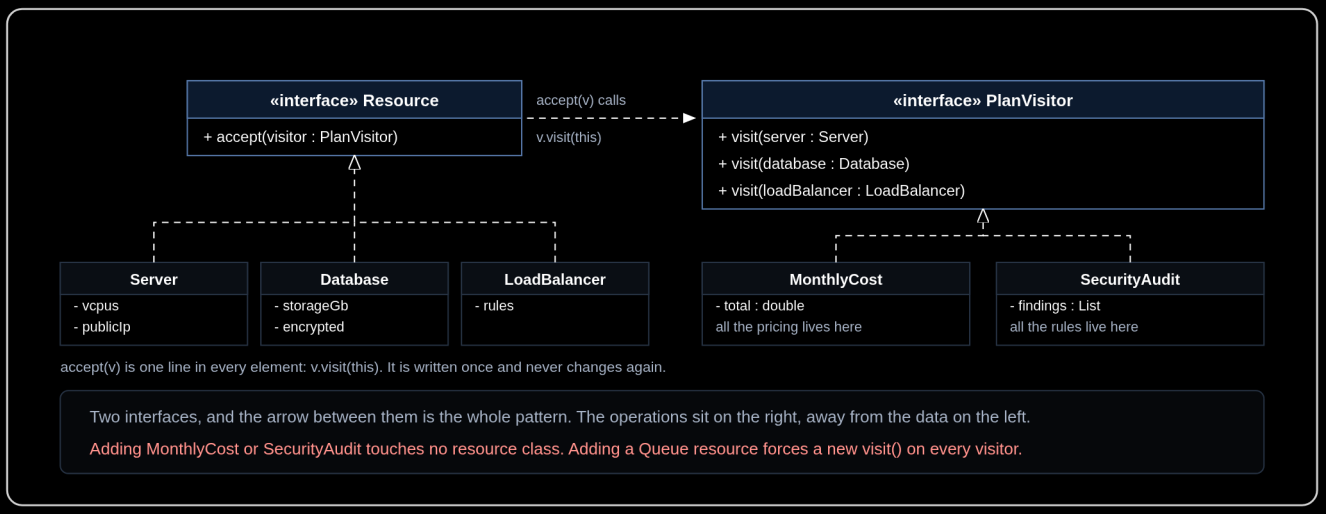
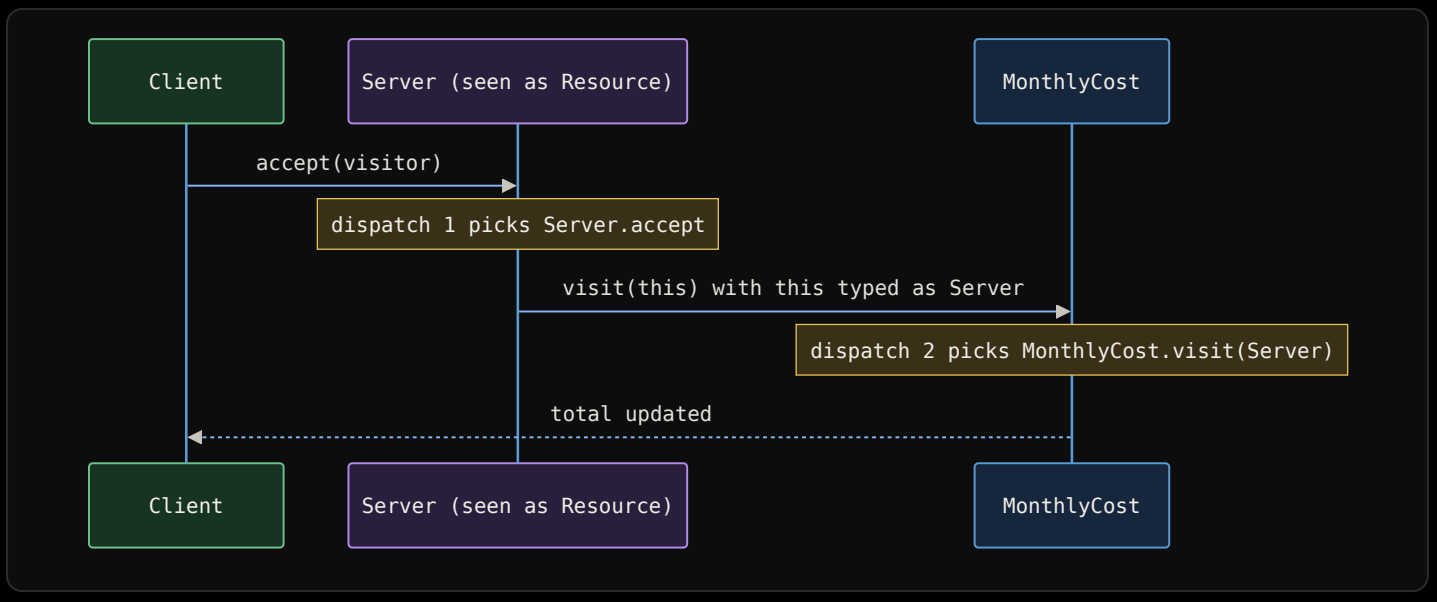


Figure — Resource with accept; PlanVisitor has one visit per resource type; MonthlyCost and SecurityAudit implement it (course diagram; it shows `total : double`, our code uses `BigDecimal`)

Each visitor depends on **every** element type — that's why **adding an element type is expensive**.



Java example

(1) **Classic Visitor** — record elements; `ResourceGroup` is a **Composite** visiting itself then its children; a new visitor per run (they accumulate). (2) **Java 21** `ModernPlan` — a sealed `Infra`, `switch` with **no** `default` (compiler-checked exhaustive), **record patterns** and **guards**; guarded cases don't count toward exhaustiveness, hence the unguarded fallbacks.

```java
// File: code/lld/ch11/visitor/PlanVisitor.java
public interface PlanVisitor {
    void visit(Server server);
    void visit(Database database);
    void visit(LoadBalancer loadBalancer);
    void visit(ResourceGroup group);
}

// File: code/lld/ch11/visitor/Server.java
public record Server(String name, int vcpus, boolean publicIp) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }   // 2nd dispatch: overload picked by static type Server
}

// File: code/lld/ch11/visitor/ResourceGroup.java
public record ResourceGroup(String name, List<Resource> children) implements Resource {
    public void accept(PlanVisitor v) {
        v.visit(this);
        for (Resource child : children) child.accept(v);
    }
}

// File: code/lld/ch11/visitor/ModernPlan.java
    public sealed interface Infra permits Vm, Sql, Lb {}
    // ... records Vm, Sql, Lb; monthlyCost(...) is the same switch without guards

    public static Optional<String> audit(Infra r) {
        return switch (r) {
            case Vm(String n, int cpus, boolean pub) when pub -> Optional.of(n + ": server has a public IP");
            case Sql(String n, int gb, boolean enc) when !enc -> Optional.of(n + ": storage not encrypted");
            case Lb(String n, int rules) when rules == 0 -> Optional.of(n + ": load balancer has no rules");
            case Vm vm -> Optional.empty();     // no finding
            case Sql db -> Optional.empty();
            case Lb lb -> Optional.empty();
        };
    }
```

Both styles print `monthly cost: 258.00` and the same findings; the modern one needs no `accept` and no visitor interface.

Full runnable code: code/lld/ch11/visitor/ (see the Full Edition).

The one question that decides it: the expression problem

	Add a new operation	**Add a new type**
Methods on the classes (plain OO)	Edit every class	One new class
Visitor (or sealed + switch)	One new visitor/function	Edit the visitor interface **and every visitor**

Ask: **which changes more often — types or operations?** Stable types + growing operations → Visitor/sealed types. Growing types → methods on classes. No plain OO design makes both cheap (the **expression problem**). Visitor is **open for new operations, closed to new element types**.

Sealed interfaces + pattern matching (Java 17 sealed; Java 21 pattern `switch` and record patterns) give Visitor's exhaustiveness check with far less code. Rules: pattern switches must be exhaustive; **unguarded** cases over every permitted subtype need no `default`; a non-sealed type needs a `default` (check lost); an unrecompiled new subtype → `MatchException`; **dominance** — guarded `case Vm(...) when pub` must precede `case Vm vm`; no `case null` → `NullPointerException` on `null`.

Use a classic Visitor when the hierarchy **can't be sealed** (library classes exposing `accept`, e.g. ASM, javac trees), for per-node callbacks over big trees, or for injectable per-operation classes. Elsewhere: Kotlin `when`, Python `singledispatch`, C++ `std::visit`, Rust `match`.

Trade-offs

Pros	Cons
New operation = one class; related logic together.	New element type changes every visitor; indirect dispatch.
Compiler reports unhandled types; elements stay small.	Elements must expose data; ceremony for small sets.

When not to use it

New types arrive often; a tiny model (three types, one operation); an operation for one type only (a method); your language checks switch exhaustiveness (use that).

Common mistakes

1. Visitor for two element types. 2. Public element fields (use accessors/records). 3. Reusing a stateful visitor without reset. 4. A `default` in a sealed-type switch — **turns off** the exhaustiveness check. 5. A composite `accept` that forgets its children (decide where traversal lives). 6. A visitor where a sealed `switch` would do.

Real-world examples

Compilers/linters (javac `TreeVisitor`, ANTLR visitors); ASM `ClassVisitor`; `FileVisitor` + `Files.walkFileTree` (really a tree-walk callback — no double dispatch); DOM serialisers; query planners.

In the interview

> **30-second answer:** "Visitor pulls an operation out of the element classes into a class with one method per element type. Each element has a one-line `accept` calling `visitor.visit(this)`, which gives double dispatch: the method chosen depends on both element and visitor types, so no `instanceof` chains. Adding operations is cheap and adding types expensive — the expression problem — so it fits stable type sets like ASTs. In Java 21 I'd usually use a sealed interface with an exhaustive pattern-matching switch instead."

- *Why not just call `visitor.visit(resource)`?* With static type `Resource` the compiler can't choose `visit(Server)`.
- *vs Iterator / Interpreter?* Iterator decides the route, Visitor the work at each stop; Interpreter keeps evaluation **in** nodes, Visitor moves it **out**.

Self-check

1. **Q:** Body of every `accept`? **A:** `visitor.visit(this)`.
2. **Q:** What's cheap and what's expensive? **A:** New operations cheap; new element types expensive.
3. **Q:** Why no `default` in `monthlyCost`? **A:** It would hide missing cases; without it the compiler forces every permitted type.

TL;DR

Operations become visitor classes; elements just `accept`; double dispatch picks the method. Cheap operations, expensive types. In Java 21 prefer **sealed interfaces + exhaustive pattern matching**.

11.6 GoF behavioral patterns: quick-reference summary

Pattern	Apply when	Example	Pros	Cons
Chain of Responsibility	Handler not known in advance	HTTP middleware	Decouples sender/receiver	May go unhandled; hard to debug
Command	Queue, log, retry, undo	GUI actions, job queues	Undo/redo; decoupled invoker	Many small classes
Interpreter	Small language to evaluate	Regex, rule DSLs	Easy to extend grammar	Unmanageable for big grammars
Iterator	Standard traversal	Playlist browsing	Simple client code	Hidden costs
Mediator	Tangled peer interactions	Chat, air traffic control	Fewer dependencies	God-object risk
Memento	Undo, checkpoints	Editor undo, save games	Atomic restore	Memory-heavy
Observer	One change must reach others	Live data feeds	Loose coupling	Leaks, no ordering
State	Lifecycle-dependent behaviour	Order lifecycles	Localised state behaviour	More classes; graph spread out
Strategy	Several ways to do one task	Retry policies	Swappable, isolated	Someone must choose
Template Method	Fixed structure, varying steps	Framework lifecycles	Reuse; enforced order	Inheritance rigidity
Visitor	Many operations, stable types	Compiler passes	Easy new operations	Hard new element types

---

11.7 Beyond GoF: patterns every LLD interview expects

From Fowler's *PoEAA* (2002), Evans's *DDD* (2003) and *Head First*; used in almost every case study.

11.7.1 Null Object

What it is

A real object implementing the interface with **neutral, do-nothing behaviour**, used instead of `null`, so callers need no `if (x != null)`. Analogy: a guest badge that opens no special doors.

The problem it solves

Returning `null` for "not found" forces every caller to remember a check; one forgotten check is a production NPE, and the checks clutter logic.

How it works

An interface (`Customer`), a real class (`RegisteredCustomer`) and a **null class** (`GuestCustomer`) returning neutral values (0% discount) or doing nothing; return it wherever you'd return `null`. Stateless null objects can be an `enum` **singleton**. Offer `isGuest()` sparingly.

Java example

```java
// File: code/lld/ch11/nullobject/GuestCustomer.java
public enum GuestCustomer implements Customer {
    INSTANCE;

    public String displayName() { return "guest"; }
    public int discountPercent() { return 0; }            // neutral value
    public void notify(String message) { }                // does nothing, on purpose
    @Override public boolean isGuest() { return true; }   // for the rare caller that must know
}

// File: code/lld/ch11/nullobject/CustomerDirectory.java
    /** Style 1: never return null; return the Null Object instead. */
    public Customer find(String id) { return byId.getOrDefault(id, GuestCustomer.INSTANCE); }

    /** Style 2: make absence explicit in the type; the caller chooses the fallback. */
    public Optional<Customer> lookup(String id) { return Optional.ofNullable(byId.get(id)); }
```

Full runnable code: code/lld/ch11/nullobject/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: no null checks, polymorphism over conditionals. Cons: can **hide real errors**. **Don't use it** when absence is an **error** (throw) or the caller must decide (return `Optional`). **Null Object vs `Optional`:** `Optional` makes absence visible in the type — use it for lookups; use Null Object for legitimately optional **collaborators** (logger, listener, discount policy). Mistakes: side effects or non-neutral values, a mutable singleton, overusing `isNull()`.

Real-world examples

`Collections.emptyList()`; `OutputStream.nullOutputStream()` (Java 11); SLF4J `NOPLogger`.

In the interview

> **30-second answer:** "Null Object replaces `null` with an implementation that does nothing or returns neutral values, so callers never need null checks — like returning an empty list. I use it for optional collaborators like loggers, and `Optional` for lookups where the caller decides. I avoid it when absence is an error, because it would hide the bug." Not GoF — Bobby Woolf (*PLoPD 3*); Fowler's "Introduce Special Case".

> **TL;DR:** Harmless "nothing" object instead of `null`; `Optional` when the caller chooses; throw when absence is a bug.

11.7.2 Dependency Injection (DI)

What it is

A class **receives** its dependencies from outside instead of **creating** them with `new`. It is the usual way to implement **Inversion of Control** — a composition root or container wires objects. (Name settled by Fowler, 2004.) Analogy: a kitchen gets deliveries from suppliers; change supplier, the recipes don't change.

The problem it solves

`private final EmailSender sender = new EmailSender();` can never send SMS, can't be unit-tested without sending email, and hides its dependencies. Fix: depend on an **interface** (`MessageSender`), receive it via the **constructor**.

How it works

Dependencies are **constructor parameters typed by interfaces** (DIP, Ch 4); a **composition root** (`main` or a container) builds and passes them; tests pass **fakes**.

Style	How	Use for
Constructor	Constructor params → `final` fields	Required dependencies — the default
Setter	`setX(...)` later	Optional/changeable dependencies
Field	Framework writes fields (`@Autowired`)	Avoid: hidden, harder to test

Java example

`OrderService` receives a `MessageSender` **and a** `Clock` (`Clock.fixed` makes time testable). `Container` is a toy IoC container: register a recipe per type; `get` builds on demand and caches singletons.

```java
// File: code/lld/ch11/di/OrderService.java
public final class OrderService {
    private final MessageSender sender;
    private final Clock clock;

    public OrderService(MessageSender sender, Clock clock) {
        this.sender = sender;
        this.clock = clock;
    }

    public void placeOrder(String customer, String orderId) {
        LocalTime now = LocalTime.now(clock);
        sender.send(customer, "order " + orderId + " placed at " + now);
    }
}
```

A test injects `RecordingSender` instead. Full runnable code: code/lld/ch11/di/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: swappable implementations, easy fakes, explicit dependencies. Cons: indirection, container "magic", misconfiguration found late. Don't inject value objects or add single-implementation interfaces "just for DI". Mistakes: **Service Locator** (classes call `container.get` — dependencies hidden again), field injection, **8+ constructor dependencies** (split the class), circular dependencies.

Real-world examples

Spring, Guice, Dagger/Hilt, Jakarta CDI, Micronaut/Quarkus. Spring can inject a `Map<String, PaymentStrategy>` — DI + Strategy.

In the interview

30-second answer: "DI means a class receives its collaborators through its constructor, typed by interfaces, instead of creating them — Dependency Inversion plus Inversion of Control: a composition root or a container like Spring wires the graph. It gives swappable implementations and easy testing with fakes — I even inject the Clock. I prefer constructor injection, avoid field injection and service locators, and treat a huge constructor as a smell." *DI vs IoC?* IoC is the general idea; DI is one way to do it. *DI vs Factory?* A factory creates on request; DI hands in ready-made dependencies.

TL;DR: Don't `new` collaborators — **receive them**; wire at the composition root.

11.7.3 Repository

What it is

A **collection-like interface for loading and saving domain objects**, in domain language (`findByEmail`, `findActive`), that **hides storage** (Fowler *PoEAA*, Evans *DDD*). Analogy: a library front desk — you ask for books by author, not shelf numbers.

The problem it solves

SQL, connections and mapping scattered in business classes make rules unreadable, untestable without a DB, and tied to one storage technology.

How it works

The interface lives in the **domain layer** (return `Optional` for "maybe"); implementations in **infrastructure** (`InMemoryUserRepository` for tests, JDBC/JPA for production); services depend on the interface via DI.

Java example

```java
// File: code/lld/ch11/repository/UserRepository.java
public interface UserRepository {
    User save(User user);
    Optional<User> findById(String id);
    Optional<User> findByEmail(String email);
    List<User> findActive();
    boolean deleteById(String id);
}

// File: code/lld/ch11/repository/SignupService.java
    public String signUp(String id, String email) {
        if (users.findByEmail(email).isPresent()) return "rejected: " + email + " already registered";
        users.save(new User(id, email, true));
        return "created " + id;
    }
```

Full runnable code: code/lld/ch11/repository/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: storage-agnostic, in-memory testable logic; swappable storage. Cons: an extra layer for trivial CRUD; storage concerns can leak. One repository per **aggregate root**, not per table. **Repository vs DAO:** DAO is table-oriented; repository is domain-oriented. Mistakes: returning `null`, exposing `ResultSet`, business rules in the repository, `findByXAndYOrZ` explosions (use Specification). **Concurrency:** `signUp` is **check-then-act** — two concurrent sign-ups can both pass; enforce with a DB **unique constraint** (or `putIfAbsent` by email).

Real-world examples

Spring Data `JpaRepository` (queries derived from method names), Micronaut Data, Android Room.

In the interview

> **30-second answer:** "A repository is a collection-like interface for aggregates in domain language — save, findById, findByEmail — that hides storage. Services depend on the interface, so logic is testable with an in-memory implementation and storage can change. In machine-coding rounds I always put an in-memory repository behind an interface."

> **TL;DR:** A **domain-language collection interface** over storage; start in-memory, swap later.

11.7.4 Specification

What it is

A **business rule as an object** with `isSatisfiedBy(candidate)`, **combinable** with `and`, `or`, `not` (Evans & Fowler). Analogy: a shopping site's filter panel.

The problem it solves

Rules like "bargain electronics = electronics AND price < 50.00 AND in stock" get duplicated across queries, validation and UI, or explode into `findByCategoryAndPriceLessThan...` methods. Specification names

each rule, makes it reusable and composable.

How it works

`Specification<T>` with default `and` / `or` / `not` returning new specs (Composite + Interpreter-like evaluation); small named factories (`inStock()`); combine and pass to a repository or stream filter.

Java example

```java
// File: code/lld/ch11/specification/Specification.java
@FunctionalInterface
public interface Specification<T> {
    boolean isSatisfiedBy(T candidate);

    default Specification<T> and(Specification<T> other) { return c -> isSatisfiedBy(c) && other.isSatisfiedBy(c);
}
    default Specification<T> or(Specification<T> other)  { return c -> isSatisfiedBy(c) || other.isSatisfiedBy(c);
}
    default Specification<T> not()                       { return c -> !isSatisfiedBy(c); }
}
```

`inCategory("electronics").and(cheaperThan(5_000)).and(inStock())` selects `[usb-cable, keyboard]`. Full runnable code: code/lld/ch11/specification/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: named, reusable, testable rules for both querying and validation. Cons: in-memory specs don't become SQL automatically (need e.g. JPA Criteria); overkill for one or two filters. Java's `Predicate<T>` already is one; a named type adds domain meaning. Mistake: filtering millions of rows in memory — push down to the DB.

Real-world examples

Spring Data JPA `Specification<T>`, search filters, eligibility rules, access policies.

In the interview

> **30-second answer:** "A specification is a named business rule object with `isSatisfiedBy`, combinable with and/or/not. It removes duplicated conditions and finder-method explosions, and the same rule can filter a query and validate an object. `Predicate` is the building block; Spring Data JPA's `Specification` translates to SQL."

> **TL;DR: Rules as small combinable objects**, reused for querying and validation.

11.7.5 MVC — Model–View–Controller (the Head First "compound pattern")

What it is

Model (data + rules, no UI knowledge), **View** (displays, receives input), **Controller** (interprets input, updates the model). Reenskaug, Smalltalk, Xerox PARC, late 1970s. *Head First* calls it a **compound pattern**:

Pattern inside MVC	Where
Observer	Model notifies views on change; views re-render.
Strategy	View delegates input handling to a swappable controller (e.g. read-only mode).
Composite	View is a tree of nested widgets.

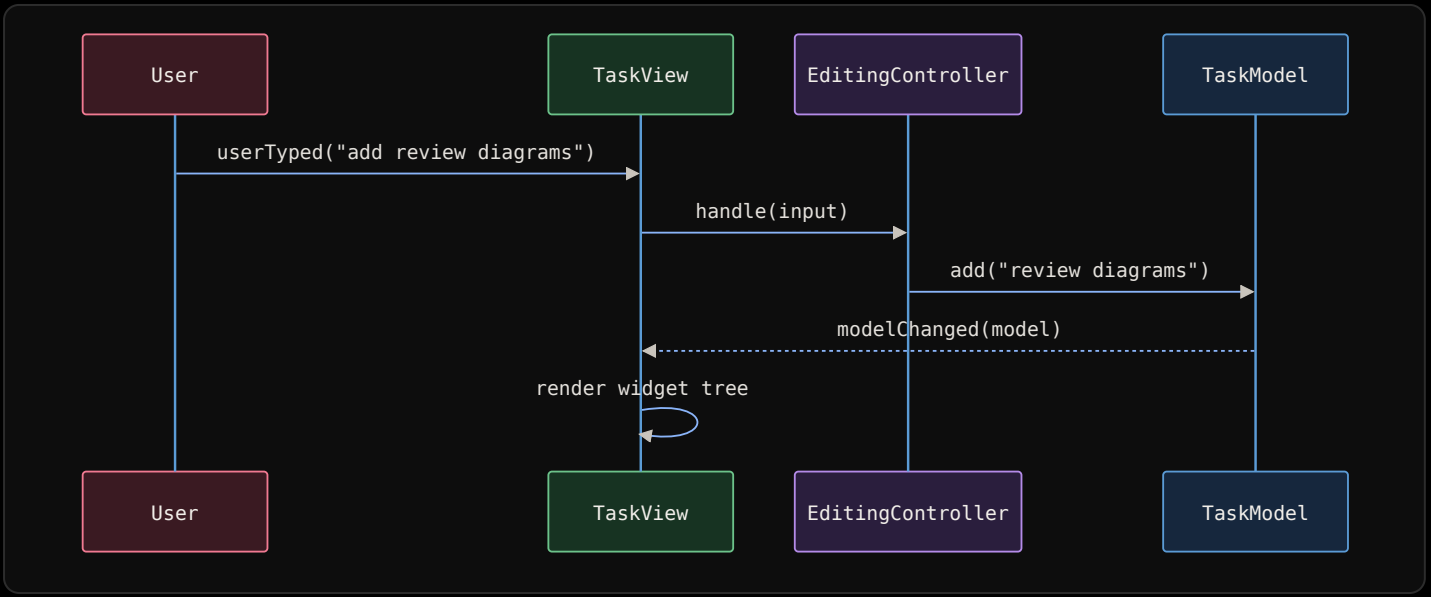
Analogy: kitchen = model, menu and plates = view, waiter = controller.

The problem it solves

Mixed UI, input and rules: can't change the screen safely, show data two ways, or test rules without a UI.

How it works

User acts on the **view** → view hands input to its **controller** → controller calls the **model** (never redraws) → model **notifies listeners** → view re-renders its **composite** tree.



Java example

A console to-do app: `TaskModel` knows only `ModelListener`s; `TaskView` implements `ModelListener`, renders `Widget.Panel`/`Widget.Label`, and forwards input to a swappable `TaskController`; switching to `ReadOnlyController` changes behaviour without touching view or model.

```
// File: code/lld/ch11/mvc/TaskModel.java
    public void add(String title) {
        if (title.isBlank()) throw new IllegalArgumentException("title must not be blank");
        tasks.add(new Task(nextId++, title, false));
        fireChanged();
    }
    // ...
    private void fireChanged() { listeners.forEach(l -> l.modelChanged(this)); }
```

Full runnable code: code/lld/ch11/mvc/ (see the Full Edition).

Trade-offs, variants and mistakes

Pros: rules testable without UI, several views per model. Cons: more classes; observer flows hard to trace; "fat" controllers. **Web MVC differs:** Spring MVC ("Model 2") has a **front controller** (`DispatcherServlet`) routing requests to `@Controller` methods; the view renders once per request and does **not** observe the model. Relatives: **MVP** (passive view) and **MVVM** (view binds to an observable view-model — Android `ViewModel` + `LiveData`/`StateFlow`, WPF; the view-model also acts as a mediator). Mistakes: fat controllers, views changing the model directly, models importing UI classes.

Real-world examples

Spring MVC, Rails, Django (MTV), Swing (`JTable` + `TableModel`), iOS UIKit, Android MVVM.

In the interview

30-second answer: "MVC separates the model — data and rules — from the view that shows it and the controller that interprets input. Head First calls it a compound pattern: the model is an Observer subject for views, the controller is a Strategy the view delegates to, and the view is a Composite

of widgets. On the web, like Spring MVC, it's request-driven with a front controller, and the view doesn't observe the model."

> **TL;DR: Model** (observable state + rules) → **View** (composite, observes) → **Controller** (strategy interpreting input).

11.7.6 Event Sourcing (lite)

What it is

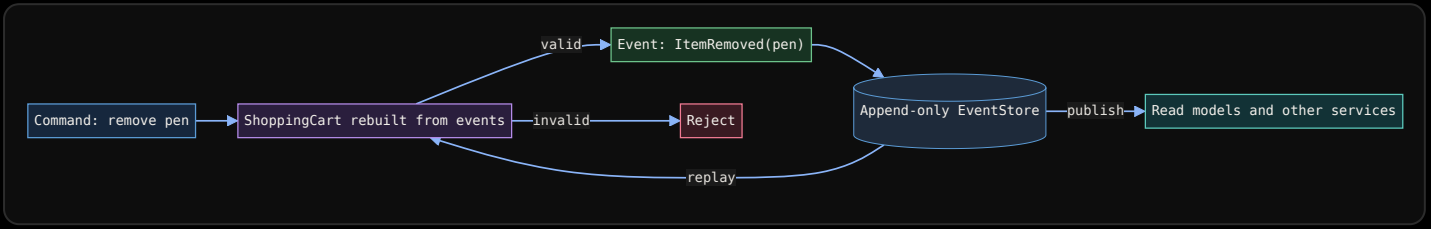
Store **every change as an immutable event** (`ItemAdded`, `ItemRemoved`, `CheckedOut`) in an **append-only log** and derive current state by **replaying** — instead of overwriting the latest state. It borrows Command (events ≈ recorded commands), Memento (snapshots) and Observer (projections) at system level. Analogy: a bank passbook — the transactions are the truth, the balance is derived.

The problem it solves

Update-in-place loses history: no "what did the cart look like before checkout?", no audit, no rebuilding a read model after a bug. Events give **time travel**, **audit**, and **new views** by replay.

How it works

1. **Command side:** replay the aggregate, **validate** the command, **emit a past-tense event**.
2. **Append** it — never update or delete events.
3. **Apply:** a pure state update, no validation (events are facts).
4. **Read side:** replay all or the first *n* events; publish events to observers building read models (**CQRS**).
5. Long streams: periodic **snapshots** (Memento) + replay of newer events.



Java example

A **sealed** `CartEvent` hierarchy; `apply` is an exhaustive pattern-matching `switch` (a forgotten event type is a compile error); `EventStore` is an append-only, `synchronized` in-memory log; prices `BigDecimal`.

```java
// File: code/lld/ch11/eventsourcing/ShoppingCart.java
    public static ShoppingCart replay(List<CartEvent> history, int n) {
        ShoppingCart cart = new ShoppingCart();
        history.stream().limit(n).forEach(cart::apply);
        return cart;
    }

    /** Pure state transition: no validation, no side effects. Events are facts. */
    private void apply(CartEvent e) {
        switch (e) {
            case CartEvent.ItemAdded a -> lineTotals.merge(a.sku(),
a.unitPrice().multiply(BigDecimal.valueOf(a.qty())), BigDecimal::add);
            case CartEvent.ItemRemoved r -> lineTotals.remove(r.sku());
            case CartEvent.CheckedOut c -> checkedOut = true;
        }
    }

    /** Command side: validate against current state, THEN emit an event. */
    public CartEvent decideRemove(String sku) {
        if (checkedOut) throw new IllegalStateException("cart already checked out");
        if (!lineTotals.containsKey(sku)) throw new IllegalArgumentException(sku + " not in cart");
        return new CartEvent.ItemRemoved(sku);
    }
```

Replaying the first *n* events shows the cart after each one ("time travel"); removing from a checked-out cart prints `command rejected: cart already checked out`.

Full runnable code: code/lld/ch11/eventsourcing/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: audit, time travel, new read models anytime. Cons: complexity, **event schema evolution**, eventual consistency. Not for plain CRUD. Mistakes: validation in `apply`; command-named events (`RemoveItem` vs `ItemRemoved`); mutable events; derived data in events; no snapshots; **no expected-version check on append** — real stores reject a second writer decided on stale state (optimistic concurrency); our toy store omits it.

Real-world examples

Bank ledgers; Git; Axon, EventStoreDB; Kafka with long retention (not *compacted* topics, which discard history).

In the interview

30-second answer: "Event sourcing stores every change as an immutable, past-tense event in an append-only log and derives state by replaying. Commands are validated against replayed state and then produce events; `apply` just folds events into state. You get audit, time travel and new read models by replay; the costs are complexity, schema evolution and eventual consistency. Snapshots keep replay fast — Memento plus a command log."

TL;DR: Store facts, not state; replay to rebuild; validate commands, never events; snapshot long streams.

11.8 Choose the right behavioral pattern

Start from the **symptom**, not the pattern name.

If you see / need...	Use	Not to be confused with
Independent checks, any may reject; per-route checks	**Chain of Responsibility**	Decorator
Undo/redo, queue, retry, log requested actions	**Command**	Strategy
Evaluate user-written rules in a small language	**Interpreter** (+ Specification)	Visitor
Walk a collection/tree/paged API	**Iterator**	Visitor
Interdependent objects where the *order* of reactions matters	**Mediator**	Observer
Roll back without exposing internals	**Memento**	Command undo
Notify an open-ended set of independent listeners	**Observer** (or a broker)	Mediator
3+ methods repeat a status `switch`; legality depends on status	**State**	Strategy
Interchangeable algorithms chosen by config/run time	**Strategy** (often a lambda)	State, Template Method
Long shared flow, a few differing steps	**Template Method** (or + callbacks)	Strategy
Many new operations over stable types	**Visitor** / sealed types + pattern matching	Iterator, Strategy
`null` checks for an optional collaborator	**Null Object** (`Optional` for lookups)	—
Classes create their own collaborators	**Dependency Injection**	Service Locator
Business logic mixed with storage	**Repository**	DAO
Same rule duplicated; finder-method explosion	**Specification**	—
UI, input and rules tangled	**MVC / MVVM**	—
Full history, audit, time travel	**Event Sourcing** (+ snapshots)	Plain audit log

Three questions for the look-alikes: (1) **Do the alternatives reference each other?** Yes → State; no → Strategy (or Bridge). (2) **Can one participant stop the flow?** Yes → CoR; all always run → Decorator/pipeline; everyone told, nobody stops → Observer. (3) **Does a central object decide what's next?** Yes → Mediator; each listener decides → Observer.

In the interview (chapter-level)

"Which patterns have you used?" Name concrete ones with their trade-off (e.g. Strategy for payment methods via a Spring-injected map; Observer via Spring events, moved to Kafka for retries; State for orders with `WHERE status = ?` guarding races). **"When would you avoid patterns?"** Two branches with no third coming, or when a lambda/enum does the job.

Self-check (whole chapter)

1. **Q:** Order placed → accepted → preparing → delivered; cancel only early. **A:** State.
2. **Q:** Card, UPI, wallet, more next quarter. **A:** Strategy via a factory/DI map.
3. **Q:** After an order: email, analytics, notify restaurant, more to come. **A:** Observer (or broker pub/sub).
4. **Q:** Type-check, optimise, pretty-print a fixed AST. **A:** Visitor, or sealed + pattern matching.
5. **Q:** Three patterns inside MVC? **A:** Observer, Strategy, Composite.

TL;DR

> - **Observer** broadcasts; **Mediator** coordinates; **CoR** lets someone stop the flow.

- **State** vs **Strategy**: states know each other; strategies are chosen from outside.
- **Template Method** = inheritance, one step varies; **Strategy** = composition, whole algorithm.
- **Visitor** = cheap operations, expensive types; Java 21 **sealed + switch** usually replaces it.
- Beyond GoF: **Null Object, DI, Repository, Specification, MVC, Event Sourcing** — the glue of real systems.

11.9 Architectural patterns, and how they differ from design patterns

A **design pattern** arranges a few classes inside a program. An **architectural pattern** (architectural style) fixes the shape of the whole system: its large parts (layers, modules, services), which part may depend on which, and how they communicate. Design patterns live *inside* those parts; a hexagonal service, for example, uses Repository and DI inside. Changing a design pattern means refactoring a few files; changing an architecture can take months. In LLD the first minute of an answer still names where classes live, and follow-ups ("microservices?", "how does notification learn an order was placed?") are architectural.

11.9.1 The styles at a glance

Style	Core idea	Use it when	Main cost	In an LLD answer
Layered (n-tier)	Layers; each calls only the one below	CRUD and business apps	Domain shaped by storage; features cut through all layers	`Controller -> Service -> Repository`
Client-server	Clients request; the server owns the data and rules	Shared data, one authority	Server bottleneck until scaled out	The API boundary; the server re-validates
Modular monolith	One deployable, modules with enforced boundaries	Default for new systems	One release train, shared failure	Packages per capability with interfaces
Microservices	Independent deployables, each owning its database	Many teams; different scaling or release needs	Network failures, sagas, eventual consistency, ops	Say where you *would* split
Event-driven	React to published events instead of calling	One action, many independent reactions	Hard to trace; duplicates, ordering	Observer in-process, a broker between services
Pipe-and-filter	Data flows through independent transforming steps	Parsing, ETL, middleware	Shared state across steps is awkward	`Filter<I, O>` plus composition
Hexagonal / clean	Core in the middle; I/O behind ports and adapters	Valuable rules, fast tests	More interfaces and mapping	No framework imports in domain; repo and gateway ports
MVC / MVP / MVVM	Separate UI state, display, input	Interactive UIs	Classes for small screens	§11.7.5; the UI edge only

11.9.2 Layered (n-tier)

Presentation → application (use cases) → domain → persistence. A **layer** is a logical separation; a **tier** is a physical one (a separate process or machine). **Strict** layering calls only the layer directly below; **relaxed** layering may skip layers. Pros: familiar, frameworks impose it. Cons: the dependency points *down to the database*; "sinkhole" layers that only forward calls. **Interview:** the default machine-coding layout. Keep rules out of controllers.

11.9.3 Client-server (and peer-to-peer)

The server owns the resource and the rules; clients start requests over a protocol (HTTP, gRPC). Never trust client-only validation. **Stateless** servers (session data in a token or a shared store) scale out behind a load balancer. **Peer-to-peer:** every node is both client and server (BitTorrent, blockchains). No central bottleneck, but consistency, discovery and trust are harder. **Interview:** state which side owns the truth, e.g. "the server re-checks seat availability under a lock".

11.9.4 Monolith, modular monolith and microservices

- **Monolith:** one deployable unit.
- **Modular monolith:** one deployable, split by business capability into modules that expose small interfaces (enforced with packages, JPMS or build rules).
- **Microservices:** independently deployable processes, each owning its own database, usually one per **bounded context** (a part of the business with one consistent vocabulary).

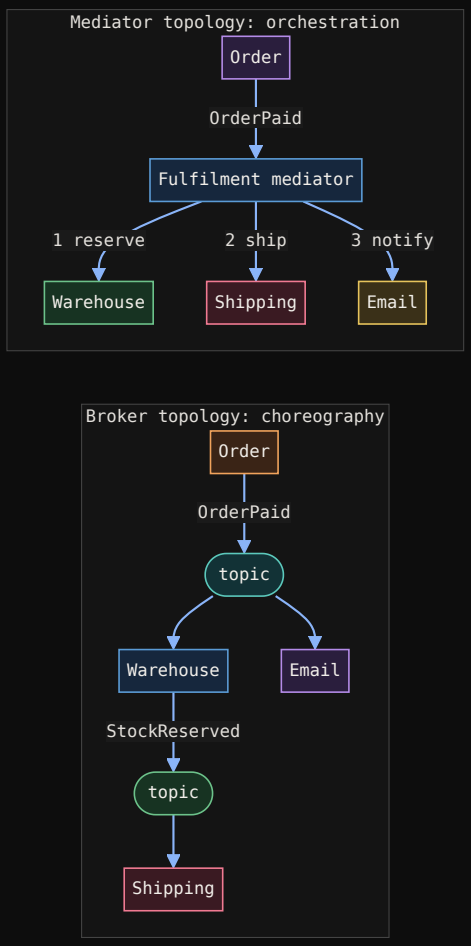
Start modular; extract a service only for a concrete reason (team independence, scaling, technology, fault isolation). Microservice costs: timeouts and retries, **idempotency** (a repeated request has no extra effect), **sagas** (local transactions with compensating steps instead of cross-service transactions), eventual consistency, ops. The worst outcome is a **distributed monolith**: services that must be deployed together. **Interview:** "each port, like `PaymentGateway`, can become a network client with the same interface."

11.9.5 Event-driven: broker vs mediator topology

Producers publish immutable events and don't know their consumers (Observer at system level; Ch 20 §20.6 builds a pub-sub system).

- **Broker topology (choreography):** consumers subscribe and may emit further events. Decoupled and scalable, but no one place sees the whole flow.
- **Mediator topology (orchestration):** a central mediator sends ordered commands and handles compensation. Visible and controllable, but coupled to every step (Mediator, §10.4, at system level).

Use a broker for independent reactions; a mediator for ordered multi-step processes. Costs: at-least-once delivery (so consumers must be idempotent), ordering only per key, eventual consistency, dead-letter queues for messages that keep failing.

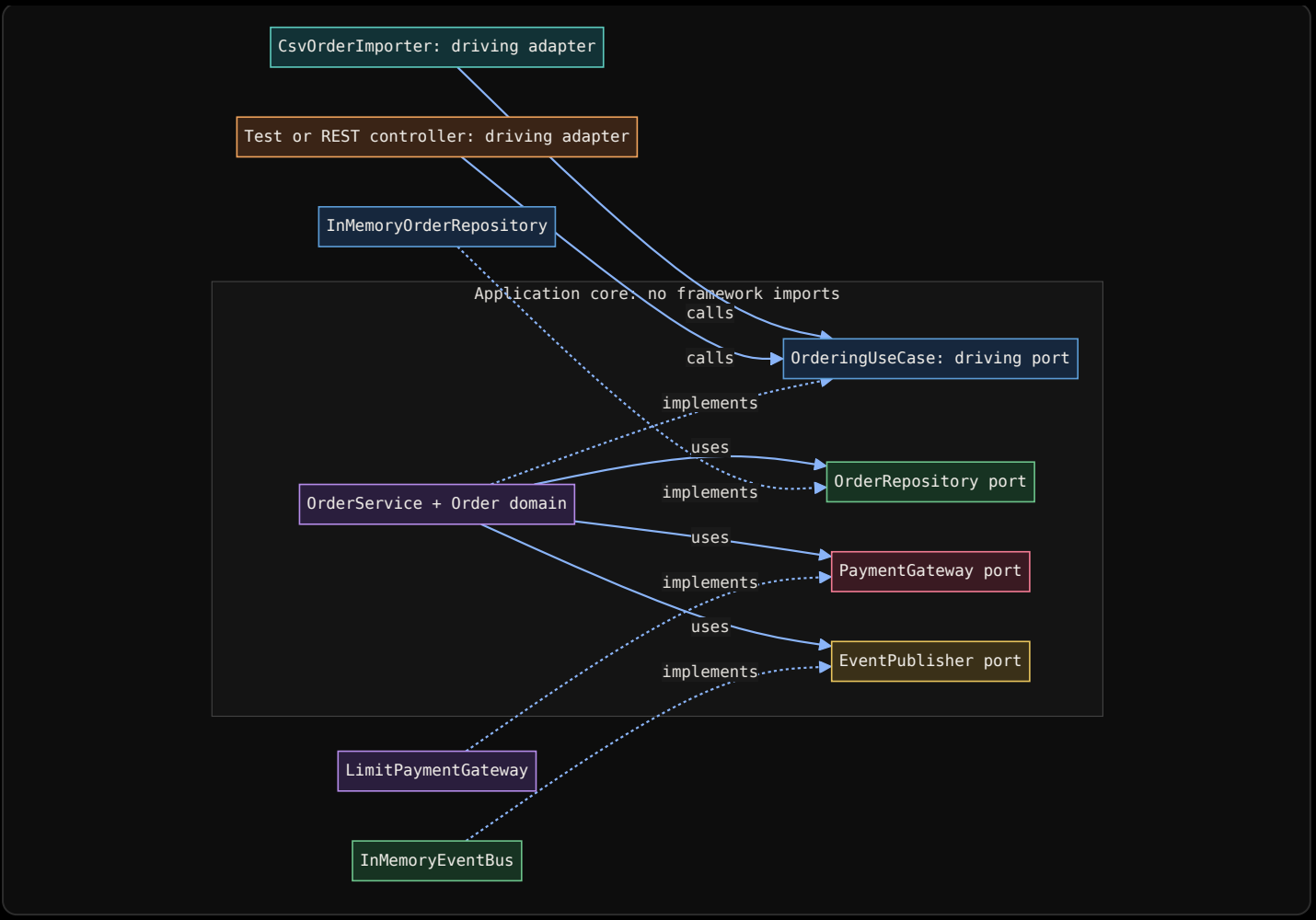


11.9.6 Pipe-and-filter

Independent **filters** transform, enrich, validate or drop items. **Pipes** connect them: function composition, or bounded queues between threads (with **backpressure**: a full queue slows the producer). Examples: Unix pipes, Java Streams, compiler phases, servlet filters. Use it for parsing, ETL, logs and media. Versus **Chain of Responsibility**: in a CoR any handler may stop the request; in a pipeline every filter runs and transforms. Versus **Decorator**: filters may change the data type at each step.

11.9.7 Hexagonal (ports and adapters) and clean architecture

The **core** (domain plus use cases) owns **ports** (interfaces). A **driving port** is a use case called by driving adapters (REST, CLI, importer, test). A **driven port** is a need (repository, payment, publisher) implemented by driven adapters. All dependencies point **inward** (DIP, Ch 4 §4.10, at system scale), and a **composition root** wires adapters to the core. **Clean** (Martin) and **onion** (Palermo) architectures are the same dependency rule drawn as circles. Versus layered: persistence now depends *in* on the domain's port. Costs: more interfaces and mapping; little gain for trivial CRUD.



11.9.8 Where MVC, MVP and MVVM fit

These are **presentation patterns** (§11.7.5). They organise only the UI edge. A controller, presenter or view-model is a **driving adapter** of the core, so MVC and hexagonal complement each other.

11.9.9 Java example

Packages `domain`, `port`, `app`, `adapter`, `pipeline` under `lld.ch11.architecture`. `CsvOrderImporter` (driving adapter) parses lines through four filters composed with `then()`. `OrderService` implements `OrderingUseCase` using only ports. `InMemoryEventBus` is a broker-style `EventPublisher`.

```java
// File: code/lld/ch11/architecture/pipeline/Filter.java
    default <R> Filter<I, R> then(Filter<O, R> next) {
        return input -> switch (process(input)) {
            case Result.Ok<O> ok -> next.process(ok.value());
            case Result.Rejected<O> rejected -> new Result.Rejected<>(rejected.reason());
        };
    }
```

```java
// File: code/lld/ch11/architecture/app/OrderService.java  (place)
        Order order = Order.place("o-" + nextId.getAndIncrement(), cmd.customerId(), lines);
        boolean paid = payments.charge(order.customerId(), order.total());
        if (paid) order.markPaid(); else order.markPaymentFailed();
        orders.save(order);
        events.publish(paid
            ? new OrderEvent.OrderPaid(order.id(), order.customerId(), order.total())
            : new OrderEvent.PaymentFailed(order.id(), order.customerId(), order.total()));
```

What the run shows:

- `pen:zero` is dropped by a filter (syntax, an adapter concern).
- `kettle` is rejected by the core (unknown product, a business rule).

- A paid order fans out to warehouse, email and audit handlers.
- Swapping the payment adapter for `(customer, amount) -> false` needs no change to the core.

Production gaps: `place` is not atomic (save `PENDING_PAYMENT` first, use an idempotency key, publish through an **outbox**); the bus is synchronous.

Full runnable code: code/lld/ch11/architecture/ (see the Full Edition).

In the interview (architectural patterns)

30-second answer: "A modular monolith with a hexagonal core: no framework imports in the domain; repositories, payment and notifications behind ports with in-memory adapters; side effects driven by domain events on a bus. Module boundaries are the future service boundaries." Choreography for independent reactions, orchestration for ordered processes with compensation. Don't draw microservices for a 90-minute machine-coding problem.

Self-check (architectural patterns)

1. **Q:** Layer vs tier? **A:** Logical vs physical separation.
2. **Q:** Who owns `OrderRepository` in hexagonal? **A:** The core; an adapter implements it.
3. **Q:** Credit check → fraud check → approval, with undo on rejection? **A:** Mediator topology (orchestration).
4. **Q:** MVC vs hexagonal? **A:** Complementary: the MVC controller is a driving adapter.

TL;DR

- Design patterns arrange classes; architectural patterns arrange systems.
- Layered: simple, but the domain depends on storage. Hexagonal flips that with ports and adapters.
- Modular monolith first. Event-driven: broker (choreography) or mediator (orchestration).
- Pipe-and-filter: every step runs and transforms.

Chapter 12 — Concurrency & Multithreading for LLD (Java) · Study Edition

> **Study edition.** Every topic of the full chapter, with the padding removed. For complete code listings, full outputs and extra follow-ups, see the Full Edition (`README.md`).
>
> **Why this chapter exists.** In an LLD interview you will hear: *"Two users click **Book** on the same seat at the same moment. What happens?"* A design with no answer is incomplete. This chapter teaches the ideas and Java 21 tools to answer it; Chapter 13 applies them to classic problems, and chapters 14–21 use them in case studies.

Every concept follows: *What it is → The problem it solves → How it works → Java example → Trade-offs → When not to use it → Common mistakes → Real-world examples → In the interview → Self-check → TL;DR.* Code lives in `code/lld/ch12/<topic>/Main.java`.

```
javac --release 21 -d /tmp/lld_12 $(find . -name '*.java')
java -cp /tmp/lld_12 lld.ch12.threads.Main
```

Part	Sections	Code package
A. Foundations	12.1–12.4	`threads`
B. What goes wrong	12.5 Races · 12.6 Java Memory Model	`race`
C. Locks	12.7–12.11	`locks`
D. Coordination	12.12 Semaphores · 12.13 Conditions · 12.14 Latch & barrier	`coordination`
E. Lock-free	12.15 CAS & atomics · 12.16 Concurrent collections	`atomics`, `collections`
F. Liveness failures	12.17 Deadlock · 12.18 Livelock · 12.19 Starvation	`deadlock`
G. Patterns	12.20–12.24	`patterns`
H. Interviews	12.25 Parking, seat booking, inventory	`lldcases`

Term	Meaning in one line
Thread	One independent sequence of instructions inside a program.
Shared state	Data more than one thread can read or write.
Thread-safe	Correct under any interleaving, with no extra locking by callers.
Atomic	One indivisible step: others see "before" or "after", never "halfway".
Lock / mutex	A token only one thread can hold at a time.
Critical section	Code touching shared state that must not run in two threads at once.
Contention	Many threads wanting the same lock at once.
Blocking	A thread waits (lock, I/O, signal) instead of using the CPU.
Liveness	"Something good eventually happens" — no deadlock, livelock, starvation.
Safety	"Nothing bad ever happens" — no corrupted data, no double booking.

Part A — Foundations

12.1 Concurrency vs parallelism

What it is

Concurrency = *dealing with* many things at once (tasks with overlapping lifetimes, maybe taking turns on one core). **Parallelism** = *doing* many things at the same instant on different cores.

The problem it solves

Programs wait a lot (disk, network, DB, users): concurrency does useful work meanwhile (responsiveness, throughput). Parallelism finishes CPU-heavy work sooner using all cores (speed-up).

How it works

The OS scheduler gives each runnable thread a time slice; when it ends or the thread blocks, a **context switch** saves one thread's registers and loads another's. On one core that is the *illusion* of simultaneity; on many cores threads really run at once.

	Concurrency	Parallelism
Core idea	Structure: many tasks in progress	Execution: many at the same instant
Needs multiple cores?	No	Yes
Main benefit	Overlapping waiting (I/O-bound)	Speed on CPU-bound work
Java tools	Threads, executors, virtual threads, `CompletableFuture`	Fork/Join, parallel streams, `Arrays.parallelSort`
Example	Web server with 10,000 connections	Sorting 100 million numbers

Java example

12.4's `Main` runs 10,000 sleeping tasks on virtual threads in well under a second (concurrency); Chapter 13's merge sort uses Fork/Join (parallelism).

Trade-offs

Concurrency adds complexity and context-switch overhead. **Amdahl's law:** with sequential fraction s, speed-up on N cores $\leq 1 / (s + (1 - s)/N)$ — 10% sequential caps it at 10×.

When not to use it

Small, fast CPU-light tasks (splitting costs more than it saves). Single-threaded designs are valid when they remove all locking (Redis's main thread, Node.js's event loop).

Common mistakes

Mixing up the two words; expecting more threads to always be faster (past the core count, CPU-bound work slows down).

Real-world examples

Concurrency: Tomcat/Jetty thread pools, Netty/Node.js event loops. Parallelism: `Arrays.parallelSort`, parallel streams, Spark/Hadoop.

In the interview

> **30-second answer:** "Concurrency is about structure — several tasks in progress, possibly interleaved on one core. Parallelism is about execution — tasks running at the same instant on several cores. Concurrency helps I/O-bound work; parallelism helps CPU-bound work."

Self-check

- **Q:** 50,000 idle chat connections — concurrency or parallelism? **A:** Concurrency: they mostly wait.

TL;DR

Concurrency = structure; parallelism = simultaneous execution. I/O-bound → concurrency; CPU-bound → parallelism.

12.2 Processes vs threads

What it is

A **process** is a running program with its **own memory**. A **thread** runs *inside* a process, **sharing the heap** but owning its **stack** and program counter.

The problem it solves

Threads share data cheaply within one program; processes give strong isolation (one crash doesn't corrupt others).

How it works

The OS gives a process a private address space and one thread (`main`); more threads share heap objects with separate stacks. Threads communicate through shared objects (with synchronization); processes via pipes, sockets, shared memory, message queues.

	Process	Thread
Memory	Private address space	Shared heap; own stack
Creation cost	High	Lower (platform thread ≈ OS thread, ~1 MB stack reserved); virtual threads much cheaper
Communication	IPC	Shared objects — fast, needs sync
Failure isolation	Strong	Weak (e.g. `OutOfMemoryError` kills all)
Context switch	Heavier	Lighter
Java	`ProcessBuilder`	`Thread`, executors

Java example

```java
// A new PROCESS (separate memory):
Process p = new ProcessBuilder("ls", "-l").inheritIO().start();
p.waitFor();

// A new THREAD (shares this JVM's heap):
Thread t = new Thread(() -> System.out.println("same heap, different stack"));
t.start();
t.join();
```

Trade-offs

Threads: fast sharing, but every shared mutable field is a potential bug. Processes: safe isolation, slower serialized communication.

When not to use it

When isolation matters more than speed (untrusted plugins, crash-prone native code) — use processes or containers.

Common mistakes

Thinking locals are shared (each thread has its own stack); one unbounded platform thread per request (use a pool or virtual threads).

Real-world examples

Chrome tabs as processes; PostgreSQL process-per-connection vs MySQL thread-per-connection.

In the interview

> **30-second answer:** "A process is an isolated program with its own memory; a thread is a path of execution inside a process that shares its heap but has its own stack. Threads are cheaper and share data easily but need synchronization; processes are isolated and safer but communicate through IPC."

Self-check

- **Q:** Two threads increment their own local `int x`. Race? **A:** No — locals live on each stack.

TL;DR

Process = isolated house; thread = worker sharing the heap, owning only its stack. Share cheaply, synchronize carefully.

12.3 Thread lifecycle & states

What it is

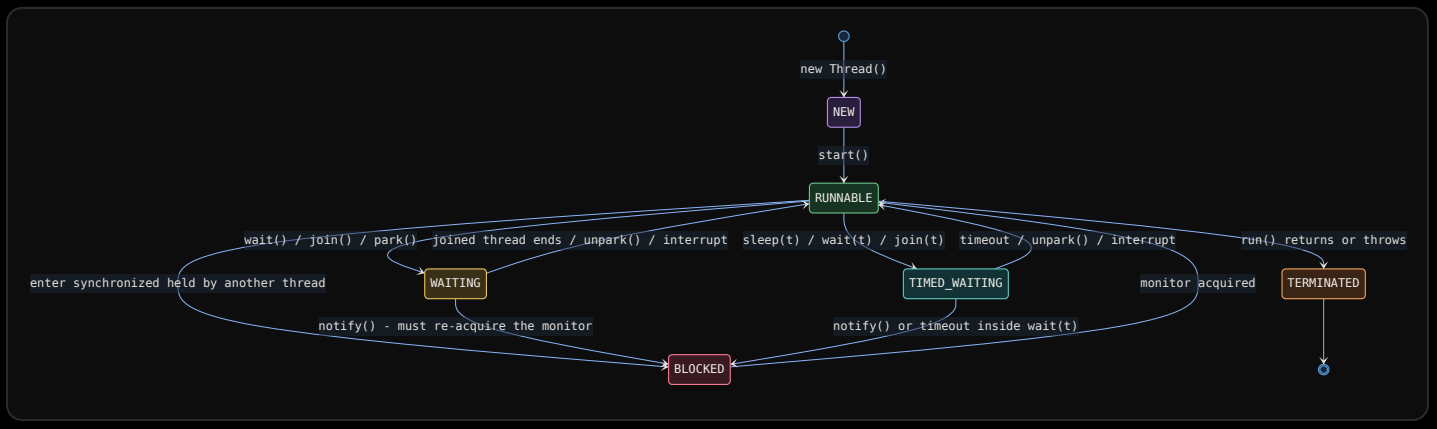
`Thread.getState()` returns one of six `Thread.State` values:

State	Meaning
`NEW`	Created, `start()` not called.
`RUNNABLE`	Running or ready (Java doesn't separate them; most blocking I/O also shows here).
`BLOCKED`	Waiting to *enter* a `synchronized` block held by another thread.
`WAITING`	Waiting indefinitely for another thread: `wait()`, `join()`, `LockSupport.park()` (used by `ReentrantLock`, `Condition`, `CountDownLatch`).
`TIMED_WAITING`	Same with a timeout: `sleep`, `wait(ms)`, `join(ms)`, `parkNanos`, `tryLock(timeout)`.
`TERMINATED`	`run()` returned or threw; cannot be restarted.

The problem it solves

Reading thread dumps (`jstack`, `jcmd <pid> Thread.print`): many `BLOCKED` on one monitor → contention or deadlock; many `WAITING` on a queue → idle pool.

How it works



Calling `start()` twice throws `IllegalThreadStateException`. A thread woken from `wait()` must re-acquire the monitor, so it may pass through `BLOCKED`. **Interruption** is cooperative: `t.interrupt()` sets a flag; `sleep` / `wait` / `join` throw `InterruptedException` and clear it. `Thread.stop()` is unsafe and no longer works.

Java example

`threadStates()` in 12.4's `Main` drives one thread into each state and prints it.

Trade-offs

A model, not a choice — but `RUNNABLE` hides whether a thread is really on a CPU or stuck in native I/O.

When not to use it

Never use `getState()` for program logic (it changes between check and act); monitoring only.

Common mistakes

Calling `run()` instead of `start()` (runs on the current thread); swallowing `InterruptedException` (rethrow or `Thread.currentThread().interrupt()`); confusing `BLOCKED` (monitor) with `WAITING` (signal).

Real-world examples

A `jstack` showing 200 Tomcat workers `BLOCKED` on one `synchronized` logger or pool method = coarse-lock bottleneck.

In the interview

> **30-second answer:** "Java threads have six states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. BLOCKED means waiting for a monitor lock; WAITING/TIMED_WAITING mean waiting for another thread's action, optionally with a timeout."

Self-check

- **Q:** Inside `Thread.sleep(1000)`? **A:** `TIMED_WAITING`.

TL;DR

NEW → RUNNABLE ↔ (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED. BLOCKED = monitor; WAITING = signal. `start()`, never `run()`; respect interrupts.

12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

What it is

Way	What you write	Result?	Use when
Extend `Thread`	`class T extends Thread { run() }`	No	Almost never — mixes task and worker
`Runnable`	`() -> { ... }`	No	Fire-and-forget task
`Callable<V>`	`() -> value` (may throw checked)	Yes, via `Future<V>`	Need a result or exception back
`ExecutorService`	`pool.submit(task)`	`Future<V>`	Default: reuse threads, limit concurrency
Virtual threads (Java 21)	`Thread.ofVirtual().start(r)` / `newVirtualThreadPerTaskExecutor()`	via `Future`	Very many *blocking* tasks

The problem it solves

An OS thread per task is expensive and unbounded. Executors separate **what** runs from **how** (pooling, queueing). Virtual threads make "one thread per request" cheap again for blocking code.

How it works

Write a `Runnable`/`Callable`, `submit` it, get a `Future`; `get()` blocks (task exceptions come wrapped in `ExecutionException`); `shutdown()` + `awaitTermination`, or try-with-resources (`close()` waits, Java 19+). **Virtual threads** (JEP 444) run on a small `ForkJoinPool` of carrier threads; blocking I/O, `sleep` or a `java.util.concurrent` lock **unmounts** them. **Pinning** keeps the carrier busy: on Java 21–23 when blocking inside `synchronized` (including `wait()`) or a native method; JDK 24 (JEP 491) removed the `synchronized` case.

Java example

```
// File: code/lld/ch12/threads/Main.java
            Future<Integer> future = pool.submit(sumTask);   // returns immediately
            System.out.println("3) Callable+Future  -> sum 1..100 = " + future.get()); // get() blocks until done
        // ...
        try (ExecutorService vexec = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<Integer>> fs = new ArrayList<>();
            for (int i = 0; i < 10_000; i++) {
                fs.add(vexec.submit(() -> { Thread.sleep(50); return 1; }));
            }
            // ... sum the futures
        } // close() waits for all tasks
```

The 10,000 sleeping tasks finish far under 500 s; the file also tours every `Thread.State`. `t.join()` creates a happens-before edge (12.6). Full runnable code: code/lld/ch12/threads/ (see the Full Edition).

Trade-offs

	Platform thread pool	Virtual threads
Cost per thread	High (OS thread + big stack)	Very low (growable heap stack)
How many	Hundreds–few thousand	Millions
Best for	CPU-bound (size ≈ cores)	Blocking I/O
Limits concurrency?	Yes, pool size	No — add a `Semaphore`
Pooling	Yes	Never pool them

When not to use it

Virtual threads for CPU-heavy work (no extra cores); blocking in `synchronized` on Java 21–23 (pinning — prefer `ReentrantLock`); `newCachedThreadPool()` for unbounded traffic.

Common mistakes

Forgetting `shutdown()` (JVM stays alive); ignoring a `Future` (exceptions silently lost); `newFixedThreadPool` for untrusted load — its queue is **unbounded** (use a bounded `ThreadPoolExecutor`, 12.21).

Real-world examples

`Executors` factory methods (Factory pattern); Spring `@Async` on a `ThreadPoolTaskExecutor`; Spring Boot 3.2+ `spring.threads.virtual.enabled=true`.

In the interview

> **30-second answer:** "I never manage raw threads in business code. I describe work as `Runnable` / `Callable` and submit it to an `ExecutorService` sized for the workload: about cores for CPU-bound work, bounded queue with a rejection policy for incoming traffic. For many blocking I/O tasks on Java 21, I use virtual threads — one per task — and protect scarce resources with a semaphore."

Self-check

- **Q:** What does `thread.run()` do? **A:** Runs synchronously on the current thread.

TL;DR

Tasks = `Runnable` / `Callable`; run with an `ExecutorService`; results via `Future`. Virtual threads make one-thread-per-blocking-task cheap. Always shut pools down.

Part B — What goes wrong

12.5 Race conditions & critical sections

What it is

A **race condition**: the result depends on unlucky thread timing. A **critical section**: code touching shared mutable state that must run in one thread at a time. A **data race** (JMM term): two threads access one variable, at least one writes, with no happens-before order.

The problem it solves

Tells you *which* code needs protection and *why*:

Shape	Example	Why it breaks
Read-modify-write	`count++`	Really read, add, write — two threads read the same old value; one update is lost.
Check-then-act	`if (seat.isFree()) seat.book(me);`	The condition can change between check and act.

How it works

1. Find all **shared mutable state**. 2. List its **invariants** ("a seat has at most one owner"). 3. Mark read-then-update code as **critical sections**. 4. Make each **atomic** — a lock, one atomic operation (`incrementAndGet`, `putIfAbsent`), or no sharing (immutability, confinement).

Java example

```
// File: code/lld/ch12/race/Main.java
    /** BROKEN: count++ is read-modify-write (3 steps), so concurrent increments get lost. */
    static class UnsafeCounter implements Counter {
        private int count;
        public void increment() { count++; }
        public int get() { return count; }
    }

    // ... Fix 1: SynchronizedCounter — synchronized increment() AND get() (reads need the lock for visibility)

    /** Fix 2: a lock-free atomic built on compare-and-swap. */
    static class AtomicCounter implements Counter {
        private final AtomicInteger count = new AtomicInteger();
        public void increment() { count.incrementAndGet(); }
        public int get() { return count.get(); }
    }
    // ... check-then-act fixed with one atomic step: seatOwner.compareAndSet(0, id)
```

Output: of 1,000,000 increments the unsafe counter lost e.g. `680044` (varies every run — that unpredictability *is* the race); both fixes give exactly 1000000.

Full runnable code: code/lld/ch12/race/ (see the Full Edition).

Trade-offs

Fix	Pros	Cons
Lock	Protects any multi-step, multi-variable invariant	Waiting; deadlock risk if nested
Atomic variable / map method	Fast, non-blocking	One variable or one map entry only
Immutability / confinement	No synchronization	Needs redesign

When not to use it

Don't lock data that is never shared (locals, confined objects) or never changed after safe construction.

Common mistakes

Protecting writes but not reads; combining two thread-safe calls and assuming atomicity (`containsKey` then `put` on a `ConcurrentHashMap` — use `putIfAbsent`/`computeIfAbsent`); locking on different objects.

Real-world examples

Double-booked tickets, oversold flash-sale stock, double payments on a double-click — check-then-act races, fixed with atomics, locks or DB constraints (unique index, `SELECT ... FOR UPDATE`, version columns).

In the interview

> **30-second answer:** "A race condition is when correctness depends on thread timing. The two common shapes are read-modify-write, like `count++`, and check-then-act, like 'if seat is free then book it'. I find the shared mutable state and its invariants, and make each critical section atomic — with a lock, an atomic operation like `putIfAbsent` or `compareAndSet`, or by removing sharing."

Self-check

- **Q:** Why does `count++` lose updates? **A:** Two threads read the same value before either writes.

TL;DR

Race = result depends on timing. Watch read-modify-write and check-then-act. Make every critical section atomic, including reads.

12.6 Java Memory Model basics: visibility, happens-before, `volatile`

What it is

The **JMM** (JLS ch. 17) defines when one thread's write is guaranteed visible to another's read. **Visibility**: will B see A's write? **Reordering**: compilers, JIT and CPUs reorder when a single thread can't tell. **Happens-before (HB)**: if X HB Y, X's effects are visible to and ordered before Y. `volatile`: reads/writes of that field are visible to all threads and ordered.

The problem it solves

`while (running) iterations++;` with a non-volatile `running` may never stop — the JIT may read it once and loop forever.

How it works

Rule	Meaning
Program order	Within a thread, each action HB the next.
Monitor lock	Unlock HB every later lock of the *same* monitor.
Volatile	A volatile write HB every later read of that field.
Thread start	`t.start()` HB everything in `t`.
Thread join	Everything in `t` HB a return from `t.join()`.
Transitivity	A HB B and B HB C ⇒ A HB C.
Final fields	Separate guarantee (JLS §17.5): after a constructor that doesn't leak `this`, every thread sees correct `final` field values, even without sync.
`java.util.concurrent`	Queue put HB take; `countDown()` HB `await()` return; `Future` completion HB `get()`.

Apply: is every write-then-read pair linked by HB? Same lock → yes. Single independent flag/reference → `volatile`. Compound update → lock or atomic.

Safe publication: share a reference via a `volatile`/`AtomicReference` field, a `final` field, a static initializer, a concurrent collection, or a lock the reader also takes. **Double-checked locking** needs `volatile`:

```java
class Config {
    private static volatile Config instance;          // volatile is essential
    static Config get() {
        Config c = instance;
        if (c == null) {
            synchronized (Config.class) {
                c = instance;
                if (c == null) instance = c = new Config();
            }
        }
        return c;
    }
}
```

(Simpler: the **holder idiom** — `private static class Holder { static final Config I = new Config(); }`.)

Java example

`Worker` in 12.5 uses `private volatile boolean running`; the output `volatile flag stopped worker: true` confirms the write became visible.

Trade-offs

Tool	Visibility	Atomic compound ops	Blocking	Cost
`volatile`	Yes	No	No	Low
`synchronized` / `Lock`	Yes (same lock)	Yes	Yes	Medium
`Atomic*`	Yes	One variable	No	Low–medium
`final`	Yes, after construction	N/A	No	Free

When not to use it

`volatile` for anything combining several steps or variables.

Common mistakes

"Works on my laptop" — x86 reorders less than ARM (Apple Silicon, Graviton); leaking `this` from a constructor; synchronizing only the writer.

Real-world examples

`volatile` shutdown flags; `ConcurrentHashMap`, `AtomicInteger`, `CopyOnWriteArrayList` use volatile + CAS internally; immutable `String`.

In the interview

> **30-second answer:** "The Java Memory Model says a thread is only guaranteed to see another thread's write if there's a happens-before edge — unlock-then-lock of the same monitor, a volatile write-then-read, thread start/join, or a concurrent-collection handoff. `volatile` gives visibility and ordering for a single variable but not atomicity; for compound actions I use locks or atomics."

Self-check

- **Q:** Non-volatile `ready = true` after preparing data; B spins on it. Two bugs? **A:** B may never see it, or see it with stale data.

TL;DR

Cross-thread visibility only via happens-before. `volatile` = visibility + ordering, not atomicity. Locks = both. Publish objects safely.

Part C — Locks

12.7 Mutex (mutual exclusion) and `synchronized`

What it is

A **mutex** is a lock at most one thread can hold. Every Java object has a built-in one — its **intrinsic lock** or **monitor** — taken by `synchronized` (instance method → `this`; static method → the `Class` object; block → the given object).

The problem it solves

Makes a critical section **atomic** and **visible** (the next lock holder sees everything done before the unlock).

How it works

Free monitor → take it; held → `BLOCKED` until free; released automatically on exit, **even by exception**; **reentrant** (the holder can re-enter).

Java example

`SynchronizedCounter` (12.5). Prefer a **private final** lock object so outside code can't take your lock:

```
private final Object lock = new Object();
void add(Item i) { synchronized (lock) { items.add(i); } }
```

Trade-offs

Pros	Cons
Simplest; auto-release; reentrant; cheap when uncontended	No timeout, `tryLock` or interruptible wait; one wait-set per object; no fairness; pins virtual threads on Java 21–23

When not to use it

When you need a timeout, interruptibility, fairness, several conditions, or read/write separation — use `ReentrantLock` / `ReentrantReadWriteLock`.

Common mistakes

Synchronizing on a *changing* reference; on string literals or boxed values (shared JVM-wide); doing slow I/O while holding the lock.

Real-world examples

`Vector`, `Hashtable`, `StringBuffer`, `Collections.synchronizedList` — safe per call but slow under contention, hence replaced by unsynchronized classes plus concurrent collections.

In the interview

> **30-second answer:** "A mutex lets one thread at a time into a critical section. In Java, `synchronized` uses the object's monitor: it's reentrant, released automatically, and gives visibility. I lock on a private final object and keep the section short."

Self-check

- **Q:** Synchronized methods on *different* objects block each other? **A:** No — different monitors.

TL;DR

One holder at a time; `synchronized` is reentrant and auto-releasing. Private final lock object; short sections.

12.8 `ReentrantLock` and reentrancy

What it is

`ReentrantLock` is an explicit mutex with `synchronized`'s meaning plus extras. **Reentrant**: the holder can acquire it again (hold count rises) and must release it as many times.

The problem it solves

Without reentrancy, a locked method calling another locked method on the same object would **deadlock with itself**. Extras over `synchronized`: `tryLock()` / `tryLock(timeout)`, `lockInterruptibly()`, optional **fairness**, multiple `Condition`s, `isLocked()` / `getHoldCount()`.

How it works

`lock()` takes it (count 0 → 1) or waits; the same thread locking again → count 2; each `unlock()` decrements, releasing at 0. **Always** `lock(); try { ... } finally { unlock(); }`.

Java example

```java
// File: code/lld/ch12/locks/Main.java
        void withdraw(int amount) {
            lock.lock();
            try {
                if (canWithdraw(amount)) balance -= amount;   // calls another locked method
            } finally {
                lock.unlock();                                // ALWAYS unlock in finally
            }
        }
        boolean canWithdraw(int amount) {
            lock.lock();                                      // same thread re-acquires: no self-deadlock
            try {
                System.out.println("hold count inside nested call = " + lock.getHoldCount());
                return balance >= amount;
            } finally {
                lock.unlock();
            }
        }
```

Output: `hold count inside nested call = 2`, `balance = 70`. The same file also demos `tryLock` (12.9), `ReadWriteLock` and `StampedLock` (12.10). Full runnable code: code/lld/ch12/locks/ (see the Full Edition).

Trade-offs

Feature	`synchronized`	`ReentrantLock`
Release	Automatic	Manual (`finally`)
Try / timeout	No	`tryLock()`, `tryLock(t, unit)`
Interruptible wait	No	`lockInterruptibly()`
Fairness	No	Optional (slower)
Conditions	One wait set	Many
Reentrant	Yes	Yes

When not to use it

If you need none of the extras, `synchronized` is shorter and can't be left locked.

Common mistakes

`lock()` *inside* `try` (a failed lock would `unlock()` → `IllegalMonitorStateException`); missing `unlock()` on an early return; fairness "to be safe" (much slower).

Real-world examples

`ArrayBlockingQueue`, `LinkedBlockingQueue`, `ThreadPoolExecutor` use it; `ReentrantLock`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch` are built on **AQS**; `CyclicBarrier` on `ReentrantLock` + `Condition`.

In the interview

> **30-second answer:** "`ReentrantLock` is an explicit mutex with the same semantics as `synchronized` plus try-lock with timeout, interruptible acquisition, optional fairness, and multiple conditions. Reentrant means the owner can re-acquire it; the hold count must drop to zero to release. I always unlock in `finally`."

Self-check

- **Q:** `lock()` ×3, `unlock()` ×2 — free? **A:** No; hold count 1.

TL;DR

`synchronized` + tryLock/timeout/interruptible/fairness/conditions. `lock(); try { … } finally { unlock(); }` always.

12.9 Try-lock & timed locking

What it is

`tryLock()` takes the lock only if free *now*; `tryLock(timeout, unit)` waits up to the timeout (interruptibly), returning `false` on failure.

The problem it solves

Plain `lock()` waits forever (hung requests, permanent deadlocks). Try-lock lets a thread **give up** — fail fast, retry, return "busy" — and breaks deadlock's *hold-and-wait* condition (12.17).

How it works

`true` → `try { work } finally { unlock(); }`; `false` → fallback. With several locks: if any fails, **release all held**, back off **randomly**, retry.

Java example

In `locks/Main.java`, `printer.tryLock(100, TimeUnit.MILLISECONDS)` fails while `main` holds it and prints `worker: printer busy for 100 ms, doing something else instead`. `transferTryLock` in 12.17 uses try-lock + random back-off.

Trade-offs

Pros	Cons
No infinite waits; breaks deadlock; predictable latency	Caller handles failure; retries without random back-off can **livelock**; timeout choice is a judgement call

When not to use it

When failing is unacceptable and waiting is fine (background batch job) — plain `lock()`.

Common mistakes

`unlock()` after a `false` → `IllegalMonitorStateException`; tight retry loops; expecting untimed `tryLock()` to be fair — it *barges*; `tryLock(0, unit)` respects fairness.

Real-world examples

Distributed locks (Redis `SET key value NX PX 30000`, Redisson `tryLock(wait, lease, unit)`, ZooKeeper); ticket seat holds that fail fast.

In the interview

> **30-second answer:** "`tryLock` with a timeout lets a thread give up instead of waiting forever. I use it when latency matters or when I need several locks: if I can't get all of them, I release what I hold, back off randomly and retry — which prevents deadlock."

Self-check

- **Q:** `tryLock(0, SECONDS)`? **A:** Tries once without waiting, but respects a fair queue.

TL;DR

Acquire only if available (within a timeout). Handle `false`, release everything on failure, back off randomly.

12.10 Read-write locks: `ReadWriteLock` and `StampedLock`

What it is

A shared **read lock** (many readers) plus an exclusive **write lock** (one writer, no readers): `ReentrantReadWriteLock`, and since Java 8 `StampedLock` with cheap *optimistic reads*.

The problem it solves

A mutex makes readers block each other; for **read-heavy** data (config, caches, catalogues) reads can run in parallel.

How it works

Readers share `readLock()` when no writer holds it; a writer waits for **no readers and no writer**. `ReentrantReadWriteLock` allows **downgrade** (write → read) but **not upgrade** (read → write deadlocks on itself). `StampedLock`: `tryOptimisticRead()` → copy fields to locals → `validate(stamp)`; on failure retry with `readLock()`.

Java example

`ConfigStore` and `Point` (optimistic read) in `locks/Main.java`; three readers inside the read lock at once print `3 readers held the read lock at the same time: true`.

Trade-offs

	Mutex	ReentrantReadWriteLock	StampedLock
Parallel readers	No	Yes	Yes (optimistic reads take no lock)
Reentrant	Yes	Yes	**No**
Conditions	Yes	Write lock only	No
Overhead	Lowest	Higher bookkeeping	Low for optimistic reads
Best when	Short or write-heavy	Reads ≫ writes, non-trivial reads	Very read-heavy, tiny sections

When not to use it

Write-heavy or tiny sections (bookkeeping makes it slower); when a **copy-on-write snapshot** is simpler (immutable map in a `volatile` field, replaced on write — 12.23).

Common mistakes

Upgrading read → write; writing under the read lock; using optimistic data before `validate` or without copying to locals.

Real-world examples

Caches, config holders, routing tables; `ConcurrentHashMap` goes further — lock-free reads.

In the interview

> **30-second answer:** "A read-write lock lets many readers in together but gives writers exclusive access. It helps when reads vastly outnumber writes. `ReentrantReadWriteLock` allows downgrade but not upgrade; `StampedLock` adds optimistic reads that take no lock and are validated afterwards, but it's not reentrant."

Self-check

- **Q:** Two readers and a writer arrive — who runs together? **A:** The two readers; the writer alone.

TL;DR

Many readers OR one writer, for read-heavy data. No upgrade; `StampedLock` = optimistic, non-reentrant.

12.11 Coarse-grained vs fine-grained locking

What it is

Coarse: one lock for a lot of state (the whole parking lot). **Fine**: many locks each guarding a small piece (per floor, spot, hash bucket).

The problem it solves

Balances **simplicity and safety** against **throughput under contention**.

How it works

Start coarse and measure; if contended, split **independent** state (**lock splitting**); for big collections use **lock striping** (lock `hash(i) % N`; Java 7 `ConcurrentHashMap` had 16 segments); take multiple fine locks in a **global order**; do slow work outside locks.

	Coarse	Fine
Simplicity	High	Lower
Concurrency	Low	High
Deadlock risk	Low	Higher — needs ordering
Overhead	One lock	Many locks
Cross-part invariants	Easy	Hard

Java example

Coarse: `SynchronizedCounter` (12.5). Fine: `Show` in 12.25 (one lock per seat, sorted order); `StripedHashMap` in Chapter 13.

Trade-offs

Correct first; go fine-grained only where profiling shows contention.

When not to use it

Don't split when an invariant spans the parts ("total booked ≤ capacity") unless you handle the cross-part check.

Common mistakes

Premature fine-graining; inconsistent lock order; fine locks that still funnel through one shared counter (use `LongAdder`).

Real-world examples

Java 8 `ConcurrentHashMap` locks one bin head; DB row vs table locks.

In the interview

> **30-second answer:** "I start with one lock for correctness, then refine where contention matters: per-floor or per-spot locks for a parking lot, per-show or per-seat locks for booking, striping for maps. Finer locks raise throughput but need a consistent lock order to avoid deadlock."

Self-check

- **Q:** Lock striping? **A:** N locks; each item maps to one by hash — fine-grained with a bounded lock count.

TL;DR

Coarse = simple, low throughput; fine = parallel, complex, deadlock-prone. Start coarse, measure, order locks.

Part D — Coordination

12.12 Semaphores

What it is

A **semaphore** holds **permits**: `acquire()` takes one (waiting if none), `release()` returns one. One permit ≈ a mutex, but with **no owner** — any thread may release.

The problem it solves

Limiting concurrency (≤ N DB connections, API calls) and **signaling** (0 permits = "wait until someone signals", used heavily in Chapter 13).

How it works

`new Semaphore(n)` is **non-fair** (callers may barge); `new Semaphore(n, true)` is FIFO (untimed `tryAcquire()` still barges; `tryAcquire(0, SECONDS)` doesn't). Also `tryAcquire(timeout)`, `acquire(k)`. Always `release()` in `finally` after a successful `acquire()`.

Java example

```java
// File: code/lld/ch12/coordination/Main.java
        Semaphore connections = new Semaphore(3);
        // ... 10 tasks on a pool of 10 threads, each doing:
                try {
                    connections.acquire();                          // blocks when 0 permits left
                    try {
                        int now = inUse.incrementAndGet();
                        maxSeen.accumulateAndGet(now, Math::max);
                        Thread.sleep(30);                           // "run a query"
                        inUse.decrementAndGet();
                    } finally {
                        connections.release();                      // give the permit back
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
```

Output: `10 queries done, never more than 3 at once: true`. The same file demos wait/notify, `Condition`, latch and barrier (12.13–12.14). Full runnable code: code/lld/ch12/coordination/ (see the Full Edition).

Trade-offs

Pros	Cons
Simple concurrency cap; doubles as a signal	No owner — buggy extra releases grow permits; protects no data by itself

When not to use it

As a mutex around shared data when a lock would do (locks have ownership and reentrancy).

Common mistakes

Releasing when `acquire` never succeeded (e.g. interrupted) → permit leak upward; not releasing on exceptions → permits vanish, system stalls.

Real-world examples

Bulkheads (Resilience4j `SemaphoreBulkhead`, Hystrix semaphore isolation); HikariCP's max pool size.

In the interview

> **30-second answer:** "A semaphore is a permit counter: acquire takes a permit or waits, release returns one. I use it to cap concurrent access to a limited resource — a bulkhead — or with zero initial permits as a signal from one thread to another. Unlike a lock it has no owner."

Self-check

- **Q:** `new Semaphore(0).acquire()`? **A:** Waits until another thread calls `release()` — a signal.

TL;DR

N permits: cap concurrency or signal. No ownership — release carefully in `finally`.

12.13 Condition variables: `wait/notify` and `Condition`

What it is

A **condition variable** lets a thread *sleep until a condition on shared state is true*, releasing the lock meanwhile, and lets another thread *wake it* after changing that state. Built-in: `wait()` / `notify()` / `notifyAll()` inside `synchronized`; explicit: `lock.newCondition()` with `await()` / `signal()` / `signalAll()`.

The problem it solves

Waiting for a state change without **busy-waiting** and without missing the moment.

How it works

The canonical pattern — memorize it:

```
synchronized (lock) {                // or: lock.lock(); try {
    while (!conditionHolds()) {      // ALWAYS a while loop, never if
        lock.wait();                 //   cond.await();  releases the lock while waiting
    }
    doAction();                      // condition is true AND we hold the lock
    lock.notifyAll();                //   cond.signal()/signalAll() if we changed state others wait for
}                                    // } finally { lock.unlock(); }
```

`wait()` **atomically releases the lock** and sleeps; it **re-acquires** it before returning. Why `while`? **Spurious wakeups** are allowed; another thread may grab the state first; with `notifyAll` only one waiter can win. `notify()` wakes one arbitrary waiter — with producers *and* consumers on one monitor it can wake the wrong kind and everyone sleeps. Use `notifyAll()`, or separate `Condition`s (`notFull`, `notEmpty`) with targeted `signal()`.

Java example

`Mailbox` (monitor, `while (item != null) wait();`) and `TurnTable` (Lock + `Condition`) in `coordination/Main.java`. Output: `received in order: [m1, m2, m3]` and `PING PONG PING PONG PING PONG`.

Trade-offs

	`wait/notify`	`Lock` + `Condition`
Conditions per lock	One wait set	Many
Targeted wake-up	No — use `notifyAll`	Yes
Timed wait	`wait(ms)`	`await(t, unit)`, `awaitNanos`
Verbosity	Less	More

When not to use it

Usually `BlockingQueue`, `Semaphore`, `CountDownLatch` or `CompletableFuture` already solve it; hand-roll only for custom coordination.

Common mistakes

`if` instead of `while`; `wait`/`notify` without holding that monitor → `IllegalMonitorStateException`; changing state without notifying (**lost wakeup**); waiting on one object while synchronized on another.

Real-world examples

`ArrayBlockingQueue`: one `ReentrantLock`, two `Condition`s (`notEmpty`, `notFull`) — built by hand in Chapter 13.

In the interview

> **30-second answer:** "A condition variable lets a thread release a lock and sleep until another thread changes the state and signals it. I always wait in a while loop that re-checks the condition, because of spurious wakeups and competing threads. With one monitor I use notifyAll; with ReentrantLock I use separate Conditions and targeted signal."

Self-check

- **Q:** What does `wait()` do with the monitor? **A:** Releases it while sleeping; re-acquires before returning.

TL;DR

Release lock, sleep until signalled, re-acquire, re-check. Always `while`; `notifyAll` or separate `Condition`s.

12.14 `CountDownLatch` and `CyclicBarrier`

What it is

`CountDownLatch(n)`: a one-time gate — `await()` returns after *n* `countDown()` calls; can't be reset. **`CyclicBarrier(n, action)`**: a reusable meeting point — when the *n*th thread calls `await()`, the action runs once, all are released, and it resets.

The problem it solves

Coordinating phases: start after all services are up (latch); finish step 1 before step 2 (barrier); release all test threads at once (latch of 1 — used throughout this book to maximise contention).

How it works

	`CountDownLatch`	`CyclicBarrier`
Who waits	Anyone calling `await()`	The *n* parties themselves
Who counts	Anyone calling `countDown()`	Arrival at `await()`
Reusable	No	Yes
Extra	Reaching 0 is permanent	Action once per trip; a failed party **breaks** it (`BrokenBarrierException`; `reset()` repairs)
Happens-before	Before `countDown()` HB after `await()`	Before `await()` HB the action HB after release

`Phaser` (Java 7) generalises both with dynamic parties across many phases.

Java example

`latchDemo` / `barrierDemo` in `coordination/Main.java` print `all services ready: true (count now 0)` and `phase 1 complete…`, `phase 2 complete…`.

Trade-offs

Latch: simple, one-shot. Barrier: reusable but fragile — one failure breaks it for all.

When not to use it

To wait for N tasks and collect results, `invokeAll` or `CompletableFuture.allOf` is clearer.

Common mistakes

A worker throws before `countDown()` → waiting forever (put it in `finally`; use `await(timeout)`); fewer threads than barrier parties.

Real-world examples

Test harnesses releasing N threads together; start-up waiting for caches, DB pools and consumers.

In the interview

> **30-second answer:** " `CountDownLatch` is a one-shot gate that opens after N countDowns; `CyclicBarrier` is a reusable rendezvous where N threads wait for each other each phase, optionally running an action when all arrive."

Self-check

- **Q:** 3 workers syncing at the end of every simulation step? **A:** `CyclicBarrier(3)`.

TL;DR

Latch = one-shot "wait for N events"; barrier = reusable "wait for N threads". `countDown` in `finally`; use timeouts.

Part E — Lock-free

12.15 Compare-and-swap (CAS) & atomics

What it is

CAS is one CPU instruction (`LOCK CMPXCHG` on x86; LL/SC on ARM): "if the value is still `expected`, set it to `new` and succeed; else fail." Java exposes it via `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference`, `AtomicIntegerArray`, `LongAdder` and `VarHandle`.

The problem it solves

Atomic updates **without locks**: no blocking, no deadlock, usually faster under low–moderate contention; the basis of the JDK's concurrency utilities.

How it works

Retry loop: read → compute → `compareAndSet(current, next)` → on failure retry. Helpers: `incrementAndGet`, `updateAndGet(fn)`, `accumulateAndGet(x, fn)` (the function may run several times — keep it side-effect-free). **`LongAdder`** spreads contended updates over cells and sums them (fast hot counters; `sum()` isn't an atomic snapshot). **ABA problem:** A → B → A makes a stale CAS succeed; fix with a version — `AtomicStampedReference` / `AtomicMarkableReference` (GC prevents classic node-reuse ABA for fresh nodes in Java).

Java example

```java
// File: code/lld/ch12/atomics/Main.java
    static int casIncrement(AtomicInteger a) {
        while (true) {
            int current = a.get();                  // 1. read
            int next = current + 1;                 // 2. compute
            if (a.compareAndSet(current, next)) {   // 3. write only if nobody changed it meanwhile
                return next;
            }                                       // 4. someone won the race: retry
        }
    }
```

The file also has a lock-free **Treiber stack** (push/pop swing one `AtomicReference` head with CAS), `LongAdder`, and an ABA demo.

Output: counter `400000`, stack popped `40000`; after A→B→A a plain CAS **succeeds** (`true`) while the stamped CAS **fails** (`false`).

Full runnable code: code/lld/ch12/atomics/ (see the Full Edition).

Trade-offs

	Locks	CAS / atomics
Blocking	Yes	No (retry)
Deadlock	Possible	Impossible
Scope	Any number of variables	One variable per CAS
Heavy contention	Threads sleep	Wasted retries — use `LongAdder`/striping
Difficulty	Moderate	Hard beyond counters/flags/one reference

When not to use it

Multi-variable invariants (moving money between two accounts) — use a lock, or put all fields in one immutable object and CAS its reference (Chapter 13's token bucket).

Common mistakes

`get()` then `set()` (still a race); side effects in `updateAndGet` lambdas; `AtomicLong` for very hot metrics (use `LongAdder`).

Real-world examples

`ConcurrentHashMap` CAS into empty bins; `ConcurrentLinkedQueue` (Michael–Scott); AQS state; JPA `@Version` / `UPDATE ... WHERE version = ?` (CAS in the DB); LMAX Disruptor.

In the interview

> **30-second answer:** "CAS atomically updates a value only if it still equals what I read; otherwise I retry. Java's atomic classes wrap it, so counters, flags and single references can be updated without locks and without deadlock. It covers one variable at a time, can suffer ABA — fixed with a stamped reference — and under heavy contention I prefer `LongAdder`."

Self-check

- **Q:** A failed `compareAndSet` means? **A:** Someone changed the value since you read it; re-read and retry.

TL;DR

CAS = update only if unchanged, retried. Lock-free, deadlock-free, one variable. Watch ABA; `LongAdder` for hot counters.

12.16 Concurrent collections

What it is

Need	Use	Key behaviour
Map	`ConcurrentHashMap`	Lock-free reads; writes lock one bin; atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; **no nulls**; weakly consistent iterators
Sorted map/set	`ConcurrentSkipListMap` / `Set`	Lock-free, sorted, O(log n)
List read ≫ write	`CopyOnWriteArrayList` / `Set`	Writes copy the array; iteration uses a snapshot
Non-blocking queue	`ConcurrentLinkedQueue` / `Deque`	Lock-free; `poll()` returns null when empty
Blocking queue	`ArrayBlockingQueue`, `LinkedBlockingQueue`, `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue`, `LinkedTransferQueue`	`put` waits when full, `take` when empty

Two `ConcurrentHashMap` guarantees to quote: **`computeIfAbsent` / `compute` / `merge` are atomic** — the mapping function runs **exactly once if absent, else not at all** (other updates to that bin block meanwhile, so keep it short and don't touch the same map; `ConcurrentSkipListMap`'s default may call it more than once). **Iterators are weakly consistent** — never throw CME, may or may not show later updates; `size()` is an estimate.

The problem it solves

One lock around a `HashMap` serialises everything; concurrent collections allow parallel access plus **atomic compound operations**.

How it works

Pick by access pattern; replace check-then-act with one call: `putIfAbsent`, `merge(k, 1, Integer::sum)`, `computeIfAbsent(k, ...)`; hand off work with a `BlockingQueue` (12.22). Sequences of calls are still not atomic.

Java example

`collections/Main.java` counts words with `merge(w, 1, Integer::sum)` from 4 threads, creates per-user lists with `computeIfAbsent`, and claims a seat with `putIfAbsent`:

```
word counts (4 threads x 1000 reps): {apple=12000, banana=8000, cherry=4000}
first putIfAbsent returns : null
second putIfAbsent returns: bob  (seat already bob's)
```

Full runnable code: code/lld/ch12/collections/ (see the Full Edition).

Trade-offs

Collection	Great for	Bad for
`ConcurrentHashMap`	Shared maps, caches, counters	Multi-key atomicity; `size()` estimate
`CopyOnWriteArrayList`	Listeners, rarely changed config	Frequent writes (O(n) copy)
`Collections.synchronizedMap`	Legacy, low contention	High contention; manual iteration locking
`ArrayBlockingQueue`	Bounded hand-off, back-pressure	Very high throughput (one lock)
`LinkedBlockingQueue`	Higher throughput (two locks)	Default is **unbounded**

When not to use it

Thread-confined objects (plain collections are faster); multi-key transactions (one lock or a DB).

Common mistakes

Slow work or same-map calls inside `computeIfAbsent` (bin locked; can throw `IllegalStateException` or deadlock); `null` in a `ConcurrentHashMap` → NPE.

Real-world examples

Caffeine on `ConcurrentHashMap`; Guava `EventBus` subscribers in a `CopyOnWriteArraySet`.

In the interview

> **30-second answer:** "I use `ConcurrentHashMap` for shared maps and its atomic methods — `putIfAbsent`, `computeIfAbsent`, `merge` — instead of check-then-act. `CopyOnWriteArrayList` for read-mostly lists like listeners. `BlockingQueue`, preferably bounded, to hand work between threads."

Self-check

- **Q:** Atomic word count? **A:** `map.merge(word, 1, Integer::sum)`.

TL;DR

CHM + atomic methods; COW for read-mostly lists; bounded `BlockingQueue` for hand-offs. Single calls atomic, sequences not.

Part F — Liveness failures

12.17 Deadlock

What it is

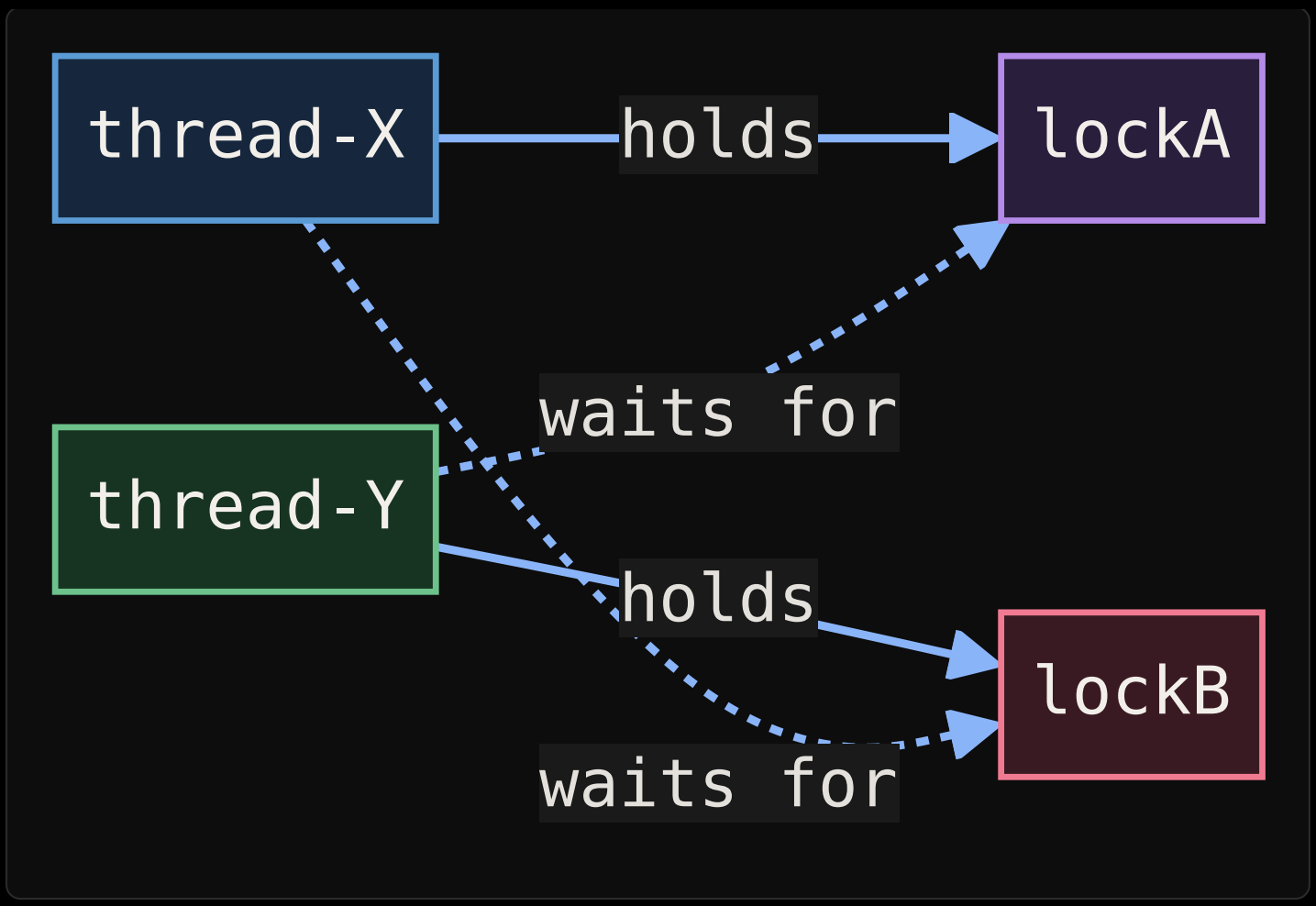
Two or more threads each wait forever for a resource another holds.

The problem it solves

Understanding it lets you *design it out*. All four **Coffman conditions** (1971) are required:

#	Condition	Meaning	How to break it
1	**Mutual exclusion**	One holder per resource	Lock-free/immutable data, shareable read locks
2	**Hold and wait**	Holding one while waiting for another	Acquire all at once, or `tryLock` and release on failure
3	**No preemption**	Only the holder releases	Timeouts, expiring leases
4	**Circular wait**	T1 waits for T2, T2 for T1	**Global lock ordering**

Break **any one** and deadlock is impossible.



How it works

1. **Lock ordering** (most common): a total order (account id, seat id, `identityHashCode` + tie-breaker lock).
2. **Try-lock**: `tryLock(timeout)`; on failure release all, random back-off, retry.
3. **Avoid nested locks**: call alien code (listeners) outside your lock.
4. **Detection**: `jstack` / `jcmd` print "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.
5. **Avoidance** (Banker's algorithm) — OS theory, rarely used in apps.

Java example

```
// File: code/lld/ch12/deadlock/Main.java
    static void transferOrdered(Account from, Account to, int amount) {
        Account first = from.id < to.id ? from : to;
        Account second = from.id < to.id ? to : from;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }
```

`transferTryLock` instead `tryLock`s both with 1 ms timeouts, releases on failure, and sleeps a random back-off.

`demonstrateDeadlock` forces opposite-order locking with a barrier and detects it:

```
deadlock detected between: [thread-X, thread-Y]  (states BLOCKED, BLOCKED)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

Full runnable code: code/lld/ch12/deadlock/ (see the Full Edition).

Trade-offs

Strategy	Pros	Cons
Lock ordering	Simple, zero cost	Needs a stable global order
Try-lock + back-off	No order needed; bounded waits	Retry logic; livelock without randomness
Single coarse lock	No deadlock among your locks	Lower throughput
Lock-free / immutable	No locks	Harder design

When not to use it

N/A — always prevent deadlock; prefer simple lock ordering to detection/recovery in app code.

Common mistakes

Calling listeners or another object's synchronized method under your lock; ordering by `hashCode()` without tie handling; **thread-starvation deadlock** — pool tasks that submit sub-tasks to the *same* pool and wait.

Real-world examples

InnoDB/PostgreSQL detect deadlocks and abort a victim; bank transfers lock accounts in id order.

In the interview

> **30-second answer:** "Deadlock needs four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. I break circular wait with a global lock order — e.g. lock seats or accounts in ascending id — or break hold-and-wait with tryLock with timeout and release-and-retry. I avoid calling external code while holding a lock, and I'd detect it in production with thread dumps."

Self-check

- **Q:** What does lock ordering break? **A:** Circular wait.

TL;DR

Deadlock = cycle of waiting; needs all 4 Coffman conditions. Lock ordering or tryLock + release. No alien code under a lock.

12.18 Livelock

What it is

Threads keep running and reacting to each other but make **no progress**.

The problem it solves

Recognising it — it often comes *from* deadlock avoidance: two threads `tryLock`, fail, release and retry at the same rhythm.

How it works (and how to fix it)

T1 takes A, fails on B, releases; T2 takes B, fails on A, releases; both retry → forever. **Fixes:** random or exponential back-off, a consistent lock order (no retries needed), or a tie-breaker (older transaction wins).

Java example

`transferTryLock` sleeps a **random** number of nanoseconds after failure; remove it and the threads could fail in lock-step.

Trade-offs

Random back-off makes progress very likely but adds latency; exponential back-off adapts to contention.

When not to use it

Never design retry loops with no randomness or tie-breaker.

Common mistakes

Fixed retry delays for everyone; retrying forever with no limit or alert.

Real-world examples

Ethernet CSMA/CD binary exponential back-off; AWS SDK and Resilience4j retries with **jitter**.

In the interview

> **30-second answer:** "Livelock is when threads keep reacting to each other without progress — like two people side-stepping in a corridor. It often comes from symmetric retry logic; I fix it with randomized or exponential back-off, or remove retries by using a lock order."

Self-check

- **Q:** Thread states in deadlock vs livelock? **A:** Deadlocked threads are blocked/waiting; livelocked ones are `RUNNABLE` but achieve nothing.

TL;DR

Busy but no progress. Break symmetry with jittered back-off or a tie-breaker.

12.19 Starvation (and fairness)

What it is

A thread waits indefinitely because others keep winning, though the system progresses.

The problem it solves

Recognising the causes: barging locks under contention, writers behind endless readers, low priorities, a long task hogging a single-thread executor.

How it works (fixes)

Fair locks/semaphores where it matters (`new ReentrantLock(true)`, `new Semaphore(n, true)`, `new ReentrantReadWriteLock(true)`); short critical sections; separate pools for slow and fast work (bulkheads); don't rely on `Thread.setPriority` (OS-dependent).

Java example

Chapter 13's Dining Philosophers `Waiter` uses `new Semaphore(n - 1, true)`.

Trade-offs

Fairness prevents starvation but reduces throughput.

When not to use it

Don't make every lock fair "just in case".

Common mistakes

Confusing it with deadlock: in starvation *someone* progresses; in deadlock nobody does.

Real-world examples

OS schedulers' *aging*; Linux CFS and EEVDF (kernel 6.6+) fair CPU shares.

In the interview

> **30-second answer:** "Starvation is when a thread never gets a resource because others always win — e.g. writers behind endless readers. Fair locks, short critical sections and separate pools fix it, at some throughput cost."

Self-check

- **Q:** Fix for writer starvation in `ReentrantReadWriteLock`? **A:** Fair mode (`new ReentrantReadWriteLock(true)`).

TL;DR

Some thread never gets a turn. Fairness, short sections, separate pools.

Part G — Concurrency patterns

The next five patterns share one runnable file, `patterns/Main.java`. Full runnable code: code/lld/ch12/patterns/ (see the Full Edition).

12.20 Signaling pattern

What it is

One thread **notifies** another that an event happened: "B must run after A".

The problem it solves

Ordering between threads without busy-waiting on a flag.

How it works

Create a signal in the "not yet" state; the waiter blocks on it; the signaller releases it; the waiter continues and (via happens-before) sees everything A did.

Tool	Reusable	Carries a value	Good for
`Semaphore(0)`	Yes	No	Ping-pong (FooBar, Ch 13)
`CountDownLatch(1)`	No	No	One-time "go"
`CompletableFuture`	No	Yes	"Here is the result"
Condition variable	Yes	Via shared state	Complex conditions

Java example

`signaling()`: B starts first but blocks on `aDone.acquire()` (a `Semaphore(0)`); A logs `charge payment` and calls `release()`, so A's line always prints first.

Trade-offs

Semaphores are compact but easy to mis-count; latches can't reset; futures are clean but single-use.

When not to use it

If the signal is really "here's a work item", use a `BlockingQueue`.

Common mistakes

A tool that *forgets* signals (bare `notify()` before the waiter waits = lost wakeup); semaphores and latches remember.

Real-world examples

`CompletableFuture` chains in WebFlux/Netty callbacks; start-up gates; "order paid → send confirmation".

In the interview

"Signaling means one thread waits for an event raised by another. I use a zero-permit semaphore for repeated hand-offs, a latch for one-time events, or a `CompletableFuture` when a result travels with the signal."

Self-check

Q: Why is bare `notify()` risky? **A:** If it happens before the wait, it's lost; pair it with a state variable checked in a loop.

TL;DR

"You may go now": Semaphore(0), latch, or future — make signals unlosable.

12.21 Thread pool pattern

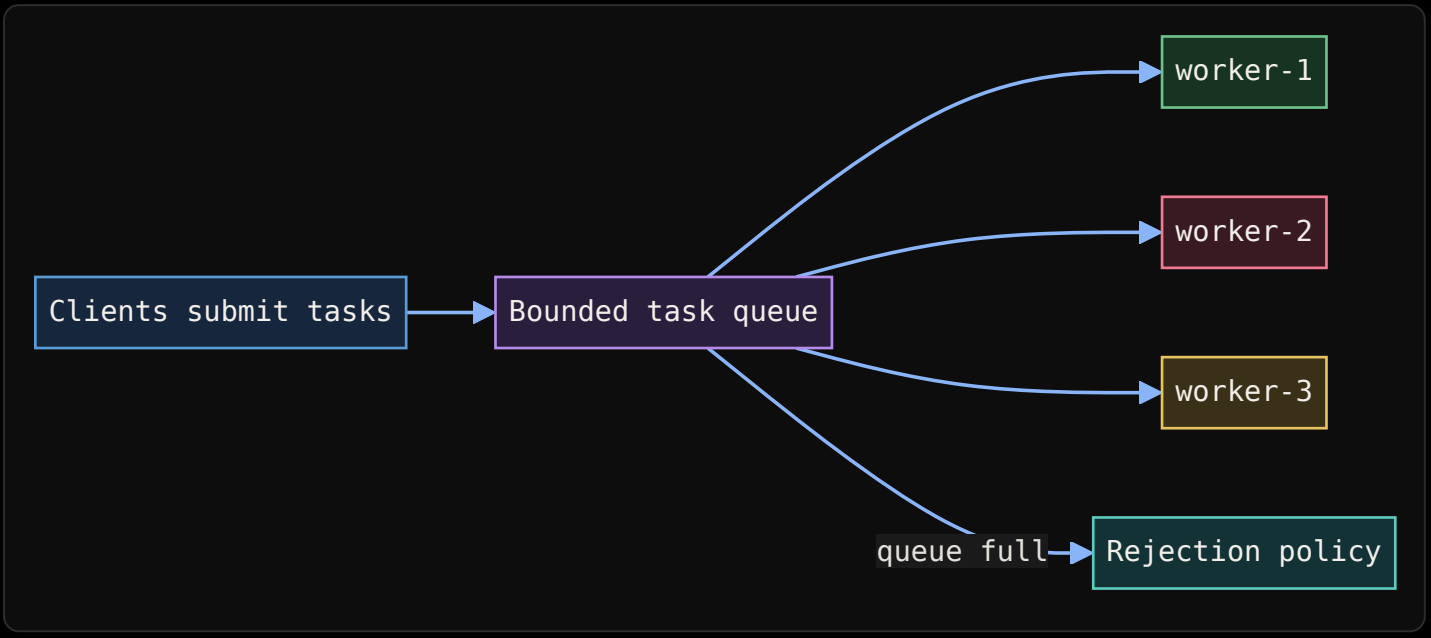
What it is

A fixed set of **worker threads** takes tasks from a shared **queue**, reusing threads.

The problem it solves

Per-task thread creation is expensive, and unbounded concurrency overwhelms CPU, memory and downstreams.

How it works



Workers loop `queue.take(); task.run()`. **`ThreadPoolExecutor` rules:** below **core** threads → new thread per task; then the **queue**; queue full → extra threads up to **max**; still full → **rejection policy** (`AbortPolicy`

default throws, `CallerRunsPolicy` runs on the submitter, `DiscardPolicy`, `DiscardOldestPolicy`). Shutdown drains the queue (one **poison pill** per worker in the hand-made pool); `submit`'s "shut down? else enqueue" is check-then-act, so it shares a lock with pill-adding or a task could land behind the pills and never run. **Sizing:** CPU-bound ≈ cores; I/O-bound ≈ cores × (1 + wait/compute), or virtual threads.

Java example

```java
// File: code/lld/ch12/patterns/Main.java
        ThreadPoolExecutor exec = new ThreadPoolExecutor(
                2, 4,                                  // core and max threads
                30, TimeUnit.SECONDS,                  // idle time before extra threads die
                new ArrayBlockingQueue<>(100),         // bounded: never an OOM from a huge backlog
                new ThreadPoolExecutor.CallerRunsPolicy()); // when full, the submitter runs it (back-pressure)
```

(`SimpleThreadPool` in the same file is a hand-made pool showing the mechanics.)

Trade-offs

Choice	Effect
Bounded queue	Protects memory; needs a rejection policy
Unbounded (`newFixedThreadPool`)	Never rejects, may OOM; max size never used
`CallerRunsPolicy`	Natural back-pressure
Many pools (bulkheads)	Isolation; more threads overall

When not to use it

Huge numbers of blocking tasks on Java 21 (virtual-thread-per-task); recursive CPU work (`ForkJoinPool`, work stealing).

Common mistakes

Tasks waiting on tasks in the same bounded pool (thread-starvation deadlock); swallowed exceptions; no `shutdown()`.

Real-world examples

Tomcat `maxThreads` (default 200), Spring `ThreadPoolTaskExecutor`, DB connection pools, `ForkJoinPool.commonPool()`.

In the interview

"A thread pool reuses a bounded set of workers fed by a queue. I size it by workload type, use a bounded queue with an explicit rejection policy for back-pressure, and give slow dependencies their own pool as a bulkhead."

Self-check

Q: When are threads beyond `corePoolSize` created? **A:** Only when the queue is full (and below max).

TL;DR

Workers + queue + rejection policy. Bound everything; size by CPU vs I/O.

12.22 Producer-consumer pattern

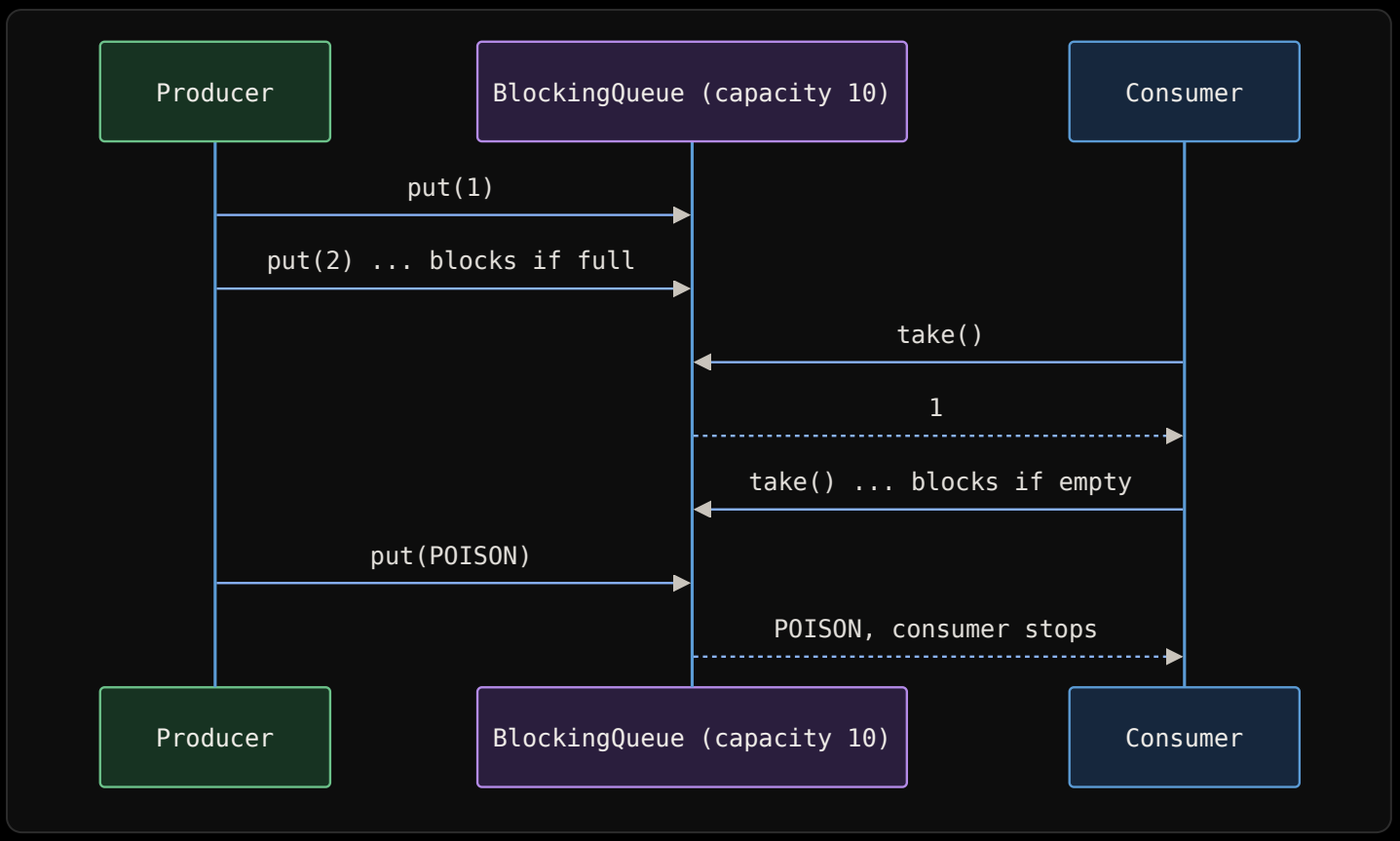
What it is

Producers put items in a shared **buffer**; **consumers** take them out; the buffer decouples their speeds.

The problem it solves

Different rates: a bounded buffer smooths bursts and applies **back-pressure**, with neither side knowing the other.

How it works



Share a bounded `BlockingQueue`; `put` waits when full, `take` when empty. Shutdown: a **poison pill per consumer**, interruption, or a done flag + `poll(timeout)`.

Java example

`producerConsumer()`: 1 producer, 2 consumers, `ArrayBlockingQueue<>(10)`, one `POISON` per consumer; the sum is exactly `500500`. Chapter 13 builds the queue itself two ways.

Trade-offs

Pros	Cons
Decouples rates; back-pressure; easy to scale consumers	Queueing latency; no ordering across consumers; needs a shutdown protocol

When not to use it

When the caller needs the result immediately — call directly.

Common mistakes

Unbounded buffer (memory blow-up); one poison pill for N consumers (N−1 hang); busy-polling `poll()` instead of `take()`.

Real-world examples

Kafka across machines (topics = buffer; consumer groups scale), and inside its Java producer (accumulator + sender thread).

In the interview

"Producer-consumer decouples producers from consumers with a bounded blocking queue: put waits when full, take waits when empty, giving natural back-pressure. I stop consumers with one poison pill each, and scale consumers independently."

Self-check

Q: Why bounded? **A:** So a slow consumer pushes back instead of the queue growing until memory runs out.

TL;DR

Bounded `BlockingQueue` between them; back-pressure for free; one poison pill per consumer.

12.23 Reader-writer pattern

What it is

Many readers may access shared data together; a writer needs exclusive access.

The problem it solves

Read-mostly data (config, catalogues, routing tables) shouldn't be serialised through one mutex.

How it works

(1) **Read-write lock** (12.10), fair if writers could starve. (2) **Copy-on-write snapshot**: an immutable object in a `volatile` field; readers just read it; a writer builds a new copy and swaps the reference.

```
private volatile Map<String, String> config = Map.of();       // immutable snapshot
String get(String k) { return config.get(k); }                  // readers: no lock
synchronized void put(String k, String v) {                      // writers: serialized
    Map<String, String> copy = new HashMap<>(config);
    copy.put(k, v);
    config = Map.copyOf(copy);                                    // publish new snapshot
}
```

	RW lock	Copy-on-write snapshot
Reader cost	Shared lock	One volatile read
Writer cost	Exclusive lock, in-place	Copy whole structure
Readers see	Latest data	Consistent, maybe slightly old snapshot
Best for	Frequent-ish writes, big data	Rare writes, small–medium data

Java example

`ConfigStore` (RW lock) in `locks/Main.java`; the snapshot above; `CopyOnWriteArrayList` in `collections/Main.java`.

Trade-offs

See the table; RW-lock readers must not upgrade.

When not to use it

Write-heavy data — a mutex or `ConcurrentHashMap`.

Common mistakes

Mutating the shared snapshot in place; read → write upgrades.

Real-world examples

Feature-flag/config clients swapping immutable snapshots; Linux RCU (read-copy-update).

In the interview

"For read-heavy data I use either a read-write lock or, if writes are rare, an immutable snapshot in a volatile field that writers replace — readers never block."

Self-check

Q: What guarantees readers see a fully built snapshot? **A:** The volatile write HB later volatile reads, and the snapshot is immutable.

TL;DR

Many readers, one writer: RW lock, or immutable snapshot + volatile swap.

12.24 Immutability & thread confinement

What it is

Thread safety **without synchronization**. **Immutability**: state can't change after construction (`String`, `LocalDate`, `BigDecimal`, records of immutables, `List.of`). **Thread confinement**: only one thread touches the data (locals, `ThreadLocal`, a single owner thread such as an event loop or actor).

The problem it solves

Races need *shared mutable* state — remove "mutable" or remove "shared".

How it works

Immutable checklist: `final` class or `record`; `private final` fields; no setters (return new objects, `withItem`); defensive-copy mutable inputs (`List.copyOf`) and never expose internals; don't leak `this`. **Confinement:** locals; `ThreadLocal.withInitial(...)` (call `remove()` in pools); a single-writer thread fed by a queue.

Java example

```java
// File: code/lld/ch12/patterns/Main.java
    record Order(String id, List<String> items) {
        Order { items = List.copyOf(items); }                 // caller's list can't change ours
        Order withItem(String item) {                          // "modify" = return a new object
            List<String> copy = new ArrayList<>(items); copy.add(item);
            return new Order(id, copy);
        }
    }

    // ---------- 5. Thread confinement ----------
    private static final ThreadLocal<StringBuilder> BUFFER = ThreadLocal.withInitial(StringBuilder::new);
```

The caller's later `mutable.add("hacked")` doesn't affect `o1` (output `o1 = [pen], o2 = [pen, book]`).

Trade-offs

	Immutability	Confinement
Cost	New object per change (GC)	Per-thread copies; `ThreadLocal` leaks if not cleaned
Sharing	Share freely	Not shareable
Reasoning	Easiest	Easy, if truly enforced

When not to use it

Very large objects changed very often — use a concurrent structure.

Common mistakes

`final List` that is mutable and exposed; `ThreadLocal` in pools without `remove()` (and heavy `ThreadLocal`s with many virtual threads — **scoped values** are the alternative, preview in 21, final in Java 25 via JEP 506); sharing `SimpleDateFormat` instead of `DateTimeFormatter`.

Real-world examples

`java.time` and `DateTimeFormatter`; Netty/Vert.x event-loop confinement; Redis's single main thread; Spring `@Transactional` state in `ThreadLocal`s.

In the interview

"The easiest thread-safety is none needed: make value objects immutable — final fields, defensive copies, 'with' methods — and confine mutable state to one thread with locals, ThreadLocal, or a single owner thread fed by a queue."

Self-check

Q: Is a class with `private final Date created` immutable? **A:** Not unless it copies the mutable `Date` in and out — prefer `Instant`.

TL;DR

No shared mutable state → no races. Immutable objects share freely; confined ones aren't shared.

Part H — Concurrency in LLD interviews

12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)

What it is

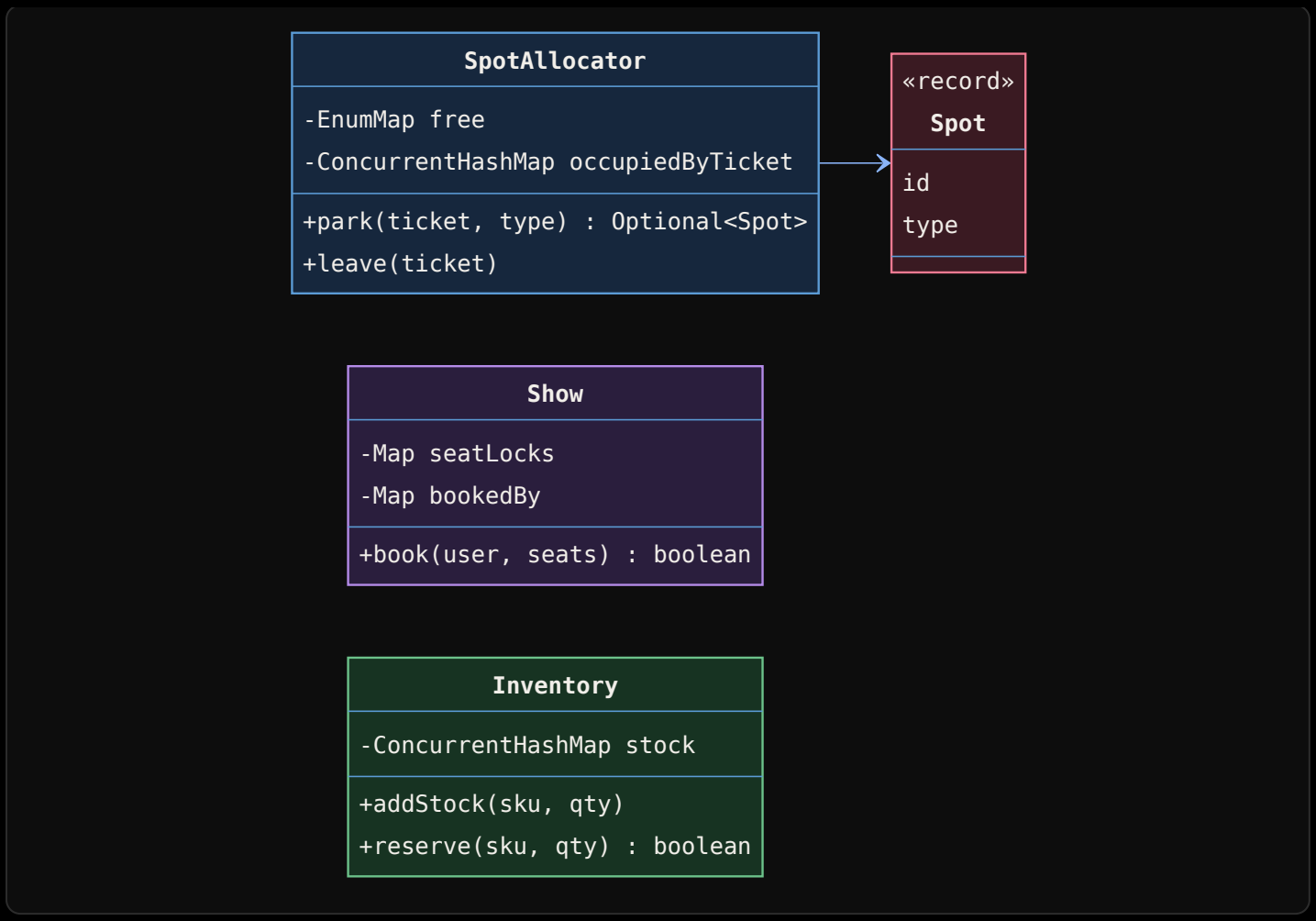
Most case studies have a **contended resource** — parking spot, seat, room, limited stock, wallet balance. The interviewer checks you prevent double allocation and still perform.

The problem it solves

Turning a single-threaded class diagram into a **thread-safe design** with the smallest correct synchronization.

How it works — a 6-step recipe

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** (usually check-then-act: *find free → assign*).
4. **Pick the smallest atomic tool:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`, CAS loop; several items all-or-nothing → per-item locks **in sorted order** (or a per-show/per-floor lock); across servers → DB constraints/transactions, optimistic versions, distributed locks.
5. **Choose granularity:** per floor/show/SKU, not global.
6. **Handle time:** holds with expiry (seat held 10 min during payment), timeouts, idempotent retries.



Java example

```java
// File: code/lld/ch12/lldcases/Main.java
        /** Locks seats in sorted order (prevents deadlock), checks all, then books all. */
        boolean book(String user, Set<String> seats) {
            List<ReentrantLock> held = new ArrayList<>();
            try {
                for (String id : new TreeSet<>(seats)) {         // global order: A1 < A2 < A3
                    ReentrantLock l = seatLocks.get(id);
                    if (l == null) return false;                  // unknown seat
                    l.lock();
                    held.add(l);
                }
                for (String id : seats) if (bookedBy.containsKey(id)) return false;   // someone got one first
                for (String id : seats) bookedBy.put(id, user);
                return true;
            } finally {
                for (int i = held.size() - 1; i >= 0; i--) held.get(i).unlock();
            }
        }
```

Parking keeps free spots in a `ConcurrentLinkedQueue` per type — `poll()` is an atomic claim, and `leave` uses `ConcurrentHashMap.remove` so a double exit is harmless. **Inventory** uses an `AtomicInteger` per SKU with a CAS loop that decrements only if `cur >= qty`. `raceAll` releases all tasks via a `CountDownLatch`. Output:

```
parking: 100 cars raced for 10 spots -> parked 10, distinct spots 10, free left 0
booking: 50 users wanted seat A2 -> successful bookings 1
booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners 1, A3 and A4 same owner: true
inventory: 200 buyers, 50 in stock -> sold 50, left 0
```

Full runnable code: code/lld/ch12/lldcases/ (see the Full Edition).

Trade-offs

Resource	In-memory	Across many servers
Parking spot	Lock-free queue per type/floor, or per-floor lock	`UPDATE ... WHERE status='FREE'` + check affected rows
Seat	Per-show/per-seat locks, sorted; hold with TTL	Unique constraint (show, seat); Redis holds (`SET NX PX`)
Inventory	`AtomicInteger` CAS / `compute`	`UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty >= 1`; expiring reservations

When not to use it

Don't bolt `synchronized` onto every method — protect only operations on contended state.

Common mistakes

A global `synchronized park()` (serialises all gates); availability check and booking as separate calls; locking seats in click order (deadlock).

Real-world examples

BookMyShow/Ticketmaster seat holds that expire (a lock with a lease); flash-sale stock decremented atomically (Redis `DECR`/Lua, conditional SQL).

In the interview

> **30-second answer:** "The critical section is 'find a free X and assign it'. Inside one JVM I make that a single atomic step — a lock-free queue poll for spots, per-seat locks taken in sorted order for multi-seat booking, a CAS decrement for stock. Across servers I'd push it into the database with a conditional update or unique constraint, and add holds with expiry for seats during payment."

Self-check

- **Q:** Which Coffman condition does sorting seat ids break? **A:** Circular wait.

TL;DR

Find the contended resource and invariant; make "check + claim" one atomic step with the smallest tool; lock fine-grained and in order; use DB/distributed mechanisms across servers.

12.26 Choosing the right tool (cheat sheet)

I need to…	Use
Increment a shared counter	`AtomicInteger` / `AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	RW lock, `StampedLock`, or volatile immutable snapshot
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition	`Lock` + `Condition` (or `wait/notifyAll`) in a `while`
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` + `putIfAbsent` / `compute` / `merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	Bounded `ExecutorService`; virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization	Immutability, thread confinement

12.27 Chapter self-check

1. **Q:** Six `Thread.State` values? **A:** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
2. **Q:** Why isn't `volatile` enough for `count++`? **A:** Visibility, not atomicity.
3. **Q:** Why `while` around `wait()`? **A:** Spurious wakeups and competing threads.
4. **Q:** Four Coffman conditions? **A:** Mutual exclusion, hold and wait, no preemption, circular wait.
5. **Q:** Deadlock vs livelock vs starvation? **A:** All wait / all run but none progress / some progress, one never does.
6. **Q:** Lock-free spot allocation across gates? **A:** Free spots in a concurrent queue; `poll()` atomically.

12.28 Chapter TL;DR

- Races come from **shared mutable state**: make critical sections atomic, or remove sharing/mutability.
- Visibility needs **happens-before**: locks, `volatile`, start/join, concurrent utilities.
- Toolbox: `synchronized` → `ReentrantLock` → RW locks → semaphores → latches/barriers → atomics/CAS → concurrent collections.
- Liveness: **lock ordering** or **tryLock + back-off** against deadlock; jitter against livelock; fairness against starvation.
- Patterns: signaling, thread pool, producer-consumer, reader-writer, immutability/confinement.
- Interviews: find the contended resource, keep "check + claim" atomic, lock fine-grained and in order, explain the multi-server version.

Chapter 13 — Classic Concurrency Problems · Study Edition

> **Study edition.** All 11 problems with their core solution code, padding removed. Demo `main` methods, helpers and full outputs are in the Full Edition (`README.md`).
>
> **Goal.** Chapter 12 gave you the toolbox; this chapter uses it on the problems interviewers ask — LeetCode's concurrency set (FooBar, Zero-Even-Odd, FizzBuzz, H2O, Dining Philosophers) and "design a thread-safe X" rounds (TTL cache, concurrent hash map, blocking queue, Bloom filter, parallel merge sort, rate limiter).

Layout per problem: *Problem → Approach → Java solution → Why it's correct → Pitfalls → Real-world examples → In the interview → Self-check → TL;DR.* Code: Java 21 in `code/lld/ch13/<problem>/Main.java`. Where event order varies, demos *verify* the result and print `true/false`.

```
javac --release 21 -d /tmp/lld_13 $(find . -name '*.java')
java -cp /tmp/lld_13 lld.ch13.foobar.Main
```

Problem map — which tool solves what

#	Problem	Core idea	Main tools
13.1	Print FooBar Alternately	Pass a "baton" between 2 threads	`Semaphore` ×2 (or monitor + turn flag)
13.2	Print Zero Even Odd	A dispatcher thread decides who goes next	`Semaphore` ×3
13.3	Fizz Buzz Multithreaded	4 threads, one counter, each waits for "its" numbers	`synchronized` + `wait/notifyAll`
13.4	Building H2O	Group threads into 2 H + 1 O	`Semaphore` (slots) + `CyclicBarrier`
13.5	Dining Philosophers	Avoid deadlock over shared forks	Lock ordering, or a "waiter" semaphore
13.6	Thread-safe cache with TTL	Expiring entries, no stampede	`ConcurrentHashMap` , `compute` , scheduled sweeper
13.7	Concurrent HashMap (simplified)	Lock striping	`ReentrantLock[]` , all-stripe resize
13.8	Bounded Blocking Queue	Wait when full / empty	`wait/notifyAll` **and** `Lock` + 2 `Condition` s
13.9	Concurrent Bloom Filter	Lock-free bit set	`AtomicLongArray` + atomic OR
13.10	Multi-threaded Merge Sort	Divide and conquer on cores	`ForkJoinPool` , `RecursiveAction`
13.11	Rate limiter (token bucket)	Atomic refill-and-take	`synchronized` , or CAS on immutable state

Checklist for any concurrency problem: (1) What is the **shared state**? (2) What **ordering or invariant** must hold? (3) Who **waits**, for **what**, and who **signals**? (4) Pick the primitive: turn-taking → semaphores; "wait until predicate" → condition variable; groups → barrier; mutual exclusion → lock; one variable → atomic. (5) Check **liveness**: can anyone wait forever, does every thread exit?

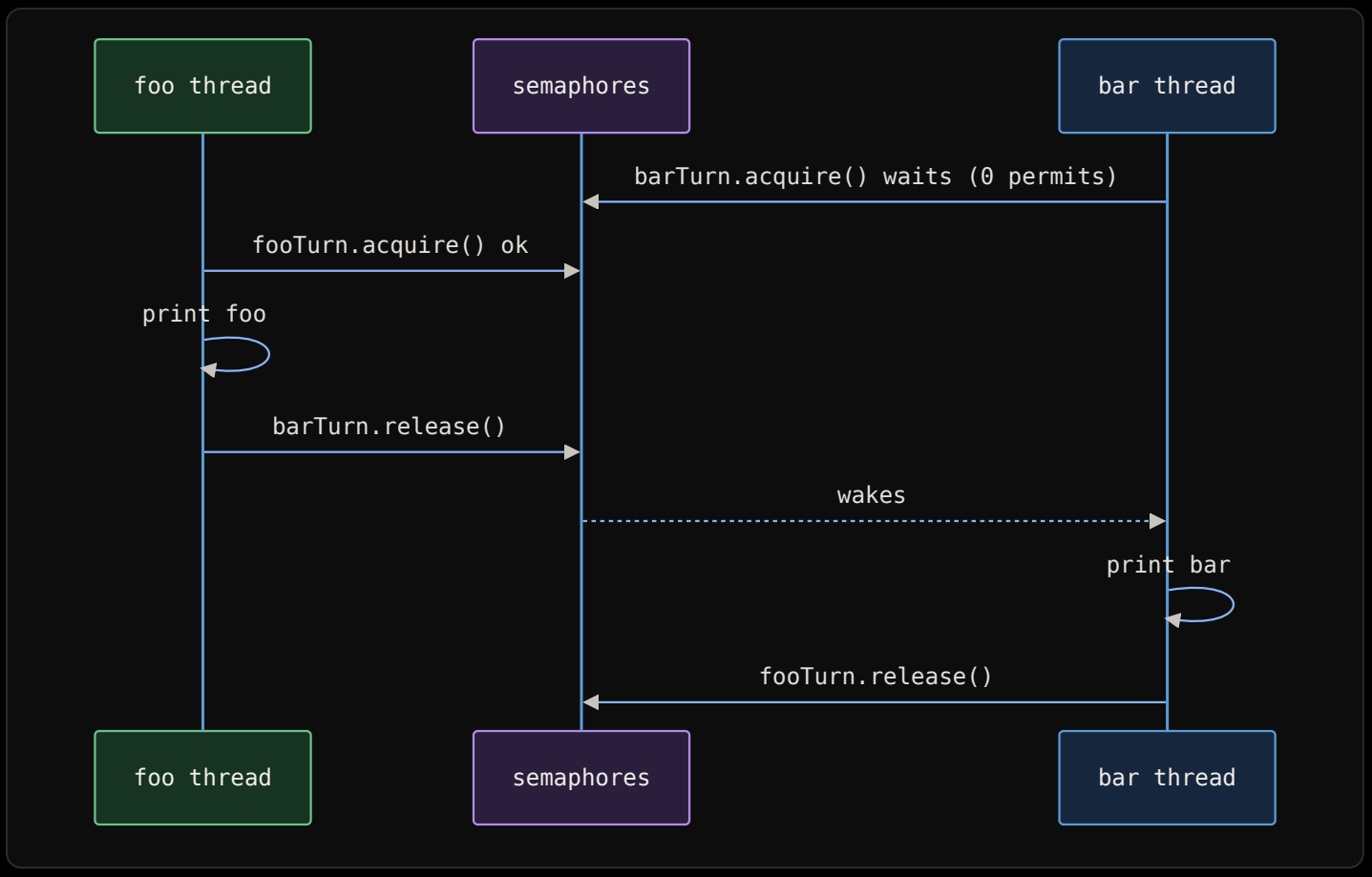
13.1 Print FooBar Alternately (LeetCode 1115)

Problem

Thread A calls `foo()` (prints "foo" *n* times), thread B calls `bar()` (prints "bar" *n* times). Output must be "foobar" × *n*, whichever starts first.

Approach

Pure **signaling** (12.20) — a baton: `fooTurn = Semaphore(1)`, `barTurn = Semaphore(0)`; each thread acquires its own, prints, releases the other's. Alternative: a monitor with a `fooTurn` flag, `while (!fooTurn) wait();`, flip, `notifyAll()`.



Java solution

```java
// File: code/lld/ch13/foobar/Main.java
    /** Solution 1: two semaphores used as a baton that is passed back and forth. */
    static class FooBarSemaphore {
        private final int n;
        private final Semaphore fooTurn = new Semaphore(1);   // foo may go first
        private final Semaphore barTurn = new Semaphore(0);   // bar must wait

        FooBarSemaphore(int n) { this.n = n; }

        void foo(Runnable printFoo) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                fooTurn.acquire();       // wait for my turn
                printFoo.run();
                barTurn.release();       // hand the baton to bar
            }
        }
        void bar(Runnable printBar) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                barTurn.acquire();
                printBar.run();
                fooTurn.release();
            }
        }
    }
    // ... Solution 2 (FooBarMonitor): synchronized + while (!fooTurn) wait(); flip; notifyAll()
```

Output (bar thread started first on purpose): `semaphores : foobarfoobarfoobarfoobar`, `both correct: true`. Full runnable code: code/lld/ch13/foobar/ (see the Full Edition).

Why it's correct

Invariant: the two semaphores hold exactly 1 permit in total, so turns strictly alternate; `release()` happens-before the `acquire()` it enables; matched loops terminate.

Pitfalls

One shared semaphore (a thread re-takes its own release); `if` instead of `while`; busy-waiting on a `volatile` flag.

Real-world examples

Ping-pong protocols, request/acknowledge stages, double-buffering.

In the interview

> **30-second answer:** "Two semaphores as a baton: foo's starts at 1, bar's at 0. Each thread acquires its own, prints, and releases the other's. The total permits are always one, so they alternate."

Self-check

- **Q:** What if `barTurn` starts at 1? **A:** bar may print first.

TL;DR

Two semaphores, one permit total, passed back and forth.

13.2 Print Zero Even Odd (LeetCode 1116)

Problem

Three threads: `zero()` prints only 0, `even()` only evens, `odd()` only odds; together `0 1 0 2 … 0 n` (n = 5 → `0102030405`).

Approach

The zero thread is a **dispatcher**: after the 0 for position *i* it releases `oddTurn` (odd *i*) or `evenTurn`; they print and release `zeroTurn`. Initial permits: zero 1, odd 0, even 0.

Java solution

```java
// File: code/lld/ch13/zeroevenodd/Main.java
        private final Semaphore zeroTurn = new Semaphore(1);
        private final Semaphore oddTurn = new Semaphore(0);
        private final Semaphore evenTurn = new Semaphore(0);
        // ...
        void zero(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i++) {
                zeroTurn.acquire();
                print.accept(0);
                // The zero thread is the "dispatcher": it decides who prints next.
                if (i % 2 == 1) oddTurn.release(); else evenTurn.release();
            }
        }
        void odd(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i += 2) {
                oddTurn.acquire();
                print.accept(i);
                zeroTurn.release();
            }
        }
        // ... even() is the same loop from 2 with evenTurn
```

Output: `n=5 -> 0102030405`, `n=10 -> 0102030405060708090 10`. Full runnable code: code/lld/ch13/zeroevenodd/ (see the Full Edition).

Why it's correct

One permit in the system → one printer at a time; loop counts match permits (zero *n*, odd ⌈n/2⌉, even ⌊n/2⌋), so everyone exits.

Pitfalls

Unsynchronized shared counters; loop bounds that don't match permits (a thread hangs).

Real-world examples

Dispatcher/worker designs (opcode routing, alternating schedulers).

In the interview

> **30-second answer:** "Three semaphores. Zero starts with one permit. After each 0 it releases odd or even depending on the index, and they release zero back. One permit total in the system, so the order is forced."

Self-check

- **Q:** For n = 4, how many times does `odd()` acquire? **A:** Twice (1 and 3).

TL;DR

Zero is the dispatcher; one permit circulates among three semaphores.

13.3 Fizz Buzz Multithreaded (LeetCode 1195)

Problem

Four threads output 1..n: multiples of 15 → `fizzbuzz`, of 3 only → `fizz`, of 5 only → `buzz`, else the number — each kind printed by its own thread.

Approach

One shared `current` guarded by the monitor; each thread loops with its own predicate: `while (current <= n && !mine(current)) wait();`, exit (with `notifyAll()`) if `current > n`, else print, `current++`, `notifyAll()`. The **condition-variable** pattern (12.13); `runWhile(predicate, action)` avoids writing the loop four times.

Java solution

```java
// File: code/lld/ch13/fizzbuzz/Main.java
        private int current = 1;                              // guarded by 'this'
        // ...
        /** Shared loop: wait until 'current' is mine (or we're done), act, advance, wake everyone. */
        private void runWhile(IntPredicate mine, IntConsumer action) throws InterruptedException {
            while (true) {
                synchronized (this) {
                    while (current <= n && !mine.test(current)) wait();
                    if (current > n) { notifyAll(); return; }     // done: wake others so they can exit too
                    action.accept(current);
                    current++;
                    notifyAll();                                  // the next number may belong to anyone
                }
            }
        }

        void fizz(Runnable printFizz) throws InterruptedException {
            runWhile(i -> i % 3 == 0 && i % 5 != 0, i -> printFizz.run());
        }
        // ... buzz: i % 5 == 0 && i % 3 != 0; fizzbuzz: i % 15 == 0; number: neither
```

Output: `1, 2, fizz, 4, buzz, fizz, 7, 8, fizz, buzz, 11, fizz, 13, 14, fizzbuzz`. Full runnable code: code/lld/ch13/fizzbuzz/ (see the Full Edition).

Why it's correct

Predicates **partition** 1..n; `current` is only touched under the monitor; `notifyAll()` is required because four predicates share one wait set; the final `notifyAll` lets everyone see `current > n` and exit.

Pitfalls

`notify()` (lost wake-ups); no `current > n` exit check; overlapping predicates (two threads claim 15).

Real-world examples

Pipeline stages waiting for a job to reach their stage.

In the interview

> **30-second answer:** "One monitor, one shared counter. Each thread waits in a while loop until the counter matches its predicate or passes n, prints, increments and notifies all. The predicates partition the numbers, so exactly one thread acts per number."

Self-check

- **Q:** Why `notifyAll`? **A:** Threads wait for *different* conditions on one monitor; only waking all guarantees the right one wakes.

TL;DR

Shared counter + per-thread predicate + `while`-wait + `notifyAll`.

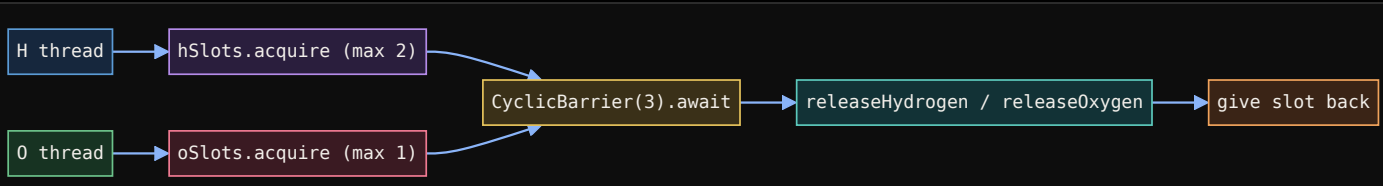
13.4 Building H2O (LeetCode 1117)

Problem

H and O threads arrive in any order and must leave in groups of exactly **2 H + 1 O**; one molecule's threads all pass before the next molecule's.

Approach

Slots — `hSlots = Semaphore(2)`, `oSlots = Semaphore(1)` cap the current molecule. **Group** — `CyclicBarrier(3)` makes the three atoms wait for each other. A thread returns its slot **after** it has been released, so the next molecule forms only when this one is done.



Java solution

```java
// File: code/lld/ch13/h2o/Main.java
    static class H2O {
        private final Semaphore hSlots = new Semaphore(2);    // at most 2 H in the current molecule
        private final Semaphore oSlots = new Semaphore(1);    // at most 1 O in the current molecule
        private final CyclicBarrier bond = new CyclicBarrier(3);   // wait until all 3 atoms are present

        void hydrogen(Runnable releaseHydrogen) throws InterruptedException {
            hSlots.acquire();
            try {
                bond.await();
            } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
            releaseHydrogen.run();
            hSlots.release();                                 // free my slot only after I have "left"
        }
        // ... oxygen() is identical with oSlots
    }
```

Output: `molecules formed: 8, every group of 3 has 2 H + 1 O: true` (orders `HHO`, `HOH`, `OHH` are all valid, so the demo verifies instead of printing). Full runnable code: code/lld/ch13/h2o/ (see the Full Edition).

Why it's correct

Slots admit ≤2 H and ≤1 O per round and the barrier needs 3 → each trip is H, H, O; slots are freed only after printing, so groups never mix; the barrier resets itself.

Pitfalls

Releasing the slot **before** printing; no barrier; H:O not exactly 2:1 (last threads hang).

Real-world examples

Batching N items of given types (matchmaking, order assembly).

In the interview

30-second answer: "Two semaphores cap the molecule at 2 H and 1 O; a CyclicBarrier of 3 makes the three wait for each other; each atom returns its slot only after it's released, so molecules never

interleave."

Self-check

- **Q:** Why is `hSlots` 2, not 1? **A:** A molecule needs two hydrogens inside the barrier at once.

TL;DR

Slots decide *who* can join; the barrier decides *when* they leave; free slots only after leaving.

13.5 Dining Philosophers (LeetCode 1226)

Problem

Five philosophers, five forks, each needs **both** adjacent forks. If everyone takes the left fork first, all may wait forever for the right — **deadlock** (all four Coffman conditions, 12.17).

Approach

Break **circular wait**: (1) **resource ordering** — always take the **lower-numbered** fork first (philosopher 4 takes fork 0 first); (2) **waiter** — a **fair** semaphore lets at most **n − 1** try at once, so someone always gets both forks. (Also: `tryLock` both with random back-off; Chandy–Misra for distributed settings.)

Java solution

```java
// File: code/lld/ch13/philosophers/Main.java
    /** Fix 1 - resource ordering: always pick up the LOWER-numbered fork first (breaks circular wait). */
    static class OrderedForks implements Table {
        private final ReentrantLock[] forks;
        // ... constructor creates n ReentrantLocks

        public void eat(int p, Runnable eatAction) {
            int left = p, right = (p + 1) % forks.length;
            ReentrantLock first = forks[Math.min(left, right)], second = forks[Math.max(left, right)];
            first.lock();
            try {
                second.lock();
                try { eatAction.run(); } finally { second.unlock(); }
            } finally { first.unlock(); }
        }
    }

    /** Fix 2 - a "waiter" lets at most n-1 philosophers try at once (breaks circular wait too). */
    static class Waiter implements Table {
        // ... forks as above, plus:
            seats = new Semaphore(n - 1, true);                // fair: nobody starves waiting for a seat
        // ... eat(): seats.acquire(); lock left, then right; eat; unlock both; seats.release() in finally
    }
```

Output: both strategies finish `[10000, 10000, 10000, 10000, 10000]` meals. Full runnable code: code/lld/ch13/philosophers/ (see the Full Edition).

Why it's correct

Ordering: increasing-index acquisition makes a waiting cycle impossible. **Waiter:** 4 diners, 5 forks → by pigeonhole someone eats. Forks are released in `finally`.

Pitfalls

Left-then-right for all (deadlock); `tryLock` retries without random back-off (livelock); the fair semaphore only makes *seats* FIFO — fair fork locks are needed for strict no-starvation.

Real-world examples

Anything needing two locks: bank transfers, multi-row DB transactions.

In the interview

> **30-second answer:** "Naively everyone takes the left fork and waits for the right — circular wait, deadlock. I break circular wait by always taking the lower-numbered fork first, or by a waiter semaphore allowing only n−1 philosophers to try; fairness prevents starvation."

Self-check

- **Q:** With ordering, who behaves "differently"? **A:** The philosopher between forks 4 and 0 — takes fork 0 (right) first.

TL;DR

Break circular wait: global fork order, or only n−1 diners.

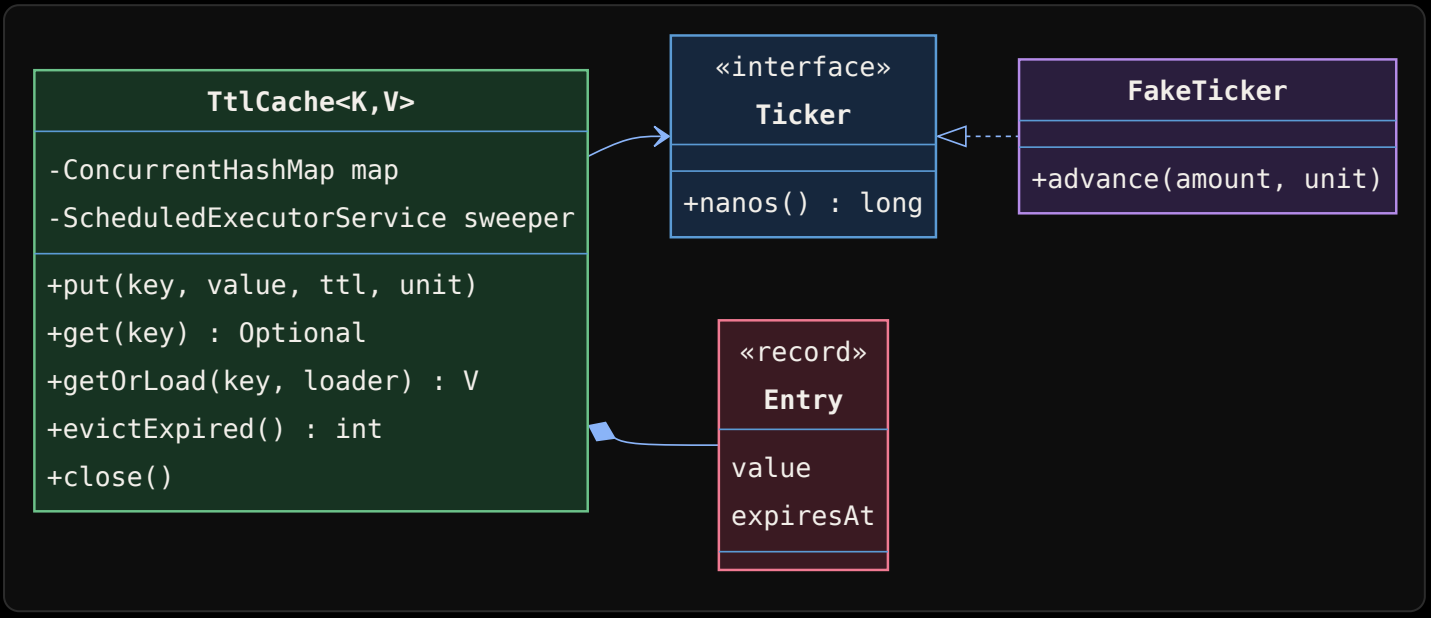
13.6 Thread-safe cache with TTL

Problem

A shared cache whose entries expire after a **TTL**: `put` (default or per-entry TTL), `get` hides expired entries, `getOrLoad` loads a miss **without a cache stampede**, expired entries leave memory, testable without sleeping.

Approach

Immutable `Entry(value, expiresAt)` in a `ConcurrentHashMap`; **lazy expiry** in `get` via conditional `remove(key, entry)`; **active expiry** by a daemon scheduled sweeper; **no stampede** via `map.compute` (loader runs once per key); an injected `Ticker` for tests.



Java solution

```
// File: code/lld/ch13/ttlcache/Main.java
        private record Entry<V>(V value, long expiresAt) { }

        private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
        // ... defaultTtlNanos, ticker, daemon sweeper scheduled at a fixed rate to call evictExpired()

        private boolean expired(Entry<V> e) { return ticker.nanos() - e.expiresAt() >= 0; }   // overflow-safe
compare

        Optional<V> get(K key) {
            Entry<V> e = map.get(key);
            if (e == null) return Optional.empty();
            if (expired(e)) {
                map.remove(key, e);   // conditional remove: never deletes a FRESH value another thread just put
                return Optional.empty();
            }
            return Optional.of(e.value());
        }

        V getOrLoad(K key, Function<? super K, ? extends V> loader) {
            return map.compute(key, (k, old) ->
                    old != null && !expired(old) ? old : new Entry<>(loader.apply(k), ticker.nanos() +
defaultTtlNanos)
            ).value();
        }
        // ... evictExpired(): for each entry, if expired, map.remove(key, entry) — the same conditional remove
```

Output: `16 concurrent misses -> loader ran 1 time(s), all got same value: true`; with the real clock the sweeper empties 1,000 entries within 200 ms.

Full runnable code: code/lld/ch13/ttlcache/ (see the Full Edition).

Why it's correct

Immutable entries in one CHM (never half-written); `remove(key, e)` removes only that exact entry; `compute` is atomic per key (one load); `now - expiresAt >= 0` survives `nanoTime` overflow; the sweeper is a daemon closed in `close()`. (Its constructor leaks `this` to the sweeper — safe only because fields are set and executor submission is a happens-before edge; prefer a static factory.)

Pitfalls

Wall-clock `currentTimeMillis()` (use `nanoTime`); slow loaders inside `compute` block the bin (store a `CompletableFuture`); lazy-only expiry leaks, active-only serves stale — use both; no size limit (add LRU/LFU).

Real-world examples

Caffeine (and Guava `CacheBuilder`); **Redis** `EXPIRE` (lazy + periodic); sessions, OTPs, seat holds.

In the interview

> **30-second answer:** "A ConcurrentHashMap of immutable entries holding value and expiry time. Get checks expiry lazily and removes with a conditional remove; a daemon scheduled sweeper removes expired keys nobody reads. getOrLoad uses compute so concurrent misses load once. Time is injected for testing."

Self-check

- **Q:** Why `remove(key, e)` not `remove(key)`? **A:** So a value written between our read and remove isn't deleted.

TL;DR

CHM + immutable (value, expiry) entries; lazy + scheduled expiry; `compute` for single-flight loads; injected clock.

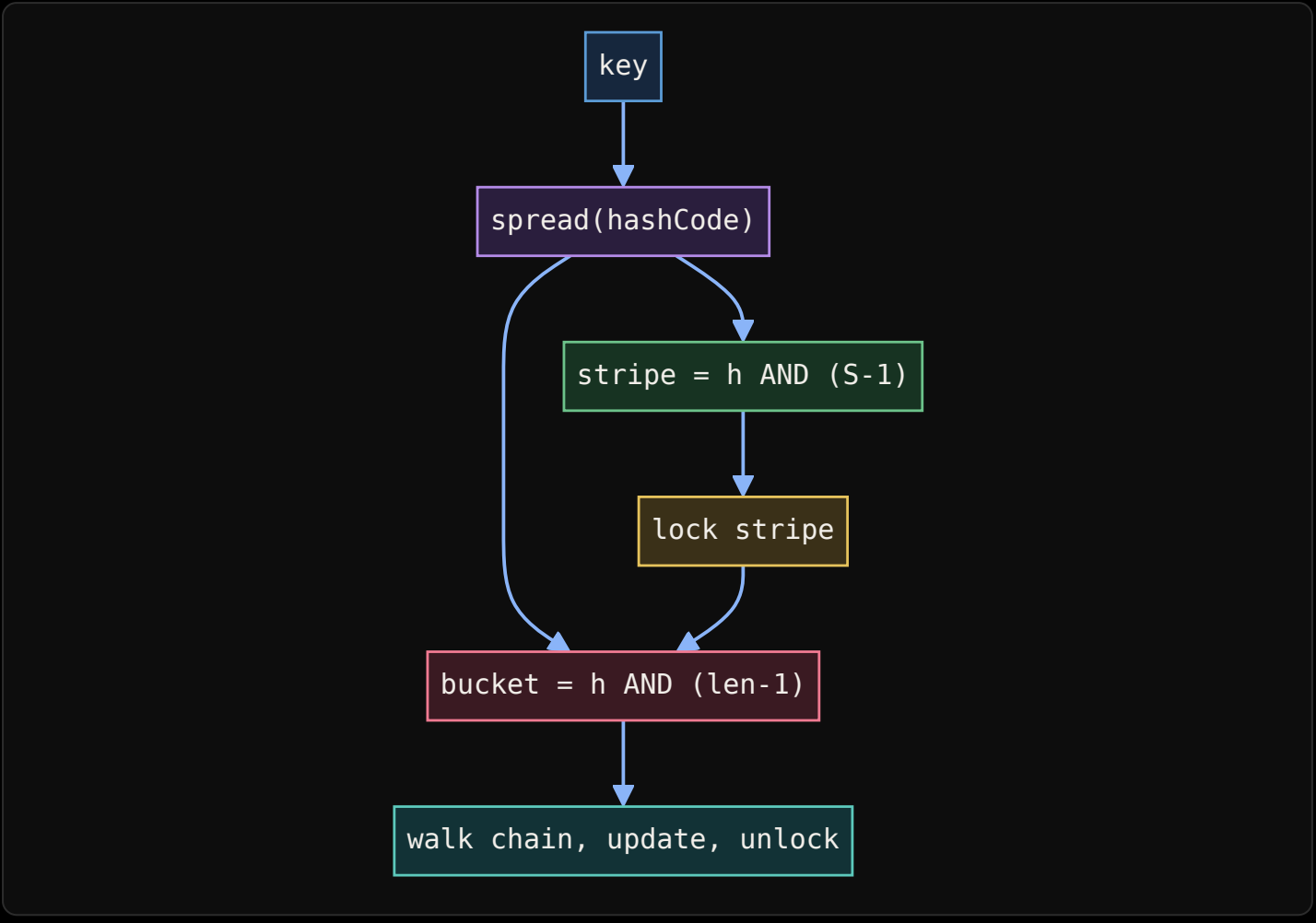
13.7 Design a (simplified) Concurrent HashMap — lock striping

Problem

A hash map with `get`, `put`, `remove`, `compute` and resizing, usable by many threads and faster than one global lock.

Approach

Lock striping (12.11): buckets of linked chains plus `S` locks (power of two). Key hash *h* → stripe `h & (S − 1)`, bucket `h & (len − 1)`; the stripe depends only on the hash, so it **never changes on resize**. Operations lock only their stripe. **Resize** (size > 0.75 × buckets): lock **all** stripes in index order, re-check, rehash into a 2× table, publish via `volatile`, unlock in reverse. Size in an `AtomicInteger`.



Java solution

```
// File: code/lld/ch13/stripedmap/Main.java
        private volatile Node<K, V>[] table;                // replaced (never mutated in place) only while ALL
locks are held
        private final ReentrantLock[] stripes;
        private final AtomicInteger size = new AtomicInteger();
        // ... constructor requires power-of-two sizes and stripes <= buckets

        // A key's stripe depends only on its hash, NOT on the table size, so it never changes on resize.
        private ReentrantLock stripeFor(int hash) { return stripes[hash & (stripes.length - 1)]; }
        // ... get/remove: lock stripeFor(h), walk t[indexFor(h, t.length)], unlock
        // ... put/compute share doCompute(): under the stripe lock find the node, apply remap,
        // ... update / unlink / insert at head; then, AFTER unlocking:
            if (grew) resize();                                     // never resize while holding one stripe

        /** Stop-the-world resize: take every stripe lock IN ORDER (no deadlock), rehash, publish. */
        private void resize() {
            for (ReentrantLock l : stripes) l.lock();
            try {
                Node<K, V>[] old = table;
                if (size.get() <= old.length * LOAD_FACTOR) return;  // another thread already resized
                Node<K, V>[] nt = (Node<K, V>[]) new Node[old.length * 2];
                // ... move every node to nt[indexFor(n.hash, nt.length)]
                table = nt;
            } finally {
                for (int i = stripes.length - 1; i >= 0; i--) stripes[i].unlock();
            }
        }
```

Output: 8 threads × 20,000 puts → `size=160000, missing=0`, table grew 16 → 262144 buckets; 80,000 concurrent `compute` increments are exact.

Full runnable code: code/lld/ch13/stripedmap/ (see the Full Edition).

Why it's correct

Each bucket has exactly one stripe (powers of two, $S \le$ length → stripe index = low bits of bucket index). **Resize** holds all locks, re-checks, publishes via `volatile`. **No deadlock:** one lock normally; resize takes all in order after releasing its own. **Compute** runs wholly under the stripe lock.

Pitfalls

Stripe from the current bucket index (changes on resize); resizing while holding a stripe; unsynchronized `table` reads; `null` keys/values.

How the real `ConcurrentHashMap` goes further

Aspect	This design	JDK 8+ `ConcurrentHashMap`
Reads	Lock the stripe	Lock-free (volatile reads)
Write locking	One of S stripe locks	CAS into empty bins; `synchronized` on the bin head otherwise
Long chains	Linked list	Red-black tree beyond 8 nodes (table ≥ 64)
Resize	Stop-the-world	Incremental; threads help transfer
Size	`AtomicInteger`	`LongAdder`-style counter cells

Real-world examples

Java 7's `ConcurrentHashMap` **segments** (16 by default); Guava `Striped<Lock>`; Caffeine's striped read buffers.

In the interview

> **30-second answer:** "An array of buckets plus a small array of locks; each key maps to a stripe by its hash bits, so different stripes work in parallel. Every bucket belongs to exactly one stripe. Resize takes all stripe locks in order, rehashes and publishes the new table through a volatile field."

Self-check

- **Q:** Why must the stripe count divide the table length? **A:** So every bucket maps to exactly one stripe.

TL;DR

N locks for M buckets, stripe by hash bits, resize under all locks in order, publish via volatile.

13.8 Thread-safe bounded Blocking Queue (two ways)

Problem

A fixed-capacity FIFO for producer-consumer: `put` blocks while full, `take` while empty, plus timed `offer(item, timeout)` and `poll(timeout)`.

Approach

A **circular array** (`head`, `tail`, `count`) and two synchronisation designs:

	Way 1: monitor	Way 2: `ReentrantLock` + 2 `Condition`s
Lock	`synchronized` methods	`lock.lockInterruptibly()`
Wait	`while (full) wait();`	`while (full) notFull.await();`
Wake	`notifyAll()` — one shared wait set	`notEmpty.signal()` / `notFull.signal()`
Timed wait	`TimeUnit.timedWait` + deadline loop	`awaitNanos` returns time left
Efficiency	Wakes everyone (thundering herd)	Wakes one thread of the right kind

Java solution

```
// File: code/lld/ch13/blockingqueue/Main.java
    // ... Way 1 (MonitorQueue): producers AND consumers share one monitor, so notifyAll():
        public synchronized void put(T item) throws InterruptedException {
            while (count == items.length) wait();
            enqueue(item);
            notifyAll();
        }
    // ...
    // ... Way 2 (ConditionQueue): ReentrantLock with TWO condition queues (how ArrayBlockingQueue is built).
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        public void put(T item) throws InterruptedException {
            if (item == null) throw new NullPointerException();
            lock.lockInterruptibly();
            try {
                while (count == items.length) notFull.await();
                enqueue(item);
            } finally { lock.unlock(); }
        }
        // ... take() mirrors put(); timed poll/offer loop on nanos = notEmpty.awaitNanos(nanos) (returns time
left)
        private void enqueue(T item) {
            items[tail] = item;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();                                  // wake ONE waiting consumer
        }
        @SuppressWarnings("unchecked")
        private T dequeue() {
            T x = (T) items[head];
            items[head] = null;
            head = (head + 1) % items.length;
            count--;
            notFull.signal();                                   // wake ONE waiting producer
            return x;
        }
```

Both pass a stress test (3 producers × 20,000, 3 consumers, capacity 2): `consumed 60000 items, sum ok=true, left in queue=0` ; `poll(30ms)` on empty → `null`, `offer(30ms)` on full → `false`. Full runnable code: code/lld/ch13/blockingqueue/ (see the Full Edition).

Why it's correct

One lock guards all fields; `while` loops handle spurious wakeups; Way 1 needs `notifyAll()` (one shared wait set); Way 2 can `signal()` (separate queues per kind); timed waits track *remaining* time.

Pitfalls

`if` instead of `while`; `notify()` on a shared monitor; not nulling `items[head]` (leak); `null` items when `null` means timeout.

Real-world examples

`ArrayBlockingQueue` = Way 2; `LinkedBlockingQueue` uses **two** locks (`putLock`, `takeLock`).

In the interview

> **30-second answer:** "A circular array with head, tail and count guarded by one lock. put waits while count == capacity, take waits while count == 0, both in while loops. With synchronized I must use notifyAll because both sides share one wait set; with ReentrantLock I use two Conditions, notFull and notEmpty, and signal just the right side."

Self-check

- **Q:** Why is `signal()` safe in Way 2 but `notify()` unsafe in Way 1? **A:** Separate condition queues guarantee the woken thread is the right kind.

TL;DR

Circular buffer + one lock + wait-while-full/empty loops. Monitor → `notifyAll`; Lock → `notFull`/`notEmpty` + `signal`.

13.9 Concurrent Bloom Filter

Problem

A thread-safe **Bloom filter** — a compact probabilistic set answering *"definitely not present"* or *"probably present"* — that many threads add to and query at once.

Approach

m bits; **add** sets *k* hashed positions; **query**: any 0 → **definitely absent**, all 1 → **probably** present (*false positive* possible). No false negatives, no deletion (a *counting* filter allows it).

Sizing for *n* items and false-positive rate *p*: $m = \lceil -n \cdot \ln p / (\ln 2)^2 \rceil$, $k = \text{round}((m / n) \cdot \ln 2)$. For $n = 80{,}000$, $p = 1\% \rightarrow m \approx 766{,}805$ bits (≈ 93.6 KiB), $k = 7$ — about **9.6 bits per item**, whatever the item size.

Double hashing (Kirsch–Mitzenmacher): positions $h1 + i \cdot h2$. **Lock-free:** an `AtomicLongArray` with **atomic OR** (`getAndAccumulate`); queries are volatile reads — bits only go 0 → 1, so a query can at most miss an unfinished `add`.

Java solution

```
// File: code/lld/ch13/bloomfilter/Main.java
        private final AtomicLongArray words;     // m bits packed into 64-bit words
        // ... numBits (m) and numHashes (k) computed from expectedItems and falsePositiveRate

        /** Adds the item. Returns true if at least one bit changed (i.e. it was definitely new). */
        boolean add(String item) {
            long base = fnv1a64(item);
            long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;   // h2 odd
            boolean changed = false;
            for (int i = 0; i < numHashes; i++) {
                int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);        // double hashing
                long mask = 1L << (bit & 63);
                long before = words.getAndAccumulate(bit >>> 6, mask, (cur, m) -> cur | m);  // atomic OR (CAS
loop)
                if ((before & mask) == 0) changed = true;
            }
            return changed;
        }
        // ... mightContain(): same k positions; any bit == 0 -> return false (definitely absent)
        // ... fnv1a64 (FNV-1a) and fmix64 (MurmurHash3 finalizer) mix the bits
```

Output: `m=766805 bits (93 KiB), k=7`; after an 8-thread insert, `false negatives: 0` and a measured false-positive rate of `1.01%` (target 1%).

Full runnable code: code/lld/ch13/bloomfilter/ (see the Full Edition).

Why it's correct

A plain `|=` can lose bits (false negatives); atomic OR retries. Bits are **monotonic** (no ABA, no locks) and OR is commutative, so the result is interleaving-independent.

Pitfalls

Non-atomic bit setting; weak hashes (clustered bits); far more than *n* items; negative indices (use `Math.floorMod`).

Real-world examples

Cassandra, HBase, RocksDB (skip disk reads); Guava `BloomFilter`; RedisBloom; "URL already crawled?" checks.

In the interview

> **30-second answer:** "A bit array of size m with k hash positions per item from double hashing; add sets k bits, query checks them — any zero means definitely absent. For concurrency I store bits in an AtomicLongArray and set them with an atomic OR; since bits only go from 0 to 1, reads need no lock. m and k come from n and the target false-positive rate."

Self-check

- **Q:** Bits per element for 1% FPR? **A:** About 9.6.

TL;DR

Bits + k hashes; "no" is certain, "yes" is probable. `AtomicLongArray` + atomic OR = lock-free and correct.

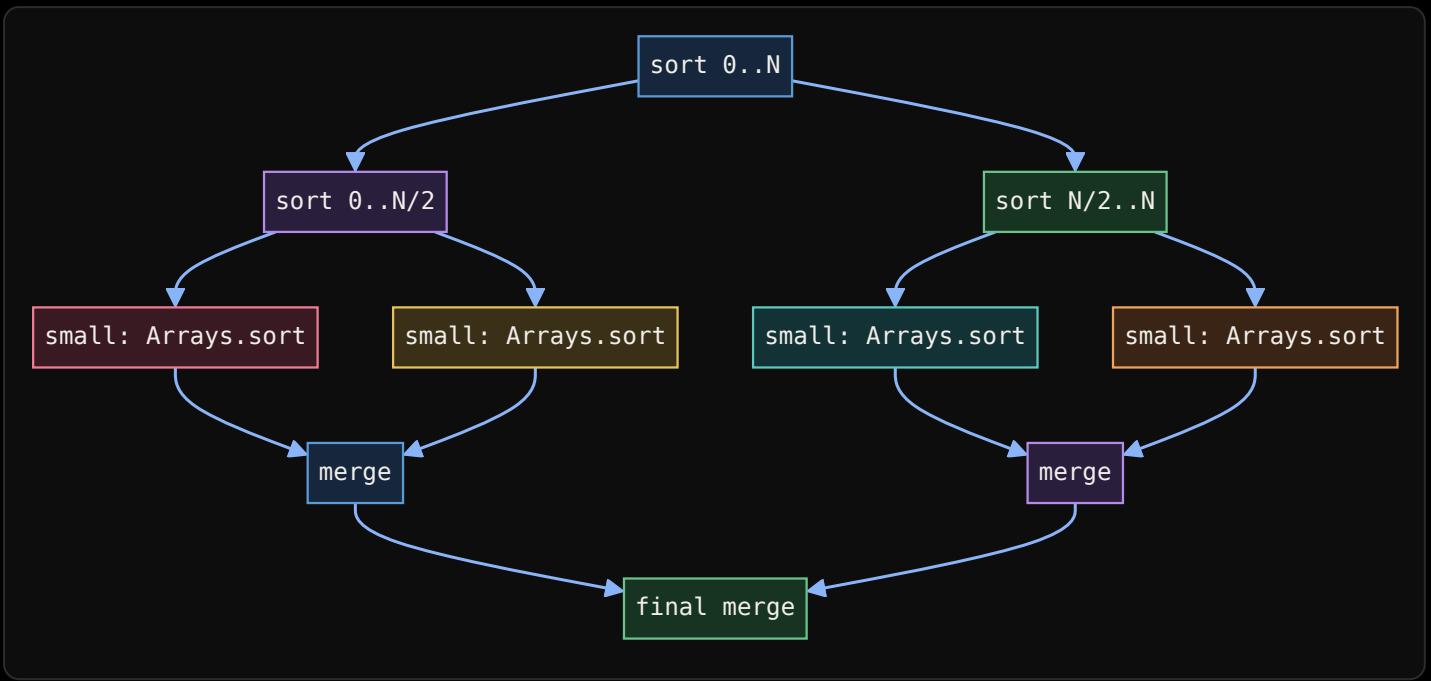
13.10 Multi-threaded Merge Sort (Fork/Join)

Problem

Sort a large `int[]` using all CPU cores.

Approach

Merge sort is **divide and conquer**; Java's **Fork/Join** (`ForkJoinPool`, `RecursiveAction`/`RecursiveTask`) fits it: ranges ≤ 8,192 → `Arrays.sort` sequentially; otherwise split at `(lo + hi) >>> 1` and `invokeAll(left, right)`; merge with **one shared scratch buffer** (tasks use disjoint `[lo, hi)` slices). **Work stealing:** idle pool threads steal tasks from the other end of busy threads' deques.



Java solution

```java
// File: code/lld/ch13/mergesort/Main.java
    static final class ParallelMergeSort extends RecursiveAction {
        private static final long serialVersionUID = 1L;
        static final int THRESHOLD = 1 << 13;   // below ~8K elements, splitting costs more than it saves
        private final int[] a, buf;             // buf = one shared scratch array; tasks use disjoint ranges
        private final int lo, hi;               // sorts a[lo, hi)

        ParallelMergeSort(int[] a, int[] buf, int lo, int hi) { this.a = a; this.buf = buf; this.lo = lo; this.hi
= hi; }

        @Override protected void compute() {
            if (hi - lo <= THRESHOLD) {
                Arrays.sort(a, lo, hi);                         // sequential base case
                return;
            }
            int mid = (lo + hi) >>> 1;                          // overflow-safe midpoint
            invokeAll(new ParallelMergeSort(a, buf, lo, mid),   // fork both halves, wait for both
                      new ParallelMergeSort(a, buf, mid, hi));
            merge(mid);
        }

        private void merge(int mid) {
            if (a[mid - 1] <= a[mid]) return;                   // already in order: skip
            System.arraycopy(a, lo, buf, lo, hi - lo);
            int i = lo, j = mid, k = lo;
            while (i < mid && j < hi) a[k++] = buf[i] <= buf[j] ? buf[i++] : buf[j++];   // <= keeps it stable
            while (i < mid) a[k++] = buf[i++];
            // right-half leftovers are already in place
        }
    }

    static void parallelSort(int[] a) {
        ForkJoinPool.commonPool().invoke(new ParallelMergeSort(a, new int[a.length], 0, a.length));
    }
```

Output: `4000000 random ints: matches Arrays.sort = true`, `Arrays.parallelSort agrees = true`. Full runnable code: code/lld/ch13/mergesort/ (see the Full Edition).

Why it's correct

Disjoint ranges → no races; `invokeAll`'s join gives happens-before for the merge; `<=` keeps it **stable**.

Pitfalls

No threshold; a new buffer per merge; blocking I/O in Fork/Join tasks; `(lo + hi) / 2` overflow; expecting speed-up on tiny arrays (Amdahl).

Real-world examples

`Arrays.parallelSort`; parallel streams; Hadoop/Spark sorts across machines.

In the interview

30-second answer: "A RecursiveAction splits the range in half until it's below a threshold, sorts small pieces sequentially, then merges using one preallocated buffer. invokeAll runs both halves in parallel on the ForkJoinPool, whose work stealing keeps cores busy. Subtasks touch disjoint ranges, so no locks are needed."

Self-check

- **Q:** Why is a shared `buf` safe? **A:** Concurrent tasks use non-overlapping ranges of it.

TL;DR

Split until small, sort sequentially, merge; Fork/Join + work stealing; disjoint ranges = no locks.

13.11 Rate limiter — token bucket (thread-safe)

Problem

Limit API calls per client (user id / API key / IP): bursts up to *B*, sustained rate *R*/s, correct under concurrency.

Approach

Token bucket: holds up to *capacity* tokens (the burst), starts full, refills at *refillRate*/s (never above capacity); each request takes one token or is rejected (HTTP 429). Refill **lazily** from `tokens` and `lastRefill` on each request — no background thread. **"Refill + check + take" must be one atomic step**, else two threads spend the same token: (1) `synchronized tryAcquire()`; (2) **lock-free** — `(tokens, lastRefill)` in one immutable `State` record in an `AtomicReference`, swapped by CAS (12.15's trick). Per client: `computeIfAbsent(clientId, …)` makes exactly one bucket. Time comes from an injected `Ticker`.

Algorithm	Bursts	Memory	Notes
Token bucket	Up to capacity	2 numbers	Most common for APIs
Leaky bucket	Smoothed constant output	Queue	Traffic shaping
Fixed window counter	2× burst at window edges	1 counter	Simplest
Sliding window log	Exact	One timestamp per request	Memory-heavy
Sliding window counter	Approximate, smooth	2 counters	Good compromise

Java solution

```
// File: code/lld/ch13/ratelimiter/Main.java
        public synchronized boolean tryAcquire() {
            long now = ticker.nanos();
            tokens = Math.min(capacity, tokens + (now - lastRefill) * tokensPerNano);   // refill + acquire are
ONE atomic step
            lastRefill = now;
            if (tokens >= 1) { tokens -= 1; return true; }
            return false;
        }
    // ...
    /** Same algorithm, lock-free: the whole state is one immutable record swapped with CAS. */
    static final class LockFreeTokenBucket implements RateLimiter {
        private record State(double tokens, long lastRefill) { }
        // ... capacity, tokensPerNano, ticker
        private final AtomicReference<State> state;
        // ...
        public boolean tryAcquire() {
            while (true) {
                State cur = state.get();
                long now = Math.max(ticker.nanos(), cur.lastRefill());   // never move time backwards
                double refilled = Math.min(capacity, cur.tokens() + (now - cur.lastRefill()) * tokensPerNano);
                boolean ok = refilled >= 1;
                State next = new State(ok ? refilled - 1 : refilled, now);
                if (state.compareAndSet(cur, next)) return ok;            // lost the race? recompute and retry
            }
        }
    }

    /** One bucket per client key (user id, API key, IP). computeIfAbsent creates each bucket exactly once. */
        boolean allow(String clientId) { return buckets.computeIfAbsent(clientId, k ->
factory.get()).tryAcquire(); }
```

Output (both versions, capacity 5, refill 2/s):

```
t=0.0s  7 requests : YYYYYnn
t=1.0s  3 requests : YYn
t=1.25s 1 request  : n  (only 0.5 token so far)
after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
```

Full runnable code: code/lld/ch13/ratelimiter/ (see the Full Edition).

Why it's correct

The decision is atomic (lock, or snapshot + CAS retry) → each token granted once; tokens clamp to `capacity`; `computeIfAbsent` prevents two buckets per client; `Math.max(..., lastRefill)` stops time going backwards.

Pitfalls

Separate refill/take calls; a thread per bucket; `currentTimeMillis()`; unbounded bucket maps (evict idle ones with 13.6's TTL cache); in-memory limits are per server — use **Redis** + Lua for global limits.

Real-world examples

Stripe (Redis token buckets); API Gateway *rate* + *burst*; Guava `RateLimiter`, Bucket4j; NGINX `limit_req` (leaky bucket).

In the interview

> **30-second answer:** "Token bucket: capacity for bursts, refill rate for the average. I refill lazily from elapsed nanoTime on each request, and make refill-check-take one atomic step — synchronized, or lock-free by CASing an immutable state record. A ConcurrentHashMap gives one bucket per client via computeIfAbsent; across servers I'd move the bucket into Redis with a Lua script."

Self-check

- **Q:** Why one immutable `State` in the lock-free version? **A:** `tokens` and `lastRefill` must change together; one CAS on one reference updates both.

TL;DR

Capacity = burst, rate = average. Lazy refill; refill+take atomic (lock or CAS on immutable state); one bucket per client; Redis for multi-server.

13.12 Chapter summary

Problem	Key primitive	The one invariant to state in the interview
FooBar	2 semaphores	Exactly one permit exists across both
Zero Even Odd	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz	monitor + `notifyAll`	Predicates partition 1..n
H2O	2 semaphores + barrier	≤2 H and ≤1 O per round; slot freed after release
Dining Philosophers	ordered locks / n−1 semaphore	No circular wait
TTL cache	CHM + `compute` + conditional remove	Never return or keep expired data; load once per key
Striped map	lock array	Each bucket belongs to exactly one stripe
Blocking queue	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort	Fork/Join	Subtasks touch disjoint ranges
Rate limiter	lock or CAS on immutable state	Refill + take is one atomic step

13.13 Chapter self-check

1. **Q:** In FizzBuzz, what bug appears with `notify()`? **A:** The wrong thread wakes and sleeps again; the right one never wakes → hang.
2. **Q:** In H2O, why release the slot after printing? **A:** So the next molecule can't form until this one has fully left.
3. **Q:** Why must the striped map's stripe depend only on the hash? **A:** So a key's lock doesn't change on resize.
4. **Q:** When is `signal()` safe instead of `signalAll()`? **A:** When each condition queue holds only threads waiting for the same thing and one change enables exactly one waiter.
5. **Q:** How do you make a token bucket global across 10 servers? **A:** Store it in Redis and update it atomically with a Lua script.

13.14 Chapter TL;DR

- **Ordering puzzles** (FooBar, ZeroEvenOdd, H2O): semaphores as batons, barriers for groups; "exactly one permit" invariants.
- **Predicate waiting** (FizzBuzz, blocking queue): lock + `while (!condition) wait/await` + `notifyAll` or targeted `signal`.
- **Multi-lock problems** (philosophers): break circular wait with ordering or limiting.
- **Thread-safe data structures** (cache, map, queue, Bloom filter, rate limiter): immutable entries, atomic compound ops, striping, CAS on immutable state, injected clocks.
- **Parallel algorithms** (merge sort): Fork/Join with thresholds and disjoint data.